

PDI & VC

Processamento Digital de Imagens e Visão Computacional

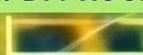
SUNFLOWER | B 86 CONF. →



SUNFLOWER | 0.98 CONF. (PDI->VC)



PDI PROCESS | EDGE DET.



GIRASSOL | 95% (VC USE)



Fundamentos, Algoritmos
e Aplicações Integradas

AUTOR: FRANCISCO DE ASSIS ZAMPIROLI



Processamento Digital de Imagens e Visão Computacional

Livro interativo com Python

Francisco de Assis Zampirolli

Sumário

PDI e VC — Python	20
Organização	20
Prefácio	21
Um livro vivo	21
Uso de ferramentas de Inteligência Artificial	21
Como este livro é produzido	21
A biblioteca <code>morph.py</code>	22
Exercícios de Programação (EPs) e Validação Automática	23
Estrutura e Validação Local	23
Integração com o Moodle (VPL)	23
Como Publicar no Moodle	24
Idiomas e Conversão de Código	24
Código aberto	24
Antes de começar: <i>Notebooks</i> em Python	24
I Parte I — Fundamentos de PDI	26
1 Fundamentos e Primeiros Passos	27
1.1 Objetivos	27
1.2 Antes de começar: <i>Notebooks</i> em Python	27
1.3 Fundamentos	27
1.3.1  Visão Computacional	28
1.3.2  Processamento Digital de Imagens (PDI)	28
1.4 Etapas do PDI	29
1.5 Formação da Imagem e o Espectro	29
1.6 O que é uma Imagem Digital?	32
Representação Matemática	32
1.7 Configuração do Ambiente	33
1.8 Fundamentos de Matrizes - atenção à cópia de referências	34
1.9 Lendo e Exibindo Imagens	35
Alternativa: <i>Download</i> da Imagem para Armazenamento Local	36
1.10 Conversão de Tipos e Limiarização	37
Conversão para Tons de Cinza	37
Limiarização (<i>Thresholding</i>)	37
Exemplo prático de conversão e limiarização	37
1.11 Limiarização pelo método de Otsu	38
1.12 Acesso a Pixels	39
1.13 Resumo	41
1.14  Uso do NotebookLM como Tutor Complementar	41
Funcionalidades Disponíveis na Plataforma	41
1.15 Lista de Exercícios	42
Referências do Capítulo	43
1.16  Parte Prática com Exercícios de Programação	43
 Objetivo deste Caderno	43

§ Instruções Passo a Passo	43
⚠ Importante: Regras e Boas Práticas	45
1.16.1 EP01_01 📏 Três métricas de distância em PDI	45
1.16.2 EP01_02 📊 Desempenho Preditivo — Métricas de ML em VC	52
1.16.3 EP01_03 ↗ <i>Mean Average Precision</i> (mAP) — Curva Precisão-Sensibilidade	54
1.16.4 EP01_04 🖼️ Leitura e Informações de uma Imagem em Matriz	57
1.16.5 EP01_05 ↺ Negativo de uma Imagem em Tons de Cinza	60
1.16.6 EP01_06 🎨 Conversão RGB → Tons de Cinza (ITU-R BT.601)	61
1.16.7 EP01_07 ⬛ Limiarização Manual: Imagem Binária	63
1.16.8 EP01_08 🎨 Remapeamento por Faixa de Intensidade	65
1.16.9 EP01_09 🏁 Padrão de Tabuleiro: Matriz de Xadrez	66
1.16.10 EP01_10 📄 Metadados: Leitura de Arquivo PGM	68
1.16.11 EP01_11 ↗ Análise de Vizinhaça: Filtro de Máximo 1D	70
2 Da Captura ao Pixel - Amostragem, Quantização e Conectividade	73
2.1 Objetivos	73
2.2 O Olho e a Câmera - Elementos da Percepção Visual	73
2.2.1 Visão humana	73
2.2.2 Sensores digitais	73
2.3 Ilusões de Ótica: Os Desafios da Percepção Visual	74
2.3.1 Ambiguidade e Contexto	74
2.3.2 Brilho e Contraste Local	75
2.3.3 Geometria e Perspectiva	75
2.4 O Modelo Matemático da Formação da Imagem	75
2.4.1 Representação Digital e Canais Espectrais	76
2.4.2 Preparando o Ambiente Prático	77
2.4.3 Implementação da Leitura de Imagem em <code>morph.py</code>	77
2.4.4 RGB e RGBA: Exemplo Prático com a Imagem Mandrill	78
2.5 Digitalização: Amostragem e Quantização	79
2.5.1 Amostragem - Discretização do espaço	79
2.5.2 Quantização - Discretização da intensidade	79
2.5.3 Efeitos da amostragem e quantização	80
2.5.4 Análise Técnica	82
2.6 Relações entre Pixels - Topologia da Imagem	83
2.6.1 Vizinhaça	83
2.6.2 Adjacência, conectividade e caminhos	83
2.6.3 Distâncias entre pixels	84
2.7 Armazenamento de Imagens	84
2.7.1 Exemplo: Extração de Metadados e Localização GPS	85
2.7.2 Por que esta separação é importante?	87
2.8 Transformações Geométricas Básicas	87
2.8.1 Translação	88
2.8.2 Rotação	89
2.8.3 Escala (Redimensionamento)	90
2.8.4 Cisalhamento (<i>Shear</i>)	91
2.9 Resumo	92
2.10 📖 Uso do NotebookLM como Tutor Complementar	92
2.11 Lista de Exercícios	93
Referências do Capítulo	93
2.12 🖱️ Parte Prática com Exercícios de Programação	93
🎯 Objetivo deste Caderno	93
2.12.1 EP02_01 ☉ Ajuste de Brilho e Contraste	94
2.12.2 EP02_02 📊 Subamostragem Espacial	96
2.12.3 EP02_03 🎨 Quantização de Níveis de Cinza	98

2.12.4	EP02_04	 Transformada de Distância em Imagem Binária	100
2.12.5	EP02_05	Translação de Imagem	102
2.12.6	EP02_06	 Rotação de Imagem	103
2.12.7	EP02_07	 Redimensionamento (Escala)	105
2.12.8	EP02_08	 Cisalhamento (Shear)	109
2.12.9	EP02_09	 Transformação Afim Genérica	110
2.12.10	EP02_10	 Correção de Perspectiva (Homografia)	114
2.12.11	EP02_11	 Correção de Perspectiva (Homografia) em Imagem Real	116
3	Operações Espaciais: Intensidade, Histograma e Filtragem		121
3.1	Objetivos		121
3.2	Operações em Nível de Intensidade		121
3.2.1	Preparando o Ambiente Prático		122
3.2.2	Operações Aritméticas		123
3.2.3	Mistura Ponderada (<i>Alpha Blending</i>)		124
3.2.4	Operações Lógicas e Máscaras Bit a Bit		126
3.3	Histograma de Imagens		127
3.3.1	Equalização de Histograma		128
3.3.2	Especificação de Histograma		132
3.4	Fundamentos Espaciais: Vizinhança, Convolução e <i>Kernels</i>		133
3.4.1	Vizinhança		133
3.4.2	Tratamento de Bordas		134
3.4.3	Correlação e Convolução		136
3.4.4	Correlação vs. Convolução no OpenCV		136
3.4.5	O Papel do <i>Kernel</i>		138
3.4.6	Exemplo Numérico: Correlação Passo a Passo		140
3.5	Filtragem Espacial de Suavização		141
3.5.1	Filtro de Média (<i>Box Filter</i>)		141
3.5.2	Filtro Gaussiano		142
3.6	Filtragem Espacial de Realce		144
3.6.1	Laplaciano		145
3.6.2	Operador de Sobel		148
3.6.3	Operador de Prewitt		150
3.6.4	<i>Unsharp Masking</i> (USM)		151
3.6.5	Detector de Canny		153
3.7	Filtros de Ordem: Filtro da Mediana		154
3.7.1	Ruído Impulsivo: Sal e Pimenta		154
3.7.2	Filtro da Mediana		155
3.8	Aplicação Prática: Pré-processamento para Segmentação		158
3.9	Resumo		159
3.10	 Uso do NotebookLM como Tutor Complementar		159
3.11	Lista de Exercícios		160
	Referências do Capítulo		161
3.12	 Parte Prática com Exercícios de Programação		161
	 Objetivo deste Caderno		161
3.12.1	EP03_01	 Adição Saturada de Constante	162
3.12.2	EP03_02	 <i>Alpha Blending</i> de Duas Imagens	164
3.12.3	EP03_03	Inversão de Imagem (Negativo Fotográfico)	165
3.12.4	EP03_04	 Equalização de Histograma (L bits)	167
3.12.5	EP03_05	Aplicação de Máscara AND Binária	168
3.12.6	EP03_06	Filtro de Média com <i>Kernel</i> $N \times N$	172
3.12.7	EP03_07	 Operador Laplaciano (w4) para Realce de Bordas	174
3.12.8	EP03_08	Gradiente de Sobel: G_x e G_y	175
3.12.9	EP03_09	 Filtro da Mediana 3×3	179

3.12.10	EP03_10	<i>Unsharp Masking (USM)</i>	181
4	Morfologia Matemática e Segmentação de Imagens		184
4.1	Objetivos		184
4.2	Limiarização		185
4.2.1	Imagem de Moedas		186
4.2.2	Escolha do Pré-processamento para o Otsu		187
4.2.3	Resultado: CLAHE como Melhor Pré-processamento		189
4.3	Morfologia Matemática		190
4.3.1	Erosão e Dilatação		190
4.3.2	Abertura e Fechamento		197
4.3.3	Operadores Geodésicos		200
4.3.4	Reconstrução Morfológica		204
4.3.5	Preenchimento de Buracos e Remoção de Bordas		206
4.3.6	<i>Pipeline</i> de Limpeza Binária com CLAHE		209
4.3.7	Morfologia em Tons de Cinza		210
4.4	Segmentação de Imagens: Fundamentação e Taxonomia		213
4.4.1	Rotulação		214
4.4.2	Transformada de Distância		218
4.4.3	Transformada de Distância Euclidiana em quatro passos		221
4.4.4	Segmentação por <i>Watershed</i>		224
4.5	Extração de Componentes e Descritores de Forma		229
4.5.1	Rotulação e Estatísticas de Componentes		230
4.5.2	Descritores de Forma		231
4.5.3	Conexão com Detecção de Objetos Moderna		233
4.6	Resumo		235
4.7	 Uso do NotebookLM como Tutor Complementar		236
4.8	Lista de Exercícios		236
	Referências do Capítulo		237
4.9	 Parte Prática com Exercícios de Programação		237
	 Objetivo deste Caderno		238
4.9.1	EP04_01	Limiarização Global por Limiar Fixo	238
4.9.2	EP04_02	 Limiarização Automática de Otsu	240
4.9.3	EP04_03	 Dilatação Binária Plana (mm.dil0)	243
4.9.4	EP04_04	Erosão Binária Plana (mm.ero0)	244
4.9.5	EP04_05	Abertura Morfológica (Remoção de Ruído)	247
4.9.6	EP04_06	 Fechamento Morfológico (Preenchimento de Falhas)	249
4.9.7	EP04_07	Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	252
4.9.8	EP04_08	Gradiente Morfológico, Top-hat e Black-hat	254
4.9.9	EP04_09	Transformada de Distância e o “Miolo” do Objeto	256
4.9.10	EP04_10	Separação de <i>Blobs</i> , Rotulação e Descritores	257
5	Transformadas e Compressão		261
5.1	Objetivos		261
5.2	Configuração do Ambiente		262
5.3	Transformada de Fourier Discreta 2D		262
5.3.1	Simulador: Reconstruindo Sinais com Senoides		262
5.3.2	Interpretação do espectro de frequência		264
5.3.3	O Experimento da Grade: Construindo uma Imagem a partir de um Único Coeficiente		264
5.3.4	Definição Matemática		265
5.3.5	Magnitude e Fase		266
5.3.6	O que a Magnitude e a Fase carregam?		267
5.4	Teorema da Convolução e Estratégias de Filtragem		269
5.4.1	<i>Kernel</i> Separável vs. Não Separável		269

5.4.2	Análise de Eficiência Computacional	270
5.4.3	Discussão dos resultados experimentais	270
5.5	Filtros no Domínio da Frequência	274
5.5.1	Filtros Passa-Baixa	275
5.5.2	Filtros Passa-Alta e Passa-Banda	275
5.5.3	Remoção de Ruído Periódico	279
	Síntese — Filtros Espectrais	281
5.6	<i>Wavelets</i> e Multirresolução	281
5.6.1	O Limite da Transformada de Fourier: localização espacial	281
5.6.2	Transformada <i>Wavelet</i> Discreta 2D	282
5.6.3	Famílias de <i>Wavelets</i>	282
5.6.4	Análise Multirresolução com a DWT 2D	286
	Síntese — Fourier vs. <i>Wavelets</i> : quando utilizar cada abordagem?	291
5.7	Compressão de Imagens	292
5.7.1	Taxonomia das Redundâncias	292
5.7.2	Transformada de Cossenos Discreta (DCT-II 2D)	293
5.7.3	As Funções de Base da DCT	293
5.7.4	Concentração de Energia e Reconstrução Progressiva	294
5.7.5	O <i>Pipeline</i> de Compressão JPEG	296
5.7.6	Simulador Interativo: Quantização DCT	299
5.8	Comparação de Formatos de Imagem	300
5.8.1	Características dos Formatos	300
5.8.2	Métricas de Avaliação de Qualidade	300
5.8.3	Inspeção Visual: Natureza dos Artefatos de Compressão	301
5.8.4	Avaliação Quantitativa e Espacial da Compressão	302
	Síntese — Compressão JPEG	307
5.9	Aplicação Prática: Remoção de Ruído por Filtragem Híbrida	308
5.9.1	Análise de Desempenho e Conclusão do Capítulo	308
5.10	Resumo do Capítulo	310
	Fundamentos Essenciais	310
5.11	 Uso do NotebookLM como Tutor Complementar	312
5.12	Lista de Exercícios	312
	Referências do Capítulo	313
5.13	 Parte Prática com Exercícios de Programação	313
	Objetivo deste Caderno	314
5.13.1	EP05_01  Filtro Passa-Baixa Ideal por Distância no Espectro	315
5.13.2	EP05_02  Filtro <i>Notch</i> : Removendo Picos Periódicos	317
5.13.3	EP05_03  Quantização DCT: a Verdadeira Fonte de Compressão	319
5.13.4	EP05_04  Implementando a DFT 2D a Partir da Definição	321
5.13.5	EP05_05  <i>Pipeline</i> JPEG Completo: DCT, Quantização e Reconstrução	322

II Parte II — Visão Computacional 325

6 Análise de Documentos e Inspeção Industrial 326

6.1	Objetivos do Capítulo	326
6.2	Configuração do Ambiente	327
6.3	Bases de Imagens para Experimentação	328
6.3.1	Imagens Públicas com <code>skimage.data</code>	328
6.4	Fundamentos de OMR e Inspeção Industrial	330
6.5	Projetos Práticos: Construção de um <i>Pipeline</i> de Análise Documental	331
6.5.1	Alinhamento Automático de Documentos (<i>OCR/OMR Pre-processing</i>)	331
6.5.2	Algoritmo de Retificação de Inclinação (<i>Deskew</i>)	332
6.5.3	Modelagem Matemática	333

6.5.4	Limitações Práticas	336
6.5.5	Normalização de Fundo e Equalização Local de Contraste	337
6.5.6	Reconhecimento Óptico de Caracteres (OCR)	339
6.5.7	Tradução Automática do Texto Reconhecido	342
6.5.8	Detecção de Bordas e Contornos	343
6.5.9	Isolamento, Segmentação e Decodificação do <i>QRCode</i>	346
6.5.10	Decodificação de Códigos de Barras	350
6.6	O MCTest como Estudo de Caso: do Protótipo ao Sistema em Produção	351
6.6.1	Obtenção e Preparação do Módulo	352
6.6.2	Extração da Área de Respostas	352
6.6.3	Segmentação do <i>QRCode</i>	355
6.6.4	Localização dos Quadros de Respostas	357
6.6.5	Leitura Automática das Respostas	357
6.7	Inspecção Industrial Automatizada	359
6.7.1	Referências e <i>Datasets</i> Públicos	360
6.7.2	Detecção de Defeitos por Análise de Textura	361
6.8	Resumo	364
	Próximos Passos	365
6.9	 Uso do NotebookLM como Tutor Complementar	365
6.10	Lista de Exercícios	365
	Referências do Capítulo	366
6.11	 Parte Prática com Exercícios de Programação	367
	Legenda de Dificuldade	367
	 Objetivo deste Caderno	368
6.11.1	EP06_01  Avaliação de Segmentação por IoU (<i>Intersection over Union</i>)	369
6.11.2	EP06_02  Filtro de Marcadores por Circularidade	371
6.11.3	EP06_03  Classificação de Marcações em Folhas de Resposta (OMR)	373
6.11.4	EP06_04  Estimador de Inclinação por Mediana Angular (<i>Deskew</i>)	375
6.11.5	EP06_05  Normalização de Fundo por Divisão (Correção de Iluminação)	377
6.11.6	EP06_06  Mapa de Variância Local para Detecção de Textura	379
6.11.7	EP06_07  <i>Pipeline</i> de Inspecção Industrial: Registro por Translação e Subtração	382
6.11.8	EP06_08  Segmentação e Decodificação Real de <i>QRCode</i> com OpenCV	383
7	Classificação de Imagens e Reconhecimento de Padrões	389
7.1	Objetivos do Capítulo	389
7.2	Configuração do Ambiente	389
7.3	Um Problema Concreto: Classificação de Frutas	390
7.4	Panorama do Capítulo: O <i>Pipeline</i> Clássico de Classificação de Imagens	392
7.5	Fundamentos de Reconhecimento de Padrões	392
7.6	Extração de Descritores Clássicos	393
7.6.1	<i>Local Binary Patterns</i> (LBP)	395
7.6.2	<i>Histogram of Oriented Gradients</i> (HOG)	396
7.6.3	O Impacto da Escala e a Normalização de Características	397
7.7	 Mapa Conceitual	399
7.8	Descritores na Prática	399
7.9	Um Problema Concreto: Simulando Descritores de Frutas	404
7.10	Classificador k-NN: Como Funciona por Dentro	405
7.10.1	Métrica de Distância	406
7.10.2	Regra de Decisão	406
7.10.3	O Papel do Parâmetro k	409
7.11	Projeto Prático 1: Classificação de Dígitos Manuscritos com k-NN	411
7.11.1	Classificação com Vetores de Intensidade	411
7.11.2	Classificação com Descritores HOG	413
7.12	Avaliação de Classificadores	414

7.12.1	Escolha de k por Validação Cruzada	415
7.12.2	O Compromisso entre Viés e Variância: Diagnóstico de <i>Overfitting</i> e <i>Underfitting</i>	416
7.13	Projeto Prático 2: Classificação de Texturas com Descritores LBP	418
7.13.1	Extração do Descritor LBP e Treinamento do Classificador	420
7.13.2	Diagnóstico Fino do Classificador: Precisão, Revocação e F1-Score	422
7.14	Limitações dos Descritores Artesanais	424
7.15	Resumo	424
7.16	 Uso do NotebookLM como Tutor	424
7.17	Lista de Exercícios	425
Referências do Capítulo		425
7.18	 Parte Prática com Exercícios de Programação	426
	Objetivo deste Caderno	427
7.18.1	EP07_01  Classificador k-NN Passo a Passo	427
7.18.2	EP07_02  Avaliação por Matriz de Confusão	429
7.18.3	EP07_03  Codificação Manual do Descritor LBP	431
7.18.4	EP07_04  Histograma de Orientações de uma Célula HOG	433
7.18.5	EP07_05  <i>Pipeline</i> Completo: Descritores + k-NN + Avaliação Multi-Classe	435
7.18.6	EP07_06  Classificação Real de um Mosaico de Texturas via LBP + k-NN	437
8	Compreendendo Cenas: Correspondência de Características, Detecção e Segmentação	443
8.1	Objetivos do Capítulo	443
8.2	Configuração do Ambiente	444
8.3	Detectores e Descritores Locais de Características	444
8.3.1	O Descritor ORB	445
8.4	Projeto Prático 1: Correspondência de Características e Homografia com ORB	445
8.4.1	Estimando a Homografia com RANSAC	445
8.5	Modelagem Matemática: Homografia e RANSAC	447
8.5.1	Homografia	447
8.5.2	RANSAC	447
8.6	Detecção de Objetos: Haar Cascade (Viola-Jones)	448
8.7	Detecção de Objetos: Caixas Delimitadoras, IoU e NMS	450
8.7.1	Intersection over Union (IoU)	450
8.7.2	Supressão de Não-Máximos (NMS)	450
8.8	Segmentação Visual: Semântica, de Instâncias e Panóptica	452
8.9	Visão Geral de Modelos Modernos	453
8.10	Limitações das Abordagens Clássicas e Motivação para o Deep Learning	454
8.11	Resumo	454
8.12	 Uso do NotebookLM como Tutor Complementar	455
8.13	Lista de Exercícios	455
8.13.1	Exercício 1: Robustez do ORB a Transformações (Básico)	455
8.13.2	Exercício 2: Ajustando o Haar Cascade (Intermediário)	455
8.13.3	Exercício 3: IoU e NMS em um Cenário Mais Denso (Intermediário/Desafio)	456
8.13.4	Exercício 4: Do Mosaico de Texturas à Segmentação (Desafio)	456
8.14	Próximos Passos	456
8.15	 Parte Prática com Exercícios de Programação	457
	Objetivo deste Caderno	457
8.15.1	EP08_01  Distância de Hamming e Correspondência de Descritores Binários	458
8.15.2	EP08_02  Homografia e RANSAC: A Votação por <i>Inliers</i>	460
8.15.3	EP08_03  Imagem Integral: Somas Retangulares em Tempo Constante	462
8.15.4	EP08_04  IoU e Supressão de Não-Máximos (NMS)	465
8.15.5	EP08_05  Rotulagem de Componentes Conectados: Segmentação Clássica de Instâncias	466
9	Deep Learning para Visão Computacional	470
9.1	Objetivos do Capítulo	470

9.2	Configuração do Ambiente	471
9.3	Da Convolução Fixa à Convolução Aprendida	472
9.3.1	Convolução como Camada Aprendida	472
9.3.2	Pooling	472
9.3.3	Arquitetura Típica	473
9.4	Projeto Prático 1: Uma CNN do Zero para os Dígitos do Capítulo 7	474
9.4.1	Comparando com o Capítulo 7	475
9.5	Transferência de Aprendizado	477
9.6	Aplicações em Larga Escala com Modelos Pré-treinados	480
9.6.1	Classificação de Imagens	480
9.6.2	Detecção de Objetos	480
9.6.3	Segmentação Semântica	481
9.7	Projeto Prático 2: Detecção de Objetos com YOLO e Transferência de Aprendizado	482
9.7.1	Gerando um Conjunto de Dados Sintético de Objetos Geométricos	482
9.7.2	Ajuste Fino de um YOLO Pré-treinado	485
9.7.3	Usando um Conjunto de Dados Real	495
9.8	Integrando Geometria e Aprendizado Profundo	495
9.8.1	Realidade Aumentada	495
9.8.2	Fotogrametria e Referência de Escala	497
9.8.3	Visão Estereoscópica	498
9.9	Resumo	499
9.10	 Uso do NotebookLM como Tutor Complementar	500
9.11	Exercícios Práticos	500
9.11.1	Exercício 1: Profundidade da Rede (Básico)	500
9.11.2	Exercício 2: Limiar de Transferência de Dados (Intermediário)	500
9.11.3	Exercício 3: Ajuste Fino Parcial (<i>Fine-Tuning</i>) (Intermediário/Desafio)	501
9.11.4	Exercício 5: Congelamento do Backbone no YOLO (Intermediário/Desafio)	501
9.11.5	Exercício 4: Disparidade e Distância de Base (Desafio)	501
9.12	Encerramento da Parte II	502
9.13	 Parte Prática com Exercícios de Programação	502
	 Objetivo deste Caderno	503
9.13.1	EP09_01  Convolução 2D Manual (Forward de uma Camada Aprendida)	503
9.13.2	EP09_02  <i>Pooling</i> Manual (Máximo e Média)	506
9.13.3	EP09_03  Contagem de Parâmetros Treináveis de uma CNN	508
9.13.4	EP09_04  Interseção sobre União (IoU) e Supressão de Não-Máximos (NMS)	510
9.13.5	EP09_05  <i>Pipeline</i> Integrado: Da Detecção à Medição do Mundo Real	512

Lista de Figuras

1.1	Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.	28
1.2	Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.	30
1.3	Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.	30
1.4	Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.	31
1.5	(A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.	31
1.6	Exemplos de imagens sintéticas representadas matricialmente.	35
1.7	Mandrill (Mandrillus sphinx) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0). . .	36
1.8	Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).	38
1.9	Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.	39
1.10	Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.	40
1.11	Estúdio do NotebookLM.	42
1.12	Simulador: Distâncias Euclidiana, <i>City-block</i> e <i>Chessboard</i>	47
1.13	Simulador: Desempenho Preditivo — Métricas de ML	54
1.14	Simulador: Mean Average Precision (mAP) - Curva P.S	58
1.15	Simulador: Estatísticas de Pixels	59
1.16	Simulador: Transformação de Negativo	61
1.17	Simulador: Percepção de Cor e Pesos ITU-R	63
1.18	Simulador: Limiarização Interativa	64
1.19	Simulador: Ajuste de Brilho e Contraste	66
1.20	Simulador: Gerador de Xadrez	68
1.21	Simulador: Estrutura do Arquivo PGM	69
1.22	Simulador: Filtro de Máximo Local 1D	72
2.1	Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.	74
2.2	Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.	75
2.3	Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — USC SIPI Image Database (domínio público).	79
2.4	Efeito da subamostragem. Os títulos exibem as dimensões ($W \times H$) e o tamanho da matriz em memória (KB).	81

2.5	Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).	82
2.6	Ilustração de vizinhanças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.	83
2.7	Area de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (<i>Malacoptila striata</i>). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).	86
2.8	Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).	88
2.9	Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.	89
2.10	Comparação de interpolação com zoom no detalhe do olho (recorte 60x60, ampliado 4x). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.	91
2.11	Exemplo de cisalhamento da imagem do pássaro: horizontal (shx=0.3), vertical (shy=0.3) e combinado (shx=0.2, shy=0.2).	92
2.12	Simulador: Ajuste de Brilho e Contraste	96
2.13	Simulador: Subamostragem	98
2.14	Simulador: Quantização	100
2.15	Simulador: Ajuste de Brilho e Contraste	102
2.16	Simulador: Translação (tx, ty)	104
2.17	Simulador: Rotação (ângulo)	106
2.18	Simulador: Redimensionamento (Nearest vs Bilinear)	108
2.19	Simulador: Cisalhamento (shx, shy)	111
2.20	Simulador: Transformação Afim (matriz 2x3)	113
2.21	Simulador: Correção de Perspectiva (Homografia)	115
2.22	Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).	117
2.23	Correção de perspectiva: original e vista frontal retificada.	118
2.24	Simulador: correção de perspectiva do Sudoku.	119
3.1	<i>Mandrill</i> (<i>Mandrillus sphinx</i>) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).	123
3.2	Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).	124
3.3	Retrato de um leopardo (<i>Panthera pardus</i>) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).	125
3.4	<i>Alpha blending</i> entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.	126
3.5	Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).	127
3.6	Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).	128
3.7	Equalização de histograma em imagem 5x5 com 3 bits (L=8): imagem original com tons concentrados em {2,3,4}, imagem equalizada com tons redistribuídos para {1,5,7}, e os respectivos histogramas evidenciando o espalhamento das frequências.	130
3.8	Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.	132
3.9	Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.	133
3.10	Correlação com <i>kernel</i> de média 3x3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.	140
3.11	<i>Patch</i> 5x5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3x3 centrada no pixel [1,1] onde a correlação será calculada.	141
3.12	Filtro de média com <i>kernels</i> de tamanho crescente (3x3, 7x7, 15x15). O borramento das bordas aumenta com o tamanho do <i>kernel</i> .	142

3.13	<i>Kernel</i> Gaussiano 5×5 ($=1$): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.	143
3.14	Comparação entre filtro de média e Gaussiano (<i>kernel</i> 9×9 , $=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.	144
3.15	Simulador: Filtragem Espacial de Realce 1D.	145
3.16	<i>Kernel</i> Laplaciano w4 aplicado ao <i>patch</i> 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.	147
3.17	Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.	148
3.18	Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $ f $ combina ambos, revelando todas as bordas independente de direção.	150
3.19	Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.	150
3.20	Realce de nitidez por <i>Unsharp Masking</i> na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.	152
3.21	<i>Unsharp Masking</i> na imagem do leopardo com $=1$ e k variando de 0.5 a 8.0. Para $k > 2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.	153
3.22	Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.	154
3.23	Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro prob indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).	155
3.24	Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.	158
3.25	<i>Pipeline</i> de pré-processamento: CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.	159
3.26	Simulador: Adição Saturada de Constante	163
3.27	Simulador: Alpha <i>Blending</i> de Duas Imagens	165
3.28	Simulador: Inversão de Imagem (Negativo Fotográfico)	167
3.29	Simulador: Equalização de Histograma (L bits)	169
3.30	Simulador: Aplicação de Máscara AND Binária	171
3.31	Simulador: Filtro de Média com <i>Kernel</i> $N \times N$	173
3.32	Simulador: Operador Laplaciano (w4) para Realce de Bordas	176
3.33	Simulador: Gradiente de Sobel: Gx e Gy	178
3.34	Simulador: Filtro da Mediana 3×3	180
3.35	Simulador: <i>Unsharp Masking</i> (USM)	183
4.1	Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).	186
4.2	Comparação dos pré-processamentos: imagem histograma+T* $^2B(T)$ Otsu. A melhor separação bimodal indica o limiar mais confiável.	189
4.3	Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.	191
4.4	Erosão e dilatação em imagem binária 10×10 com elemento estruturante 'L' assimétrico 3×3 . Validação da dualidade erosão-dilatação.	197
4.5	Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A.	198
4.6	Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13	200
4.7	Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\text{delta}_g^{(n)}(f)$ encontra a saída do labirinto.	201

4.8	Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do <i>pipeline</i> , o fluxo é resolvido diretamente no domínio complementar através de <i>mm.cero</i> e <i>mm.suprec</i> , eliminando re-inversões redundantes.	204
4.9	<i>Pipeline</i> de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstroem sua forma exata até a convergência.	206
4.10	<i>Pipeline</i> de preenchimento de buracos (<i>clohole</i>): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.	208
4.11	<i>Pipeline</i> de eliminação de estruturas de borda (<i>edgeoff</i>): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.	209
4.12	<i>Pipeline</i> completo de segmentação com CLAHE: Otsu $\rightarrow$ abertura ($r=9$) $\rightarrow$ <i>clohole</i> $\rightarrow$ abertura ($r=33$) $\rightarrow$ <i>edgeoff</i>	210
4.13	Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D.	211
4.14	Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19.	213
4.15	Algoritmo de rotulação por <i>flood-fill</i> com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.	215
4.16	Simulador interativo de rotulação de componentes conexas (<i>connected component labeling</i>): visualização da expansão <i>flood-fill</i> , conectividade-4 e conectividade-8.	216
4.17	Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.	218
4.18	Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.	219
4.19	Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.	220
4.20	Transformada de Distância em imagem binária 10×10 . Esquerda: imagem original (<i>foreground</i> = 255). Centro: <i>mm.dist1</i> iterativa (erosões com elemento cruz). Direita: <i>mm.dist</i> (L2 Euclidiana).	221
4.21	Simulador interativo 2D (4×4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.	222
4.22	Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando <i>mm.gdist</i> . Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.	223
4.23	Algoritmo Didático de <i>Watershed</i> por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.	226
4.24	Simulador interativo do Algoritmo <i>Watershed</i> por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.	227
4.25	<i>Pipeline watershed</i> delimitado por máscara em imagem binária 20×20	228
4.26	<i>Pipeline Watershed</i> para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.	229
4.27	Componentes conexas extraídos após o <i>pipeline</i> CLAHE $\rightarrow$ Otsu $\rightarrow$ limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.	231
4.28	Contornos extraídos com <i>findContours</i> . Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.	233

4.29	<i>Bounding boxes</i> derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (<i>classe cx cy w h</i>), com coordenadas normalizadas para o intervalo $[0,1]$	234
4.30	Simulador: Limiarização Global por Limiar Fixo	240
4.31	Simulador: Limiarização Automática de Otsu	242
4.32	Simulador: Dilatação Binária Plana (mm.dil0)	245
4.33	Simulador: Erosão Binária Plana (mm.ero0)	247
4.34	Simulador: Abertura Morfológica (erosão + dilatação)	249
4.35	Simulador: Fechamento Morfológico (dilatação + erosão)	251
4.36	Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	253
4.37	Simulador: Gradiente Morfológico, Top-hat e Black-hat	255
4.38	Simulador: Transformada de Distância	258
4.39	Simulador: Separação de Blobs, Rotulação e Descritores	260
5.1	Decomposição de Fourier 1D: uma onda quadrada (linha tracejada) é aproximada pela soma das primeiras senoides (linhas coloridas). Quanto mais termos, melhor a aproximação.	262
5.2	Simulador interativo da decomposição de Fourier 1D: visualização da soma de senoides com diferentes frequências, amplitudes e fases. Adicione termos e observe a convergência para formas de onda arbitrárias.	263
5.3	Toda frequência no espectro (ponto isolado) corresponde a uma onda senoidal 2D rotacionada no domínio espacial.	265
5.4	Simulador interativo da síntese de Fourier 2D. Altere a posição horizontal (u) e vertical (v) do coeficiente no espectro de frequências centrado e observe como a distância em relação ao centro dita a frequência espacial (espessura) e o ângulo dita a orientação da onda senoidal gerada.	265
5.5	Diagrama conceitual do espectro de Fourier 2D centrado.	267
5.6	Experimento de troca de fase: Imagem A (moedas) e Imagem B (padrão geométrico) reconstruídas com magnitudes e fases trocadas. O resultado mostra que a estrutura visual é muito mais sensível à fase do que à magnitude: quando a fase de B é mantida, a imagem resultante preserva a organização espacial de B, mesmo com a magnitude de A. A magnitude, por sua vez, influencia principalmente o contraste e a textura. Observe que a qualidade da reconstrução não é perfeita — há artefatos visíveis —, evidenciando a interdependência entre fase e magnitude para uma representação fiel da imagem.	269
5.7	Comparação de eficiência: Convolução Não Separável (Espacial 2D), Separável (Espacial 1D) e via FFT.	271
5.8	Sem <i>padding</i> , um deslocamento severo faz a imagem vazar para o lado oposto (convolução circular).	272
5.9	Verificação do Teorema da Convolução: a diferença pixel a pixel entre a convolução espacial (cv2.filter2D) e a multiplicação em frequência (FFT) é numericamente nula — confirmando a equivalência teórica.	273
5.10	A Dualidade Perigosa: O corte abrupto na Frequência (Cilindro) transforma-se obrigatoriamente numa Sinc espacial. Suas ondulações causam o <i>ringing</i> fantasma nas bordas da imagem.	274
5.11	Filtros passa-baixa.	275
5.12	Simulador interativo de filtros no domínio da frequência.	276
5.13	Comparação entre filtros passa-baixa: Ideal ($D=30$), Gaussiano ($D=30$) e Butterworth ($D=30$, $n=2$). Perfis de $H(u,v)$ ao longo de uma linha central e imagens filtradas correspondentes.	278
5.14	Filtro passa-alta Gaussiano. (a) Original; (b) Filtro passa-alta ($D=30$) - as bordas das moedas e fundo texturizado são realçados.	279
5.15	Remoção de ruído periódico via filtro <i>notch</i> no domínio da frequência: (a) imagem com ruído senoidal, (b) espectro mostrando os picos do ruído, (c) máscara <i>notch</i> centrada nos picos, (d) imagem restaurada.	280
5.16	Fourier global é cego para a posição. Os espectros não dizem onde as bordas estão.	282

5.17	Funções da <i>Wavelet</i> (). Note como elas rapidamente decaem para zero (suporte compacto), ao contrário das senoides infinitas de Fourier.	284
5.18	Diagrama da decomposição <i>wavelet</i> 2D em dois níveis.	284
5.19	Simulação da decomposição <i>wavelet</i> 2D.	285
5.20	Decomposição <i>wavelet</i> 2D de 2 níveis com <i>wavelet</i> Haar: subbandas LL, LH, HL, HH em cada nível. As subbandas de detalhe revelam estruturas orientadas em diferentes escalas utilizando um padrão sintético.	287
5.21	Comparação entre famílias de <i>wavelets</i> : Haar, db4, sym4 e bior2.2. Subbanda LL (aproximação) e HH (diagonal) para cada escolha, ilustrando o compromisso entre compactação e suavidade com base no padrão sintético.	289
5.22	Reconstrução <i>wavelet</i> com limiarização de coeficientes (<i>hard thresholding</i>): à medida que o limiar aumenta, mais detalhes são zerados, produzindo imagens progressivamente mais suaves. Métrica PSNR quantifica a perda de qualidade sobre o padrão sintético.	291
5.23	O Alfabeto Visual do JPEG: As 64 funções de base da DCT-II. O coeficiente DC fica no topo esquerdo (suave). Ao descer e avançar à direita, a oscilação espacial aumenta drasticamente.	294
5.24	DCT 2D em bloco 8×8: coeficientes e reconstrução progressiva.	296
5.25	<i>Pipeline</i> JPEG simplificado aplicado à imagem clássica do <i>Cameraman</i> : DCT em blocos 8×8, quantização com diferentes fatores de qualidade e reconstrução via IDCT. Os artefatos de bloco (<i>blocking artifacts</i>) tornam-se visualmente evidentes em fatores de qualidade reduzidos ($Q = 10$ e $Q = 25$).	299
5.26	Simulador interativo de compressão DCT-JPEG: ajuste o fator de qualidade e visualize em tempo real os coeficientes zerados, o bloco reconstruído e o erro de quantização.	300
5.27	Análise comparativa de artefatos de compressão sob fator de qualidade reduzido ($Q = 10$). À esquerda, observa-se o artefato de bloco característico da discretização por DCT no JPEG. À direita, evidencia-se o efeito de atenuação e suavização de bordas intrínseco ao padrão WebP.	302
5.28	Curva taxa-distorção: PSNR vs tamanho de arquivo para JPEG, WebP e PNG aplicada à imagem do <i>Cameraman</i>	304
5.29	Análise espacial de degradação: imagens reconstruídas e respectivos mapas de erro absoluto normalizados para diferentes fatores de qualidade JPEG.	306
5.30	<i>Pipeline</i> completo de remoção de ruído misto: (1) adição de ruído gaussiano e periódico; (2) identificação de picos de interferência no espectro de frequências; (3) aplicação de máscara <i>notch</i> com atenuação gaussiana suave; (4) pós-processamento via filtro bilateral para eliminação do ruído estocástico residual.	310
5.31	Mapa conceitual das transformações e propriedades no domínio da frequência.	311
5.32	Simulador: Filtro Passa-Baixa Ideal no Espectro	316
5.33	Simulador: Filtro Notch	318
5.34	Simulador: Quantização DCT (round-trip)	320
5.35	Simulador: DFT 2D manual	322
5.36	Simulador: <i>Pipeline</i> JPEG completo em bloco	324
6.1	Amostra de imagens públicas do <i>skimage.data</i> , agrupadas por área de aplicação.	330
6.2	<i>Pipeline</i> de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.	332
6.3	Simulador interativo de correção de inclinação (<i>deskew</i>): mova o <i>slider</i> para inclinar o documento e observe as três etapas do <i>pipeline</i> — imagem inclinada, bordas Canny e resultado corrigido.	335
6.4	<i>Pipeline</i> de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.	336

6.5	Comparação entre estratégias de pré-processamento para binarização da imagem de texto: (a) imagem original; (b) limiarização direta pelo método de Otsu; (c) fundo estimado por filtragem Gaussiana; (d) imagem após normalização de fundo (divisão pela imagem suavizada); (e) CLAHE seguido de limiarização por Otsu; e (f) normalização de fundo seguida de limiarização por Otsu. As imagens ilustram o efeito de cada técnica na compensação de variações de iluminação e na qualidade da segmentação.	338
6.6	Comparação do texto extraído pelo Tesseract a partir de quatro versões da mesma imagem: (a) imagem original; (b) CLAHE seguido de limiarização por Otsu; (c) normalização de fundo seguida de limiarização por Otsu; e (d) normalização de fundo em tons de cinza, sem limiarização. A comparação evidencia que a binarização externa nem sempre favorece o reconhecimento, uma vez que o Tesseract já realiza sua própria binarização adaptativa internamente.	341
6.7	Fluxo de reconhecimento e tradução automática. A imagem pré-processada por normalização de fundo, sem limiarização, é utilizada como entrada para o Tesseract OCR, e o texto reconhecido é traduzido do inglês para o português por meio da biblioteca <i>deep-translator</i> . . .	343
6.8	Detecção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.	345
6.9	Simulador interativo de filtragem por circularidade: mova o <i>slider</i> para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).	346
6.10	<i>Pipeline</i> de processamento do <i>QRCode</i> : limiarização, abertura morfológica, inversão e recorte final para decodificação.	349
6.11	Simulador interativo do <i>pipeline</i> de isolamento do <i>QRCode</i> : navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a detecção do contorno quadrado sobre a imagem original do gabarito.	349
6.12	Decodificação de código de barras linear com <i>pyzbar</i> : imagem de entrada e dados extraídos. .	351
6.13	Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado. . .	354
6.14	Área de respostas extraída por <i>getAnswerArea</i> : região retificada contendo os quadros de marcação.	355
6.15	Região do <i>QRCode</i> isolada por <i>segmentQRcode</i> dentro da área de respostas retificada. . . .	356
6.16	Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.	357
6.17	Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.	359
6.18	Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.	361
6.19	Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.	362
6.20	Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia. . .	364
6.21	Simulador: IoU entre máscara de referência e máscara predita	371
6.22	Simulador: Filtro de Marcadores por Circularidade	373
6.23	Simulador: Classificação de Marcações OMR	375
6.24	Simulador: Estimador de Inclinação por Mediana Angular	377
6.25	Simulador: Normalização de Fundo por Divisão	379
6.26	Simulador: Mapa de Variância Local para Detecção de Textura	381
6.27	Simulador: <i>Pipeline</i> de Inspeção — Registro por Translação e Subtração	384
6.28	Simulador: Segmentação Geométrica + Verificação por Decodificação de <i>QRCode</i>	388
7.1	Exemplo motivacional: diferentes frutas formando agrupamentos distintos em um espaço de características.	392
7.2	Panorama do <i>pipeline</i> clássico de classificação de imagens: extração de descritores (LBP, HOG), formação do espaço de características e classificação via k-NN. Não se aplica a modelos de <i>deep learning</i> (CNNs, YOLO), que aprendem <i>features end-to-end</i> diretamente dos pixels. Fonte: elaborado com auxílio do NotebookLM (Google, 2025).	393

7.3	Comparação visual de diferentes descritores aplicados à mesma imagem. Cada descritor revela aspectos distintos da cena.	395
7.4	Importância da normalização das características para o classificador k-NN.	399
7.5	Mapa conceitual do processo de classificação de imagens utilizando descritores clássicos (LBP, HOG, pixels brutos) e classificador tradicional (k-NN). Não se aplica a modelos de <i>deep learning</i> (CNNs, YOLO), que aprendem <i>features end-to-end</i> diretamente dos pixels.	400
7.6	Matrizes de confusão obtidas pelo classificador k-NN utilizando três descritores diferentes. As linhas representam a classe real (Maçã, Banana e Laranja) e as colunas a classe predita. Quanto maior a concentração de valores na diagonal principal, melhor o desempenho do descritor.	403
7.7	Comparação detalhada de acurácia global em cenário realista. Observe como os descritores extraídos estruturalmente (LBP e HOG) superam o uso de intensidades puras de pixels brutos.	404
7.8	Classificação de uma nova amostra pelo algoritmo k-NN. O ponto de teste (estrela) é classificado a partir dos três vizinhos mais próximos, destacados por círculos.	409
7.9	Simulador interativo da fronteira de decisão do k-NN: adicione pontos de treinamento e ajuste o valor de k para observar o efeito sobre a região de decisão.	410
7.10	Amostra de dígitos manuscritos da base <i>load_digits</i> , utilizada como estudo de caso de classificação.	411
7.11	Matriz de confusão do classificador k-NN treinado com vetores de intensidade brutos (pixels).	413
7.12	Comparação de acurácia entre descritores de pixels brutos e HOG para o classificador k-NN (k=3) na base de dígitos.	414
7.13	Acurácia média por validação cruzada (5 partições) em função do parâmetro k, para a base de dígitos com vetores de intensidade brutos.	416
7.14	Acurácia nos conjuntos de treinamento e teste para diferentes valores de k. As regiões coloridas representam, de forma conceitual, tendências de comportamento do classificador: maior risco de sobreajuste (vermelho), compromisso entre viés e variância (verde) e maior risco de subajuste (azul).	418
7.15	Amostras sintéticas das três classes de textura utilizadas no experimento de classificação, geradas com ruído gaussiano e variabilidade intra-classe.	420
7.16	Matriz de confusão do classificador k-NN treinado com descritores LBP para as três classes de textura sintética, incluindo ruído e variabilidade intra-classe.	421
7.17	Métricas de avaliação detalhadas para o classificador k-NN com descritores LBP	423
7.18	Simulador: Classificador k-NN Passo a Passo	429
7.19	Simulador: Precisão x Revocação	431
7.20	Simulador: Código LBP	432
7.21	Simulador: Histograma HOG de uma Celula	434
7.22	Simulador: <i>Pipeline</i> k-NN com escolha de metrica	436
7.23	Mosaicos de referência (formato PGM ASCII) utilizados no EP07_06. Caso 1: grade 2x2 com uma amostra de cada classe. Caso 2: grade 3x3 com classes repetidas e maior variabilidade.	441
7.24	Simulador: Classificacao de um Mosaico de Texturas via LBP + k-NN	441
8.1		445
8.2		446
8.3		446
8.4	Simulador interativo do algoritmo RANSAC: ajuste o limiar de distância e execute o algoritmo para observar a separação entre inliers e outliers.	448
8.5		449
8.6		451
8.7		452
8.8	Simulador: Distância de Hamming entre Dois Descritores Binários	460
8.9	Simulador: RANSAC — Votação por Inliers entre Modelos Candidatos	462
8.10	Simulador: Imagem Integral e Consulta Retangular em O(1)	464
8.11	Simulador: IoU e Supressão de Não-Máximos	467
8.12	Simulador: Rotulagem de Componentes Conectados — Conectividade 4 vs. 8	469

9.1	Simulador interativo de convolução: escolha um kernel e avance passo a passo (ou calcule tudo de uma vez) para observar a construção do mapa de características.	473
9.2	Curva de treinamento de uma CNN simples na base de dígitos: perda de treinamento e acurácia no conjunto de teste ao longo das épocas.	475
9.3	Comparação de acurácia entre os classificadores clássicos do Capítulo 7 (pixels brutos e HOG com k-NN) e a CNN treinada neste capítulo, na mesma base de dígitos.	476
9.4	Comparação de acurácia entre treinar do zero e reaproveitar (transferir) um extrator de características já treinado, ambos com o mesmo pequeno número de exemplos rotulados na tarefa de destino.	479
9.5	Amostras do conjunto de dados sintético de objetos geométricos, com as caixas delimitadoras no formato YOLO sobrepostas.	485
9.6	Detecções do YOLO ajustado por transferência de aprendizado em uma imagem de validação não vista durante o treinamento.	490
9.7	Realidade aumentada baseada em marcador: um marcador ArUco é detectado na cena e sua homografia é reaproveitada para posicionar uma imagem virtual sobre ele.	496
9.8	Medição de um objeto a partir de uma referência de escala conhecida (um cartão de 8,56 cm de largura).	498
9.9	Par estéreo sintético (um objeto em primeiro plano deslocado mais do que o fundo) e o mapa de disparidade calculado — regiões mais claras indicam maior disparidade (objetos mais próximos).	499
9.10	Simulador: Convolução 2D Manual (correlação cruzada + viés + ReLU)	506
9.11	Simulador: Pooling Manual (máximo vs. média)	508
9.12	Simulador: Contagem de Parâmetros — Convolução vs. Camada Totalmente Conectada	510
9.13	Simulador: IoU e Supressão de Não-Máximos	513
9.14	Simulador: Pipeline Integrado — NMS + Medição por Referência de Escala	515

Lista de Tabelas

1.1	Conexão entre PDI, VC e outras áreas da ciência.	29
1.2	Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.	32
1.3	Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.	33
2.1	Convenções de escala e ordem de canais nas principais bibliotecas de PDI.	76
2.2	Comparativo entre estratégias de compactação e eficiência de processamento.	83
2.3	Comparativo de métricas de distância aplicadas à malha de pixels.	84
2.4	Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.	84
2.5	Métodos de interpolação disponíveis em <code>mm.resize</code> e seus respectivos números de vizinhos utilizados no cálculo.	90
3.1	Algoritmo de equalização de histograma.	128
3.2	Interpretação típica dos coeficientes do <i>kernel</i>	138
3.3	Etapas do <i>Unsharp Masking</i>	151
3.4	Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.	154
4.1	Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.	187
4.2	Propriedades de abertura e fechamento.	197
4.3	Sequências do filtro sequencial alternado <code>mm.asf</code>	199
4.4	Comparação entre os operadores <i>clohole</i> e <i>edgeoff</i>	207
4.5	Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.	214
4.6	<i>Pipeline</i> completo do <i>watershed</i> baseado em marcadores para imagens reais.	224
4.7	<i>Pipeline</i> simplificado utilizado no exemplo didático da Figure 4.25.	225
4.8	Comparação entre as abordagens baseadas em componentes conexos e em contornos.	230
5.1	Correspondência entre as regiões do espectro de magnitude após a aplicação do FFT Shift.	264
5.2	Comparação da complexidade da convolução direta, separável e via Transformada Rápida de Fourier (FFT).	270
5.3	Subbandas produzidas pela Transformada <i>Wavelet</i> Discreta 2D (DWT), indicando os filtros aplicados em cada direção e o conteúdo predominante de cada componente.	282
5.4	Comparação entre famílias de <i>wavelets</i> , destacando o comprimento do suporte, o número de momentos nulos, a simetria e aplicações típicas.	283
5.5	Comparação entre a Transformada Discreta de Fourier (DFT) e a Transformada <i>Wavelet</i> Discreta (DWT), destacando suas principais características e aplicações.	291
5.6	Categorias de redundância em imagens digitais e seus respectivos mecanismos de exploração.	292
5.7	Etapas do <i>pipeline</i> de compressão JPEG e seus respectivos fundamentos de projeto.	296
5.8	Comparação estrutural entre os principais formatos de imagem rasterizados.	300
5.9	Síntese das etapas do <i>pipeline</i> de compressão JPEG e seus respectivos impactos.	307
6.1	Imagens do <code>skimage.data</code> utilizadas nos experimentos deste capítulo.	328
6.2	Seleção de imagens públicas representativas disponíveis no módulo <i>skimage.data</i> , organizadas por área de aplicação.	329

7.1	Conjunto de descritores cromáticos, texturais e geométricos utilizados para representar imagens de frutas.	404
7.2	Exemplo simplificado de classificação de frutas utilizando duas características normalizadas. .	406

PDI e VC — Python

Bem-vindo ao livro de PDI e Visão Computacional — versão Python / Português.

Organização

- **Parte I — Fundamentos de PDI** — Representação, histogramas, filtragem, morfologia
 - **Parte II — Visão Computacional** — Segmentação, descritores, detecção, deep learning
-

Prefácio

Este livro constitui uma **obra em permanente atualização** nas áreas de **Processamento Digital de Imagens (PDI)** e **Visão Computacional (VC)**, concebida como material didático interativo para cursos de graduação e pós-graduação em Computação, Engenharia e áreas afins.

! Destaque

A obra fundamenta-se na metodologia descrita em Zampirolli et al. (2025) — extensão de Zampirolli et al. (2024), trabalho premiado na trilha **Recursos e Ambientes Educacionais** da **EduComp 2024**. O conteúdo integra a biblioteca `morph.py`, desenvolvida pelo autor.

Um livro vivo

Este não é um livro estático. Seu conteúdo evolui continuamente à medida que exemplos são aprimorados, novas seções são incorporadas e as abordagens pedagógicas são refinadas com base na experiência de uso e no retorno de estudantes e docentes. Dessa forma, a obra é tratada como um *projeto em constante evolução*, buscando acompanhar tanto os avanços tecnológicos quanto as melhores práticas de ensino em PDI e VC.

Uso de ferramentas de Inteligência Artificial

A elaboração deste livro contou com o apoio de **ferramentas de Inteligência Artificial (IA)** para aprimorar a qualidade do texto, dos exemplos de código e dos materiais de apoio. Ao longo do desenvolvimento, diferentes assistentes de IA, como **DeepSeek**, **Gemini**, **ChatGPT** e **Claude**, foram utilizados de forma iterativa para auxiliar em atividades como:

- aprimoramento da clareza, da precisão e da fluidez das explicações;
- identificação e correção de erros de sintaxe, lógica e digitação em exemplos de código;
- sugestão de melhorias na organização, na estrutura e na apresentação do conteúdo.

Além disso, **algumas ilustrações foram geradas com o auxílio do Gemini**, com o objetivo de facilitar a visualização de conceitos abstratos e complementar as explicações apresentadas ao longo do texto.

Embora essas ferramentas tenham contribuído significativamente para o processo de produção, **a seleção, a validação técnica, a organização e a responsabilidade pelo conteúdo final são inteiramente do autor**.

Mais do que utilizar IA para gerar conteúdo, o maior desafio consiste em selecionar, verificar, adaptar e organizar esse material de forma pedagogicamente consistente. Nesse contexto, merece destaque o conjunto de **simuladores interativos** desenvolvidos em JavaScript e incorporados às versões HTML e IPYNB do livro, os quais proporcionam experimentação prática e contribuem para a consolidação dos conceitos apresentados.

Como este livro é produzido

O conteúdo é integralmente desenvolvido em **Quarto** e armazenado na pasta `all` do repositório github.com/fzampirolli/pdi-vc. A partir de um único código-fonte, o livro é automaticamente publicado em diferentes formatos:

Formato	Descrição
HTML	Versão <i>web</i> com simuladores interativos e navegação entre capítulos: fzampirolli.github.io/pdi-vc
PDF	Versão para impressão ou leitura <i>offline</i> : livro.pt.py.pdf
Notebooks (.ipynb)	Compatíveis com Jupyter e Google Colab , permitindo executar, modificar e experimentar todos os exemplos de código e os Exercícios de Programação (EPs). Um filtro personalizado preserva referências cruzadas, numeração de figuras e tabelas, além de formatar automaticamente as citações no padrão ABNT.

Cada capítulo disponibiliza botões **Executar Colab**, que direcionam para dois ambientes complementares:

1. **Parte Teórica**, localizada na abertura do capítulo, contendo todos os exemplos executáveis.
2. **Parte Prática**, na seção  **Parte Prática com Exercícios de Programação**, dedicada aos EPs e seus **simuladores interativos**.

A geração dos *notebooks* é totalmente automatizada pelo *script* `gerar_notebooks_alunos.py`, que adapta o conteúdo para ambientes interativos, permitindo sua execução sem necessidade de instalar o Quarto.

A biblioteca `morph.py`

A biblioteca `morph.py` (Zampirolli et al., 2025) acompanha toda a obra como uma ferramenta de apoio ao ensino. Seu objetivo não é substituir bibliotecas consolidadas, como **OpenCV**, **scikit-image**, **NumPy** ou **Matplotlib**, nem competir com elas em desempenho ou abrangência. Em vez disso, procura **tornar transparentes os algoritmos fundamentais de PDI-VC**, permitindo que o estudante compreenda sua implementação, modifique o código e desenvolva novas funcionalidades.

Sempre que possível, a biblioteca concilia clareza didática com desempenho. Assim, versões voltadas ao ensino coexistem com implementações otimizadas para aplicações práticas.

Herança Técnica

A estrutura da `morph.py` tem como base ferramentas de Morfologia Matemática desenvolvidas no Brasil, como **MMachLib** (Lotufo et al., 1997) e **MMach** (Barrera et al., 1998), além da **mmorph**, utilizada na obra de Dougherty; Lotufo (2003). Essas bibliotecas serviram de referência para o ensino da área durante décadas.

A filosofia da biblioteca pode ser observada, por exemplo, nos operadores morfológicos de dilatação:

- `mm.dil`: versão de uso geral, que utiliza implementações otimizadas da OpenCV para imagens maiores;
- `mm.dil0`: implementação didática da dilatação para **elementos estruturantes planares**, seguindo diretamente a definição clássica da Morfologia Matemática;
- `mm.dil1`: implementação didática da dilatação para **funções estruturantes (*kernels* não planares)**, seguindo rigorosamente sua formulação matemática.

As implementações `mm.dil0` e `mm.dil1` priorizam fidelidade às equações apresentadas ao longo do livro em detrimento do desempenho. Dessa forma, o código permanece simples, permitindo que o estudante acompanhe cada etapa do algoritmo, estabelecendo uma correspondência direta entre a teoria e sua implementação.

Essa filosofia estende-se a diversos operadores da biblioteca, como rotulação, transformada de distância e *watershed*. Em vez de tratar esses algoritmos como caixas-pretas, a biblioteca incentiva o leitor a compreender sua implementação, modificá-la e ampliá-la de forma colaborativa.

A biblioteca também oferece funções para leitura, exibição, conversão de cores, filtragem e morfologia matemática por meio de uma interface simples:

```
from morph import mm

img = mm.gray(mm.read('lena.jpg')) # leitura e conversão para níveis de cinza
grad = mm.gradm(img)               # gradiente morfológico
mm.show(grad)                       # exibição
```

O código-fonte está disponível em:

<https://github.com/fzampirolli/pdi-vc/tree/master/morph>

Exercícios de Programação (EPs) e Validação Automática

Cada unidade do livro inclui **Exercícios de Programação (EPs)** práticos e de complexidade crescente, projetados para consolidar os conceitos apresentados ao longo do capítulo. Cada EP é acompanhado por um **simulador interativo**, disponível nas versões HTML e IPYNB, que permite ao estudante manipular parâmetros, visualizar o comportamento dos algoritmos e desenvolver uma compreensão intuitiva do problema antes de iniciar sua implementação. Dessa forma, o processo de aprendizagem combina experimentação, programação e validação automática.

Estrutura e Validação Local

A validação das soluções é feita localmente pela classe `TestSuite` (`testsuite.py`), que compara a saída do programa com os arquivos de casos de teste (`.cases`):

```
TestSuite("EP01_01.py").run()
```

O sistema suporta múltiplas linguagens (Python, Java, C++, C, JavaScript e R) e pode ser integrado diretamente ao Moodle.

Integração com o Moodle (VPL)

Os EPs dos *notebooks* podem ser convertidos em atividades **VPL** (*Virtual Programming Lab*) no Moodle. O *script* `ep_tools.py` (executado via `make build`) automatiza a exportação dos EPs do final de cada capítulo em duas etapas:

1. **Extração** (`make eps`): gera um HTML interativo independente por EP, com enunciado, exemplos e simulador, em `gen/book/eps/<versao>/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_01.html
2. **Conversão** (`make moodle`): transforma o HTML em um fragmento autocontido, sem dependência de CSS externo, pronto para ser colado no editor do Moodle, em `gen/book/eps/<versao>_moodle/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt/_moodle/EP01_01.html

Ao publicar com `make publish`, essas **duas versões** de cada EP ficam disponíveis nos *links* indicados. O professor pode combinar as duas estratégias: colar a versão Moodle no editor VPL (com simulador funcional) e incluir no enunciado um *link* para a versão completa, onde as fórmulas são renderizadas corretamente pelo MathJax.

⚠ Revisão obrigatória antes de publicar no Moodle

Embora automatizada, a conversão exige revisão do professor em razão das limitações do editor TinyMCE/HTML Purifier do Moodle:

- **Fórmulas matemáticas** — O TinyMCE descarta barras invertidas (`\`), corrompendo equações LaTeX. Por isso, a versão Moodle converte fórmulas simples para HTML puro (entidades como `≥`, `×`). Fórmulas complexas (`\begin{cases}`, matrizes, integrais) podem sair incomple-

tas — revise e, se necessário, reescreva-as em texto ou indique a versão completa via *link*.

- **Figuras externas** — Imagens complementares devem ser inseridas manualmente na atividade VPL.
- **Simuladores** — Funcionam perfeitamente desde que utilizem JavaScript padrão com estilos *inline* e manipulação direta do DOM (`getElementById`). **Atenção:** se incluir fórmulas em LaTeX que contenham o caractere `\` no Moodle, os simuladores podem parar de funcionar.

Como Publicar no Moodle

1. Acesse a versão Moodle do EP desejado:

```
https://fzampirolli.github.io/pdi-vc/eps/py.pt_moodle/EPXX_YY.html
```

2. Copie o código-fonte completo da página (`Ctrl+U` → `Ctrl+A` → `Ctrl+C`).
3. No Moodle, crie a atividade VPL, acesse o editor HTML, cole o conteúdo (`Ctrl+V`) e salve.
4. Importe os arquivos `.cases` da pasta `all/capXX/casos/` nas configurações de testes do VPL.

Reaproveitamento dos Casos de Teste

Os arquivos `.cases` são compatíveis com o VPL, permitindo usar o mesmo conjunto de testes tanto no desenvolvimento local quanto na correção automatizada no Moodle.

Idiomas e Conversão de Código

O **núcleo de referência** é escrito em **Português** com exemplos em **Python**. Versões em outros idiomas e linguagens de programação são geradas via **APIs de IA**, por meio de um fluxo automatizado que realiza a tradução textual e a conversão de sintaxe do código.

Nota

As versões traduzidas via IA servem como **ponto de partida** e podem conter imprecisões; recomenda-se revisão antes da aplicação em sala de aula.

Código aberto

O projeto é regido por princípios de código aberto. O repositório público reúne o texto, os códigos, as imagens e os *scripts*: github.com/fzampirolli/pdi-vc

Cada versão do livro é arquivada permanentemente no **Zenodo** com um DOI citável. Para citar este material em trabalhos acadêmicos:

ZAMPIROLI, Francisco de Assis. **PDI+VC — Processamento Digital de Imagens e Visão Computacional**. UFABC, 2026. DOI: [10.5281/zenodo.20784606](https://doi.org/10.5281/zenodo.20784606)

Antes de começar: *Notebooks* em Python

O conceito de **Literate Programming** (Programação Literária), proposto por Donald Knuth (Knuth, 1984), fundamenta a estrutura deste material. A lógica inverte o paradigma tradicional: o programa é escrito para a leitura humana, assemelhando-se a um ensaio, enquanto o código é extraído separadamente para execução computacional.

O conteúdo é estruturado em *notebooks* — documentos que intercalam **células de texto** (em [Markdown](#)) e **células de código** (em Python).

- **Execução:** células de código são identificadas por `[ ]`. A execução pode ser realizada com **Shift + Enter** ou pelo botão  da interface.
- **Ambientes:** os *notebooks* podem ser executados localmente, por meio do [Jupyter](#) ou do [Visual Studio Code \(VS Code\)](#), bem como em ambientes de nuvem, como o [Google Colab](#).

 Nota sobre o formato

Em ambientes interativos, o código pode ser modificado e executado. Nas versões estáticas (HTML ou PDF), os blocos de código têm finalidade de leitura e referência, sem prejuízo à integridade das explicações.

Parte I

Parte I — Fundamentos de PDI

Capítulo 1

Fundamentos e Primeiros Passos



Este capítulo inaugura a **Parte 1** do livro, dedicada aos fundamentos do Processamento Digital de Imagens (PDI). São apresentados a representação matemática de imagens digitais e os principais métodos para sua manipulação, utilizando a linguagem Python e a biblioteca `morph.py` (Zampirolli et al., 2025).

1.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Compreender a natureza física e matemática da imagem digital $f(x, y)$.
- Identificar as faixas do espectro eletromagnético relevantes para PDI.
- Configurar o ambiente de desenvolvimento em Python.
- Realizar operações básicas: leitura, exibição e salvamento de imagens.
- Manipular estruturas de matrizes (NumPy) sem cair em armadilhas de memória.
- Acessar e modificar intensidades de pixels individualmente.
- Aplicar limiarização manual.

1.2 Antes de começar: *Notebooks* em Python

Este material foi construído sob o conceito de *Literate Programming* (Programação Literária), idealizado por Donald Knuth na década de 1980 (Knuth, 1984). Knuth — também criador do sistema **TeX** para tipografia digital — propôs que os programas fossem escritos como uma narrativa lógica para seres humanos, intercalando código e documentação.

Para executar uma célula, pressione Shift + Enter ou clique no botão ▷.

i Nota sobre o formato

Nas versões renderizadas (PDF ou HTML), o código é apresentado em blocos estáticos para fins de leitura e referência. A execução interativa requer o acesso via Google Colab (disponível no topo da página) ou em ambiente local via VSCode ou Jupyter Notebook.

1.3 Fundamentos

O estudo de sistemas baseados em imagens compreende um ecossistema de disciplinas integradas que transformam dados visuais brutos em conhecimento estruturado. Enquanto algumas áreas focam na geração de representações, outras dedicam-se ao tratamento e à análise desses dados para dar suporte a aplicações tecnológicas complexas.

O diagrama apresentado no Figure 1.1 estabelece a distinção e complementaridade entre o Processamento Digital de Imagens (PDI) e a Visão Computacional (VC). O PDI, destacado em **verde**, tem como foco

a transformação de imagem para imagem, visando melhoria de qualidade ou pré-processamento, como a remoção de ruídos e realce de contraste.

Em contrapartida, a VC, assinalada em **azul**, foca na interpretação do conteúdo visual para extrair modelos ou informações, como o reconhecimento de objetos e gestos. A região de intersecção ilustra a sinergia entre as áreas, onde o PDI prepara os dados visuais para a interpretação pela VC. O mapa também demonstra as interconexões de ambas as disciplinas com áreas como Robótica, Computação Gráfica, Inteligência Artificial (IA) e Neurociência.

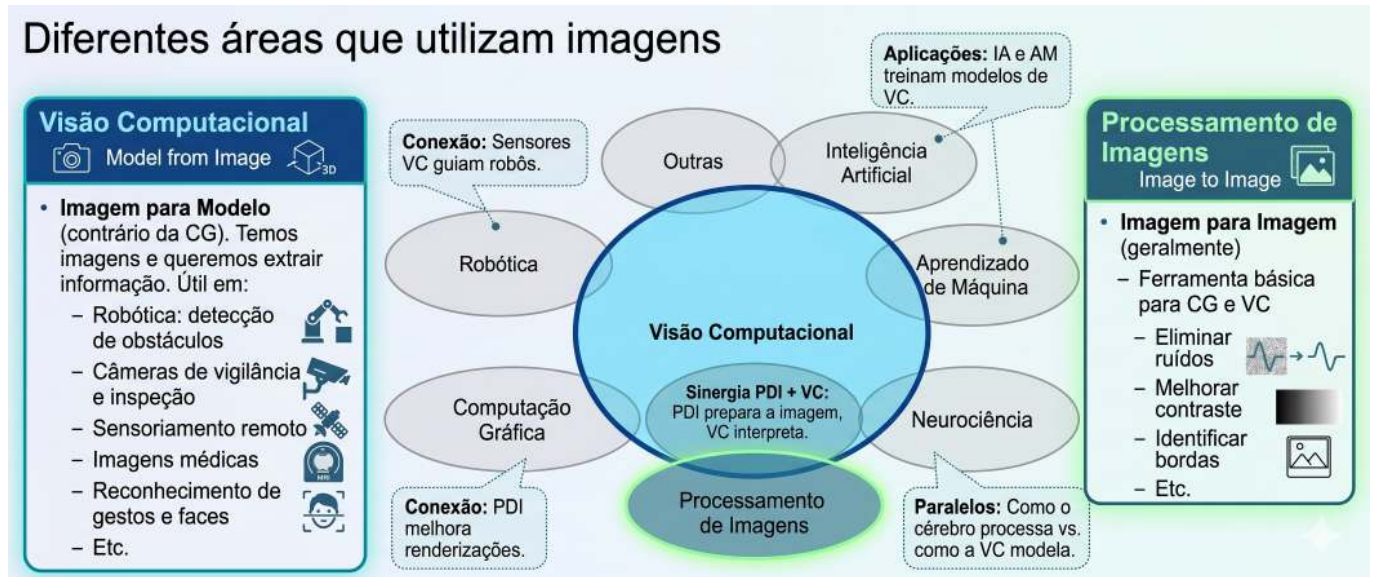


Figura 1.1: Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.

1.3.1 Visão Computacional

- **Foco:** *Imagem* → *Modelo* (caminho inverso da Computação Gráfica).
- **Objetivo:** Extrair informação de alto nível a partir de imagens ou vídeos.
- **Aplicações típicas:**
 - Robótica - detecção de obstáculos, localização e navegação autônoma.
 - Vigilância e inspeção - reconhecimento de eventos, leitura de placas, controle de qualidade.
 - Sensoriamento remoto - análise de imagens de satélite, mapeamento ambiental.
 - Imagens médicas - detecção de tumores, segmentação de órgãos, auxílio ao diagnóstico.
 - Interação humano-computador - reconhecimento de gestos, expressões faciais, rastreamento ocular.
- **Relação com outras áreas:** utiliza técnicas de Aprendizado de Máquina e IA para classificar e interpretar cenas; serve como “olhos” da Robótica.

1.3.2 Processamento Digital de Imagens (PDI)

- **Foco:** *Imagem* → *Imagem* (geralmente - transformação de uma imagem em outra).
- **Objetivo:** Melhorar a qualidade visual ou extrair características de baixo nível.
- **Aplicações comuns:**
 - Eliminação de ruídos (filtros de média, mediana, gaussiano).
 - Melhoria de contraste (equalização de histograma, ajuste gamma).
 - Detecção de bordas (Sobel, Canny, Laplaciano).
 - Segmentação (limiarização, crescimento de regiões, *watershed*).

- Transformações geométricas (redimensionamento, rotação, correção de perspectiva).
- **Relação com outras áreas (ver Table 1.1):**
 - É a **base** para a maioria dos sistemas de VC (pré-processamento).
 - A Computação Gráfica frequentemente aplica PDI para pós-processamento (ex.: suavização, realce).
 - Técnicas de IA podem otimizar parâmetros de processamento (ex.: aprendizado de filtros).

Tabela 1.1: Conexão entre PDI, VC e outras áreas da ciência.

Área	Relação com PDI e VC
Inteligência Artificial	Fornecer modelos (redes neurais, SVM) que interpretam saídas da VC.
Robótica	Consome dados de VC para tomar decisões (navegação, manipulação).
Aprendizado de Máquina	Usa descritores extraídos pelo PDI/VC para treinar classificadores.
Computação Gráfica	Caminho inverso: <i>modelo</i> $\rightarrow$ <i>imagem</i> ; muitas vezes aplica PDI para renderização realista.
Neurociência	Inspira modelos de PDI (ex.: filtros semelhantes a células ganglionares da retina).

1.4 Etapas do PDI

As etapas do PDI são apresentadas na Figure 1.2, que podem ser compreendidas como uma cadeia de transformações que reduz a redundância dos dados em busca de significado:

- **Baixo Nível:** Atua diretamente sobre os pixels da imagem ruidosa para realizar melhorias e filtrações, gerando como saída uma imagem limpa ou realçada.
- **Médio Nível:** Recebe a imagem tratada e realiza a segmentação e descrição, transformando a matriz de pixels em atributos estruturados (forma, tamanho e textura).
- **Alto Nível:** Utiliza a tabela de atributos para alimentar processos de lógica e inteligência artificial, resultando na decisão ou reconhecimento final (como o diagnóstico médico).

A Figure 1.3 detalha a sequência completa do PDI, desde a captura até a interpretação. O fluxo inicia-se na Aquisição da Imagem (1) e prossegue com o Melhoramento (2) e a Restauração (3). Em seguida, o conteúdo é isolado pela Segmentação (4) e refinado pela Morfologia (5). A transição crucial ocorre na Representação e Descrição (6), onde objetos visuais são convertidos em dados matemáticos (área, perímetro etc.), permitindo o Reconhecimento (7). Processos auxiliares incluem o Processamento de Imagem Colorida e a Compressão, que contribuem para a eficiência do armazenamento e da análise.

1.5 Formação da Imagem e o Espectro

O processo de formação de uma imagem é fundamentado na interação entre a matéria e a energia radiante. Essencialmente, uma imagem é concebida quando um **sensor registra a radiação** resultante da interação com um objeto físico. No contexto da visão humana e da fotografia convencional, este fenômeno depende de uma fonte de luz que ilumine a cena; as características dos objetos são então codificadas através das **variações de intensidade e cor** da luz que atinge o sensor, conforme ilustrado na Figure 1.4.

A **luz visível** ocupa apenas uma pequena faixa do espectro eletromagnético — entre **380 nm (violeta)** e **750 nm (vermelho)** — conforme ilustrado na Figure 1.5. Sensores digitais convencionais operam nessa mesma janela, mas equipamentos especializados podem captar radiações invisíveis ao olho humano, como o infravermelho e os raios X. Em PDI, a imagem formada depende diretamente da sensibilidade espectral do sensor utilizado.

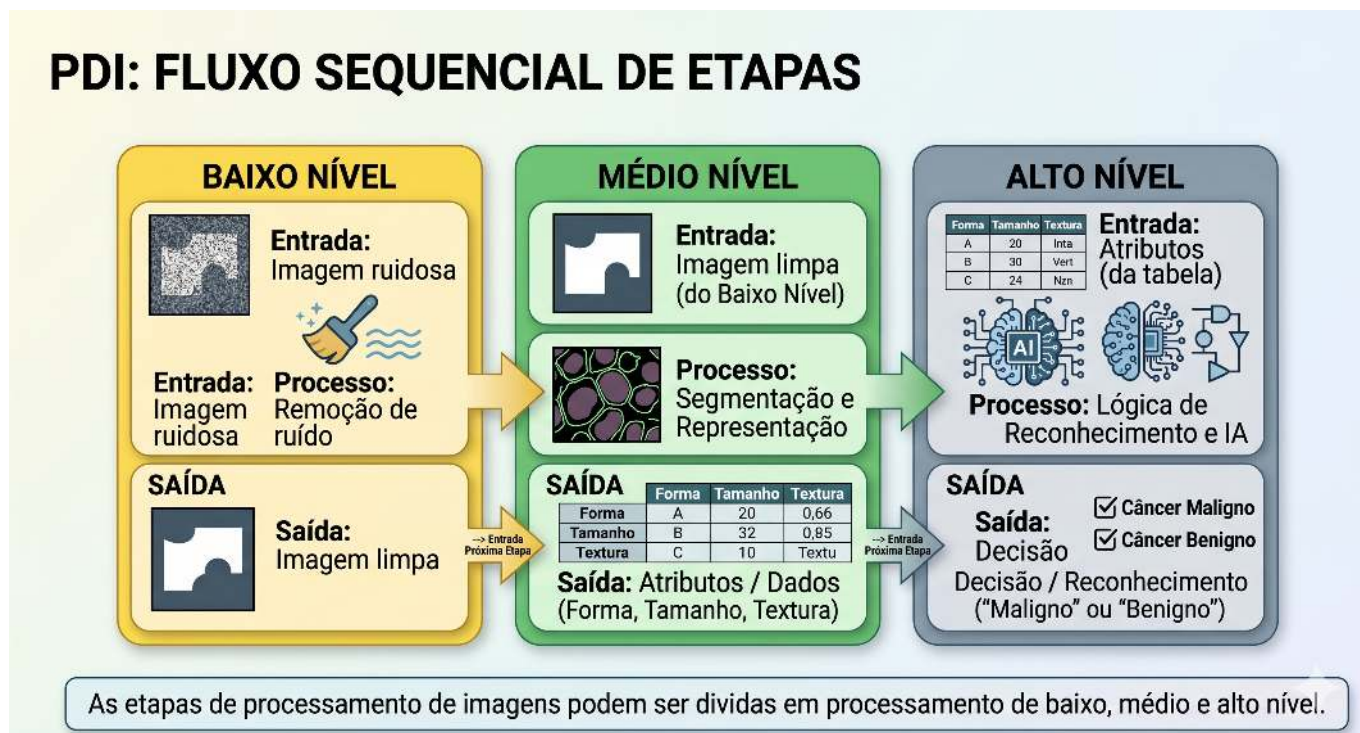


Figura 1.2: Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.

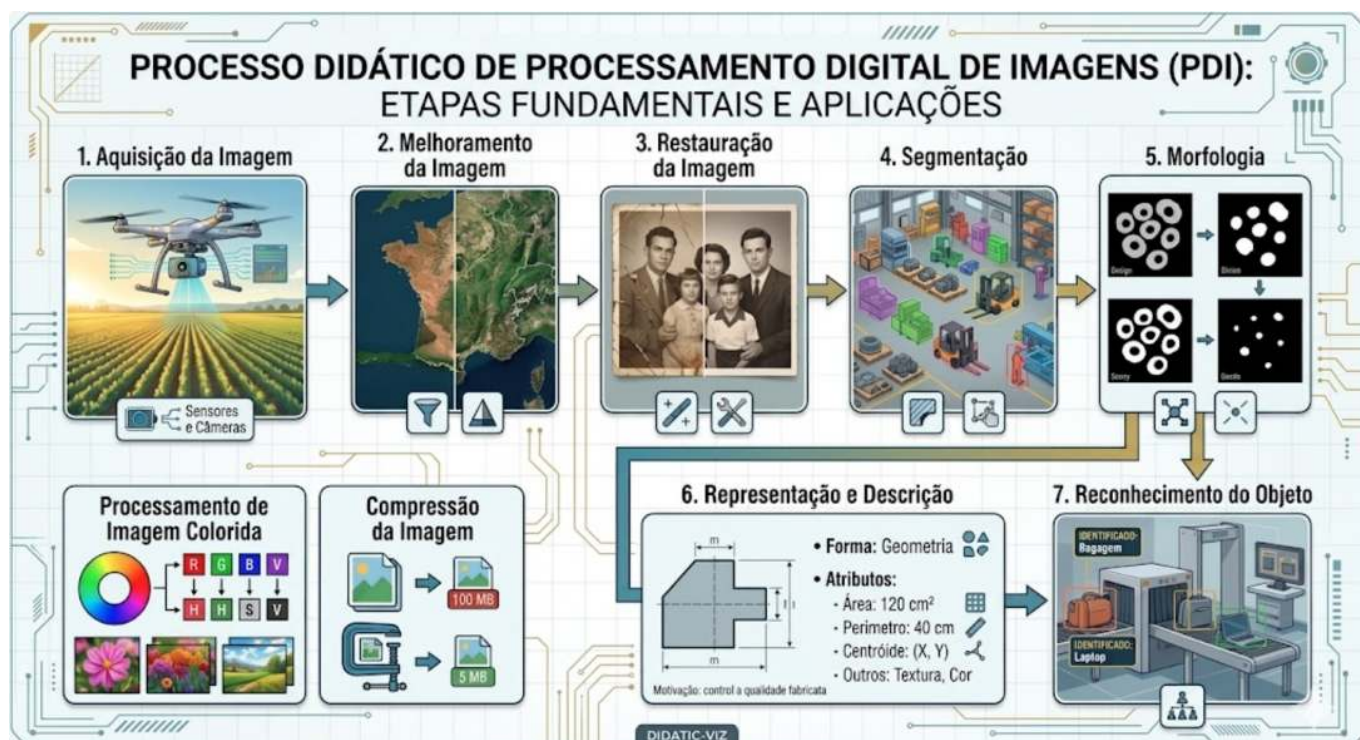


Figura 1.3: Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.

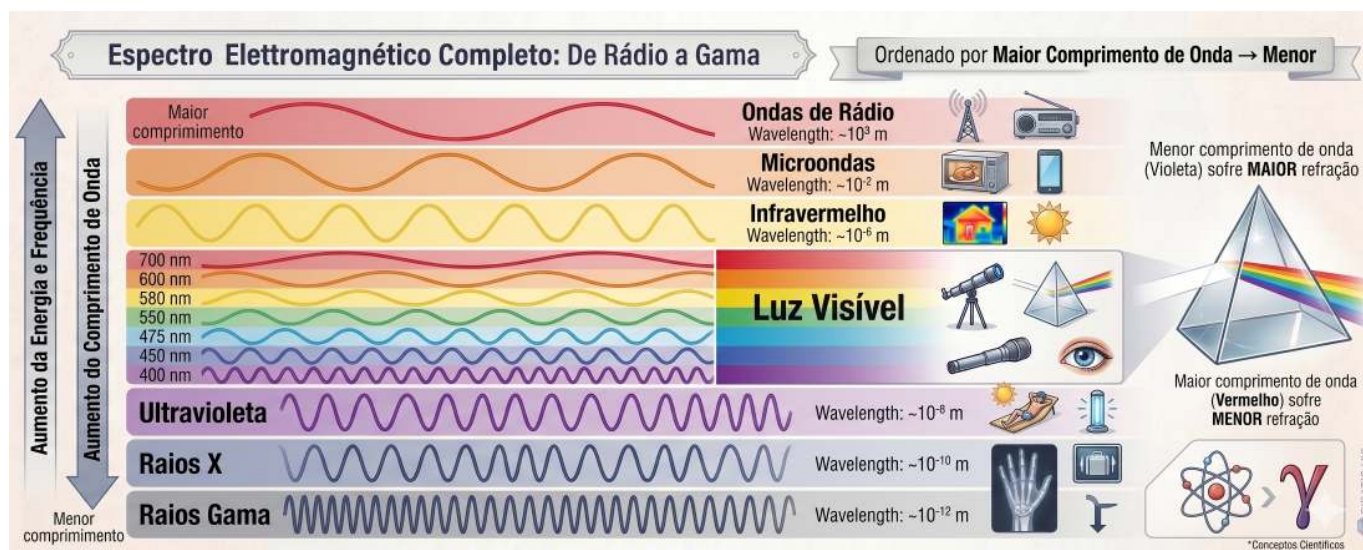


Figura 1.4: Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.

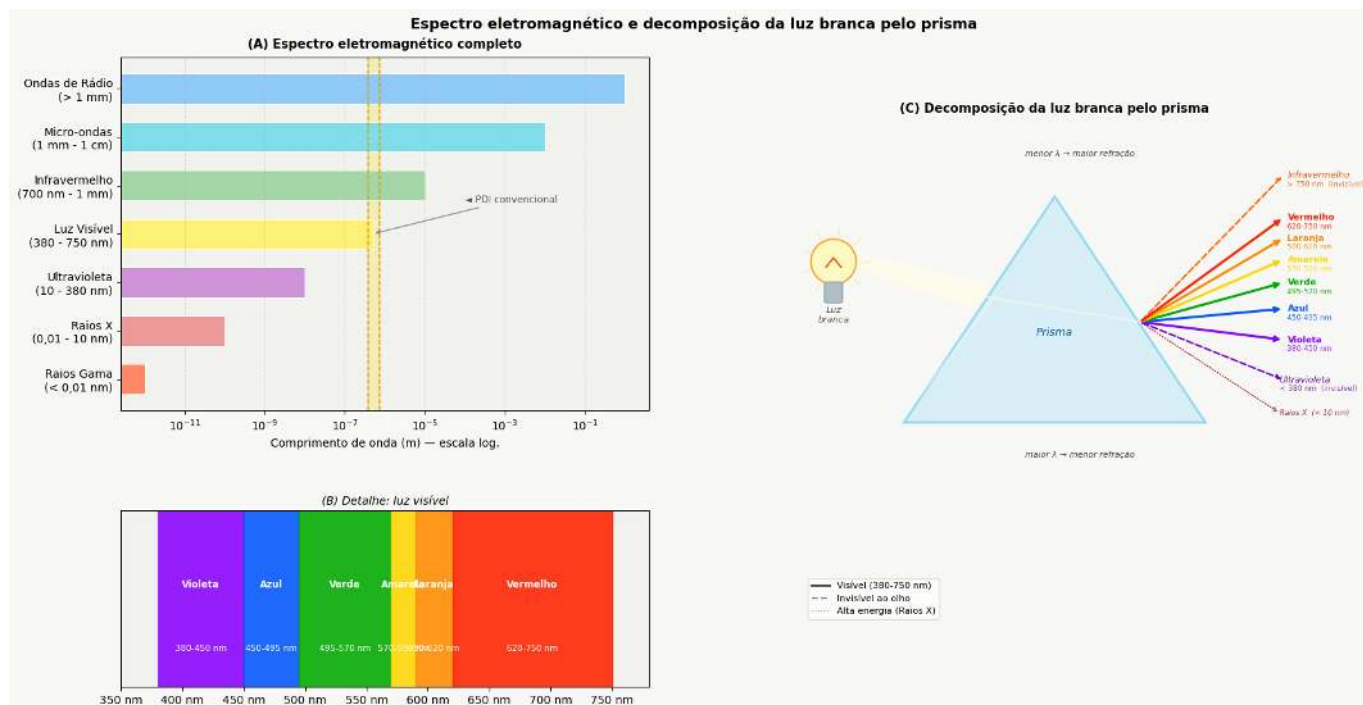


Figura 1.5: (A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.

A partir desse processo físico de aquisição, torna-se possível modelar matematicamente a imagem digital como uma função bidimensional discreta, na qual cada ponto da cena é representado por amostras numéricas de intensidade luminosa, formalizando assim os conceitos de pixel e de imagem digital apresentados na próxima seção.

1.6 O que é uma Imagem Digital?

Uma **imagem digital** é formada por uma grade de **pixels** (*Picture Elements*), onde cada pixel é a menor unidade elementar da imagem.

💡 O que é um Pixel?

Um **pixel** é a menor unidade endereçável que compõe uma imagem digital. Cada pixel ocupa uma posição única na grade e armazena um ou mais valores numéricos que representam sua intensidade ou cor.

Representação Matemática

Diferente de uma função contínua, o domínio de uma imagem digital é um plano retangular finito $\mathbb{E} \subset \mathbb{Z}^2$, que representa a grade de amostragem. Este domínio é indexado por coordenadas inteiras:

$$\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\} \tag{1.1}$$

Onde:

- L : representa a largura da imagem (número de colunas).
- H : representa a altura da imagem (número de linhas).

A imagem digital é uma função que associa cada par de coordenadas (x, y) a um ou mais valores que descrevem a aparência do pixel.

$$f : \mathbb{E} \rightarrow \mathcal{V} \tag{1.2}$$

O conjunto $\mathcal{V}$ define os valores possíveis para o pixel (codomínio), variando conforme o tipo de imagem, como demonstrado na Table 1.2.

Tabela 1.2: Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.

Tipo de imagem	$\mathcal{V}$ (valores do pixel)	Representação
Binária	$\{0, 1\}$ ou $\{0, 255\}$	■ □
Tons de cinza	$\{0, 1, \dots, 255\}$	[] [=] [#] ■
Colorida (RGB)	$\{0, \dots, 255\}^3$ (triplas ordenadas de valores)	■ ■ ■

Exemplo prático: Uma imagem colorida no modelo RGB pode ser representada matematicamente por uma função que associa três valores de intensidade a cada pixel. Computacionalmente, essa representação corresponde a três matrizes sobrepostas — os canais vermelho (*Red*), verde (*Green*) e azul (*Blue*) — nas quais cada elemento armazena a intensidade luminosa do respectivo canal em uma determinada posição da imagem.

Para viabilizar os experimentos práticos de PDI e VC, este livro utiliza o ecossistema científico do Python, integrando bibliotecas voltadas à manipulação matricial, ao processamento de imagens e à visualização de resultados, apresentadas a seguir.

1.7 Configuração do Ambiente

Este material fundamenta-se no ecossistema científico do Python, com destaque para o **NumPy** (matemática matricial), **OpenCV** (VC padrão de mercado) e a biblioteca **morph.py** (Zampirolli et al., 2025), desenvolvida especificamente para fins didáticos neste livro, ver Table 1.3.

Atualmente, o projeto disponibiliza a versão em **Python** e **Português**. A arquitetura do sistema, no entanto, foi projetada para expansão futura, permitindo a adaptação automatizada para outras linguagens de programação (como C++ e Java) e idiomas, por meio de APIs de tradução.

Os **Exercícios de Programação (EPs)** do final de cada capítulo integram o módulo `testsuite.py`, já disponível para práticas em Python, C, C++, Java, JavaScript e R, garantindo a validação imediata das soluções propostas nos *notebooks* de estudo.

Tabela 1.3: Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.

Biblioteca	Função principal
<code>numpy</code>	Representação matricial de imagens
<code>opencv-python</code>	Leitura, escrita e operações de VC
<code>matplotlib</code>	Visualização de imagens e gráficos
<code>morph.py</code>	Abstração didática das operações de PDI

Versões do morph.py

A biblioteca `morph.py` possui duas versões públicas:

- **Versão 1.0** — versão original, publicada junto ao artigo apresentado no EduComp 2024: <https://github.com/fzampirolli/morph>
- **Versão 1.1** — versão compacta, utilizada nas atividades deste livro: <https://github.com/fzampirolli/pdi-vc/blob/master/morph/morph.py>

A versão 1.1 foi necessária para uso nas atividades do **Moodle/VPL** (Laboratório Virtual de Programação). Na versão 1.0, bibliotecas como `matplotlib`, `requests` e `skimage` eram carregadas no momento do `import morph`, causando erro de memória (Jail: out of memory, 128MiB) no ambiente restrito do VPL.

Na versão 1.1, esses *imports* foram convertidos para **carregamento lazy**: cada biblioteca é importada apenas dentro do método que a utiliza, e somente quando esse método é de fato chamado. Assim, um simples `import morph` carrega apenas `numpy` e `cv2`, que são suficientes para a maioria dos EPs.

```
import os, importlib, urllib.request, numpy as np, matplotlib.pyplot as plt

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

# Verifica se o atributo existe antes de imprimir
version = getattr(morph, "__version__", "local_file")

print(f" Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: {version}).")
print(f" Funções disponíveis no mm: \n{dir(mm)[-5:]}...")
```


Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: 1.1.0).
 Funções disponíveis no mm:
 ['verifyBoundingBox', 'watershed', 'watershed0', 'watershedB', 'write']...

1.8 Fundamentos de Matrizes - atenção à cópia de referências

Como uma imagem digital é uma matriz, precisamos saber criá-las corretamente. Em Python, existe uma armadilha comum ao usar o operador `*` em listas:

⚠ Atenção à cópia de referências

Ao fazer `m = [[0]*2]*3`, você não cria 3 linhas independentes, mas sim 3 referências para a **mesma** linha. Alterar um valor em uma linha alterará todas as outras!

Para visualizar esse comportamento, você pode testar o código no [Python Tutor](#) e comparar com a forma correta de criar matriz com listas: `m = [[0]*2 for _ in range(3)]`.

A forma recomendada e padrão em PDI é usar **NumPy**, que cria matrizes com dados independentes e oferece eficiência computacional. O código a seguir mostra diferentes formas de criação de imagens sintéticas — o resultado é exibido na Figure 1.6.

```
# Criando uma imagem preta (zeros) de 4, 6 pixels
img_preta = np.zeros((4, 6), dtype='uint8')
img_preta[0,0] = 255 # pixel branco no canto superior esquerdo

# Criando uma imagem branca (255) de 4, 6 pixels
img_branca = np.ones((4, 6), dtype='uint8') * 255
img_branca[3,5] = 0 # pixel preto no canto inferior direito

# Criando uma imagem aleatória para testes (ruído)
img_random = mm.randomImage(4, 6, maxValue=255)

print("Matriz aleatória gerada:")
print(mm.drawImg(img_random))

mm.show(
    [img_preta, img_branca, img_random],
    titles=[
        "Predominantemente preta\n(com 1 pixel branco em (0,0))",
        "Predominantemente branca\n(com 1 pixel preto em (3,5))",
        "Imagem aleatória\n(simulação de ruído)"
    ],
    cols=3,
    figsize=(9, 3),
    axis=True
)
```

Matriz aleatória gerada:

77	111	74	119	46	136
219	115	171	222	22	29
76	66	110	242	146	133
133	42	182	15	37	103

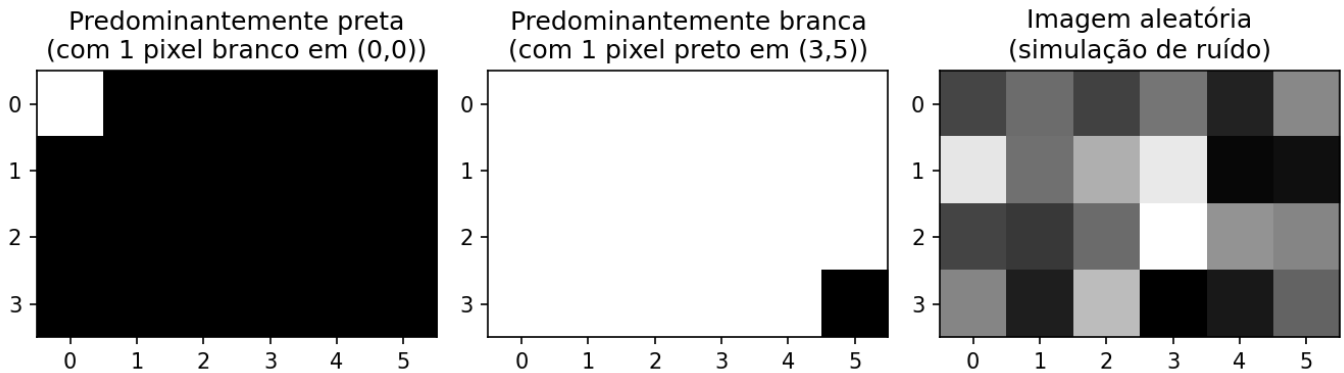


Figura 1.6: Exemplos de imagens sintéticas representadas matricialmente.

1.9 Lendo e Exibindo Imagens

Em bibliotecas como o [NumPy](#), [OpenCV](#) e [scikit-image](#), imagens digitais são frequentemente representadas computacionalmente como *arrays* do NumPy. Dessa forma, operações matriciais e conceitos de álgebra linear podem ser aplicados diretamente ao PDI.

Uma das operações fundamentais em PDI é a leitura de imagens. Na biblioteca `morph.py`, a função `mm.read()` permite carregar imagens tanto de arquivos locais quanto de URLs, como ilustrado na Figure 2.7.

```
import os
import numpy as np

url = "https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg"
caminho = "imagens/mandrill.png"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img = np.array(img_obj)

print(f"Dimensões (H, W, Canais): {img.shape}")
print(f"Tipo de dado: {img.dtype}")

mm.show(img, title="Exemplo: Captura RGB")
```

```
Dimensões (H, W, Canais): (1365, 2048, 3)
Tipo de dado: uint8
```

Exemplo: Captura RGB



Figura 1.7: Mandrill (*Mandrillus sphinx*) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0).

Alternativa: *Download* da Imagem para Armazenamento Local

Em ambientes nos quais a leitura direta de URLs esteja indisponível — devido a restrições de rede, políticas de *firewall* ou ausência de conectividade — uma alternativa consiste em realizar previamente o *download* da imagem para o sistema de arquivos local. Após o armazenamento, a imagem pode ser carregada normalmente pela função `mm.read()`. No exemplo a seguir, o arquivo é salvo localmente com o nome `mandrill.png`.

```
!wget -O mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

```
img = mm.read('mandrill.png')
mm.show(img, title="Exemplo: Captura RGB (arquivo local)")
```

Explicação

- `wget -O mandrill.png <URL>` realiza o *download* da imagem e a armazena localmente com o nome especificado;
- `mm.read('mandrill.png')` efetua a leitura do arquivo diretamente do sistema de arquivos, sem necessidade de requisições HTTP adicionais;
- essa estratégia reduz a dependência de conectividade durante a execução dos experimentos e evita *downloads* repetidos da mesma imagem.

Nota

O prefixo `!` é utilizado em ambientes baseados em *notebooks*, como [Jupyter Notebook](#), [JupyterLab](#) e [Google Colab](#), para executar comandos do sistema operacional diretamente em células de código. Em terminais convencionais, o comando deve ser utilizado sem o prefixo `!`.

Caso o utilitário `wget` não esteja instalado, pode-se utilizar alternativamente:

```
!curl -o mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

1.10 Conversão de Tipos e Limiarização

Conforme vimos, uma imagem colorida no espaço RGB é representada pela função:

$$f : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}^3$$

Ou seja, para cada pixel (x, y) , temos três valores (R, G, B) que definem sua cor.

Conversão para Tons de Cinza

Para converter uma imagem RGB em tons de cinza (*grayscale*), é necessário combinar os três canais em um único valor de intensidade g , que representa o brilho percebido. Como o olho humano não é igualmente sensível ao vermelho, verde e azul, utiliza-se uma **média ponderada**. O padrão [ITU-R BT.601](#) (ITU-R, 2011) define os seguintes pesos:

$$g = 0.299 R + 0.587 G + 0.114 B \quad (1.3)$$

Após o cálculo, o valor g é arredondado para o inteiro mais próximo e ajustado ao intervalo $[0, 255]$. O resultado é uma nova imagem, agora em tons de cinza, representada por:

$$f_{\text{cinza}} : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}$$

Limiarização (*Thresholding*)

A partir da imagem em tons de cinza $f_{\text{cinza}}(x, y)$, uma operação fundamental é a **limiarização**, que produz uma imagem **binária** (apenas preto e branco). Para isso, escolhe-se um valor de corte T (geralmente no intervalo $[0, 255]$) e define-se:

$$f_{\text{bin}}(x, y) = \begin{cases} 255 & \text{se } f_{\text{cinza}}(x, y) > T \\ 0 & \text{caso contrário} \end{cases} \quad (1.4)$$

Exemplo: Com $T = 128$, pixels com intensidade acima de 128 tornam-se brancos (255); os demais tornam-se pretos (0).

A limiarização é amplamente usada para **segmentar objetos do fundo**, extrair bordas ou criar máscaras binárias para processamento posterior.

Nota: O valor 255 representa o branco máximo em imagens de 8 bits, enquanto 0 representa o preto absoluto.

Exemplo prático de conversão e limiarização

A Figure 1.8 ilustra os principais passos para transformar uma imagem colorida em tons de cinza e, em seguida, convertê-la em uma imagem binária por limiarização. O código a seguir implementa essas etapas:

```
# 1. Converter para Tons de Cinza
img_gray = mm.gray(img)

# 2. Aplicar limiar (Pixels > 128 tornam-se 255, outros 0)
limiar = 128
img_binaria = mm.threshold(img_gray, limiar)

# Uso da nova função
mm.show(
    [img, img_gray, img_binaria],
```

```
titles=["Original", "Tons de Cinza", f"Binária (T={limiar})"],
cols=3
)
```



Figura 1.8: Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).

1.11 Limiarização pelo método de Otsu

Conforme apresentado na Equation 1.4, a limiarização converte uma imagem em tons de cinza para binária usando um valor de corte T . Até agora fixamos $T = 128$ manualmente.

```
img_bin_fixo = mm.threshold(img_gray, T=128)
```

Entretanto, a escolha manual de T nem sempre é trivial. A biblioteca `mm` oferece uma alternativa automática: quando o parâmetro `limiar` **não é fornecido**, a função `mm.threshold(img_gray)` calcula o valor de T pelo **método de Otsu** (Otsu, 1979). Esse método, que será detalhado em capítulos futuros, maximiza a variância entre classes do histograma (frequência de cada tom de cinza), separando automaticamente os pixels de objeto e fundo.

O código abaixo compara a limiarização manual ($T = 128$) com a automática (Otsu), mostrando também o valor de T calculado:

```
import cv2
# Limiarização com T fixo (manual)
T_fixo = 128
img_bin_fixo = mm.threshold(img_gray, T_fixo)

# Limiarização pelo método de Otsu (T automático)
T_otsu, img_bin_otsu = cv2.threshold(img_gray, 0, 255,
                                     cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f'Limiar calculado por Otsu: T = {T_otsu}')
# ou simplesmente:
# img_bin_otsu = mm.threshold(img_gray)

# Exibição lado a lado
mm.show(
    [img_gray, img_bin_fixo, img_bin_otsu],
    titles=["Tons de Cinza", f"Binária (T={T_fixo})", f"Binária (Otsu, T={T_otsu})"],
    cols=3
)
```

```
Limiar calculado por Otsu: T = 95.0
```




Figura 1.9: Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.

A Figure 1.9 mostra que o limiar obtido por Otsu se adapta automaticamente à imagem, resultando em uma binarização mais eficiente do que um valor fixo, especialmente quando as intensidades do objeto e do fundo são bem separadas no histograma. Essa técnica é amplamente utilizada em sistemas de VC para binarização de documentos, detecção de objetos e pré-processamento de imagens.

💡 Simplicidade da biblioteca `morph.py`

Enquanto o OpenCV exige a chamada completa:

```
T_otsu, img_bin = cv2.threshold(img_gray, 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

a biblioteca `mm` abstrai toda essa complexidade: basta chamar `mm.threshold(img_gray)`. O limiar de Otsu é calculado automaticamente e a imagem binária é retornada diretamente. Essa abordagem permite concentrar-se no conceito, não nos detalhes de implementação.

1.12 Acesso a Pixels

Em Python com NumPy, uma imagem é estruturada como um **array multidimensional**. O acesso a um pixel específico utiliza a convenção matricial **linha** (eixo Y) e **coluna** (eixo X): `img[linha, coluna]`.

O código da Figure 1.10 demonstra como extrair esses valores em imagens coloridas (RGB) e em tons de cinza, além de isolar a vizinhança imediata do ponto de interesse.

```
import matplotlib.pyplot as plt

# 1. Definição das coordenadas do pixel alvo
r, c = 600, 800

# 2. Acesso direto aos valores dos pixels
pixel_cinza = img_gray[r, c]
pixel_rgb = img[r, c]

print(f" VALORES NO PIXEL ALVO ({r}, {c}):")
print(f"   • Tons de cinza (Escalar) : {pixel_cinza}")
print(f"   • Colorido (Vetor RGB)      : R={pixel_rgb[0]}, G={pixel_rgb[1]}, B={pixel_rgb[2]}")
print(f"  "-" * 50)

# 3. Exibição didática da vizinhança 3x3 ao redor do pixel (r, c)
# Recorta da linha r-1 até r+1, e da coluna c-1 até c+1
vizinhanca_gray = img_gray[r-1:r+2, c-1:c+2]

print(f" MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:")
```

```

print(f"    (0 pixel alvo central está destacado com entre colchetes)\n")

# Impressão formatada para destacar o pixel central
for i, linha in enumerate(vizinhanca_gray):
    linhas_str = []
    for j, valor in enumerate(linha):
        if i == 1 and j == 1:
            linhas_str.append(f"[{valor:3d}]") # Destaca o centro (100, 100)
        else:
            linhas_str.append(f"{valor:3d} ")
    print("    " + " ".join(linhas_str))

# 4. Demonstração visual usando o mm.show (opcional, mas excelente para o livro)
# Mostra a imagem inteira com um ponto demarcando a região
plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.plot(c, r, 'ro', markersize=8, label=f'Pixel ({r},{c})') # Plota (x, y) -> (c, r)
plt.title(f"Localização do Pixel ({r}, {c}) na Imagem")
plt.legend()
plt.axis('on') # Mantém os eixos para o aluno ver as coordenadas 100, 100
plt.show()

```

VALORES NO PIXEL ALVO (600, 800):

- Tons de cinza (Escalar) : 81
- Colorido (Vetor RGB) : R=92, G=78, B=67

MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:

(0 pixel alvo central está destacado com entre colchetes)

```

83    96   105
85 [ 81]   96
83    77   84

```

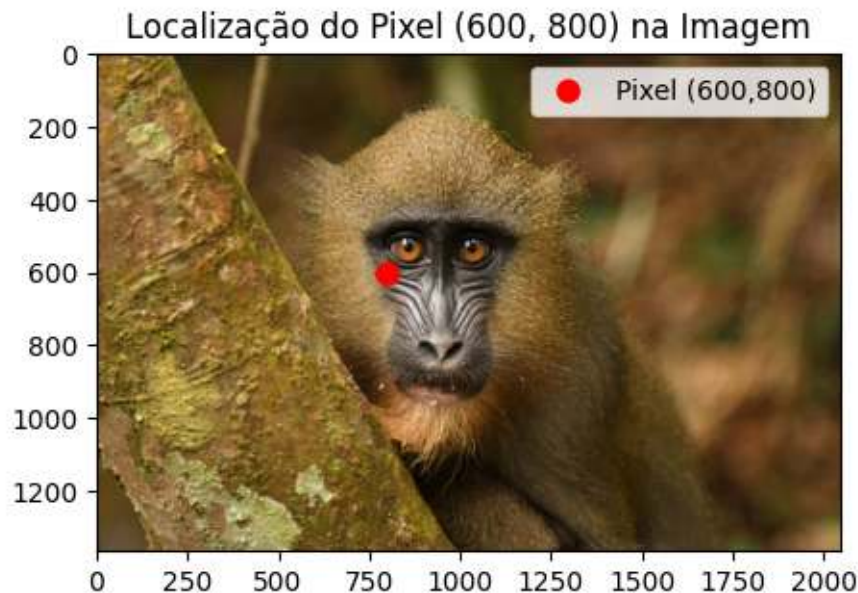


Figura 1.10: Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.

Explicação linha a linha:

1. `img_gray[r, c]` — Retorna um único valor inteiro (escalar) entre 0 e 255, que representa a intensidade de brilho do cinza.

2. `img[r, c]` — Retorna um vetor com três valores `[R, G, B]`, correspondentes às intensidades dos canais Vermelho, Verde e Azul.
3. `img_gray[r-1:r+2, c-1:c+2]` — Realiza um fatiamento (*slicing*) para extrair a submatriz 3×3 ao redor do pixel. Esta operação é a base para a implementação de filtros espaciais e convoluções.
4. O laço `for` subsequente apenas formata essa submatriz no console, exibindo o pixel central entre colchetes `[ ]` para fins didáticos.
5. A função `plt.plot(c, r, 'ro')` plota um ponto vermelho sobre a imagem para correlacionar a coordenada numérica com sua posição visual. Note que o Matplotlib utiliza a ordem padrão de tela `(X, Y)`, invertendo para `(c, r)`.

Como a indexação em Python começa em zero (*zero-based*), o canto superior esquerdo da imagem é a coordenada `(0,0)`. Guarde sempre esta regra: a primeira dimensão da matriz controla a **altura** (linhas/Y) e a segunda controla a **largura** (colunas/X).

1.13 Resumo

Neste capítulo, apresentaram-se os fundamentos de representação de imagens digitais: a definição de pixel, a estruturação de imagens em matrizes e o impacto da amostragem e da quantização na qualidade final:

- **Imagem digital** = função $f(x, y)$ que mapeia coordenadas para intensidades (escalares ou vetoriais).
- **Domínio**: conjunto finito $\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\}$.
- **Tipos principais**: binária ($\mathcal{V} = \{0, 255\}$), tons de cinza ($\mathcal{V} = [0, 255]$) e RGB ($\mathcal{V} = [0, 255]^3$).
- A biblioteca `morph.py` (ou `mm`) oferece funções didáticas para operações básicas de PDI, como `mm.gray()`, `mm.threshold()`, `mm.show_multiple()`.
- **Armadilha NumPy**: nunca use `[[0]*n]*m` para criar matrizes — use sempre `np.zeros()` ou `np.ones()`.
- **Limiarização** converte tons de cinza em binária; o **método de Otsu** determina o valor de corte automaticamente maximizando a variância entre classes.
- Acesso a pixels via `img[linha, coluna]`, com indexação *zero-based*.

O Capítulo 2 abordará **histogramas e equalização de contraste**.

1.14 Uso do NotebookLM como Tutor Complementar

Nesta edição, além dos *notebooks* interativos no Google Colab, o **NotebookLM** está disponível como ferramenta complementar de estudo. A plataforma utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Estude com o Tutor Inteligente

Acesse o ambiente do capítulo pelo *link* abaixo e explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 01](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

Funcionalidades Disponíveis na Plataforma

O **NotebookLM** oferece uma suíte avançada de ferramentas baseadas em IA para transformar o conteúdo estático do livro em uma experiência de aprendizado dinâmica e multimídia. Conforme ilustrado na

Figure 1.11, a plataforma utiliza técnicas de **RAG** (*Retrieval-Augmented Generation*), fundamentadas no trabalho de Lewis et al. (2020), para basear as respostas estritamente nos documentos fornecidos e minimizar a ocorrência de alucinações.

As principais funcionalidades incluem:

- **Resumos Multimodais (Áudio e Vídeo):** Geração de conversas naturais entre especialistas no formato de **Resumo em Áudio** (estilo *podcast*) e **Resumo em Vídeo**, discutindo os temas centrais do capítulo, como as diferenças entre PDI e VC, ou a interpretação de transformações como a limiarização e o método de Otsu.
- **Visualização de Estruturas (Mapa Mental e Infográfico):** Criação automática de diagramas que conectam visualmente os conceitos, por exemplo, o fluxo de processamento desde a captura da imagem digital, passando pela conversão para tons de cinza, limiarização e segmentação binária.
- **Ferramentas de Avaliação (Teste e Cartões Didáticos):** Geração de **Testes** de múltipla escolha e **Cartões Didáticos** (*flashcards*) para fixação de conhecimento, baseados no texto autoral (ex.: perguntas sobre a fórmula de conversão RGB→cinza ou sobre o funcionamento do limiar global e de Otsu).
- **Apoio à Apresentação (Slides e Relatórios):** Auxílio na estruturação de **Apresentações de Slides** e na redação de **Relatórios** técnicos, facilitando a comunicação de resultados de experimentos com imagens.
- **Análise de Dados (Tabela de Dados):** Organização de dados extraídos do texto em tabelas estruturadas, auxiliando na compreensão de exemplos práticos, como a comparação entre diferentes valores de limiar.
- **Chat Contextualizado:** Permite o questionamento direto sobre o código e a teoria, como: “*Como implementar a conversão de RGB para tons de cinza usando os pesos do padrão ITU-R BT.601?*” ou “*O que acontece com a imagem binária se eu escolher um limiar $T=200$ em vez de $T=128$?*”.



Figura 1.11: Estúdio do NotebookLM.

1.15 Lista de Exercícios

1. (15%) Com suas próprias palavras, defina **imagem digital** e **pixel**. Dê um exemplo concreto de como uma imagem colorida (RGB) é representada matricialmente no computador.
2. (15%) Explique as diferenças entre **imagem binária**, **tons de cinza (8 bits)** e **colorida RGB**, indicando a faixa de valores possíveis para cada pixel em cada tipo.
3. (20%) Considerando a fórmula de conversão RGB → tons de cinza do padrão ITU-R BT.601:

$$g = 0.299 R + 0.587 G + 0.114 B$$

Calcule o valor do pixel em tons de cinza para $(R, G, B) = (80, 180, 30)$. Arredonde para o inteiro mais próximo.

4. (20%) O que é **limiarização** (*thresholding*)? Explique a diferença entre escolher um limiar T fixo (ex.: $T = 128$) e utilizar o **método de Otsu** para determinação automática do limiar. Em poucas palavras, como o método de Otsu escolhe o limiar?
5. (15%) No contexto da biblioteca didática `mm` discutida no capítulo, responda:
 - a) (7,5%) Como se acessa o valor do pixel na posição (linha=50, coluna=60) de uma imagem em tons de cinza `img_gray`?
 - b) (7,5%) Qual é a vantagem de usar `mm.threshold(img_gray)` sem passar o limiar? Compare com a chamada equivalente no OpenCV.
6. (15%) O que a propriedade `img.shape` retorna para uma imagem NumPy no formato RGB? Dê um exemplo concreto com uma imagem de 640×480 pixels.

Referências do Capítulo

A fundamentação teórica deste capítulo compreende as seguintes obras de PDI e VC:

- Gonzalez; Woods (2018) para os fundamentos de **Processamento Digital de Imagens** (PDI).
- Singh (2019) para a implementação prática de métodos de **processamento e análise de imagens**.
- Szeliski (2022) para o estudo de VC e algoritmos fundamentais.
- Bradski; Kaehler (2008) para a aplicação da biblioteca **OpenCV** em ambiente Python.
- Lewis et al. (2020) para o conceito de **geração aumentada por recuperação (RAG)**, utilizado no suporte ao processamento de informações deste material.



1.16 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

Os **Exercícios de Programação (EPs)** apresentados a seguir podem ser submetidos também em atividades do Moodle (atividades VPL) que fornecem *feedback* automático.

Este caderno foi desenvolvido para superar limitações de uso do Moodle. Com ele, deve-se:

1. **Desenvolver:** Escrever e editar sua solução diretamente no ambiente Colab.
2. **Validar:** Testar seu código localmente utilizando os **mesmos casos de teste** do Moodle.
3. **Organizar:** Salvar seus códigos das atividades VPL de forma segura.
4. **Avaliar:** Quando estiver conectado ao Moodle, basta copiar sua solução e clicar em **Avaliar** no Moodle (se estiver na rede da UFABC) para registrar sua nota oficial.

§ Instruções Passo a Passo

Em um ambiente de execução (como VSCode, Jupyter ou Colab), siga a ordem abaixo para configurar o ambiente e validar seus exercícios:

Preparação do Ambiente

Execute a célula de código abaixo para baixar `morph.py` e `testsuite.py` do repositório da disciplina — apenas se ainda não existirem no diretório local. Com ambos em `./`, o *notebook* e os subprocessos do `TestSuite` encontram o módulo sem configurações extras de caminho.

Nota: O script `testsuite.py` buscará automaticamente os casos de teste em `all/{cap}/cases` no [GitHub](#).

Escrevendo o Código

Salve sua solução em uma célula de código usando o comando mágico `%%writefile`. O nome do arquivo deve seguir o padrão `EPX_Y.*`, onde `X` é o capítulo, `Y` é o exercício e `*` é a extensão da linguagem.

Exemplo: `%%writefile EP01_01.py`

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.1 | TestSuite: 1.1.0

Executando os Testes

Após salvar o arquivo com sua solução, execute o comando abaixo (em uma nova célula) para avaliar os testes automáticos:

```
TestSuite("EP01_01.extensão").run()
```

Substitua a extensão conforme a linguagem usada:

Linguagem	Extensão
Python	.py
Java	.java
C	.c
C++	.cpp
JavaScript	.js
R	.r

Como funciona: O `TestSuite` baixa os casos de teste do GitHub, executa seu programa com cada entrada e compara a saída com a esperada – calculando automaticamente sua nota.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

⚠ Importante: Regras e Boas Práticas

♦ Sobre a Entrada de Dados

O seu programa deve ler a entrada padrão (teclado).

- **Python:** Utilize `input()`.
- **Outras linguagens:** Utilize o comando de leitura padrão equivalente (`cin`, `Scanner`, etc.).

♦ Configuração de IA no Colab

Para um melhor aprendizado, recomenda-se desativar o preenchimento automático de código por IA, pois não estará disponível durante as avaliações. Por exemplo no navegador Chrome:

- Vá em: **Ferramentas > Configurações > IA generativa**
- Desmarque: **Habilitar geração de código**

♦ Integridade Acadêmica (Plágio)

Este recurso de testes locais aplica-se a EPs **sem variações**. No entanto:

- **Individualidade:** Cada aluno deve desenvolver sua própria solução.
- **Detecção de Similaridade:** O professor utiliza ferramentas que detectam cópias, **mesmo com alteração de nomes de variáveis ou espaços em branco**.

1.16.1 EP01_01 📏 Três métricas de distância em PDI

Nesta atividade, você deve escrever um programa que calcule as três distâncias clássicas em PDI: **Euclidiana (L2)**, **City-Block (L1)** e **Chessboard (L ∞)**.

- Leia **4 números reais** que representam as coordenadas: A_x, A_y, B_x, B_y .
- Calcule as três distâncias utilizando as fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$$

$$d_{\text{City-block}} = |B_x - A_x| + |B_y - A_y|$$

$$d_{\text{Chessboard}} = \max(|B_x - A_x|, |B_y - A_y|)$$

- Imprima os três resultados, cada um em uma linha, formatados com **duas casas decimais**, na ordem: **Euclidiana**, **City-block**, **Chessboard**.

📌 Importante:

- Utilize as funções matemáticas padrão da sua linguagem: `math.sqrt`, `abs` (ou `fabs`) e `max`.
- A saída deve conter **apenas os números** (um por linha), sem textos adicionais.
- Ver um simulador interativo para esta questão na **Figure 1.12** (gráfico com arrasto dos pontos e visualização das três métricas).

1.16.1.1 🖼 Por que isso importa? – Custo computacional

Em uma imagem **1000×1000 pixels** (1 milhão de pixels), calcular a distância de cada pixel a um ponto de referência exige **1 milhão de operações**. A escolha da métrica afeta o desempenho:

Métrica	Operações por pixel	Custo relativo (1M pixels)	Quando usar
Euclidiana (L2)	2 subtrações, 2 multiplicações, 1 soma, 1 sqrt	 Mais custosa – sqrt é cara	Distância “real” no espaço contínuo
City-block (L1)	2 subtrações, 2 abs , 1 soma	 Moderada – sem raiz quadrada	Grids, robótica, imagens binárias
Chessboard (L∞)	2 subtrações, 2 abs , 1 max	 Mais eficiente	Movimentos de peças, morfologia

A função **sqrt** é computacionalmente mais cara que operações como adição, subtração, multiplicação e valor absoluto. Em CPUs modernas, a diferença pode ser pequena (cerca de $1,5\times$ a $3\times$), mas em sistemas embarcados ou em laços de milhões de iterações, qualquer ganho importa. Por isso, **quando o objetivo é apenas comparar distâncias** (ex.: encontrar o ponto mais próximo), use a **distância euclidiana ao quadrado**.

1.16.1.2 Tarefa (especificação para VPL)

Entrada:

Uma única linha com quatro números reais: A_x A_y B_x B_y

Saída:

Três linhas, cada uma com um número real de duas casas decimais (Euclidiana, City-block, Chessboard).

1.16.1.3 Exemplos

Entrada	Saída	Observação
0	5.00	Triângulo 3-4-5
0	7.00	
3	4.00	
4		
0	1.41	Diagonal unitária
0	2.00	
1	1.00	
1		

Exemplo de teste de **sqrt** em Python, com **timeit** isolando cada operação:

```
import math
import timeit

N = 50_000_000

def apenas_soma():
    a, b = 3.0, 4.0
    return a + b

def soma_e_sqrt():
    a, b = 3.0, 4.0
    return math.sqrt(a*a + b*b)
```

```

t_soma = timeit.timeit(apenas_soma, number=N)
t_sqrt = timeit.timeit(soma_e_sqrt, number=N)

print(f"Soma simples      : {t_soma:.3f} s")
print(f"Soma + sqrt       : {t_sqrt:.3f} s")
print(f"Razão (sqrt/soma)  : {t_sqrt/t_soma:.2f}x")

```

```

Soma simples      : 2.967 s
Soma + sqrt       : 5.276 s
Razão (sqrt/soma) : 1.78x

```

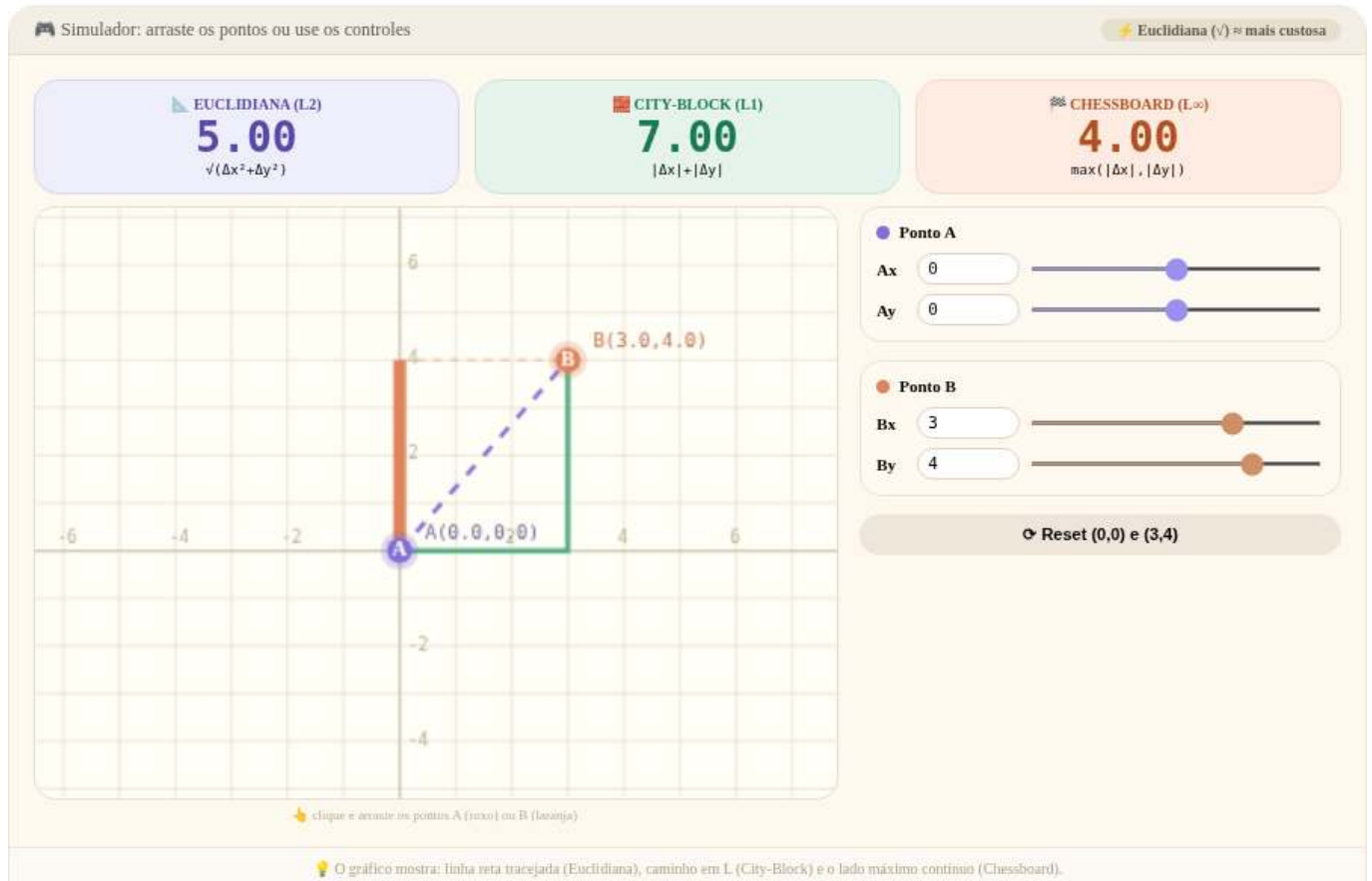


Figura 1.12: Simulador: Distâncias Euclidiana, *City-block* e *Chessboard*

1.16.1.4 🐍 Python

Basta criar uma célula de código normal e inserir o código Python. A entrada pode ser simulada usando `input()`, que funciona normalmente.

Exemplo de célula:

```

%%writefile EP01_01.py
# Código Python
x1,y1,x2,y2 = int(input()), int(input()), int(input()), int(input())
# Cálculo das diferenças
dx = abs(x2 - x1)
dy = abs(y2 - y1)

# 1. Distância Euclidiana (L2)
dist_euclidiana = (dx**2 + dy**2)**0.5

```

```
# 2. Distância City-block / Manhattan (L1)
dist_city_block = dx + dy

# 3. Distância Chessboard / Chebyshev (Linf)
dist_chessboard = max(dx, dy)

# Saída formatada conforme os casos de teste
print(f"{dist_euclidiana:.2f}")
print(f"{dist_city_block:.2f}")
print(f"{dist_chessboard:.2f}")
```

Overwriting EP01_01.py

```
# Espera que você digite 4 números inteiros ao executar esta célula.
# No Jupyter ou Google Colab, a mágica %run -i permite que o script leia do teclado.
# No terminal comum, você usaria: python3 EP01_01.py (sem o '!' e sem '%run').

# %run -i EP01_01.py
```

```
# Envia 4 inteiros como entrada padrão (stdin) para o script EP01_01.py usando um pipe
!echo -e "0\n0\n4\n4" | python3 EP01_01.py
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.py").run()
```

1.16.1.5 Java

Para executar Java no Colab, você precisa usar uma célula com o prefixo `%%writefile` para salvar o código em arquivo, compilar e executar.

```
%%writefile EP01_01.java
import java.util.Scanner;

class EP01_01 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        double x1 = s.nextDouble(), y1 = s.nextDouble();
        double x2 = s.nextDouble(), y2 = s.nextDouble();

        double dx = Math.abs(x2 - x1);
        double dy = Math.abs(y2 - y1);

        System.out.printf("%.2f\n", Math.sqrt(dx*dx + dy*dy));
        System.out.printf("%.2f\n", dx + dy);
        System.out.printf("%.2f\n", Math.max(dx, dy));
    }
}
```

Overwriting EP01_01.java

```
!javac EP01_01.java
!echo -e "0\n0\n4\n4" | java EP01_01
```



```
5,66
8,00
4,00
```

```
TestSuite("EP01_01.java").run()
```

1.16.1.6 C

De forma similar, use `%%writefile` para salvar o código, depois compile e execute. Para C, utilizamos o compilador **GCC**.

```
# Instalar o compilador GCC e ferramentas de build
# O build-essential inclui gcc, g++, make, etc.
import platform, shutil, subprocess

def instalar_gcc():
    if shutil.which("gcc"):
        print(" GCC já disponível."); return
    if platform.system() != "Linux":
        print(" Mac: xcode-select --install | Windows: WSL ou MinGW"); return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" build-essential instalado!")

instalar_gcc()
```

GCC já disponível.

```
%%writefile EP01_01.c
#include <stdio.h>
#include <math.h>

int main() {
    double x1, y1, x2, y2;
    if (scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2) != 4) return 0;

    double dx = fabs(x2 - x1);
    double dy = fabs(y2 - y1);

    // Euclidiana, City-block e Chessboard
    printf("%.2f\n", sqrt(dx * dx + dy * dy));
    printf("%.2f\n", dx + dy);
    printf("%.2f\n", fmax(dx, dy));

    return 0;
}
```

Overwriting EP01_01.c

```
# Compila o arquivo .c gerando o executável EP01_01
# -lm é usado para linkar a biblioteca matemática (math.h) se necessário

!gcc EP01_01.c -o EP01_01 -lm
!echo -e "0\n0\n4\n4" | ./EP01_01
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.c").run()
```

1.16.1.7 C++

De forma similar ao Java, use `%%writefile` para salvar o código, depois compile e execute. Lembre-se de que, no Colab, também é necessário instalar o seguinte:

```
# Instalar o compilador G++ para C++
# O build-essential inclui o g++, make, etc.
import platform, shutil, subprocess

def instalar_gpp():
    if shutil.which("g++"):
        print(" G++ já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: xcode-select --install")
        else:
            print(" Windows: use WSL ou MinGW (https://www.mingw-w64.org)")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" Compilador C++ pronto.")

instalar_gpp()
```

G++ já disponível.

```
%%writefile EP01_01.cpp
#include <iostream>
#include <iomanip>
#include <cmath>
#include <algorithm>

int main() {
    double x1, y1, x2, y2;
    if (!(std::cin >> x1 >> y1 >> x2 >> y2)) return 0;

    double dx = std::abs(x2 - x1);
    double dy = std::abs(y2 - y1);

    std::cout << std::fixed << std::setprecision(2);

    // Euclidiana, City-block e Chessboard
    std::cout << std::sqrt(dx*dx + dy*dy) << std::endl;
    std::cout << (dx + dy) << std::endl;
    std::cout << std::max(dx, dy) << std::endl;

    return 0;
}
```

Overwriting EP01_01.cpp

```
!g++ EP01_01.cpp -o EP01_01
!echo -e "0\n0\n4\n4" | ./EP01_01
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.cpp").run()
```

1.16.1.8 JavaScript (Node.js)

Para JavaScript, use `%%writefile` para criar o arquivo e execute com Node:

```
%%writefile EP01_01.js
function escreva(s) { try { document.write(s + "<br>"); } catch(e) { console.log(s); } }

process.stdin.once('data', data => {
  const valores = data.toString().trim().split(/\s+/);
  const x1 = parseFloat(valores[0]);
  const y1 = parseFloat(valores[1]);
  const x2 = parseFloat(valores[2]);
  const y2 = parseFloat(valores[3]);

  const dx = Math.abs(x2 - x1);
  const dy = Math.abs(y2 - y1);

  // Euclidiana, City-block e Chessboard
  escreva(Math.sqrt(dx * dx + dy * dy).toFixed(2));
  escreva((dx + dy).toFixed(2));
  escreva(Math.max(dx, dy).toFixed(2));
});
```

```
Overwriting EP01_01.js
```

```
TestSuite("EP01_01.js").run()
```

1.16.1.9 R

No Colab, pode-se executar código R diretamente utilizando a mágica `%%R`.

O programa deve ler **quatro números** (x_1 , y_1 , x_2 , y_2) e exibir a distância euclidiana com **duas casas decimais**.

Exemplo de célula:

```
%%R
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
x1 <- dados[1]; y1 <- dados[2]; x2 <- dados[3]; y2 <- dados[4]
dist <- sqrt((x2 - x1)^2 + (y2 - y1)^2)
cat(sprintf("%.2f", dist))
```

```
# Instalar o R e o Rscript
# O r-base instala o ambiente R completo, incluindo o Rscript
import platform, shutil, subprocess
```

```
def instalar_r():
    if shutil.which("R"):
        print(" R já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: https://cran.r-project.org/bin/macosx/")
        else:
            print(" Windows: https://cran.r-project.org/bin/windows/base/")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "r-base"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "r-base"]
    subprocess.run(cmd, check=True)
    print(" Ambiente R pronto.")

instalar_r()
```

R já disponível.

Para testar da mesma forma que nos exemplos anteriores, deve-se usar a entrada padrão (stdin) no terminal ou adaptar o código conforme abaixo:

```
%%writefile EP01_01.r
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
dx <- abs(dados[3] - dados[1])
dy <- abs(dados[4] - dados[2])

# Saídas: Euclidiana, City-block e Chessboard
cat(sprintf("%.2f\n%.2f\n%.2f\n",
    sqrt(dx^2 + dy^2),
    dx + dy,
    max(dx, dy)))
```

Overwriting EP01_01.r

```
!echo -e "0\n0\n4\n4" | Rscript EP01_01.r
```

5.66
8.00
4.00

```
TestSuite("EP01_01.r").run()
```

1.16.2 EP01_02 Desempenho Preditivo — Métricas de ML em VC

Nesta atividade, você entrará no mundo do **Aprendizado de Máquina** (*Machine Learning*). Seu objetivo é avaliar o desempenho de um classificador binário calculando métricas a partir de uma **Matriz de Confusão**.

1.16.2.1 Por que a métrica certa importa?

Imagine um detector de **moedas de R\$ 1,00**. O impacto do erro define a métrica prioritária:

Métrica	Exemplo Prático	Importância em PDI/VC
Acurácia	Contagem de Grãos	Útil quando as classes são equilibradas (ex: metade dos grãos com defeito, metade saudáveis).

Métrica	Exemplo Prático	Importância em PDI/VC
Precisão	Segurança/Biometria	Crucial para evitar Falsos Positivos (ex: não permitir que um impostor acesse um sistema por erro de reconhecimento).
Sensibilidade	Saúde (Tumores)	Crucial para evitar Falsos Negativos (ex: não deixar um tumor passar despercebido em um exame de Raio-X).
F1-score	Cédulas de Dinheiro	Ideal para um equilíbrio entre não rejeitar notas verdadeiras e não aceitar notas falsas.

1.16.2.2 A Matriz de Confusão

	Predita Positiva	Predita Negativa
Real Positiva	VP (Verdadeiro Positivo)	FN (Falso Negativo)
Real Negativa	FP (Falso Positivo)	VN (Verdadeiro Negativo)

Tarefa:

1. Leia **4 valores inteiros** na ordem: VP, FN, FP, VN.
2. Calcule as métricas utilizando as fórmulas:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Sensibilidade (Recall)} = \frac{VP}{VP + FN}$$

$$\text{F1-score} = \frac{2 \times \text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}}$$

3. Imprima os resultados formatados com **duas casas decimais**, um por linha.

Importante:

- Use divisão em ponto flutuante para evitar resultados truncados.
- A ordem de saída deve ser: **Acurácia**, **Precisão**, **Sensibilidade** e **F1-score**.
- Veja a simulação interativa deste EP na Figure 1.13.

1.16.2.3 Exemplo de Execução

Entrada	Saída	Observação
40	0.75	Acurácia
10	0.73	Precisão
15	0.80	Sensibilidade

Entrada	Saída	Observação
35	0.76	F1-score

(Nota: $VP=40$, $FN=10$, $FP=15$, $VN=35$. Total de casos = 100)

Simulador: desempenho de classificadores

Ajuste os valores da matriz de confusão e veja as métricas calculadas em tempo real.

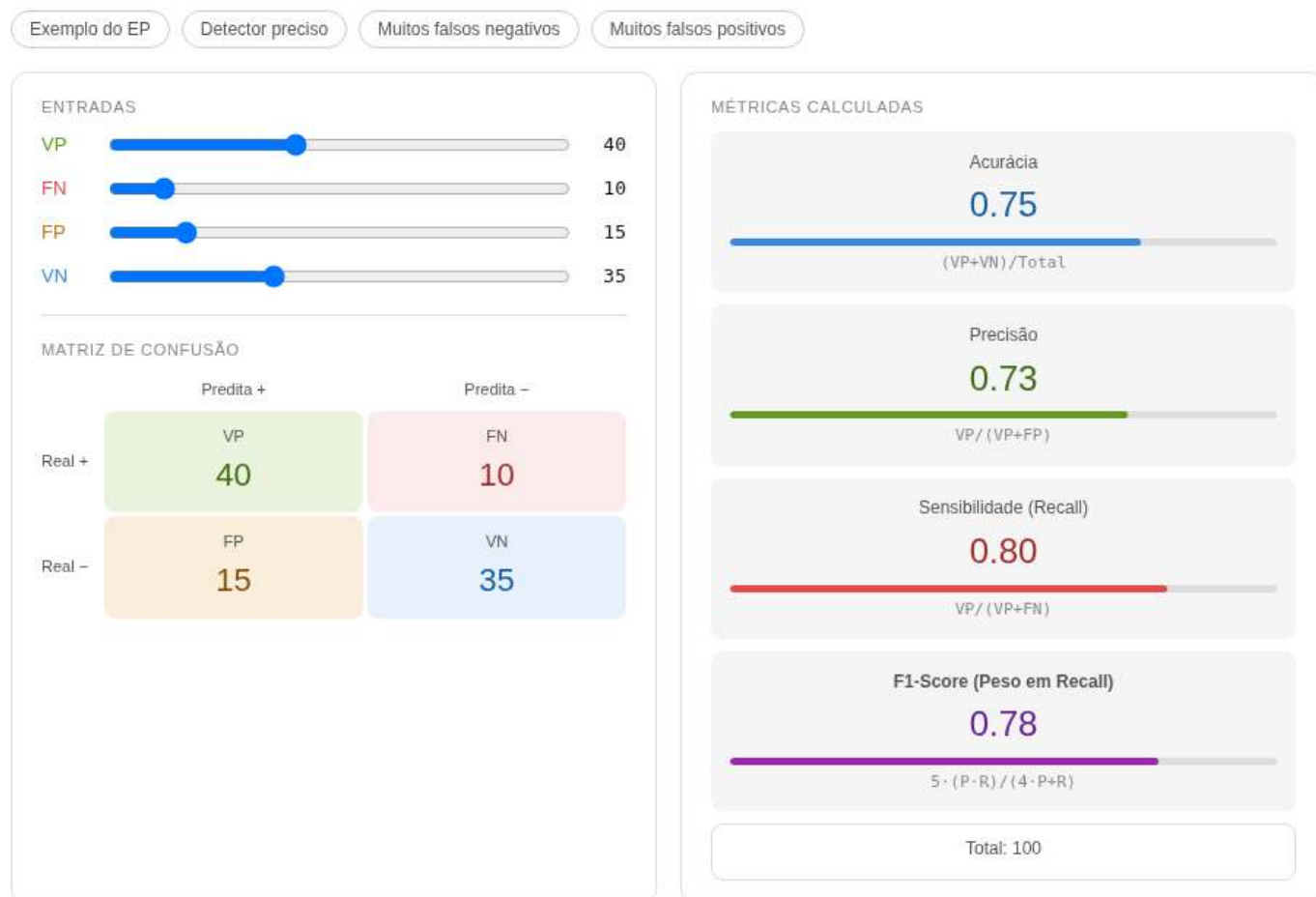


Figura 1.13: Simulador: Desempenho Preditivo — Métricas de ML

```
%%writefile EP01_02.py
# sua solução
```

Overwriting EP01_02.py

```
TestSuite("EP01_02.py").run()
```

1.16.3 EP01_03 ↗ Mean Average Precision (mAP) — Curva Precisão-Sensibilidade

Nesta atividade, você avaliará um classificador binário (ex.: detecção de desmatamento em imagens de satélite, ver dgi.inpe.br) através da **curva Precisão-Sensibilidade** e da métrica **mAP** (*Mean Average*

Precision). O mAP é padrão em competições como [COCO \(Common Objects in Context\)](#) e [PASCAL VOC \(Visual Object Classes\)](#) e dos modelos [YOLO \(You Only Look Once\)](#).

1.16.3.1 Por que o mAP é a métrica-padrão?

Na EP01_02 você viu que a escolha do limiar altera significativamente a Precisão e a Sensibilidade. O **mAP (Mean Average Precision)** resolve isso: avalia o modelo em **vários limiares** (cada limiar deve gerar uma matriz de confusão diferente) e resume o desempenho pela **área sob a curva Precisão-Sensibilidade (P-S)**.

Enquanto o *F1-Score* olha para um único ponto de equilíbrio, o mAP considera a curva inteira. Quanto mais próximo de **1,0**, melhor o detector em todos os limiares e classes (ex.: moedas de 25, 50 e 1 real).

Métrica	O que resume	Limitação
F1-Score	Equilíbrio $P \times S$ num único limiar	Depende do limiar escolhido
AP	Área sob a curva P-S de uma classe	Válida apenas para uma classe
mAP	Média das APs sobre todas as classes	Mais complexo de implementar

Referências: [Roboflow — mAP](#) · [Vídeo explicativo](#)

1.16.3.2 Como o mAP é calculado — passo a passo

1. **Limiares fixos** (use sempre esta lista):

```
limiares = [0.00, 0.09, 0.21, 0.31, 0.39, 0.52, 0.60, 0.71, 0.81, 0.89, 1.00]
```

2. Para cada limiar (t), classifique as amostras: **predito = 1 se confiança t , senão 0**. Calcule VP, FP, FN, VN e obtenha $\text{Precisão}((t))$ e $\text{Sensibilidade}((t))$.
3. Monte a curva P-S: pares ($\text{Sensibilidade}((t))$, $\text{Precisão}((t))$), ordenados por Sensibilidade crescente.
4. **Monotonize a Precisão**:

$$P_{\text{mono}}[i] = \max_{j \geq i} P[j]$$

5. Calcule a **AP** (área sob a curva monotônica) usando a **regra do trapézio** (aproximação mais precisa que a simples soma de Riemann):

$$AP = \sum_{i=1}^{m-1} \frac{P_{\text{mono}}[i-1] + P_{\text{mono}}[i]}{2} \cdot (S[i] - S[i-1])$$

6. **mAP** = média das APs de todas as classes. Neste EP há apenas **1 classe**, portanto $\text{mAP} = \text{AP}$.

Note

Diferença resumida:

A *soma de Riemann* aproxima a área por retângulos, podendo subestimar ou superestimar. A **regra do trapézio** usa trapézios, reduzindo o erro ao considerar a média entre os valores nos extremos do intervalo, sendo geralmente mais precisa para funções suaves por partes, como a curva Precisão-Sensibilidade.

1.16.3.3 Tarefa

Leia um inteiro n (quantidade de amostras). Em seguida leia n linhas, cada uma com: **verdade** (0 ou 1) e **confiança** (float 0.0–1.0).

Calcule e imprima, para o **limiar 0.85** (índice 9 da lista):

- Matriz de Confusão (VP, FN, FP, VN)
- Acurácia, Precisão, Sensibilidade e F1-Score

Em seguida, para **todos os limiares**, imprima:

- Precisões cruas, Precisões monotônicas e Sensibilidades, separadas por ,
- mAP final

1.16.3.4 📌 Importante

- Limiar fixo para as métricas individuais: **0.85**
- Divisão segura: se denominador for zero, use 0
- Formatação: duas casas decimais
- Monotonize de trás para frente
- A Figure 1.14 apresenta uma simulação desta questão

1.16.3.5 📌 Exemplo de Execução

Entrada	Saída Esperada
7	# MÉTRICAS PARA O LIMAR 0.85 #
0 0.94	Matriz de Confusão:
1 0.80	VP = 0, FN = 5
1 0.69	FP = 1, VN = 1
0 0.67	
1 0.30	Métricas de Avaliação:
1 0.15	Acurácia: 0.14
1 0.15	Precisão: 0.00
	Sensibilidade: 0.00
	F1-Score: 0.00
	# MÉTRICAS PARA TODOS OS LIMIARES #
	Precisões: 0.00, 0.00, 0.00, 0.50, 0.50, 0.50, 0.50, 0.50, 0.60, 0.71, 0.71
	Precisões mon.: 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71
	Sensibilidades: 0.00, 0.00, 0.00, 0.20, 0.40, 0.40, 0.40, 0.40, 0.60, 1.00, 1.00
	mAP: 0.71

1.16.3.6 🐡 Dica para calcular o AP (com regra do trapézio)

```
def calcular_AP(verdades, confiancas, limiares):
    m = len(limiares)
    precisoes = [0.0] * m
    sensibilidades = [0.0] * m
    for i in range(m):
        p, s = calcular_metricas(verdades, confiancas, limiares[i])
        precisoes[m-1-i] = p
        sensibilidades[m-1-i] = s
    prec_mono = precisoes.copy()
    for i in range(m-2, -1, -1):
        if prec_mono[i] < prec_mono[i+1]:
            prec_mono[i] = prec_mono[i+1]
    AP = 0.0
    for i in range(1, m):
```

```
# Regra do trapézio: média das alturas vezes a base
area_trapezio = (prec_mono[i-1] + prec_mono[i]) / 2.0
AP += area_trapezio * (sensibilidades[i] - sensibilidades[i-1])
return precisoes, prec_mono, sensibilidades, AP
```

```
%%writefile EP01_03.py
# sua solução
```

Overwriting EP01_03.py

```
TestSuite("EP01_03.py").run()
```

1.16.4 EP01_04 Leitura e Informações de uma Imagem em Matriz

Nesta atividade, você deve escrever um programa que processe uma imagem digital representada como uma matriz de pixels em tons de cinza.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os **L * C** valores inteiros que compõem a matriz da imagem (cada valor entre 0 e 255).
- Calcule e imprima as seguintes informações:
 1. O número de linhas.
 2. O número de colunas.
 3. O valor do maior pixel (**Máximo**).
 4. O valor do menor pixel (**Mínimo**).
 5. A **Média** aritmética de todos os pixels.

Importante:

- A saída deve seguir exatamente o formato rotulado (ex: **Linhas: X**).
- O valor da média deve ser formatado com **duas casas decimais**.
- Ver um simulador interativo para esta questão na **Figure 1.15** (grade interativa para visualização de intensidades e cálculos em tempo real).

1.16.4.1 Por que isso importa? – A Imagem como Dados

Toda imagem digital é, no fundo, uma estrutura de dados. Em tons de cinza de 8 bits, cada pixel é um valor escalar. Extrair estatísticas básicas é o primeiro passo para:

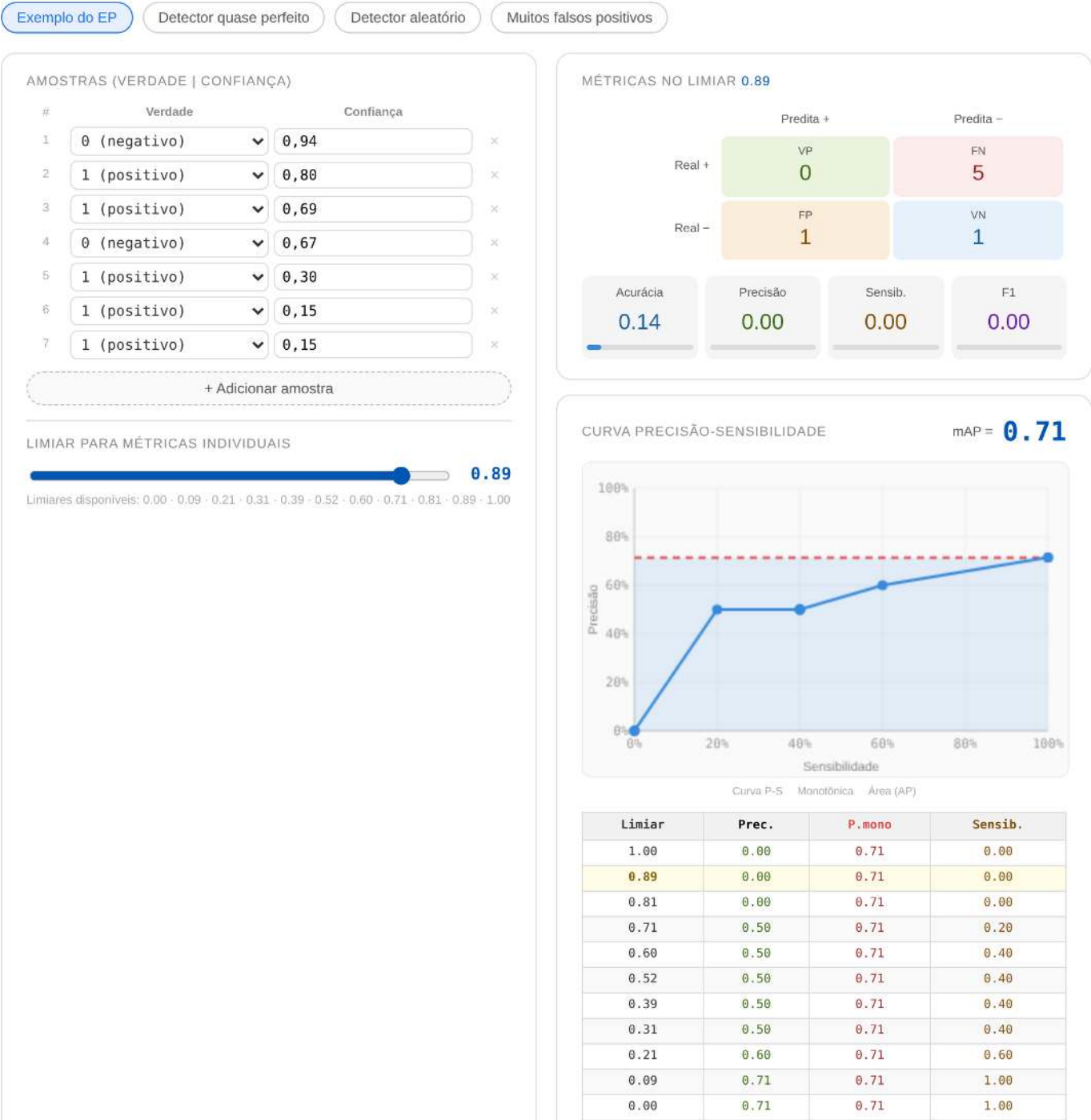
Operação	Utilidade Prática
Máximo/Mínimo	Identificar se a imagem está “lavada” (baixo contraste) ou saturada.
Média	Calcular o brilho global da cena para ajustes de exposição.
Normalização	Redimensionar os valores para intervalos como [0, 1] em redes neurais.

1.16.4.2 Tarefa (especificação para VPL)

Entrada:

Simulador: Curva P-S e mAP

Edite as amostras (verdade e confiança) e veja a curva Precisão-Sensibilidade e o mAP calculados em tempo real.



CURVA PRECISÃO-SENSIBILIDADE

mAP = 0.71

Limiar	Prec.	P.mono	Sensib.
1.00	0.00	0.71	0.00
0.89	0.00	0.71	0.00
0.81	0.00	0.71	0.00
0.71	0.50	0.71	0.20
0.60	0.50	0.71	0.40
0.52	0.50	0.71	0.40
0.39	0.50	0.71	0.40
0.31	0.50	0.71	0.40
0.21	0.60	0.71	0.60
0.09	0.71	0.71	1.00
0.00	0.71	0.71	1.00

Figura 1.14: Simulador: Mean Average Precision (mAP) - Curva P.S

A primeira linha contém o inteiro L (linhas).

A segunda linha contém o inteiro C (colunas).

As linhas seguintes contêm os elementos da matriz.

Saída:

Cinco linhas formatadas conforme o exemplo:

Linhas: L

Colunas: C

Max: V

Min: V

Media: V.VV

1.16.4.3 Exemplos

Entrada	Saída	Observação
2	Linhas: 2	Imagem pequena de alto contraste
3	Colunas: 3	
0 128 255	Max: 255	
50 100 200	Min: 0	
	Media: 122.17	



Figura 1.15: Simulador: Estatísticas de Pixels

```
%%writefile EP01_04.py
# sua solução
```

Overwriting EP01_04.py


```
TestSuite("EP01_04.py").run()
```

1.16.5 EP01_05 Negativo de uma Imagem em Tons de Cinza

Nesta atividade, deve-se escrever um programa que calcule o negativo de uma imagem digital.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os valores inteiros que compõem a matriz da imagem.
- Para cada pixel, aplique a transformação de inversão:

$$pixel_{negativo} = 255 - pixel_{original}$$

- Imprima a matriz resultante, mantendo o formato original (L linhas e C colunas).

Importante:

- Os valores de cada linha na saída devem ser separados por um **espaço em branco**.
- A saída deve conter **apenas os números** da matriz resultante.
- Ver um simulador interativo para esta questão na **Figure 1.16** (comparação em tempo real entre a matriz original e seu negativo).

1.16.5.1 Por que isso importa? – Inversão de Intensidade

O **negativo** é uma transformação linear básica que inverte a escala de brilho. É uma ferramenta essencial para o olho humano identificar detalhes claros que estão “escondidos” em fundos mais escuros, sendo amplamente utilizada em:

Aplicação	Utilidade
Imagens Médicas	Melhora a visualização de anomalias em tecidos densos (ex: Raios-X).
Astronomia	Destacar galáxias e nebulosas tênues contra o vazio do espaço.
Artes Digitais	Efeitos estéticos e preparação de máscaras de seleção.

1.16.5.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro **L**.

A segunda linha contém o inteiro **C**.

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz invertida com **L** linhas e **C** colunas.

1.16.5.3 Exemplos

Entrada	Saída	Observação
2	255 127 0	Onde era 0 (preto) vira 255 (branco)
3	205 155 55	
0 128 255		
50 100 255		



Figura 1.16: Simulador: Transformação de Negativo

```
%%writefile EP01_05.py
# sua solução
```

Overwriting EP01_05.py

```
TestSuite("EP01_05.py").run()
```

1.16.6 EP01_06 🎨 Conversão RGB → Tons de Cinza (ITU-R BT.601)

Nesta atividade, você deve escrever um programa que converta pixels coloridos (RGB) para tons de cinza utilizando a ponderação fisiológica do padrão ITU-R BT.601.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia **L × C** triplas de inteiros, onde cada tripla representa os canais **R** (Red), **G** (Green) e **B** (Blue) de um pixel.
- Para cada pixel, calcule o valor de cinza (*g*) usando a fórmula:

$$g = \text{round}(0.299 \times R + 0.587 \times G + 0.114 \times B)$$

- Imprima a matriz resultante (L linhas e C colunas) contendo os valores inteiros convertidos.

📌 **Importante:**

- Utilize a função `round()` da sua linguagem para garantir o arredondamento correto para o inteiro mais próximo.
- A saída deve conter apenas os valores de cinza, mantendo a estrutura de matriz (separados por espaço na linha).
- Ver um simulador interativo para esta questão na **Figure 1.17** (ajuste os sliders para ver como cada cor contribui para o brilho final).

1.16.6.1 🌸 Por que não usar apenas a média?

O olho humano não percebe todas as cores com a mesma intensidade. Somos muito mais sensíveis ao **Verde** do que ao **Azul** devido à nossa evolução biológica. O padrão ITU-R BT.601 utiliza pesos específicos para criar uma imagem em cinza que pareça naturalmente correta para nossa visão:

Canal	Peso	Percepção Humana
 Verde	58.7%	Máxima sensibilidade (distinção de folhagens).
 Vermelho	29.9%	Sensibilidade média.
 Azul	11.4%	Baixa sensibilidade (tons mais escuros).

1.16.6.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L.

A segunda linha contém o inteiro C.

As linhas seguintes contêm triplas de inteiros R G B para cada pixel.

Saída:

A matriz de cinzas com L linhas e C colunas.

1.16.6.3 📌 Exemplos

Entrada	Saída	Observação
1 3 255 0 0 0 255 0 0 0 255	76 150 29	Note como o Verde (150) é mais brilhante que o Azul (29)

```
%%writefile EP01_06.py
# sua solução
```

```
Overwriting EP01_06.py
```

```
TestSuite("EP01_06.py").run()
```

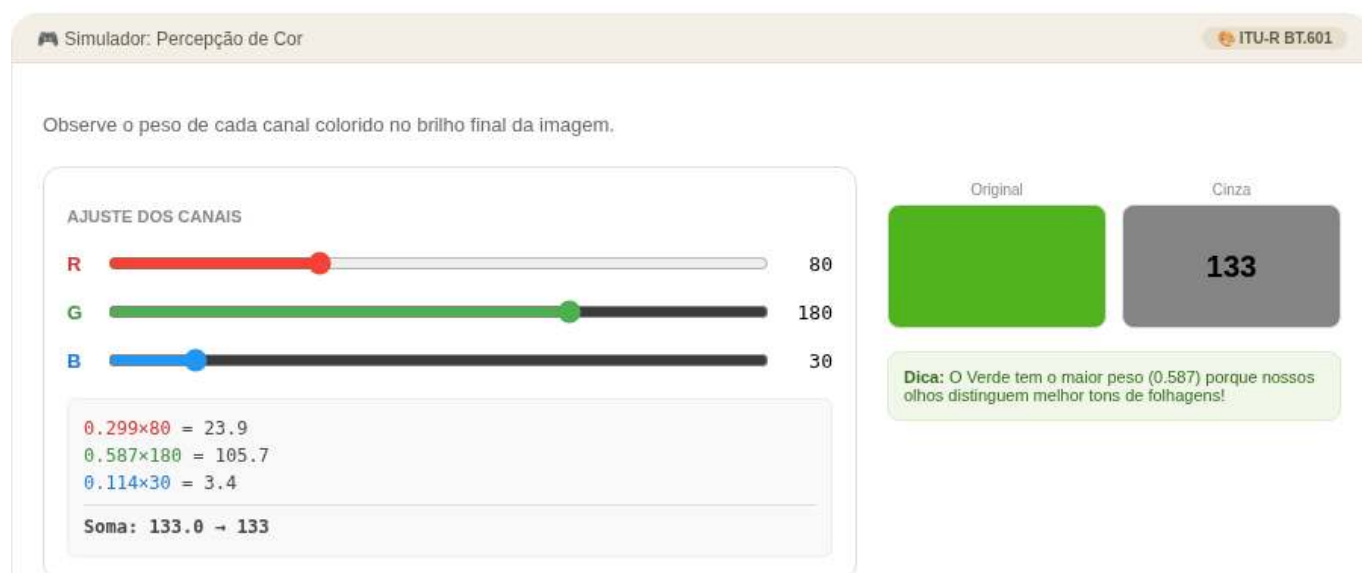


Figura 1.17: Simulador: Percepção de Cor e Pesos ITU-R

1.16.7 EP01_07 ● Limiarização Manual: Imagem Binária

Nesta atividade, você deve escrever um programa que realize a segmentação de uma imagem através da limiarização (*thresholding*).

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia um inteiro **T**, que será o valor do limiar (corte).
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a seguinte regra de binarização:

$$\text{resultado} = \begin{cases} 255 & \text{se } p > T \\ 0 & \text{se } p \leq T \end{cases}$$

- Imprima a matriz resultante contendo apenas os valores 0 ou 255.

📌 Importante:

- Preste atenção ao operador: o pixel só vira branco (255) se for **estritamente maior** que T .
- A saída deve manter a estrutura de matriz (L linhas e C colunas).
- Ver um simulador interativo para esta questão na **Figure 1.18** (ajuste o slider de T para observar como os objetos são isolados do fundo).

1.16.7.1 🧠 O que é Segmentação?

A limiarização é o método mais simples de separar objetos de interesse do fundo da imagem. Ao transformar tons de cinza em preto e branco puro, criamos um mapa binário que facilita a contagem de objetos ou a identificação de formas:

Valor do Pixel (p)	Condição	Resultado Final
Escuro ($p \leq T$)	Fundo/Ruído	0 (Preto)
Claro ($p > T$)	Objeto/Destaque	255 (Branco)

1.16.7.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L .

A segunda linha contém o inteiro C .

A terceira linha contém o inteiro T (limiar).

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz binarizada (0 ou 255) com L linhas e C colunas.

1.16.7.3 📌 Exemplos

Entrada	Saída	Observação
2	0 0 0 255	Note que o valor 128 virou 0 (pois $128 \leq 128$)
4	0 255 255 0	
128		
0 100 128 200		
50 129 255 64		

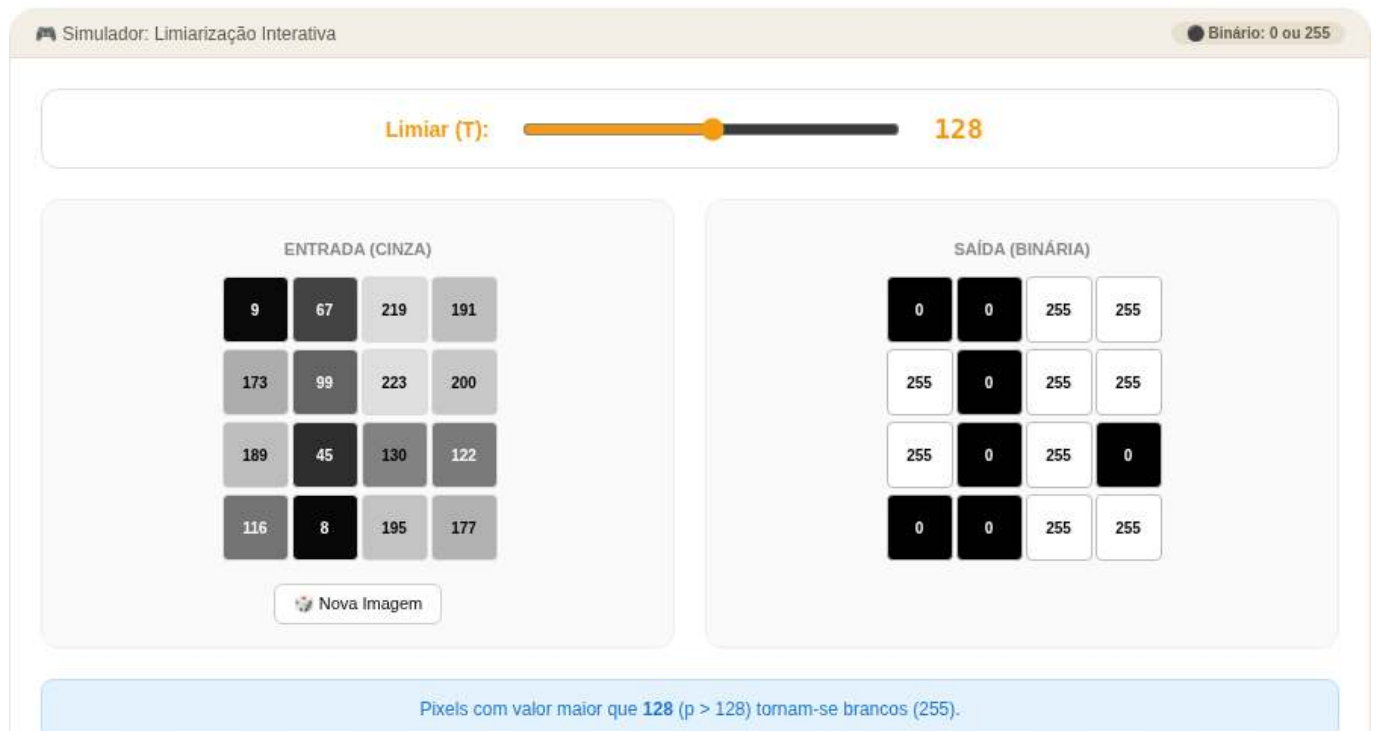


Figura 1.18: Simulador: Limiarização Interativa

```
%%writefile EP01_07.py
# sua solução
```

Overwriting EP01_07.py

```
TestSuite("EP01_07.py").run()
```

1.16.8 EP01_08 🧠 Remapeamento por Faixa de Intensidade

Nesta atividade, você deve escrever um programa que aplique transformações lineares distintas a diferentes regiões de intensidade da imagem.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia os inteiros **T** (limiar), δ_1 (delta 1) e δ_2 (delta 2).
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a regra de remapeamento condicional:

$$\text{resultado} = \begin{cases} p + \delta_1 & \text{se } p < T \\ p + \delta_2 & \text{se } p \geq T \end{cases}$$

- Imprima a matriz resultante com os novos valores de intensidade.

📌 Importante:

- δ_1 é o offset aplicado aos pixels escuros (abaixo do limiar).
- δ_2 é o offset aplicado aos pixels claros (maior ou igual ao limiar).
- Os casos de teste garantem que o resultado estará sempre no intervalo válido de **0 a 255**, portanto, não é necessário tratar saturação ou arredondamentos.
- A saída deve manter a estrutura de matriz (**L** linhas e **C** colunas).

1.16.8.1 🧠 Transformação Condicional de Pixels

Em Processamento Digital de Imagens (PDI), frequentemente precisamos tratar regiões de forma independente. Esta técnica permite, por exemplo, clarear apenas as sombras de uma fotografia (aumentando os pixels escuros) sem estourar o brilho das áreas já claras, ou vice-versa.

Faixa de Intensidade	Condição	Operação
Pixels Escuros	$p < T$	$p + \delta_1$
Pixels Claros	$p \geq T$	$p + \delta_2$

1.16.8.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém os inteiros **L** e **C**.

A segunda linha contém os inteiros **T**, δ_1 e δ_2 .

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz transformada com **L** linhas e **C** colunas, com valores separados por espaço.

1.16.8.3 📌 Exemplos

Entrada	Saída	Observação
2 4 128 60 -40 0 100 150 255 80 128 200 30	60 160 110 215 140 88 160 90	Pixels < 128 somam 60. Pixels 128 subtraem 40.

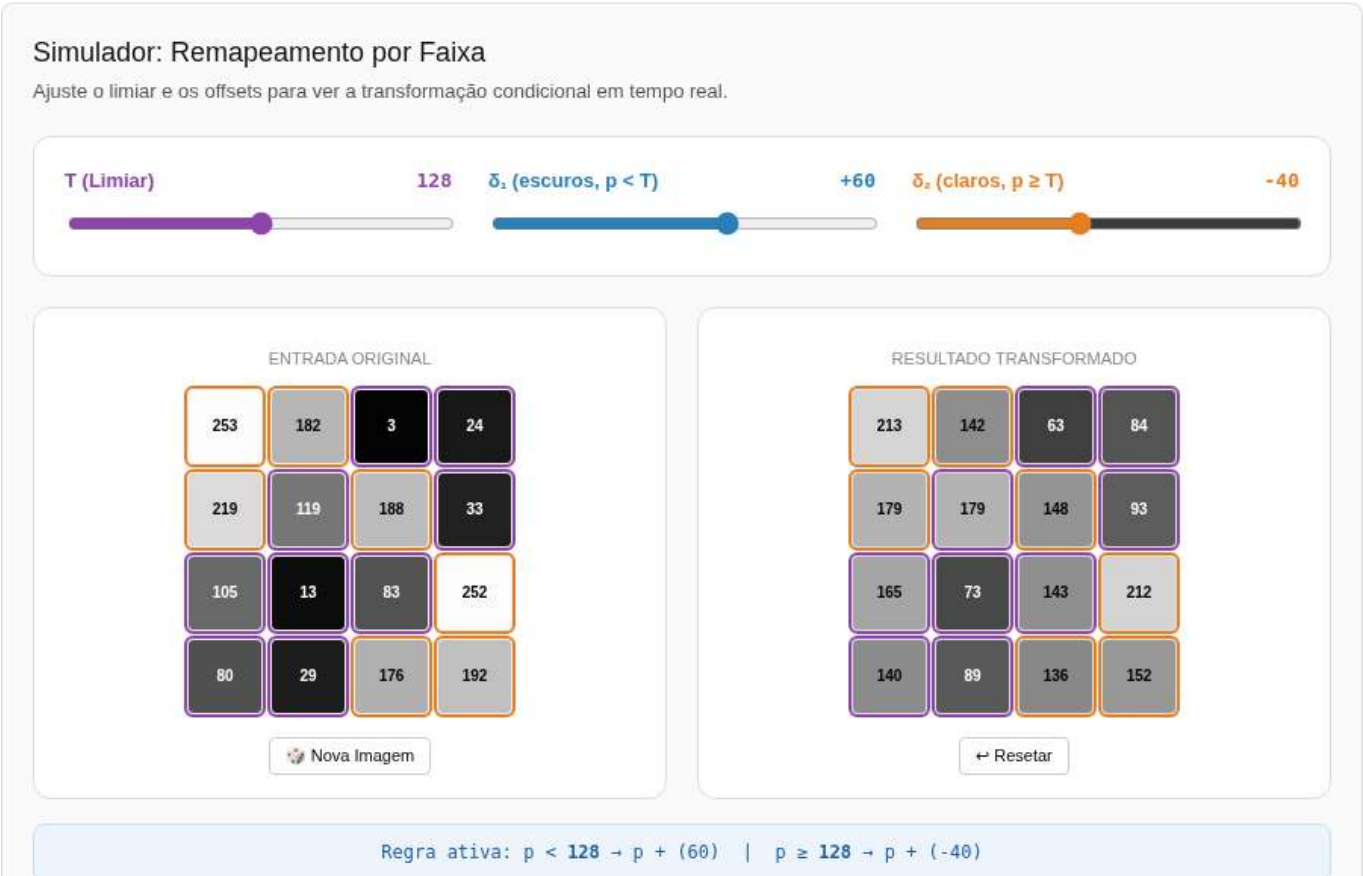


Figura 1.19: Simulador: Ajuste de Brilho e Contraste

```
%%writefile EP01_08.py
# sua solução

Overwriting EP01_08.py

TestSuite("EP01_08.py").run()
```

1.16.9 EP01_09 🏁 Padrão de Tabuleiro: Matriz de Xadrez

Nesta atividade, você deve escrever um programa que gere uma imagem sintética no padrão de um tabuleiro de xadrez.

- Leia dois inteiros **L** (linhas) e **C** (colunas).
- Gere uma matriz onde os valores alternam entre **0** (preto) e **1** (branco).

- A lógica de preenchimento deve seguir a regra da paridade:
- O elemento na posição $(0, 0)$ é sempre **0**.
- Um pixel na posição (i, j) será **1** se a soma dos índices $(i + j)$ for **ímpar**.
- Um pixel na posição (i, j) será **0** se a soma dos índices $(i + j)$ for **par**.

📌 Importante:

- As cores devem alternar corretamente tanto horizontalmente quanto verticalmente.
- A saída deve ser a matriz impressa linha por linha, com os elementos separados por um espaço.
- Ver um simulador interativo para esta questão na **Figure 1.20** (ajuste as dimensões para visualizar a construção da malha e a saída textual correspondente).

1.16.9.1 🧠 Padrões Sintéticos

Criar padrões geométricos é um exercício fundamental para dominar a **lógica de índices** em matrizes. Em Processamento Digital de Imagens, o padrão de xadrez não é apenas estético; ele é amplamente utilizado para:

Aplicação	Utilidade
Calibração de Câmera	Estimar parâmetros intrínsecos e extrínsecos da lente.
Correção de Distorção	Identificar e corrigir o efeito de “barril” ou “almofada” em lentes grande-angulares.
Mapeamento 3D	Projetar padrões conhecidos para reconstruir superfícies em sistemas de luz estruturada.

1.16.9.2 📋 Tarefa (especificação para VPL)

Entrada:

Uma linha contendo o inteiro L (linhas).

Uma linha contendo o inteiro C (colunas).

Saída:

A matriz de xadrez com L linhas e C colunas, impressa com espaços entre os elementos.

1.16.9.3 📌 Exemplos

Entrada	Saída	Observação
3	0 1 0 1	Note que cada linha começa com o inverso da anterior
4	1 0 1 0	
	0 1 0 1	

```
%%writefile EP01_09.py
# sua solução
```

```
Overwriting EP01_09.py
```

```
TestSuite("EP01_09.py").run()
```

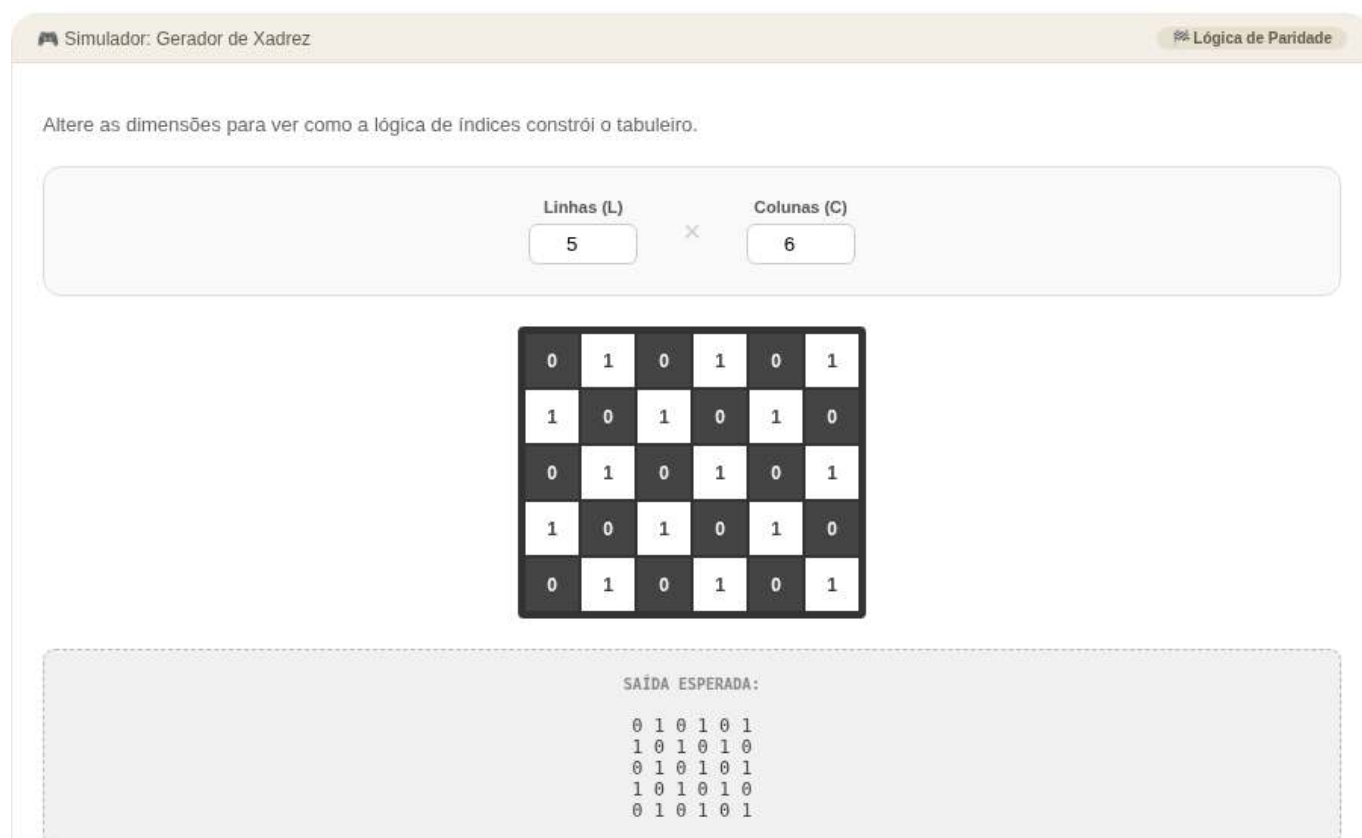


Figura 1.20: Simulador: Gerador de Xadrez

1.16.10 EP01_10 Metadados: Leitura de Arquivo PGM

Nesta atividade, você deve ler um arquivo de imagem no formato **PGM (Portable Gray Map)** e extrair suas dimensões a partir do cabeçalho.

- O formato **PGM (P2)** é um arquivo de texto simples (ASCII) que armazena imagens em tons de cinza.
- O arquivo possui um **cabeçalho** estruturado da seguinte forma:
 1. **Versão:** O identificador P2.
 2. **Comentários:** Linhas opcionais que começam com # (devem ser ignoradas).
 3. **Dimensões:** Dois inteiros representando **Largura** e **Altura**.
 4. **Máximo:** Um inteiro representando a intensidade máxima (geralmente 255).
- Após o cabeçalho, seguem-se os dados dos pixels.

📌 Importante:

- **Leitura de Arquivo:** Você deve abrir o arquivo indicado no exemplo utilizando a função `open()` do Python.
- **Ordem de Saída:** Ao contrário da ordem presente no arquivo, a saída esperada deve estar no formato de tupla: (**Altura**, **Largura**, **Canais**).
- Como arquivos PGM são em tons de cinza, o número de **Canais** é sempre **1**.
- Ver um simulador interativo para esta questão na **Figure 1.21** (ajuste as dimensões para ver como o cabeçalho ASCII é gerado).

1.16.10.1 🧠 Entendendo o formato PGM

O formato PGM é um dos mais simples para processamento de imagens. Por ser em texto puro, ele permite visualizar os metadados e até os valores dos pixels abrindo o arquivo em um bloco de notas:

Componente	Exemplo	Significado
Magic Number	P2	Identifica que é um PGM em formato de texto (ASCII).
Comentário	# CREATOR...	Linha informativa ignorada pelo processador.
Dimensões	397 343	397 colunas (Largura) e 343 linhas (Altura).
Intensidade	255	Define o valor do branco puro (escala de 0 a 255).

1.16.10.2 📋 Tarefa (especificação para VPL)

Entrada:

Nenhuma entrada via teclado. O programa deve ler o arquivo "aula01fig03b.pgm" presente no diretório de execução.

Saída:

Uma tupla contendo (Altura, Largura, 1).

1.16.10.3 📌 Exemplos

Nome do Arquivo	Saída Esperada	Observação
"aula01fig03b.pgm"	(343, 397, 1)	Note a inversão da ordem: Altura primeiro



Figura 1.21: Simulador: Estrutura do Arquivo PGM

i Nota

O arquivo necessário para este EP será baixado automaticamente do repositório por meio do seguinte código:

```
import os, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/all/cap01/imagens"
file = "aula01fig03b.pgm"

opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
urllib.request.install_opener(opener)

for f in [file]:
    if not os.path.exists(f):
        url = f"{BASE_URL}/{f}"
        try:
            urllib.request.urlretrieve(url, f)
            print(f" Arquivo baixado: {f}")
        except urllib.error.HTTPError as e:
            print(f" Erro {e.code}: Não encontrado em {url}")
```

```
%%writefile EP01_10.py
# sua solução
```

```
Overwriting EP01_10.py
```

```
TestSuite("EP01_10.py").run()
```

1.16.11 EP01_11 ↗ Análise de Vizinhaça: Filtro de Máximo 1D

Nesta atividade, você deve implementar um filtro morfológico simples de máximo operando em um sinal unidimensional (vetor).

- Leia um inteiro **n**, representando o tamanho do vetor.
- Leia os **n** elementos inteiros que compõem o vetor original **v1**.
- Crie um novo vetor **v2**, onde cada posição *i* é o resultado da comparação entre o elemento atual e seus vizinhos imediatos:

$$v2[i] = \max(v1[i-1], v1[i], v1[i+1])$$

 Importante:

- **Bordas:** Nas extremidades do vetor (índices 0 e $n-1$), a vizinhaça possui apenas dois elementos (o próprio e o único vizinho disponível). No índice 0, compare apenas $v1[0]$ e $v1[1]$. No último índice, compare apenas $v1[n-2]$ e $v1[n-1]$.
- **Saída:** Imprima o cabeçalho “v2:” seguido pelos valores do vetor resultante, um por linha.
- Ver um simulador interativo para esta questão na **Figure 1.22** (passe o mouse sobre os resultados para visualizar a janela de vizinhaça utilizada no cálculo).

1.16.11.1 🧠 Por que analisar vizinhos?

Em processamento de imagens, o valor de um pixel raramente é isolado; ele depende do contexto ao seu redor. O **Filtro de Máximo** é a base da operação de **Dilatação** em morfologia matemática, servindo para:

Função	Efeito Visual
Realce	Expande estruturas brilhantes e “engorda” objetos claros.
Remoção de Ruído	Elimina pequenos pontos pretos (ruído de “sal e pimenta” escuro).
Preenchimento	Fecha pequenos buracos ou lacunas em formas binárias.

1.16.11.2 📋 Tarefa (especificação para VPL)

Entrada:

Um inteiro n .

Nas linhas seguintes, os n elementos inteiros do vetor.

Saída:

A string $v2$: na primeira linha.

Nas linhas seguintes, cada elemento de $v2$ (um por linha).

1.16.11.3 📌 Exemplos

Entrada	Saída	Observação
5	$v2$:	No índice 1: $\max(10, 20, 5) = 20$
10	20	
20	20	
5	30	
30	30	
15	30	

```
%%writefile EP01_11.py
# sua solução
```

Overwriting EP01_11.py

```
TestSuite("EP01_11.py").run()
```




Figura 1.22: Simulador: Filtro de Máximo Local 1D

Capítulo 2

Da Captura ao Pixel - Amostragem, Quantização e Conectividade



Este capítulo aprofunda a compreensão da imagem digital, transitando da natureza física da captura para a sua representação matemática discreta. Investigamos como a luz se torna dado e como a organização espacial dos pixels define as relações de vizinhança e conectividade essenciais para algoritmos avançados de Visão Computacional (VC).

2.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Explicar o modelo físico de formação da imagem baseado em iluminação e refletância.
- Diferenciar os mecanismos da visão humana e dos sensores digitais.
- Compreender os processos de **amostragem** (discretização do espaço) e **quantização** (discretização da intensidade).
- Descrever as relações topológicas entre pixels: vizinhança, adjacência, conectividade e distâncias.
- Realizar transformações geométricas básicas (translação, rotação, escala) preservando a qualidade.
- Visualizar na prática os efeitos da variação da resolução espacial e da profundidade de bits.

2.2 O Olho e a Câmera - Elementos da Percepção Visual

A formação de uma imagem digital começa com a captura da luz refletida pelos objetos. Como ilustrado na Figure 2.1, tanto o olho humano quanto as câmeras digitais seguem princípios ópticos semelhantes para focar a luz em uma superfície sensível, embora utilizem mecanismos biológicos e eletrônicos distintos para a transdução do sinal.

2.2.1 Visão humana

O olho funciona como um sistema óptico complexo: a luz atravessa a córnea, o humor aquoso, a pupila (controlada pela íris) e o cristalino — que ajusta o foco dinamicamente — até atingir a retina. Na retina, encontram-se os fotorreceptores: os **cones** (6 milhões), concentrados na fóvea, são responsáveis pela visão de cores e detalhes, enquanto os **bastonetes** (120 milhões) garantem a visão em baixa luminosidade (visão escotópica), detectando apenas intensidades de cinza. O ponto cego é a região de onde parte o nervo óptico, carecendo de receptores.

2.2.2 Sensores digitais

Nas câmeras, os sensores de imagem desempenham o papel da retina. Os dois tipos mais comuns são o **CCD** (*Charge-Coupled Device* - Dispositivo de Carga Acoplada) e o **CMOS** (*Complementary Metal-Oxide-Semiconductor* - Semicondutor de Óxido Metálico Complementar). O sensor é composto por uma matriz de fotossítios (pixels) que acumulam carga elétrica proporcional à luz incidente.

Para a reconstrução de cores, utiliza-se o **Filtro de Bayer**, uma matriz de filtros coloridos que permite que cada pixel capture apenas uma componente de cor: vermelho, verde ou azul (RGGB - *Red, Green, Green, Blue*). Posteriormente, um **ADC** (*Analog-to-Digital Converter* - Conversor Analógico-Digital) quantiza essa carga em valores numéricos, definidos por uma profundidade de bits (ex.: 8 bits, resultando em 256 níveis de intensidade).

i Curiosidade

Embora o olho humano tenha milhões de receptores, a resolução de alta definição é restrita à fóvea (visão central), equivalente a aproximadamente 120×120 pixels. A percepção de uma cena completa em alta resolução é resultado de intenso pós-processamento realizado pelo cérebro.

O Olho e a Câmera - Elementos da Percepção Visual

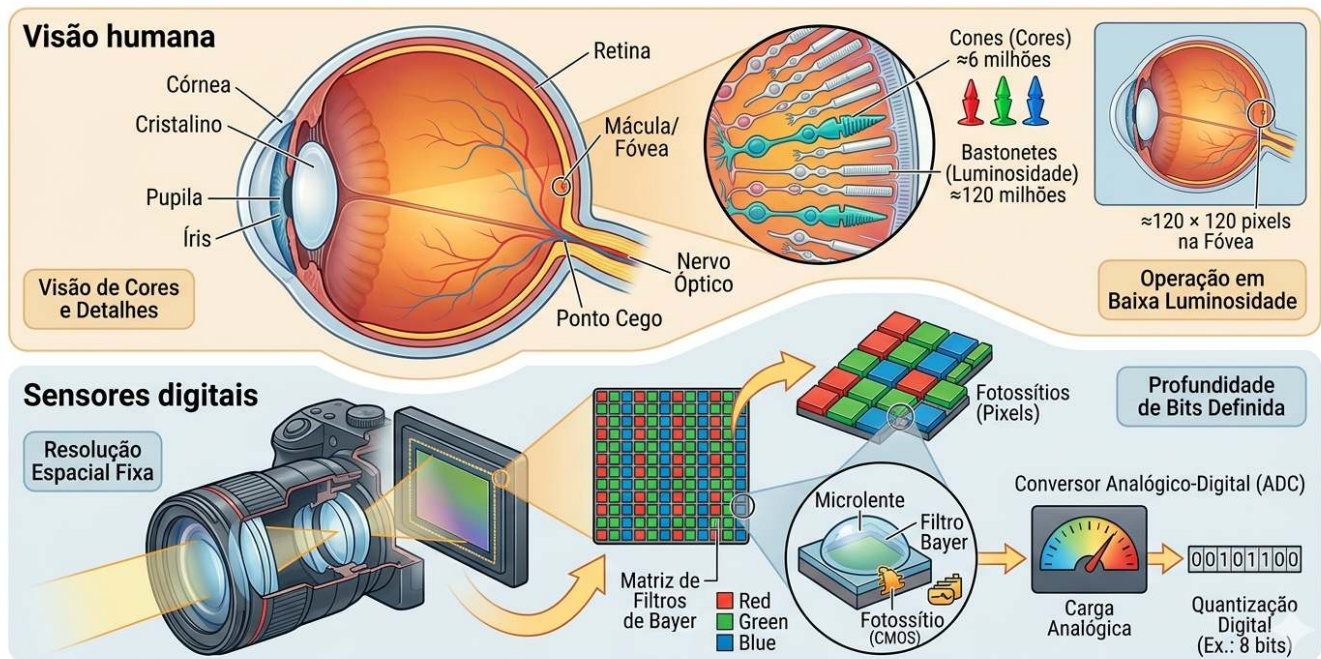


Figura 2.1: Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.

2.3 Ilusões de Ótica: Os Desafios da Percepção Visual

Enquanto os sensores digitais capturam a intensidade da luz de forma linear e objetiva, o sistema visual humano interpreta a cena com base em contexto, experiências prévias e mecanismos biológicos de sobrevivência. As ilusões de ótica não são “erros” do olho, mas evidências do intenso **pós-processamento cerebral** realizado no córtex visual.

2.3.1 Ambiguidade e Contexto

O cérebro busca constantemente dar sentido a padrões ambíguos. No exemplo do **Vaso de Rubin** (veja a Figure 2.2), a percepção alterna entre a figura (vaso) e o fundo (dois rostos), demonstrando que não conseguimos processar ambas as interpretações simultaneamente. Já a ilusão do **Elefante de Shepard** brinca com a nossa incapacidade de reconciliar linhas de contorno que sugerem volume em posições logicamente impossíveis.

2.3.2 Brilho e Contraste Local

Muitas ilusões decorrem da **inibição lateral**, mecanismo pelo qual neurônios vizinhos na retina competem entre si para realçar bordas. Na **Grade de Scintillating**, pontos escuros “fantasmas” parecem surgir nas interseções brancas devido a esse processamento local de contraste.

A ilusão da **Sombra no Tabuleiro de Adelson** é talvez a mais impactante para a VC: o quadrado “A” e o quadrado “B” possuem exatamente o mesmo valor de cinza no sensor (ou arquivo digital), mas o cérebro “corrige” o brilho de “B” por entender que ele está sob uma sombra projetada, percebendo-o como mais claro.

2.3.3 Geometria e Perspectiva

A percepção de profundidade pode ser enganada por construções geométricas que desafiam a lógica tridimensional a partir de um ângulo de visão específico. A **Escada de Schröder** utiliza a ambiguidade da perspectiva para criar um objeto que parece subir ou descer dependendo de como é observado, evidenciando como a nossa interpretação de “cima” e “baixo” depende do ponto de fuga.

Ilusões de Ótica: Os Desafios da Percepção Visual

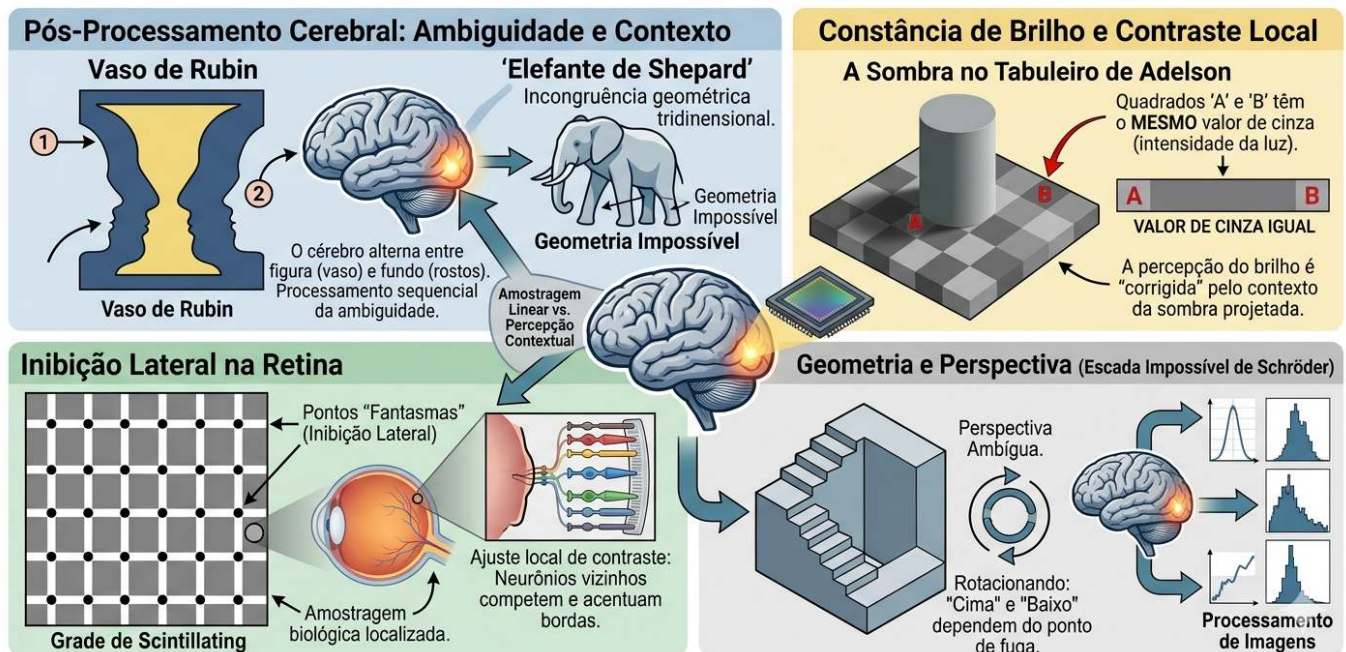


Figura 2.2: Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.

2.4 O Modelo Matemático da Formação da Imagem

Uma imagem pode ser modelada como o produto de duas funções:

$$f(x, y) = i(x, y) \cdot r(x, y) \quad (2.1)$$

onde:

- $i(x, y)$ é a **iluminação** incidente sobre a cena (energia luminosa por unidade de área), determinada pela fonte de luz;

- $r(x, y)$ é a **refletância** do objeto (fração da luz refletida), determinada pelas propriedades ópticas da superfície, com $0 < r(x, y) < 1$.

Na prática, as duas componentes variam em escalas espaciais distintas: $i(x, y)$ tende a variar lentamente no espaço, enquanto $r(x, y)$ pode apresentar variações abruptas associadas a texturas, bordas e detalhes finos. Técnicas de PDI frequentemente procuram separar ou compensar essas componentes, como em métodos de correção de iluminação não uniforme.

A Figure 2.3 ilustra como esse modelo se manifesta em imagens digitais coloridas, representadas por múltiplos canais espectrais (RGB) e, opcionalmente, por um canal adicional de transparência (RGBA).

2.4.1 Representação Digital e Canais Espectrais

Em imagens digitais coloridas, a função $f(x, y)$ da Equation 2.1 é representada por múltiplos canais espectrais. No padrão **RGB**, cada pixel armazena três amostras independentes:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y)]$$

correspondentes às intensidades das componentes vermelha (*Red*), verde (*Green*) e azul (*Blue*). Em imagens do tipo **RGBA**, adiciona-se um quarto canal:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y), A(x, y)]$$

onde $A(x, y)$ representa o canal **alfa** (*Alpha*), responsável por codificar a opacidade do pixel. Por convenção, o valor $A = 0$ indica um pixel totalmente transparente, enquanto $A = 255$ (ou 1) representa um pixel totalmente opaco.

Assim, uma imagem digital colorida é estruturada como uma matriz multidimensional de dimensões:

- **RGB:** $M \times N \times 3$
- **RGBA:** $M \times N \times 4$

em que cada posição (x, y) armazena as amostras associadas às propriedades ópticas daquela coordenada espacial.

Convenção de escala entre bibliotecas: Diferentes ecossistemas de programação adotam faixas de valores e ordenações distintas para representar os canais digitais, conforme sintetizado na Table 2.1.

Tabela 2.1: Convenções de escala e ordem de canais nas principais bibliotecas de PDI.

Biblioteca	Tipo padrão	Faixa de valores	Ordem das bandas	Forma do array/tensor
OpenCV	uint8	[0, 255]	BGR	$H \times W \times C$
Pillow	uint8	[0, 255]	RGB	$H \times W \times C$
NumPy	uint8/float32	[0, 255] ou [0, 1]	—	$H \times W \times C$
scikit-image	float64	[0, 1]	RGB	$H \times W \times C$
PyTorch	float32	[0, 1]	RGB	$C \times H \times W$
TensorFlow/Keras	float32	[0, 1]	RGB	$H \times W \times C$

A conversão entre essas faixas de intensidade é direta e dada por:

$$f_{\text{norm}}(x, y) = \frac{f(x, y)}{255}$$

onde $f(x, y) \in [0, 255]$ e $f_{\text{norm}}(x, y) \in [0, 1]$.

Negligenciar essas discrepâncias é uma fonte frequente de erros em fluxos de trabalho de VC — como, por exemplo, injetar uma imagem lida via OpenCV (padrão BGR, tipo `uint8`) diretamente em um modelo profundo do PyTorch que pressupõe o formato RGB normalizado.

2.4.2 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório e demonstra, para um pixel sintético, as diferentes convenções de escala e ordem de canais adotadas pelas principais bibliotecas de PDI.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")

# Demonstração das convenções de escala
img_exemplo = np.array([[200, 100, 50]], dtype=np.uint8) # 1 pixel RGB

print("\n Convenções de escala por biblioteca ")
print(f" NumPy / Pillow / OpenCV (uint8): {img_exemplo[0,0]} → faixa [0, 255]")
print(f" scikit-image / PyTorch (float): {img_exemplo[0,0] / 255.0} → faixa [0.0, 1.0]")
print(f" OpenCV: atenção - lê em BGR: {img_exemplo[0,0,:-1]} (canais invertidos)")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.0).

```
Convenções de escala por biblioteca
NumPy / Pillow / OpenCV (uint8): [200 100 50] → faixa [0, 255]
scikit-image / PyTorch (float): [0.78431373 0.39215686 0.19607843] → faixa [0.0, 1.0]
OpenCV: atenção - lê em BGR: [ 50 100 200] (canais invertidos)
```

2.4.3 Implementação da Leitura de Imagem em `morph.py`

O trecho a seguir exibe o código-fonte da função `mm.read`, permitindo verificar diretamente como a biblioteca `morph.py` implementa a leitura de imagens.

```
import inspect
print(inspect.getsource(mm.read))
```

```
@staticmethod
def read(file, pil=False, grayscale=False):
    """Lê imagem (local, URL ou Google Drive) → PIL.Image, ndarray 2D ou RGB 3D."""
    import re, requests, numpy as np
    from PIL import Image
    from io import BytesIO
    from urllib.request import urlopen, Request

    # - fonte: URL / Google Drive -
    if isinstance(file, str) and file.startswith(("http://", "https://", "id=")):
```



```

m = re.search(r"id=(\[w-]+\)", file) or re.search(r"/d/(\[w-]+\)", file)
url = f"https://drive.google.com/uc?export=view&id={m.group(1)}" \
    if m and ("id=" in file or "drive.google.com" in file) else file
hdr = {"User-Agent": "Mozilla/5.0 AppleWebKit/537.36 Chrome/124 Safari/537.36"}
try:
    r = requests.get(url, headers=hdr, timeout=20)
    if r.status_code == 429: raise requests.exceptions.HTTPError()
    r.raise_for_status()
    file = BytesIO(r.content)
except:
    file = BytesIO(urlopen(Request(url, headers=hdr), timeout=20).read())

img = Image.open(file)
img.load()
if pil:
    return img
if grayscale:
    return np.array(img.convert("L")) # (H,W)
if img.mode == "L":
    return np.array(img) # (H,W)
if img.mode == "RGBA":
    # fundo branco
    bg = Image.new("RGB", img.size, (255, 255, 255))
    bg.paste(img, mask=img.split()[3])
    return np.array(bg)
return np.array(img.convert("RGB")) # (H,W,3)

```

2.4.4 RGB e RGBA: Exemplo Prático com a Imagem Mandrill

O exemplo a seguir carrega a imagem Mandrill em formato RGB, constrói artificialmente um canal alfa com gradiente horizontal e exibe ambas as representações, ilustrando concretamente as estruturas matriciais $M \times N \times 3$ e $M \times N \times 4$.

```

import os
import numpy as np

# https://commons.wikimedia.org/wiki/File:Mandrill-k-means.png
url = "https://upload.wikimedia.org/wikipedia/commons/a/ab/Mandrill-k-means.png"
caminho = "imagens/mandrill.png"

# Leitura RGB
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True)
    mm.write(img_pil, caminho)
else:
    img_pil = mm.read(caminho, pil=True)

img_rgb = np.array(img_pil.convert("RGB")) # shape: (M, N, 3), uint8

# Criação do canal alfa (gradiente horizontal)
h, w, _ = img_rgb.shape
alpha = np.tile(np.linspace(0, 255, w, dtype=np.uint8), (h, 1))
img_rgba = np.dstack([img_rgb, alpha]) # shape: (M, N, 4), uint8

# Diagnóstico
print(f"RGB → shape={img_rgb.shape}, dtype={img_rgb.dtype}, "
      f"faixa=[{img_rgb.min()}, {img_rgb.max()}]")
print(f"RGBA → shape={img_rgba.shape}, dtype={img_rgba.dtype}, "
      f"faixa=[{img_rgba.min()}, {img_rgba.max()}]")
print(f"Pixel (0,0): RGB={img_rgb[0,0]} | RGBA={img_rgba[0,0]}")

# Normalização float32 (padrão scikit-image / PyTorch)
img_rgb_norm = img_rgb.astype(np.float32) / 255.0

```

```
print(f"\nNormalizado (float32): faixa=[{img_rgb_norm.min():.2f}, {img_rgb_norm.max():.2f}]\n")
print(f"Pixel (0,0) normalizado: {img_rgb_norm[0,0]}")

# Visualização
mm.show(
    [img_rgb, img_rgba],
    title=["RGB - $M \\times N \\times 3$", "RGBA - $M \\times N \\times 4$"],
    cols=2, axis=True
)
```

```
RGB → shape=(512, 512, 3), dtype=uint8, faixa=[14, 248]
RGBA → shape=(512, 512, 4), dtype=uint8, faixa=[0, 255]
Pixel (0,0): RGB=[125 110 58] | RGBA=[125 110 58 0]
```

```
Normalizado (float32): faixa=[0.05, 0.97]
Pixel (0,0) normalizado: [0.49019608 0.43137255 0.22745098]
```

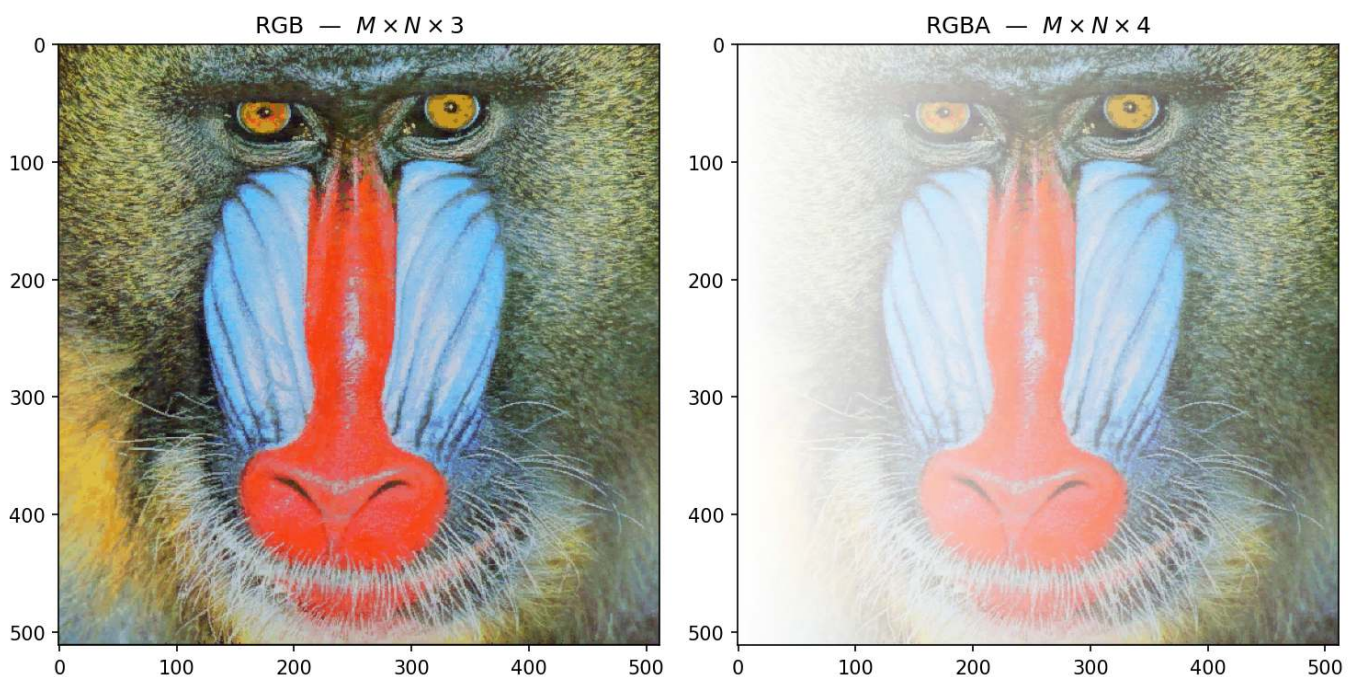


Figura 2.3: Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — [USC SIPI Image Database](#) (domínio público).

2.5 Digitalização: Amostragem e Quantização

Para transformar uma cena contínua em uma imagem digital, dois processos são necessários: amostragem e quantização.

2.5.1 Amostragem - Discretização do espaço

A amostragem consiste em medir o valor da função $f(x, y)$ em pontos igualmente espaçados, formando uma matriz de M linhas (altura) e N colunas (largura). Cada elemento dessa matriz é um **pixel**. A **resolução espacial** é dada por $M \times N$. Quanto maior a resolução, mais detalhes espaciais são preservados, mas maior também o custo computacional e de armazenamento.

2.5.2 Quantização - Discretização da intensidade

A quantização associa a cada pixel um valor numérico discreto, geralmente representado por um inteiro de b bits. A **profundidade de bits** define o número de níveis de intensidade: 2^b . Imagens em tons de cinza

costumam usar 8 bits (256 níveis). Imagens coloridas usam três canais de 8 bits (24 bits no total).

Ilustração: Se usarmos apenas 1 bit por pixel (preto e branco), perdemos todos os tons intermediários. Com 2 bits (4 níveis) já se percebe degradês grosseiros. Com 8 bits, o olho humano dificilmente percebe a discretização (visão contínua).

⚠ Erro de quantização

Erro de quantização é a diferença entre o valor analógico real e o valor discreto atribuído. Ele se manifesta como ruído de quantização, visível em regiões com gradiente suave quando se usa poucos bits.

2.5.3 Efeitos da amostragem e quantização

Os experimentos a seguir mostram como a redução da resolução espacial (subamostragem) e da profundidade de bits degradam a qualidade visual. Use o código para explorar diferentes fatores e níveis de cinza.

```
# Carregar imagem de exemplo
base = "https://upload.wikimedia.org/wikipedia/commons"

arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)

url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True) # retorna PIL Image com EXIF
    mm.write(img_pil, caminho)      # salva preservando EXIF
else:
    img_pil = mm.read(caminho, pil=True)

img_numpy = np.array(img_pil)

exif = img_pil._getexif() # agora funciona - img_pil é PIL Image

img_color = mm.read(caminho)
img_gray0 = mm.gray(img_color)
print(f"Imagem original: {img_color.shape}")
print(f"Tipo da imagem: {type(img_color)}")
```

```
Imagem original: (3067, 2047, 3)
Tipo da imagem: <class 'numpy.ndarray'>
```

O experimento apresentado na Figure 2.4 ilustra o compromisso entre a **resolução espacial** e o **custo de armazenamento** em memória. O código utiliza a técnica de subamostragem por fatiamento (*slicing*) para reduzir a matriz original de pixels conforme um fator f , resultando em uma economia de memória — por exemplo, um fator $f = 8$ reduz o tamanho do dado em 64 vezes (8^2). Para fins de comparação visual, as imagens reduzidas são restauradas às dimensões originais (512×512) através da interpolação por **vizinho mais próximo** (*nearest*). Este processo não recupera a informação perdida, mas torna evidente o efeito de *aliasing* e a estrutura de blocos (pixelização) gerada pela baixa densidade de dados da matriz amostrada.

```
def subsample_simple(image, f):
    # Subamostragem via fatiamento (slicing)
    reduced = image[::f, ::f]
```

```
# Cálculo de memória em KB
mem_kb = reduced.nbytes / 1024
label = f"{reduced.shape[1]}x{reduced.shape[0]}, {int(mem_kb)} KB\n(Fator {f})"

# Restaura o tamanho para visualização (H, W originais)
res = mm.resize(reduced, (image.shape[1], image.shape[0]), method='nearest')
return res, label

factors = [1, 4, 8, 12]
img_gray = img_gray0[820:1850, 890:1550] # Recorte para melhor visualização dos detalhes

# Gera os resultados e separa em listas para o mm.show
results = [subsample_simple(img_gray, f) for f in factors]
imgs_list = [r[0] for r in results]
titles_list = [r[1] for r in results]

mm.show(imgs_list, titles=titles_list, cols=4, figsize=(16, 12))
```

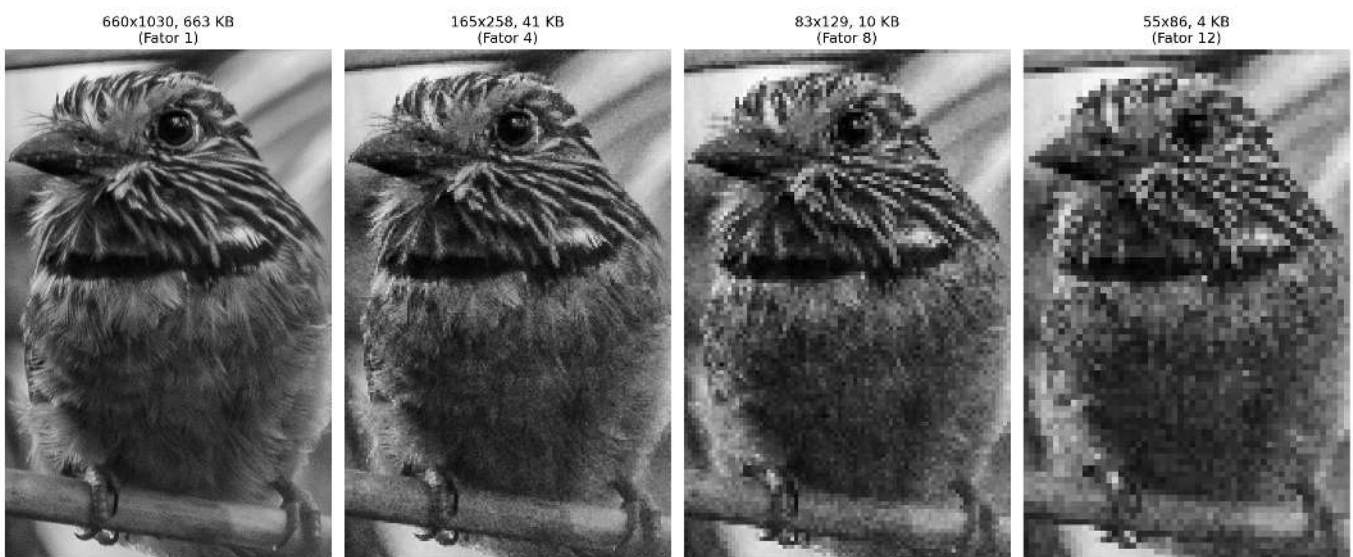


Figura 2.4: Efeito da subamostragem. Os títulos exibem as dimensões (W x H) e o tamanho da matriz em memória (KB).

O experimento na Figure 2.5 foca na **quantização de intensidade**, o processo de discretização da amplitude da função $f(x, y)$. Enquanto a subamostragem afeta a grade espacial, a redução da profundidade de bits limita a quantidade de tons de cinza disponíveis para representar o brilho.

Ao reduzir a profundidade de 8 bits (256 níveis) para valores menores, surge o efeito de **posterização**, onde os gradientes suaves de uma cena são substituídos por transições abruptas. No limite de 1 bit, a imagem torna-se estritamente binária, preservando apenas a silhueta e perdendo detalhes de textura e volume.

```
def quantize_simple(image, bits):
    """Reduz a profundidade de bits e calcula metadados de memória."""
    levels = 2 ** bits
    # Normalização e quantização uniforme
    quantized = (np.floor(image / 256 * levels) / levels * 255).astype(np.uint8)

    # Cálculo de memória em KB
    mem_kb = quantized.nbytes / 1024
    label = f"{bits} bits ({levels} níveis)\n{int(mem_kb)} KB"

    return quantized, label
```



```
# Lista de bits para teste (8 é o padrão, 1 é o binário)
bits_test = [8, 4, 2, 1]

# Gera os resultados e separa em listas para o mm.show
results_q = [quantize_simple(img_gray, b) for b in bits_test]
imgs_q = [r[0] for r in results_q]
titles_q = [r[1] for r in results_q]

mm.show(imgs_q, titles=titles_q, cols=4, figsize=(16, 12))
```



Figura 2.5: Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).

2.5.4 Análise Técnica

- **Domínio vs. Codomínio:** Note que a resolução espacial (dimensões da matriz) permanece constante em 512x512; o que se altera é apenas o codomínio da função da imagem.
- **Constância de Memória:** Observe nos títulos que o tamanho em KB não diminui. Isso ocorre porque o NumPy armazena cada pixel quantizado em um contêiner de 8 bits (`uint8`), independentemente de o valor real ser apenas 0 ou 1.
- **Percepção:** A degradação visual torna-se crítica abaixo de 4 bits, onde o olho humano começa a perceber as “fronteiras” artificiais criadas pela falta de tons intermediários.

A limitação de tipos de dados menores que um byte no ecossistema Python/NumPy deve-se à arquitetura de hardware, que endereça a memória em blocos de 8 bits (Bytes). Para que se mantenha a compatibilidade com o OpenCV e se garanta a eficiência, mapeiam-se até mesmo elementos binários para contêineres de 1 byte (`uint8` ou `bool8`).

Embora linguagens como ANSI C permitam a compactação de 8 pixels por byte (*bit-packing*), tal abordagem exige a descompactação constante para cálculos e impõe alta complexidade na manipulação de ponteiros. Conforme a Table 2.2, privilegia-se o uso de `uint8` pela facilidade no acesso a vizinhos e pela versatilidade em transformações geométricas. Além disso, métodos nativos do NumPy e OpenCV executam o processamento internamente em baixo nível (C/C++), o que torna operações vetorizadas mais rápidas do que implementações manuais com laços aninhados em Python.

Tabela 2.2: Comparativo entre estratégias de compactação e eficiência de processamento.

Característica	Python (NumPy/OpenCV)	ANSI C (Bit-packing)
Menor Unidade	1 Byte (8 bits)	1 Bit
Memória (Binária)	256 KB (para 512x512)	32 KB (para 512x512)
Velocidade	Alta (Vetorização em C)	Variável (Lenta se houver bit-shift)
Complexidade	Baixa: Métodos prontos	Alta: Ponteiros e Máscaras

2.6 Relações entre Pixels - Topologia da Imagem

Os pixels não são elementos isolados; suas posições relativas definem importantes conceitos para processamento.

2.6.1 Vizinhaça

Dado um pixel de coordenadas (x, y) , definem-se dois tipos principais de vizinhaça (para imagens em *grid* retangular):

- **Vizinhaça-4** (von Neumann): inclui os pixels nas posições $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, $(x, y+1)$.
- **Vizinhaça-8** (Moore): inclui todos os oito pixels adjacentes (acrescenta os quatro diagonais).

A escolha da vizinhaça influencia operações como detecção de bordas, cálculo de gradientes e conectividade.

```
# # Criação de uma matriz 3x3 para exemplo topológico
# viz = np.zeros((3, 3), dtype='uint8')

# # Definindo Vizinhaça-4 (N4) com valor diferente para destaque
# viz[0, 1] = viz[2, 1] = viz[1, 0] = viz[1, 2] = viz[1, 1] = 255

# ou simplesmente (teste com números como argumentos):
viz = mm.secross()

# Exibição da matriz para análise de coordenadas
mm.drawImgPlt(viz, scale=40)
```

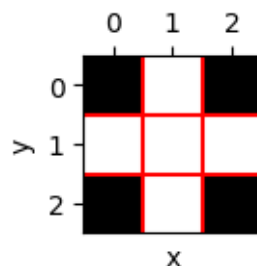


Figura 2.6: Ilustração de vizinhaças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.

```
import morph
importlib.reload(morph)
from morph import mm
```

2.6.2 Adjacência, conectividade e caminhos

Dois pixels são **adjacentes** se estão em contato segundo uma vizinhaça definida e satisfazem um critério de valor (ex.: mesmo nível de intensidade). Uma **conectividade** define uma relação de equivalência entre pixels que formam uma região conexa. Um **caminho** é uma sequência de pixels adjacentes.

A **conectividade-4** (N4) e **conectividade-8** (N8) podem produzir resultados diferentes na segmentação e no cálculo de componentes conexas (etiquetagem). Por exemplo, um padrão de tabuleiro de xadrez pode ser completamente desconexo em N4, mas totalmente conexo em N8.

2.6.3 Distâncias entre pixels

As métricas de distância são fundamentais para quantificar a proximidade física e a conectividade entre os elementos que compõem a grade digital. Conforme demonstrado na Table 2.3, a escolha da métrica define o custo de deslocamento entre pixels e altera o comportamento de algoritmos de segmentação e análise morfológica.

Para medir a distância entre dois pixels $p(x_1, y_1)$ e $q(x_2, y_2)$, utilizam-se diferentes funções métricas que impõem restrições de movimento distintas sobre o *grid*:

Tabela 2.3: Comparativo de métricas de distância aplicadas à malha de pixels.

Métrica	Definição	Interpretação
Euclidiana	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Distância exata em linha reta (contínua)
Manhattan (<i>City block</i>)	$ x_1 - x_2 + y_1 - y_2 $	Movimentos horizontais + verticais
Chebyshev (<i>Tabuleiro</i>)	$\max(x_1 - x_2 , y_1 - y_2)$	Maior deslocamento entre os eixos

Essas distâncias são aplicadas em diversos contextos de PDI, incluindo algoritmos de interpolação geométrica, transformadas de distância, crescimento de regiões e análise de formas.

i Exemplo prático

Considerando dois pixels com deslocamentos relativos $\Delta x = 3$ e $\Delta y = 4$:

- **Euclidiana**: $\sqrt{3^2 + 4^2} = 5$ (hipotenusa do triângulo retângulo).
- **Manhattan**: $3 + 4 = 7$ (soma dos catetos).
- **Chebyshev**: $\max(3, 4) = 4$ (predomínio do maior deslocamento).

2.7 Armazenamento de Imagens

A escolha do formato de arquivo é um passo decisivo no fluxo de processamento, pois determina como os dados de amostragem e quantização serão preservados ou descartados. Conforme se apresenta na Table 2.4, cada extensão equilibra de forma distinta a fidelidade dos dados e a eficiência de armazenamento.

Tabela 2.4: Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.

Formato	Características	Uso típico
PGM	Formato simples de mapa de cinzas (texto ou binário).	Pesquisa acadêmica e ferramentas Unix.
BMP	Não comprimido (ou compressão simples).	Windows, aplicações legado.
PNG	Compressão sem perdas (<i>lossless</i>).	Web, imagens com transparência.
JPEG	Compressão com perdas (<i>lossy</i>), ideal para fotografias.	Fotos, câmeras digitais.
TIFF	Suporta múltiplas camadas e compressão variada.	Editoração, arquivamento.
RAW	Dados brutos do sensor, sem processamento.	Fotografia profissional.

Formato	Características	Uso típico
DICOM	Padrão médico com metadados clínicos embutidos (paciente, equipamento, protocolo).	Radiologia, tomografia, ressonância magnética.

Os metadados de uma imagem incluem parâmetros como largura, altura, profundidade de bits e codificação de cor. Em formatos científicos, preservam-se também informações de calibração e detalhes da captura. Ao utilizar-se a função `mm.read()`, a biblioteca `morph` preserva automaticamente esses dados para que se respeitem as propriedades originais da imagem.

Em contextos científicos e hospitalares, privilegia-se o padrão **DICOM** (*Digital Imaging and Communications in Medicine*) para garantir que não haja perda de precisão diagnóstica. Repositórios públicos como o [The Cancer Imaging Archive \(TCIA\)](#), o [Alzheimer’s Disease Neuroimaging Initiative \(ADNI\)](#) e o [PhysioNet](#) disponibilizam vastos conjuntos de dados neste formato, incluindo metadados clínicos anonimizados que são essenciais para a investigação científica.

2.7.1 Exemplo: Extração de Metadados e Localização GPS

Diferente da matriz de pixels pura obtida pela leitura convencional — como na imagem do pássaro apresentada no início deste capítulo —, o uso do argumento `pil=True` no método `mm.read()` altera a natureza do objeto retornado (ver Figure 2.7). Enquanto o padrão (`pil=False`) retorna um `numpy.ndarray` RGB, a leitura com `pil=True` retorna um objeto especializado da biblioteca Pillow, capaz de interpretar o cabeçalho **EXIF**.

O cabeçalho **EXIF** (*Exchangeable Image File Format*) funciona como um repositório técnico da captura, permitindo que o objeto Pillow interprete uma vasta gama de informações que vão muito além das coordenadas de GPS. Ao utilizar `pil=True`, o sistema ganha acesso ao “DNA” da imagem, incluindo metadados de hardware (marca e modelo da câmera), configurações ópticas (abertura, distância focal e tempo de exposição) e parâmetros de iluminação (uso de flash e balanço de branco).

Essa distinção é importante no PDI, pois transforma a matriz de amostragem em um conjunto de dados contextualizado, onde as características físicas do sensor e da lente podem ser utilizadas para normalizar brilhos ou corrigir distorções geométricas.

```
from PIL.ExifTags import TAGS
import os, numpy as np

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)
url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

# 1. Leitura - preserva objeto PIL com EXIF
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho) # salva preservando EXIF
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj) # conversão para NumPy

# 2. Extração e conversão de GPS (Tag 34853)
exif = img_obj._getexif()
```

```

if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(-(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
                                   else (dms[0]+dms[1]/60+dms[2]/3600))
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy, scale=30)

```

```

GPS Decimal: -24.587955, -48.629758
Maps: https://www.google.com/maps/search/?api=1&query=-24.587955,-48.629758333333335

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (2047, 3067)
Pillow (0,0): (58, 96, 0)
Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (3067, 2047, 3)
NumPy [0,0]: [58 96  0]

```

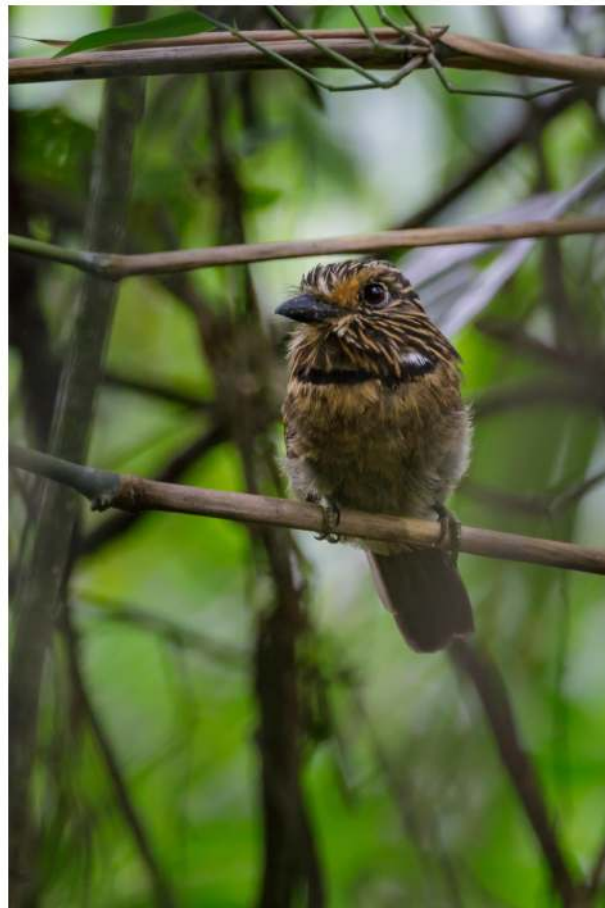


Figura 2.7: Área de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (*Malacoptila striata*). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).

i Nota Pedagógica: A sutil diferença das dimensões

Repare que a representação das dimensões muda conforme a estrutura de dados utilizada:

- **No Pillow (.size):** Retorna (Largura, Altura) — no exemplo: (2047, 3067). É uma visão orientada ao arquivo de imagem.
- **No NumPy (.shape):** Segue a convenção matemática de matrizes: (Linhas/Altura, Colunas/Largura, Canais) — no exemplo: (3067, 2047, 3).

Esta distinção é fundamental para evitar erros de indexação ao implementar filtros manuais. Enquanto o objeto Pillow carrega o “**onde**” e o “**quando**” (contexto), o array NumPy carrega o “**quanto**” de luz (intensidade) existe em cada ponto da imagem.

2.7.2 Por que esta separação é importante?

Ao carregar uma imagem via `cv2.imread()`, o resultado é um `<class 'numpy.ndarray'>`, que contém estritamente os valores numéricos resultantes da **quantização** e **amostragem**. No entanto, ao utilizar `pil=True`, o `mm.read()` retorna um objeto da classe `PIL.JpegImagePlugin.JpegImageFile`.

Esta classe mantém o arquivo “aberto” para permitir o acesso ao contexto da captura antes que os dados sejam convertidos em uma matriz bruta. Note que o pixel (0, 0) é idêntico nas duas representações — (58, 96, 0) no Pillow e [58, 96, 0] no NumPy —, confirmando que ambos descrevem os mesmos dados, apenas com interfaces distintas. Essa separação é vital: pixels servem para algoritmos; metadados servem para georreferenciamento, catalogação científica e correções baseadas no hardware de aquisição.

Para inspecionar todos os metadados EXIF de um objeto Pillow:

```
from PIL import ExifTags

exif_raw = img_obj._getexif()
if exif_raw:
    for tag_id, valor in sorted(exif_raw.items()):
        tag_nome = ExifTags.TAGS.get(tag_id, f"TAG_{tag_id}")
        print(f" {tag_nome:40s} : {valor}")
```

2.8 Transformações Geométricas Básicas

As transformações geométricas alteram a posição dos pixels, mantendo os valores de intensidade. São fundamentais para alinhamento, correção de distorções e aumento de dados (*data augmentation*) em aprendizado de máquina.

Uma **transformação afim** é qualquer mapeamento que preserve colinearidade (pontos sobre uma reta permanecem sobre uma reta) e razões de distâncias entre pontos colineares. Em 2D, toda transformação afim pode ser expressa em **coordenadas homogêneas** por uma matriz 3×3 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{T}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.2)$$

A sub-matriz 2×2 superior esquerda $\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ controla rotação, escala e cisalhamento; o vetor $(t_x, t_y)^\top$ controla a translação. As transformações geométricas mais usadas em PDI — translação, rotação e escala — são casos particulares de $\mathbf{T}$, e podem ser **compostas** por multiplicação de matrizes, na ordem $\mathbf{T} = \mathbf{T}_n \cdots \mathbf{T}_2 \mathbf{T}_1$.

i Transformação inversa e interpolação

Na implementação prática (`cv2.warpAffine`), aplica-se a **transformação inversa**: para cada pixel (x', y') da imagem destino, calcula-se a posição de origem $(x, y) = \mathbf{T}^{-1}(x', y')$ e interpola-se o valor. Isso evita buracos na imagem resultante causados pelo mapeamento direto de pixels inteiros para posições não inteiras.

2.8.1 Translação

A translação é a transformação afim mais simples: desloca todos os pixels por um vetor (t_x, t_y) . Em coordenadas homogêneas, é expressa pela matriz:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases} \quad (2.3)$$

A terceira linha da matriz garante que a operação permaneça no espaço afim, permitindo que translação, rotação e escala sejam combinadas por simples multiplicação de matrizes. Na prática, `cv2.warpAffine` usa apenas as duas primeiras linhas (matriz 2×3), pois a terceira é sempre $[0, 0, 1]$.

Pixels deslocados para além da área original são descartados; áreas descobertas são preenchidas com 0 (preto). Ver um exemplo na Figure 2.8.

```
img_tx1 = mm.translate(img_gray, 50, 50)
img_tx2 = mm.translate(img_gray, 100, 50)
mm.show([img_gray, img_tx1, img_tx2],
        titles=["Original", "Translação (50,50)", "Translação (100,50)"],
        cols=3, figsize=(16, 12))
```



Figura 2.8: Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).

2.8.2 Rotação

A rotação por ângulo θ em torno de um ponto central (c_x, c_y) é composta por três transformações afins: translação para a origem, rotação pura e translação de volta. A matriz resultante é:

$$\mathbf{T}_{\text{rot}} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

Na implementação, `cv2.getRotationMatrix2D` gera diretamente as duas primeiras linhas de $\mathbf{T}_{\text{rot}}$ (matriz 2×3 para `warpAffine`), aceitando também um fator de escala s que multiplica $\cos \theta$ e $\sin \theta$. Ver exemplo na Figure 2.9.

Como a rotação desloca pixels para novas posições não-inteiras, `cv2.warpAffine` precisa estimar a cor de cada pixel de destino a partir dos vizinhos — processo chamado **interpolação**. O parâmetro `interp` controla esse comportamento:

- **nearest** (`INTER_NEAREST`): atribui o valor do pixel mais próximo. Rápido, mas produz serrilhado (*aliasing*) em bordas diagonais.
- **bilinear** (`INTER_LINEAR`, padrão): média ponderada dos 4 vizinhos mais próximos. Equilibra qualidade e desempenho — adequado para a maioria dos casos.
- **bicubic** (`INTER_CUBIC`): considera os 16 vizinhos em uma superfície cúbica. Produz bordas mais suaves, ao custo de maior processamento.

```
img_rot30 = mm.rotate(img_gray, 30, interp='bilinear')
img_rot45 = mm.rotate(img_gray, 45, interp='bilinear')

mm.show([img_gray, img_rot30, img_rot45],
        titles=["Original", "Rotação 30°", "Rotação 45°"],
        cols=3, figsize=(16, 12))
```



Figura 2.9: Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.

2.8.3 Escala (Redimensionamento)

O redimensionamento por fatores (s_x, s_y) é uma transformação afim com matriz:

$$\mathbf{T}_{\text{escala}} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.5)$$

Quando $s > 1$ (ampliação), pixels da imagem destino mapeiam para posições não inteiras na origem — exigindo interpolação para estimar o valor. Quando $s < 1$ (redução), múltiplos pixels de origem contribuem para um único pixel destino — exigindo **decimação**. Os mesmos três métodos descritos na rotação estão disponíveis em `mm.resize`, com a diferença de que aqui o impacto visual é mais perceptível: na ampliação, **nearest** produz efeito de blocos (*pixelation*), enquanto **bicubic** preserva melhor a nitidez das bordas, conforme a Table 2.5:

Tabela 2.5: Métodos de interpolação disponíveis em `mm.resize` e seus respectivos números de vizinhos utilizados no cálculo.

Método	Vizinhos usados	Característica
'nearest'	1	Rápido; produz efeito de blocos (<i>pixelation</i>)
'bilinear'	4	Bom compromisso qualidade/custo; bordas suaves
'bicubic'	16	Maior nitidez; preferido em softwares profissionais

O parâmetro `size_or_factor` aceita tanto um escalar (fator uniforme, e.g. 0.5 para reduzir à metade) quanto uma tupla (largura, altura) para dimensões absolutas.

```
# 1. Recorte da região do bico
y, x, offset = 210, 40, 40
crop = img_gray[y-offset:y+offset, x-offset:x+offset]
print(f"Imagem: {img_gray.shape} | Crop: {crop.shape}")

# 2. Ampliar 4× com mm.resize
crop_nearest = mm.resize(crop, size_or_factor=4, method='nearest')
crop_bilinear = mm.resize(crop, size_or_factor=4, method='bilinear')

# 3. Exibição comparativa
mm.show(
    [crop, crop_nearest, crop_bilinear],
    titles=["Original (recorte)", "Vizinho mais próximo (4×)", "Bilinear (4×)"],
    cols=3, figsize=(16, 12), dpi=200
)
```

Imagem: (1030, 660) | Crop: (80, 80)



Figura 2.10: Comparação de interpolação com zoom no detalhe do olho (recorte 60×60 , ampliado $4 \times$). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.

2.8.4 Cisalhamento (*Shear*)

O cisalhamento é uma transformação afim que distorce a imagem ao deslocar cada pixel proporcionalmente à sua posição em um eixo. A matriz geral combina cisalhamento horizontal (sh_x) e vertical (sh_y):

$$\mathbf{T}_{\text{cisalh}} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.6)$$

Para $sh_x \neq 0$ e $sh_y = 0$, cada linha é deslocada horizontalmente proporcionalmente à sua posição vertical — produzindo o efeito de “inclinação” característico. Ver exemplo na Figure 2.11.

```
img_shx = mm.shear(img_gray, shx=0.3)
img_shy = mm.shear(img_gray, shy=0.3)
img_shc = mm.shear(img_gray, shx=0.2, shy=0.2)

mm.show(
    [img_gray, img_shx, img_shy, img_shc],
    titles=["Original", "Horiz. (shx=0.3)", "Vert. (shy=0.3)", "Combinado (0.2, 0.2)"],
    cols=4, figsize=(16, 12)
)
```

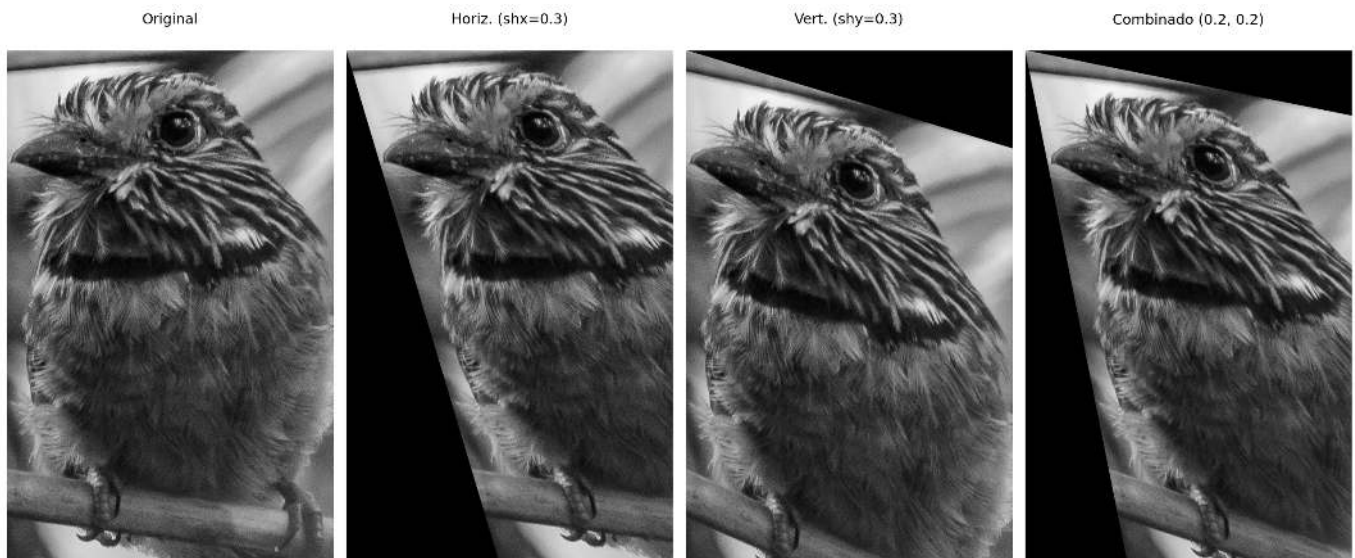


Figura 2.11: Exemplo de cisalhamento da imagem do pássaro: horizontal ($shx=0.3$), vertical ($shy=0.3$) e combinado ($shx=0.2$, $shy=0.2$).

2.9 Resumo

Neste capítulo foram apresentados os fundamentos de digitalização e topologia de imagens:

- **Formação da imagem:** $f(x, y) = i(x, y) \cdot r(x, y)$.
- **Amostragem:** discretização do espaço $\rightarrow$ resolução espacial $M \times N$.
- **Quantização:** discretização da intensidade $\rightarrow$ profundidade de bits b .
- **Relações topológicas:** vizinhança-4, vizinhança-8, conectividade, distâncias (Euclidiana, Manhattan, Chebyshev).
- **Transformações geométricas:** translação, rotação, escala (com interpolação bilinear ou vizinho mais próximo).
- **Formatos de arquivo:** BMP, PNG, JPEG, TIFF, RAW; cada um com diferentes compromissos entre qualidade e tamanho.

O Capítulo 3 abordará **operações espaciais** como convolução, filtragem e morfologia matemática (erosão, dilatação).

2.10 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentivamos o uso do **NotebookLM** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Para cada capítulo, preparamos um projeto específico na plataforma. Para uma experiência de estudo ampliada, utilize o acesso abaixo:

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o link a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 02](#)

⚠ Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

2.11 Lista de Exercícios

1. (15%) Explique, com suas próprias palavras, a diferença entre **amostragem** e **quantização**. Dê um exemplo concreto de cada uma no contexto de uma imagem digital.
2. (15%) Considere uma imagem com resolução espacial de 1024×768 pixels e profundidade de 24 bits (8 bits por canal RGB). Calcule o tamanho total não comprimido da imagem em bytes e em megabytes.
3. (20%) Utilizando o código do laboratório, modifique o fator de subamostragem para 3 e para 6. Descreva visualmente o que ocorre com as bordas dos objetos. O que é o efeito de *aliasing*?
4. (20%) Para a imagem em tons de cinza, aplique quantização com 3 bits (8 níveis) e 5 bits (32 níveis). Compare os resultados e explique por que 5 bits já pode ser considerado suficiente para muitas aplicações.
5. (15%) Dados dois pixels $A = (10, 20)$ e $B = (15, 25)$, calcule as distâncias Euclidiana, Manhattan e Chebyshev entre eles.
6. (15%) Usando a função `mm.rotate`, gire a imagem do pássaro em ângulos de 90° , 180° e 270° com interpolação bilinear. Compare com a rotação usando `method='nearest'`. Em quais situações a interpolação vizinho mais próximo ainda é útil?

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de amostragem, quantização e relações entre pixels.
- Szeliski (2022) para transformações geométricas e conectividade.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.



2.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.1 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (.py, .java, .c, .cpp, .js ou .r). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

2.12.1 EP02_01 ☉ Ajuste de Brilho e Contraste

Nesta atividade, o objetivo é implementar um operador de ponto para **transformação linear de intensidade**, aplicando o ajuste dinâmico de brilho e contraste em uma imagem digital.

2.12.1.1 📋 Diretrizes de Implementação

O algoritmo deve seguir o fluxo de execução abaixo:

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas) da matriz.
2. **Parâmetros:** Ler o valor real α (fator de contraste) e o inteiro β (fator de brilho).
3. **Dados:** Ler os valores inteiros da matriz original.
4. **Maapeamento:** Para cada pixel p , calcular o novo valor p' pela equação:

$$p' = \text{clip}(\text{round}(\alpha \cdot p + \beta))$$

5. **Saída:** Exibir a matriz resultante com as dimensões $L \times C$.

2.12.1.2 📌 Restrições Computacionais

- **Arredondamento (*Round*):** Aplica-se o arredondamento matemático para o inteiro mais próximo antes da conversão de tipo.
- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$ para preservar o padrão de 8 bits:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Simulação:** O efeito dos parâmetros α e β na correção histogramática pode ser observado na Figure 2.12.

2.12.1.3 Fundamentação Teórica

As alterações modificam o histograma da imagem para ajustar o perfil de iluminação e a distinção tonal.

Parâmetro	Função	Impacto Visual
α (Alpha)	Escalar	Modula o Contraste . Se $\alpha > 1$, expande o histograma; se $0 \leq \alpha < 1$, comprime.
β (Beta)	Aditiva	Modula o Brilho . Se positivo, translada o histograma para a direita; se negativo, para a esquerda.
clip	Limitador	Restringe a faixa dinâmica, impedindo erros de <i>underflow</i> e <i>overflow</i> .

2.12.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Valores de **alpha** (α) e **beta** (β).
- Linhas seguintes: Elementos numéricos da matriz original.

Saída:

- Matriz transformada estruturada em L linhas e C colunas.

2.12.1.5 Exemplos

A tabela a seguir apresenta um exemplo prático do comportamento esperado do algoritmo, destacando a atuação dos operadores de arredondamento e saturação.

Entrada	Saída	Observação
1 4 1.5 -30 0 100 180 255	0 120 240 255	Note o efeito de saturação no último pixel

Executando os Testes

Para rodar os testes, execute `TestSuite("EP02_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.



Figura 2.12: Simulador: Ajuste de Brilho e Contraste

```
%%writefile EP02_01.py
# sua solução
```

```
Overwriting EP02_01.py
```

```
TestSuite("EP02_01.py").run()
```

2.12.2 EP02_02 Subamostragem Espacial

Nesta atividade, você deve implementar a redução da resolução espacial de uma imagem através do processo de subamostragem.

- Leia dois inteiros L e C , representando as dimensões da matriz original.
- Leia um valor inteiro f ($f \geq 1$), que representa o fator de amostragem.
- Leia os valores inteiros da matriz original.
- A nova imagem deve ser construída selecionando o pixel da posição $(f \cdot i, f \cdot j)$ da imagem original.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.13 uma simulação deste EP.

Importante:

- **Dimensões Finais:** A imagem amostrada terá dimensões $\lceil L/f \rceil \times \lceil C/f \rceil$. No contexto de programação, isso equivale ao tamanho resultante de um fatiamento (slicing) com passo f .
- **Implementação:** Não utilize funções prontas de bibliotecas de processamento de imagens (como OpenCV ou PIL) para o redimensionamento. Implemente a lógica de seleção de pixels manualmente ou via fatiamento de matrizes.
- **Aliasing:** Note que este processo pode causar o efeito de *aliasing* (serrilhamento), onde detalhes finos são perdidos ou padrões indesejados aparecem.

2.12.2.1 🧠 Discretização do Espaço

A subamostragem reduz a resolução espacial de uma imagem, selecionando apenas um pixel a cada f pixels em cada direção. É o processo inverso da interpolação:

Parâmetro	Função	Efeito
Fator f	Salto de amostragem	Define o intervalo de seleção. Um fator 2 reduz a largura e altura pela metade.
Resolução	Densidade de pixels	Diminui a quantidade total de informação espacial da imagem.
Aliasing	Efeito colateral	Surgimento de padrões em escada ou blocos devido à perda de detalhes finos.

2.12.2.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o fator f .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz reduzida com as dimensões correspondentes ao fatiamento por f .

2.12.2.3 📌 Exemplos

Entrada	Saída	Observação
2 4 2 10 20 30 40 50 60 70 80	10 30	O fator 2 seleciona os pixels (0,0) e (0,2) da primeira linha. A segunda linha é ignorada.

```
%%writefile EP02_02.py
# Código Python
```

```
Overwriting EP02_02.py
```

```
TestSuite("EP02_02.py").run()
```

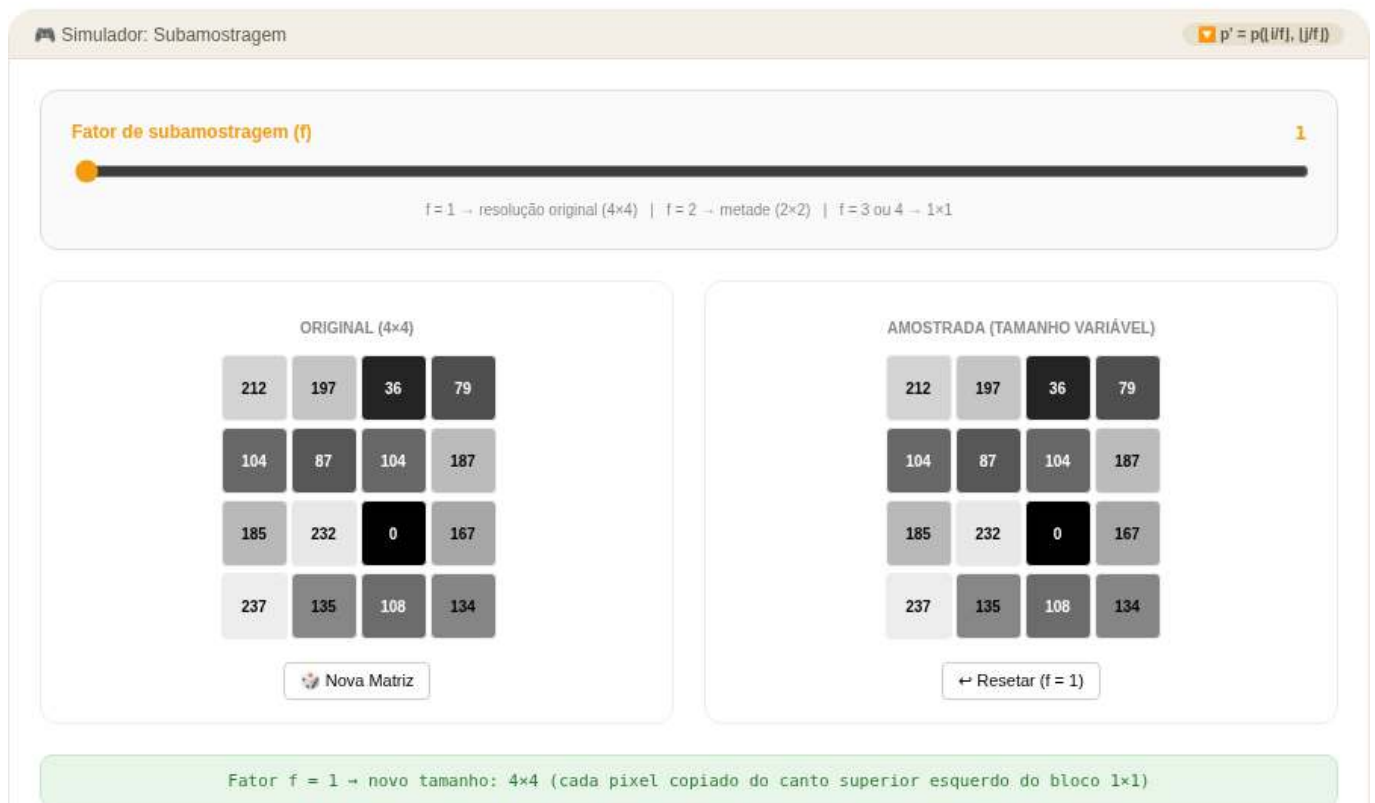


Figura 2.13: Simulador: Subamostragem

2.12.3 EP02_03 🎮 Quantização de Níveis de Cinza

Nesta atividade, você deve implementar a quantização uniforme de uma imagem, reduzindo a quantidade de níveis de intensidade de cinza originais para uma nova escala baseada em um número menor de bits.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia um inteiro k ($1 \leq k \leq 8$), representando o novo número de bits da imagem.
- Calcule o número de níveis ($N = 2^k$) e o tamanho do intervalo (passo).
- Para cada pixel p , calcule o novo valor p' mapeando-o para o índice do nível discretizado correspondente (variando de 0 a $2^k - 1$).
- Imprima a matriz resultante com os mesmos valores de dimensões originais.
- Ver na Figure 2.14 uma simulação deste EP.

📌 Importante:

- **Posterização:** Ao reduzir drasticamente os níveis (ex: $k = 2$), você notará que degradês suaves se transformam em faixas abruptas de cor devido à perda de resolução de amplitude.
- **Cálculo do Passo:** O intervalo entre cada nível é definido por $passo = 256/2^k$.
- **Mapeamento:** O método de quantização uniforme por truncamento que mapeia o pixel para o índice do seu respectivo nível discretizado é dado por:

$$p' = \left\lfloor \frac{p}{passo} \right\rfloor$$

Em termos de implementação (como em Python), isso equivale à divisão inteira: $p' = p // passo$.

2.12.3.1 🧠 Discretização da Amplitude

Enquanto a subamostragem lida com a resolução espacial, a quantização foca na precisão da cor (amplitude). Reduzir os bits significa simplificar a informação cromática:

Parâmetro	Função	Efeito
Bits (k)	Profundidade de cor	Define quantos tons diferentes a imagem pode ter (2^k).
Passo	Intervalo de tom	Espaçamento entre os níveis de cinza permitidos.
Posterização	Fenômeno visual	Transformação de variações contínuas em blocos de cor sólida.

2.12.3.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o número de bits k .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com os índices dos níveis quantizados, mantendo o tamanho original $L \times C$.

2.12.3.3 📌 Exemplos

Entrada	Saída	Observação
1 4 2 0 80 170 255	0 1 2 3	Com $k = 2$, temos $2^2 = 4$ níveis discretos disponíveis (0, 1, 2, 3). O passo é $256/4 = 64$. Aplicando a divisão inteira por elemento: $0//64 = 0$, $80//64 = 1$, $170//64 = 2$, $255//64 = 3$.
1 5 1 10 50 120 200 250	0 0 0 1 1	Com $k = 1$, temos $2^1 = 2$ níveis (0 e 1). Passo = $256/2 = 128$. Pixels menores que 128 resultam em 0, e pixels maiores ou iguais a 128 resultam em 1.

```
%%writefile EP02_03.py
# Código Python
```

```
Overwriting EP02_03.py
```

```
TestSuite("EP02_03.py").run()
```

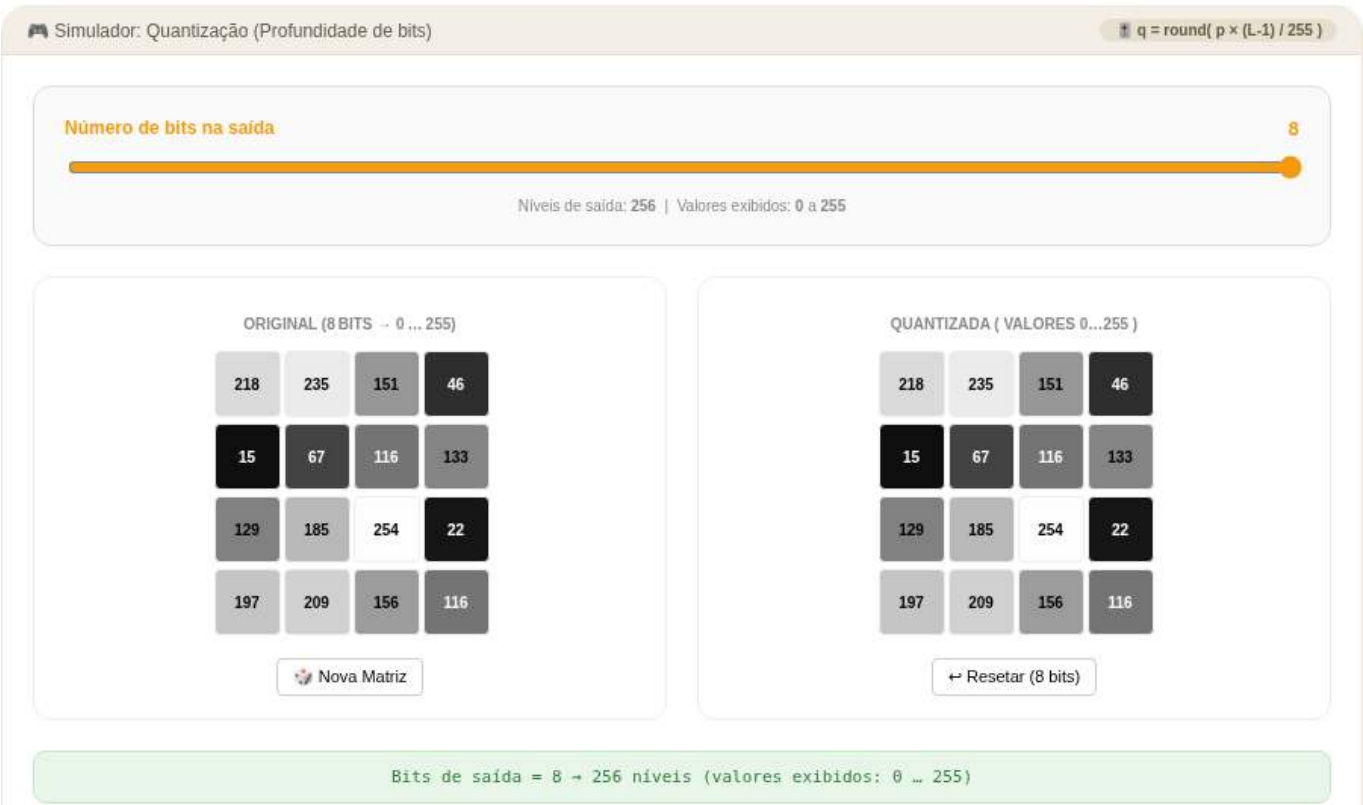


Figura 2.14: Simulador: Quantização

2.12.4 EP02_04 Transformada de Distância em Imagem Binária

Dada uma imagem binária onde pixels de valor **1** representam o *objeto* e pixels **0** representam o *fundo*, a distância de um pixel de fundo recebe a **menor distância ao pixel de objeto mais próximo**. Pixels de objeto recebem distância **0**. Para simplificar, considere que a imagem tem **apenas um único objeto com um pixel de valor 1**.

Problema: Leia uma imagem binária $L \times C$ e uma métrica, e calcule essa distância simplificada aplicando uma das três fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(\Delta r)^2 + (\Delta c)^2}$$

$$d_{\text{City-block}} = |\Delta r| + |\Delta c|$$

$$d_{\text{Chessboard}} = \max(|\Delta r|, |\Delta c|)$$

onde Δr é a diferença de linhas e Δc a diferença de colunas entre dois pixels.

2.12.4.1 Por que isso importa? - Aplicações da DT

A Transformada de Distância (DT) aparece em dezenas de pipelines de visão computacional:

Métrica	Complexidade	Aplicação típica
Euclidiana	$O(n^2)$ ingênuo	Esqueletização, matching de formas

Métrica	Complexidade	Aplicação típica
City-block	● $O(n)$ com 2 passes	Morfologia, dilatação/erosão
Chessboard	● $O(n)$ com 2 passes	Morfologia, dilatação/erosão

2.12.4.2 📌 Requisitos Técnicos

- **Entrada:** * Primeira linha: L e C (inteiros).
 - Segunda linha: nome da métrica (`euclidean`, `cityblock` ou `chessboard`).
 - Em seguida, a matriz binária $L \times C$ (valores 0 ou 1).
- **Pixels de objeto (1):** distância = 0 (ou 0.00 para euclidiana).
- **Pixels de fundo (0):** distância ao único pixel de objeto na imagem.
- **Arredondamento (euclidiana):** imprimir com **2 casas decimais** (formato `:.2f`). City-block e Chessboard produzem inteiros — imprimir sem decimais.
- **Saída:** valores separados por espaço, uma linha por linha da matriz.
- Ver na Figure 2.15 uma simulação deste EP.

2.12.4.3 📌 Exemplos

Entrada	Saída	Observação
4	2 2 2 2	A distância Chessboard é $\max(\ dx\ , \ dy\)$. O único pixel objeto é $(2, 2) = 0$; os demais armazenam sua distância mínima até ele.
4	2 1 1 1	
chessboard	2 1 0 1	
0 0 0 0	2 1 1 1	
0 0 0 0		
0 0 1 0		
0 0 0 0		

2.12.4.4 📌 Observações finais

- Como a imagem tem **apenas um objeto de um pixel**, a distância de cada pixel de fundo é simplesmente a distância desse pixel ao único ponto objeto.
- A implementação pode usar força bruta (percorrer todos os pixels da imagem e calcular a distância diretamente), pois L e C são pequenos nos casos de teste.
- Este problema é um aquecimento para a Transformada de Distância geral, que será trabalhada em capítulos posteriores com múltiplos objetos e algoritmos otimizados.

```
%%writefile EP02_04.py
# Código Python
```

```
Overwriting EP02_04.py
```

```
TestSuite("EP02_04.py").run()
```




Figura 2.15: Simulador: Ajuste de Brilho e Contraste

2.12.5 EP02_05 Translação de Imagem

Nesta atividade, você deve implementar o deslocamento espacial de uma imagem. A translação move cada pixel da imagem original para uma nova posição com base em um vetor de deslocamento.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia dois inteiros t_x (deslocamento horizontal) e t_y (deslocamento vertical).
- Leia os valores inteiros da matriz original.
- Calcule a nova posição (x', y') para cada pixel (x, y) original.
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.16 uma simulação deste EP.

Importante:

- **Preenchimento:** Pixels que “entram” na imagem devido ao deslocamento e não possuem correspondente na original devem ser preenchidos com **0** (preto).
- **Descarte:** Pixels que, após a translação, ficarem fora dos limites da matriz ($0 \dots L - 1$ ou $0 \dots C - 1$) devem ser ignorados.
- **Coordenadas:** Considere x como o índice da linha e y como o índice da coluna.

2.12.5.1 Deslocamento Espacial

Transladar uma imagem significa mover todos os seus pontos por uma distância fixa em direções especificadas. Matematicamente, usando coordenadas homogêneas, a operação é descrita como:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Que resulta nas equações simples:

- $x' = x + t_x$
- $y' = y + t_y$

2.12.5.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os inteiros t_x e t_y .

As linhas seguintes contém os elementos da matriz $L \times C$.

Saída:

A matriz resultante com as mesmas dimensões $L \times C$ após o deslocamento.

2.12.5.3 Exemplos

Entrada	Saída	Observação
2 2 1 1 10 20 30 40	0 0 0 10	Deslocamento ($t_x = 1, t_y = 1$): Cada pixel move uma posição para a direita (horizontal) e uma para baixo (vertical). O pixel $(0, 0) = 10$ vai para o destino $(1, 1)$ (canto inferior direito). As posições vazias são preenchidas com 0.
3 3 -1 0 1 2 3 4 5 6 7 8 9	2 3 0 5 6 0 8 9 0	
		Deslocamento ($t_x = -1, t_y = 0$): Cada pixel move uma posição para a esquerda (horizontal). A primeira coluna original (1, 4, 7) é descartada, as demais colunas movem-se para a esquerda, e a última coluna resultante é preenchida com zeros (0).

```
%%writefile EP02_05.py
# Código Python
```

```
Overwriting EP02_05.py
```

```
TestSuite("EP02_05.py").run()
```

2.12.6 EP02_06 Rotação de Imagem

Nesta atividade, você deve implementar a rotação de uma imagem em torno do seu centro geométrico. Esta operação requer o mapeamento de coordenadas e o uso de técnicas de interpolação para determinar os novos valores dos pixels.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia um valor real θ (ângulo em graus) e uma string representando o **método** de interpolação (**nearest** ou **bilinear**).

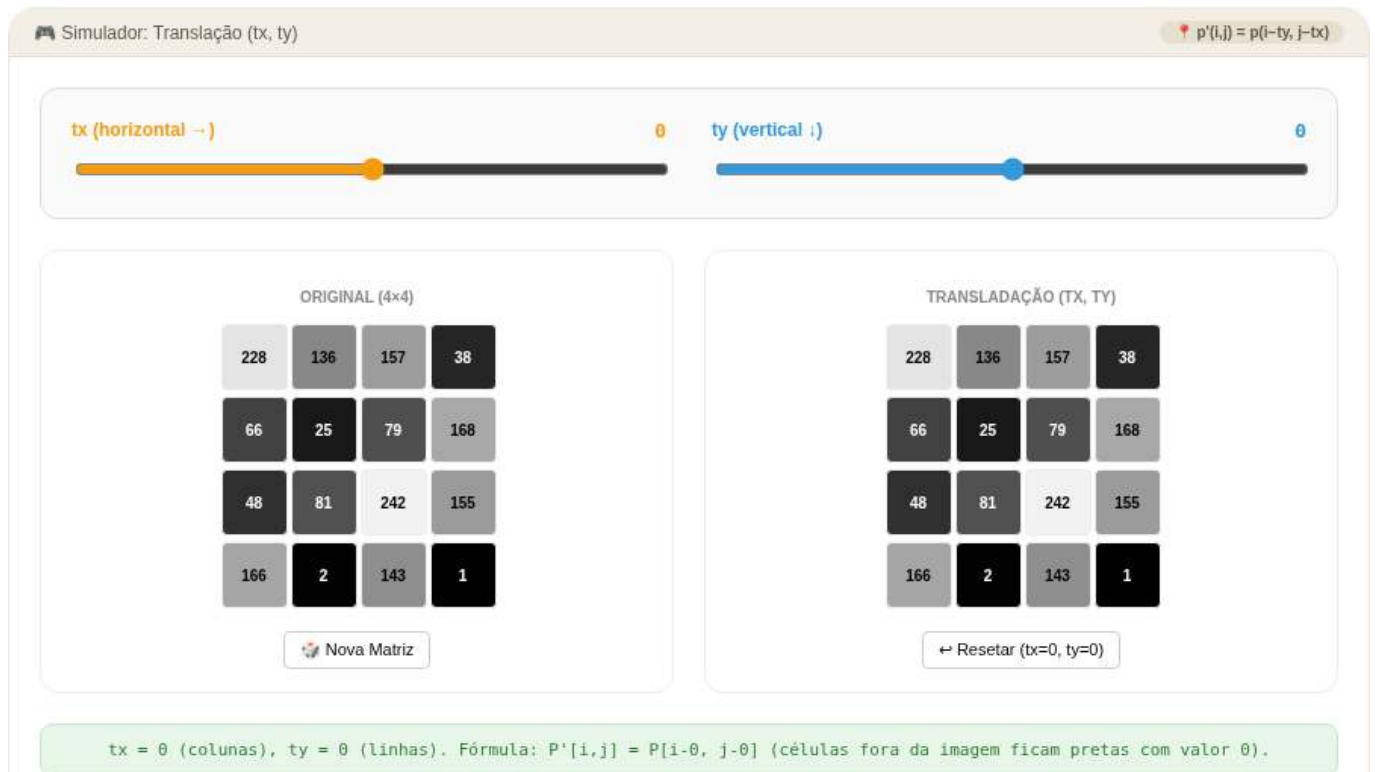


Figura 2.16: Simulador: Translação (tx, ty)

- Leia os valores inteiros da matriz original.
- Realize a rotação em torno do centro da imagem $(L/2, C/2)$.
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.17 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para evitar “buracos” na imagem final, percorra cada pixel (x', y') da imagem de destino e calcule sua posição correspondente (x, y) na imagem original usando a matriz de rotação inversa.
- **Interpolação:**
 - **nearest:** Atribui o valor do pixel mais próximo da coordenada calculada.
 - **bilinear:** Calcula uma média ponderada baseada nos 4 vizinhos mais próximos.
- **Bordas:** Pixels cuja origem (x, y) caia fora dos limites da imagem original devem ser preenchidos com 0.

2.12.6.1 🌸 Transformação por Ângulo

A rotação de um ponto (x, y) em relação à origem por um ângulo θ é dada pela matriz de transformação. Para rotacionar em torno de um centro (x_c, y_c) , primeiro transladamos o centro para a origem, rotacionamos e transladamos de volta:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x_c \\ \sin \theta & \cos \theta & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \\ 1 \end{bmatrix}$$

Dica: Use o mapeamento inverso para garantir que todos os pixels da imagem de saída sejam preenchidos corretamente.

2.12.6.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o ângulo **theta** (em graus) e o método **interp** (**nearest** ou **bilinear**).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz rotacionada com L linhas e C colunas.

2.12.6.3 Exemplos

Entrada	Saída	Observação
2	3 1	Rotação de 90° horário: a coluna 0 vira a linha 0 (de baixo para cima). $(0, 0) = 1 \rightarrow (1, 0)$, $(1, 0) = 3 \rightarrow (0, 0)$, $(0, 1) = 2 \rightarrow (1, 1)$, $(1, 1) = 4 \rightarrow (0, 1)$.
2	4 2	
90 nearest		
1 2		
3 4		
3	0 180 0	Rotação de 45°: o pixel central permanece 255; os vizinhos diretos recebem valor interpolado ≈ 180 por bilinear; os cantos permanecem 0.
3	180 255 180	
45 bilinear	0 180 0	
0 0 0		
0 255 0		
0 0 0		

```
%%writefile EP02_06.py
# Código Python
```

```
Overwriting EP02_06.py
```

```
TestSuite("EP02_06.py").run()
```

2.12.7 EP02_07 Redimensionamento (Escala)

Nesta atividade, você deve implementar o redimensionamento de uma imagem utilizando fatores de escala. Diferente da subamostragem simples, aqui utilizaremos técnicas de interpolação para permitir tanto a ampliação quanto a redução da imagem.

- Leia dois inteiros L e C , representando as dimensões da matriz original.
- Leia dois valores reais s_x (escala nas linhas) e s_y (escala nas colunas).
- Leia uma string representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.

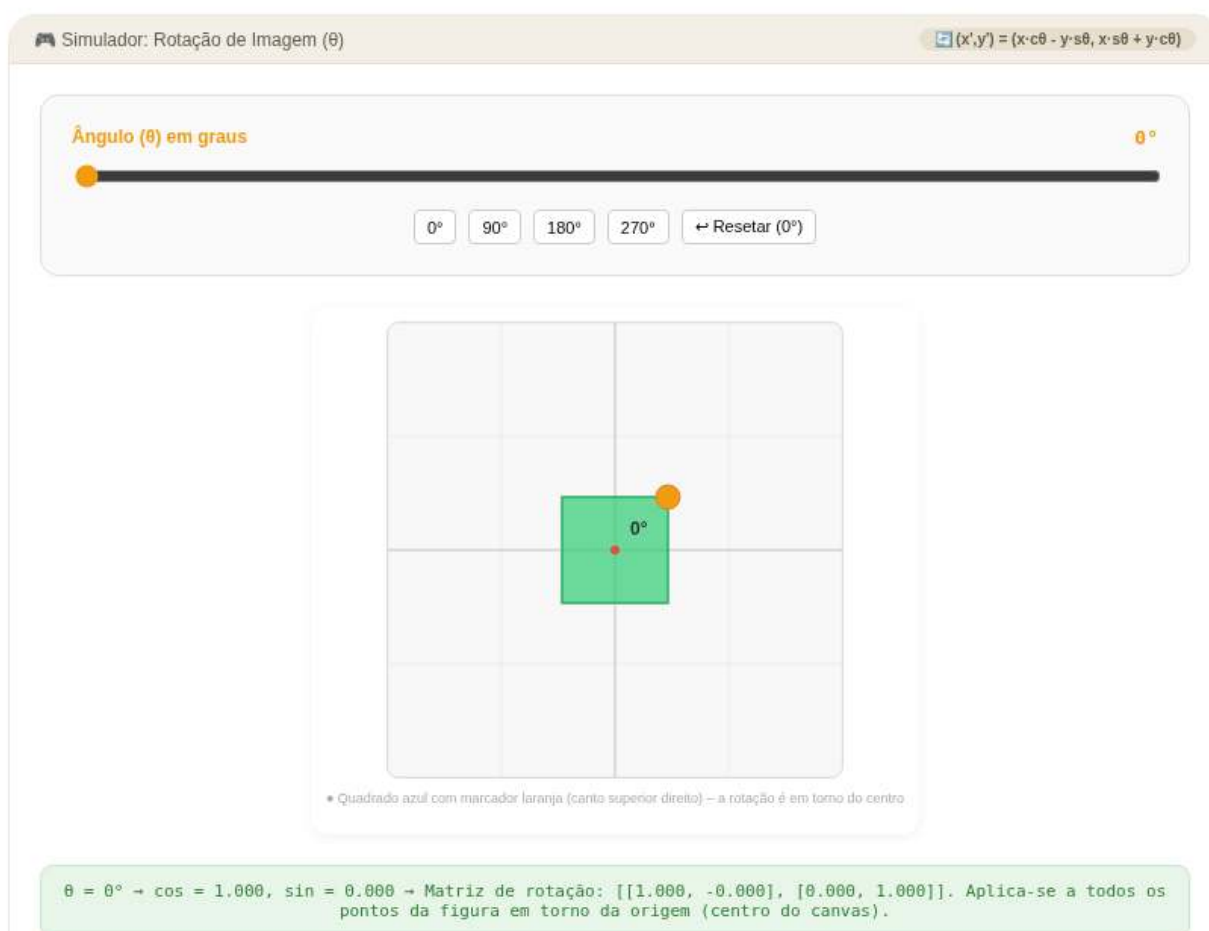


Figura 2.17: Simulador: Rotação (ângulo)

- Calcule as novas dimensões: $L' = \text{round}(L \times s_x)$ e $C' = \text{round}(C \times s_y)$.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.18 uma simulação deste EP.

Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, encontre a posição correspondente na origem usando $(x, y) = (x'/s_x, y'/s_y)$.
- **Interpolação:**
 - **nearest:** Seleciona o valor do pixel mais próximo (arredondamento das coordenadas).
 - **bilinear:** Realiza uma interpolação linear dupla entre os quatro pixels vizinhos mais próximos na imagem original.
 - **Bordas:** Certifique-se de que o mapeamento não tente acessar índices fora do intervalo $[0, L - 1]$ e $[0, C - 1]$.

2.12.7.1 Interpolação para Ampliação/Redução

Redimensionar uma imagem por fatores (s_x, s_y) exige o preenchimento de vazios (na ampliação) ou a fusão de informações (na redução). O método de interpolação define a qualidade visual do resultado:

Método	Funcionamento	Efeito Visual
Nearest	Pega o valor do vizinho mais próximo.	Rápido, mas gera efeito “pixelado” ou blocos.
Bilinear	Média ponderada dos 4 vizinhos (2×2) .	Suaviza a imagem, reduzindo o serrilhamento.

2.12.7.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os fatores s_x e s_y .

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz redimensionada com dimensões $L' \times C'$.

2.12.7.3 Exemplos

Entrada	Saída	Observação
2	1 1 2 2	Ampliação 2×: cada pixel original é replicado em um bloco 2×2. A imagem 2 × 2 vira 4 × 4.
2	1 1 2 2	
2.0 2.0	3 3 4 4	
nearest	3 3 4 4	
1 2		
3 4		Redução 0.5×: a imagem 2 × 2 vira 1 × 1. Com nearest, o único pixel de saída amostra a posição (0,0) = 10.
2	10	
2		
0.5 0.5		
nearest		
10 20		
30 40		

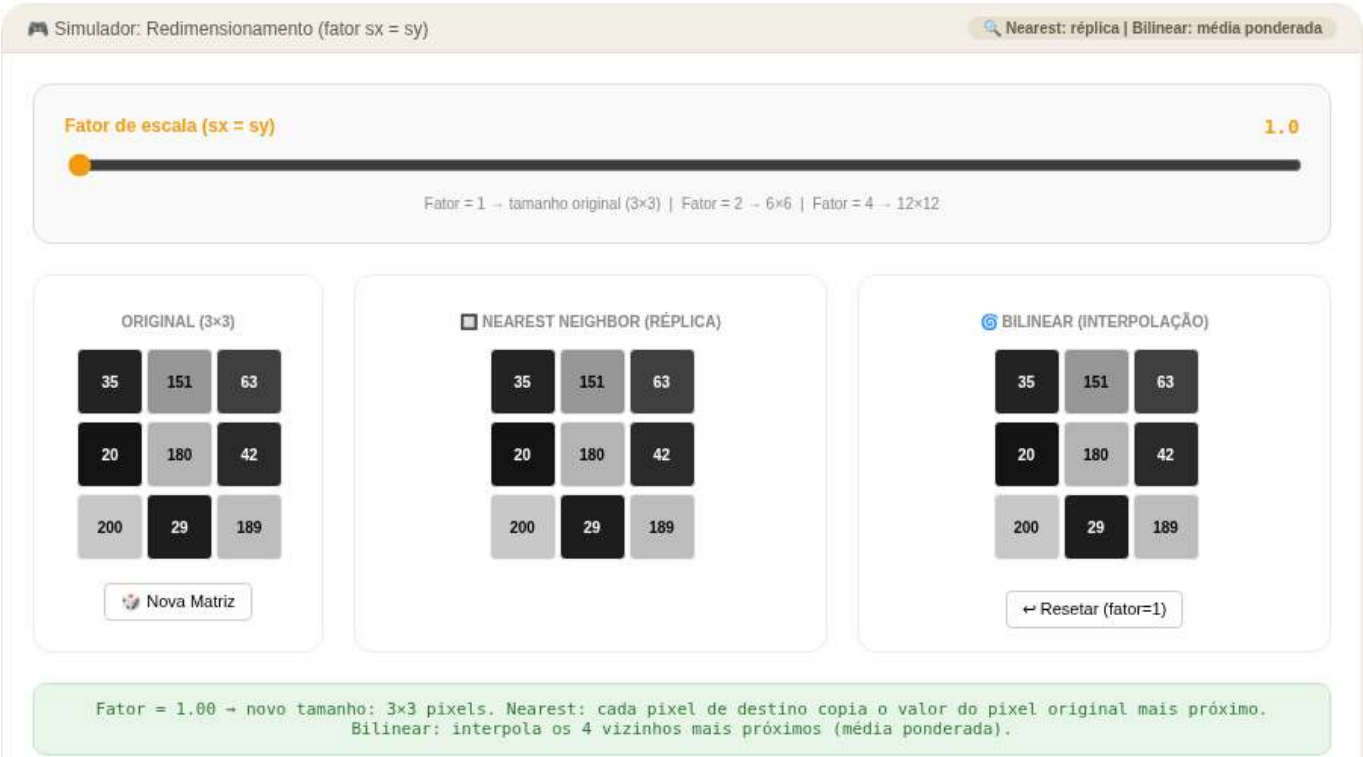


Figura 2.18: Simulador: Redimensionamento (Nearest vs Bilinear)

```
%%writefile EP02_07.py
# Código Python
import numpy as np
from morph import mm

# 1. Leitura das dimensões, fatores e método
l = int(input())
c = int(input())
sx, sy = map(float, input().split())
interp = input().strip()

# 2. Leitura da imagem original
img = mm.readImg(l, c)

# 3. Novas dimensões
```

```

l_new = round(l * sx)
c_new = round(c * sy)

# 4. Redimensionamento usando mm.resize
# cv2.resize usa (largura, altura) = (colunas, linhas)
resultado = mm.resize(img, (c_new, l_new), method=interp)

# 5. Exibição
print(mm.drawImg(resultado))

```

Writing EP02_07.py

TestSuite("EP02_07.py").run()

2.12.8 EP02_08 ✂ Cisalhamento (Shear)

Nesta atividade, você deve implementar a transformação de cisalhamento em uma imagem. O cisalhamento é uma transformação afim que desloca cada ponto em uma direção fixada, por um valor proporcional à sua distância de uma reta paralela a essa direção, resultando em um efeito de inclinação.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia dois valores reais sh_x (cisalhamento horizontal) e sh_y (cisalhamento vertical).
- Leia uma string representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original da imagem (cortando o que ultrapassar os limites).
- Imprima a matriz resultante com as dimensões $L \times C$.
- Ver na Figure 2.19 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, calcule a posição correspondente na origem (x, y) utilizando a matriz de cisalhamento inversa.
- **Preenchimento:** Coordenadas que resultarem em posições fora da matriz original devem ser preenchidas com 0.
- **Coordenadas:** Para fins desta implementação, considere x como o índice da linha e y como o índice da coluna.

2.12.8.1 🧠 Distorção Afim

O cisalhamento altera a geometria da imagem inclinando seus eixos. A relação entre as coordenadas originais (x, y) e as transformadas (x', y') é dada por:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Isso resulta nas equações:

- $x' = x + sh_x \cdot y$
- $y' = y + sh_y \cdot x$

2.12.8.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os fatores `shx` e `shy`.

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as mesmas dimensões $L \times C$.

2.12.8.3 Exemplos

Entrada	Saída	Observação
3	10 20 30	Cisalhamento horizontal: linha i desloca $\lfloor i \cdot 0.5 \rfloor$ pixels. Linha $0 \rightarrow 0\text{px}$, linha $1 \rightarrow 0\text{px}$, linha $2 \rightarrow 1\text{px}$. Os pixels deslocados para fora são descartados e as posições vazias preenchidas com 0.
3	0 40 50	
0.5 0.0	0 0 70	
nearest		
10 20 30		
40 50 60		Cisalhamento vertical: coluna j desloca $\lfloor j \cdot 1.0 \rfloor$ pixels para baixo. Coluna $0 \rightarrow 0\text{px}$ (inalterada), coluna $1 \rightarrow 1\text{px}$: 20 desce para $(1, 1)$ e $(0, 1)$ fica 0.
70 80 90		
2	10 0	
2	30 20	
0.0 1.0		
nearest		
10 20		
30 40		

```
%%writefile EP02_08.py
# Código Python
```

```
Overwriting EP02_08.py
```

```
TestSuite("EP02_08.py").run()
```

2.12.9 EP02_09 Transformação Afim Genérica

Nesta atividade, você deve implementar uma transformação afim arbitrária em uma imagem. Esta operação é a generalização de todas as transformações lineares (escala, rotação, cisalhamento) combinadas com a translação, permitindo manipulações geométricas complexas através de uma única matriz.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia **seis valores reais** (a, b, t_x, c, d, t_y) que compõem a matriz de transformação afim 2×3 .
- Leia uma string representando o **método** de interpolação (`nearest` ou `bilinear`).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original $L \times C$.

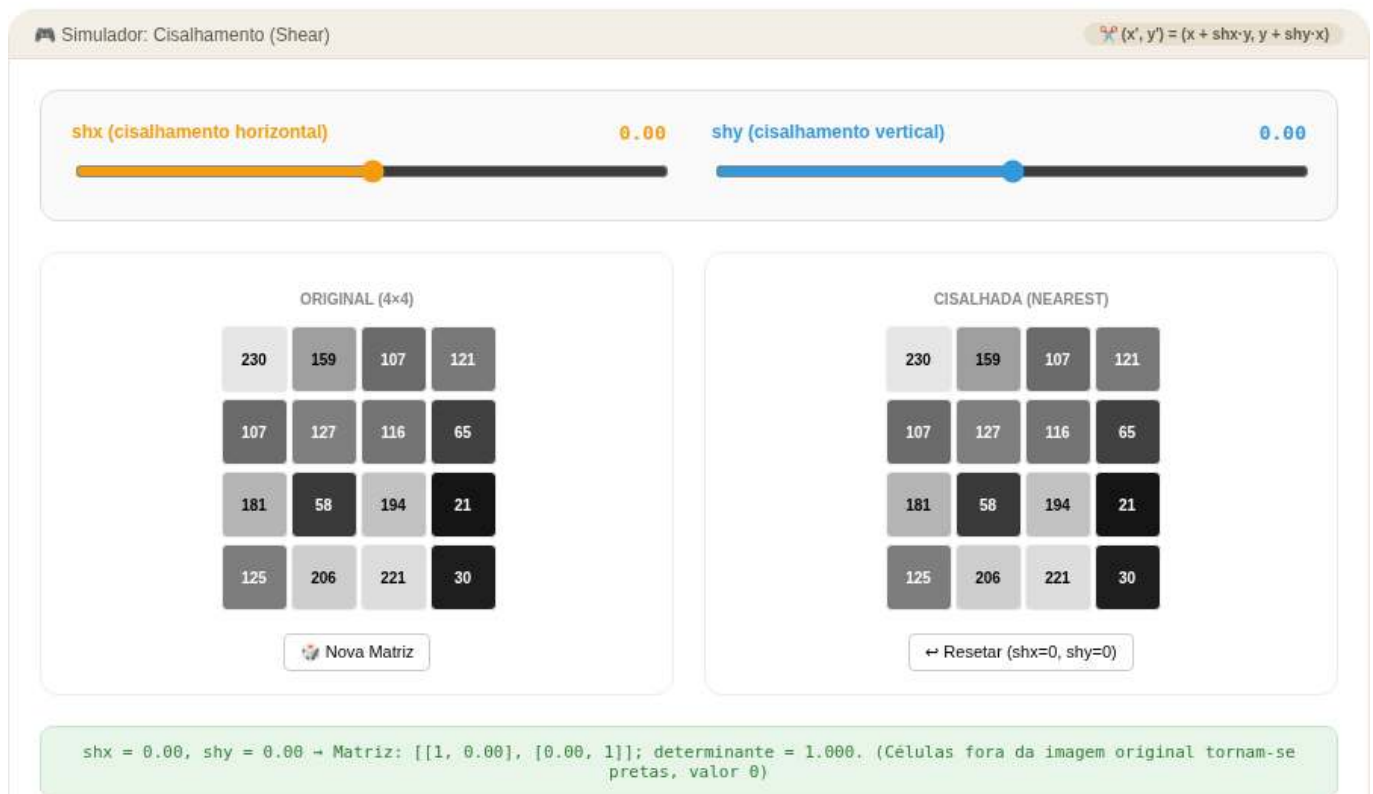


Figura 2.19: Simulador: Cisalhamento (shx, shy)

- Imprima a matriz resultante.
- Ver na Figure 2.20 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para calcular o valor de cada pixel na imagem de destino, você deve utilizar a **inversa** da matriz de transformação afim fornecida para encontrar a coordenada correspondente na imagem original.
- **Preenchimento:** Coordenadas calculadas que caíam fora dos limites $[0, L - 1]$ e $[0, C - 1]$ da imagem original devem resultar em um pixel de valor **0**.
- **Flexibilidade:** Esta implementação deve ser capaz de realizar qualquer uma das tarefas anteriores (translação, rotação, etc.) bastando alterar os parâmetros da matriz.

Dica:

```
flags = cv2.INTER_NEAREST if interp == 'nearest' else \
        cv2.INTER_CUBIC   if interp == 'bicubic'   else \
        cv2.INTER_LANCZOS4 if interp == 'lanczos'   else \
        cv2.INTER_LINEAR

r = cv2.warpAffine(img, M, (C, L), flags=flags)
```

2.12.9.1 🧠 Combinação de Operações

A transformação afim preserva pontos, retas e planos. No processamento de imagens, ela mapeia a posição (x, y) para (x', y') seguindo o sistema:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Ou, de forma compacta em coordenadas homogêneas:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2.12.9.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém seis floats: a b t_x c d t_y .

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as dimensões originais $L \times C$.

2.12.9.3 Exemplos

Entrada	Saída	Observação
2	15 20	Translação fracionária ($t_x = 0.5, t_y = 0.5$): cada pixel de saída (i, j) amostra a posição $(i + 0.5, j + 0.5)$ da entrada via bilinear. Ex: (0,0) interpola os quatro vizinhos $\rightarrow 15$.
2	25 30	
1.0 0.0 0.5 0.0 1.0 0.5		
bilinear		
10 20		
30 40		Escala $2\times$ via matriz afim ($a = 2, d = 2$): cada pixel de saída (i, j) amostra a posição $(2i, 2j)$ da entrada com nearest. Ex: (0,2) $\rightarrow$ (0,4) fora da imagem $\rightarrow$ nearest clipa para (0,2) = 3... aguarda confirmação da lógica de borda.
3	1 1 2	
3	1 1 2	
2.0 0.0 0.0 0.0 2.0 0.0	4 4 5	
nearest		
1 2 3		
4 5 6		
7 8 9		

```
%%writefile EP02_09.py
# Código Python
```

```
Overwriting EP02_09.py
```

```
TestSuite("EP02_09.py").run()
```

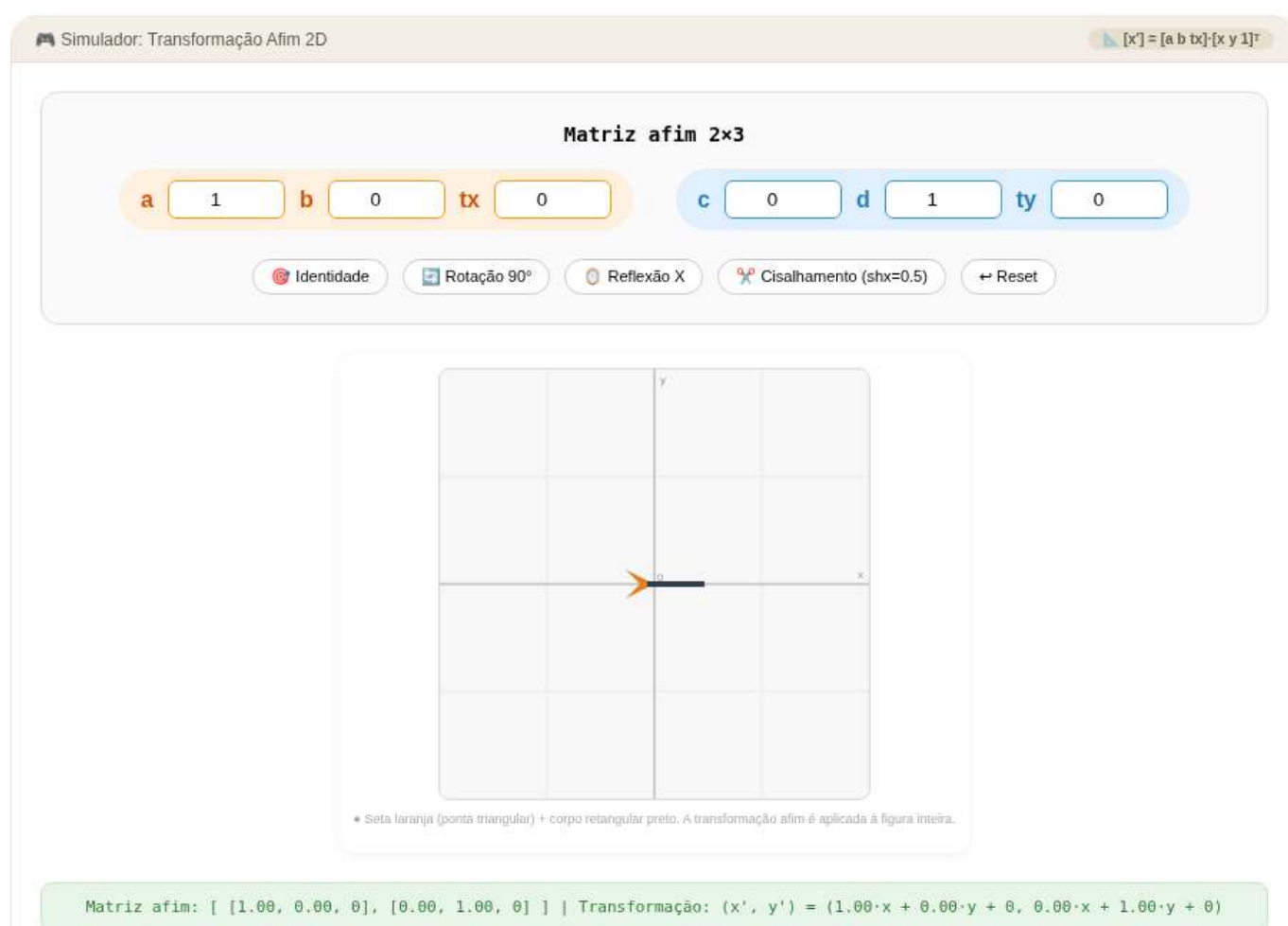


Figura 2.20: Simulador: Transformação Afim (matriz 2×3)

2.12.10 EP02_10 🎯 Correção de Perspectiva (Homografia)

Nesta atividade, você deve implementar a transformação de perspectiva, também conhecida como homografia. Diferente das transformações afins, a perspectiva não preserva o paralelismo, permitindo “retificar” objetos inclinados, como documentos ou placas capturados em ângulos oblíquos.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz original.
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de origem (objeto distorcido).
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de destino (onde o objeto deve ser mapeado).
- Leia os valores da matriz original.
- Calcule a matriz de homografia 3×3 e aplique a transformação.
- Imprima a matriz resultante com as dimensões de saída especificadas.
- Ver na Figure 2.21 uma simulação deste EP.

📌 Importante:

- **Graus de Liberdade:** A homografia possui 8 graus de liberdade (o nono elemento da matriz 3×3 é uma constante de normalização, geralmente 1), exigindo no mínimo 4 pontos correspondentes para ser calculada.
- **Projeção:** Após multiplicar as coordenadas pela matriz, é necessário dividir os resultados x' e y' pela componente homogênea w para retornar ao plano 2D.
- **Uso de Bibliotecas:** Para esta tarefa, você pode utilizar as funções `cv2.getPerspectiveTransform` para obter a matriz e `cv2.warpPerspective` para aplicar a transformação, ou implementar o sistema linear e o mapeamento inverso manualmente para um desafio extra.

```
# Dimensões de saída: bounding box dos pontos destino + 1
w = int(max(pts2[:, 0])) + 1; h = int(max(pts2[:, 1])) + 1
# M = cv2.getPerspectiveTransform(pts1, pts2)
# dst = cv2.warpPerspective(img, M, (w, h))
# ou
dst = mm.perspective_transform(img, pts1, pts2, size=(w, h))
```

2.12.10.1 🧠 Deformação não-afim

Enquanto transformações afins mapeiam paralelogramos em paralelogramos, a homografia mapeia qualquer quadrilátero em outro quadrilátero. Isso é essencial para visão computacional:

Operação	Característica	Aplicação Típica
Homografia	Projeção em plano	Correção de documentos, escaneamento de placas.
Ponto de Fuga	Convergência de linhas	Reconstrução 3D a partir de imagens 2D.
Warping	Deformação de malha	Estabilização de vídeo e panoramas (stitching).

2.12.10.2 📌 Exemplos

Entrada	Saída	Observação
4 4	10 20 30 40	As 4 primeiras linhas após as dimensões são os pontos de origem; as 4 seguintes são os destinos. Com pontos idênticos, a transformação de perspectiva é a identidade e a imagem é preservada.
0 0	50 60 70 80	
3 0	90 100 110 120	
0 3	130 140 150 160	
3 3		
0 0		
3 0		
0 3		
3 3		
10 20 30 40		
50 60 70 80		
90 100 110 120		
130 140 150 160		

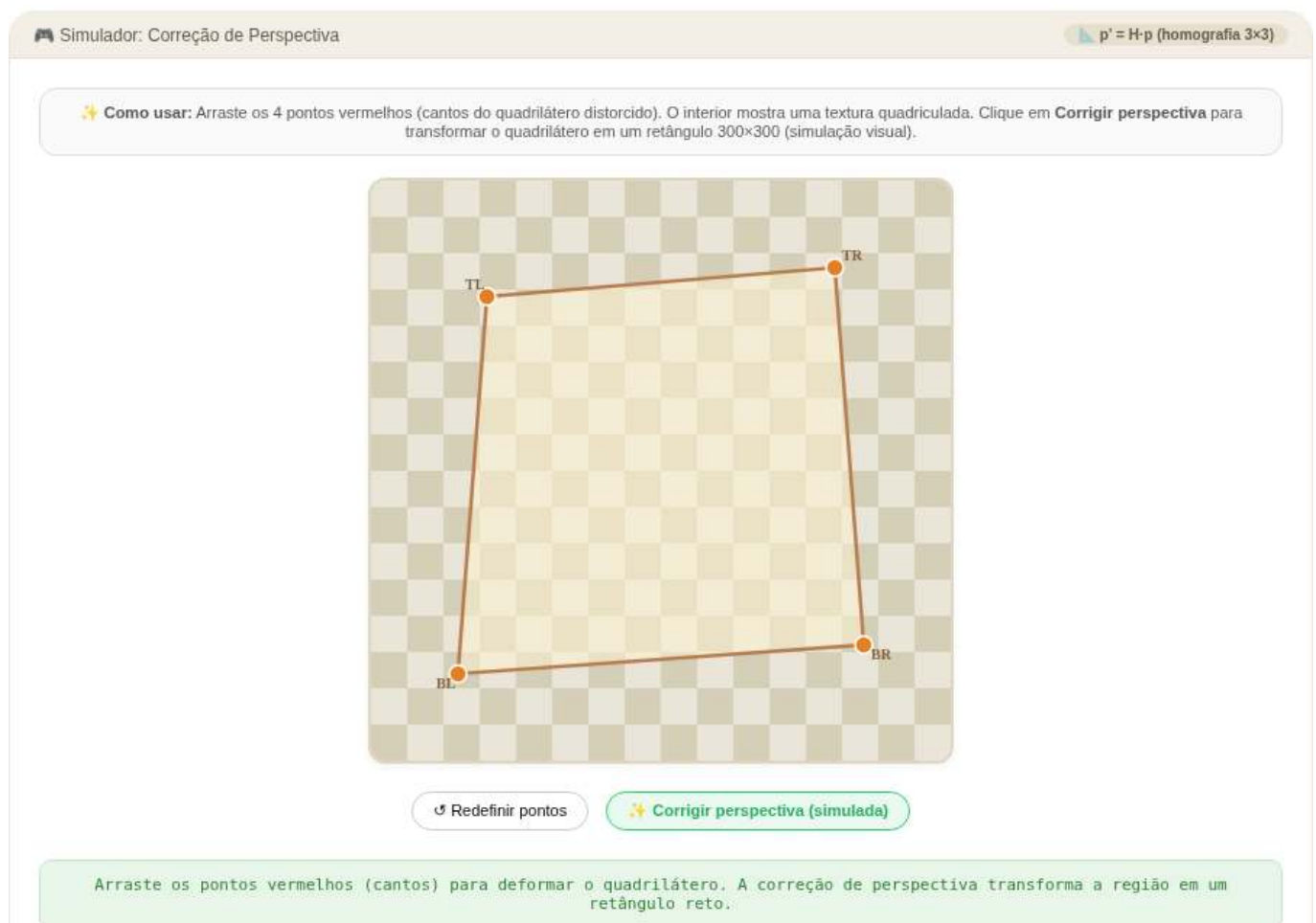


Figura 2.21: Simulador: Correção de Perspectiva (Homografia)

```
%%writefile EP02_10.py
# Código Python
```

```
Overwriting EP02_10.py
```

```
TestSuite("EP02_10.py").run()
```

2.12.11 EP02_11 🎯 Correção de Perspectiva (Homografia) em Imagem Real

Nesta atividade, o objetivo é aplicar a transformação de perspectiva (homografia) para “retificar” um objeto inclinado em uma fotografia real. Você lidará com a imagem de um jornal, onde a grade de um jogo de Sudoku está distorcida devido ao ângulo em que a foto foi capturada.

Seu programa deve ler os parâmetros de entrada do terminal, carregar a imagem, calcular a matriz de homografia 3×3 , aplicar a transformação geométrica e exibir um indicador global de validação.

- Leia dois inteiros **L** e **C**, representando as dimensões de linhas e colunas (altura e largura) que a imagem retificada de saída deve ter.
- Leia **quatro pares de coordenadas** (x, y) via terminal, representando os quatro cantos do quadrilátero de origem (o Sudoku distorcido na imagem original).
- Calcule automaticamente os **quatro pares de coordenadas de destino** utilizando as dimensões L e C fornecidas, mapeando os cantos para as extremidades da nova imagem: $(0, 0)$, $(C - 1, 0)$, $(0, L - 1)$ e $(C - 1, L - 1)$.
- Carregue a imagem local `sudoku.png` e converta-a para tons de cinza (grayscale).
- Calcule a matriz de homografia e aplique a transformação espacial na imagem.
- **Saída:** Calcule e imprima a **soma de todos os pixels** da imagem resultante.

📌 Importante:

- **Arquivo de entrada:** A imagem `sudoku.png` deve estar na mesma pasta do script. O programa deve lê-la diretamente do disco (ex: usando `mm.read("sudoku.png")` ou `cv2.imread()`).
- **Ordem dos Pontos:** Garanta que a leitura dos 4 pontos de origem e a geração dos 4 pontos de destino sigam rigorosamente a mesma ordem dos cantos: Superior-Esquerdo (TL), Superior-Direito (TR), Inferior-Esquerdo (BL) e Inferior-Direito (BR).
- **Dimensões no OpenCV:** Lembre-se que funções como `cv2.warpPerspective` esperam o tamanho da imagem de saída no formato `(largura, altura)`, o que equivale a (C, L) .
- **Interpolação:** Para garantir a consistência matemática da soma dos pixels com o corretor automático, utilize a interpolação bilinear padrão (`flags=cv2.INTER_LINEAR`).
- **Créditos:** A imagem utilizada é “Sudoku en periódico” de Héctor Rodríguez, sob licença **CC BY 2.0**.

2.12.11.1 🧠 Contexto do Problema

A homografia possui 8 graus de liberdade, exigindo no mínimo 4 correspondências de pontos para ser calculada. Ao contrário de transformações afins, ela mapeia qualquer quadrilátero em outro quadrilátero, permitindo que linhas que convergem para pontos de fuga voltem a ser paralelas:

Operação	Característica	Aplicação Típica
Homografia	Projeção entre planos	Retificação de documentos, escaneamento de placas e QR Codes.
Mapeamento Inverso	Varredura do destino para a origem	Evita “buracos” ou pixels vazios na imagem final retificada.
Warping	Reamostragem espacial	Correção de distorção de lentes e montagem de panoramas (<i>stitching</i>).

2.12.11.2 📌 Exemplos

Entrada	Saída	Observação
500 500 100 120 420 95 80 440 450 460	32982820	As duas primeiras entradas são as dimensões de saída (L e C). As 4 linhas seguintes são as coordenadas (x, y) dos cantos do Sudoku na imagem original + PAD. A saída é a soma total dos pixels da imagem retificada.
200 200 100 120 420 95 80 440 450 460	5277150	

2.12.11.3 Aquisição da imagem do sudoku e conversão para níveis de cinza

A Figure 2.22 mostra a leitura da imagem original seguida da conversão para tons de cinza e redimensionamento para uma matriz de 500×500 pixels, preparando os dados para a etapa seguinte. A correção de perspectiva, aplicada na Figure 2.23 via matriz de homografia, elimina as deformações causadas pelo ângulo da câmera e produz uma visão frontal e regular da grade do Sudoku.



Figura 2.22: Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).

```
import cv2
import numpy as np

# --- 1. Carrega a imagem salva (sudoku.png) ---
img = mm.read("sudoku.png")          # BGR, 500x500

# --- 2. Padding para não cortar vértices ---
```

```

PAD = 60
img_pad = cv2.copyMakeBorder(
    img, PAD, PAD, PAD, PAD,
    cv2.BORDER_CONSTANT, value=[255, 255, 255]
)

# --- 3. Pontos de origem (cantos da grade na imagem expandida) ---
pts1 = np.float32([
    [100, 160], # TL
    [390, 45],  # TR
    [200, 580], # BL
    [570, 420], # BR
])
#      W      H

# --- 4. Pontos de destino (vista frontal 500x500) ---
SIZE = 500
pts2 = np.float32([
    [0, 0],
    [SIZE, 0],
    [0, SIZE],
    [SIZE, SIZE],
])

# --- 5. Homografia e retificação ---
img_rect = mm.perspective_transform(img_pad, pts1, pts2, size=(SIZE, SIZE))

# --- 6. Exibição ---
mm.show(
    [img_pad, img_rect],
    titles=["Original (com padding)", "Vista frontal retificada"],
    cols=2, figsize=(10, 6), axis=True
)

```

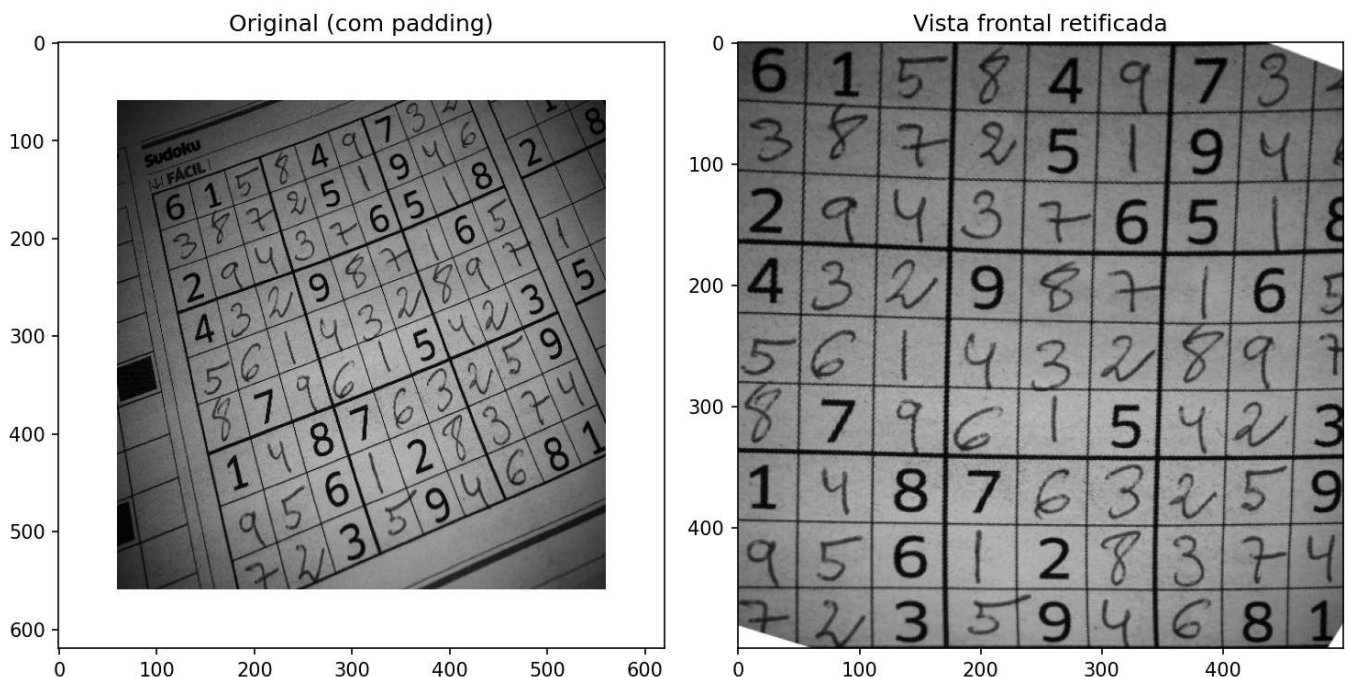


Figura 2.23: Correção de perspectiva: original e vista frontal retificada.

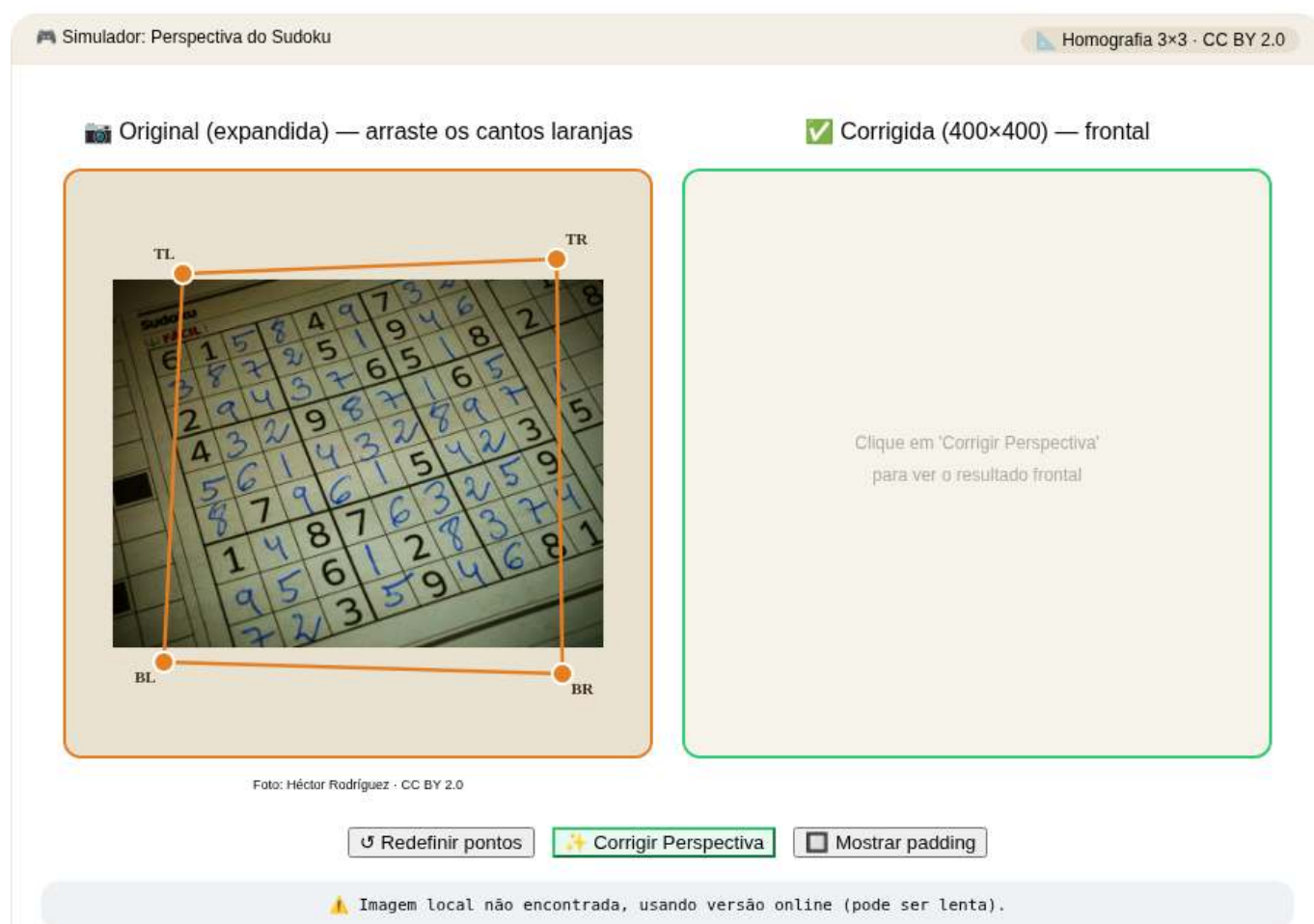


Figura 2.24: Simulador: correção de perspectiva do Sudoku.


```
%%writefile EP02_11.py  
# Código Python
```

```
Overwriting EP02_11.py
```

```
TestSuite("EP02_11.py").run()
```

Capítulo 3

Operações Espaciais: Intensidade, Histograma e Filtragem



Este capítulo aprofunda o processamento de imagens no domínio espacial, partindo da manipulação direta de pixels e histogramas para o realce de contraste, até a aplicação de filtros locais por convolução para suavização, redução de ruído e detecção de bordas. O objetivo é desenvolver a intuição matemática e computacional que sustenta grande parte dos algoritmos modernos de Visão Computacional.

3.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Manipular intensidade e pixels:** Executar operações aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para combinação e seleção de regiões de interesse (ROI), e aplicar *alpha blending* (`mm.blend`) para fusão ponderada de imagens;
- **Processar histogramas:** Interpretar o histograma como diagnóstico tonal, aplicar equalização global (`mm.equalize`) e adaptativa (CLAHE), e realizar especificação de histograma para transferência de perfil tonal entre imagens;
- **Compreender fundamentos espaciais:** Entender vizinhança, *padding* de borda, e a diferença entre correlação cruzada (`mm.conv`) e convolução — incluindo por que kernels assimétricos como Sobel produzem resultados distintos nas duas operações;
- **Aplicar filtragem de suavização:** Utilizar o filtro de média (`mm.conv` com kernel uniforme) e o filtro Gaussiano (`cv2.GaussianBlur`) para redução de ruído, compreendendo a vantagem da ponderação radial e da separabilidade Gaussiana;
- **Aplicar filtragem de realce:** Usar o Laplaciano (w_4 e w_8) para realce isotrópico de bordas, o operador de Sobel para gradiente direcional (G_x , G_y , magnitude e direção), e o *Unsharp Masking* para amplificação controlada de alta frequência pelo parâmetro k ;
- **Utilizar filtros de ordem:** Aplicar o filtro da mediana (`cv2.medianBlur`) para remoção eficaz de ruído sal e pimenta, compreendendo por que sua natureza não linear e robustez a outliers o tornam superior aos filtros lineares nesse cenário;
- **Resolver problemas práticos:** Encadear técnicas em pipelines de pré-processamento (ex.: CLAHE → Gaussiano → Canny) e utilizar as funções da biblioteca `morph.py` (`mm.conv`, `mm.histImg`, `mm.equalize`, `mm.drawImageKernel`) para análise e visualização didática de cada etapa.

3.2 Operações em Nível de Intensidade

O nível mais elementar de processamento de imagens atua diretamente sobre os valores dos pixels, sem considerar vizinhança. Essas operações — chamadas de **transformações de ponto** (*point operations*) — são as mais rápidas computacionalmente e formam a base para técnicas mais complexas.

Formalmente, uma transformação de ponto pode ser descrita como:

$$g(x, y) = T[f(x, y)] \quad (3.1)$$

onde $f(x, y)$ é a imagem de entrada, $g(x, y)$ é a saída e T é uma função aplicada a cada pixel individualmente.

3.2.1 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão:{version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.1).

Como objeto de estudo ao longo deste capítulo, utilizaremos as imagens de vida selvagem apresentadas nas Figure 3.1 e Figure 3.3. A partir delas, exploraremos operações espaciais sobre intensidade, histogramas e filtragem, analisando seus efeitos no realce, na suavização, na redução de ruído e na detecção de bordas, de modo a compreender os fundamentos matemáticos e computacionais do PDI.

```
import os

# Fonte : https://commons.wikimedia.org/wiki/File:Carlos_Guilherme_Rodrigues_(76515283).jpeg
# Mapa : https://www.google.com/maps?q=-26.204734,28.226624

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Carlos_Guilherme_Rodrigues_%2876515283%29.jpeg"
url = f"{base}/9/9b/{arquivo}"
caminho = "imagens/mandrill-exif.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_color = np.array(img_obj)
img_gray = mm.gray(img_color)

# 2. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Tipo NumPy: {type(img_color)} | Dimensões [y,x,c]: {img_color.shape}")

# 3. Exibição
mm.show(img_color, scale=90)
```

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (960, 540)
 Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (540, 960, 3)



Figura 3.1: *Mandrill* (*Mandrillus sphinx*) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).

3.2.2 Operações Aritméticas

Operações aritméticas entre imagens são amplamente usadas em PDI para combinar, comparar ou realçar informações. A **subtração de imagens** é especialmente poderosa para detectar diferenças entre dois quadros — por exemplo, na remoção de fundo estático em câmeras de vigilância:

$$g(x, y) = f_1(x, y) - f_2(x, y) \quad (3.2)$$

A **adição saturada** limita o resultado ao intervalo $[0, 255]$: valores acima de 255 são fixados em 255, evitando o *overflow* silencioso do tipo `uint8` (ex.: $200 + 100 = 44$ em vez de 300). A **subtração saturada** aplica o mesmo princípio pelo lado inferior: valores negativos são fixados em 0.

⚠ Saturação e *overflow*

Operações aritméticas em `uint8` sofrem *overflow* silencioso: $200 + 100 = 44$ (não 300). As funções `mm.addm` e `mm.subm` usam `cv2.add` e `cv2.subtract`, que realizam saturação automática. Para o *blending*, converta para `float32` antes de operar e aplique `np.clip(..., 0, 255).astype(np.uint8)` ao final, pois a operação envolve pesos fracionários.

A Figure 3.2 demonstra adição de uma constante (clareamento) e subtração de uma constante (escurecimento com saturação em 0).

```
fundo = 60

img_add = mm.addm(img_gray, fundo)
img_sub = mm.subm(img_gray, fundo)

mm.show(
    [img_gray, img_add, img_sub],
    titles=["Original", "addm (+60)", "subm (-60)"],
    cols=3
)
```

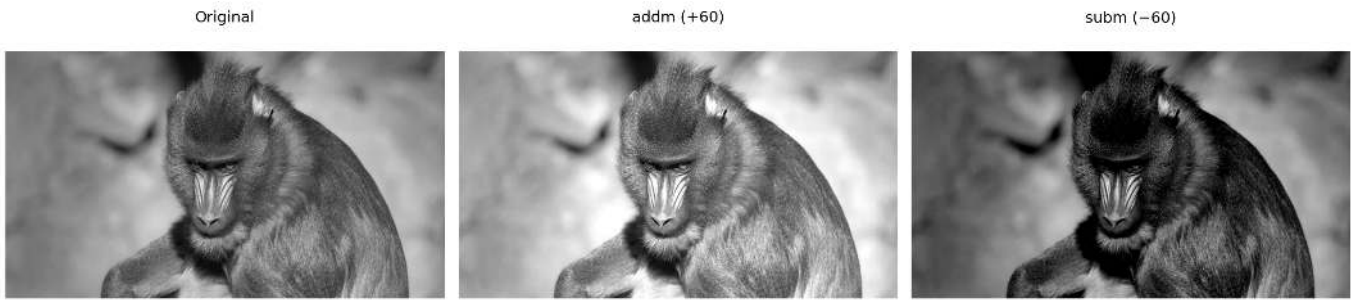


Figura 3.2: Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).

3.2.3 Mistura Ponderada (*Alpha Blending*)

A **mistura ponderada** (*alpha blending*) combina duas imagens utilizando pesos complementares α e $(1 - \alpha)$:

$$g(x, y) = \alpha f_1(x, y) + (1 - \alpha) f_2(x, y), \quad \alpha \in [0, 1] \quad (3.3)$$

Quando $\alpha = 1$, obtém-se apenas a imagem f_1 ; quando $\alpha = 0$, apenas f_2 . Valores intermediários produzem uma transição suave entre ambas, sendo amplamente utilizados em composição de imagens, sobreposição de camadas, marcas d'água e efeitos de fusão visual.

Para que a combinação produza um resultado coerente, é necessário alinhar previamente as regiões de interesse das imagens. Na Figure 3.4, utiliza-se um recorte do rosto do leopardo:

```
leo = img_gray_leo[250:-300, 100:-200]
```

e um recorte central da região facial do mandril:

```
mandrill = img_gray[100:400, 380:530]
```

As duas regiões são escolhidas de forma que olhos e estrutura facial permaneçam aproximadamente alinhados. Em seguida, o recorte do leopardo é redimensionado para coincidir com as dimensões do mandril antes da aplicação da mistura ponderada.

A operação é realizada em `float32` para evitar problemas de *overflow* durante as operações aritméticas, seguida de *clipping* e conversão final para `uint8`.

```
import os
from PIL.ExifTags import TAGS

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Leopard_%28Panthera pardus%29_portrait.jpg"
url = f"{base}/9/92/{arquivo}"
caminho = "imagens/leopardo.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj)
img_gray_leo = mm.gray(img_numpy)

# 2. Extração e conversão de GPS (Tag 34853)
```

```

exif = img_obj._getexif()
if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(
        -(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
        else (dms[0]+dms[1]/60+dms[2]/3600)
    )
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy)

```

```

GPS Decimal: -19.140200, 23.814872
Maps: https://www.google.com/maps/search/?api=1&query=-19.1402,23.814872222222224

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (1242, 1324)
Pillow (0,0): (192, 134, 86)
Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (1324, 1242, 3)
NumPy [0,0]: [192 134 86]

```



Figura 3.3: Retrato de um leopardo (*Panthera pardus*) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).

```

leo = img_gray_leo[250:-300, 100:-200]
mandrill = img_gray[100:400, 380:530]
img_leopardo = mm.resize(leo, (mandrill.shape[1], mandrill.shape[0]), method='bilinear')

alphas = [1.0, 0.8, 0.6, 0.4, 0.2, 0.0]
imgs_blend = []

```



```
titles_blend = []

for a in alphas:
    imgs_blend.append(mm.blend(mandrill, img_leopardo, alpha=a))
    titles_blend.append(f"={a:.1f}")

mm.show(imgs_blend, titles=titles_blend, cols=6, figsize=(18, 16))
```

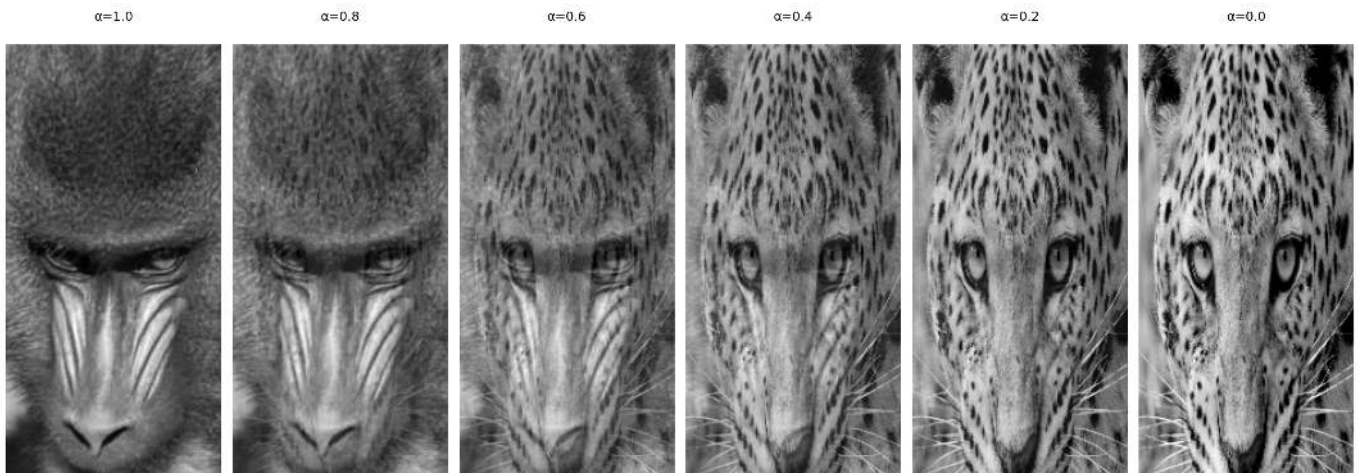


Figura 3.4: *Alpha blending* entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.

3.2.4 Operações Lógicas e Máscaras Bit a Bit

As operações lógicas bit a bit (AND, OR e NOT) atuam diretamente sobre os bits de cada pixel e são a base para criação e aplicação de **máscaras** (*masks*) — imagens binárias com apenas 0 (preto) e 255 (branco) usadas para isolar **Regiões de Interesse (ROI)**.

O comportamento de cada operação decorre da representação binária do 255 (11111111) e do 0 (00000000):

- **AND** com a máscara: onde $m = 255$, os bits originais são preservados; onde $m = 0$, o pixel é zerado. Resultado: recorte da ROI.
- **OR** com a máscara: onde $m = 255$, o pixel é forçado a branco; onde $m = 0$, o valor original é mantido. Resultado: iluminação da ROI.
- **NOT** (sem máscara): inverte todos os bits ($g = 255 - f$), produzindo o negativo fotográfico da imagem.

A Figure 3.5 ilustra as três operações aplicadas à imagem do mandrill com uma máscara circular.

```
import cv2
h, w = img_gray.shape

# Máscara circular centrada na imagem
mask_circ = np.zeros((h, w), dtype=np.uint8)
cv2.circle(mask_circ, (w//2, h//2), min(h, w)//3 - 10, 255, -1)

# Operações via morph
img_not = mm.bnot(img_gray) # NOT: negativo fotográfico
img_and = mm.band(img_gray, mask_circ) # img_gray & mask_circ: preserva apenas a ROI circular
img_or = mm.bor(img_gray, mask_circ) # img_gray | mask_circ: ilumina a região da máscara

mm.show(
    [img_gray, img_and, img_or, img_not],
```

```
titles=["Original", "AND (ROI circular)", "OR (ilumina ROI)", "NOT (negativo)"],
cols=4
)
```



Figura 3.5: Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).

3.3 Histograma de Imagens

O **histograma** de uma imagem em tons de cinza é uma função discreta que descreve a distribuição de frequências das intensidades:

$$h(r_k) = n_k, \quad k = 0, 1, \dots, L - 1 \quad (3.5)$$

onde r_k é o k -ésimo nível de intensidade, n_k é o número de pixels com essa intensidade e L é o total de níveis (tipicamente 256 para 8 bits). O histograma normalizado estima a probabilidade de cada nível:

$$p(r_k) = \frac{n_k}{MN} \quad (3.6)$$

onde MN é o total de pixels. Por ser uma **estatística global**, o histograma não carrega informação posicional, mas revela características essenciais como brilho médio, contraste e distribuição tonal. Na prática, `mm.hist(img)` retorna o vetor de contagens $h(r_k)$, que serve tanto para visualização (via `mm.histImg`) quanto para cálculos como **função de distribuição acumulada (CDF)** e equalização.

i Interpretação do Histograma

- **Estreito à esquerda:** imagem subexposta (escura).
- **Estreito à direita:** imagem superexposta (clara).
- **Concentrado no centro:** baixo contraste.
- **Distribuído por toda a faixa:** alto contraste, boa utilização dos tons disponíveis.

A Figure 3.6 apresenta o histograma da imagem do mandrill, bem como versões escurecida (`mm.subm`) e clareada (`mm.addm`). Observa-se o deslocamento da distribuição de intensidades para a esquerda e para a direita, respectivamente. Note que o intervalo representado no eixo x não corresponde necessariamente a toda a faixa de 0 a 255.

```
img_dark = mm.subm(img_gray, 80)
img_high = mm.addm(img_gray, 80)

images = [img_gray, img_dark, img_high]
titles = ["Original", "Escurecida (-80)", "Clareada (+80)"]

def hist_title(img, t):
    H = mm.hist(img)
    n0 = int(H[0])
    n255 = int(H[255]) if len(H) > 255 else 0
    return f"Histograma - {t}\n0:{n0:},px 255:{n255:},px"
```

```

mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [hist_title(img, t) for img, t in zip(images, titles)],
    rows=2, cols=3,
    figsize=(14, 8)
)

```

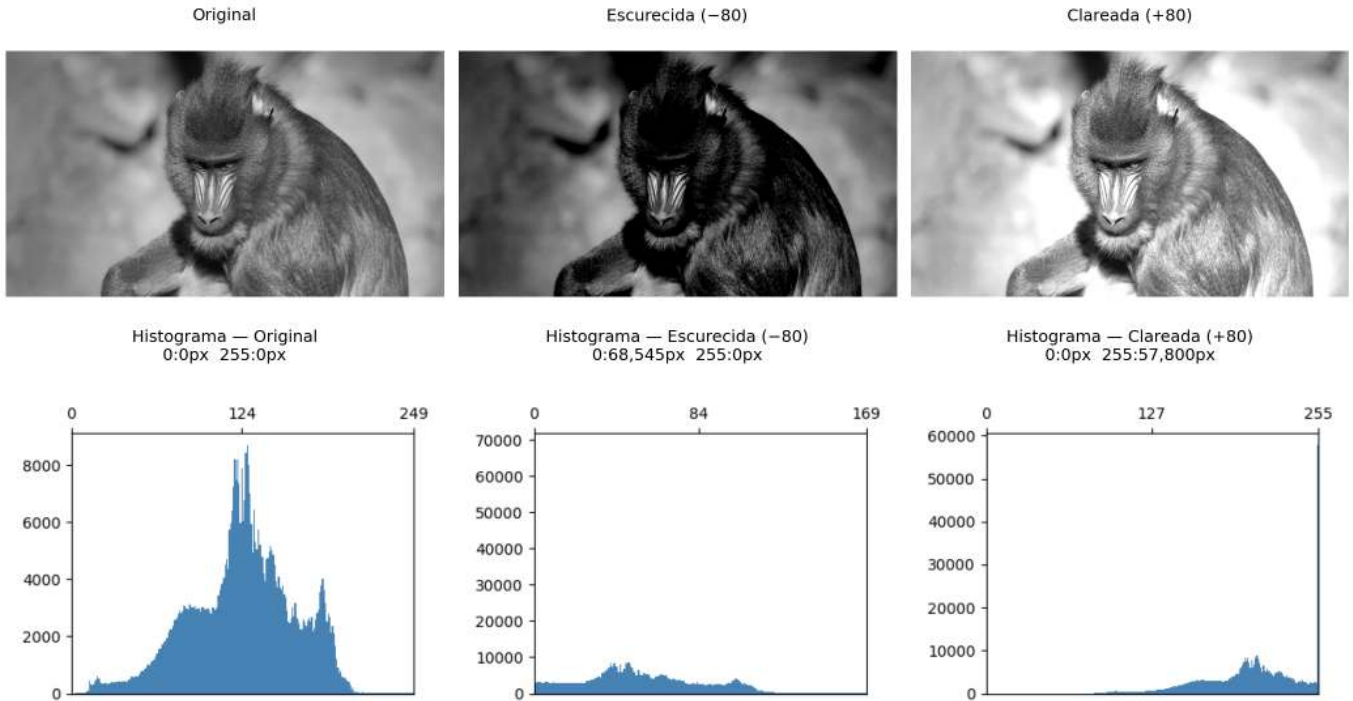


Figura 3.6: Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).

3.3.1 Equalização de Histograma

A **equalização de histograma** redistribui as intensidades para que o histograma resultante seja o mais uniforme possível. O mapeamento é dado pela **função de distribuição acumulada (CDF)**:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p(r_j) = \frac{L - 1}{MN} \sum_{j=0}^k n_j \quad (3.7)$$

A transformação é monotônica: níveis frequentes recebem intervalos maiores no domínio de saída (maior separação → mais contraste), enquanto níveis raros são comprimidos.

O algoritmo completo, em cinco etapas, é apresentado na Table 3.1.

Tabela 3.1: Algoritmo de equalização de histograma.

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow$ número de pixels com intensidade k , $k = 0 \dots L - 1$
2	Probabilidade	$p[k] \leftarrow h[k]/MN$
3	CDF	$\text{cdf}[k] \leftarrow \sum_{j=0}^k p[j]$ (soma acumulada)
4	Look-Up Table (mapeamento)	$\text{lut}[k] \leftarrow \text{round}(\text{cdf}[k] \times (L - 1))$

Etapa	Operação	Fórmula
5	Aplicação	$g[i, j] \leftarrow \text{lut}[f[i, j]]$ (para todo pixel)

Note na Figure 3.7 que a equalização **redistribui** os tons existentes para posições mais espaçadas na faixa $[0, L - 1]$, mas não cria novos tons — a imagem equalizada continua com exatamente 3 tons distintos, agora em $\{1, 5, 7\}$ em vez de $\{2, 3, 4\}$.

```
img5 = np.array([[3, 4, 2, 3, 4],
                 [4, 3, 3, 4, 3],
                 [2, 3, 4, 3, 2],
                 [3, 4, 3, 2, 3],
                 [4, 3, 2, 3, 4]], dtype=np.uint8)
L = 8 # 3 bits: níveis 0..7

# 1. Histograma com tamanho garantido até L
h = mm.hist(img5, 3) # B=3 → 2³=8 níveis, tamanho garantido

p = h / h.sum() # 2. Probabilidade de cada nível p[k] = h[k] / MN

cdf = np.cumsum(p) # 3. CDF normalizada ou
# cdf = np.zeros(len(p))
# cdf[0] = p[0]
# for k in range(1, len(p)):
#     cdf[k] = cdf[k-1] + p[k] # cdf[k] = Σ p[j], j=0..k

lut = np.round(cdf * (L - 1)).astype(np.uint8) # 4. LUT: mapeamento para [0, L-1]

img5_eq = lut[img5] # 5. Aplica LUT pixel a pixel ou, equivalente a:
# l, c = img5.shape
# img5_eq = np.zeros((l, c), dtype=np.uint8)
# for i in range(l):
#     for j in range(c):
#         img5_eq[i, j] = lut[img5[i, j]] # g[i,j] = lut[f[i,j]]

print("Imagem original (5×5, 3 bits):")
print(img5)
print()
header = f"{'k':>3} {'h[k]':>6} {'p[k]':>7} {'CDF[k]':>8} {'lut[k]':>7}"
print(header)
print("-" * len(header))
for k in range(L):
    print(f"{k:>3} {h[k]:>6} {p[k]:>7.4f} {cdf[k]:>8.4f} {lut[k]:>7}")
print()
print("Imagem equalizada (5×5):")
print(img5_eq)

# Conta tons distintos
tons_orig = len(np.unique(img5))
tons_eq = len(np.unique(img5_eq))

mm.show(
    [img5, img5_eq, mm.histImg(img5, L-1), mm.histImg(img5_eq, L-1)],
    titles=[f"Original ({tons_orig} tons: {sorted(np.unique(img5).tolist())})",
           f"Equalizada ({tons_eq} tons: {sorted(np.unique(img5_eq).tolist())})",
           "Histograma - Original",
           "Histograma - Equalizada"],
    rows=2, cols=2, figsize=(5, 4), dpi=100
```

Imagem original (5×5, 3 bits):

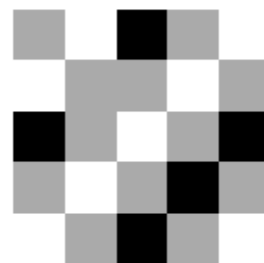
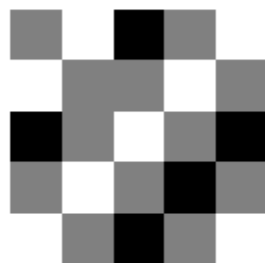
```
[[3 4 2 3 4]
 [4 3 3 4 3]
 [2 3 4 3 2]
 [3 4 3 2 3]
 [4 3 2 3 4]]
```

k	h[k]	p[k]	CDF[k]	lut[k]
0	0	0.0000	0.0000	0
1	0	0.0000	0.0000	0
2	5	0.2000	0.2000	1
3	12	0.4800	0.6800	5
4	8	0.3200	1.0000	7
5	0	0.0000	1.0000	7
6	0	0.0000	1.0000	7
7	0	0.0000	1.0000	7

Imagem equalizada (5×5):

```
[[5 7 1 5 7]
 [7 5 5 7 5]
 [1 5 7 5 1]
 [5 7 5 1 5]
 [7 5 1 5 7]]
```

Original (3 tons: [2, 3, 4]) Equalizada (3 tons: [1, 5, 7])



Histograma — Original

Histograma — Equalizada

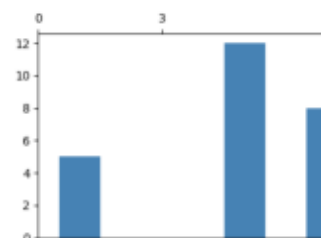
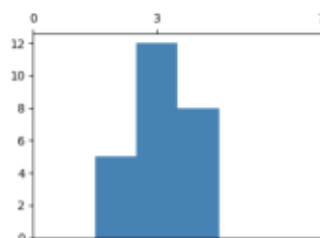


Figura 3.7: Equalização de histograma em imagem 5×5 com 3 bits ($L=8$): imagem original com tons concentrados em $\{2,3,4\}$, imagem equalizada com tons redistribuídos para $\{1,5,7\}$, e os respectivos histogramas evidenciando o espalhamento das frequências.

Limitação: a equalização global pode super-realçar ruídos e produzir contraste excessivo em regiões homogêneas. O **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) reduz esse problema ao aplicar a equalização em blocos locais (*tiles*) e limitar a altura dos picos do histograma antes da equalização.

A Figure 3.8 compara a imagem original, a equalização global via `mm.equalize` e o CLAHE do OpenCV, exibindo também os histogramas resultantes. Diferentemente da equalização global, que utiliza uma única transformação baseada na CDF de toda a imagem, o CLAHE adapta o contraste a cada região, sendo particularmente útil em imagens com iluminação não uniforme.

No exemplo, foi utilizado `clipLimit=2.0` e `tileGridSize=(32,32)`. O parâmetro `clipLimit` define o quanto os picos do histograma local podem crescer antes de serem cortados (*clipped*). No OpenCV, esse valor é um fator relativo: o limite real é aproximadamente calculado como `clipLimit × (número de pixels do bloco / número de níveis de cinza)`. Por exemplo, em um bloco com 4096 pixels e uma imagem de 8 bits (256 níveis de cinza), a frequência média por nível é $4096/256 = 16$. Assim, `clipLimit=2.0` permite picos de aproximadamente $2 \times 16 = 32$ ocorrências antes do corte. As ocorrências excedentes não são descartadas: elas são redistribuídas entre os demais níveis de cinza do histograma, reduzindo a concentração excessiva em poucos níveis e evitando uma amplificação exagerada do contraste local. Valores menores limitam mais o contraste e reduzem a amplificação de ruído, enquanto valores maiores permitem um realce mais intenso, mas podem introduzir artefatos.

- `clipLimit=1.0`: realce suave e conservador;
- `clipLimit=2.0`: bom equilíbrio entre contraste e naturalidade;
- `clipLimit=4.0`: maior destaque para detalhes locais;
- `clipLimit=8.0`: contraste agressivo, com possível amplificação de ruído.

Assim, o CLAHE costuma produzir resultados mais naturais do que a equalização global, especialmente em imagens com sombras, reflexos ou iluminação desigual.

```
img_eq    = mm.equalize(img_gray)
clahe     = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(32, 32))
img_clahe = clahe.apply(img_gray)

images = [img_gray, img_eq, img_clahe]
titles = ["Original", "mm.equalize (CDF)", "CLAHE"]

mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [f"Histograma - {t}" for t in titles],
    rows=2, cols=3,
    figsize=(14, 8)
)
```

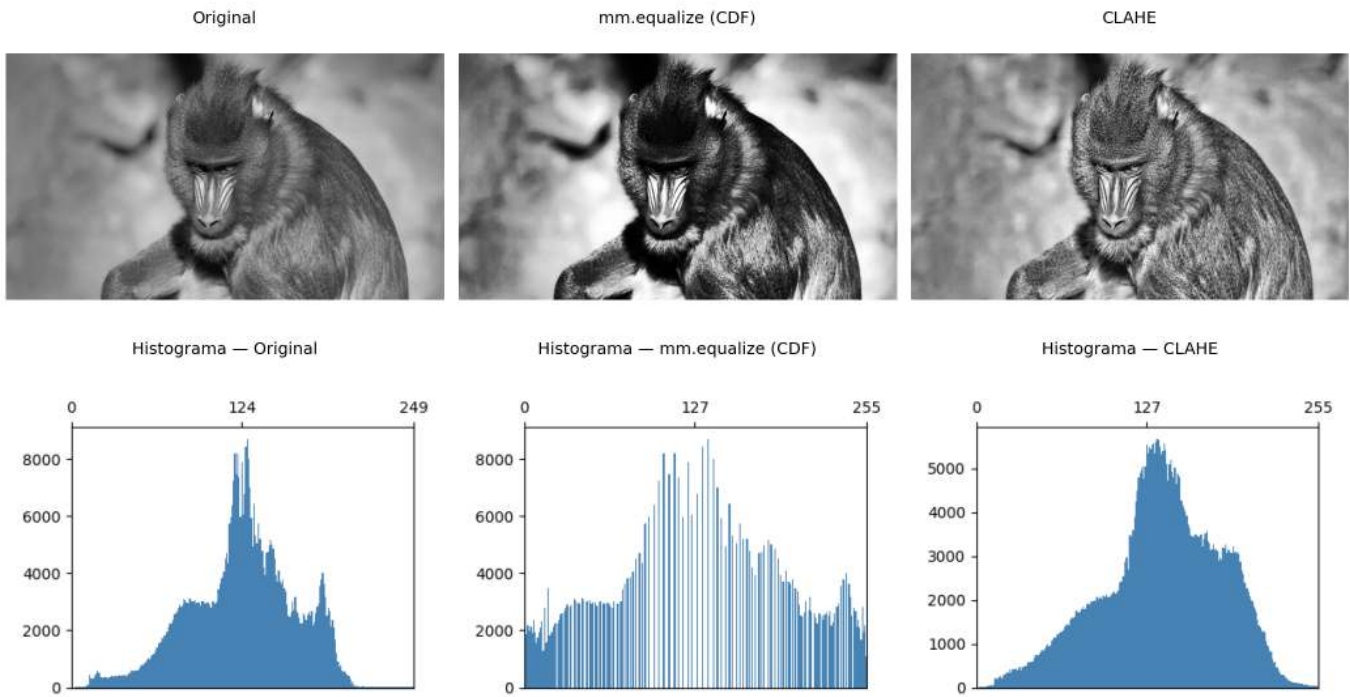



Figura 3.8: Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.

3.3.2 Especificação de Histograma

Enquanto a equalização impõe uma distribuição uniforme, a **especificação de histograma** (*histogram matching*) permite que o histograma da imagem de saída siga uma distribuição **arbitrária** — por exemplo, o histograma de outra imagem de referência.

O procedimento envolve três etapas:

1. Calcular a CDF da imagem de entrada: $P_r(r_k)$.
2. Calcular a CDF da imagem de referência: $P_z(z_k)$.
3. Para cada nível r_k , encontrar o nível z que minimiza $|P_z(z) - P_r(r_k)|$.

$$T(r_k) = \arg \min_z |P_z(z) - P_r(r_k)| \quad (3.8)$$

Na Figure 3.9, transferimos o perfil tonal do leopardo (Figure 3.3) para a imagem do mandrill — uma aplicação direta do conceito visto no *blending*: em vez de fundir pixels, aqui fundimos distribuições tonais.

```
img_leop_gray = mm.gray(img_numpy)

def hist_specify(src, ref):
    """Mapeia src para o perfil tonal de ref via CDF."""
    cdf_src = np.cumsum(mm.hist(src) / src.size)
    cdf_ref = np.cumsum(mm.hist(ref) / ref.size)
    lut = np.array([np.argmin(np.abs(cdf_ref - v)) for v in cdf_src], dtype=np.uint8)
    return lut[src]

img_spec = hist_specify(img_gray, img_leop_gray)

images = [img_gray, img_leop_gray, img_spec]
titles = ["Mandrill (original)", "Leopardo (referência)", "Mandrill → perfil do leopardo"]

mm.show()
```

```

images + [mm.histImg(img) for img in images],
titles=titles + [f"Histograma - {t}" for t in titles],
rows=2, cols=3,
figsize=(12, 9)
)

```

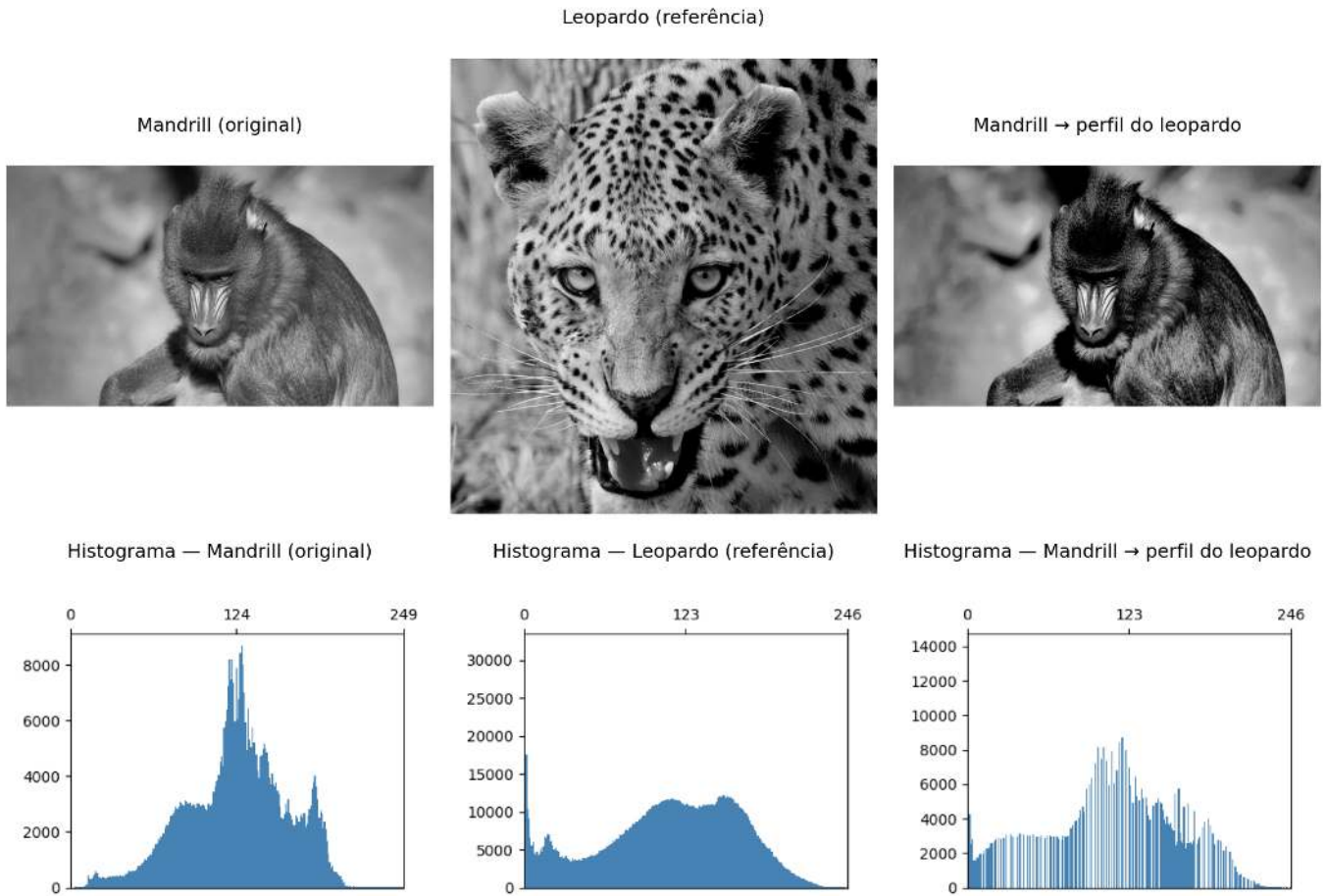


Figura 3.9: Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.

3.4 Fundamentos Espaciais: Vizinhança, Convolução e *Kernels*

As operações de filtragem espacial não atuam em um único pixel isolado, mas em uma **vizinhança** ao seu redor. Para isso, utiliza-se uma pequena matriz de coeficientes denominada **kernel** (ou máscara), que percorre toda a imagem por meio de uma **janela deslizante** (*sliding window*).

As janelas mais comuns são 3×3 , 5×5 e 7×7 . Em uma janela 3×3 , por exemplo, o pixel central é processado juntamente com seus oito vizinhos imediatos. Em cada posição da janela, os valores dos pixels são combinados com os coeficientes do *kernel*, produzindo um novo valor para o pixel central.

3.4.1 Vizinhança

Considere uma janela 3×3 centrada no pixel (x, y) :

$$\begin{bmatrix} (x-1, y-1) & (x, y-1) & (x+1, y-1) \\ (x-1, y) & (x, y) & (x+1, y) \\ (x-1, y+1) & (x, y+1) & (x+1, y+1) \end{bmatrix} \quad (3.9)$$

De forma geral, uma janela de tamanho $(2a + 1) \times (2b + 1)$ abrange todos os pixels situados até a posições na horizontal e até b posições na vertical em relação ao pixel central. Assim, uma janela 3×3 corresponde a $a = b = 1$, uma janela 5×5 a $a = b = 2$, e assim por diante.

Matematicamente, a vizinhança é definida por

$$\mathcal{V}(x, y) = \{(x + s, y + t) : -a \leq s \leq a, -b \leq t \leq b\} \quad (3.10)$$

3.4.2 Tratamento de Bordas

Pixels próximos às bordas possuem parte de sua vizinhança fora da imagem. Para aplicar filtros nessas regiões, é necessário definir como os valores externos serão obtidos. As estratégias mais comuns são:

- **Zero-padding** (BORDER_CONSTANT): completa a região externa com zeros.
- **Replicação** (BORDER_REPLICATE): repete os valores da borda.
- **Reflexão** (BORDER_REFLECT_101): espelha os pixels vizinhos à borda.

O OpenCV utiliza, por padrão, BORDER_REFLECT_101, pois essa estratégia preserva melhor a continuidade dos níveis de cinza e reduz artefatos introduzidos pelo tratamento das bordas.

O exemplo a seguir compara os três métodos usando uma matriz 3×3 . Observe como cada estratégia preenche os pixels externos necessários para que filtros 3×3 possam ser aplicados também nos cantos da imagem.

```
import cv2
import numpy as np

img = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
], dtype=np.uint8)

bordas = {
    "CONSTANT (zero-padding)": cv2.BORDER_CONSTANT,
    "REPLICATE": cv2.BORDER_REPLICATE,
    "REFLECT_101": cv2.BORDER_REFLECT_101
}

print("Imagem original:")
print(mm.drawImg(img))

for k in [3, 5]:
    b = k // 2

    print(f"=== Kernel {k}x{k} ===")

    for nome, tipo in bordas.items():
        pad = cv2.copyMakeBorder(
            img, b, b, b, b,
            tipo,
            value=0
        )

        print(f"{nome}")
        print(mm.drawImg(pad))
```

```
Imagem original:
1 2 3
4 5 6
7 8 9
```

```

=== Kernel 3x3 ===
CONSTANT (zero-padding)
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0

REPLICATE
1 1 2 3 3
1 1 2 3 3
4 4 5 6 6
7 7 8 9 9
7 7 8 9 9

REFLECT_101
5 4 5 6 5
2 1 2 3 2
5 4 5 6 5
8 7 8 9 8
5 4 5 6 5

=== Kernel 5x5 ===
CONSTANT (zero-padding)
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 1 2 3 0 0
0 0 4 5 6 0 0
0 0 7 8 9 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

REPLICATE
1 1 1 2 3 3 3
1 1 1 2 3 3 3
1 1 1 2 3 3 3
4 4 4 5 6 6 6
7 7 7 8 9 9 9
7 7 7 8 9 9 9
7 7 7 8 9 9 9

REFLECT_101
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1
6 5 4 5 6 5 4
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1

```

Observe que o resultado de um filtro pode variar significativamente conforme o tratamento adotado para as bordas da imagem.

Na biblioteca didática `morph.py`, os algoritmos implementados diretamente em Python (sem utilizar OpenCV ou *scikit-image*) não utilizam *padding*. Para cada pixel, em geral, os elementos da vizinhança são examinados individualmente, e apenas os vizinhos cujas coordenadas pertencem ao domínio da imagem participam do cálculo. Assim, nas regiões próximas às bordas, a vizinhança efetiva pode conter menos pixels do que a janela especificada.

3.4.3 Correlação e Convolução

Existem dois mecanismos matematicamente relacionados:

Correlação cruzada (*cross-correlation*) — o *kernel* é aplicado diretamente:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (3.11)$$

Convolução bidimensional — o *kernel* é rotacionado 180° antes da aplicação:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (3.12)$$

Para *kernels* simétricos (Gaussiano, Laplaciano, média) as duas operações produzem resultados idênticos. Para *kernels* assimétricos (Sobel, Prewitt) a diferença é significativa.

3.4.4 Correlação vs. Convolução no OpenCV

`cv2.filter2D` implementa **correlação**. Para convolução verdadeira com *kernel* assimétrico, rotacione o *kernel* 180° (`np.rot90(w, 2)`) antes de passar para `filter2D`.

3.4.4.1 Correlação (*OpenCV*: `cv2.filter2D`)

```
import cv2
import numpy as np

# imagem 4x4
img = np.array([
    [ 1,  2,  3,  4],
    [ 5,  6,  7,  8],
    [ 9, 10, 11, 12],
    [13, 14, 15, 16]
], dtype=np.float32)

# kernel assimétrico
w = np.array([
    [0, 1, 2],
    [0, 0, 0],
    [0, 0, 0]
], dtype=np.float32)

corr = cv2.filter2D(
    img, -1, w, # -1 indica que o tipo da saída é o mesmo da entrada
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel:")
print(mm.drawImg(w))

print("Resultado da correlação:")
print(mm.drawImg(corr))
```

Imagem original:

1	2	3	4
---	---	---	---

```

5   6   7   8
9  10  11  12
13  14  15  16

```

Kernel:

```

0   1   2
0   0   0
0   0   0

```

Resultado da correlação:

```

0   0   0   0
5   8  11   4
17  20  23   8
29  32  35  12

```

O método `cv2.filter2D` realiza correlação, isto é, aplica o *kernel* exatamente na orientação fornecida.

3.4.4.2 Convolução

```

import cv2
import numpy as np

# mesmo kernel
w_conv = np.rot90(w, 2)

conv = cv2.filter2D(
    img, -1, w_conv,
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel original:")
print(mm.drawImg(w))

print("Kernel rotacionado 180°:")
print(mm.drawImg(w_conv))

print("Resultado da convolução:")
print(mm.drawImg(conv))

```

Imagem original:

```

1   2   3   4
5   6   7   8
9  10  11  12
13  14  15  16

```

Kernel original:

```

0   1   2
0   0   0
0   0   0

```

Kernel rotacionado 180°:

```

0   0   0
0   0   0
2   1   0

```

Resultado da convolução:

```

5  16  19  22

```


9	28	31	34
13	40	43	46
0	0	0	0

A convolução utiliza o *kernel* rotacionado em 180°. Por isso, para reproduzir a definição matemática de convolução usando `cv2.filter2D`, é necessário aplicar previamente `np.rot90(w, 2)`.

Quando o *kernel* é simétrico (por exemplo, filtros de média ou Gaussianos), correlação e convolução produzem o mesmo resultado. A diferença aparece apenas para *kernels* assimétricos, como o exemplo apresentado.

3.4.5 O Papel do *Kernel*

Os coeficientes do *kernel* determinam completamente o efeito produzido pelo filtro, conforme resumido na Table 3.2.

Tabela 3.2: Interpretação típica dos coeficientes do *kernel*.

Característica	Efeito típico
Coeficientes positivos com soma 1	Suavização (<i>passa-baixa</i>)
Soma igual a 0, com valores positivos e negativos	Detecção de bordas (<i>passa-alta</i>)
Coeficiente central positivo dominante e vizinhos negativos	Realce de nitidez
Coeficientes assimétricos	Gradiente direcional

Exemplos:

Suavização:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Detecção de bordas:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Realce de nitidez:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Gradiente direcional (Sobel):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

A Figure 3.10 demonstra o mecanismo passo a passo: para cada posição da janela, multiplica-se cada coeficiente do *kernel* pelo pixel correspondente da vizinhança e somam-se os produtos obtidos. O resultado é exatamente o valor definido pela Equation 3.11 para aquela posição da imagem. Embora as imagens produzidas por `mm.conv0` e `cv2.filter2D` (ou `mm.conv`) sejam visualmente muito semelhantes, a implementação baseada em OpenCV é milhares de vezes mais rápida, como mostrado a seguir.

⚠ Desempenho: laços Python vs. operações vetorizadas

A função `mm.conv0` implementa a correlação diretamente em Python por meio de laços aninhados. Embora essa abordagem seja adequada para fins didáticos, ela executa um grande número de operações e torna-se lenta para imagens maiores.

Já `mm.conv` utiliza `cv2.filter2D`, implementado em C++ e otimizado para operações matriciais. No exemplo apresentado, a versão vetorizada foi mais de **3000 vezes mais rápida** que a implementação didática, produzindo um resultado visualmente equivalente.

As diferenças numéricas observadas concentram-se principalmente nas bordas da imagem. Em `mm.conv0`, os pixels da borda permanecem inalterados, enquanto `mm.conv` utiliza uma estratégia de reflexão das bordas (`cv2.BORDER_REFLECT_101`, padrão do `cv2.filter2D`).

Por isso, `mm.conv0` deve ser utilizado para compreender o algoritmo, enquanto `mm.conv` é a opção recomendada para aplicações práticas.

```
import time

w_mean = np.ones((3,3), dtype=np.float32) / 9.0
img_gray = img_leop_gray
# Demonstração numérica em patch 5x5
patch = img_gray[250:255, 250:255].astype(np.float32)
roi = patch[0:3, 0:3]
print("Patch 5x5 (intensidades):")
print(patch.astype(np.int32))
print(f"\nKernel 3x3 de média:\n{w_mean}")
print(f"\nCorrelação no pixel central [1,1]: \
      {(w_mean * roi).sum():.1f} (original: {patch[1,1]:.0f})")

# Comparação de tempo na imagem completa
t0 = time.perf_counter()
img_conv0 = mm.conv0(img_gray, w_mean)
t_conv0 = time.perf_counter() - t0

t0 = time.perf_counter()
img_conv = mm.conv(img_gray, w_mean)
t_conv = time.perf_counter() - t0

diff = np.abs(img_conv0.astype(int) - img_conv.astype(int)).max()
print(f"\nTempos na imagem {img_gray.shape[0]}x{img_gray.shape[1]}:")
print(f"  mm.conv0 (laços Python): {t_conv0:.3f} s")
print(f"  mm.conv (cv2.filter2D): {t_conv:.5f} s")
print(f"  Aceleração: {t_conv0/t_conv:.0f}x mais rápido")
print(f"  Diferença máxima: {diff} (bordas com padding diferente)")

mm.show(
    [img_gray, img_conv0, img_conv],
    titles=["Original", f"conv0 ({t_conv0:.2f}s)", f"conv ({t_conv:.4f}s)"],
    cols=3
)
```

Patch 5x5 (intensidades):

```
[[ 91  95 108 116 114]
 [106 107 108 111 114]
 [102 103 107 112 116]
 [ 83  90 111 125 123]
 [ 79  88 110 126 124]]
```

Kernel 3x3 de média:

```
[[0.11111111 0.11111111 0.11111111]]
```

```
[0.11111111 0.11111111 0.11111111]
[0.11111111 0.11111111 0.11111111]]
```

Correlação no pixel central [1,1]: 103.0 (original: 107)

Tempos na imagem 1324x1242:

mm.conv0 (laços Python): 5.963 s

mm.conv (cv2.filter2D): 0.00213 s

Aceleração: 2796× mais rápido

Diferença máxima: 39 (bordas com padding diferente)

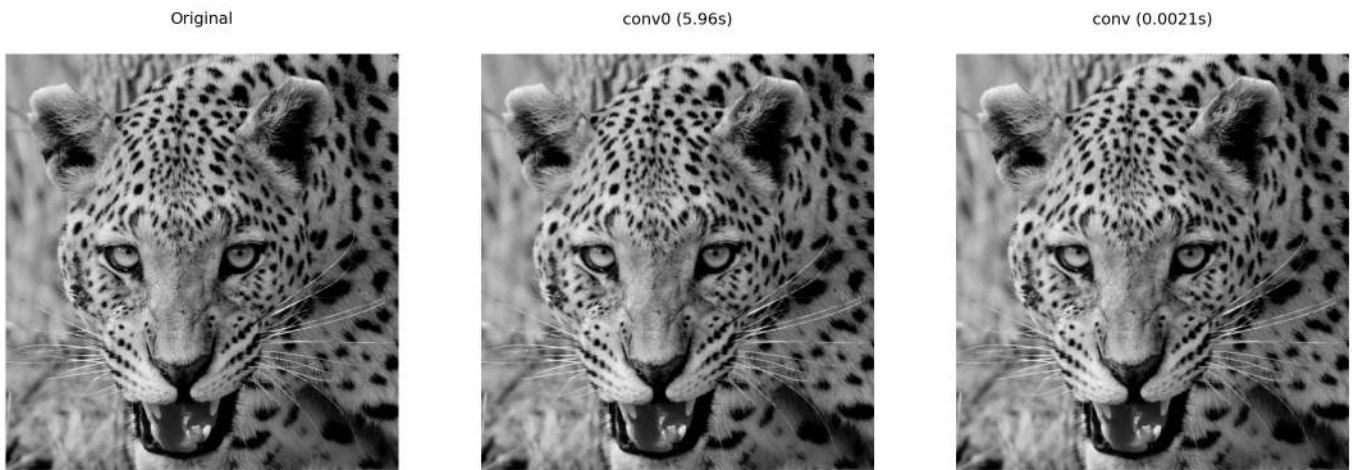


Figura 3.10: Correlação com *kernel* de média 3×3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.

3.4.6 Exemplo Numérico: Correlação Passo a Passo

Para tornar concreto o mecanismo da Equation 3.11, considere o *kernel* de média 3×3 ($a = b = 1$, todos os coeficientes = $1/9 \approx 0,111$) aplicado ao *patch* 5×5 extraído da imagem do leopardo. A Figure 3.11 exhibe o *patch* com grade e destaca em amarelo a janela 3×3 centrada no pixel [1,1]:

```
patch = img_gray[250:255, 250:255]
B      = np.ones((3,3), dtype=np.uint8)

print("Patch 5x5 (intensidades):")
print(mm.drawImg(patch))

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch 5x5 (intensidades):
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

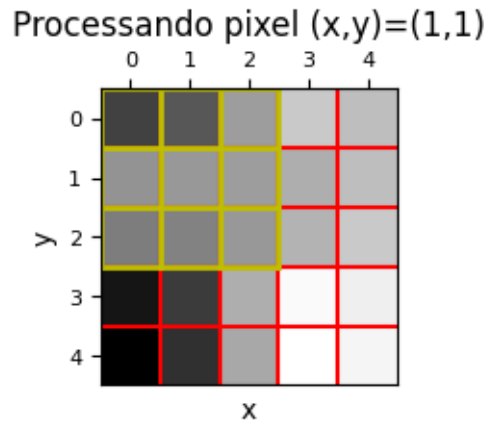


Figura 3.11: *Patch* 5×5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3×3 centrada no pixel [1,1] onde a correlação será calculada.

Para ilustrar o cálculo da correlação, considere o pixel na posição [1, 1] do *patch* 5×5 mostrado na Figure 3.11. Essa posição foi escolhida apenas por conveniência didática, pois possui uma vizinhança 3×3 completa ao seu redor.

Os valores dessa vizinhança correspondem à submatriz superior esquerda do *patch*:

$$\text{vizinhança} = \begin{bmatrix} 91 & 95 & 108 \\ 106 & 107 & 108 \\ 102 & 103 & 107 \end{bmatrix}$$

Aplicando a Equation 3.11 com o kernel de média:

$$g[1, 1] = \frac{91 + 95 + 108 + 106 + 107 + 108 + 102 + 103 + 107}{9} = \frac{927}{9} = 103$$

O resultado (103) é ligeiramente menor que o valor original do pixel central (107), pois a média incorpora vizinhos de menor intensidade, produzindo o efeito de suavização. Na prática, o algoritmo inicia o processamento em [0, 0] e repete esse mesmo cálculo para cada posição da imagem, deslocando a janela até cobrir todo o domínio.

3.5 Filtragem Espacial de Suavização

Os filtros de suavização (*smoothing filters*) atenuam variações bruscas de intensidade, reduzindo ruído e detalhes de alta frequência. São filtros **passa-baixa** — preservam as componentes de baixa frequência (estruturas grandes) e atenuam as de alta frequência (ruído, bordas).

3.5.1 Filtro de Média (*Box Filter*)

O filtro de média utiliza um *kernel* uniforme de tamanho $n \times n$, onde todos os coeficientes valem $1/n^2$:

$$w_{\text{média}} = \frac{1}{n^2} \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}_{n \times n} \quad (3.13)$$

Cada pixel de saída é a média aritmética dos n^2 pixels de sua vizinhança. Note que a soma dos coeficientes é sempre 1 — o brilho médio da imagem é preservado. *Kernels* maiores produzem suavização mais agressiva, mas borram progressivamente as bordas.

A Figure 3.12 mostra o efeito do filtro de média com *kernels* 3×3 , 7×7 e 15×15 sobre um detalhe da imagem do leopardo. Os resultados foram obtidos com `mm.blur`, que implementa o filtro de média por meio da função `cv2.blur`, equivalente à convolução da imagem com um *kernel* uniforme cujos coeficientes são $h(x, y) = 1/N^2$; de forma equivalente, o mesmo resultado pode ser obtido com `mm.conv`, calculando ($g = f * h$). À medida que o *kernel* aumenta, mais pixels contribuem para cada valor de saída, intensificando a suavização, reduzindo o ruído e tornando detalhes finos e bordas progressivamente mais borrados.

```
# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

sizes = [3, 7, 15]
imgs = [img_gray_crop] + \
    [mm.blur(img_gray_crop, k) for k in sizes] # ou
    #[mm.conv(img_gray_crop, np.ones((k,k), dtype=np.float32)/(k*k)) for k in sizes]
titles = ["Original"] + [f"Média {k}x{k}" for k in sizes]

mm.show(imgs, titles=titles, cols=4)
```

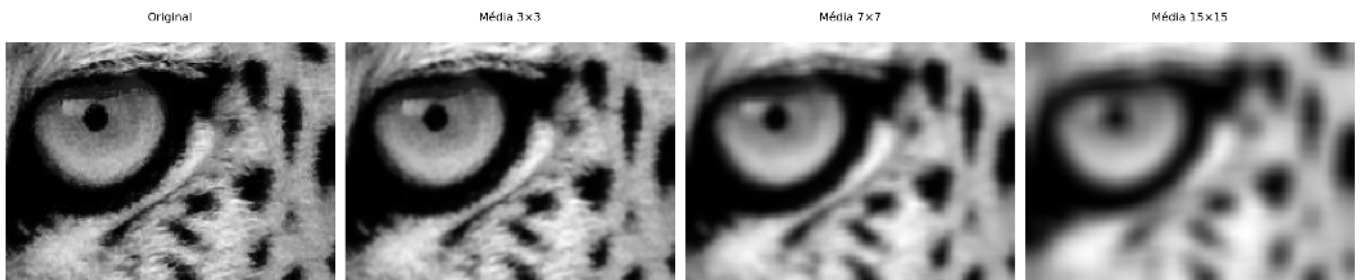


Figura 3.12: Filtro de média com *kernels* de tamanho crescente (3×3 , 7×7 , 15×15). O borramento das bordas aumenta com o tamanho do *kernel*.

3.5.2 Filtro Gaussiano

O filtro Gaussiano pesa os pixels da vizinhança de acordo com uma função Gaussiana bidimensional:

$$G(s, t) = \frac{1}{2\pi\sigma^2} e^{-\frac{s^2+t^2}{2\sigma^2}} \quad (3.14)$$

onde σ é o desvio padrão e controla o raio de influência. Pixels mais próximos do centro têm peso maior; pixels distantes são progressivamente ignorados.

A Figure 3.13 apresenta o *kernel* Gaussiano 5×5 gerado para $\sigma = 1$. O *kernel* foi construído a partir do produto externo de dois vetores Gaussianos unidimensionais e posteriormente normalizado para que a soma de seus coeficientes seja igual a 1. Observa-se que os maiores pesos concentram-se no centro da matriz, decrescendo radialmente em direção às bordas. Essa distribuição faz com que os pixels centrais tenham maior influência no resultado da filtragem, contribuindo para uma suavização mais natural e com melhor preservação de bordas do que o filtro de média.

```
# Gera kernel Gaussiano 5x5 via cv2
k_gauss = cv2.getGaussianKernel(5, 1)
w_gauss = k_gauss @ k_gauss.T          # @ => multiplicação matricial: G(s,t) = G(s)·G(t)
w_gauss = w_gauss / w_gauss.sum()      # garante soma = 1

print("Kernel Gaussiano 5x5 (=1), normalizado:")
for row in w_gauss:
```

```
print(" " + " ".join(f"{v:.4f}" for v in row))
print(f"\nSoma dos coeficientes: {w_gauss.sum():.6f}")
print(f"Peso central vs. canto: {w_gauss[2,2]:.4f} vs. {w_gauss[0,0]:.4f} "
      f"({w_gauss[2,2]/w_gauss[0,0]:.1f}× maior)")

mm.drawImgPlt((w_gauss * 1000).astype(np.uint8), scale=40)
```

Kernel Gaussiano 5×5 (=1), normalizado:

```
0.0030  0.0133  0.0219  0.0133  0.0030
0.0133  0.0596  0.0983  0.0596  0.0133
0.0219  0.0983  0.1621  0.0983  0.0219
0.0133  0.0596  0.0983  0.0596  0.0133
0.0030  0.0133  0.0219  0.0133  0.0030
```

Soma dos coeficientes: 1.000000

Peso central vs. canto: 0.1621 vs. 0.0030 (54.6× maior)

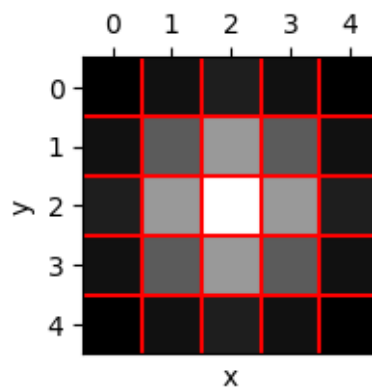


Figura 3.13: *Kernel* Gaussiano 5×5 (=1): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.

i Vantagem computacional da separabilidade

Considere um *kernel* quadrado de tamanho $n \times n$. Se esse filtro for separável (como o Gaussiano), a convolução 2D pode ser decomposta em duas convoluções 1D: uma horizontal e outra vertical. Nesse caso, o custo por pixel passa de aproximadamente $O(n^2)$ operações (convolução 2D direta) para $O(2n)$ operações (duas convoluções 1D). Assim, a complexidade é reduzida de forma significativa, tornando o processamento mais eficiente

Comparado ao filtro de média, o Gaussiano:

- **Preserva melhor as bordas** — a ponderação radial suaviza sem criar transições abruptas;
- **Não introduz anéis** (*ringing*) no domínio da frequência, pois a Gaussiana é sua própria transformada de Fourier (Capítulo 5);
- **É controlado por σ** — aumentar σ equivale a aumentar o raio de suavização de forma contínua e previsível.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados à imagem do leopardo usando uma janela 9×9 . O filtro de média foi implementado por convolução com um *kernel* uniforme, em que todos os 81 pixels da vizinhança possuem o mesmo peso ($1/81$), enquanto o filtro Gaussiano foi obtido com `cv2.GaussianBlur`, utilizando pesos definidos por uma distribuição Gaussiana. Ambos reduzem ruído e suavizam a imagem, porém o filtro Gaussiano preserva melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados a um detalhe da imagem do leopardo

com *kernels* 9×9 . O filtro de média foi obtido com `mm.blur`, equivalente à convolução com um *kernel* uniforme cujos coeficientes valem $1/81$, enquanto o filtro Gaussiano foi obtido com `mm.gaussian`, equivalente à convolução com um *kernel* gerado a partir de uma distribuição Gaussiana. Ambos promovem suavização e redução de ruído, porém o filtro Gaussiano atribui maior peso aos pixels centrais da vizinhança, preservando melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

```
# w_media9 = np.ones((9,9), dtype=np.float32) / 81.0
# img_media9 = mm.conv(img_gray, w_media9) # ou
img_media9 = mm.blur(img_gray, 9)
# img_gauss9 = cv2.GaussianBlur(img_gray, (9,9), 0) # ou
img_gauss9 = mm.gaussian(img_gray, 9, 0)

# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

mm.show(
    [crop(img_gray), crop(img_media9), crop(img_gauss9)],
    titles=["Detalhe: Original", "Média 9×9", "Gaussiano 9×9"],
    cols=3, figsize=(12, 8)
)
```



Figura 3.14: Comparação entre filtro de média e Gaussiano (*kernel* 9×9 , $\sigma=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.

3.6 Filtragem Espacial de Realce

Os filtros de realce (*sharpening filters*) enfatizam transições abruptas de intensidade, aumentando a nitidez e a visibilidade de bordas. São filtros **passa-alta** — amplificam as componentes de alta frequência (bordas, textura) e suprimem as de baixa frequência (regiões uniformes).

A intuição é simples: se subtrairmos de uma imagem sua versão suavizada (que contém apenas as baixas frequências), o que resta são as altas frequências — bordas e detalhes. Somando esse resíduo de volta à imagem original, o contraste local aumenta:

$$g = f + k(f - f_{\text{suave}}), \quad k > 0 \quad (3.15)$$

O Figure 3.15 ilustra esse processo em um sinal 1D sintético com três estruturas distintas: um degrau largo, um pico fino e uma rampa suave. No painel , o sinal original $f(x)$; no , a versão suavizada $f_{\text{suave}}(x)$ obtida por média móvel — note como o pico fino é atenuado. O painel exibe o resíduo $f - f_{\text{suave}}$, que retém apenas as transições abruptas. Por fim, o painel mostra $g(x)$: o pico, antes atenuado, é restaurado e amplificado

em relação ao original. Ajuste k e o tamanho da janela para observar o *trade-off* entre nitidez e amplificação de ruído.

Os filtros de realce formalizam essa ideia diretamente no *kernel*, sem precisar de duas etapas separadas.



Figura 3.15: Simulador: Filtragem Espacial de Realce 1D.

3.6.1 Laplaciano

O Laplaciano é um operador de **segunda derivada** isotrópico, ou seja, responde igualmente a variações em todas as direções, ao contrário de operadores de primeira derivada, como Sobel e Prewitt, que são direcionais:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (3.16)$$

Uma propriedade importante da segunda derivada é que seu valor é **próximo de zero em regiões uniformes** e elevado nas transições de intensidade. Assim, ao subtrair o Laplaciano da imagem original, reforçam-se bordas e detalhes, aumentando o contraste local:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \quad (3.17)$$

Na forma discreta, a segunda derivada em x é aproximada por $f(x+1, y) - 2f(x, y) + f(x-1, y)$, e analogamente em y . Somando as duas direções, obtém-se o *kernel* w_4 (4-vizinhos) ou w_8 (8-vizinhos, incluindo diagonais):

$$w_4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad w_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (3.18)$$

i Soma zero e centro negativo

Ambos os *kernels* têm **soma dos coeficientes igual a zero**: em regiões uniformes, a saída é 0 — o Laplaciano não altera o brilho médio, apenas detecta variações. O **centro negativo** indica que o pixel é comparado com seus vizinhos: quanto mais ele se destacar (para cima ou para baixo), maior o valor absoluto do Laplaciano naquele ponto.

No exemplo a seguir, o pixel central $[1, 1] = 107$ possui vizinhos $\{95, 106, 108, 103\}$. Como esses valores são próximos entre si, a região é quase uniforme e o Laplaciano retorna um valor baixo, produzindo pouco realce. Em regiões de borda, onde há diferenças maiores entre o pixel central e seus vizinhos, o Laplaciano assume valores mais elevados (positivos ou negativos), e a operação de Equation 3.17 intensifica essas transições.

A Figure 3.16 ilustra o cálculo do Laplaciano com o *kernel* w_4 em uma vizinhança 3×3 destacada dentro de um *patch* 5×5 . O exemplo mostra o valor obtido pelo operador e o correspondente pixel realçado na imagem de saída, que passa de 107 para 123.

```
patch = img_gray[250:255, 250:255]
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
B = np.ones((3,3), dtype=np.uint8)

p = patch.astype(np.float32)
lap_11 = int(p[0,1] + p[1,0] - 4*p[1,1] + p[1,2] + p[2,1])
real_11 = int(np.clip(p[1,1] - lap_11, 0, 255))

print("Patch original:")
print(mm.drawImg(patch))
print(f"Laplac. w4 em [1,1]: ", end="")
print(f"{p[0,1]:.0f} + {p[1,0]:.0f} - 4*{p[1,1]:.0f} + {p[1,2]:.0f} + {p[2,1]:.0f} = {lap_11}")
print(f"Pixel realçado [1,1]: {p[1,1]:.0f} - ({lap_11}) = {real_11}")

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Laplac. w4 em [1,1]: 95 + 106 - 4*107 + 108 + 103 = -16
Pixel realçado [1,1]: 107 - (-16) = 123
```

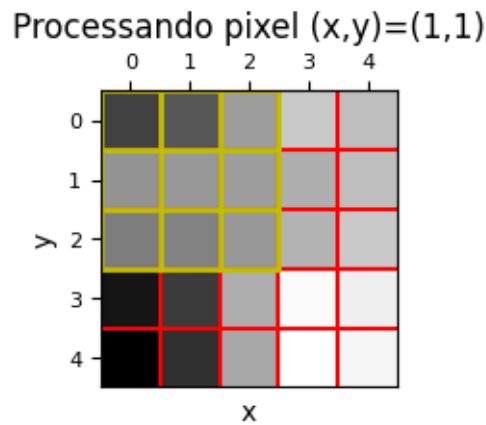


Figura 3.16: *Kernel* Laplaciano w_4 aplicado ao *patch* 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.

A Figure 3.17 compara a aplicação dos *kernels* Laplacianos w_4 e w_8 em um recorte maior da imagem do leopardo. Para cada caso, são mostradas a resposta bruta do operador, que evidencia as bordas e transições de intensidade, e a imagem obtida após o realce por subtração do Laplaciano. Observa-se que o *kernel* w_8 , por considerar também os vizinhos diagonais, produz uma resposta mais intensa e detecta variações em mais direções, resultando em um realce ligeiramente mais acentuado.

```
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
w8 = np.array([[1, 1, 1],
               [1,-8, 1],
               [1, 1, 1]], dtype=np.float32)

mm.show(
    [img_gray_crop, mm.laplacian_viz(img_gray_crop, w4), mm.laplacian(img_gray_crop, w4),
     img_gray_crop, mm.laplacian_viz(img_gray_crop, w8), mm.laplacian(img_gray_crop, w8)],
    titles=["Original", "Laplaciano w4", "Realce w4",
           "Original", "Laplaciano w8", "Realce w8"],
    rows=2, cols=3, figsize=(14, 8)
)
```



Figura 3.17: Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.

3.6.2 Operador de Sobel

O operador de Sobel estima as **derivadas parciais de primeira ordem** nas direções horizontal e vertical. Diferente do Laplaciano (segunda derivada), o Sobel é direcional e mais robusto ao ruído, pois cada *kernel* combina uma derivada com uma suavização Gaussiana perpendicular:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * f \quad (3.19)$$

G_x detecta bordas **verticais** (variação na direção x); G_y detecta bordas **horizontais** (variação na direção y). Os pesos $\{1, 2, 1\}$ na direção perpendicular correspondem à suavização Gaussiana 1D, que reduz a sensibilidade ao ruído.

i Sobel é correlação, não convolução

Os *kernels* de Sobel são **assimétricos** — a rotação de 180° altera o resultado. `cv2.Sobel` implementa correlação cruzada (como `cv2.filter2D`). Para obter a derivada direcional correta, os sinais já estão definidos para correlação: G_x retorna valores positivos onde a intensidade cresce da esquerda para a direita.

A magnitude do **gradiente** combina os dois componentes, representando a força da borda independente de direção:

$$|\nabla f| = \sqrt{G_x^2 + G_y^2} \quad (3.20)$$

E a **direção** do gradiente (perpendicular à borda) é:

$$\theta = \arctan \left(\frac{G_y}{G_x} \right) \quad (3.21)$$

Para ilustrar numericamente, calcula-se G_x e G_y manualmente no pixel central $[1, 1]$ do *patch* 5×5:

```
patch = img_gray[250:255, 250:255]
p = patch.astype(np.float32)

# Gx no pixel [1,1]: coluna direita - coluna esquerda (ponderada)
gx = (p[0,2] + 2*p[1,2] + p[2,2]) - (p[0,0] + 2*p[1,0] + p[2,0])

# Gy no pixel [1,1]: linha inferior - linha superior (ponderada)
gy = (p[2,0] + 2*p[2,1] + p[2,2]) - (p[0,0] + 2*p[0,1] + p[0,2])

mag = np.sqrt(gx**2 + gy**2)
theta = np.degrees(np.arctan2(gy, gx))

print("Patch 5x5:")
print(mm.drawImg(patch))
print(f"Gx em [1,1]: ({p[0,2]:.0f} + 2*{p[1,2]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[1,0]:.0f} + {p[2,0]:.0f}) = {gx:.1f}")
print(f"Gy em [1,1]: ({p[2,0]:.0f} + 2*{p[2,1]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[0,1]:.0f} + {p[0,2]:.0f}) = {gy:.1f}")

# Substituído o | pelo prefixo mag para não quebrar o interpretador de tabelas do Quarto
print(f"mag(Grad f) = sqrt({gx:.1f}^2 + {gy:.1f}^2) = {mag:.1f}")
print(f"Theta = arctan({gy:.1f}/{gx:.1f}) = {theta:.1f} graus")
```

Patch 5x5:

```
91 95 108 116 114
106 107 108 111 114
102 103 107 112 116
83 90 111 125 123
79 88 110 126 124
```

```
Gx em [1,1]: (108 + 2*108 + 107) - (91 + 2*106 + 102) = 26.0
Gy em [1,1]: (102 + 2*103 + 107) - (91 + 2*95 + 108) = 26.0
mag(Grad f) = sqrt(26.0^2 + 26.0^2) = 36.8
Theta = arctan(26.0/26.0) = 45.0 graus
```

O valor reduzido de $|\nabla f|$ nesse *patch* confirma que a região é quase uniforme, pois o gradiente assume valores elevados apenas onde há mudanças significativas de intensidade. A Figure 3.18 aplica o operador de Sobel a um recorte maior da imagem do leopardo. São apresentadas as respostas horizontal (G_x) e vertical (G_y), obtidas por convolução com os respectivos *kernels* de Sobel, além da magnitude $|\nabla f|$, calculada a partir da combinação de ambas. Enquanto G_x destaca bordas verticais e G_y bordas horizontais, a magnitude evidencia bordas em qualquer direção.

```
def norm255(img):
    mn, mx = img.min(), img.max() #+1e-9 para evitar divisão por zero
    return ((img - mn) / (mx - mn + 1e-9) * 255).astype(np.uint8)

sobelx = cv2.Sobel(img_gray_crop, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img_gray_crop, cv2.CV_64F, 0, 1, ksize=3)
magnitudo = np.sqrt(sobelx**2 + sobely**2)
direcao = np.degrees(np.arctan2(sobely, sobelx))

# magnitudo = mm.sobel(img_gray_crop) # ou, mas com clip em vez de norm255

mm.show()
```



```
[img_gray_crop, norm255(np.abs(sobelx)), norm255(np.abs(sobely)),
 norm255(magnitude), norm255(direcao % 180)],
titles=["Original", "Gx (bordas vert.)", "Gy (bordas horiz.)",
        "Magnitude |f|", "Direção (mod 180°)"],
cols=5
)
```



Figura 3.18: Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $|f|$ combina ambos, revelando todas as bordas independente de direção.

3.6.3 Operador de Prewitt

O operador de Prewitt é estruturalmente idêntico ao Sobel, mas substitui a ponderação Gaussiana $\{1, 2, 1\}$ por pesos uniformes $\{1, 1, 1\}$:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} * f \quad (3.22)$$

A magnitude e a direção do gradiente seguem as mesmas equações do Sobel (Equation 3.20 e Equation 3.21). A diferença prática é que o Prewitt é ligeiramente mais sensível ao ruído — a suavização perpendicular uniforme pondera menos o pixel central da linha — mas computacionalmente mais simples. Em imagens com baixo ruído os resultados são equivalentes.

```
Kx = np.array([[ -1, 0, 1], [ -1, 0, 1], [ -1, 0, 1]], dtype=np.float32)
Ky = np.array([[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]], dtype=np.float32)
f = img_gray_crop.astype(np.float32)

mm.show(
    [img_gray_crop,
     norm255(np.abs(cv2.filter2D(f, -1, Kx))),
     norm255(np.abs(cv2.filter2D(f, -1, Ky))),
     mm.prewitt(img_gray_crop)],
    titles=["Original", "Gx", "Gy", "Magnitude |f|"],
    cols=4
)
```

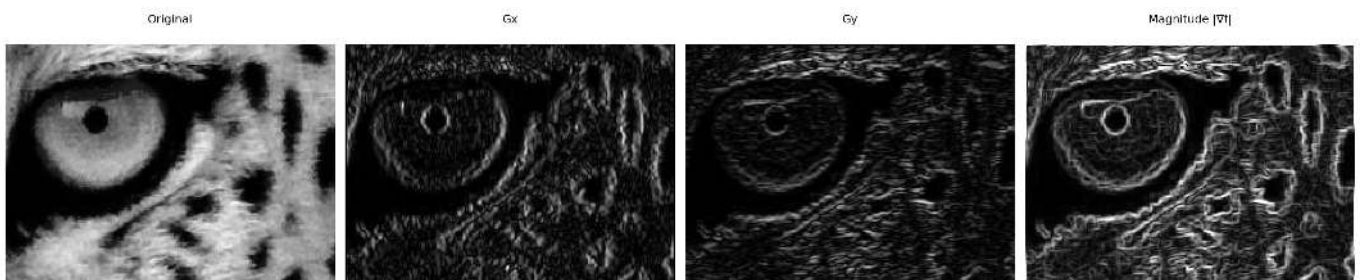


Figura 3.19: Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.

3.6.4 Unsharp Masking (USM)

O *Unsharp Masking* é uma técnica clássica de realce de nitidez originária da fotografia analógica, hoje amplamente usada em software de edição de imagens. A ideia central é extrair as **componentes de alta frequência** da imagem (bordas e detalhes) e somá-las de volta à original com um peso k :

Tabela 3.3: Etapas do *Unsharp Masking*.

Etapas	Operação	Descrição
1	$\bar{f} = f * G_{\sigma}$	Suaviza com Gaussiana — retém baixas frequências
2	$m = f - \bar{f}$	Máscara: diferença = altas frequências (bordas)
3	$g = f + k \cdot m$	Soma ponderada da máscara à original

Substituindo a etapa 2 na etapa 3, obtém-se a expressão compacta:

$$g = f + k(f - f * G_{\sigma}) = (1 + k)f - k(f * G_{\sigma}) \quad (3.23)$$

O parâmetro k controla a intensidade do realce:

- $k = 0$: sem realce ($g = f$);
- $k = 1$: USM clássico — duplica a contribuição das altas frequências;
- $k > 1$: *High Boost Filtering* — amplificação além do dobro, útil para imagens muito borradas.

⚠ Amplificação de ruído

O USM não distingue bordas de ruído — ambos são componentes de alta frequência. Para k elevado, o ruído presente na imagem é amplificado junto com as bordas. Por isso, é recomendável aplicar uma leve suavização antes do USM em imagens ruidosas, ou usar σ pequeno na Gaussiana.

Para ilustrar as etapas do USM, a Figure 3.20 aplica o método a um *patch* 30×30 da imagem do leopardo, utilizando $\sigma = 1$ e $k = 1$. Inicialmente, a imagem é suavizada por um filtro Gaussiano. Em seguida, a máscara de alta frequência é obtida pela diferença entre a imagem original e a suavizada. Por fim, essa máscara é somada à imagem original, reforçando bordas e detalhes. A figura apresenta as três etapas do processo e o resultado final do realce.

```
patch = img_gray_crop[35:65, 45:75]
k = 1.0

p = patch.astype(np.float32)
sigma = 1.0

# Etapa 1: suavização Gaussiana
ksize = int(6 * sigma + 1) | 1 # operador binário garante tamanho ímpar
p_suave = cv2.GaussianBlur(p, (ksize, ksize), sigma)

# Etapa 2: máscara de alta frequência
mascara = p - p_suave

# Etapa 3: realce
p_usm = np.clip(p + k * mascara, 0, 255).astype(np.uint8)

# p_usb = mm.usm(p, k) # ou

print(f"Pixel central [1,1]: original={int(p[1,1])} suave={p_suave[1,1]:.1f}")
```

```

f" mascara={mascara[1,1]:.1f} realcado={p_usm[1,1]}")

mm.show(
    [patch,
     p_suave.astype(np.uint8),
     np.clip(mascara + 128, 0, 255).astype(np.uint8), # tornar os valores negativos visíveis na figura
     p_usm],
    titles=["Patch original",
            f"Suavizado (={sigma})",
            "Máscara (alta freq., +128)",
            f"Realçado USM (k={k})"],
    cols=4, figsize=(12, 4)
)

```

Pixel central [1,1]: original=16 suave=20.2 mascara=-4.2 realcado=11

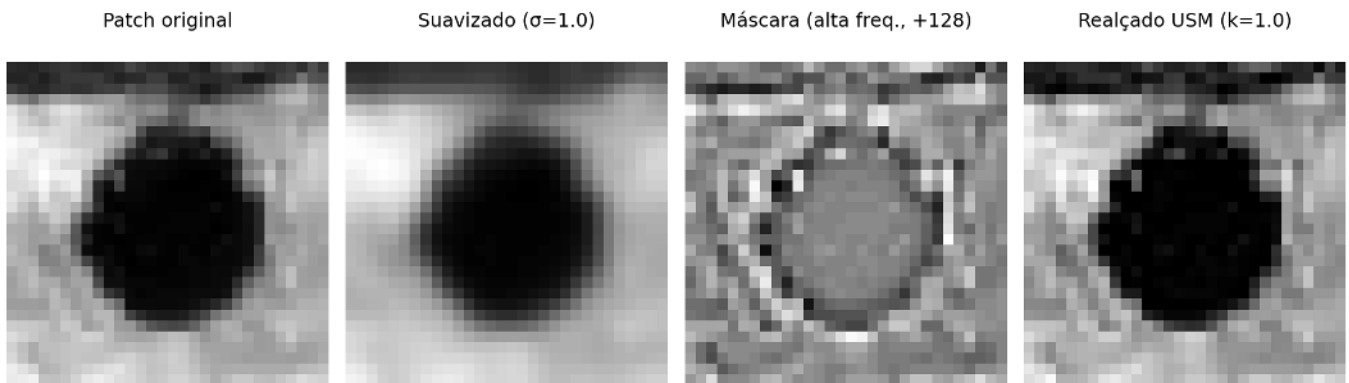


Figura 3.20: Realce de nitidez por *Unsharp Masking* na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.

A Figure 3.21 aplica o método USM a um recorte maior da imagem do leopardo utilizando $\sigma = 1$ e diferentes valores do fator de ganho k . Em todos os casos, a máscara de alta frequência é obtida pela diferença entre a imagem original e sua versão suavizada por filtro Gaussiano. O parâmetro k controla a intensidade do realce: valores menores produzem um aumento sutil de nitidez, enquanto valores maiores reforçam progressivamente bordas e detalhes. Observa-se que, para valores elevados de k , surgem halos ao redor das bordas e o ruído presente na imagem passa a ser amplificado.

```

ks = [0.5, 1.0, 3.0, 5.0, 8.0]

imgs = [img_gray_crop] + [mm.usm(img_gray_crop, k) for k in ks]
titles = ["Original"] + [f"USM k={k}" for k in ks]

mm.show(imgs, titles=titles, cols=3, rows=2, figsize=(14, 8))

```

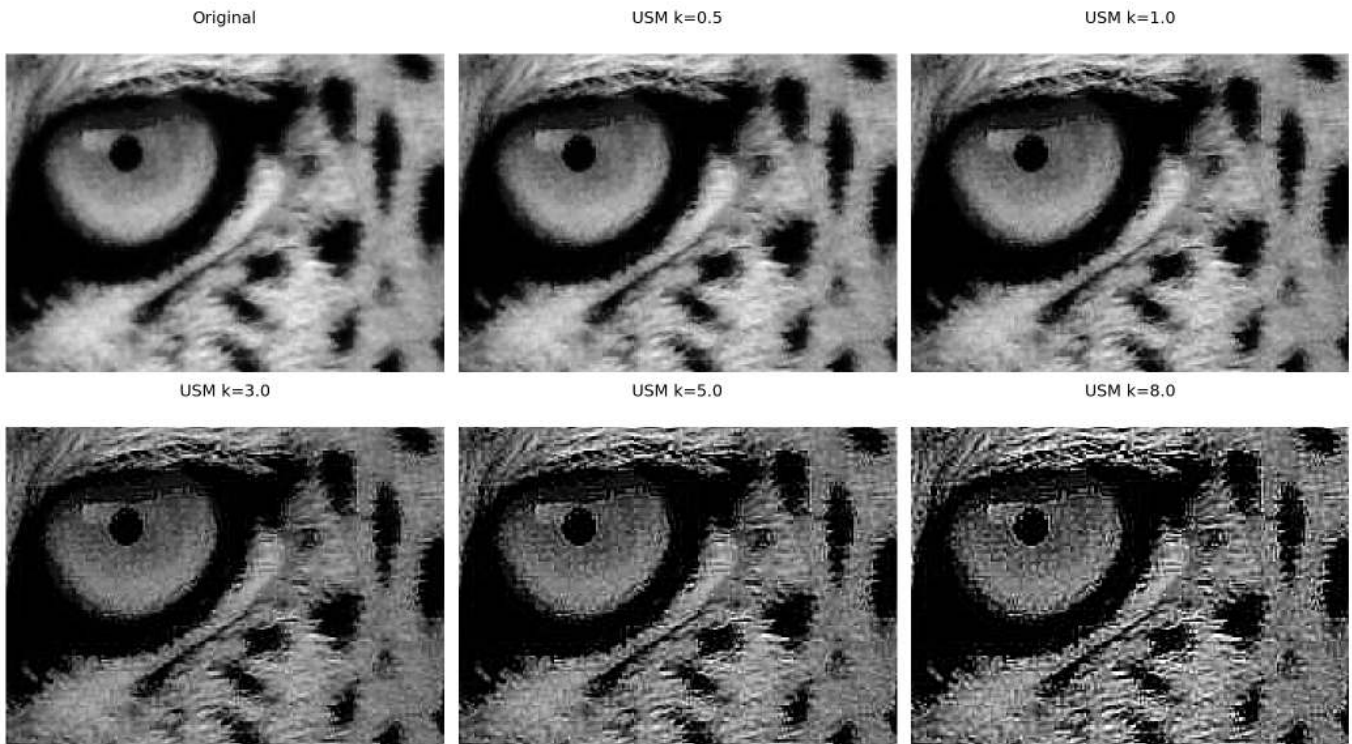


Figura 3.21: *Unsharp Masking* na imagem do leopardo com $\alpha=1$ e k variando de 0.5 a 8.0. Para $k>2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.

3.6.5 Detector de Canny

O Canny combina quatro etapas em sequência — suavização Gaussiana, gradiente de Sobel, supressão de não-máximos e histerese por duplo limiar — para produzir bordas **finas, binárias e conectadas**. Diferente de Sobel e Prewitt, o resultado não é um mapa de gradiente contínuo, mas uma máscara onde cada pixel é borda ou não.

O parâmetro central é o par de limiares (T_{low}, T_{high}). Pixels com gradiente acima de T_{high} são bordas certas; abaixo de T_{low} , descartados. Os pixels ambíguos — entre os dois limiares — são decididos por **histerese**: tornam-se borda se estiverem conectados a uma borda certa, e descartados caso contrário. Isso evita tanto a perda de trechos fracos de bordas reais quanto a inclusão de ruído isolado. Uma heurística comum é $T_{high} = 3 \times T_{low}$.

```
mm.show(
    [img_gray_crop,
     mm.canny(img_gray_crop, 30, 90),
     mm.canny(img_gray_crop, 60, 180),
     mm.canny(img_gray_crop, 120, 240)],
    titles=["Original", "Canny (30/90)", "Canny (60/180)", "Canny (120/240)"],
    cols=4
)
```

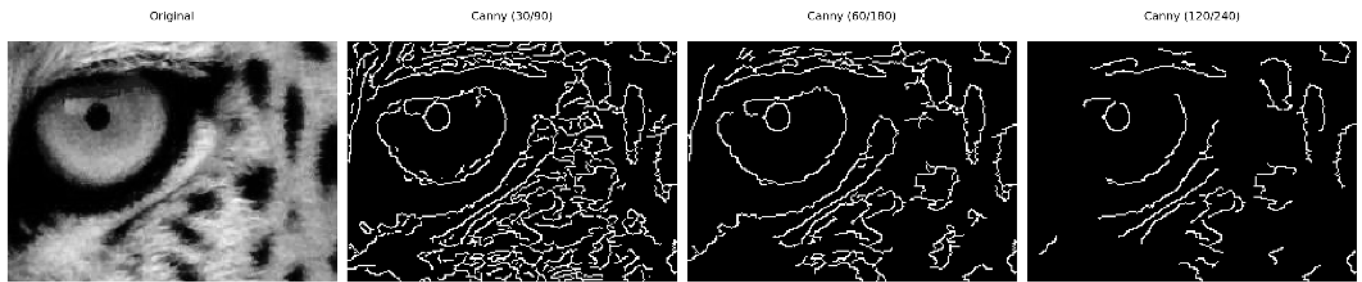


Figura 3.22: Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.

i Escolha dos limiares

Uma heurística comum é $T_{high} = 3 \times T_{low}$. Valores típicos dependem do intervalo de gradiente da imagem — `cv2.Canny` aceita valores absolutos em $[0, 255]$. Para imagens com contraste variável, calcular os limiares a partir de percentis da magnitude de Sobel é mais robusto do que valores fixos.

3.7 Filtros de Ordem: Filtro da Mediana

Os filtros de ordem (*order-statistic filters*) substituem o pixel central pelo valor de um **percentil** da distribuição de intensidades da vizinhança — ao contrário dos filtros lineares, que calculam combinações ponderadas. O mais importante é o **filtro da mediana**.

3.7.1 Ruído Impulsivo: Sal e Pimenta

O ruído **sal e pimenta** (*salt-and-pepper noise*) substitui pixels aleatórios por valores extremos: 0 (pimenta, preto) ou 255 (sal, branco). É comum em transmissão de imagens com erros de bit e em câmeras com sensores defeituosos.

Para entender por que os filtros lineares falham, considere uma vizinhança 3×3 onde um único pixel foi corrompido para 255:

$$\text{vizinhança} = \begin{bmatrix} 102 & 98 & 105 \\ 100 & \mathbf{255} & 97 \\ 103 & 99 & 101 \end{bmatrix}$$

Tabela 3.4: Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.

Método	Cálculo	Resultado
Média	$(102+98+\dots+255+\dots+101)/9$	140
Mediana	$\{97, 98, 99, 100, \mathbf{101}, 102, 103, 105, 255\}$	101

⚠ Por que filtros de média falham com ruído impulsivo?

A média é sensível a **outliers** — um único pixel com valor 255 em uma vizinhança de valor 100 eleva a saída para 140, espalhando o ruído pela imagem. A mediana, por ser um **estimador robusto**, seleciona o valor central da distribuição ordenada, descartando naturalmente os extremos sem nenhum ajuste especial.

O exemplo a seguir ilustra o comportamento da média e da mediana na presença de um pixel corrompido por ruído impulsivo. Observa-se que a média é fortemente influenciada pelo valor extremo (255), produzindo uma estimativa distante dos valores predominantes da vizinhança. Já a mediana permanece próxima do valor original da região, evidenciando sua maior robustez a *outliers* e justificando seu uso na remoção de ruído sal e pimenta.

```
# Exemplo numérico: média vs. mediana com pixel corrompido
vizinhanca = np.array([102, 98, 105, 100, 255, 97, 103, 99, 101])
print(f"Vizinhança: {sorted([int(x) for x in vizinhanca])}")
print(f"Média:      {vizinhanca.mean():.1f} (deslocada pelo outlier 255)")
print(f"Mediana:    {int(np.median(vizinhanca))} (ignora o outlier)")
print(f"Valor original: ~100")
```

```
Vizinhança: [97, 98, 99, 100, 101, 102, 103, 105, 255]
Média:      117.8 (deslocada pelo outlier 255)
Mediana:    101 (ignora o outlier)
Valor original: ~100
```

A Figure 3.23 apresenta o efeito do ruído sal e pimenta em diferentes densidades. O ruído foi gerado substituindo aleatoriamente uma fração dos pixels por valores mínimos (0, pimenta) e máximos (255, sal). À medida que a densidade aumenta de 2% para 10%, cresce a quantidade de pixels corrompidos, tornando a degradação visual mais evidente e dificultando a percepção de detalhes da imagem.

```
def add_salt_pepper(img, prob=0.05, seed=42):
    """Adiciona ruído sal e pimenta com probabilidade total prob."""
    noisy = img.copy()
    rnd = np.random.default_rng(seed).random(img.shape)
    noisy[rnd < prob / 2] = 0 # pimenta
    noisy[rnd > 1 - prob / 2] = 255 # sal
    n_corrompidos = int((rnd < prob/2).sum() + (rnd > 1-prob/2).sum())
    return noisy, n_corrompidos

probs = [0.02, 0.05, 0.10]
imgs_noise, titles_noise = [img_gray_crop], ["Original"]

for p in probs:
    noisy, n = add_salt_pepper(img_gray_crop, p)
    imgs_noise.append(noisy)
    titles_noise.append(f"Ruído {int(p*100)}%\n({n:,} pixels)")

mm.show(imgs_noise, titles=titles_noise, cols=4)
```

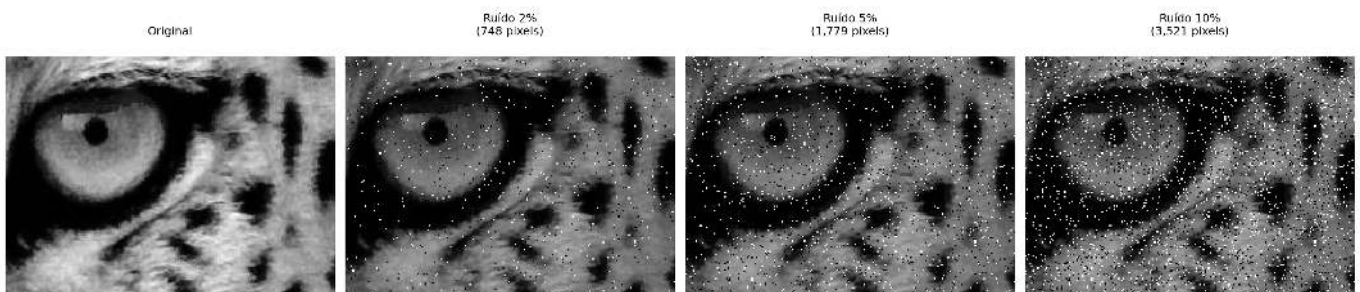


Figura 3.23: Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro *prob* indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).

3.7.2 Filtro da Mediana

O filtro da mediana substitui cada pixel pelo **valor mediano** dos pixels da sua vizinhança $n \times n$:

$$g(x, y) = \text{med}_{(s,t) \in \mathcal{V}_n} \{f(x + s, y + t)\} \quad (3.24)$$

O valor mediano é aquele que ocupa a posição central quando os n^2 valores da vizinhança são ordenados. Para uma janela 3×3 ($n^2 = 9$ pixels), a mediana é o 5º valor da sequência ordenada.

Para ilustrar, considere o mesmo *patch* 5×5 com um pixel corrompido artificialmente em $[1, 1]$:

```
patch = img_gray[250:255, 250:255].copy()
corrompido = patch.copy()
corrompido[1, 1] = 255 # injeta pixel sal no centro

# Vizinhança 3x3 centrada em [1,1]
viz_orig = sorted(patch[0:3, 0:3].ravel().tolist())
viz_corr = sorted(corrompido[0:3, 0:3].ravel().tolist())

print("Patch original:")
print(mm.drawImg(patch))
print("\nPatch com pixel corrompido [1,1]=255:")
print(mm.drawImg(corrompido))
print(f"\nVizinhança 3x3 original (ordenada): {viz_orig}")
print(f"Mediana original: {viz_orig[4]}")
print(f"\nVizinhança 3x3 corrompida (ordenada): {viz_corr}")
print(f"Mediana corrompida: {viz_corr[4]} ← ignora o 255")
print(f"Média corrompida: {np.mean(viz_corr):.1f} ← distorcida pelo 255")
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Patch com pixel corrompido [1,1]=255:
```

```
 91  95 108 116 114
106 255 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Vizinhança 3x3 original (ordenada): [91, 95, 102, 103, 106, 107, 107, 108, 108]
Mediana original: 106
```

```
Vizinhança 3x3 corrompida (ordenada): [91, 95, 102, 103, 106, 107, 108, 108, 255]
Mediana corrompida: 106 ← ignora o 255
Média corrompida: 119.4 ← distorcida pelo 255
```

O exemplo confirma: mesmo com o pixel corrompido a 255, a mediana retorna o valor central correto — o *outlier* ocupa a última posição na ordenação e é descartado naturalmente.

Por ser baseada em ordenação e não em soma, a mediana possui três propriedades fundamentais que a diferenciam dos filtros lineares:

- **Robusta** ao ruído impulsivo — outliers vão para as extremidades da sequência ordenada e não afetam o valor central;
- **Preservadora de bordas** — transições abruptas de intensidade são mantidas, pois a mediana seleciona um valor que já existe na vizinhança, sem criar novos níveis intermediários;

- **Não linear** — não pode ser expressa como convolução, portanto `mm.conv` não se aplica; usa-se `cv2.medianBlur`.

A Figure 3.24 compara diferentes técnicas de remoção de ruído sal e pimenta aplicadas a uma imagem com 10% de pixels corrompidos. Foram avaliados os filtros Gaussiano, Média, Mediana, Bilateral e Morfológico (próximo capítulo), permitindo observar o compromisso entre remoção de ruído e preservação de detalhes. Em geral, os filtros de média e Gaussiano reduzem o ruído, mas tendem a borrar as bordas, enquanto a mediana apresenta melhor desempenho para ruído impulsivo. O filtro bilateral preserva melhor as bordas, e o filtro morfológico remove boa parte dos pixels corrompidos sem degradar excessivamente a estrutura da imagem.

```
# 10% de ruído para evidenciar diferenças entre filtros
noisy_5, _ = add_salt_pepper(img_gray_crop, prob=0.1)

# Filtros
f_gauss = cv2.GaussianBlur(noisy_5, (5, 5), 1.0)
f_media = mm.conv(noisy_5, np.ones((5, 5), dtype=np.float32) / 20.0)
f_median3 = cv2.medianBlur(noisy_5, 3)
f_median5 = cv2.medianBlur(noisy_5, 5)
f_bilat = cv2.bilateralFilter(noisy_5, d=9, sigmaColor=75, sigmaSpace=75)

# filtros morfológicos no próximo capítulo, mas já adiantados aqui para comparação
# B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
# f_morf = cv2.morphologyEx( # apresentado no próximo capítulo
#     cv2.morphologyEx(noisy_5, cv2.MORPH_OPEN, B),
#     cv2.MORPH_CLOSE, B)
B = mm.secross() # elemento estruturante de caixa 3x3
f_morf = mm.asf(noisy_5, 'OC', B) # equivalente via morph

# MSE
def mse(a, b):
    return float(np.mean((a.astype(np.float32) - b.astype(np.float32))**2))

print(f"{'Filtro':<22} {'MSE':>8}")
print("-" * 32)
pares = [("Gaussiano 5x5", f_gauss),
         ("Média 5x5", f_media),
         ("Mediana 3x3", f_median3),
         ("Mediana 5x5", f_median5),
         ("Bilateral", f_bilat),
         ("Morf. open+close", f_morf)]
for nome, img in pares:
    print(f"{nome:<22} {mse(img_gray_crop, img):>8.2f}")

imgs = [img_gray_crop, noisy_5, f_gauss, f_media,
        f_median3, f_median5, f_bilat, f_morf]
titles = ["Original", "Ruído 10%", "Gaussiano 5x5", "Média 5x5",
         "Mediana 3x3", "Mediana 5x5", "Bilateral", "Morf. open+close"]

mm.show(
    imgs,
    titles=[f"Crop: {t}" for t in titles],
    cols=4, rows=2, figsize=(14, 8)
)
```

Filtro	MSE
Gaussiano 5x5	260.84
Média 5x5	872.57
Mediana 3x3	31.26

Mediana 5×5	65.86
Bilateral	1001.22
Morf. open+close	65.16

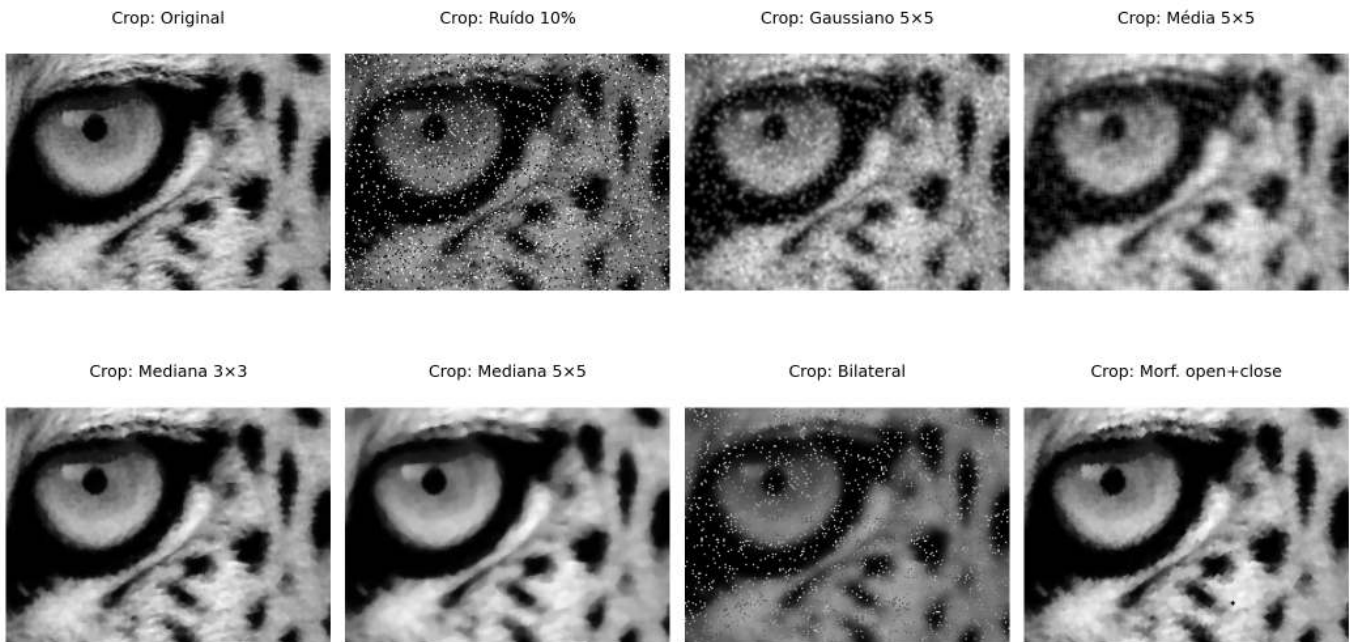


Figura 3.24: Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.

3.8 Aplicação Prática: Pré-processamento para Segmentação

Na prática, as técnicas deste capítulo raramente são usadas isoladamente. Um *pipeline* de **pré-processamento** típico combina várias etapas em sequência, adaptando-se ao tipo de imagem e à aplicação. A Figure 3.25 ilustra um *pipeline* completo:

1. **Equalização de histograma (CLAHE)**: normaliza o contraste independente das condições de iluminação;
2. **Filtro Gaussiano**: suaviza ruído de aquisição sem destruir bordas;
3. **Deteção de bordas (Sobel/Canny)**: extrai estruturas relevantes para segmentação.

i Ordem importa

A ordem das operações afeta o resultado final. Em geral: **(1) normalização de intensidade → (2) redução de ruído → (3) realce/segmentação**. Inverter a ordem pode amplificar ruído ou perder bordas antes de detectá-las.

```
# Etapa 1: CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img_gray_crop)

# Etapa 2: Gaussiano
img_gauss = cv2.GaussianBlur(img_clahe, (5, 5), 0)

# Etapa 3: Canny
edges = cv2.Canny(img_gauss, 50, 150)

# Comparação: pipeline vs. Canny direto
edges_direct = cv2.Canny(img_gray_crop, 50, 150)
```

```
mm.show(
    [img_gray_crop, img_clahe, img_gauss, edges, edges_direct],
    titles=["Original", "1. CLAHE", "2. Gaussiano", "3. Canny (pipeline)", "Canny (direto)"],
    cols=5
)
```



Figura 3.25: *Pipeline* de pré-processamento: CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.

3.9 Resumo

Neste capítulo foram apresentadas as principais técnicas de processamento no domínio espacial, da manipulação direta de pixels até a filtragem por vizinhança:

- **Operações de ponto:** aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para recorte de ROI e combinação de imagens; *alpha blending* (`mm.blend`) para fusão ponderada com peso $\alpha \in [0, 1]$.
- **Histograma:** função discreta de distribuição de intensidades; visualizado com `mm.histImg` e calculado com `mm.hist`; base para diagnóstico tonal e para as técnicas de equalização e especificação.
- **Equalização:** redistribuição automática das intensidades pela CDF (`mm.equalize`), com variante adaptativa CLAHE para controle local do contraste.
- **Especificação de histograma:** transferência do perfil tonal de uma imagem de referência via mapeamento inverso da CDF — generalização da equalização para distribuições arbitrárias.
- **Correlação e convolução:** mecanismo de janela deslizante implementado em `mm.conv` (`cv2.filter2D`); diferenciados pela rotação de 180° do *kernel* — relevante apenas para kernels assimétricos.
- **Filtros de suavização:** média (*kernel* uniforme, borra bordas proporcionalmente ao tamanho) e Gaussiano (ponderação radial, separável, sem *ringing*, preserva melhor as bordas).
- **Filtros de realce:** Laplaciano (w_4/w_8 , segunda derivada isotrópica), Sobel (gradiente direcional de primeira ordem, com magnitude $|\nabla f|$ e direção θ) e *Unsharp Masking* (amplificação das altas frequências com parâmetro k).
- **Filtro da mediana:** não linear, robusto a outliers, preserva bordas — superior aos filtros lineares para ruído sal e pimenta.
- **Pipeline prático:** encadeamento CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny como estratégia de pré-processamento; `mm.drawImgKernel` para visualização didática da janela deslizante.

O Capítulo 4 abordará a **morfologia matemática** (erosão, dilatação, abertura e fechamento), explorando em profundidade as funções `mm.ero` e `mm.dil` da biblioteca `morph.py`. Em seguida, o Capítulo 5 apresentará o **processamento no domínio da frequência**, com foco na Transformada de Fourier e em técnicas de filtragem espectral.

3.10 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentivamos o uso do **NotebookLM** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro — incluindo as funções da biblioteca `morph.py` e os experimentos realizados neste capítulo.

Para cada capítulo, preparamos um projeto específico na plataforma com o PDF do capítulo, os *notebooks* e materiais auxiliares. Sugerimos explorar especialmente:

- **Guia de Estudo:** resumo estruturado dos conceitos, ideal para revisão antes de provas;
- **Conversa:** tire dúvidas sobre equalização, convolução, filtros e pipelines diretamente com o tutor;
- **Perguntas frequentes:** questões típicas sobre a diferença entre média e mediana, USM, Laplaciano vs. Sobel.

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o *link* a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR NOTEBOOKLM: CAPÍTULO 03](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

3.11 Lista de Exercícios

1. (10%) Explique a diferença entre **convolução** e **correlação cruzada**. Para quais tipos de *kernel* os resultados são idênticos? Dê um exemplo de *kernel* assimétrico (como Sobel G_x) e mostre numericamente que os resultados diferem aplicando-o ao patch 5×5 do capítulo das duas formas.
2. (15%) Considere uma imagem 5×5 com intensidades concentradas entre os níveis 3 e 5 (baixo contraste, 3 bits). Aplique manualmente o algoritmo de equalização da Table 3.1, preenchendo todas as colunas da tabela (k , $h[k]$, $p[k]$, $\text{cdf}[k]$, $\text{lut}[k]$). Verifique o resultado com `mm.equalize`.
3. (15%) Usando `mm.conv`, aplique o filtro de média com kernels de tamanho 3×3 , 9×9 e 21×21 à imagem do mandrill. Para cada versão, calcule o **PSNR** (*Peak Signal-to-Noise Ratio*) em relação à original:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{MN} \sum_{i,j} (f - g)^2$$

Plote o PSNR em função do tamanho do kernel e explique o que a queda progressiva indica sobre a relação entre suavização e perda de informação.

4. (15%) Usando `add_salt_pepper` com densidade de 5%, aplique e compare: (a) `mm.conv` com média 3×3 , (b) `cv2.GaussianBlur` com $\sigma = 1$, (c) `cv2.medianBlur` com janela 3×3 e (d) `cv2.medianBlur` com janela 5×5 . Exiba as imagens com `mm.show` em grade 2×4 (linha 1: imagens, linha 2: histogramas via `mm.histImg`). Explique por que a mediana supera os filtros lineares usando o argumento da Table 3.4.
5. (15%) Implemente `mm.conv0` usando apenas operações NumPy vetorizadas — sem laços Python e sem `cv2.filter2D` — com o operador de *stride tricks* (`np.lib.stride_tricks.sliding_window_view`). Compare o resultado e o tempo de execução com `mm.conv0` (laços) e `mm.conv` (cv2) para kernels 3×3 e 15×15 na imagem do mandrill.
6. (15%) Aplique o *Unsharp Masking* com $\sigma = 1$ e $k \in \{0.5, 1.0, 2.0, 4.0\}$ usando a função `usm` do capítulo. Para cada valor de k : (a) calcule a diferença absoluta $|g - f|$, (b) exiba as imagens e as diferenças com `mm.show`, e (c) plote o histograma das diferenças com `mm.histImg`. Identifique a partir de qual k os artefatos (halos e amplificação de ruído) tornam-se visualmente inaceitáveis.

7. (15%) Escolha uma imagem de raio-X ou tomografia disponível publicamente (ex.: via `mm.read` de URL) e projete um pipeline de pré-processamento com pelo menos 4 etapas sequenciais, justificando cada escolha com base nos conceitos do capítulo. Exiba com `mm.show` em grade: imagem original, cada etapa intermediária e o resultado final com seus histogramas (`mm.histImg`).

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de operações de intensidade, histograma, convolução e filtragem espacial.
- Szeliski (2022) para a visão computacional e aplicações práticas de filtragem.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.



3.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f"\u2705 Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.0 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP03_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:


```

codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP03_01").run_code(codigo)

```

3.12.1 EP03_01 + Adição Saturada de Constante

Em sistemas de vigilância por vídeo, câmeras em ambientes com iluminação variável produzem imagens subexpostas. O ajuste de brilho por **adição saturada de uma constante** é a operação mais simples para correção imediata, sendo aplicada em tempo real nos *chips* de câmeras embarcadas e em *pipelines* de pré-processamento de robôs móveis.

Ver na ?@fig-03-sim-adicao uma simulação deste EP.

3.12.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Constante:** Ler o inteiro k (valor a ser somado).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \text{clip}(p + k)$$

5. **Saída:** Exibir a matriz resultante com dimensões $L \times C$.

3.12.1.2 📌 Restrições Computacionais

- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Tipo:** O resultado final deve ser inteiro (sem casas decimais).
- **k pode ser negativo:** valores negativos escurecem a imagem; positivos clareiam.

3.12.1.3 🧠 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
$k > 0$	Inteiro	Clareia a imagem; pixels próximos de 255 saturam em branco
$k < 0$	Inteiro	Escurece a imagem; pixels próximos de 0 saturam em preto
$k = 0$	Inteiro	Imagem inalterada

3.12.1.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .

- Linha 2: Inteiro C .
- Linha 3: Inteiro k .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz transformada em L linhas e C colunas, valores inteiros separados por espaço.

3.12.1.5 Exemplos

Entrada	Saída	Observação
2	50 150 250	Saturação em 255 nos pixels altos
3	255 255 255	
50		
0 100 200		
210 240 255		
1	0 0 170 225	Saturação em 0 nos pixels baixos
4		
-30		
0 20 200 255		

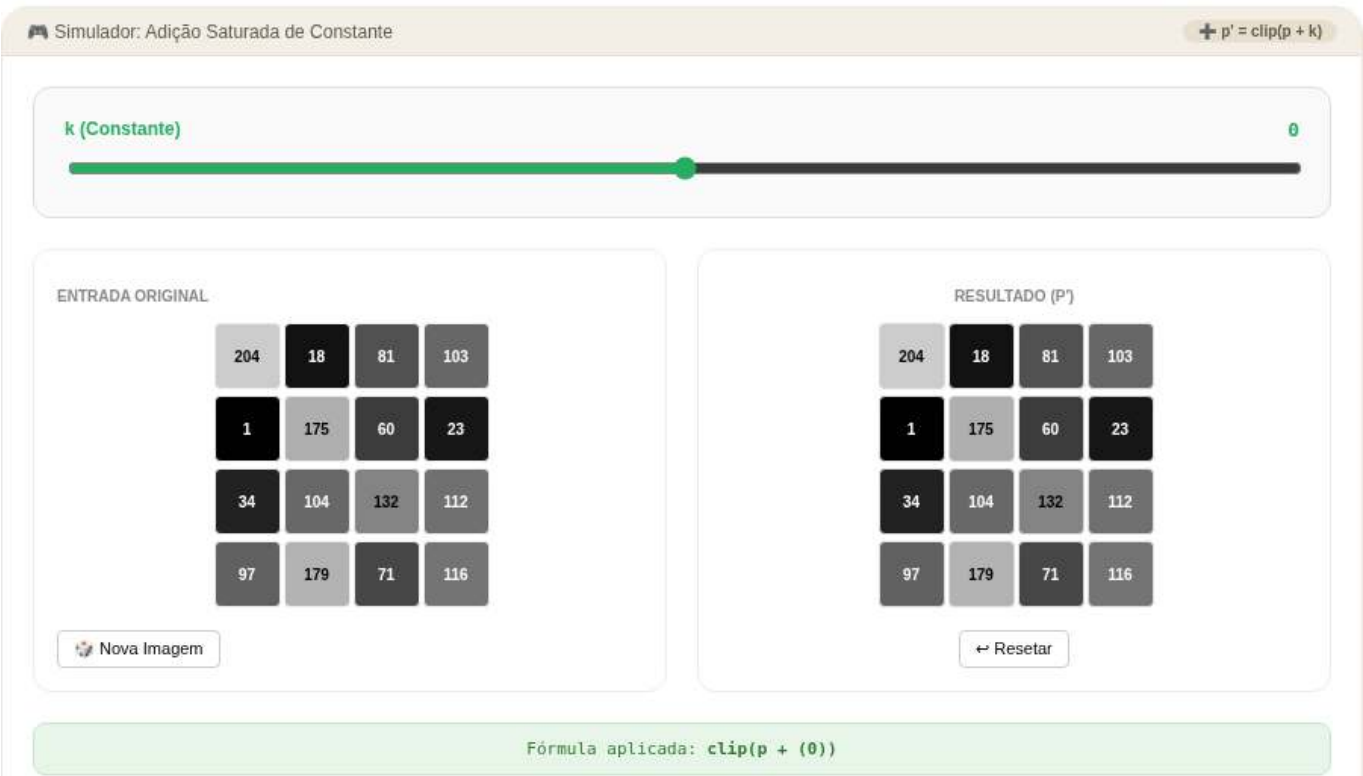


Figura 3.26: Simulador: Adição Saturada de Constante

```
%%writefile EP03_01.py
# Código Python

Writing EP03_01.py

TestSuite("EP03_01.py").run()
```

3.12.2 EP03_02 Alpha *Blending* de Duas Imagens

Em medicina nuclear, imagens de diferentes modalidades (tomografia computadorizada e ressonância magnética) são fundidas para auxiliar no diagnóstico. A **mistura ponderada** (*alpha blending*) é a operação fundamental desse processo, permitindo ao radiologista controlar interativamente o peso de cada modalidade na imagem exibida.

Ver na Figure 3.27 uma simulação deste EP.

3.12.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real $\alpha \in [0, 1]$.
3. **Dados:** Ler os valores inteiros da matriz f_1 (imagem 1) e em seguida da matriz f_2 (imagem 2).
4. **Mapeamento:** Para cada posição (i, j) , calcular:

$$g(i, j) = \text{clip}(\text{round}(\alpha \cdot f_1(i, j) + (1 - \alpha) \cdot f_2(i, j)))$$

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.2.2 Restrições Computacionais

- **Arredondamento:** Aplicar **round** antes da conversão para inteiro.
- **Saturação:** Confinar ao intervalo $[0, 255]$ com $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operação em float:** Realize a operação em ponto flutuante antes de arredondar.

3.12.2.3 Fundamentação Teórica

Valor de α	Resultado
$\alpha = 1.0$	Apenas f_1
$\alpha = 0.5$	Média aritmética de f_1 e f_2
$\alpha = 0.0$	Apenas f_2

3.12.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real α .
- Linhas seguintes: Elementos de f_1 (L linhas com C valores cada).
- Linhas seguintes: Elementos de f_2 (L linhas com C valores cada).

Saída:

- Matriz resultante $L \times C$.

3.12.2.5 📌 Exemplos

Entrada	Saída	Observação
1 3 0.5 0 100 200 100 200 50	50 150 125	Média entre as duas imagens
1 3 1.0 10 20 30 90 80 70	10 20 30	Apenas f_1 (alpha=1)



Figura 3.27: Simulador: Alpha *Blending* de Duas Imagens

```
%%writefile EP03_02.py
# Código Python

Writing EP03_02.py

TestSuite("EP03_02.py").run()
```

3.12.3 EP03_03 Inversão de Imagem (Negativo Fotográfico)

Em radiologia, as imagens de raio-X são tradicionalmente visualizadas em negativo: ossos aparecem em preto sobre fundo branco. A operação de **negativo fotográfico** é aplicada rotineiramente em PACS (*Picture Archiving and Communication Systems*) para facilitar a detecção de fraturas e densidades ósseas.

Ver na Figure 3.28 uma simulação deste EP.

3.12.3.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original.
3. **Mapeamento:** Para cada pixel p , calcular o negativo:

$$p' = 255 - p$$

4. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.3.2 Restrições Computacionais

- **Sem *clipping* necessário:** O resultado de $255 - p$ com $p \in [0, 255]$ é sempre $\in [0, 255]$.
- **Tipo inteiro:** Saída deve ser valores inteiros.
- **Equivalência lógica:** A operação é idêntica ao NOT bit a bit (`mm.bnot`) em imagens de 8 bits.

3.12.3.3 Fundamentação Teórica

Pixel Original p	Pixel Negativo p'	Observação
0 (preto)	255 (branco)	Inversão total
128 (cinza médio)	127 (cinza médio)	Valor central
255 (branco)	0 (preto)	Inversão total

3.12.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz negativa em L linhas e C colunas.

3.12.3.5 Exemplos

Entrada	Saída	Observação
140 128 200 255	255 127 55 0	Inversão de cada pixel
2 2 10 20 30 40	245 235 225 215	Matriz 2x2 invertida

```
%%writefile EP03_03.py
# Código Python
```

```
Writing EP03_03.py
```

```
TestSuite("EP03_03.py").run()
```



Figura 3.28: Simulador: Inversão de Imagem (Negativo Fotográfico)

3.12.4 EP03_04 Equalização de Histograma (L bits)

Em imagens de satélite de sensoriamento remoto, a variação de iluminação ao longo do dia produz imagens de baixo contraste. A **equalização de histograma** é aplicada automaticamente em satélites como o Landsat para redistribuir os tons, revelando detalhes de vegetação, relevo e zonas urbanas invisíveis na imagem original.

Ver na Figure 3.29 uma simulação deste EP.

3.12.4.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e B (número de bits, com $L_{\max} = 2^B$).
2. **Dados:** Ler a matriz de pixels f com valores em $[0, 2^B - 1]$.
3. **Histograma:** Calcular $h[k] = \text{número de pixels com intensidade } k$, para $k = 0 \dots 2^B - 1$.
4. **Probabilidade:** $p[k] = h[k] / (L \cdot C)$.
5. **CDF:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$; função de distribuição acumulada.
6. **LUT:** $\text{lut}[k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$; *Look-Up Table* (tabela de consulta).
7. **Aplicação:** $g[i, j] = \text{lut}[f[i, j]]$.
8. **Saída:** Exibir a matriz equalizada $L \times C$.

3.12.4.2 Restrições Computacionais

- **Arredondamento:** Usar arredondamento matemático (**round**) na LUT.
- **Bits:** O número de níveis é 2^B (ex.: $B = 3 \Rightarrow 8$ níveis, $B = 8 \Rightarrow 256$ níveis).
- **CDF acumulada:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$, com $\text{cdf}[2^B - 1] = 1.0$.

3.12.4.3 Fundamentação Teórica

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow \text{nº pixels com intensidade } k$

Etapa	Operação	Fórmula
2	Probabilidade	$p[k] = h[k]/(L \cdot C)$
3	CDF	$\text{cdf}[k] = \sum_{j=0}^k p[j]$
4	LUT	$\text{lut}[\text{Look} - \text{UpTable}(\text{tabeladeconsulta})k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$
5	Aplicação	$g[i, j] = \text{lut}[f[i, j]]$

3.12.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro B (número de bits).
- Linhas seguintes: Elementos inteiros da matriz.

Saída:

- Matriz equalizada em L linhas e C colunas.

3.12.4.5 Exemplos

Entrada	Saída	Observação
5	5 7 1 5 7	Exemplo 3 bits do capítulo
5	7 5 5 7 5	
3	1 5 7 5 1	
3 4 2 3 4	5 7 5 1 5	
4 3 3 4 3	7 5 1 5 7	
2 3 4 3 2		
3 4 3 2 3		Histograma bimodal extremo
4 3 2 3 4		
1	0 0 7 7	
4		
3		
0 0 7 7		

```
%%writefile EP03_04.py
# Código Python
```

```
Writing EP03_04.py
```

```
TestSuite("EP03_04.py").run()
```

3.12.5 EP03_05 Aplicação de Máscara AND Binária

Em sistemas de inspeção industrial por visão computacional, é necessário isolar regiões de interesse (ROI) em imagens de peças para verificar defeitos de fabricação. A operação **AND bit a bit com uma máscara**

Simulador: Equalização de Histograma

Escolha o número de bits e gere uma imagem para ver a equalização em tempo real.

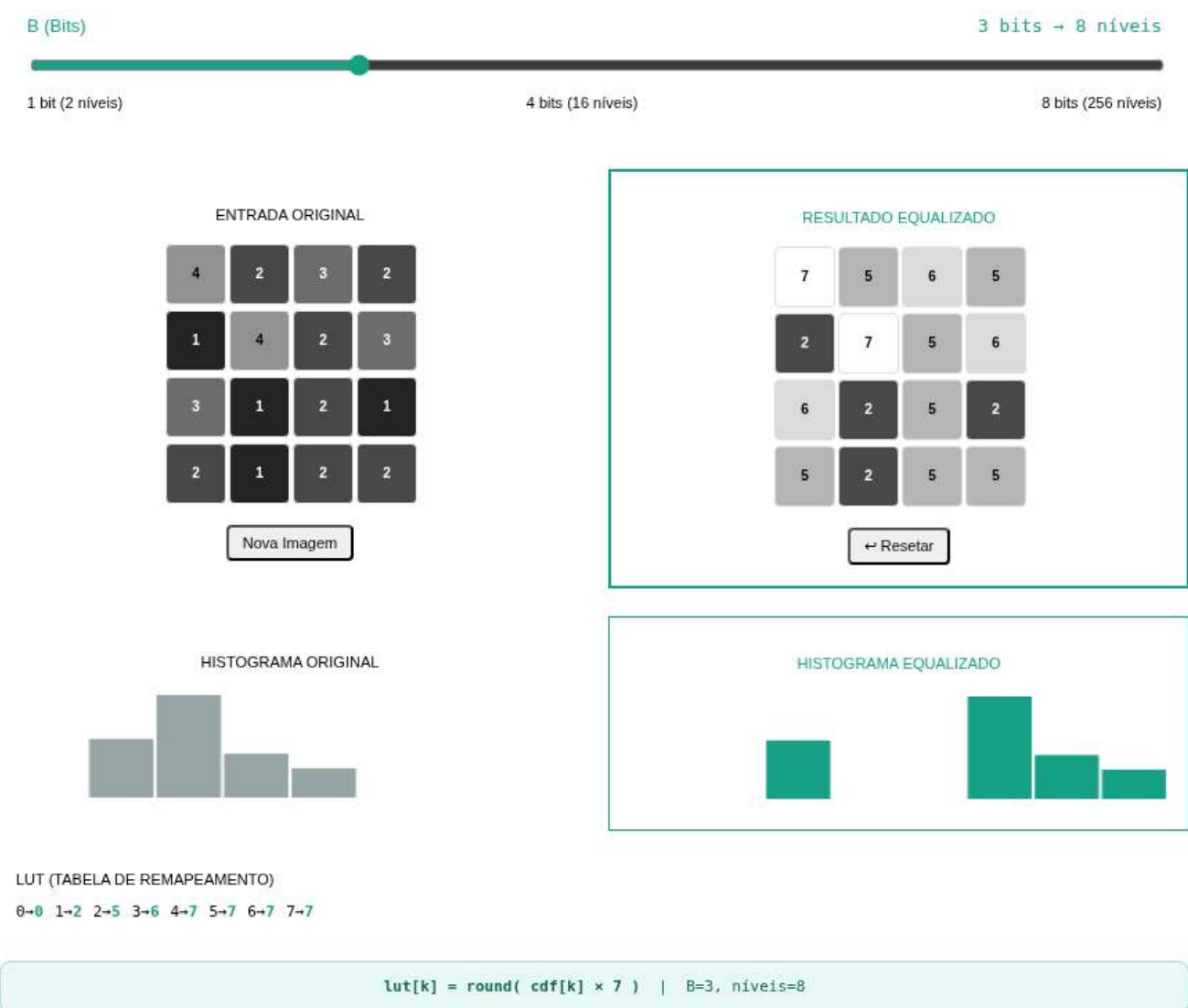


Figura 3.29: Simulador: Equalização de Histograma (L bits)

binária é o mecanismo fundamental para recortar exatamente a área de inspeção, zerando todos os pixels fora dela.

Ver na Figure 3.30 uma simulação deste EP.

3.12.5.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f (valores $\in [0, 255]$).
3. **Máscara:** Ler a matriz binária m (valores: apenas 0 ou 255).
4. **Mapeamento:** Para cada pixel (i, j) , aplicar o AND bit a bit:

$$g(i, j) = f(i, j) \text{ AND } m(i, j)$$

onde $255 = 11111111$ e $0 = 00000000$ em binário.

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.5.2 Restrições Computacionais

- **AND com 255:** $p \text{ AND } 255 = p$ (todos os bits preservados).
- **AND com 0:** $p \text{ AND } 0 = 0$ (todos os bits zerados).
- **Máscara:** Os únicos valores possíveis na máscara são 0 e 255.
- **Implementação:** Em Python, o AND bit a bit entre inteiros usa o operador `&`.

3.12.5.3 Fundamentação Teórica

Pixel f	Máscara m	Resultado $f \text{ AND } m$
qualquer v	255 (11111111)	v (preservado)
qualquer v	0 (00000000)	0 (zerado)

3.12.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos de f (L linhas).
- Linhas seguintes: Elementos de m (L linhas com valores 0 ou 255).

Saída:

- Matriz resultante $L \times C$.

3.12.5.5 Exemplos

Entrada	Saída	Observação
2	100 150 0	Máscara seleciona região
3	0 80 120	
100 150 200		
50 80 120		
255 255 0		
0 255 255		Alternado preservado/zerado
1	10 0 30 0	
4		
10 20 30 40		
255 0 255 0		

Simulador de máscara AND binária: aplica operação AND pixel a pixel entre imagem e máscara interativa.

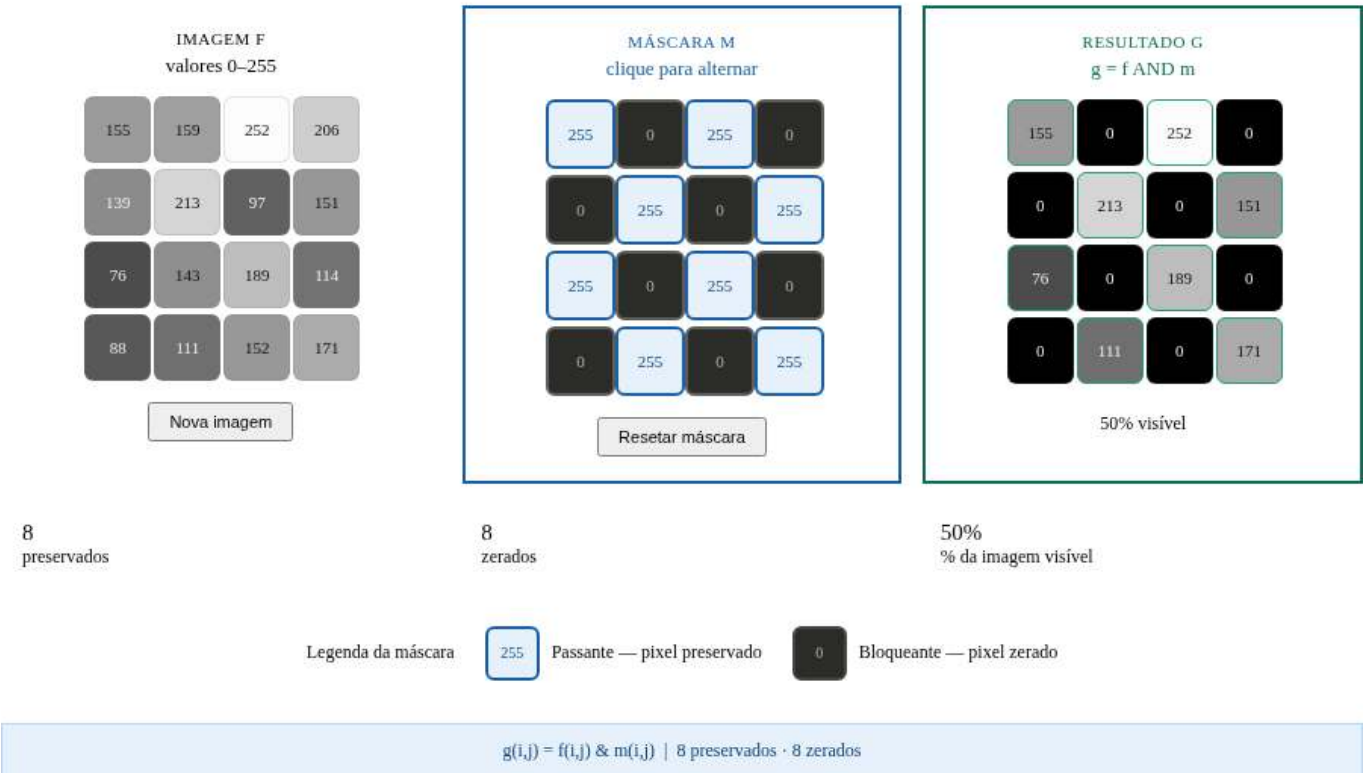


Figura 3.30: Simulador: Aplicação de Máscara AND Binária

```
%%writefile EP03_05.py
# Código Python

Writing EP03_05.py

TestSuite("EP03_05.py").run()
```

3.12.6 EP03_06 Filtro de Média com *Kernel* $N \times N$

Em câmeras de veículos autônomos, imagens capturadas sob chuva ou névoa apresentam ruído gaussiano. O **filtro de média** é amplamente utilizado para sua redução em tempo real, sendo implementado diretamente no **ISP** (*Image Signal Processor*) de sensores **CMOS** (*Complementary Metal-Oxide-Semiconductor*).

Os sensores CMOS são os sensores de imagem usados na maioria das câmeras modernas (*smartphones*, *webcams*, câmeras automotivas etc.). Eles convertem a luz em sinais elétricos, e o ISP processa esses sinais em tempo real — aplicando operações como redução de ruído, balanço de branco e outros ajustes de imagem.

Ver na Figure 3.31 uma simulação deste EP.

3.12.6.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e N (tamanho do *kernel*, sempre ímpar).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro de Média:** Para cada pixel (i, j) **interno** (sem bordas), calcular:

$$g(i, j) = \text{round} \left(\frac{1}{N^2} \sum_{s=-r}^r \sum_{t=-r}^r f(i+s, j+t) \right), \quad r = \lfloor N/2 \rfloor$$

4. **Tratamento de Borda:** Pixels na borda (onde a janela $N \times N$ ultrapassa os limites) devem ser **copiados diretamente** do original sem modificação.
5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.6.2 Restrições Computacionais

- **Raio:** $r = \lfloor N/2 \rfloor$ (metade do *kernel*, inteiro).
- **Pixels internos:** (i, j) com $r \leq i < L - r$ e $r \leq j < C - r$.
- **Arredondamento:** Usar arredondamento matemático antes de converter para inteiro.
- **Sem *clipping*:** A média de valores $\in [0, 255]$ permanece em $[0, 255]$.

3.12.6.3 Fundamentação Teórica

Tamanho N	Coefficiente	Pixels na janela	Efeito
3	$1/9 \approx 0.111$	9	Suave
5	$1/25 = 0.04$	25	Médio
7	$1/49 \approx 0.020$	49	Forte

3.12.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro N (ímpar, $N \geq 3$).
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz filtrada $L \times C$.

3.12.6.5 Exemplos

Entrada	Saída	Observação
3	10 20 30	Apenas borda ($3 \times 3 =$ borda total)
3	40 50 60	
3	70 80 90	
10 20 30		
40 50 60		
70 80 90		
5	0 0 0 0 0	Pixel isolado: todos os 9 pixels internos cuja janela 3×3 inclui o valor 100 recebem $\text{round}(100/9)=11$
5	0 11 11 11 0	
3	0 11 11 11 0	
0 0 0 0 0	0 11 11 11 0	
0 0 0 0 0	0 0 0 0 0	
0 0 100 0 0		
0 0 0 0 0		
0 0 0 0 0		

Simulador de filtro de média com kernel 3×3 ou 5×5 : passe o mouse nas células do resultado para visualizar a janela de vizinhança.

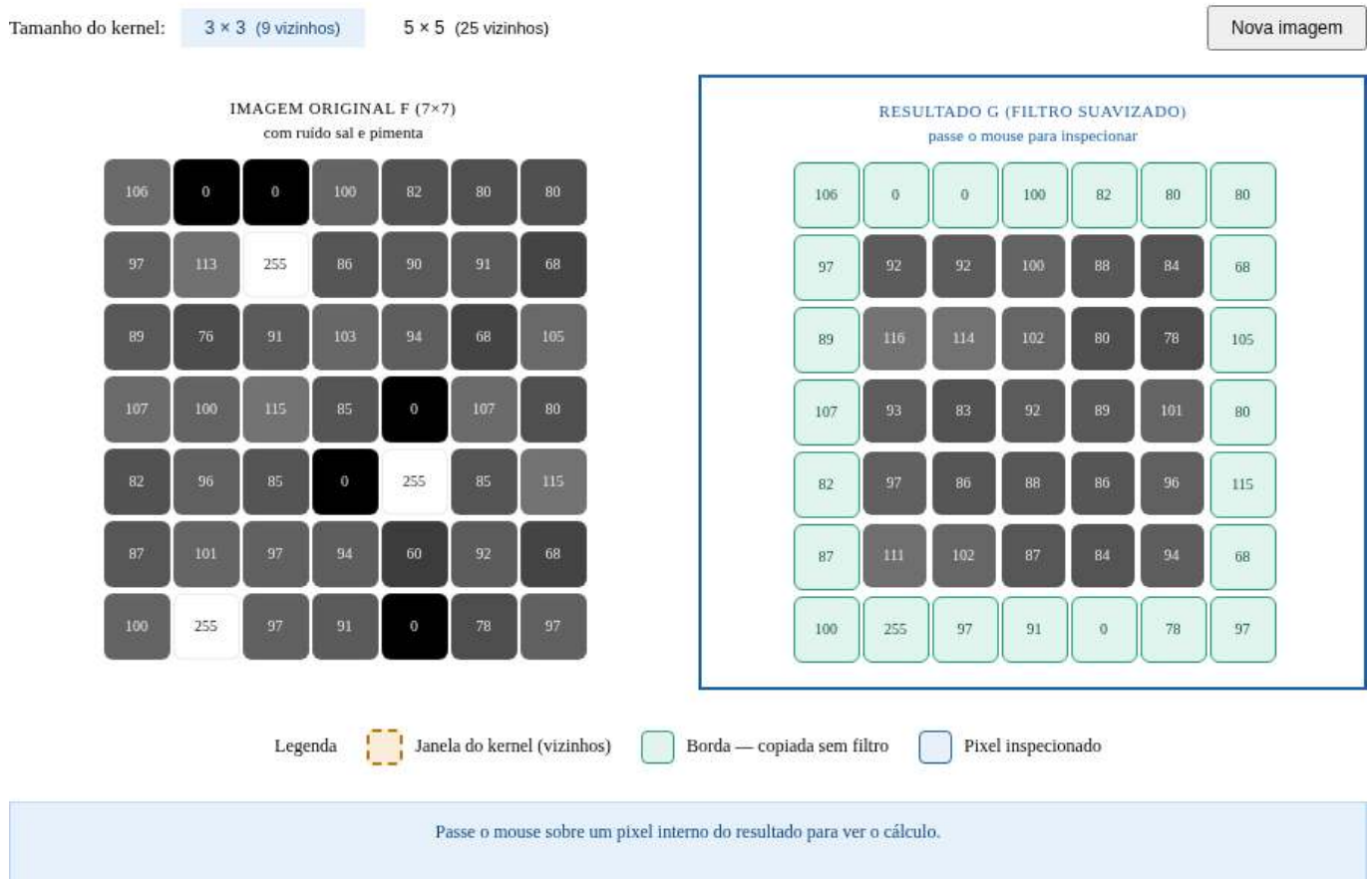


Figura 3.31: Simulador: Filtro de Média com *Kernel* $N \times N$

```
%%writefile EP03_06.py
# Código Python
```



```
Writing EP03_06.py
```

```
TestSuite("EP03_06.py").run()
```

3.12.7 EP03_07 🔍 Operador Laplaciano (w4) para Realce de Bordas

Em tomografias de alta resolução, a nitidez das bordas entre tecidos é crítica para diagnóstico. O **operador Laplaciano** é amplamente utilizado em *pipelines* de pré-processamento de imagens médicas para realçar automaticamente os contornos anatômicos antes da segmentação, evitando intervenção manual do radiologista.

Ver na Figure 3.32 uma simulação deste EP.

3.12.7.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Laplaciano (w4):** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$, calcular:

$$\nabla^2 f(i, j) = f(i - 1, j) + f(i + 1, j) + f(i, j - 1) + f(i, j + 1) - 4 \cdot f(i, j)$$

4. **Realce:** Calcular a imagem realçada:

$$g(i, j) = \text{clip}(f(i, j) - \nabla^2 f(i, j))$$

5. **Borda:** Pixels na borda são copiados diretamente: $g(i, j) = f(i, j)$.
6. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.7.2 📌 Restrições Computacionais

- **Kernel w4:** $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ — apenas 4-vizinhos.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$ aplicado ao resultado do realce.
- **Sem arredondamento:** O Laplaciano usa apenas somas/subtrações de inteiros.

3.12.7.3 🧠 Fundamentação Teórica

Região	$\nabla^2 f$	Efeito do Realce
Uniforme	≈ 0	Sem alteração
Borda crescente	< 0	Pixel clareado
Borda decrescente	> 0	Pixel escurecido

3.12.7.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .

- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.7.5 Exemplos

Entrada	Saída	Observação
3 3 0 0 0 0 100 0 0 0 0	0 0 0 0 255 0 0 0 0	Pico isolado: $\text{lap}=-400$, $g=100-(-400)=500 \rightarrow \text{clip}=255$
3 3 50 50 50 50 50 50 50 50 50	50 50 50 50 50 50 50 50 50	Região uniforme: Laplaciano=0, sem alteração

```
%%writefile EP03_07.py
# Código Python
```

```
Writing EP03_07.py
```

```
TestSuite("EP03_07.py").run()
```

3.12.8 EP03_08 Gradiente de Sobel: G_x e G_y

Em robôs exploradores de Marte (como o Perseverance), a detecção de obstáculos é realizada em tempo real por câmeras estereoscópicas. O **operador de Sobel** calcula o gradiente direcional da cena e é utilizado no algoritmo de detecção de bordas para identificar rochas, fissuras e desníveis do terreno que possam comprometer a navegação.

Ver na Figure 3.33 uma simulação deste EP.

3.12.8.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz f .
3. **G_x e G_y :** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:

$$G_x(i, j) = [f(i - 1, j + 1) + 2f(i, j + 1) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i, j - 1) + f(i + 1, j - 1)]$$

$$G_y(i, j) = [f(i + 1, j - 1) + 2f(i + 1, j) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i - 1, j) + f(i - 1, j + 1)]$$

4. **Magnitude:** $|\nabla f(i, j)| = \text{clip}(\text{round}(\sqrt{G_x^2 + G_y^2}))$.

Simulador do operador Laplaciano para realce de bordas: imagem original, laplaciano e resultado.

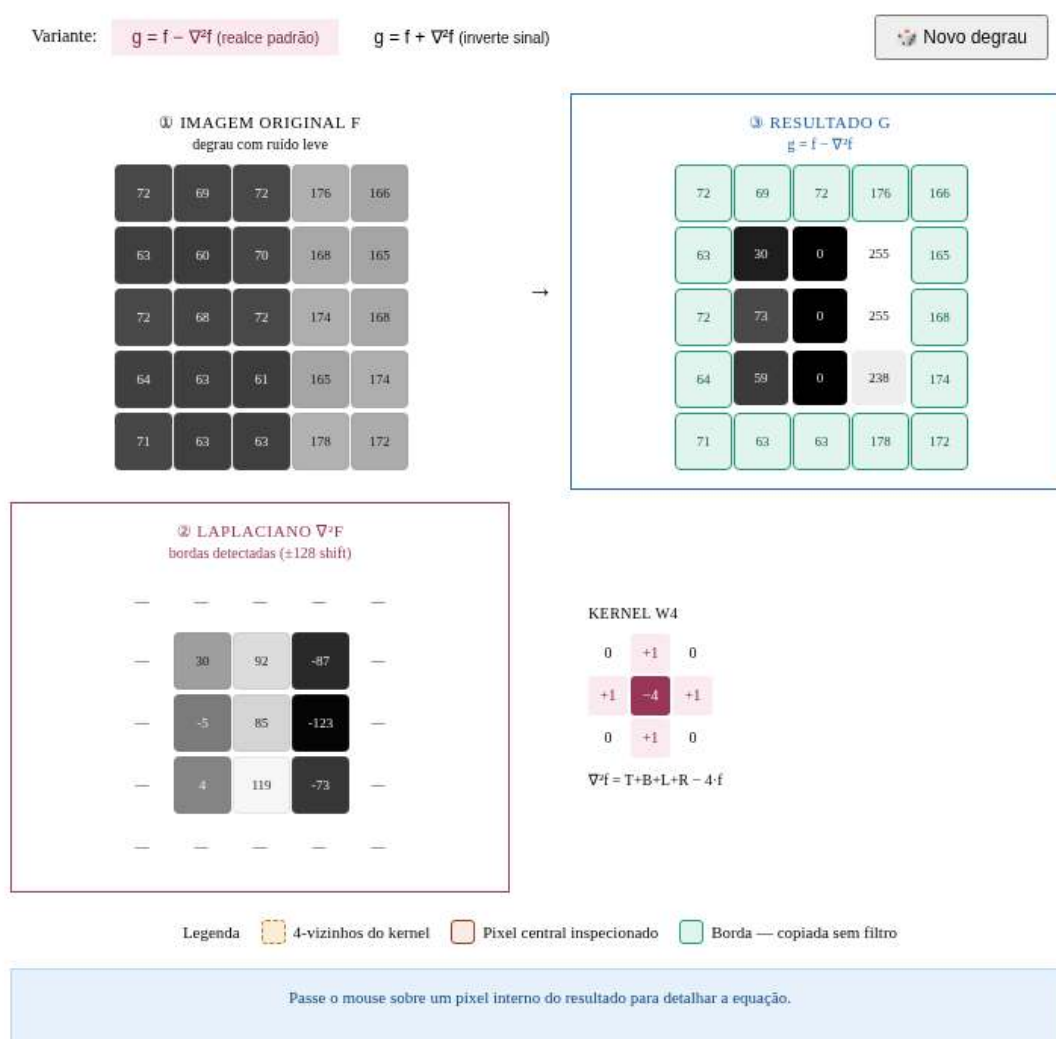


Figura 3.32: Simulador: Operador Laplaciano (w4) para Realce de Bordas

5. **Borda:** Pixels de borda recebem magnitude 0.
6. **Saída:** Exibir a magnitude $L \times C$.

3.12.8.2 Restrições Computacionais

- **Arredondamento:** Aplicar round antes de converter para inteiro.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Raiz quadrada:** Usar $\sqrt{G_x^2 + G_y^2}$ (não a aproximação $|G_x| + |G_y|$).

3.12.8.3 Fundamentação Teórica

Operador	Detecta	Coefficientes diagonais
G_x	Bordas verticais	± 1
G_y	Bordas horizontais	± 1
$ \nabla f $	Todas as bordas	Combinado

3.12.8.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Magnitude do gradiente, matriz $L \times C$.

3.12.8.5 Exemplos

Entrada	Saída	Observação
3	0 0 0	Imagem nula: gradiente zero
3	0 0 0	
0 0 0	0 0 0	
0 0 0		
0 0 0		
3	0 0 0	Borda vertical central: Gx alto
3	0 255 0	
0 0 255	0 0 0	
0 0 255		
0 0 255		

```
%%writefile EP03_08.py
# Código Python
```

Writing EP03_08.py

```
TestSuite("EP03_08.py").run()
```

Simulador do gradiente de Sobel: imagem original, Gx, Gy e magnitude do gradiente.



Figura 3.33: Simulador: Gradiente de Sobel: Gx e Gy

3.12.9 EP03_09 Filtro da Mediana 3×3

Imagens de radar de abertura sintética (SAR) usadas em monitoramento ambiental e militar sofrem de um tipo específico de ruído chamado *speckle*, que possui características similares ao ruído sal e pimenta. O **filtro da mediana** é o método padrão de remoção desse ruído porque preserva as bordas das estruturas enquanto elimina os pontos espúrios.

Ver na Figure 3.34 uma simulação deste EP.

3.12.9.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro da Mediana 3×3:** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:
 - Coletar os 9 pixels da vizinhança 3×3 : $\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$.
 - Ordenar os 9 valores em ordem crescente.
 - Atribuir $g(i, j)$ ao valor central (posição índice 4, considerando índice 0).

$$g(i, j) = \text{mediana}\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$$

4. **Borda:** Copiar diretamente: $g(i, j) = f(i, j)$.
5. **Saída:** Exibir a matriz filtrada $L \times C$.

3.12.9.2 Restrições Computacionais

- **Janela:** Sempre $3 \times 3 = 9$ elementos.
- **Mediana:** O elemento central da sequência ordenada (índice 4 de 0 a 8).
- **Sem clipping:** A mediana de valores em $[0, 255]$ permanece em $[0, 255]$.
- **Não linear:** O filtro mediana não pode ser expresso como convolução linear.

3.12.9.3 Fundamentação Teórica

Ruído	Filtro de Média	Filtro de Mediana
Sal e pimenta (0 ou 255)	Espalha o ruído	Remove sem distorcer bordas
Gaussiano	Reduz eficazmente	Reduz parcialmente

3.12.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Matriz filtrada $L \times C$.

3.12.9.5 📌 Exemplos

Entrada	Saída	Observação
3	100 100 100	Ponto preto eliminado: mediana de $8 \times 100 + 1 \times 0 = 100$
3	100 100 100	
100 100 100	100 100 100	
100 0 100		
100 100 100		
3	50 50 50	Ponto branco (sal) eliminado
3	50 50 50	
50 50 50	50 50 50	
50 255 50		
50 50 50		

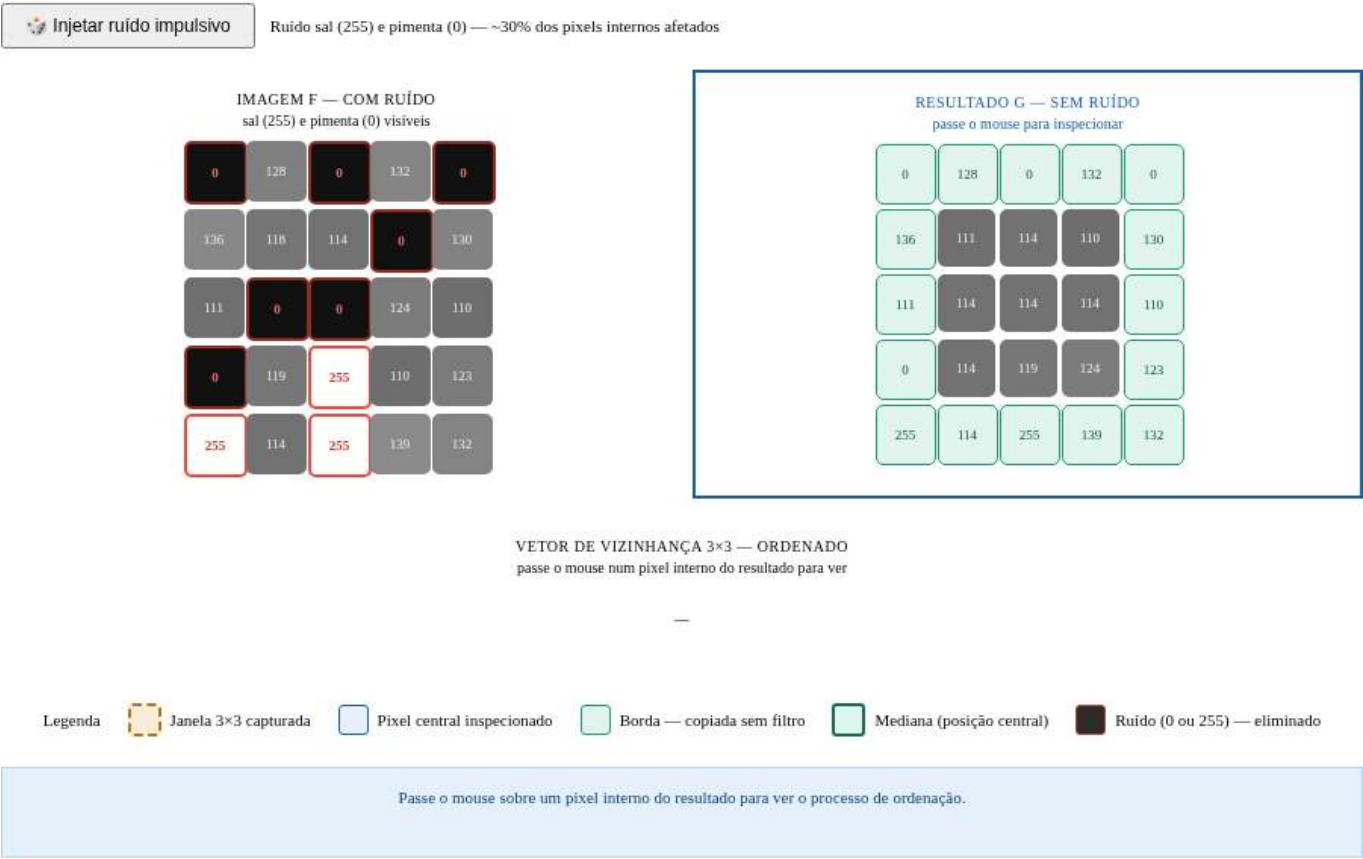


Figura 3.34: Simulador: Filtro da Mediana 3×3

```
%%writefile EP03_09.py
# Código Python

Writing EP03_09.py

TestSuite("EP03_09.py").run()
```

3.12.10 EP03_10 *Unsharp Masking* (USM)

Em sistemas de digitalização de documentos históricos e obras de arte, a nitidez das imagens é fundamental para leitura de textos manuscritos e detalhes ornamentais. O *Unsharp Masking* (USM) é o algoritmo de realce de nitidez padrão utilizado em *scanners* profissionais e softwares como Adobe Photoshop, controlado pelo parâmetro k que determina a intensidade do realce.

Ver na Figure 3.35 uma simulação deste EP.

3.12.10.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real k (intensidade do realce, $k \geq 0$).
3. **Dados:** Ler a matriz de pixels f .
4. **Suavização:** Calcular $\bar{f}$ com filtro de média 3×3 (apenas pixels internos; bordas mantidas):

$$\bar{f}(i, j) = \frac{1}{9} \sum_{s=-1}^1 \sum_{t=-1}^1 f(i+s, j+t)$$

5. **Máscara de alta frequência:** $m(i, j) = f(i, j) - \bar{f}(i, j)$.
6. **Realce USM:** Para cada pixel interno:

$$g(i, j) = \text{clip}(\text{round}(f(i, j) + k \cdot m(i, j)))$$

7. **Borda:** $g(i, j) = f(i, j)$ (cópia direta).
8. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.10.2 Restrições Computacionais

- **Arredondamento:** Aplicar `round` antes do clipping.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operações em float:** Calcular $\bar{f}$ e m em ponto flutuante antes de arredondar o resultado final.
- $k = 0$: Sem realce — a saída é idêntica à entrada (exceto pelas bordas).

3.12.10.3 Fundamentação Teórica

Etapas	Operação	Descrição
1	$\bar{f} = f * \frac{1}{9} \mathbf{1}_{3 \times 3}$	Suavização (baixas frequências)
2	$m = f - \bar{f}$	Máscara (altas frequências)
3	$g = \text{clip}(\text{round}(f + k \cdot m))$	Realce ponderado

3.12.10.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real k .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.10.5 Exemplos

Entrada	Saída	Observação
3	100 100 100	k=0: sem realce
3	100 100 100	
0.0	100 100 100	
100 100 100		
100 100 100		
100 100 100		
3	50 50 50	k=1: pixel central realçado e saturado
3	50 255 50	
1.0	50 50 50	
50 50 50		
50 200 50		
50 50 50		

```
%%writefile EP03_10.py
# Código Python
```

Writing EP03_10.py

```
TestSuite("EP03_10.py").run()
```

Simulador de Unsharp Masking (USM): visualização do pipeline com imagem original, versão desfocada, máscara de alta frequência e resultado realçado.

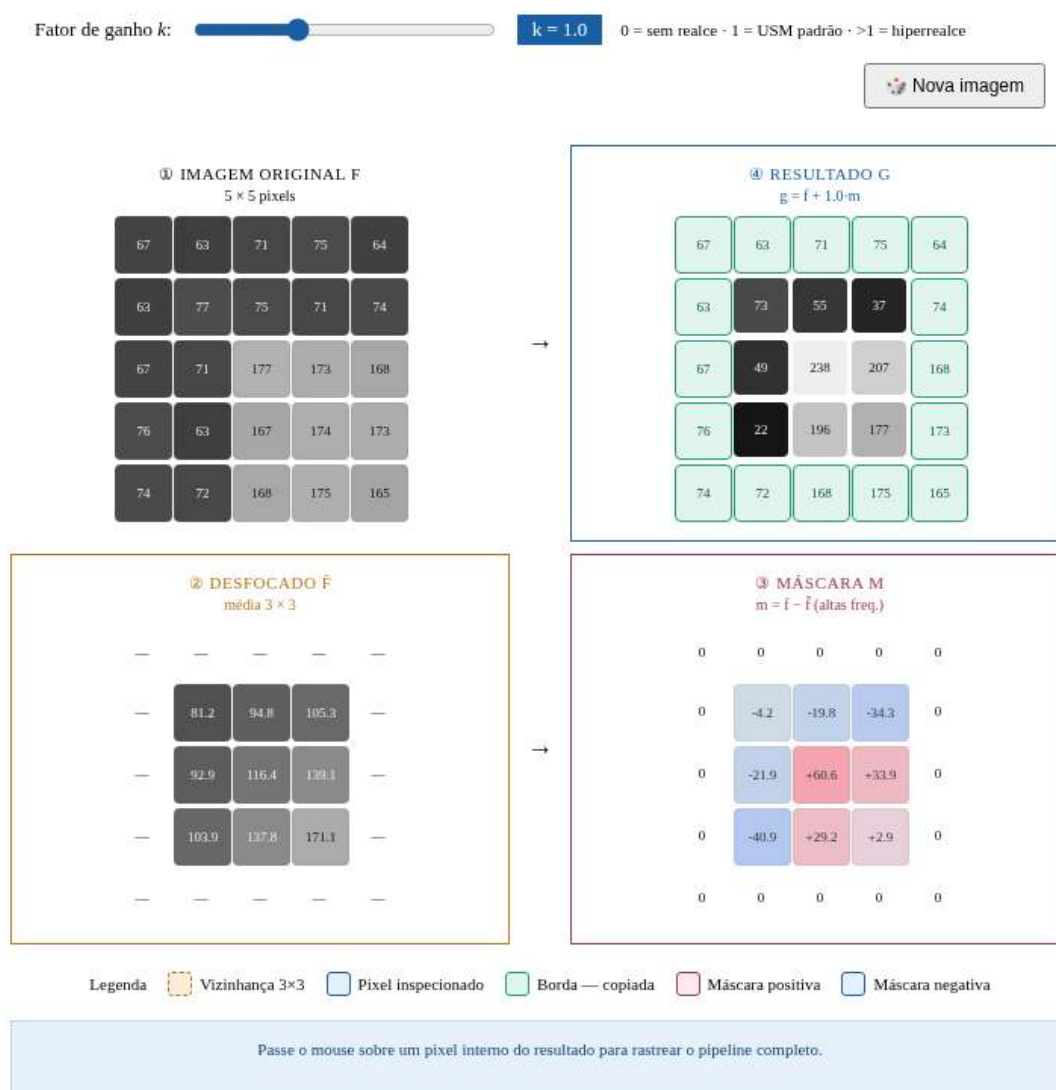


Figura 3.35: Simulador: *Unsharp Masking* (USM)

Capítulo 4

Morfologia Matemática e Segmentação de Imagens



Este capítulo apresenta dois temas fundamentais do Processamento Digital de Imagens (PDI): a **morfologia matemática** e a **segmentação de imagens**. A morfologia matemática fornece um arcabouço teórico baseado na teoria dos conjuntos para analisar, refinar e quantificar a forma de objetos em imagens binárias e em tons de cinza, por meio de operadores fundamentais como **erosão** e **dilatação**. A segmentação, por sua vez, tem como objetivo particionar a imagem em regiões de interesse, separando objetos do fundo e produzindo representações adequadas para análise e interpretação.

O capítulo inicia com a **limiarização**, uma das técnicas mais importantes de segmentação, introduzindo o método automático de **Otsu** e revisitando a análise de histogramas por meio da variância interclasses, apresentada no Capítulo 1. Em seguida, são estudados os principais operadores da morfologia matemática, incluindo erosão, dilatação, abertura, fechamento e reconstrução morfológica, que permitem refinar máscaras binárias e preservar estruturas relevantes dos objetos. Por fim, são apresentadas técnicas de segmentação baseada em regiões, como a **rotulação de componentes conexos**, a **transformada de distância** e o algoritmo *watershed* baseado em marcadores, culminando na extração de descritores geométricos e na geração de *bounding boxes* compatíveis com sistemas modernos de detecção de objetos.

4.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Aplicar limiarização:** Compreender o critério automático de Otsu por maximização da variância interclasses (σ_B^2) e selecionar estratégias adequadas de pré-processamento para facilitar a segmentação;
- **Dominar morfologia binária:** Compreender e aplicar erosão ($A \ominus B$) e dilatação ($A \oplus B$) como operadores fundamentais, derivando abertura ($A \circ B$), fechamento ($A \bullet B$) e operações baseadas em reconstrução morfológica, como `mm.clohole` e `mm.edgeoff`;
- **Aplicar morfologia em tons de cinza:** Utilizar gradiente morfológico e filtros *top-hat* para realce e análise de estruturas locais;
- **Rotular componentes conexos:** Identificar e separar regiões conectadas em imagens binárias por meio de algoritmos de rotulação;
- **Aplicar transformada de distância:** Interpretar e calcular distâncias ao fundo utilizando abordagens morfológicas e métricas geométricas;
- **Segmentar por regiões:** Construir *pipelines* de segmentação baseados em marcadores utilizando Transformada de Distância e o algoritmo *watershed*;
- **Extraír descritores geométricos:** Calcular propriedades como área, perímetro, centróide, circularidade e *bounding boxes* por meio de `cv2.connectedComponentsWithStats` e `cv2.findContours`;
- **Relacionar PDI e visão computacional:** Compreender como descritores extraídos por segmentação podem ser convertidos para formatos utilizados por detectores modernos, como YOLO.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.2).

4.2 Limiarização

A **limiarização** (*thresholding*) é uma das formas mais simples e eficientes de segmentação de imagens. Seu objetivo é classificar cada pixel em duas classes de intensidade, normalmente associadas a *objeto* e *fundo*:

$$g(x, y) = \begin{cases} 255, & \text{se } f(x, y) > T \\ 0, & \text{caso contrário} \end{cases} \quad (4.1)$$

em que $f(x, y)$ representa a intensidade do pixel na imagem original e $g(x, y)$ a imagem binária resultante.

A escolha do limiar T é importante para a qualidade da segmentação. O método de **Otsu** determina automaticamente o limiar ótimo ao maximizar a **variância interclasses** σ_B^2 e definida por:

$$\sigma_B^2(T) = w_0(T) w_1(T) [\mu_0(T) - \mu_1(T)]^2 \quad (4.2)$$

em que:

- $w_0(T)$ e $w_1(T)$ são as probabilidades acumuladas das classes fundo e objeto;
- $\mu_0(T)$ e $\mu_1(T)$ são as médias de intensidade dessas classes;
- $\sigma_B^2(T)$ representa a variância interclasses para um dado limiar T .

O método funciona melhor quando o histograma apresenta dois grupos de intensidades relativamente separados. Para isso, o algoritmo avalia todos os limiares possíveis da imagem — tipicamente no intervalo $[0, 255]$ para imagens de 8 bits — e seleciona o valor que maximiza a variância entre classes, denotada por σ_B^2 :

$$T^* = \arg \max_{T \in [0, 255]} \sigma_B^2(T)$$

i Otsu assume histogramas bimodais

O método de Otsu produz melhores resultados quando o histograma apresenta dois picos bem definidos (*bimodalidade*), correspondentes ao fundo e ao objeto. Quanto maior a separação entre esses picos e mais pronunciado o máximo de σ_B^2 , mais confiável tende a ser o limiar obtido.

Em imagens com iluminação não uniforme ou múltiplas regiões de intensidade, técnicas de **limiarização adaptativa** — nas quais o limiar é calculado localmente — costumam produzir segmentações mais robustas.

O índice B em σ_B^2 significa *between classes* (*entre classes*). Assim, σ_B^2 representa a **variância entre as classes** (*between-class variance*).

4.2.1 Imagem de Moedas

A imagem utilizada para praticar a segmentação é uma fotografia de coleção de moedas de diferentes países e épocas (Figure 4.1). Crédito: GAZI.MD.AHAD (CC BY-SA 4.0). Ela apresenta objetos circulares com bordas bem definidas, sendo ideal para demonstrar limiarização, operadores morfológicos, transformada de distância, *watershed* e descritores de forma.

```
import os

url      = "https://upload.wikimedia.org/wikipedia/commons/2/25/GAZI.MD.AHAD_11.jpg"
caminho = "imagens/coins.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_coins_color = np.array(img_obj)
img_coins_gray  = mm.gray(img_coins_color)

print(f"Dimensões [y,x,c]: {img_coins_color.shape}")
mm.show(img_coins_color, scale=30)
```

Dimensões [y,x,c]: (2560, 1920, 3)



Figura 4.1: Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).

4.2.2 Escolha do Pré-processamento para o Otsu

A qualidade do método de Otsu depende diretamente de quão **bimodal** é o histograma da imagem de entrada. A imagem original das moedas apresenta iluminação não uniforme e moedas escuras (cobre oxidado) com intensidades próximas às do fundo escuro, tornando o histograma pouco bimodal.

Para minimizar essas limitações, serão avaliadas duas técnicas clássicas de pré-processamento, apresentadas no capítulo anterior, aplicadas antes da etapa de limiarização. A Table 4.1 resume as características de cada abordagem. Essas técnicas buscam aumentar a separação entre objeto e fundo, tornando o histograma mais próximo de uma distribuição bimodal.

Tabela 4.1: Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.

Técnica	O que faz	Quando usar
CLAHE	Equalização de histograma adaptativa local	Baixo contraste global ou regional
Gaussiano	Suavização por convolução com gaussiana	Ruído de alta frequência (textura do fundo)

A função `cv2.createCLAHE(clipLimit, tileGridSize)` divide a imagem em blocos e aplica equalização de histograma em cada um deles, limitando a amplificação de ruído por meio do parâmetro `clipLimit`. Já o filtro Gaussiano (`cv2.GaussianBlur`) suaviza texturas finas que poderiam criar falsos picos no histograma.

Para comparar objetivamente qual pré-processamento produz a melhor entrada para o método de Otsu, serão apresentados, para cada versão da imagem, a própria imagem, o histograma com o limiar ótimo T^* destacado, a curva $\sigma_B^2(T)$ e o resultado da binarização.

i Critério de comparação

A versão que apresentar o maior valor de σ_B^2 em seu pico fornece a melhor separação entre as classes de fundo e objeto e, conseqüentemente, a melhor entrada para o método de Otsu (Figure 4.2).

```
import cv2, io

def otsu_criterio(img):
    h=mm.hist(img); p=h/h.sum(); # histograma e probabilidades
    sigma2=np.zeros(len(p))
    for T in range(1,len(p)): # percorre limiares
        w0,w1=p[:T].sum(),p[T:].sum() # probabilidades das classes
        if w0*w1==0: continue # evita divisão por zero
        mu0=(np.arange(T)*p[:T]).sum()/w0 # média fundo
        mu1=(np.arange(T,len(p))*p[T:]).sum()/w1 # média objeto
        sigma2[T]=w0*w1*(mu0-mu1)**2 #  $\sigma_B^2(T)$ 
    return sigma2,np.argmax(sigma2) # curva e T ótimo

def fig2img(fig):
    b=io.BytesIO(); fig.savefig(b,format='png',dpi=100) # figura → buffer
    plt.close(fig); b.seek(0)
    return np.array(plt.imread(b)) # buffer → array

def plot_curve(y,T,title,ylabel,color):
    fig,ax=plt.subplots(figsize=(4,3))
    ax.plot(y,color=color) if ylabel==" $\sigma_B^2$ " else ax.bar(range(len(y)),y,color=color,width=1)
    ax.axvline(T,color='red',lw=2,label=f"T*={T}") # limiar ótimo
    ax.set(xlabel="T" if ylabel==" $\sigma_B^2$ " else "Intensidade",ylabel=ylabel)
    ax.legend(fontsize=8); plt.tight_layout()
```

```

return fig2img(fig)

# Pré-processamentos
clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))
img_clahe = clahe.apply(img_coins_gray)
img_gauss = cv2.GaussianBlur(img_clahe,(5,5),0)
imgs0=[("Original",img_coins_gray),
        ("CLAHE",img_clahe),
        ("CLAHE+Gauss",img_gauss)]

# Tabela comparativa
print(f"{'Versão':<18}{'T*':>6}{'2B pico':>14}")
print("-"*40)

imgs,titles=[],[]
for nome,img in imgs0:
    sigma2,T=otsu_criterio(img) # calcula 2B(T)
    print(f"{'nome':<18}{'T':>6}{'sigma2[T]:>14.4e}")
    imgs += [
        img, # imagem
        plot_curve(mm.hist(img),T,f"Hist T*={T}", "Freq.", "steelblue"),
        plot_curve(sigma2,T,"2B(T)", "2B", "darkorange"),
        mm.threshold(img) # Otsu final
    ]
    titles += [nome,f"Hist T*={T}", "2B(T)",f"Otsu T*={T}"]

# Exibição final
mm.show(imgs,titles=titles,cols=4,figsize=(12,12),dpi=200)

```

Versão	T*	² B pico
Original	105	1.9437e+03
CLAHE	122	2.4695e+03
CLAHE+Gauss	123	2.4283e+03

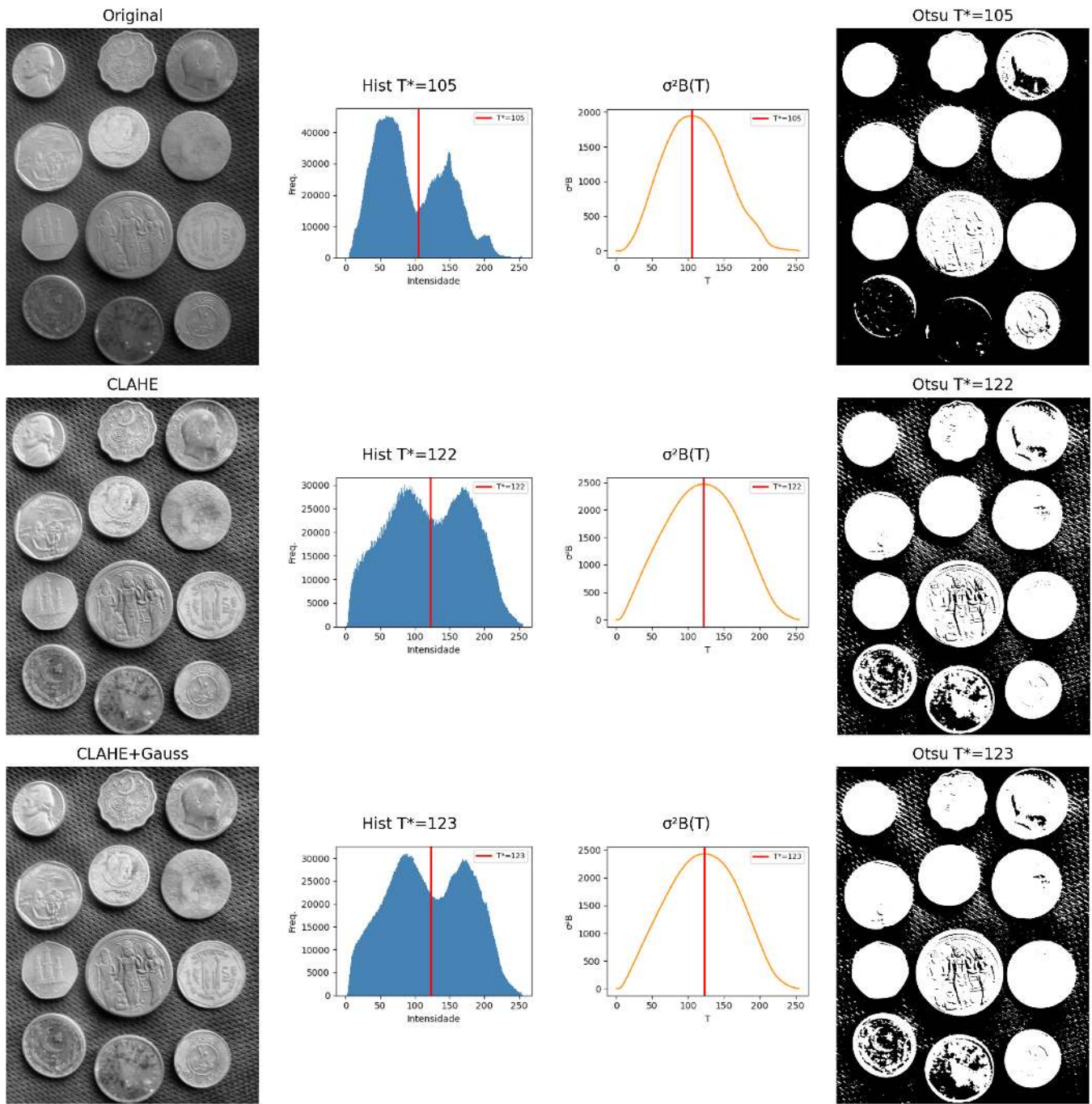


Figura 4.2: Comparação dos pré-processamentos: imagem | histograma+ T^* | $\sigma^2 B(T)$ | Otsu. A melhor separação bimodal indica o limiar mais confiável.

4.2.3 Resultado: CLAHE como Melhor Pré-processamento

A análise da Figure 4.2 indica que o **CLAHE** obteve o maior valor da variância inter-classes ($\sigma_B^2 \approx 2,47 \times 10^3$), com limiar ótimo $T^* = 122$. Embora a combinação **CLAHE+Gaussiano** tenha produzido resultado muito semelhante ($\sigma_B^2 \approx 2,43 \times 10^3$, $T^* = 123$), o critério quantitativo do método de Otsu favorece ligeiramente o uso do CLAHE isoladamente.

Em termos visuais, as imagens binarizadas obtidas com CLAHE e CLAHE+Gaussiano são praticamente equivalentes. A diferença entre as duas abordagens torna-se mais evidente na análise dos histogramas e dos valores de $\sigma_B^2(T)$ do que na inspeção direta das segmentações resultantes. Assim, a escolha do CLAHE baseia-se principalmente na maximização da separação estatística entre as classes de fundo e objeto.

💡 Interpretação dos resultados

Observe que os pré-processamentos com CLAHE e CLAHE+Gaussiano produzem histogramas e limiares ótimos muito próximos ($T^* = 122$ e $T^* = 123$). Consequentemente, as imagens binarizadas resultantes também são bastante semelhantes. Nesse caso, a decisão não é baseada em diferenças visuais marcantes, mas no critério objetivo do método de Otsu: o maior valor de σ_B^2 indica a melhor separação entre as classes.

4.3 Morfologia Matemática

A **morfologia matemática** é uma teoria baseada em conjuntos utilizada para analisar a forma e a estrutura de objetos em imagens. Diferentemente dos filtros lineares apresentados no Capítulo 3, os operadores morfológicos são **não lineares**, pois se baseiam em operações de mínimo, máximo e inclusão espacial, em vez de combinações lineares de intensidades. Esses operadores atuam sobre a vizinhança de cada pixel por meio de um **elemento estruturante** $\mathbb{B}$, responsável por definir a forma e o tamanho da região analisada.

Em imagens binárias e em tons de cinza com elementos planos, o elemento estruturante transladado para a posição x é definido espacialmente como:

$$\mathbb{B}_x = \{x + b \mid b \in \mathbb{B}\}$$

Nas regiões de borda da imagem, parte do conjunto $\mathbb{B}_x$ pode extrapolar o domínio físico da cena ($\mathbb{E}$). Para garantir a consistência matemática dos operadores primitivos nessas fronteiras, assume-se teoricamente que o espaço exterior ao domínio da imagem é preenchido com o **elemento neutro** da operação correspondente (infinito positivo para a erosão e infinito negativo para a dilatação), impedindo que o ambiente externo corrompa as estruturas internas do objeto.

Quando o elemento estruturante associa pesos a seus elementos — isto é, $b : \mathbb{B} \rightarrow \mathbb{Z}$ — ele é denominado **função estruturante** ou **elemento estruturante não plano**.

Desenvolvida por Matheron e Serra na década de 1960 para imagens binárias e posteriormente estendida a tons de cinza, a morfologia matemática fundamenta operadores como gradiente morfológico, *top-hat*, *watershed* e transformada de distância, todos derivados de dois primitivos: **erosão** e **dilatação** (Matheron, 1975; Serra, 1982).

4.3.1 Erosão e Dilatação

Os dois operadores primitivos são definidos de forma unificada para imagens em tons de cinza ($f : \mathbb{E} \rightarrow \mathbb{Z}$) e, por restrição ao domínio $\{0, 1\}$, também para imagens binárias.

4.3.1.1 Erosão

A **Erosão** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\varepsilon_b(f)(x) = (f \ominus b)(x) = \min_{z \in \mathbb{B}} \{ f(x + z) - b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.3)$$

Na prática, a erosão substitui a intensidade do pixel x pelo mínimo valor resultante da diferença entre a imagem e o elemento estruturante na vizinhança definida pelo domínio $\mathbb{B}$. Valores positivos nos pesos de $b(z)$ forçam o resultado local para baixo, “cavando” o relevo da imagem mais profundamente e intensificando a erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão simplifica-se para o mínimo local puro:

$$\varepsilon_B(f)(x) = \min\{f(y) : y \in \mathbb{B}_x\}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto $\mathbb{B}$, transladado para a coordenada x , esteja **completamente contido** no objeto A :

$$A \ominus \mathbb{B} = \{z \in \mathbb{E} \mid \mathbb{B}_z \subseteq A\}$$

Efeito Visual: *Encolhe* objetos e estruturas claras, eliminando protuberâncias, picos brilhantes ou ruídos que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.2 Implementação da erosão

A versão didática `mm.ero0` implementa o caso particular de erosão com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o menor valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de mínimo local descrita na Equation 4.3 para $b \equiv 0$.

Observe que os valores dos vizinhos são sempre lidos de forma estática da imagem original f ; a matriz de saída g é utilizada exclusivamente para registrar o mínimo acumulado da vizinhança corrente. Dessa forma, o resultado final é invariante em relação à ordem de varredura dos pixels (seja por linhas ou colunas).

A função auxiliar `_viz` calcula as coordenadas dos vizinhos válidos dentro dos limites físicos da imagem. Nas bordas, a inicialização do acumulador em 255 emula com exatidão o preenchimento por elemento neutro exigido pela teoria. Já a função de interface `mm.ero` recorre à implementação nativa e otimizada do OpenCV (`cv2.erode`) quando o elemento estruturante é plano, chaveando para a rotina geral `mm.ero1` caso o elemento possua pesos topográficos.

O exemplo computacional a seguir ilustra a aplicação de um elemento estruturante em cruz (`mm.secross()`) em destaque na Figure 4.3, comparando a execução da variante didática em laço (`mm.ero0`) com o motor computacional do OpenCV (`mm.ero`).

```
B_cruz = mm.secross()
mm.drawImgPlt(B_cruz, scale=20)
```

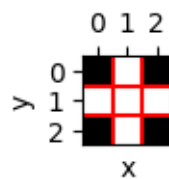


Figura 4.3: Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.

```
def _viz(f, B, y, x):
    """Gera (vy, vx, b_val) para cada vizinho válido de (y,x)."""
    H, W = f.shape
    Bh, Bw = B.shape
    oh, ow = -Bh/2 + 0.5, -Bw/2 + 0.5 # offsets fixos
    for by, bx in np.ndindex(Bh, Bw):
        vy, vx = int(y + by + oh), int(x + bx + ow)
        if 0 <= vy < H and 0 <= vx < W:
            yield vy, vx, B[by, bx]

def ero(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Erosão (OpenCV ou com pesos)."""
    try:
        return cv2.erode(f, Bc)
```



```

except: return mm.ero1(f, Bc)

def ero0(f, Bc=np.ones((3,3),dtype='uint8')):
    """Erosão clássica sem pesos."""
    g = np.empty_like(f)
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 255
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] > f[vy,vx]: g[y,x] = f[vy,vx]
    return g

# Script de Teste e Validação
B = mm.seccross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Erosão Plana Didática (ero0):")
print(mm.drawImage(ero0(f, B)))

print("Erosão Otimizada OpenCV (ero):")
print(mm.drawImage(ero(f, B)))

```

```

Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
1 9 8 0 8
3 7 7 0 2
8 5 5 1 2
4 3 6 6 0
8 9 4 1 9

Erosão Plana Didática (ero0):
1 1 0 0 0
1 3 0 0 0
3 3 1 0 0
3 3 3 0 0
4 3 1 1 0

Erosão Otimizada OpenCV (ero):
1 1 0 0 0
1 3 0 0 0
3 3 1 0 0
3 3 3 0 0
4 3 1 1 0

```

A função `_viz` utiliza `yield` para gerar cada vizinho sob demanda, sem armazenar todos os resultados em memória em uma lista. No experimento abaixo, uma janela de 3000×3000 produz 9 milhões de vizinhos. A implementação baseada em lista consumiu mais de 1 GB de RAM e levou aproximadamente 46 s, enquanto a versão com `yield` consumiu memória desprezível e executou em 38 s. Em processamento de imagens, geradores são especialmente úteis para percorrer grandes vizinhanças de forma eficiente.

```
import tracemalloc

def lista(n): return [(i, i) for i in range(n)]
def gera(n):
    for i in range(n): yield i, i

for nome, f in [("LISTA", lista), ("YIELD", gera)]:
    tracemalloc.start()
    sum(x+y for x,y in f(9_000_000))
    _, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(f"{nome}: {pico/1024/1024:.1f} MB")
```

```
LISTA: 830.8 MB
YIELD: 0.0 MB
```

4.3.1.3 Dilatação

A **Dilatação** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\delta_b(f)(x) = (f \oplus b)(x) = \max_{z \in \mathbb{B}} \{ f(x - z) + b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.4)$$

Na prática, a dilatação substitui a intensidade do pixel x pelo maior valor resultante da soma entre a imagem e o elemento estruturante na vizinhança definida. O argumento de inversão espacial ($x - z$) indica que a dilatação avalia implicitamente o elemento transposto (refletido) $\hat{b}$, propriedade fundamental para assegurar a dualidade matemática em relação à erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão reduz-se ao máximo local puro:

$$\delta_B(f)(x) = \max \{ f(y) : y \in \mathbb{B}_x \}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto refletido $\hat{\mathbb{B}}$, transladado para a coordenada x , possua **interseção não vazia** com o objeto A :

$$A \oplus \mathbb{B} = \{ z \in \mathbb{E} \mid \hat{\mathbb{B}}_z \cap A \neq \emptyset \}$$

Efeito Visual: *Expand*e as estruturas claras da imagem, aumentando o preenchimento de objetos, conectando componentes próximos e eliminando canais, fossas escuras ou vales que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.4 Implementação da dilatação

A versão didática `mm.dil0` implementa o caso particular de dilatação com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o maior valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de máximo local para $b \equiv 0$.

Antes de iniciar a varredura espacial, o elemento estruturante sofre uma reflexão geométrica por meio da instrução `np.flip(Bc)` para construir explicitamente a matriz transposta $\hat{B}$ exigida pela teoria. Em máscaras perfeitamente simétricas (como cruzeiros, quadrados e discos centrados na origem), essa reflexão não altera o arranjo dos pixels; contudo, para elementos assimétricos, tal etapa é estritamente necessária para garantir a equivalência com as definições formais e salvaguardar as leis de dualidade.

Assim como verificado no operador de erosão, os valores dos vizinhos são sempre lidos de forma estática a partir da matriz original f , enquanto a matriz de saída g atua puramente como o registrador do máximo acumulado da vizinhança. Nas fronteiras da imagem, a inicialização do acumulador em 0 emula com exatidão

o preenchimento externo por elemento neutro ($-\infty$, ou zero em representações de 8 bits), garantindo que as bordas físicas da cena sejam dilatadas em perfeita conformidade com o padrão adotado pelo OpenCV.

O exemplo computacional abaixo ilustra a aplicação prática de um elemento em cruz (`mm.secross()`), validando a consistência entre a lógica em laços (`mm.dil0`) e o método nativo industrial (`mm.dil`).

```
def dil(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação (OpenCV ou com pesos)."""
    try:    return cv2.dilate(f, Bc)
    except: return mm.dil1(f, Bc)

def dil0(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação plana seguindo rigorosamente a teoria."""
    g = np.empty_like(f)
    Bc = np.flip(Bc)      # reflexão explícita: B
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 0 # Inicializa com o valor mínimo para buscar o máximo
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] < f[vy,vx]:
                    g[y,x] = f[vy,vx]

    return g

# Script de Teste e Validação
B = mm.secross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Dilatação Plana Didática (dil0):")
print(mm.drawImage(dil0(f, B)))

print("Dilatação Otimizada OpenCV (dil):")
print(mm.drawImage(dil(f, B)))
```

```
Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
8 3 5 7 1
5 6 1 8 9
0 1 8 1 0
5 5 4 4 1
9 4 0 6 9

Dilatação Plana Didática (dil0):
8 8 7 8 9
8 6 8 9 9
5 8 8 8 9
9 5 8 6 9
9 9 6 9 9

Dilatação Otimizada OpenCV (dil):
8 8 7 8 9
8 6 8 9 9
```

```

5 8 8 8 9
9 5 8 6 9
9 9 6 9 9

```

i Nota: O Confronto dos Sinais ($f(x+z)$ vs $f(x-z)$)

Compare as definições formais da erosão (Equation 4.3) e da dilatação (Equation 4.4). Considere um elemento estruturante assimétrico à direita $\mathbb{B} = \{0, 1\}$ (origem e um pixel à direita) aplicado na posição $x = 10$.

1. Na Erosão (Equation 4.3):

$$\min\{f(x+z) - b(z)\}$$

- $z = 0 \Rightarrow f(10+0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10+1) = \mathbf{f(11)}$

O operador consulta o pixel atual (10) e o pixel à direita (11), preservando a orientação original de $\mathbb{B}$.

2. Na Dilatação (Equation 4.4):

$$\max\{f(x-z) + b(z)\}$$

- $z = 0 \Rightarrow f(10-0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10-1) = \mathbf{f(9)}$

Devido ao sinal negativo ($-z$), avançar no elemento estruturante corresponde a recuar na imagem, fazendo com que a dilatação consulte o pixel à esquerda (9).

A função `_viz`, utilizada em `morph.py`, gera vizinhos por deslocamentos aditivos da forma $x+z$. Por esse motivo, a implementação de `mm.dil0` reflete previamente o elemento estruturante por meio de `np.flip(B)`. Após a reflexão, a varredura baseada em $x+z$ passa a acessar exatamente os mesmos pontos definidos pela expressão teórica $f(x-z)$ da dilatação em Equation 4.4.

Para elementos estruturantes simétricos (como discos, quadrados e cruzes centradas), a reflexão não altera a máscara. Já para elementos assimétricos, essa etapa é indispensável para que a implementação reproduza corretamente a definição matemática da dilatação e preserve a dualidade erosão–dilatação.

i Dualidade erosão–dilatação

Erosão e dilatação são **duais pelo complemento**. Isso significa que um operador pode ser completamente obtido a partir do outro, desde que se atue sobre o complemento da imagem utilizando o elemento estruturante refletido $\hat{B}$:

$$(A \ominus B)^c = A^c \oplus \hat{B} \quad \Longleftrightarrow \quad A \ominus B = (A^c \oplus \hat{B})^c$$

De forma análoga, a **dilatação também pode ser obtida a partir da erosão**:

$$(A \oplus B)^c = A^c \ominus \hat{B} \quad \Longleftrightarrow \quad A \oplus B = (A^c \ominus \hat{B})^c$$

Em termos práticos, a erosão de um objeto pode ser obtida pela dilatação de seu complemento, seguida da complementação do resultado (e vice-versa). Na implementação do pacote `morph.py`, as versões didáticas `mm.ero0` e `mm.dil0` tornam explícita essa estrutura por meio de laços (*loops*), enquanto `mm.ero` e `mm.dil` delegam as operações ao OpenCV visando maior eficiência computacional.

i Condições de contorno e imagens finitas

Na morfologia matemática clássica, definida sobre um domínio infinito (tipicamente $\mathbb{Z}^2$), essa dualidade é exata. Em imagens digitais, entretanto, trabalha-se com matrizes finitas, e o resultado passa a depender da forma como são tratados os pixels localizados fora da imagem.

Para que as identidades de dualidade permaneçam válidas, o complemento deve ser definido em relação ao mesmo universo e as condições de contorno adotadas para a erosão e para a dilatação devem ser complementares entre si. Por exemplo, se a erosão assume que os pixels externos pertencem ao objeto (255), então a dilatação aplicada ao complemento deve assumir que esses mesmos pixels externos pertencem ao fundo (0).

Quando diferentes estratégias de preenchimento são utilizadas (replicação, reflexão, valor constante etc.), a dualidade teórica pode deixar de ser satisfeita exatamente nas regiões próximas às bordas da imagem.

Para ilustrar numericamente os operadores morfológicos e a dualidade erosão–dilatação, a Figure 4.4 apresenta uma imagem binária 10×10 processada com um elemento estruturante em formato de “L”. Na implementação de `morph.py`, a origem de B é fixada no centro geométrico da máscara — posição (1,1) para um *kernel* 3×3 — e deve corresponder a um elemento ativo para que a erosão se comporte corretamente (conforme discutido anteriormente). O elemento estruturante B_L definido a seguir satisfaz essa condição. A Figure 4.5 complementa a análise com um simulador interativo da erosão, permitindo visualizar o deslocamento do elemento estruturante sobre a imagem e identificar as posições em que ele permanece completamente contido no objeto.

```
A = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,1,1,1,0,0,0,0],
    [0,0,1,1,1,1,1,0,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,0,0,0],
    [0,1,1,1,1,1,1,0,0,0],
    [0,0,1,1,1,1,1,1,0,0],
    [0,0,0,1,1,1,1,0,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_L = np.array([[1,0,0],
                [1,1,0],
                [1,1,0]], dtype=np.uint8) # origem no centro (1,1), elemento ativo

# Elemento estruturante refletido B
B_hat = np.flip(B_L)
print("Elemento estruturante B_L:")
print(mm.drawImg(B_L))
print("Elemento estruturante refletido B:")
print(mm.drawImg(B_hat))

# Erosão de A por B_L
img_ero0 = mm.ero0(A, B_L)

# Validação da dualidade: (A ∖ B)c = Ac ∖ B
A_c = 255 - A # complemento de A
ero_c = 255 - img_ero0 # complemento da erosão - lado esquerdo
dil_Ac = mm.dil0(A_c, B_hat) # lado direito

dualidade_ok = np.array_equal(ero_c, dil_Ac)
print(f"Dualidade (A ∖ B)c == Ac ∖ B : {dualidade_ok}")
# Transposto (reflexão em ambos os eixos): B usado na dilatação da dualidade
B_hat = np.flip(B_L)
mm.show(
    [A, A_c, img_ero0, ero_c, dil_Ac],
    titles=["A", "A ", "A ∖ B", "(A ∖ B) ", "A ∖ B"],
    cols=5,
```

```
figsize=(15, 3)
)
```

Elemento estruturante B_L:

```
1 0 0
1 1 0
1 1 0
```

Elemento estruturante refletido B:

```
0 1 1
0 1 1
0 0 1
```

Dualidade $(A \ominus B)^c == A^c \oplus B$: True

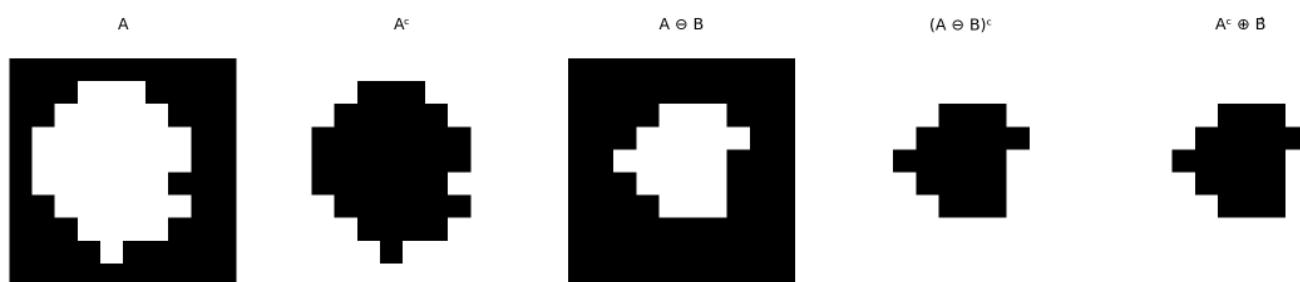


Figura 4.4: Erosão e dilatação em imagem binária 10×10 com elemento estruturante ‘L’ assimétrico 3×3. Validação da dualidade erosão–dilatação.

4.3.2 Abertura e Fechamento

Combinando erosão e dilatação obtêm-se dois operadores de grande utilidade prática: a **abertura** e o **fechamento**, definidos pelas Equações Equation 4.5 e Equation 4.6. Seus principais efeitos são resumidos na Tabela Table 4.2.

Abertura (*opening*) — erosão seguida de dilatação pelo mesmo B :

$$A \circ B = (A \ominus B) \oplus B \quad (4.5)$$

Fechamento (*closing*) — dilatação seguida de erosão pelo mesmo B :

$$A \bullet B = (A \oplus B) \ominus B \quad (4.6)$$

Na prática, o OpenCV aplica o mesmo elemento estruturante nas duas etapas. Para elementos estruturantes simétricos (os mais comuns), essa implementação coincide com a definição matemática apresentada acima.

Tabela 4.2: Propriedades de abertura e fechamento.

Operador	Sequência	Efeito principal
Abertura $A \circ B$	erosão $\rightarrow$ dilatação	Remove estruturas incapazes de conter o elemento estruturante; suaviza contornos externos
Fechamento $A \bullet B$	dilatação $\rightarrow$ erosão	Preenche buracos menores que B ; suaviza contornos internos

Propriedade importante: ambos são **idempotentes**. Por exemplo,

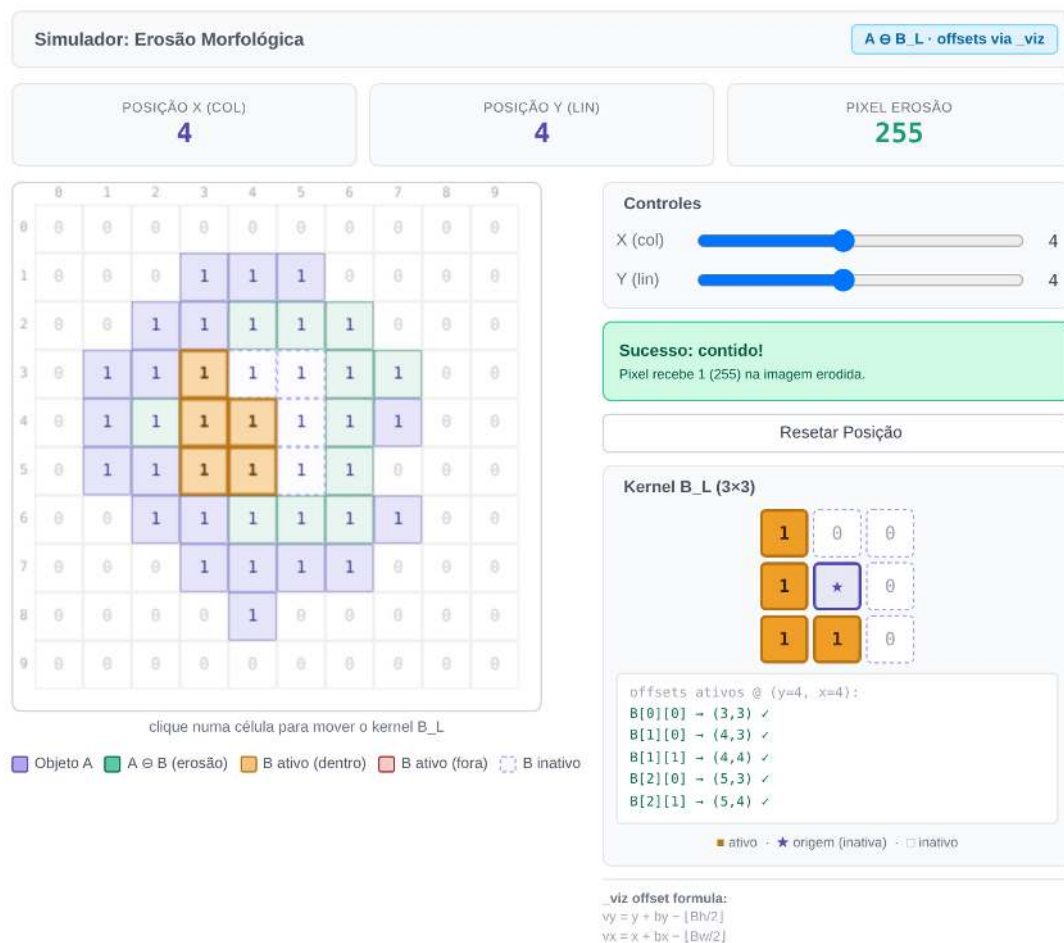


Figura 4.5: Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A .

$$(A \circ B) \circ B = A \circ B,$$

ou seja, após a primeira aplicação, novas aplicações do mesmo operador não alteram mais o resultado.

4.3.2.1 Implementação da abertura e do fechamento

Diferentemente da erosão e da dilatação, abertura e fechamento não introduzem novos mecanismos computacionais. Ambos são obtidos pela composição sequencial dos operadores primitivos já apresentados:

```
def open0(f, B):
    return mm.dil0(mm.ero0(f, B), B)

def close0(f, B):
    return mm.ero0(mm.dil0(f, B), B)
```

A função `mm.open` delega a operação para `cv2.morphologyEx(..., MORPH_OPEN, B)`, enquanto `mm.close` utiliza `cv2.morphologyEx(..., MORPH_CLOSE, B)`, produzindo o mesmo resultado de forma mais eficiente.

A abertura herda da erosão a capacidade de remover estruturas menores que o elemento estruturante e da dilatação a restauração parcial das regiões preservadas. O fechamento realiza o processo inverso: primeiro expande os objetos e depois restaura suas dimensões originais, preenchendo lacunas e buracos menores que o elemento estruturante.

4.3.2.2 Filtro Sequencial Alternado

Na prática, abertura e fechamento são frequentemente aplicados em sequência para remover simultaneamente ruído externo e preencher buracos internos. A função `mm.asf` (*Alternating Sequential Filter*) generaliza essa estratégia ao aplicar aberturas e fechamentos alternadamente com elementos estruturantes progressivamente maiores. As sequências disponíveis são apresentadas na Table 4.3.

Tabela 4.3: Sequências do filtro sequencial alternado `mm.asf`.

Sequência	Ordem	Uso típico
'OC'	abertura → fechamento	remove ruído externo antes de preencher pequenos buracos
'CO'	fechamento → abertura	preenche pequenos buracos antes de remover ruído externo
'OCO'	abertura → fechamento → abertura	ênfatisa a remoção de ruído externo
'COC'	fechamento → abertura → fechamento	ênfatisa o preenchimento de buracos e lacunas

O parâmetro `n` controla o número de escalas utilizadas pelo filtro. Em cada iteração i , o elemento estruturante é ampliado por soma de Minkowski (`mm.sesum(b, i)`), produzindo uma sequência de filtros morfológicos cada vez mais abrangentes. Diferentemente de uma única abertura ou fechamento com um elemento estruturante grande, o ASF realiza uma suavização progressiva em múltiplas escalas, preservando melhor a geometria dos objetos relevantes enquanto elimina estruturas menores. A Figure 4.6 apresenta um exemplo de aplicação da abertura, do fechamento e do ASF na imagem das moedas.

```
img_bin    = mm.threshold(img_clahe)
B_disk     = mm.sedisk(19)

img_open   = mm.open(img_bin, B_disk)           # erosão → dilatação
img_close  = mm.close(img_bin, B_disk)          # dilatação → erosão
img_oc     = mm.close(img_open, B_disk)         # abertura seguida de fechamento
img_asf    = mm.asf(img_bin, 'OC', mm.sedisk(3), n=7)
# ASF: disco base 3×3, cresce a cada iteração
```

```
mm.show(
    [img_bin, img_open, img_close, img_oc, img_asf],
    titles=["Binarização Otsu (CLAHE)", "Abertura (A∘B)", "Fechamento (A⋄B)",
           "Abertura→Fechamento", "ASF-OC (n=7)"],
    cols=5, figsize=(15, 12)
)
```

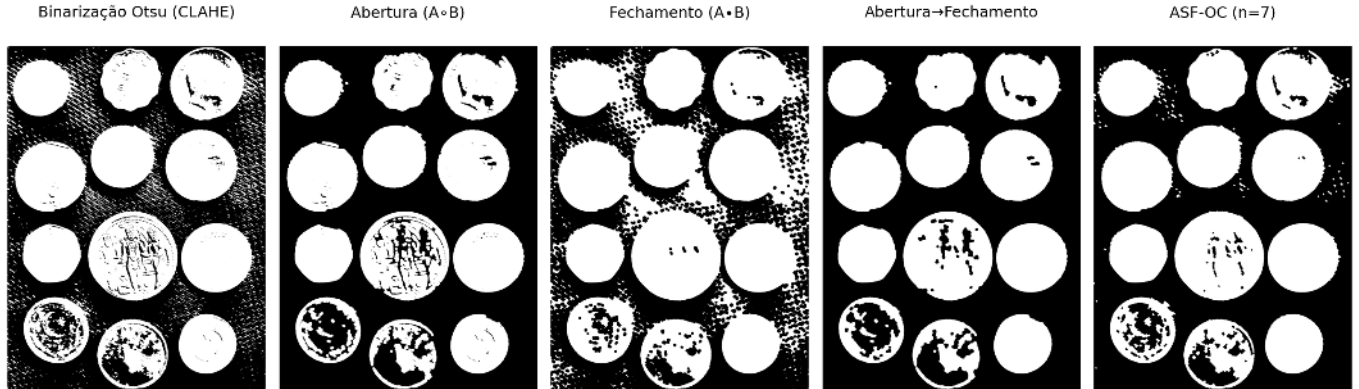


Figura 4.6: Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13 .

4.3.3 Operadores Geodésicos

Os operadores geodésicos introduzem uma restrição adicional aos operadores morfológicos clássicos por meio de uma imagem de controle denominada **máscara** g . Em vez de permitir que a erosão ou a dilatação se propaguem livremente pela imagem, o resultado de cada iteração é limitado ponto a ponto pelos valores da máscara, restringindo a evolução da operação às regiões permitidas.

4.3.3.1 Dilatação Geodésica

A **dilatação geodésica** de uma imagem marcador f sob uma imagem máscara g , utilizando um elemento estruturante plano b , é definida por:

$$f \oplus_g b = (f \oplus b) \wedge g, \quad (4.7)$$

onde $\wedge$ representa o mínimo ponto a ponto.

Em outras palavras, realiza-se inicialmente uma dilatação convencional sobre o marcador e, em seguida, o resultado é restringido pela máscara g . Dessa forma, a propagação nunca pode ultrapassar as regiões permitidas pela máscara.

A formulação clássica da dilatação geodésica pressupõe que o marcador esteja contido na máscara, isto é, $f \leq g$, garantindo que a evolução da operação permaneça sempre limitada pela máscara.

4.3.3.2 Implementação da dilatação geodésica

A função `mm.cdil` implementa diretamente esse operador e permite executar múltiplas iterações consecutivas:

```
def cdil(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Dilatação geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.minimum(cv2.dilate(y, b), g)
    return y
```

A instrução `np.minimum(cv2.dilate(y, b), g)` implementa exatamente a definição matemática da dilatação geodésica, isto é, $(y \oplus b) \wedge g$.

Quando $n = 1$, a função executa uma única dilatação geodésica. Para $n > 1$, o resultado de cada etapa torna-se o marcador da etapa seguinte, produzindo uma propagação progressiva controlada pela máscara.

O simulador interativo Figure 4.7 permite acompanhar, passo a passo, a propagação do marcador f ao longo dos corredores do labirinto. A cada iteração de `mm.cdil`, a frente de dilatação avança para as células vizinhas livres — aquelas em que $g = 1$ —, enquanto as paredes ($g = 0$) permanecem intransponíveis. O número de passos necessários para que o marcador alcance a saída corresponde exatamente ao comprimento geodésico do caminho mais curto dentro da máscara, evidenciando a conexão direta entre a dilatação geodésica iterada e a noção de distância em grafos.

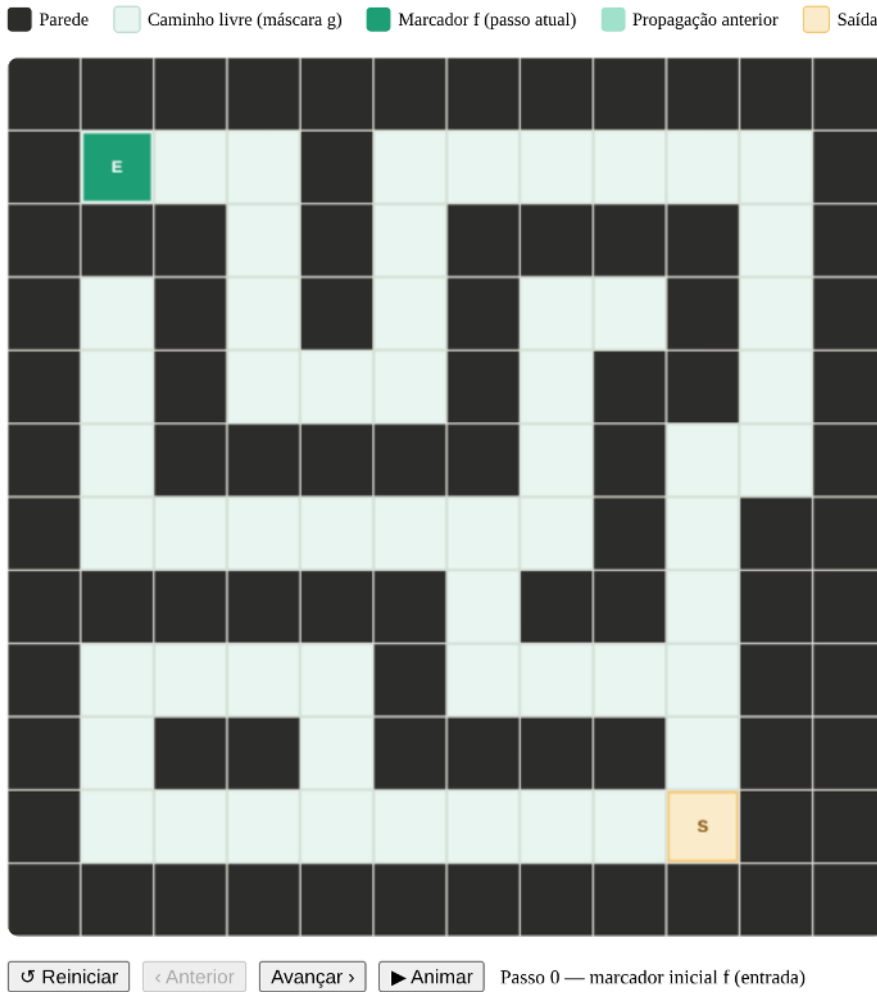


Figura 4.7: Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\delta_g^{(n)}(f)$ encontra a saída do labirinto.

4.3.3.3 Erosão Geodésica

De forma dual, a **erosão geodésica** de uma imagem marcador f sob uma imagem máscara g é definida por:

$$f \ominus_g b = (f \ominus b) \vee g, \quad (4.8)$$

onde $\vee$ representa o operador de máximo ponto a ponto.

Nesse caso, a erosão convencional do marcador é seguida por uma restrição inferior imposta pela máscara. Assim, nenhum pixel do resultado pode assumir valor inferior ao correspondente pixel da máscara.

A formulação clássica da erosão geodésica pressupõe a condição dual

$$f \geq g,$$

de modo que a máscara atue como limite inferior durante todo o processo.

4.3.3.4 Implementação da erosão geodésica

A função estática `mm.cero` materializa esse operador:

```
@staticmethod
def cero(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Erosão geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.maximum(cv2.erode(y, b), g)
    return y
```

A instrução `np.maximum(cv2.erode(y, b), g)` implementa diretamente a expressão $(y \ominus b) \vee g$.

Assim como na dilatação geodésica, o parâmetro n define quantas erosões geodésicas sucessivas serão computadas antes de retornar a imagem final.

i Relação com a reconstrução morfológica

A reconstrução morfológica apresentada na próxima seção é obtida pela aplicação iterativa da dilatação geodésica (`mm.cdil`) até que um ponto fixo seja alcançado, isto é, até que nenhuma célula mude de valor entre duas iterações consecutivas. Em outras palavras, a reconstrução consiste em uma sequência de dilatações geodésicas sucessivas que se propagam dentro da máscara até não haver mais alterações. De forma dual, também é possível definir reconstruções baseadas em erosão geodésica por meio de aplicações sucessivas de `mm.cero`.

4.3.3.5 Exemplo: propagação em um labirinto via dualidade

A Figure 4.8 ilustra a resolução do problema de conectividade de um labirinto utilizando a dualidade morfológica por meio das funções `mm.cero` e `mm.suprec`.

Em vez de propagar um marcador pelos corredores livres através de dilatações geodésicas, o problema é formulado no domínio complementar. Inicialmente, a máscara original é invertida,

$$g = 1 - g_{orig},$$

fazendo com que as paredes passem a assumir valor 1 e os corredores valor 0. De forma análoga, o marcador é construído nesse mesmo domínio complementar, contendo um único valor 0 na posição de entrada do labirinto e valor 1 nos demais pixels.

Utilizando o elemento estruturante em cruz (`mm.secross()`), a erosão geodésica atua sobre o marcador complementado. A cada iteração de `mm.cero`, a região conectada ao marcador inicial sofre erosões sucessivas, enquanto a máscara impõe um limite inferior que impede a propagação através das paredes do labirinto.

As imagens intermediárias mostram estados da evolução após diferentes números de iterações ($n=5$, $n=12$ e $n=22$). O resultado final é obtido pela reconstrução geodésica por erosão (`mm.suprec`), que aplica erosões geodésicas sucessivas até atingir um ponto fixo, isto é, uma situação em que nenhuma alteração adicional ocorre entre duas iterações consecutivas.

No domínio complementar, a região reconstruída corresponde exatamente ao conjunto de corredores conectados à entrada do labirinto. Assim, a conectividade entre entrada e saída pode ser determinada diretamente a partir da imagem reconstruída.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

# 1. Máscara original g (0 = Parede, 1 = Corredor)
g_orig = np.array([
    [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    [0, 1, 0, 1, 0, 0, 0, 0, 0, 1],
    [0, 1, 0, 1, 1, 1, 1, 1, 1, 1],
    [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]], dtype=np.uint8)

# Inversão global no início (Dualidade Morfológica)
g = 1 - g_orig # Agora: 1 = Parede, 0 = Corredor
f = np.ones((10, 10), dtype=np.uint8)
f[0, 1] = 0 # Semente injetada como 0 no corredor

# Elemento estruturante em cruz
B_cruz = mm.secross()

# Processamento direto usando mm.cero e mm.suprec no domínio invertido
passo_5 = mm.cero(f, g, B_cruz, n=5)
passo_12 = mm.cero(f, g, B_cruz, n=12)
passo_22 = mm.cero(f, g, B_cruz, n=22)
ponto_fixo = mm.suprec(f, g, B_cruz)

# --- CONFIGURAÇÃO DA EXIBIÇÃO GRÁFICA ---

titles = [
    "Máscara (~g)", "Marcador (~f)",
    "Avanço (n=5)", "Avanço (n=12)",
    "Avanço (n=22)", "~mm.suprec"
]

images = [g, f, passo_5, passo_12, passo_22, ponto_fixo]

fig, axes = plt.subplots(1, 6, figsize=(16, 4), facecolor='#fcfcfc')

# Mapa de cores adaptado para o domínio complementar:
# No domínio invertido: 1 = Parede (Azul Escuro)
# Onde g == 0 e imagem == 1 = Corredor Livre (Cinza Claro)
# Onde g == 0 e imagem == 0 = Onda Geodésica Ativa (Ouro)
cmap_pipeline = mcolors.ListedColormap(['#ffc13b', '#e0e0e0', '#1e3d59'])

for i, ax in enumerate(axes):
    if i == 0 or i == 1:
        # Para as condições iniciais (~g e ~f)
        cmap_init = mcolors.ListedColormap(['#e0e0e0', '#1e3d59'])
        ax.imshow(images[i], cmap=cmap_init, vmin=0, vmax=1)
    else:
        # Renderização baseada nos estados complementares
        render_step = np.zeros_like(g, dtype=np.uint8)
```



```

render_step[g == 1] = 2      # Parede (1 original do mapa de cores)
render_step[g == 0] = 1      # Corredor padrão
render_step[images[i] == 0] = 0 # Onda ativa (0 original do mapa de cores)
ax.imshow(render_step, cmap=cmap_pipeline, vmin=0, vmax=2)

ax.set_title(titles[i], fontsize=10, fontweight='bold', color='#1e3d59', pad=12)
ax.set_xticks(np.arange(-0.5, 10, 1), minor=True)
ax.set_yticks(np.arange(-0.5, 10, 1), minor=True)
ax.grid(which='minor', color='ffffff', linestyle='--', linewidth=1)
ax.tick_params(which='both', bottom=False, left=False, labelbottom=False, labelleft=False)

# Indicadores gráficos de Entrada e Saída
ax.plot(1, 0, marker='v', color='#2ecc71', markersize=7, markeredgewidth=1.5)
ax.plot(8, 9, marker='o', color='#e74c3c', markersize=6, fillstyle='none', markeredgewidth=2)

plt.tight_layout()
plt.show()

```

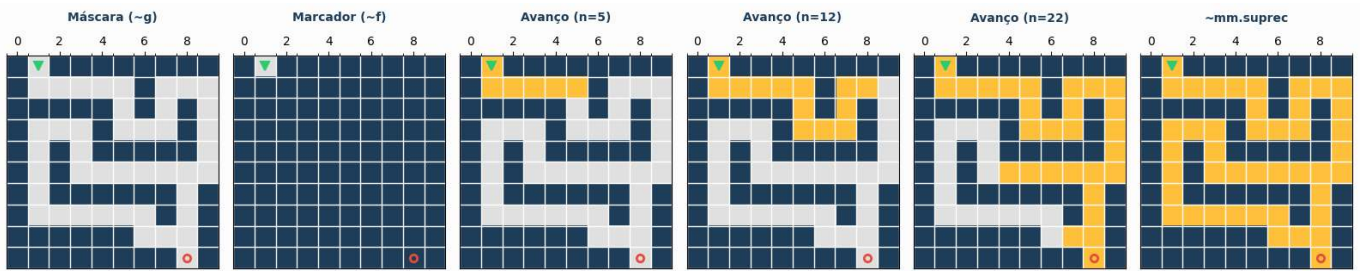


Figura 4.8: Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do *pipeline*, o fluxo é resolvido diretamente no domínio complementar através de *mm.cero* e *mm.suprec*, eliminando re-inversões redundantes.

4.3.4 Reconstrução Morfológica

A **reconstrução morfológica** propaga uma imagem **marcador** f dentro de uma imagem **máscara** g , garantindo que o resultado nunca ultrapasse os valores de intensidade impostos pela máscara. O operador fundamental que viabiliza essa propagação contida é a **dilatação geodésica**, definida pela Equation 4.7.

A reconstrução é obtida pela aplicação iterativa dessa dilatação condicionada. Inicialmente, o marcador é limitado pela máscara para estabelecer o estado inicial:

$$X^{(0)} = f \wedge g,$$

e as iterações subsequentes são definidas de forma recursiva por:

$$X^{(k)} = (X^{(k-1)} \oplus b) \wedge g.$$

A sequência cresce monotonicamente até atingir um ponto fixo, produzindo a **reconstrução morfológica por dilatação** (também conhecida na literatura como *inf-reconstrução*):

$$R_g^\delta(f) = \lim_{k \rightarrow \infty} X^{(k)} = X^{(k)} \quad \text{quando} \quad X^{(k)} = X^{(k-1)}. \quad (4.9)$$

A subida iterativa é interrompida assim que a estabilidade é alcançada, isto é, quando duas iterações consecutivas produzem matrizes com valores absolutamente idênticos.

4.3.4.1 Implementação da reconstrução morfológica

A rotina didática `mm.infrec` implementa diretamente o algoritmo iterativo de ponto fixo. Inicialmente, o marcador efetivo inicial $X^{(0)}$ é determinado pela operação `np.minimum(f, g)`. Para garantir que o laço de verificação execute a primeira passada sem disparar falsas convergências prematuras, a variável de controle da iteração anterior (`y1`) é inicializada preenchida com um valor sentinela fora do domínio de dados (ou simplesmente com uma matriz que force a primeira execução).

```
def infrec(f, g, b=np.zeros((3,3), dtype='uint8')):
    """Inf-reconstrução: dilata o marcador (f g) até convergir sob a máscara g."""
    y = np.minimum(f, g)
    # Inicializa y1 com valores impossíveis para forçar a entrada no laço
    y1 = np.full_like(f, 256, dtype=np.int16)
    while not np.array_equal(y, y1):
        y1 = y.copy()
        # Aplica a dilatação geodésica: (y b) g
        y = np.minimum(cv2.dilate(y, b), g)
    return y.astype('uint8')
```

No interior do laço `while`, a variável `y` armazena a estimativa corrente da reconstrução $X^{(k)}$, enquanto `y1` preserva a imagem do estágio imediatamente anterior $X^{(k-1)}$. A instrução de controle condicional `np.minimum(cv2.dilate(y, b), g)` traduz fielmente a dilatação geodésica teórica, onde a expansão morfológica convencional comandada pelo OpenCV é imediatamente “podada” e limitada pelas barreiras de intensidade da máscara `g`. O laço cessa quando nenhuma modificação de pixel é registrada entre os passos.

4.3.4.2 Vantagens da reconstrução morfológica

A reconstrução morfológica é significativamente mais robusta que a abertura convencional porque é capaz de remover estruturas indesejadas sem distorcer ou alterar a morfologia dos objetos que devem ser preservados.

Enquanto a abertura clássica suaviza cantos, elimina pontas e deforma contornos devido à imposição geométrica rígida do elemento estruturante, a reconstrução geodésica utiliza a máscara para recuperar com exatidão os limites e formatos originais dos objetos que possuem conectividade com o marcador original.

Em termos intuitivos, o marcador atua como uma semente de contágio que se expande progressivamente, mas apenas trafegando pelas regiões permitidas pela máscara. Componentes que não possuem nenhuma intersecção com o marcador jamais serão reconstruídas (sendo eliminadas), enquanto as componentes tocadas pela semente expandem-se até restaurar integralmente a sua geometria original.

Esse comportamento discriminatório e conservativo é ilustrado na Figure 4.9, utilizando o elemento estruturante em cruz apresentado na Figure 4.3.

```
# 1. Definição da Máscara (g): Imagem 10x10 com dois objetos isolados
g = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,1,1,0],
    [0,1,1,1,1,0,0,1,1,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,0,1,1,1,0,0,1,0,0],
    [0,0,1,1,1,0,0,1,1,0],
    [0,0,0,1,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_cruz = mm.secross()

# 2. Geração do Marcador (f)
f = mm.ero(g, mm.sebox())
```

```
# 3. Iterações da Dilatação Condicionada automatizadas em laço
iteracoes = []
titulos_iteracoes = []
img_atual = f.copy()

for i in range(1, 6):
    img_add = img_atual*80
    img_atual = mm.cdil(img_atual, g, B_cruz)
    iteracoes.append(img_add+img_atual)
    titulos_iteracoes.append(f"Dilatação Cond. (n={i})")

# Reconstrução Geodésica Final via biblioteca (ponto de controle)
img_reconstruida = mm.infrec(f, g, B_cruz)

# Verificação de convergência (o passo 5 deve ser idêntico à reconstrução final)
convergencia_ok = np.array_equal(img_reconstruida, iteracoes[-1])
print(f" Reconstrução alcançou estabilidade na iteração 5: {convergencia_ok}")

# 4. Exibição do pipeline evolutivo (Montagem dinâmica das listas)
mm.show(
    [g, f] + iteracoes + [img_reconstruida],
    titles=["Máscara (g)", "Marcador (f)"] + titulos_iteracoes + ["Reconstrução Final R_g(f)"],
    cols=8,
    figsize=(18, 3)
)
```

Reconstrução alcançou estabilidade na iteração 5: False

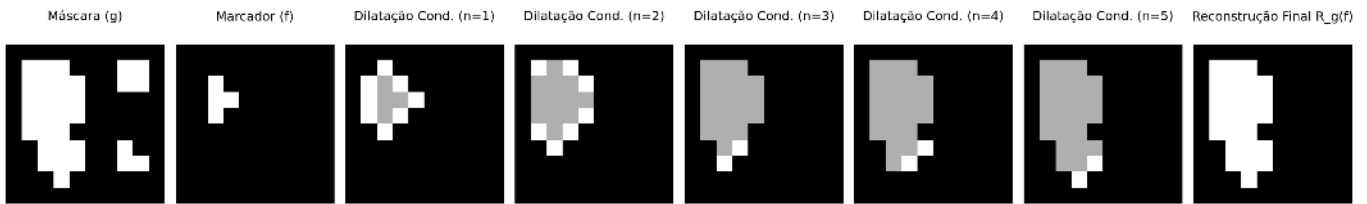


Figura 4.9: *Pipeline* de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstróem sua forma exata até a convergência.

4.3.5 Preenchimento de Buracos e Remoção de Bordas

Dois operadores baseados em reconstrução morfológica completam o *pipeline* de limpeza binária. Suas características principais são resumidas na Table 4.4.

Preenchimento de buracos (`mm.clohole`) remove cavidades completamente cercadas pelo objeto, independentemente do tamanho, sem alterar os contornos externos. O procedimento atua no complemento da imagem, utilizando como marcador uma restrição da moldura (*frame*) ao fundo:

$$\text{clohole}(f) = (R_{f^c}^{\delta}(\text{frame}(f) \wedge f^c))^c \quad (4.10)$$

Em termos operacionais, primeiro reconstrói-se o fundo externo e, em seguida, aplica-se a complementação para recuperar os objetos com buracos preenchidos.

Remoção de objetos de borda (`mm.edgeoff`) elimina todos os objetos que tocam a borda da imagem, preservando apenas os componentes totalmente internos. O marcador é obtido pela interseção entre a moldura (*frame*) e os objetos da imagem:

$$\text{edgeoff}(f) = f \setminus R_f^\delta(\text{frame}(f) \wedge f) \quad (4.11)$$

Tabela 4.4: Comparação entre os operadores *clohole* e *edgeoff*.

Operador	Marcador	Máscara	Efeito
<code>mm.clohole</code>	<i>frame</i> restrito ao fundo (f^c)	f^c	Preenche buracos internos
<code>mm.edgeoff</code>	<i>frame</i> restrito ao objeto (f)	f	Remove objetos conectados à borda

A evolução passo a passo dessas transformações geodésicas pode ser acompanhada nas figuras a seguir. A Figure 4.10 ilustra o mecanismo de inundação controlada do operador `mm.clohole`, no qual a reconstrução ocorre a partir do fundo externo e impede a propagação para regiões internas não conectadas ao exterior, resultando no preenchimento consistente das cavidades internas. Em contrapartida, a Figure 4.11 detalha a dinâmica do operador `mm.edgeoff`, em que apenas os componentes conectados à borda são reconstruídos e posteriormente removidos, preservando exclusivamente os objetos totalmente contidos no interior da imagem.

A conectividade da propagação geodésica é controlada pelo elemento estruturante: `mm.sebox()` (vizinhança de 8) inclui conexões diagonais, enquanto `mm.secross()` (vizinhança de 4) as exclui. Consequentemente, a escolha do elemento estruturante afeta quais componentes são alcançados pela reconstrução e, portanto, quais serão preservados ou removidos.

4.3.5.1 Conformidade com a implementação

As definições acima estão diretamente alinhadas com a implementação em `morph.py`, reproduzida a seguir:

```
@staticmethod
def clohole(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito ao fundo da imagem
    marcador = mm.frame(f, border=1) & mm.neg(f)
    return mm.neg(mm.infrec(marcador, mm.neg(f), b))

@staticmethod
def edgeoff(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito aos objetos da imagem
    marcador = mm.frame(f, border=1) & f
    return mm.subm(f, mm.infrec(marcador, f, b))
```

Essas implementações deixam explícito que ambos os operadores são instâncias diretas de reconstrução morfológica por dilatação geodésica com `mm.infrec`, diferindo apenas na escolha do marcador e da máscara: `clohole` atua no complemento da imagem, enquanto `edgeoff` atua diretamente no domínio dos objetos.

```
# Função auxiliar unificada para gerar o pipeline iterativo com realce visual
def gerar_pipeline_reconstrucao(marcador, mascara, kernel, passos=4, fator=80):
    iteracoes, titulos = [], []
    img_atual = marcador.copy()
    for i in range(1, passos + 1):
        img_visual = img_atual * fator
        img_atual = mm.cdil(img_atual, mascara, kernel)
        iteracoes.append(img_visual + img_atual)
        titulos.append(f"Iter. (n={i})")
    return iteracoes, titulos

# Imagem binária 10x10 com 3 cenários: objeto com buraco, objeto isolado e objeto na borda
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
```

```

[0,1,1,1,1,0,0,0,1],
[0,1,0,0,1,0,0,1,0,1],
[0,1,0,0,1,0,0,1,0,1],
[0,1,1,1,1,0,0,1,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,1,1,1,0,1,0,1,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,0,0,0,0,0,0,0,1]], dtype=np.uint8) * 255

B_cruz = mm.secross()
f_c = mm.neg(f)          # Utiliza o operador de negação nativo da mm

# PIPELINE CLOHOLE
# 0 marcador do clohole teórico é a borda externa
marcador_ch = mm.frame(f, border=1)
iters_ch, tits_ch = gerar_pipeline_reconstrucao(marcador_ch, f_c, B_cruz, passos=5)

# Resultado final calculado por extenso e validado com a função nativa mm.clohole
img_clohole = mm.neg(mm.infrec(marcador_ch, f_c, B_cruz))
print(f" Validação clohole: {np.array_equal(img_clohole, mm.clohole(f))}")

mm.show(
    [f, marcador_ch] + iters_ch + [img_clohole],
    titles=["f original", "Marcador (Borda)"] + tits_ch + ["clohole(f)"],
    cols=8, figsize=(18, 3), axis=True
)

```

Validação clohole: True

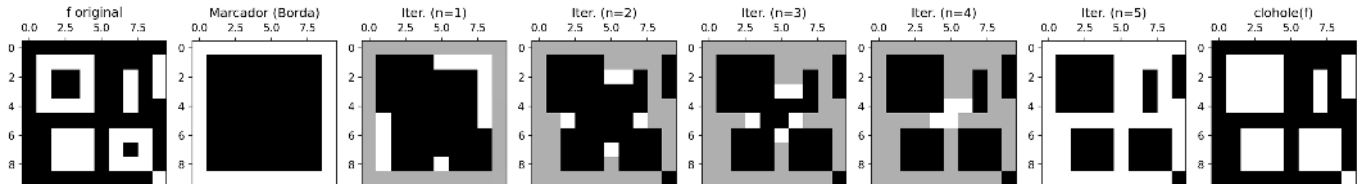


Figura 4.10: *Pipeline* de preenchimento de buracos (*clohole*): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.

```

# PIPELINE EDGEOFF
B_box = mm.sebox()
marcador_eo = marcador_ch & f
iters_eo, tits_eo = gerar_pipeline_reconstrucao(marcador_eo, f, B_box, passos=5)

# Resultado final por definição e validação nativa
img_edgoff = mm.edgoff(f, B_box)
print(f" Validação edgoff: {np.array_equal(img_edgoff, mm.edgoff(f))}")

mm.show(
    [f, marcador_eo] + iters_eo + [img_edgoff],
    titles=["f original", "Marcador (Borda)"] + tits_eo + ["edgoff(f)"],
    cols=8, figsize=(18, 3)
)

```

Validação edgoff: True

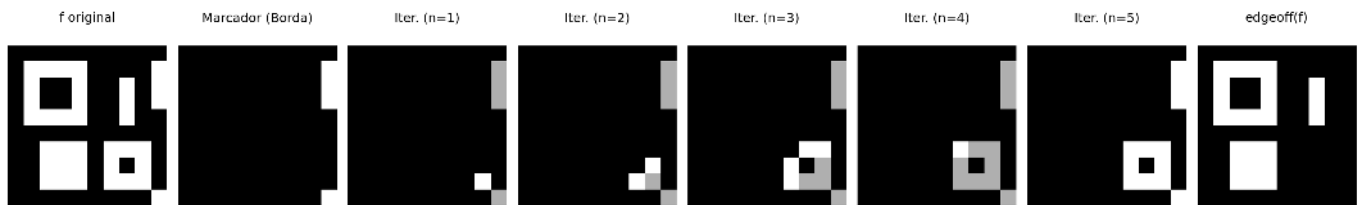
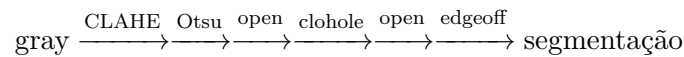


Figura 4.11: *Pipeline* de eliminação de estruturas de borda (*edgeoff*): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.

4.3.6 Pipeline de Limpeza Binária com CLAHE

Com base na análise anterior — na qual o CLAHE produziu o maior valor da variância interclasses ($\sigma_B^2 \approx 2,47 \times 10^3$), ver Figure 4.2, e o operador `mm.clohole` mostrou-se eficaz no preenchimento das cavidades internas —, o *pipeline* final de segmentação, ilustrado na Figure 4.12, é estruturado pelo seguinte fluxo computacional:



Após a etapa de `mm.clohole`, aplica-se uma segunda abertura morfológica com um elemento estruturante maior (`mm.sedisk(33)`, disco de diâmetro 33). Essa operação remove pequenas regiões residuais e artefatos que possam ter permanecido após a segmentação. Em particular, o preenchimento geodésico pode transformar pequenas cavidades isoladas em componentes conectados ao objeto, tornando conveniente uma etapa adicional de filtragem baseada em tamanho. O diâmetro foi escolhido de modo que as moedas permaneçam capazes de conter o elemento estruturante, enquanto componentes significativamente menores sejam eliminados.

Nesta imagem, nenhuma moeda está conectada à borda da matriz. Consequentemente, a aplicação de `mm.edgeoff` não altera o resultado obtido após a segunda abertura. Ainda assim, essa etapa é mantida no *pipeline* por robustez, pois em outras imagens podem existir objetos parcialmente visíveis ou conectados às bordas, que devem ser removidos antes da etapa de análise.

💡 Por que a abertura após o clohole?

O operador `mm.clohole` preenche todas as cavidades fechadas presentes nos objetos segmentados. Em algumas situações, pequenas regiões indesejadas podem permanecer após essa etapa ou tornar-se conectadas aos objetos principais. A abertura morfológica subsequente remove componentes menores que o elemento estruturante, preservando as moedas devido ao seu tamanho significativamente maior.

```
# Pré-processamento: apenas CLAHE
img_clahe0 = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(img_coins_gray)

# Etapa 1: Binarização Otsu
img_bin = mm.threshold(img_clahe0)

# Etapa 2: Abertura - remove ruídos brancos de fundo
img_open = mm.open(img_bin, mm.sedisk(9))

# Etapa 3: clohole - fecha todos os buracos internos
img_hole = mm.clohole(img_open)

# Etapa 4: Abertura (kernel grande) - remove artefatos do clohole
img_limpo = mm.open(img_hole, mm.sedisk(33))

# Etapa 5: edgeoff - remove objetos que tocam a borda
img_final = mm.edgeoff(img_limpo, border=1)
```



```

mm.show(
    [img_clahe0, img_bin,          img_open,
     img_hole,  img_limpo,        img_final],
    titles=["CLAHE",              "Otsu",          "Abertura (r=9)",
           "clohole",            "Abertura (r=33)", "Final (edgeoff)"],
    cols=6, figsize=(18, 6)
)

```

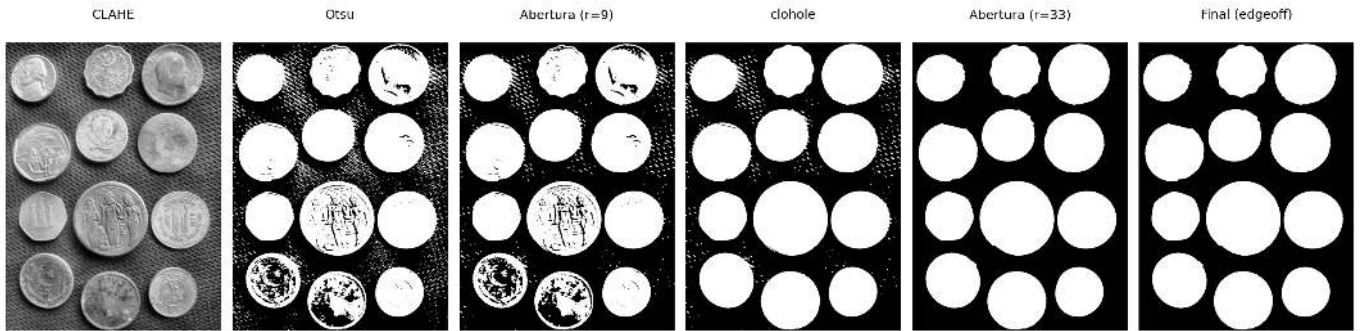


Figura 4.12: *Pipeline* completo de segmentação com CLAHE: Otsu → abertura (r=9) → clohole → abertura (r=33) → edgeoff.

4.3.7 Morfologia em Tons de Cinza

Os operadores morfológicos estendem-se naturalmente para imagens em tons de cinza. Nessa formulação, a erosão e a dilatação passam a atuar diretamente sobre os níveis de intensidade da imagem. Para elementos estruturantes planos ($b \equiv 0$), a erosão corresponde ao **mínimo local** e a dilatação ao **máximo local** dentro da vizinhança definida pelo elemento estruturante.

A interpretação intuitiva é simples: a erosão **escurece** regiões ao substituir cada pixel pelo menor valor presente em sua vizinhança, enquanto a dilatação **clareia** regiões ao utilizar o maior valor disponível. A combinação desses operadores permite construir transformações capazes de realçar bordas, remover tendências de iluminação e destacar estruturas locais.

Três operadores derivados são especialmente úteis:

Gradiente morfológico — destaca bordas como a diferença entre dilatação e erosão:

$$\text{grad}_B(f) = (f \oplus B) - (f \ominus B) \quad (4.12)$$

Top-hat — destaca estruturas brilhantes menores que o elemento estruturante (diferença entre a imagem original e sua abertura):

$$\text{top-hat}_B(f) = f - (f \circ B) \quad (4.13)$$

Black-hat — destaca estruturas escuras menores que o elemento estruturante (diferença entre o fechamento e a imagem original):

$$\text{black-hat}_B(f) = (f \bullet B) - f \quad (4.14)$$

O *Top-hat* extrai detalhes brilhantes que não sobrevivem à abertura, enquanto o *Black-hat* evidencia detalhes escuros removidos pelo fechamento. Já o gradiente morfológico realça transições abruptas de intensidade, produzindo uma representação semelhante à de um detector de bordas.

Para compreender o mecanismo desses operadores em nível local, a Figure 4.13 apresenta um simulador interativo de morfologia em tons de cinza. O simulador permite editar livremente o elemento estruturante,

visualizar seu deslocamento sobre a imagem e acompanhar simultaneamente o perfil unidimensional das intensidades. Dessa forma, torna-se possível observar diretamente como a erosão seleciona mínimos locais, como a dilatação seleciona máximos locais e como o gradiente morfológico emerge da diferença entre esses dois operadores.

O botão localizado no canto superior direito permite alternar entre a visualização original em tons de cinza e uma representação pseudocolorida (*colormap*) apenas nos três tipos gradientes. A versão colorida facilita a percepção visual das variações de intensidade, tornando mais evidente a ação dos operadores morfológicos sobre máximos, mínimos e transições locais da imagem.

Os operadores apresentados estão disponíveis em `morph.py` por meio das funções `mm.gradm`, `mm.tophat` e `mm.blackhat`.

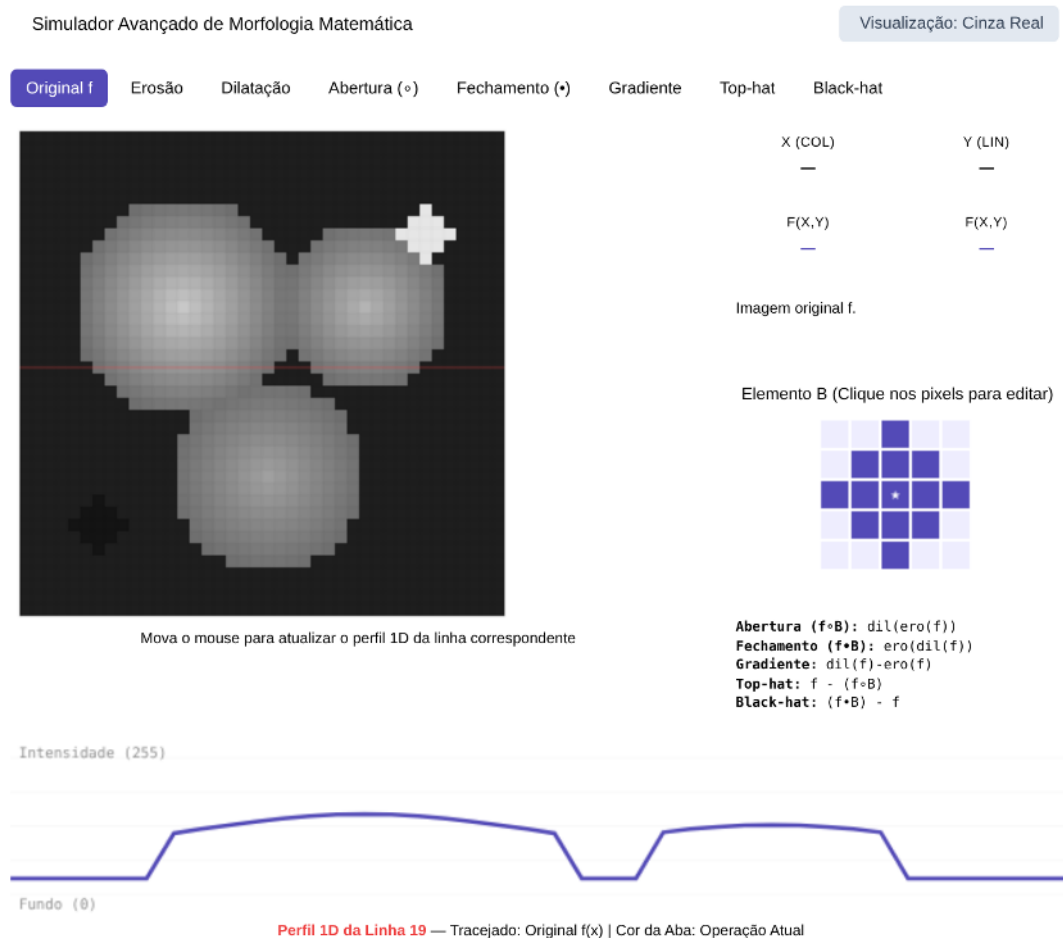


Figura 4.13: Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D.

A Figure 4.14 ilustra os efeitos desses operadores sobre a imagem das moedas e seus respectivos histogramas. Observe que a erosão desloca a distribuição para intensidades mais baixas, enquanto a dilatação a desloca para intensidades mais altas. O gradiente concentra valores nas regiões de contorno, e os operadores *Top-hat* e *Black-hat* produzem histogramas fortemente concentrados em baixos níveis de intensidade, pois apenas pequenas estruturas locais são realçadas.

```
import io
import matplotlib.pyplot as plt
import numpy as np

def fig2img(fig):
    b = io.BytesIO(); fig.savefig(b, format='png', dpi=100); plt.close(fig); b.seek(0)
```

```

    return (plt.imread(b)[: , : , :3] * 255).astype(np.uint8)

def plot_hist(img, title):
    fig, ax = plt.subplots(figsize=(4, 3))
    h = mm.hist(img)
    # CORREÇÃO: range usa len(h) dinamicamente para casar com o retorno da biblioteca
    ax.bar(range(len(h)), h, color='steelblue', width=1, edgecolor='steelblue')
    ax.set(xlim=(0, 255)); plt.tight_layout()
    return fig2img(fig)

# 1. Processamento morfológico base
B = mm.sedisk(19)
operadores = [
    ("Original", img_coins_gray),
    ("Erosão", mm.ero(img_coins_gray, B)),
    ("Dilatação", mm.dil(img_coins_gray, B)),
    ("Gradiente Morf.", mm.gradm(img_coins_gray, B)),
    ("Top-hat", mm.tophat(img_coins_gray, B)),
    ("Black-hat", mm.blackhat(img_coins_gray, B))
]

# 2. Montagem dinâmica do par: [Imagem, Histograma]
imgs, titles = [], []
for nome, img in operadores:
    imgs += [img, plot_hist(img, f"Hist. {nome}")]
    titles += [nome, f"Hist. {nome}"]

# 3. Exibição final em grade de duas colunas (Imagem | Histograma)
mm.show(imgs, titles=titles, cols=4, figsize=(10, 8))

```

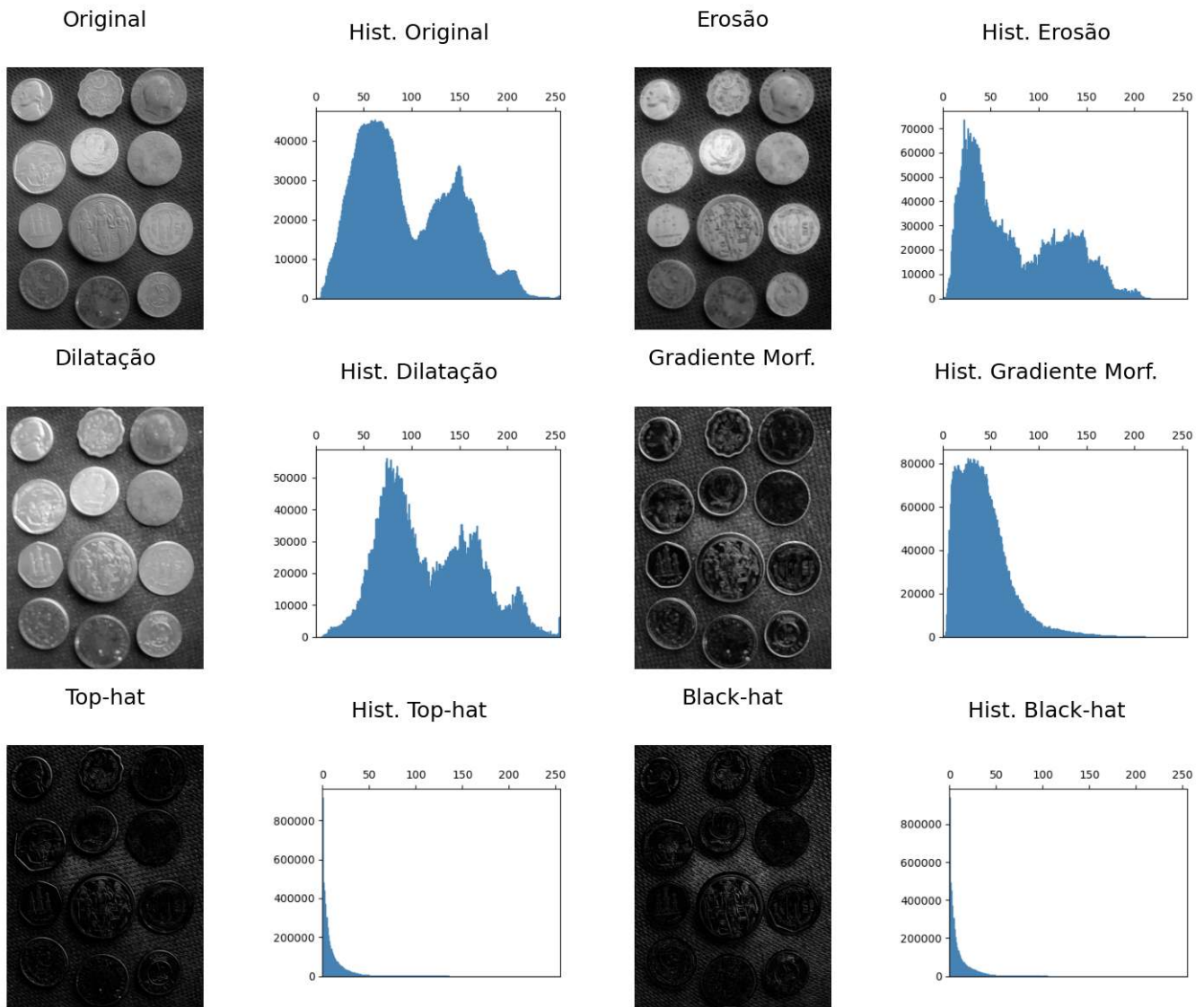


Figura 4.14: Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19.

Os operadores morfológicos apresentados anteriormente serão agora utilizados como ferramentas de refinamento e geração de marcadores para métodos de segmentação mais avançados, apresentados a seguir.

4.4 Segmentação de Imagens: Fundamentação e Taxonomia

A **segmentação de imagens** consiste em dividir a imagem em regiões associadas a objetos ou estruturas de interesse. Em PDI, ela representa a transição entre o processamento de baixo nível — como filtragem e realce — e etapas de análise mais avançadas, como extração de características, reconhecimento e interpretação da cena.

Formalmente, o objetivo da segmentação consiste em decompor o domínio espacial completo de uma imagem, denotado por Ω , em uma partição de subconjuntos $\{R_1, R_2, \dots, R_n\}$ que satisfaça simultaneamente os critérios de **completeza** e **disjunção**:

$$\bigcup_{i=1}^n R_i = \Omega, \quad R_i \cap R_j = \emptyset \quad \forall i \neq j \quad (4.15)$$

Além das propriedades de completeza e disjunção expressas na Equation 4.15, cada sub-região R_i deve constituir um domínio **homogêneo** segundo um predicado de similaridade definido sobre propriedades locais — intensidade, cor ou textura — e, simultaneamente, ser **distinta** das regiões adjacentes.

As técnicas de segmentação podem ser organizadas em diferentes famílias. Neste capítulo serão enfatizadas as abordagens resumidas na Table 4.5, fundamentadas principalmente em critérios de intensidade, conectividade e proximidade espacial.

Tabela 4.5: Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.

Abordagem	Critério de Segmentação	Operadores de Referência
Limiarização	Particionamento do espaço de intensidades	Critério de Otsu, limiarização global e local
Morfologia Matemática	Relações espaciais definidas por elementos/funções estruturantes	Erosão, dilatação, abertura, fechamento e reconstrução
Baseada em Regiões	Homogeneidade local e conectividade espacial	Rotulação de componentes conexos, Transformada de Distância e <i>Watershed</i>

Até este ponto, o desenvolvimento prático concentrou-se na **limiarização**, por meio da combinação entre equalização adaptativa CLAHE e o método global de Otsu. Essa etapa foi complementada por operadores de **reconstrução morfológica** baseados em dilatações geodésicas, implementados pelas funções `mm.infrec`, `mm.clohole` e `mm.edgeoff`, produzindo uma máscara binária limpa e adequada para análise.

Contudo, em cenários onde objetos distintos aparecem conectados na máscara binária — seja por contato físico, sobreposição parcial ou por pontes estreitas de pixels produzidas pela segmentação —, a limiarização deixa de ser suficiente para individualizar cada objeto. Nesses casos, múltiplos objetos passam a compor um único componente conexo, dificultando etapas posteriores de medição e interpretação.

Para superar essa limitação, as próximas seções introduzem três ferramentas complementares: a **Rotulação de Componentes Conexos**, a **Transformada de Distância** e o algoritmo de segmentação por **Watershed** baseado em marcadores. Em conjunto, essas técnicas permitem separar objetos adjacentes, identificar regiões individualmente e extrair descritores geométricos consistentes para análise quantitativa.

4.4.1 Rotulação

A **rotulação de componentes conexas** (*connected component labeling*) é o operador que atribui um identificador inteiro único a cada conjunto de pixels pertencentes à mesma componente conexa em uma imagem binária.

i Definição formal

Dada uma imagem binária f e uma relação de conectividade definida por um elemento estruturante B (tipicamente conectividade-4 ou conectividade-8), o algoritmo de rotulação da Figure 4.15 produz uma imagem g na qual todos os pixels pertencentes à mesma componente conexa recebem o mesmo rótulo inteiro positivo, enquanto pixels pertencentes a componentes distintas recebem rótulos diferentes.

A conectividade define quais pixels são considerados vizinhos diretos de um pixel (x, y) . As definições mais utilizadas são:

- **Conectividade-4:** considera apenas os quatro vizinhos ortogonais (norte, sul, leste e oeste).
- **Conectividade-8:** considera os quatro vizinhos ortogonais e os quatro diagonais, totalizando oito vizinhos.

A escolha da conectividade influencia diretamente a formação das componentes conexas e, consequentemente, o resultado da rotulação, como ilustrado na Figure 4.17. Um exemplo adicional pode ser explorado interativamente no simulador apresentado na Figure 4.16.

A implementação em `morph.py` disponibiliza duas versões desse operador. A função `mm.label0` reproduz explicitamente o algoritmo de *flood-fill* utilizando uma pilha e permite controlar a conectividade por meio do elemento estruturante adotado. Já `mm.label` delega a operação à implementação otimizada do OpenCV (`cv2.connectedComponents`). Em ambos os casos, o resultado é uma imagem rotulada na qual cada componente conexa recebe um identificador inteiro distinto.

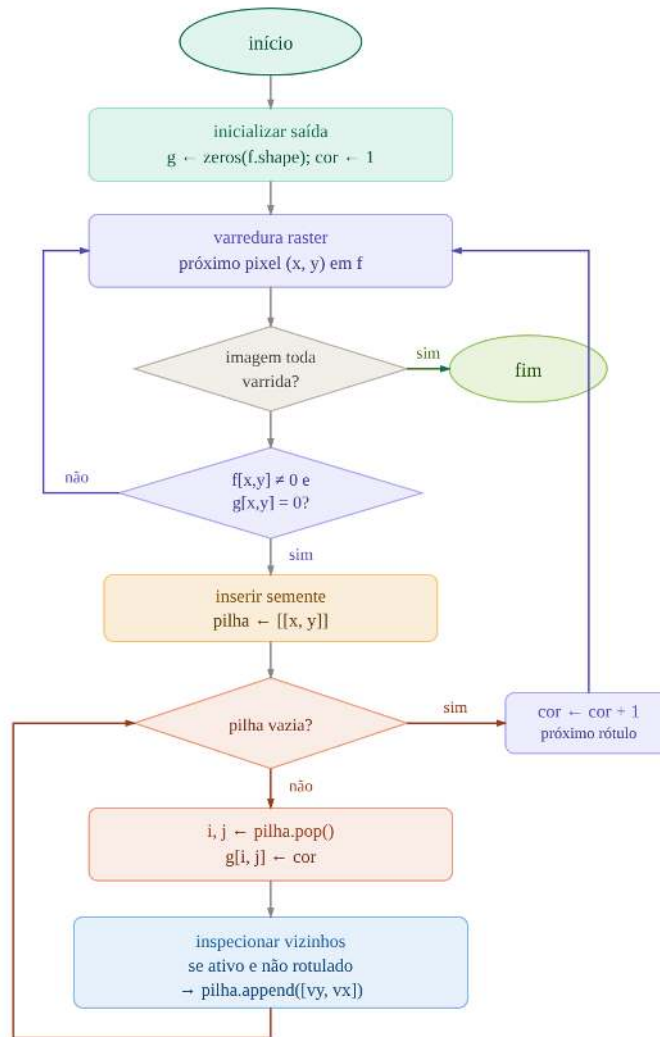


Figura 4.15: Algoritmo de rotulação por *flood-fill* com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.

O auxiliar `_viz` itera sobre a janela estruturante `b` centrada em (i, j) , gerando somente os vizinhos válidos dentro dos limites da imagem — a conectividade desejada é inteiramente determinada pela forma de `b` passada ao algoritmo.

Exemplo didático — efeito da conectividade:



Figura 4.16: Simulador interativo de rotulação de componentes conexas (*connected component labeling*): visualização da expansão flood-fill, conectividade-4 e conectividade-8.

```

# Imagem binária 10×10 com componentes diagonalmente adjacentes
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0,0,0], # diagonal com linha anterior
    [0,0,0,1,0,0,1,1,0,0],
    [0,0,0,0,0,0,1,1,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [0,1,1,1,0,0,0,0,1,0], # diagonal com linha anterior
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

# Elemento estruturante cruz (conectividade-4) e quadrado (conectividade-8)
B4 = mm.secross() # conectividade-4
B8 = mm.sebox()   # conectividade-8

# Rotulação com label0 (didático) e label (cv2)
lbl4_didatico = mm.label0(f, B4)
lbl8_didatico = mm.label0(f, B8)

_, lbl4_cv2 = cv2.connectedComponents(f, connectivity=4)
_, lbl8_cv2 = cv2.connectedComponents(f, connectivity=8)

# Validação cruzada
print(f" label0 (C4) == cv2 (C4): {np.array_equal(lbl4_didatico, lbl4_cv2)}")
print(f" label0 (C8) == cv2 (C8): {np.array_equal(lbl8_didatico, lbl8_cv2)}")
print(f" Componentes C4: {lbl4_cv2.max()} | Componentes C8: {lbl8_cv2.max()}")

# Normalização para visualização
def norm_label(lbl):
    out = np.zeros_like(lbl, dtype=np.uint8)
    for i, v in enumerate(np.unique(lbl)[1:], 1):
        out[lbl == v] = int(i * 255 / lbl.max())
    return out

mm.show(
    [f, norm_label(lbl4_cv2), norm_label(lbl8_cv2)],
    titles=[
        "f original",
        f"Rotulação C4\n({lbl4_cv2.max()} componentes)",
        f"Rotulação C8\n({lbl8_cv2.max()} componentes)"
    ],
    cols=3, figsize=(12, 4), axis=True
)

```

```

label0 (C4) == cv2 (C4): True
label0 (C8) == cv2 (C8): True
Componentes C4: 7 | Componentes C8: 4

```

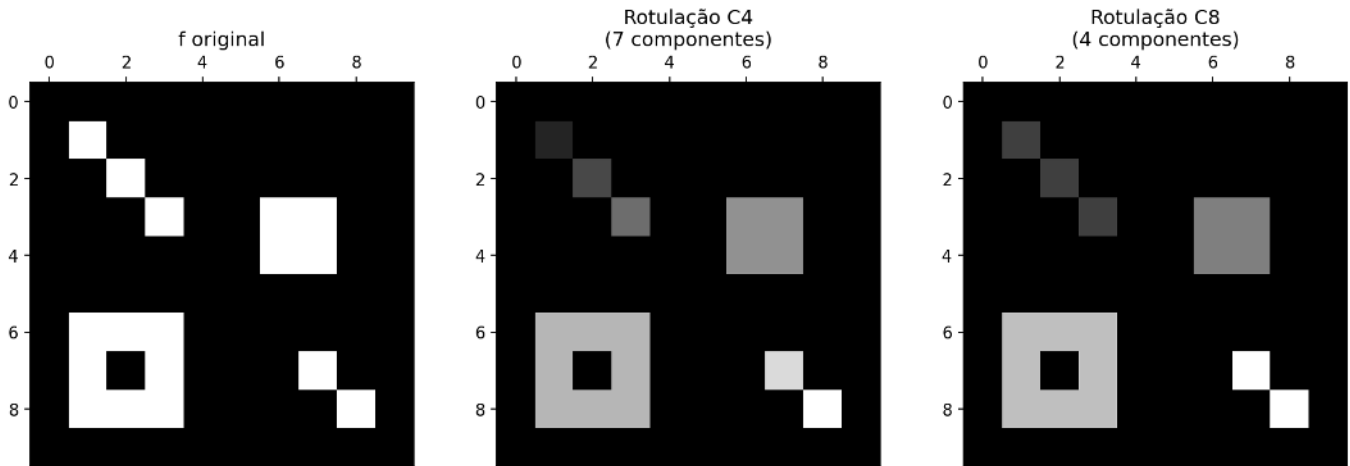


Figura 4.17: Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.

4.4.2 Transformada de Distância

A **Transformada de Distância** (TD) é um operador que, aplicado a uma imagem binária f , produz uma imagem em níveis de cinza D na qual cada pixel pertencente ao objeto ($f(x, y) \neq 0$) recebe como valor a distância geométrica até o pixel de fundo ($f(x', y') = 0$) mais próximo:

$$D(x, y) = \min_{(x', y') : f(x', y') = 0} d((x, y), (x', y'))$$

onde $d(\cdot, \cdot)$ é uma métrica de distância — tipicamente a distância Euclidiana (L_2). O resultado é uma representação topográfica dos objetos: pixels localizados no interior assumem valores elevados, enquanto pixels próximos às bordas apresentam baixos valores de distância. Os **máximos locais** de D correspondem aos pontos mais afastados da borda do objeto, frequentemente próximos aos seus centros geométricos ou centros de máxima inscrição — propriedade particularmente útil para a geração automática de marcadores no algoritmo *watershed*.

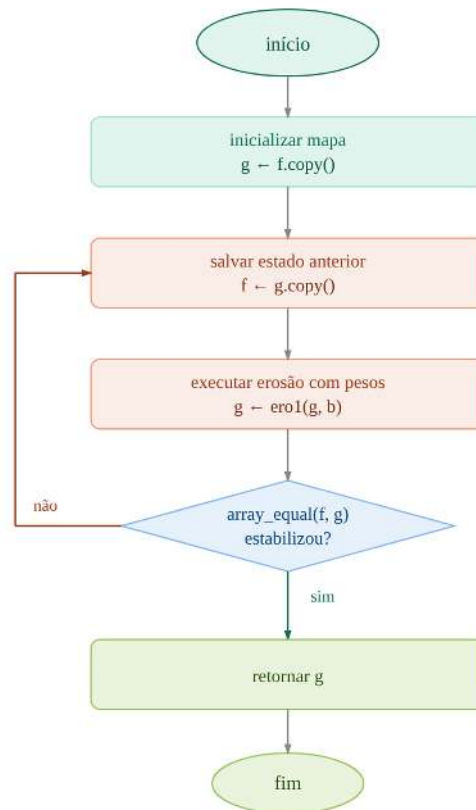
i Definição formal usando erosões

A TD admite ainda uma interpretação morfológica iterativa, conforme algoritmo da Figure 4.18. Considere uma função estruturante b cujo valor central é nulo e cujos vizinhos possuem custos negativos associados ao deslocamento. Ao aplicar erosões sucessivas com essa função estruturante particular, os valores dos pixels dos objetos (que devem assumir a distância máxima possível da imagem) são progressivamente reduzidos segundo os custos definidos por b . O valor acumulado dessa propagação passa então a representar a distância ao fundo segundo a métrica induzida pela função estruturante.

Essa interpretação é implementada em `mm.dist1()`, que acumula erosões sucessivas utilizando a operação `mm.ero1()`. Já `mm.dist()` delega o cálculo da distância Euclidiana ao operador otimizado do OpenCV `cv2.distanceTransform(f, cv2.DIST_L2, 5)`, onde f é a imagem binária de entrada, `cv2.DIST_L2` especifica a métrica Euclidiana (L_2) e 5 indica o uso de uma máscara 5×5 para aproximar a distância com elevada precisão.

A função `dist1` produz uma transformada de distância discreta cuja métrica é determinada pela geometria e pelos pesos da função estruturante utilizada. Por exemplo, utilizando uma função estruturante em cruz com custo unitário para os quatro vizinhos ortogonais, obtém-se a distância de **Manhattan** (L_1). Outras escolhas de vizinhança e pesos induzem métricas diferentes. Já `mm.dist()` calcula uma aproximação eficiente da distância Euclidiana (L_2).

Por exigir sucessivas erosões sobre toda a imagem, a abordagem `dist1` possui custo computacional significativamente maior que `mm.dist()`, sendo empregada neste livro principalmente para fins didáticos e para evidenciar a relação entre morfologia matemática e transformadas de distância.



Ponto Fixo Numérico: A distância emerge da propagação matemática do valor zero.

Figura 4.18: Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.

A Figure 4.19 apresenta um simulador interativo da TD: é possível posicionar o cursor sobre diferentes pixels do objeto e observar, em tempo real, o valor da distância associado àquela posição, isto é, a distância até o pixel de fundo mais próximo. A Figure 4.20 apresenta um exemplo prático dessa execução em ambiente Python.

```

import numpy as np

# Imagem binária 10×10 com um único pixel de fundo (0,0) e o resto como objeto (255)
f = np.ones((10,10), dtype=np.uint8) * 255
f[0,0] = 0

B_cruz = np.array([
    [-np.inf, -1, -np.inf],
    [-1,      0, -1],
    [-np.inf, -1, -np.inf]
], dtype=float)

d_iter = mm.dist1(f, B_cruz)
  
```

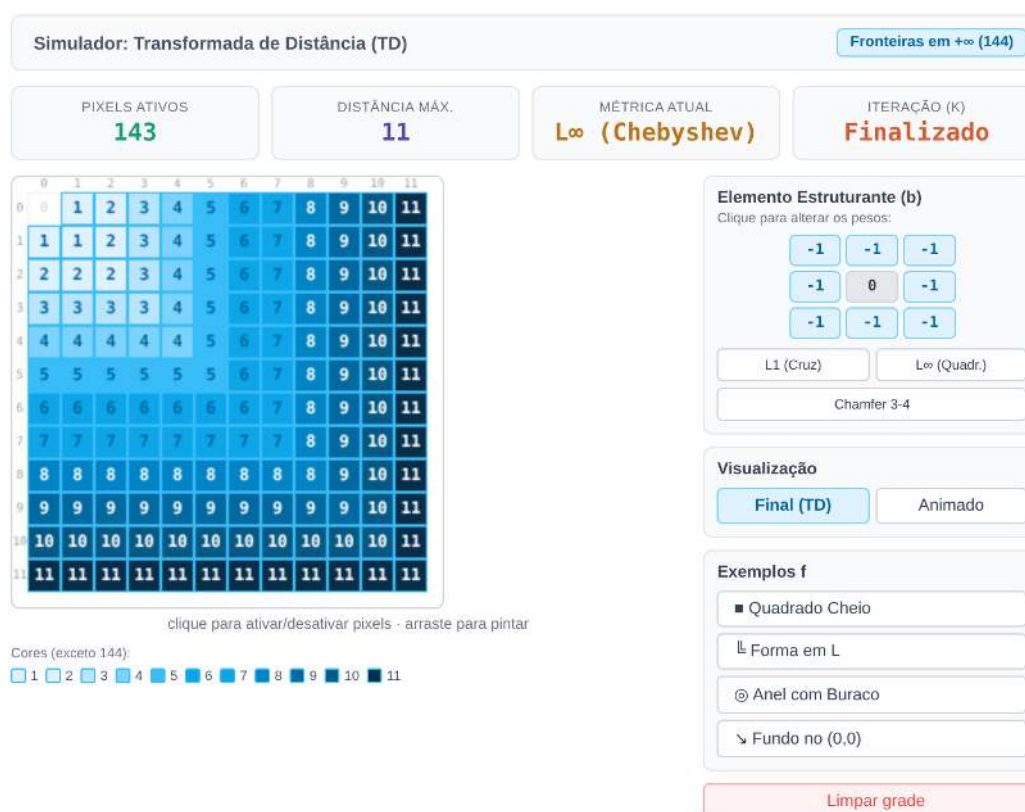


Figura 4.19: Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.

```

d_l2 = mm.dist(f)

print(f"Máx. dist1 (erosões) : {d_iter.max()} px")
print(f"Máx. dist (L2)      : {d_l2.max():.2f} px")
print(f"Posição do máximo dist1 : {np.where(d_iter == d_iter.max())}")
print(f"Posição do máximo dist  : {np.where(d_l2 == d_l2.max())}")

mm.show(
    [f, d_iter, d_l2],
    titles=["f original", "mm.dist1\n(erosões com cruz)", "mm.dist\n(L2 Euclidiana)"],
    cols=3, figsize=(12, 4), axis=True
)

```

```

Máx. dist1 (erosões) : 18 px
Máx. dist (L2)      : 12.00 px
Posição do máximo dist1 : (array([9]), array([9]))
Posição do máximo dist  : (array([9]), array([9]))

```

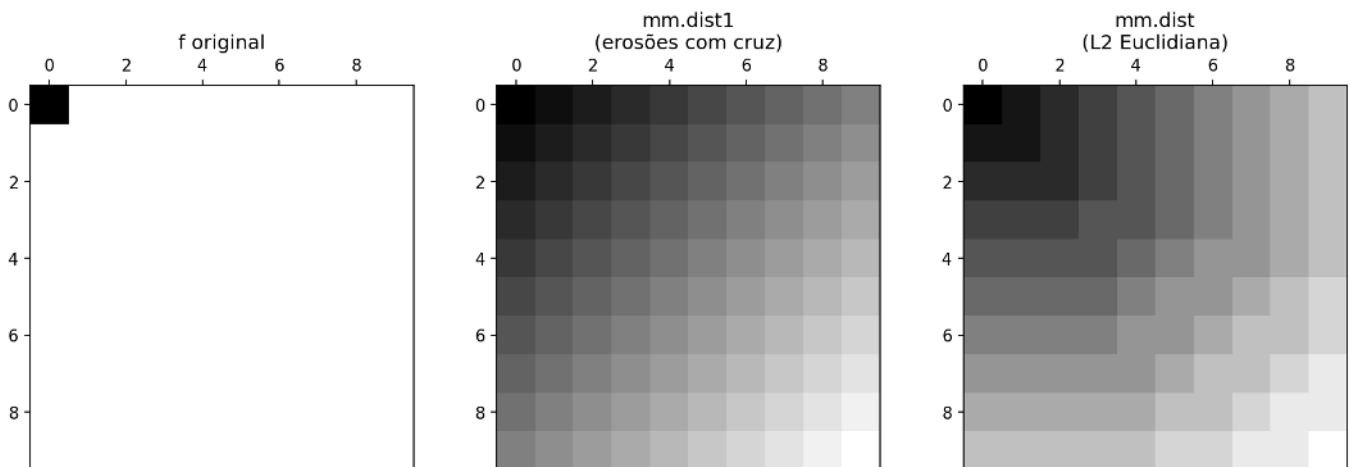


Figura 4.20: Transformada de Distância em imagem binária 10×10. Esquerda: imagem original (*foreground* = 255). Centro: mm.dist1 iterativa (erosões com elemento cruz). Direita: mm.dist (L2 Euclidiana).

A anotação dos valores numéricos diretamente sobre os pixels permite verificar como `dist1` propaga as distâncias segundo a métrica induzida pela função estruturante utilizada. No caso do elemento cruz com custo unitário, os valores obtidos correspondem à distância de *Manhattan* (L_1). Embora `dist1` e `mm.dist` produzam valores numéricos distintos por adotarem métricas diferentes, ambas as transformadas preservam a estrutura topográfica dos objetos, fazendo com que seus máximos ocorram em regiões centrais semelhantes. Essa propriedade justifica o uso de `mm.dist` em aplicações práticas, devido à sua elevada eficiência computacional.

4.4.3 Transformada de Distância Euclidiana em quatro passos

O simulador da Figure 4.21 implementa o algoritmo da TDE de Lotufo; Zampiroli (2001) em duas etapas. Na primeira etapa, a função `edt1` realiza uma transformação unidimensional vertical de forma sequencial (*in-place*), percorrendo cada coluna em *raster* ($\downarrow$) e *anti-raster* ($\uparrow$) para calcular as distâncias na direção vertical (dois passos: Sul e Norte). Na segunda etapa, a função `edt2` utiliza esse resultado como entrada e realiza uma propagação horizontal por filas: para cada linha da matriz, duas filas de prioridade, `Eq` e `Wq`, são inicializadas percorrendo os índices de coluna em sentidos opostos (`Eq` de `W-1` até 1, `Wq` de 2 até `W`), de modo que cada pixel atualizado enfileira imediatamente seus vizinhos para reprocessamento dentro da mesma rodada (mais dois passos: Leste e Oeste). Esse mecanismo de fila permite aplicar erosões sucessivas com pesos ímpares crescentes ($b = 1, 3, 5, \dots$, incrementado a cada iteração do laço externo) sem que a propagação fique presa em valores desatualizados, já que cada rodada resolve a cadeia de dependências horizontal por completo antes do próximo incremento de `b`. A convergência dessa propagação produz a

Transformada de Distância Euclidiana em toda a matriz, combinando a informação vertical obtida em **edt1** com a propagação horizontal em fila realizada em **edt2**.



Figura 4.21: Simulador interativo 2D (4x4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.

4.4.3.1 Transformada de Distância Geodésica

A transformada de distância geodésica associa a cada pixel a menor distância até um marcador, sob a restrição imposta por uma máscara. Dessa forma, a propagação ocorre exclusivamente pelos pixels permitidos, preservando a conectividade do domínio.

A Figura Figure 4.22 ilustra esse processo em um labirinto: (a) a máscara **g**; (b) a distância geodésica **D1** calculada a partir da entrada; (c) a distância **D2** calculada a partir da saída; e (d) o caminho mínimo obtido a partir dessas duas transformadas.

O caminho ótimo é determinado pela soma das distâncias ($D1 + D2$). Os pixels pertencentes à trajetória mínima são aqueles para os quais essa soma assume seu menor valor, definindo uma conexão entre entrada e saída com comprimento geodésico mínimo.

Esse princípio permite resolver labirintos sem a necessidade de explorar explicitamente todas as possibilidades de percurso. A solução emerge diretamente da propagação de distâncias em um domínio restrito. Essa abordagem é particularmente relevante em labirintos de elevada complexidade, como os construídos a partir de estruturas quasicristalinas e ciclos hamiltonianos descritos por Singh; Lloyd; Flicker (2024). Em Zampirolli et al. (2025), esse mesmo formalismo é empregado para resolver um labirinto complexo; a seguir, o método é ilustrado em uma versão simplificada do problema.

```
import numpy as np

# 1 = corredor, 0 = parede
g = np.array([
  [0,1,0,0,0,0,0,0,0,0],
  [0,1,1,1,1,1,0,1,1,1],
  [0,0,0,0,0,1,0,1,0,1],
  [0,1,1,1,0,1,1,1,0,1],
  [0,1,0,1,0,0,0,0,0,1],
  [0,1,0,1,1,1,1,1,1,1],
  [0,1,0,0,0,0,0,0,1,0],
  [0,1,1,1,1,1,1,0,1,0],
])
```

```

[0,0,0,0,0,0,1,1,1,0],
[0,0,0,0,0,0,0,0,1,0]
], dtype=np.uint8)

# marcador da entrada
entrada = np.zeros_like(g, dtype=np.uint8)
entrada[0,1] = 1

# marcador da saída
saida = np.zeros_like(g, dtype=np.uint8)
saida[9,8] = 1

# distâncias geodésicas
D1 = mm.gdist(g, entrada)
D2 = mm.gdist(g, saida)

# soma das distâncias
S = D1 + D2

# menor valor válido da soma
dmin = np.min(S[S > 0])

# pixels pertencentes a um caminho ótimo
caminho = (S == dmin)

print("Distância geodésica mínima:", dmin)

mm.show(
    [g, D1, D2, caminho],
    titles=[
        "Labirinto",
        "Distância da Entrada",
        "Distância da Saída",
        f"Menor Caminho\n(d={dmin})"
    ],
    cols=4,
    figsize=(14,4),
    axis=True
)

```

Distância geodésica mínima: 15

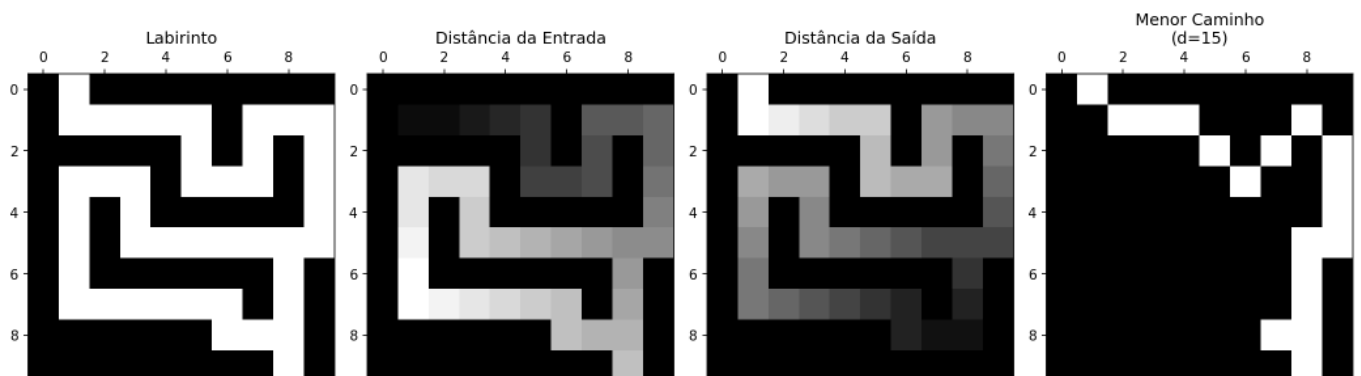


Figura 4.22: Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando `mm.gdist`. Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.

4.4.4 Segmentação por *Watershed*

O algoritmo ***Watershed*** interpreta uma imagem em níveis de cinza como uma superfície topográfica, na qual valores elevados correspondem a montanhas e valores baixos correspondem a vales ou bacias de drenagem (*catchment basins*). No contexto da segmentação baseada em marcadores, os máximos da Transformada de Distância são frequentemente utilizados para identificar regiões internas aos objetos, fornecendo sementes confiáveis para o processo de inundação.

A segmentação é então realizada por uma simulação conceitual de inundação progressiva a partir desses marcadores. À medida que as bacias associadas a diferentes sementes se expandem, regiões vizinhas eventualmente entram em contato. Nesse instante são construídas barreiras virtuais, denominadas *watershed lines*, que passam a delimitar os objetos da cena. Esse mecanismo permite separar objetos adjacentes ou parcialmente sobrepostos, mesmo quando eles formam uma única componente conexa após a limiarização.

A implementação didática apresentada neste capítulo explora inicialmente o conceito de crescimento de regiões (*region growing*) confinado por uma máscara binária, conforme detalhado no algoritmo iterativo da Figure 4.23.

i Versão didática versus implementação clássica

A função `mm.watershed0` não implementa o algoritmo *watershed* clássico. Seu objetivo é ilustrar, de forma simplificada, a propagação de marcadores por crescimento de regiões (*region growing*), permitindo visualizar como diferentes sementes competem pela ocupação do espaço disponível. O crescimento é delimitado por uma máscara binária de suporte e monitorado por um controle de estagnação, gerando um resultado semelhante a uma partição de Voronoi restrita à geometria dos objetos de entrada.

Já a função `mm.watershed` utiliza a implementação otimizada do OpenCV (`cv2.watershed`), que realiza a inundação sobre uma superfície topográfica definida pela imagem de entrada. Nesse caso, a propagação dos marcadores é influenciada pelos valores dos pixels, fazendo com que as linhas de separação se formem naturalmente sobre as cristas do relevo.

A Figure 4.24 apresenta um simulador iterativo que ilustra a propagação dos marcadores pela região de interesse. Cada marcador atua como uma fonte de inundação que expande sua área de influência até encontrar regiões provenientes de outras sementes. No algoritmo do OpenCV, os pixels pertencentes às linhas divisoras são identificados pelo valor `-1`, representando as fronteiras entre bacias adjacentes.

Pipeline Morfológico do *Watershed*

O *watershed* baseado em marcadores normalmente integra um fluxo mais amplo de segmentação. Em imagens reais, etapas de pré-processamento são frequentemente necessárias para melhorar o contraste, reduzir ruídos e gerar marcadores confiáveis. Esse fluxo completo está resumido na Table 4.6.

Tabela 4.6: *Pipeline* completo do *watershed* baseado em marcadores para imagens reais.

Etapa	Operação	Finalidade
1	CLAHE + Suavização	Realce de contraste e redução de ruído
2	Limiarização	Separação inicial entre objeto e fundo
3	Abertura/Fechamento	Remoção de ruídos e pequenas imperfeições
4	Dilatação da Máscara	Identificação do Fundo Certo (<i>Sure Background</i>)
5	Transformada de Distância + Limiar	Identificação do Objeto Certo (<i>Sure Foreground</i>)
6	Região Incerta	Diferença entre Fundo Certo e Objeto Certo

Etapa	Operação	Finalidade
7	<code>cv2.watershed</code>	Propagação dos marcadores pela região incerta

Para enfatizar exclusivamente os conceitos de Transformada de Distância, marcadores e inundação topográfica, o exemplo da Figure 4.25 utiliza uma imagem binária sintética e adota um fluxo simplificado, resumido na Table 4.7.

Tabela 4.7: *Pipeline* simplificado utilizado no exemplo didático da Figure 4.25.

Etapa	Operação	Finalidade
1	Transformada de Distância	Construção da superfície topográfica
2	Limiar da TD	Extração dos marcadores (<i>Sure Foreground</i>)
3	Dilatação	Determinação do Fundo Certo (<i>Sure Background</i>)
4	Região Incerta	Diferença entre fundo e marcadores
5	<code>cv2.watershed</code>	Propagação dos marcadores e geração das fronteiras

```
import cv2

# 1. Imagem sintética e Transformada de Distância
f_sint = np.zeros((20, 20), dtype=np.uint8)
cv2.circle(f_sint, (6, 10), 5, 255, -1)
cv2.circle(f_sint, (14, 10), 5, 255, -1)
dist = mm.dist(f_sint)

# 2. Marcadores (picos da distância)
m = (dist > 0.8 * dist.max()).astype(np.uint8) * 255

# 3. Execução do Watershed0 condicional (m=marcadores primeiro, mask=f_sint)
w_reg = mm.watershed0(m, mask=f_sint, op='region')
w_line = mm.watershed0(m, mask=f_sint, op='line')

#w_reg = mm.watershedB(m, mask=f_sint, op='region')
#w_line = mm.watershedB(m, mask=f_sint, op='line')

w_reg = mm.watershed(m, mask=f_sint, op='region')
w_line = mm.watershed(m, mask=f_sint, op='line')

# 4. Exibição dos resultados
mm.show([f_sint, dist, m, w_reg, w_line], cols=5, figsize=(16, 4),
        titles=["Original", "Distância", "Marcadores", "Regiões", "Linhas"])
```

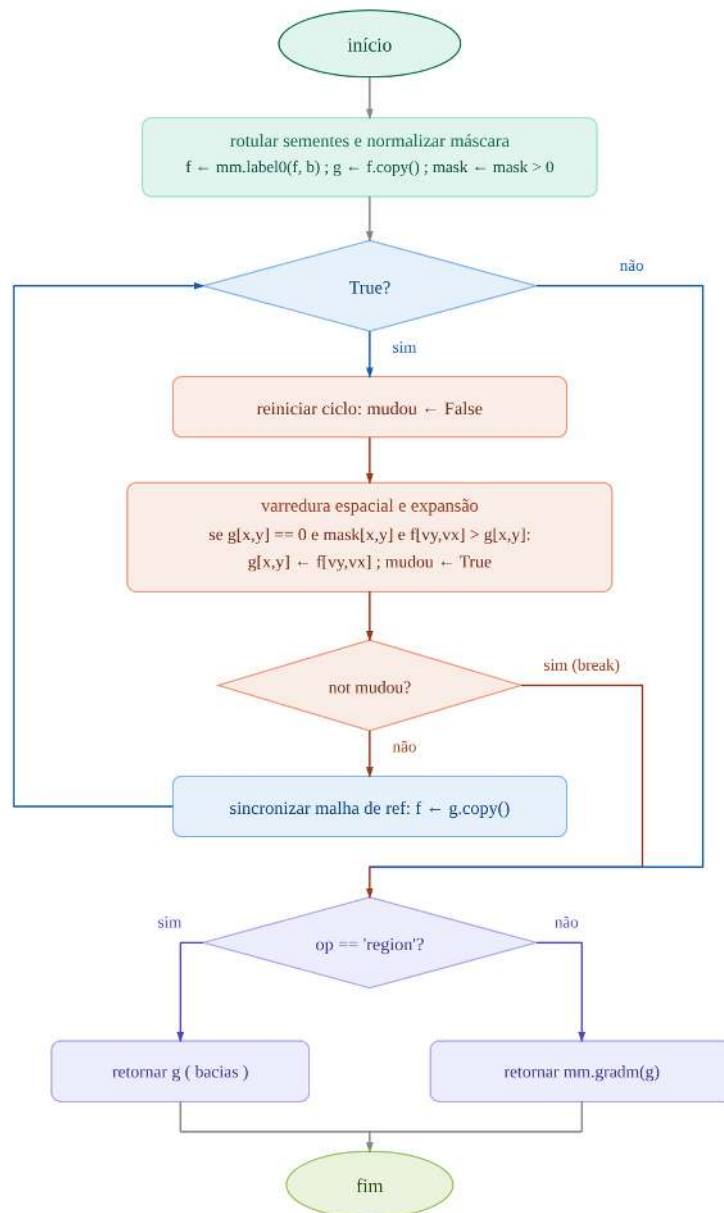


Figura 4.23: Algoritmo Didático de *Watershed* por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.

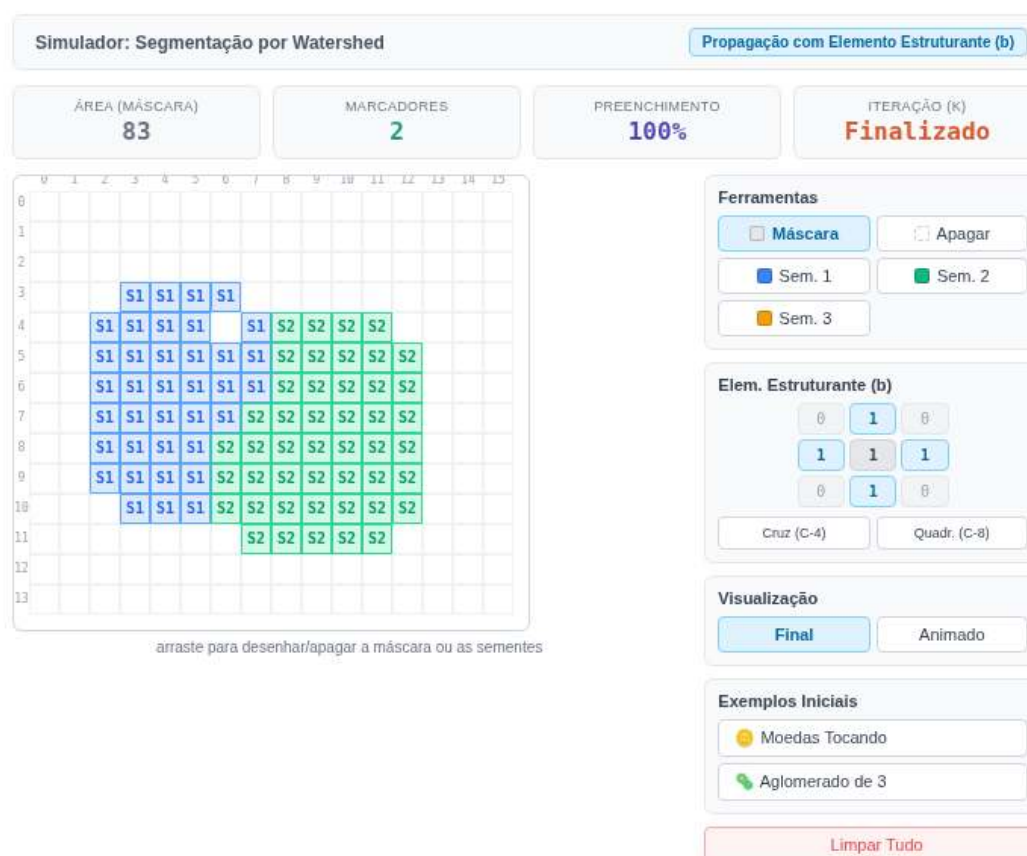


Figura 4.24: Simulador interativo do Algoritmo *Watershed* por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.

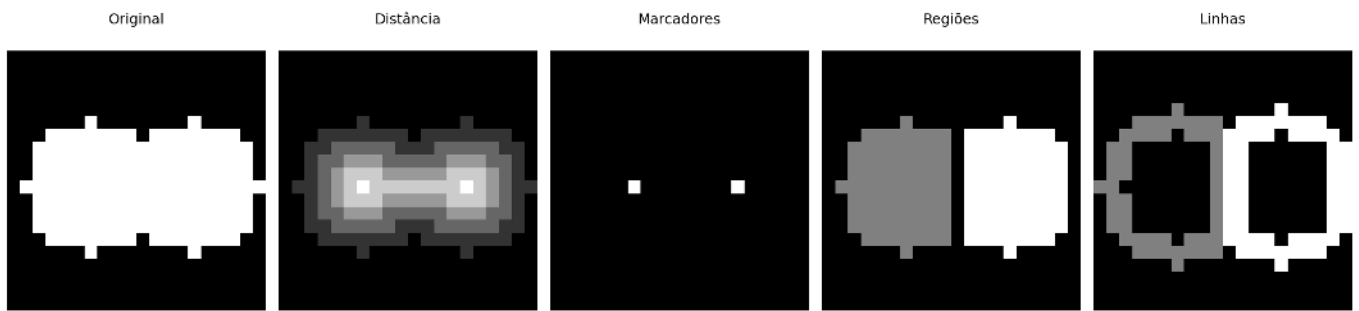


Figura 4.25: *Pipeline watershed* delimitado por máscara em imagem binária 20×20.

Aplicação em moedas sobrepostas:

```
img_base = img_coins_gray

# 1. Simular moedas sobrepostas/conectadas (usando a máscara binária base)
img_sobrepostas = mm.dil(img_final, mm.sebox(40))

# 2. Abertura morfológica para limpar ruídos
opening = mm.open(img_sobrepostas, mm.sebox(2))

# 3. Transformada de Distância
dist = mm.dist(opening)
dist_vis=(255*(dist/dist.max())).astype(np.uint8) if dist.max() > 0 else dist.astype(np.uint8)

# 4. Picos seguros (Marcadores das moedas)
picos = (dist > 0.5 * dist.max()).astype(np.uint8) * 255

# 5. Execução do Watershed (Ajustado para usar a nova assinatura)
# Passamos 'opening' direto em mask, pois ela delimita o escopo de expansão das moedas
ws_region = mm.watershed(picos, mask=opening, op='region')
ws_line = mm.watershed(picos, mask=opening, op='line')

# 6. Contagem de objetos (Ignora fundo 0)
labels = np.unique(ws_region)
labels = labels[labels > 0]
print(f"Objetos detectados: {len(labels)}")

# 7. Anotação Final
img_annotated = cv2.cvtColor(img_base, cv2.COLOR_GRAY2BGR)

for idx, label_id in enumerate(labels):
    mask_reg = (ws_region == label_id).astype(np.uint8)
    if mask_reg.sum() < 500: continue

    cy, cx = np.mean(np.where(mask_reg), axis=1).astype(int)
    cv2.putText(img_annotated, str(idx + 1), (cx - 25, cy + 20),
                cv2.FONT_HERSHEY_SIMPLEX, 3.2, (0, 255, 0), 8, cv2.LINE_AA)

# Desenha as linhas de separação em vermelho
mask_ann = mm.dil(ws_line, np.ones((11, 11), np.uint8))
img_annotated[mask_ann > 0] = [255, 0, 0]

# Exibição
mm.show(
    [img_base, img_sobrepostas, dist_vis, picos, ws_region, img_annotated],
    titles=["Original", "Sobrepostas", "Distância", "Marcadores", "Watershed", "Anotado"],
    cols=3, rows=2, figsize=(12, 8))
```

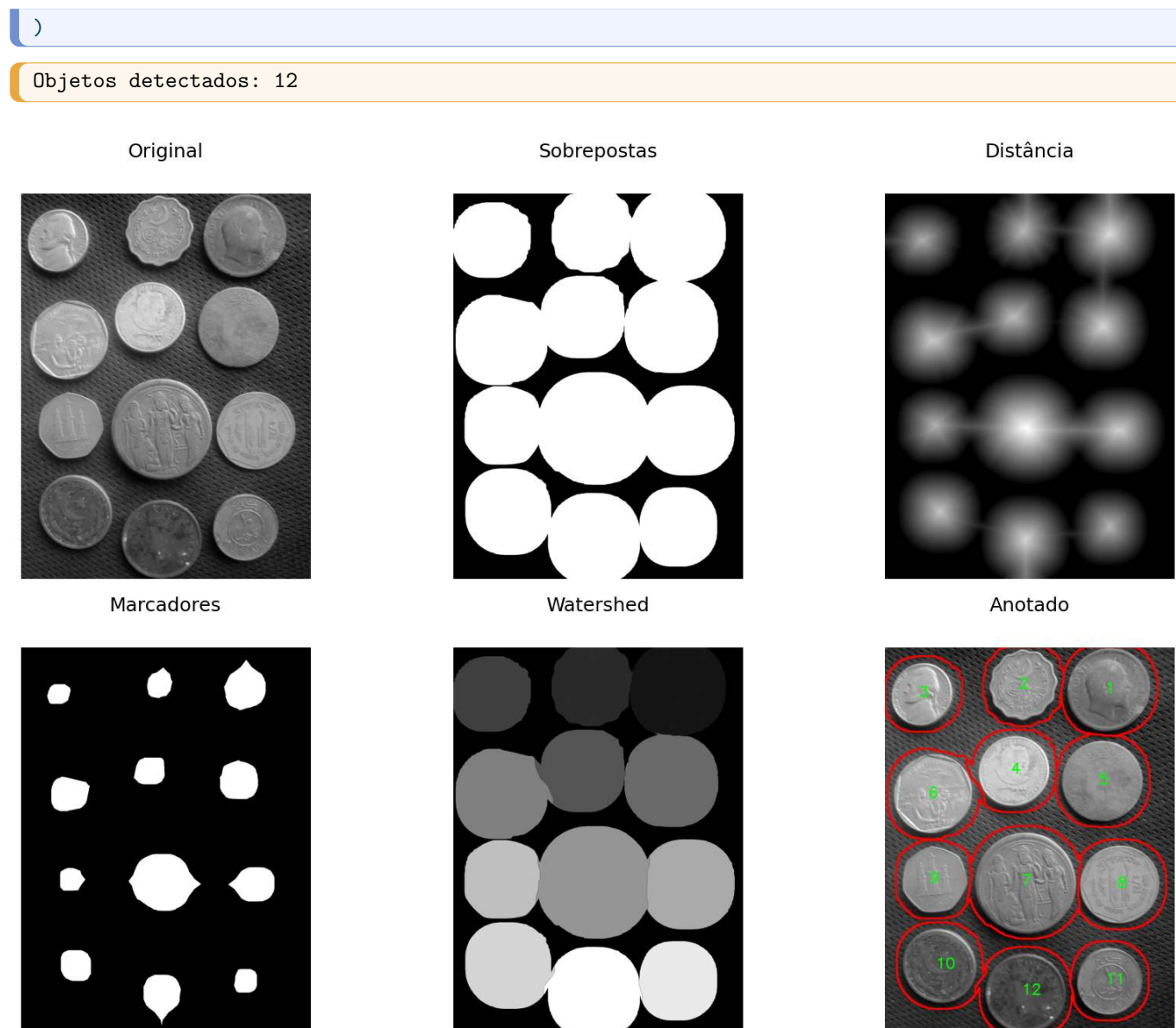


Figura 4.26: *Pipeline Watershed* para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.

4.5 Extração de Componentes e Descritores de Forma

Após a segmentação e o refinamento morfológico, o próximo passo consiste em identificar individualmente cada objeto presente na imagem e extrair suas propriedades geométricas. Essa etapa é fundamental para tarefas de medição, classificação e reconhecimento de padrões.

Um **componente conexo** é um conjunto maximal de pixels pertencentes ao objeto que permanecem mutuamente conectados segundo uma relação de conectividade previamente definida (4- ou 8-conectividade). Após a rotulação, cada componente recebe um identificador único, permitindo que suas características sejam analisadas individualmente.

O OpenCV oferece duas abordagens complementares para essa análise, resumidas na Table 4.8.

Tabela 4.8: Comparação entre as abordagens baseadas em componentes conexos e em contornos.

	<code>connectedComponentsWithStats</code>	<code>findContours</code>
Retorna	rótulo por pixel e estatísticas por componente	sequência de pontos que descreve a borda
Descritores diretos	área, <i>bounding box</i> e centróide	perímetro, forma e hierarquia
Objetos em contato	tende a fundir regiões conectadas	tende a produzir um único contorno externo
Uso típico	contagem, filtragem e rotulação	análise geométrica e descritores de forma

4.5.1 Rotulação e Estatísticas de Componentes

A função `cv2.connectedComponentsWithStats` realiza simultaneamente a rotulação dos componentes conexos e a extração de descritores básicos de cada região. O operador retorna:

- uma imagem de rótulos (`labels`);
- estatísticas geométricas (`stats`);
- coordenadas dos centróides (`centroids`).

As estatísticas incluem área, largura, altura e posição da *bounding box* mínima alinhada aos eixos da imagem.

A Figure 4.27 ilustra a aplicação desse operador após o *pipeline* de segmentação das moedas. Cada componente conexo recebe uma cor distinta e sua área é anotada diretamente sobre a imagem.

```
# 1. Rotulagem e estatísticas
n, labels, stats, centroids = cv2.connectedComponentsWithStats(
    img_final, connectivity=8
)

# 2. Coloração dos componentes
np.random.seed(4)
colors = np.random.randint(50, 255, (n, 3), dtype=np.uint8)
colors[0] = [0, 0, 0] # fundo preto
img_colored = colors[labels]
img_annotated = img_colored.copy()

# 3. Tabela de descritores
print(f"Componentes detectados (excluindo fundo): {n - 1}")
print(f"\n{'ID':>4} {'Área':>8} {'cx':>6} {'cy':>6} {'w':>6} {'h':>6}")
print("-" * 42)
for i in range(1, n):
    area = stats[i, cv2.CC_STAT_AREA]
    cx, cy = int(centroids[i, 0]), int(centroids[i, 1])
    w, h = stats[i, cv2.CC_STAT_WIDTH], stats[i, cv2.CC_STAT_HEIGHT]
    print(f"{i:>4} {area:>8} {cx:>6} {cy:>6} {w:>6} {h:>6}")
    for cor, esp in [(0,0,0), 10), ((255,0,0), 5)]:
        cv2.putText(img_annotated, f"{i}: {area}",
                    (cx - 150, cy + 18),
                    cv2.FONT_HERSHEY_SIMPLEX, 2.0, cor, esp, cv2.LINE_AA)

mm.show(
    [img_coins_gray, img_final, img_colored, img_annotated],
    titles=["Original", "Segmentação Final", "Componentes Conexos", "Áreas Anotadas"],
    cols=4, figsize=(18, 6)
)
```

Componentes detectados (excluindo fundo): 12

ID	Área	cx	cy	w	h
1	231969	1484	280	553	542
2	158967	917	250	453	468
3	139229	250	312	433	419
4	175550	857	821	474	465
5	222882	1442	889	539	531
6	210043	316	977	527	516
7	343376	934	1559	667	665
8	213641	1555	1574	530	515
9	147880	328	1543	432	438
10	187280	363	2111	487	492
11	150110	1487	2215	433	444
12	215387	932	2292	528	522

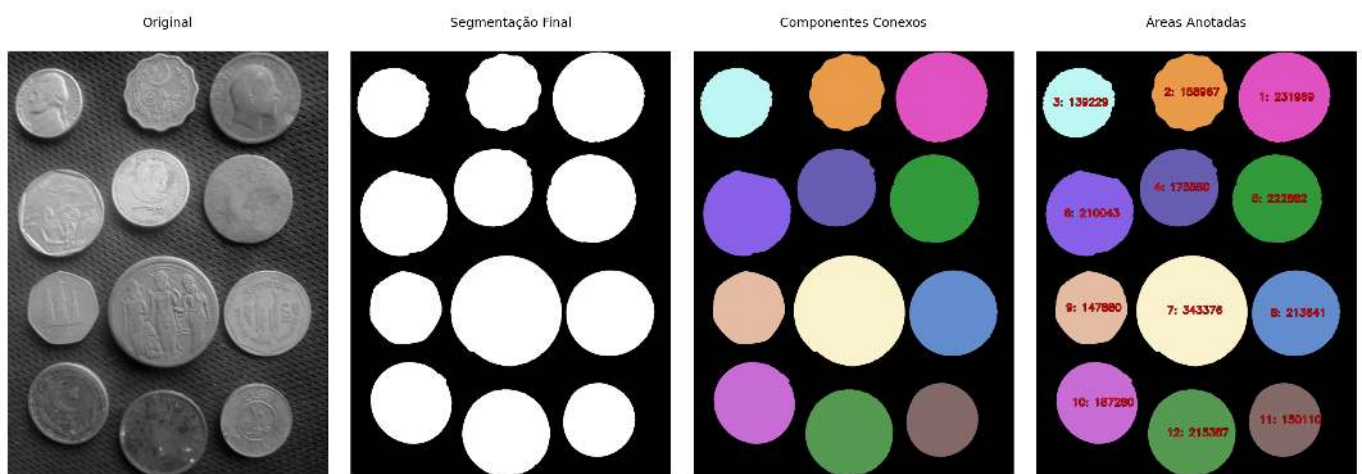


Figura 4.27: Componentes conexos extraídos após o *pipeline* CLAHE → Otsu → limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.

4.5.2 Descritores de Forma

Enquanto `connectedComponentsWithStats` opera sobre regiões, `cv2.findContours` atua diretamente sobre suas fronteiras. O operador retorna, para cada objeto, uma sequência ordenada de pontos que descreve seu contorno.

A partir dessa representação é possível calcular descritores geométricos que não são fornecidos diretamente pela rotulação de componentes:

- **Área** (`cv2.contourArea`)
- **Perímetro** (`cv2.arcLength`);
- **Circularidade** ($C = \frac{4\pi A}{P^2}$, onde A é a área e P o perímetro);
- **Aproximação poligonal** (`cv2.approxPolyDP`);
- **Fecho convexo** (*convex hull*);
- **Hierarquia de contornos**, permitindo representar relações pai-filho entre regiões e seus buracos internos.

A circularidade assume valor máximo igual a 1 para um círculo perfeito e diminui à medida que o objeto se torna mais alongado ou apresenta irregularidades em sua borda.

A Figure 4.28 apresenta os contornos extraídos das moedas segmentadas, juntamente com os valores de circularidade calculados para cada objeto.

```
contornos, hierarquia = cv2.findContours(
    img_final, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

img_contornos = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)
img_circulares = img_contornos.copy()

print(f"Contornos detectados: {len(contornos)}")
print(f"\n{'ID':>4} {'Área':>8} {'Perímetro':>10} {'Circularidade':>14}")
print("-" * 42)

for i, cnt in enumerate(contornos, start=1):
    area = cv2.contourArea(cnt)
    perim = cv2.arcLength(cnt, closed=True)
    circ = (4 * np.pi * area / perim**2) if perim > 0 else 0
    M = cv2.moments(cnt)
    cx = int(M["m10"] / M["m00"]) if M["m00"] > 0 else 0
    cy = int(M["m01"] / M["m00"]) if M["m00"] > 0 else 0

    print(f"{'i':>4} {'area':>8.0f} {'perim':>10.1f} {'circ':>14.3f}")

    cv2.drawContours(img_contornos, [cnt], -1, (0, 255, 0), 3)
    cv2.drawContours(img_circulares, [cnt], -1, (0, 255, 0), 3)
    for cor, esp in [(0,0,0), 8), ((255,0,0), 3)]:
        cv2.putText(img_circulares, f"{circ:.2f}",
                    (cx - 80, cy + 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 3.6, cor, esp, cv2.LINE_AA)

mm.show(
    [img_final, img_contornos, img_circulares],
    titles=["Segmentação Final", "Contornos", "Circularidade"],
    cols=3, figsize=(18, 6)
)
```

Contornos detectados: 12

ID	Área	Perímetro	Circularidade

1	214626	1769.6	0.861
2	149480	1465.1	0.875
3	186570	1654.9	0.856
4	147252	1458.6	0.870
5	212886	1756.3	0.867
6	342424	2232.5	0.863
7	209282	1767.7	0.842
8	222108	1809.7	0.852
9	174854	1616.4	0.841
10	138602	1471.5	0.804
11	158306	1538.7	0.840
12	231181	1847.4	0.851



Figura 4.28: Contornos extraídos com *findContours*. Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.

4.5.3 Conexão com Detecção de Objetos Moderna

Os descritores extraídos nas seções anteriores — especialmente *bounding boxes*, centróides, áreas e medidas de forma — estabelecem uma ponte natural entre a segmentação morfológica clássica e os sistemas modernos de detecção de objetos. Embora as técnicas estudadas neste capítulo utilizem operações sobre pixels e regiões segmentadas, muitas das representações produzidas são diretamente compatíveis com os formatos empregados em modelos contemporâneos de visão computacional.

Detectores baseados em aprendizado profundo, como a família YOLO (*You Only Look Once*) (Redmon et al., 2016), operam diretamente sobre imagens coloridas e produzem, para cada objeto detectado, uma *bounding box* descrita pelo centro (cx, cy) e pelas dimensões (w, h), além de uma classe e uma pontuação de confiança. Essa representação compartilha a mesma estrutura geométrica básica obtida por `connectedComponentsWithStats`, embora seja produzida por um modelo aprendido e não por segmentação explícita.

A Figure 4.29 ilustra como as *bounding boxes* obtidas por morfologia podem ser exportadas no formato YOLO para compor conjuntos de dados utilizados no treinamento ou na avaliação de detectores.

```
H_img, W_img = img_coins_gray.shape
img_bbox = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)

CLASSE = 0 # 0 = moeda (única categoria neste exemplo)

print(f"{'cls':>4} {'cx_n':>8} {'cy_n':>8} {'w_n':>8} {'h_n':>8} ← formato YOLO")
print("-" * 54)

yolo_linhas = []
for i in range(1, n):
    x0 = stats[i, cv2.CC_STAT_LEFT]
    y0 = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]

    cx_n = (x0 + w / 2) / W_img
    cy_n = (y0 + h / 2) / H_img
    w_n = w / W_img
    h_n = h / H_img

    yolo_linhas.append(f"{CLASSE} {cx_n:.4f} {cy_n:.4f} {w_n:.4f} {h_n:.4f}")
print(f"{CLASSE:>4} {cx_n:>8.4f} {cy_n:>8.4f} {w_n:>8.4f} {h_n:>8.4f}")
```

```

cv2.rectangle(img_bbox, (x0, y0), (x0 + w, y0 + h), (0, 255, 0), 4)
cv2.putText(img_bbox, f"moeda", (x0 + 8, y0 + 60),
            cv2.FONT_HERSHEY_SIMPLEX, 3.8, (255, 0, 0), 5, cv2.LINE_AA)

# Exportar arquivo de anotação no formato YOLO
with open("moedas.txt", "w") as f:
    f.write("\n".join(yolo_linhas))
print("\nAnotação salva em moedas.txt")

mm.show(
    [img_coins_gray, img_final, img_bbox],
    titles=["Original", "Segmentação Final", "Bounding Boxes (formato YOLO)"],
    cols=3, figsize=(18, 6)
)

```

cls	cx_n	cy_n	w_n	h_n	← formato YOLO
0	0.7753	0.1102	0.2880	0.2117	
0	0.4784	0.0984	0.2359	0.1828	
0	0.1331	0.1225	0.2255	0.1637	
0	0.4464	0.3217	0.2469	0.1816	
0	0.7523	0.3471	0.2807	0.2074	
0	0.1664	0.3812	0.2745	0.2016	
0	0.4862	0.6104	0.3474	0.2598	
0	0.8115	0.6154	0.2760	0.2012	
0	0.1724	0.6023	0.2250	0.1711	
0	0.1893	0.8258	0.2536	0.1922	
0	0.7763	0.8652	0.2255	0.1734	
0	0.4854	0.8953	0.2750	0.2039	

Anotação salva em moedas.txt

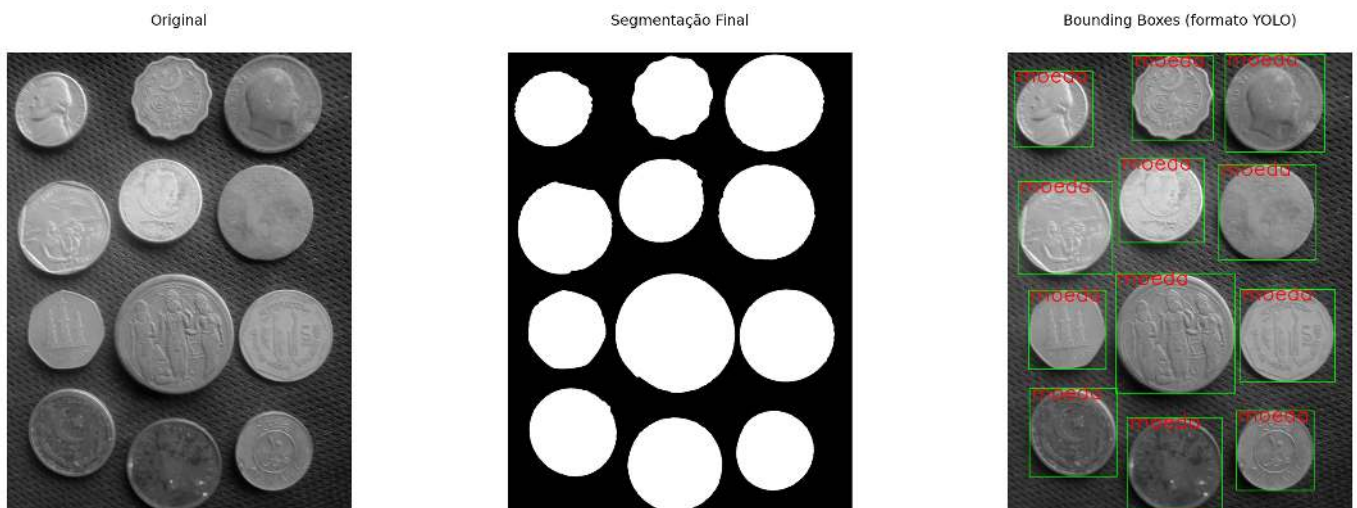


Figura 4.29: *Bounding boxes* derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (*classe cx cy w h*), com coordenadas normalizadas para o intervalo $[0,1]$.

O formato YOLO armazena cada objeto em uma linha contendo cinco campos:

classe cx cy w h

onde (cx, cy) representa o centro da *bounding box* e (w, h) suas dimensões. Todos os valores geométricos são normalizados para o intervalo $[0, 1]$ em relação à largura e à altura da imagem. A **classe** é um identificador

inteiro associado a uma categoria definida pelo conjunto de dados (por exemplo, $0 \rightarrow \text{moeda}$). Quando há múltiplas categorias — como moeda de ouro (0), moeda de prata (1) e disco plástico (2) — basta atribuir o identificador correspondente a cada objeto antes da exportação, mantendo exatamente o mesmo formato de anotação. No exemplo anterior, essas informações foram armazenadas no arquivo `moedas.txt`.

O fluxo apresentado neste capítulo — segmentação $\rightarrow$ rotulação $\rightarrow$ extração de *bounding boxes* — corresponde conceitualmente à etapa de **anotação** (*labeling*) empregada na construção de conjuntos de treinamento para detectores modernos. Ferramentas especializadas, como Label Studio e Roboflow, automatizam esse processo em imagens complexas, mas a lógica fundamental permanece a mesma: associar a cada objeto uma região de interesse e uma classe. Em cenários controlados, com fundo uniforme e objetos bem separados, técnicas morfológicas podem inclusive gerar anotações automaticamente ou servir como ponto de partida para a rotulação manual, reduzindo significativamente o esforço de construção do conjunto de dados. Em aplicações reais mais complexas, entretanto, a validação humana continua sendo necessária para garantir a qualidade das anotações.

Avaliação: IoU (*Intersection over Union*)

Uma forma simples de avaliar a qualidade de uma segmentação consiste em compará-la com uma máscara de referência (*ground truth*). A métrica mais utilizada para essa finalidade é a **IoU** (*Intersection over Union*):

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \quad (4.16)$$

onde A representa a segmentação produzida pelo algoritmo e B a segmentação de referência.

O valor da IoU varia entre 0 e 1. Quanto maior o valor, maior a sobreposição entre as máscaras. Uma IoU igual a 1 indica correspondência perfeita entre a segmentação obtida e a referência.

A mesma métrica também é amplamente utilizada em detecção de objetos, sendo aplicada às *bounding boxes* previstas e anotadas. Nessa área, valores de IoU superiores a 0,5 são frequentemente adotados como critério mínimo para considerar uma detecção correta.

Além da Morfologia

As técnicas estudadas neste capítulo segmentam objetos explorando conectividade espacial, operações morfológicas e relevo topográfico. Existem, entretanto, abordagens alternativas baseadas em agrupamento de características, como o algoritmo *k-means*, modelos de mistura Gaussiana (GMM) e métodos mais recentes baseados em aprendizado profundo. Essas técnicas serão retomadas na Parte II do livro, dedicada à Visão Computacional.

4.6 Resumo

Neste capítulo foram apresentadas as principais técnicas de segmentação e morfologia matemática, concluindo o estudo do PDI no domínio espacial.

- **Pré-processamento e limiarização:** A combinação entre equalização adaptativa CLAHE e o método de Otsu mostrou-se eficaz para induzir separação bimodal no histograma e simplificar a binarização de imagens com iluminação não uniforme.
- **Erosão e dilatação:** Operadores morfológicos fundamentais baseados na busca por mínimos e máximos locais em uma vizinhança definida pelo elemento estruturante B . São operadores duais pelo complemento e foram implementados por meio das funções `mm.ero` e `mm.dil`.
- **Abertura e fechamento:** Composições de erosão e dilatação que permitem remover ruídos, suavizar contornos e preencher pequenas lacunas, preservando a estrutura global dos objetos.
- **Reconstrução morfológica:** Processo geodésico iterativo que propaga um marcador dentro dos limites impostos por uma máscara, constituindo a base de operadores como `mm.clohole` e `mm.edgeoff`.

- **Pipeline de limpeza binária:** Fluxo consolidado composto por CLAHE → Otsu → abertura → `mm.clohole` → abertura restrita → `mm.edgeoff`, produzindo máscaras adequadas para análise quantitativa.
- **Morfologia em tons de cinza:** Extensão algébrica baseada em mínimos e máximos ponderados, viabilizando operadores como gradiente morfológico e filtros *top-hat* para realce de estruturas locais.
- **Transformada de Distância e Watershed:** A Transformada de Distância permitiu gerar marcadores automáticos para o algoritmo *watershed*, possibilitando a separação de objetos adjacentes ou parcialmente sobrepostos.
- **Componentes conexos e descritores:** A rotulação de regiões (`cv2.connectedComponentsWithStats`) e a extração de contornos (`cv2.findContours`) permitiram calcular descritores geométricos como área, centróide, perímetro, circularidade e *bounding boxes*.
- **Conexão com Visão Computacional Moderna:** As *bounding boxes* extraídas por morfologia foram exportadas no formato YOLO, evidenciando a ligação entre técnicas clássicas de segmentação e sistemas modernos de detecção de objetos.

O Capítulo 5 introduzirá técnicas de processamento no **domínio da frequência**, abordando a **Transformada de Fourier**, filtragem espectral e os fundamentos da **compressão de imagens**, incluindo DCT, JPEG e *wavelets*.

4.7 Uso do NotebookLM como Tutor Complementar

Nesta edição, o uso do **NotebookLM** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 04](#)

Aviso sobre Conteúdo Gerado por IA

Embora seja uma ferramenta valiosa de apoio aos estudos, o NotebookLM pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

4.8 Lista de Exercícios

1. (10%) Implemente manualmente o critério de Otsu sem utilizar `cv2.threshold`. Calcule a variância interclasses $\sigma_B^2(T)$ para todos os limiares $T \in [0, 255]$ usando `mm.hist`, identifique o limiar ótimo T^* e compare o resultado com o valor obtido pelo OpenCV. Plote σ_B^2 em função de T e destaque o ponto de máximo.
2. (15%) Aplique limiarização adaptativa com blocos de tamanho 11, 31 e 51 a uma imagem contendo iluminação não uniforme. Compare os resultados com a limiarização global de Otsu e discuta as vantagens e limitações de cada abordagem.
3. (15%) Execute o *pipeline watershed* da imagem de moedas variando o limiar aplicado à Transformada de Distância (0.3, 0.5 e 0.7 vezes o valor máximo). Explique como esse parâmetro influencia a geração dos marcadores, a separação de objetos adjacentes e a ocorrência de sobre-segmentação.
4. (15%) Utilizando `mm.drawImage`, construa uma demonstração visual passo a passo da erosão de uma imagem binária 7×7 com elemento estruturante quadrado 3×3 . Para cada posição analisada, indique

se o elemento estruturante está completamente contido no objeto e justifique o valor atribuído ao pixel de saída.

5. (15%) Demonstre experimentalmente a dualidade entre erosão e dilatação verificando a identidade $(A \ominus B)^c = A^c \oplus \hat{B}$ utilizando `mm.ero`, `mm.dil` e `mm.bnot`. Calcule a diferença pixel a pixel entre os dois lados da equação e apresente o resultado utilizando `mm.histImg` ou visualização equivalente.
6. (15%) Implemente manualmente o gradiente morfológico utilizando apenas `mm.ero` e `mm.dil`, comparando o resultado com `cv2.morphologyEx(img, cv2.MORPH_GRADIENT, B)`. Avalie o efeito de diferentes elementos estruturantes (quadrado 3×3 , disco 5×5 e linha 1×9) sobre a detecção de bordas.
7. (15%) Construa um *pipeline* completo para contagem e classificação de moedas por tamanho (pequena, média e grande) utilizando área e circularidade como descritores. Gere uma máscara de referência (*ground truth*) manualmente e calcule a métrica IoU (*Intersection over Union*) para avaliar a qualidade da segmentação. Apresente os resultados em tabela e por meio de visualizações produzidas com `mm.show`.

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de segmentação, limiarização de Otsu, Transformada de Distância, *watershed*, morfologia matemática e descritores de forma.
- Matheron (1975) e Serra (1982) para a fundamentação teórica original, formulação algébrica e desenvolvimento da Morfologia Matemática.
- Szeliski (2022) para segmentação baseada em regiões, rotulação de componentes conexos, *watershed* baseado em marcadores e avaliação de segmentação por meio da métrica IoU.
- Bradski; Kaehler (2008) para a utilização prática da biblioteca OpenCV, incluindo funções como `cv2.distanceTransform`, `cv2.watershed`, `cv2.connectedComponentsWithStats` e `cv2.findContours`.
- Redmon et al. (2016) para a introdução aos detectores modernos da família YOLO e sua relação com descritores geométricos como *bounding boxes* extraídas por segmentação.
- Singh; Lloyd; Flicker (2024) para a construção de labirintos complexos baseados em ciclos hamiltonianos sobre mosaicos quasicristalinos, utilizados como exemplo de aplicação da Transformada de Distância Geodésica e de algoritmos de busca de caminhos.
- Zampirolli et al. (2025) para a implementação dos operadores morfológicos, transformadas geodésicas e resolução de labirintos por propagação de distâncias em domínios restritos.



4.9 Parte Prática com Exercícios de Programação

Esta lista transforma os conceitos do Capítulo 4 em uma trilha prática de segmentação e morfologia matemática. Os EPs começam com limiarização e avançam até rotulação e descritores de componentes, sempre com matrizes pequenas para que cada pixel possa ser conferido à mão.

! Regra comum dos EPs morfológicos

Nas operações com vizinhança, **não faça padding**. Para cada pixel, avalie apenas as posições do elemento estruturante que caem dentro do domínio da imagem. Essa é a mesma ideia das implementações didáticas em `morph.py`, como `mm.dil0`, `mm.ero0`, `mm.dil1` e `mm.label0`: a vizinhança é recortada pelo domínio válido da imagem.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.0

Executando os Testes

Para rodar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

4.9.1 EP04_01 Limiarização Global por Limiar Fixo

Em **scanners de documentos** e em **sistemas de leitura de código de barras**, a primeira etapa do processamento é sempre separar o que é “objeto” (tinta, texto, barras) do que é “fundo” (papel, embalagem). A **limiarização global** faz exatamente isso: compara cada pixel a um único limiar T e decide, em tempo real, se ele pertence à classe clara ou à classe escura. É o operador de segmentação mais simples — e mesmo assim, está por trás de boa parte dos *pipelines* industriais de inspeção visual. Ver na Figure 4.30 uma simulação deste EP.

4.9.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Limiar:** Ler o inteiro T (limiar de decisão).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \begin{cases} 255, & \text{se } p > T \\ 0, & \text{se } p \leq T \end{cases}$$

5. **Saída:** Exibir a matriz binarizada com dimensões $L \times C$.

4.9.1.2 Restrições Computacionais

- **Binarização:** A saída contém **apenas** os valores 0 ou 255.
- **Comparação estrita:** O critério usa $> T$ (pixels iguais a T tornam-se fundo).
- **Tipo:** O resultado final deve ser inteiro.
- **Observação:** Este EP segue a convenção da OpenCV (`cv2.THRESH_BINARY`): apenas pixels com valor **maior que** T tornam-se brancos (255); pixels com valor **igual a** T permanecem pretos (0).

4.9.1.3 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
T pequeno	Inteiro	A maioria dos pixels torna-se branca
T grande	Inteiro	A maioria dos pixels torna-se preta
T bem escolhido	Inteiro	Separa nitidamente objeto e fundo

4.9.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro T .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255 separados por espaço.

4.9.1.5 Exemplos

Entrada	Saída	Observação
2 4 100 0 99 100 180 255 30 120 80	0 0 0 255 255 0 255 0	$T = 100$: apenas pixels com valor maior que 100 tornam-se brancos; por isso, 99 e 100 tornam-se pretos.
1 3 0 0 50 255	0 255 255	
		$T = 0$: apenas pixels com valor estritamente maior que 0 tornam-se brancos.

```
%%writefile EP04_01.py
# Código Python
```

```
Writing EP04_01.py
```

```
TestSuite("EP04_01.py").run()
```

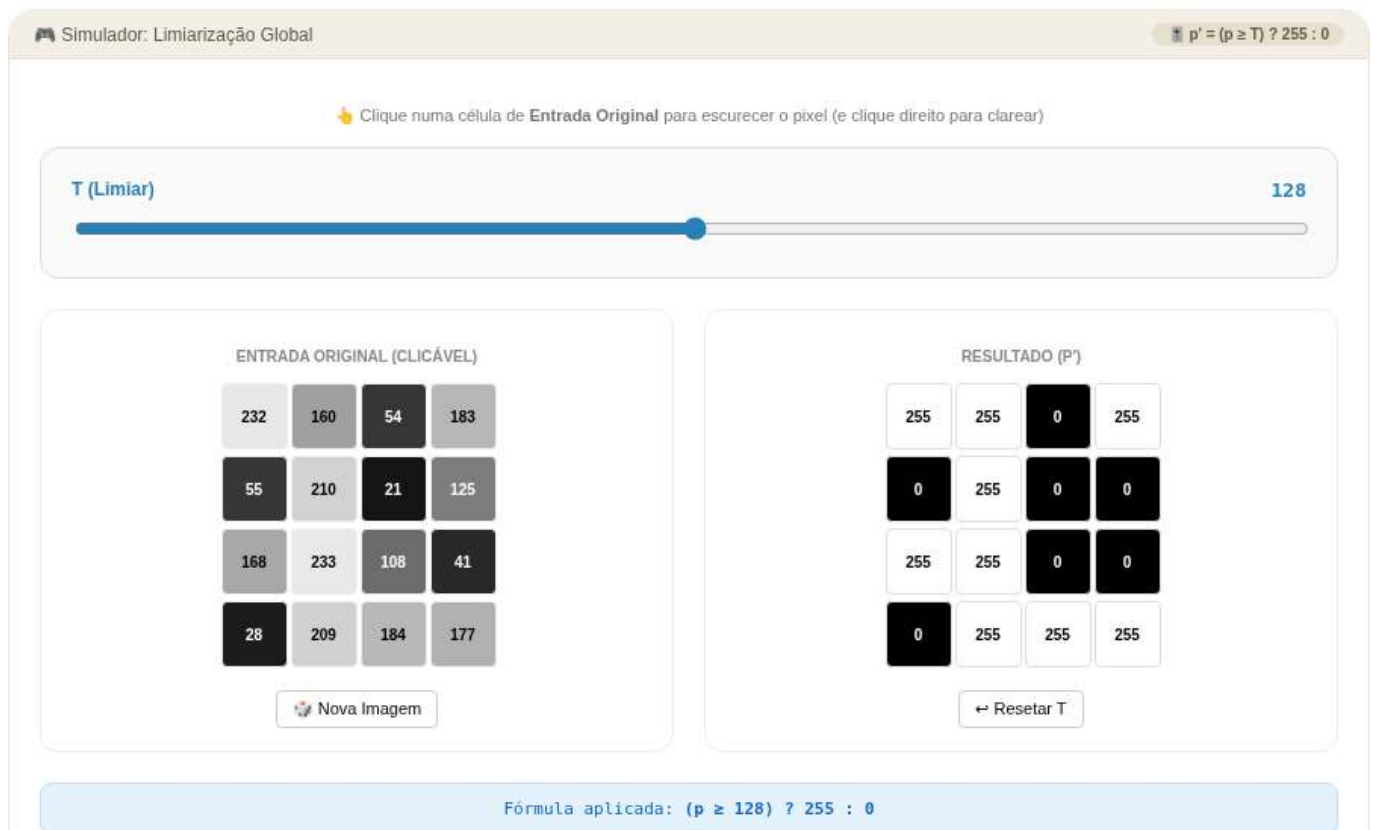


Figura 4.30: Simulador: Limiarização Global por Limiar Fixo

4.9.2 EP04_02 Limiarização Automática de Otsu

Escolher manualmente o limiar T funciona quando a iluminação é estável, mas em **microscopia digital** e em **inspeção de lâminas de sangue**, cada amostra tem um contraste diferente — um limiar fixo falharia de imagem para imagem. O **método de Otsu** resolve isso encontrando, sozinho, o limiar que **maximiza a separação estatística** entre as duas classes de pixels, tornando a segmentação automática e adaptativa. Ver na Figure 4.31 uma simulação deste EP.

4.9.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original linha a linha.
3. **Histograma:** Construir o histograma $h[i]$, $i = 0, \dots, 255$, contando quantos pixels têm valor i .
4. **Busca do limiar:** Para cada candidato T de 1 a 255, calcular a **variância entre classes**:

$$\sigma_B^2(T) = \frac{n_0 \cdot n_1}{N^2} (m_0 - m_1)^2$$

onde n_0, n_1 são as quantidades de pixels com valor $< T$ e $\geq T$, m_0, m_1 são suas médias, e $N = L \times C$.

5. **Escolha:** O limiar ótimo T^* é o que maximiza $\sigma_B^2(T)$ (em caso de empate, manter o **primeiro** encontrado).
6. **Aplicação:** Binarizar a imagem usando T^* , aplicando:

$$p' = \begin{cases} 255, & \text{se } p > T^* \\ 0, & \text{se } p \leq T^* \end{cases}$$

4.9.2.2 📌 Restrições Computacionais

- **Candidatos válidos:** Ignorar T que deixe $n_0 = 0$ ou $n_1 = 0$ (classe vazia).
- **Empate:** Sempre manter o **primeiro** T que atingiu o valor máximo de σ_B^2 .
- **Tipo:** T^* e a matriz de saída devem ser inteiros.
- **Convenção OpenCV:** A binarização segue `cv2.THRESH_BINARY`; pixels com valor exatamente igual a T^* tornam-se pretos.

4.9.2.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto
$\sigma_B^2(T)$ alta	Classes bem separadas em T	T é um bom candidato a limiar
Histograma bimodal	Dois “morros” distintos	Otsu encontra o vale entre eles
Histograma unimodal	Um único “morro”	Otsu ainda escolhe <i>algum</i> T , mas a segmentação é pouco confiável

4.9.2.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255.

4.9.2.5 📌 Exemplos

Entrada	Saída	Observação
4	0 0 0 255	Histograma bimodal claro: 12 e 200
4	0 0 255 255	
12 12 12 200	0 255 255 255	
12 12 200 200	255 255 255 255	
12 200 200 200		Apenas dois valores: T^* fica no maior
200 200 200 200		
1	0 250	
2		
10 250		

```
%%writefile EP04_02.py
# Código Python
```

```
Writing EP04_02.py
```

```
TestSuite("EP04_02.py").run()
```

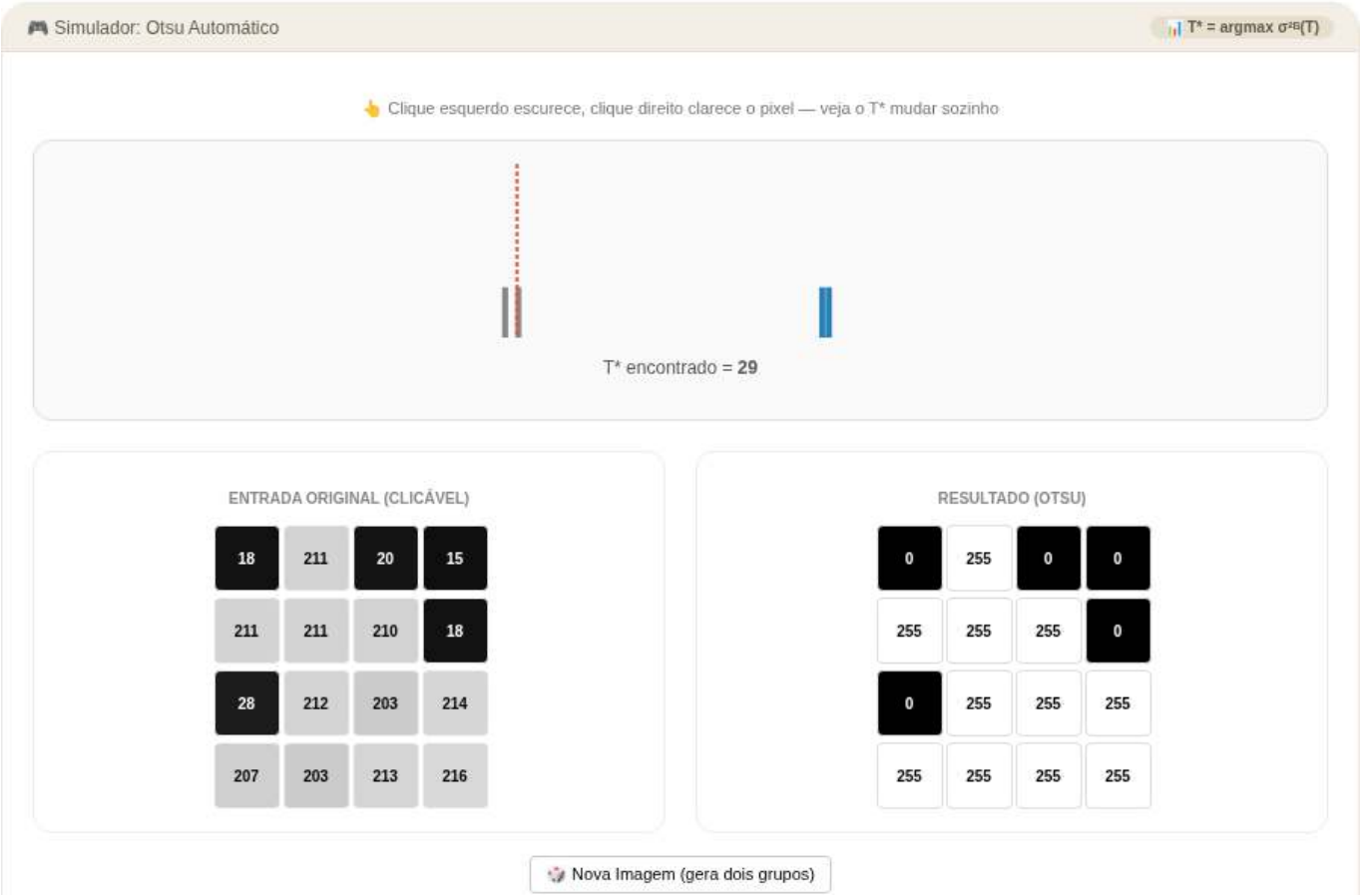


Figura 4.31: Simulador: Limiarização Automática de Otsu

4.9.3 EP04_03 🌱 Dilatação Binária Plana (mm.dil0)

Em **microscopia de partículas** e em **OCR de placas de carro desgastadas**, traços finos ou descontínuos precisam ser “engrossados” para que o reconhecimento funcione. A **dilatação morfológica** faz exatamente isso: expande regiões claras usando um elemento estruturante B — a mesma operação implementada em `morph.py` como `mm.dil0(f, B)`, usada quando B é **plano** (sem pesos, só 0/1). Ver na Figure 4.32 uma simulação deste EP.

4.9.3.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Reflexão:** Construir B_{ref} , a versão de B refletida em 180° (linhas e colunas invertidas) — exatamente como faz `mm.dil0` internamente.
6. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer as posições (by, bx) de B_{ref} centradas em (y, x) , usando o deslocamento

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$ — **não preencher com zeros**.

7. **Mapeamento:** Calcular cada pixel de saída como o **máximo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B_{ref} vale 1:

$$g(y, x) = \max \left(f(y, x), \max_{\substack{(v_y, v_x) \text{ válido} \\ B_{ref}(by, bx)=1}} f(v_y, v_x) \right)$$

8. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.3.2 🚫 Restrições Computacionais

- **Sem padding:** Jamais inventar vizinhos fora da imagem; usar apenas os que existem de fato.
- **Reflexão obrigatória:** B deve ser refletido antes de aplicado (é o que diferencia `mm.dil0` de uma simples busca por máximo).
- **Robustez de borda:** Se nenhuma posição válida de $B_{ref} = 1$ cair dentro do domínio para um dado pixel, ele **mantém seu valor original**.

4.9.3.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Dilatação	$g \geq f$ sempre (extensiva)	Regiões claras crescem, buracos escuros encolhem
B maior	Vizinhança mais ampla	Crescimento mais agressivo
Reflexão de B	$B_{ref}(y, x) = B(-y, -x)$	Garante a definição formal de Minkowski da dilatação

4.9.3.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .

- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.3.5 Exemplos

Entrada	Saída	Observação
3	0 9 0	B em cruz simétrico: ponto isolado se expande em cruz
3	9 9 9	
3	0 9 0	
3		
0 1 0		
1 1 1		
0 1 0		
0 0 0		
0 9 0		
0 0 0		
1	200 200 200 80	B horizontal: cada pixel “puxa” o máximo dos vizinhos da linha
4		
1		
3		
1 1 1		
10 200 5 80		

```
%%writefile EP04_03.py
# Código Python
```

```
Writing EP04_03.py
```

```
TestSuite("EP04_03.py").run()
```

4.9.4 EP04_04 Erosão Binária Plana (mm.ero0)

Se a dilatação engrossa, a **erosão** afina. Em **sistemas de contagem de células**, ela é usada para **separar células que se tocam**: ao “comer” as bordas de cada região, conexões finas entre objetos desaparecem antes mesmo de qualquer contagem ser feita. Em **morph.py**, essa é a operação **mm.ero0(f, B)** — a **dual** exata da dilatação, e a única das duas que **não** reflete o elemento estruturante. Ver na Figure 4.33 uma simulação deste EP.

4.9.4.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Vizinhança sem padding (sem reflexão!):** Para cada pixel (y, x) , percorrer as posições (by, bx) de B na ordem original (sem refletir), usando o mesmo deslocamento do EP04_03:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

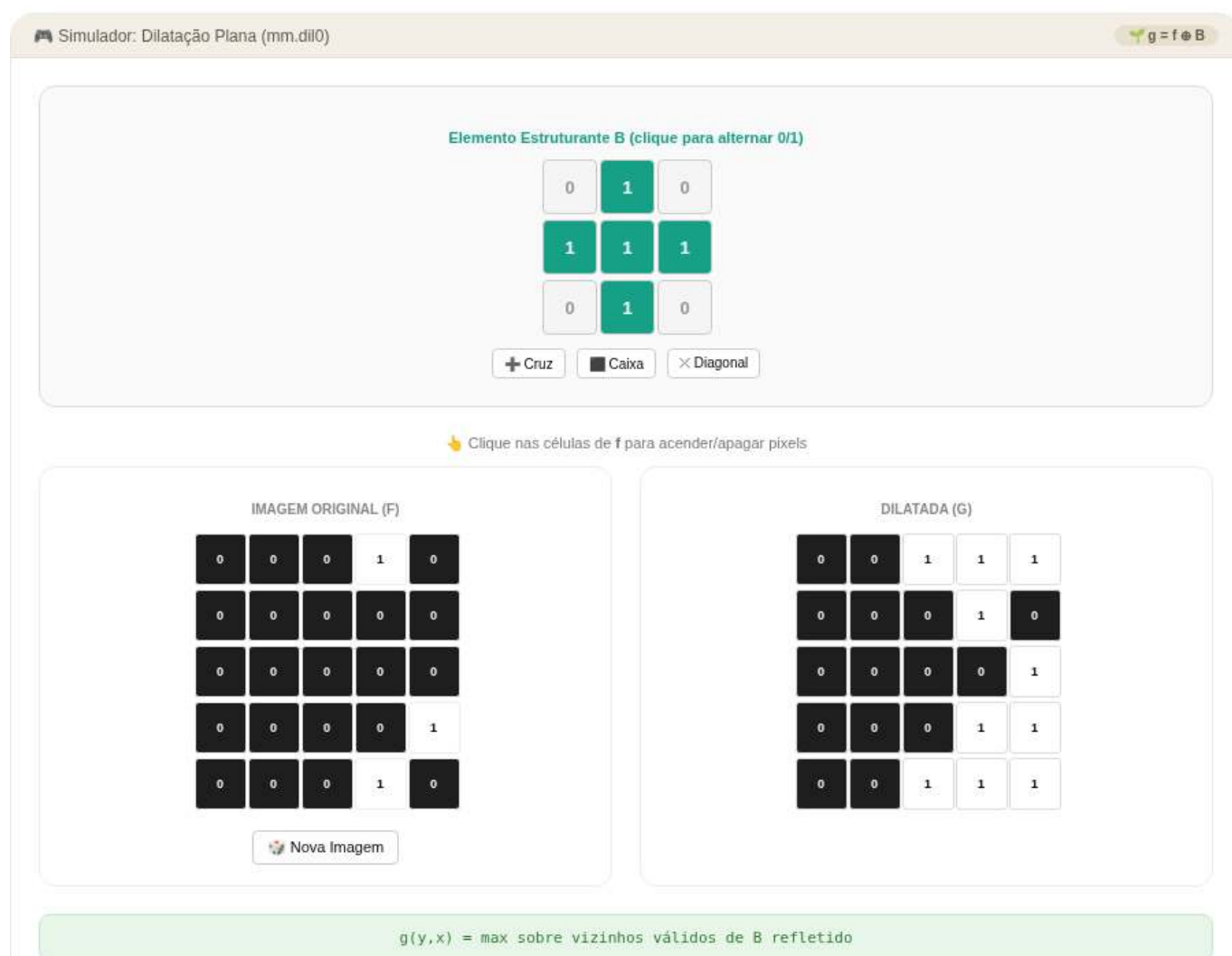


Figura 4.32: Simulador: Dilatação Binária Plana (mm.dil0)

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$. 6. **Mapeamento:** Calcular cada pixel de saída como o **mínimo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B vale 1:

$$g(y, x) = \min \left(f(y, x), \min_{\substack{(v_y, v_x) \text{ válido} \\ B(by, bx)=1}} f(v_y, v_x) \right)$$

7. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.4.2 Restrições Computacionais

- **Sem reflexão:** Diferente da dilatação, B é usado **exatamente como lido** — refletir aqui seria um erro conceitual grave.
- **Sem padding:** Vizinhos fora da imagem são simplesmente ignorados, nunca tratados como 0.
- **Robustez de borda:** Se nenhuma posição válida de $B = 1$ cair dentro do domínio, o pixel mantém seu valor original.

4.9.4.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Erosão	$g \leq f$ sempre (anti-extensiva)	Regiões claras encolhem, ruído pontual desaparece
Dualidade	$\text{ero}(f, B) = -\text{dil}(-f, B_{ref})$	Erosão e dilatação são “espelhos” matemáticos
B maior	Erosão mais agressiva	Objetos finos somem completamente

4.9.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.4.5 Exemplos

Entrada	Saída	Observação
3	9 0 9	O “buraco” central (0) se propaga em cruz
3	0 0 0	
3	9 0 9	
3		
0 1 0		
1 1 1		
0 1 0		
9 9 9		
9 0 9		
9 9 9		

Entrada	Saída	Observação
1	10 5 5 80	B horizontal: cada pixel “puxa” o mínimo dos vizinhos da linha
4		
1		
3		
1 1 1		
10 200 5 80		

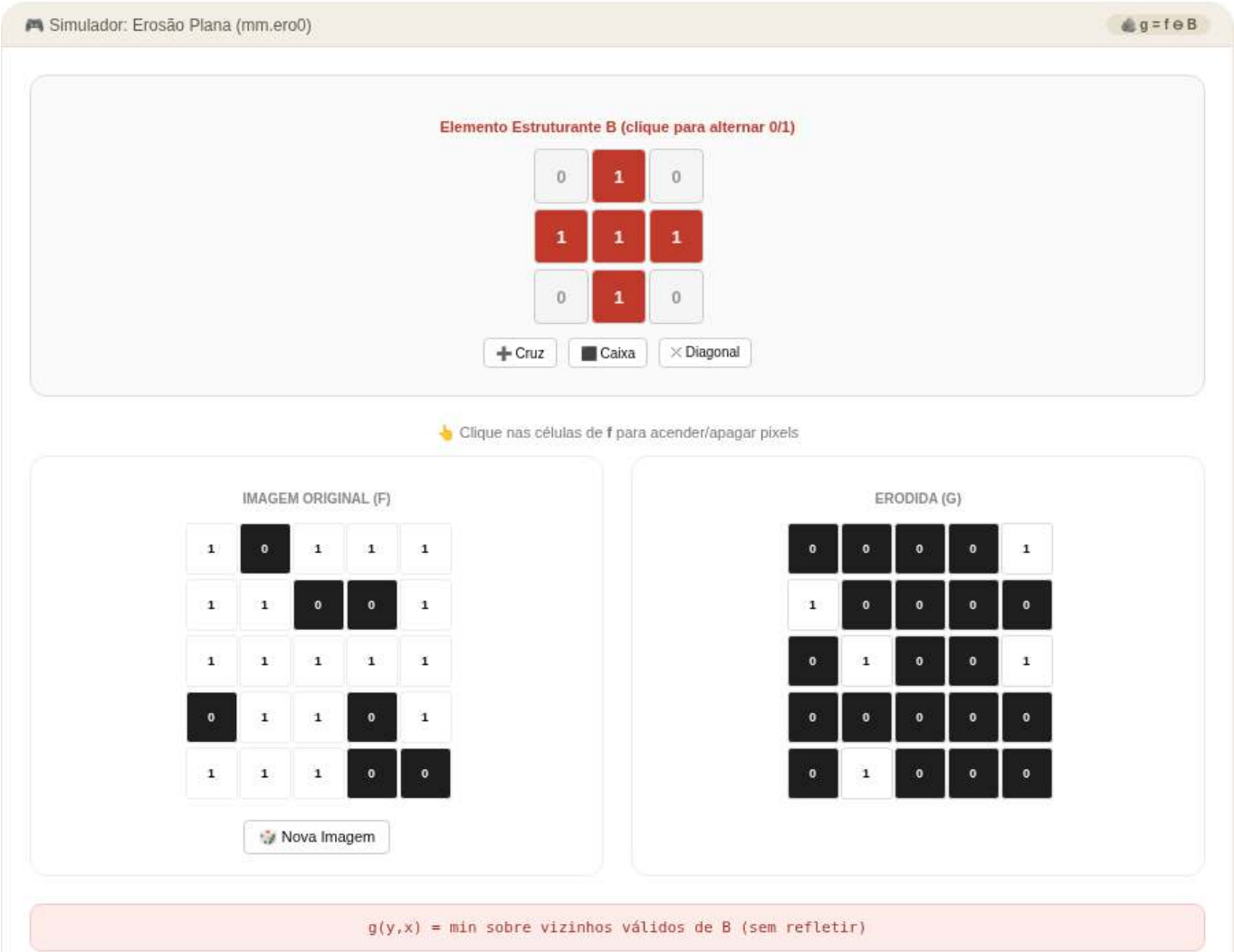


Figura 4.33: Simulador: Erosão Binária Plana (mm.ero0)

```
%%writefile EP04_04.py
# Código Python

Writing EP04_04.py

TestSuite("EP04_04.py").run()
```

4.9.5 EP04_05 Abertura Morfológica (Remoção de Ruído)

Imagens capturadas por **sensores de baixo custo**, como os de drones agrícolas, costumam vir salpicadas de pequenos pontos de ruído — pixels isolados que não representam nada real. Aplicar erosão seguida de

dilatação com o **mesmo** elemento estruturante produz a **abertura**: ela “limpa” pontos e protuberâncias finas, mas devolve ao objeto principal praticamente seu tamanho original. É a combinação clássica usada em **pré-processamento de imagens de satélite** antes de qualquer contagem de área plantada. Ver na Figure 4.34 uma simulação deste EP.

4.9.5.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Erosão:** Calcular $e = f \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding).
6. **Dilatação:** Calcular $g = e \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding) — mas agora aplicado sobre e , não sobre f .
7. **Saída:** Exibir a matriz resultante g (a **abertura** de f por B) com dimensões $L \times C$.

4.9.5.2 Restrições Computacionais

- **Ordem fixa:** É sempre erosão primeiro, depois dilatação — a ordem inversa define outro operador (fechamento, do próximo EP).
- **Mesmo B :** O elemento estruturante usado na erosão e na dilatação deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.5.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Anti-extensividade	$g \subseteq f$ sempre	A abertura nunca cria pixel novo, só remove
Idempotência	$\text{abertura}(\text{abertura}(f)) = \text{abertura}(f)$	Aplicar de novo não muda mais nada
Pontos isolados	Menores que B	São completamente eliminados
Núcleo do objeto	Maior que B	É recuperado quase intacto pela dilatação final

4.9.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.5.5 Exemplos

Entrada	Saída	Observação
7	0 0 0 0 0 0	Pontos isolados e a protuberância fina desaparecem; o quadrado central sobrevive
7	0 0 0 0 0 0	
3	0 0 1 1 1 0 0	
3	0 0 1 1 1 0 0	
1 1 1	0 0 1 1 1 0 0	
1 1 1	0 0 0 0 0 0 0	
1 1 1	0 0 0 0 0 0 0	
0 0 0 0 0 0 0		
0 1 0 0 0 1 0		
0 0 1 1 1 0 0		
0 0 1 1 1 0 0		
0 0 1 1 1 1 0		
0 0 0 0 0 0 0		
0 1 0 0 0 0 1		

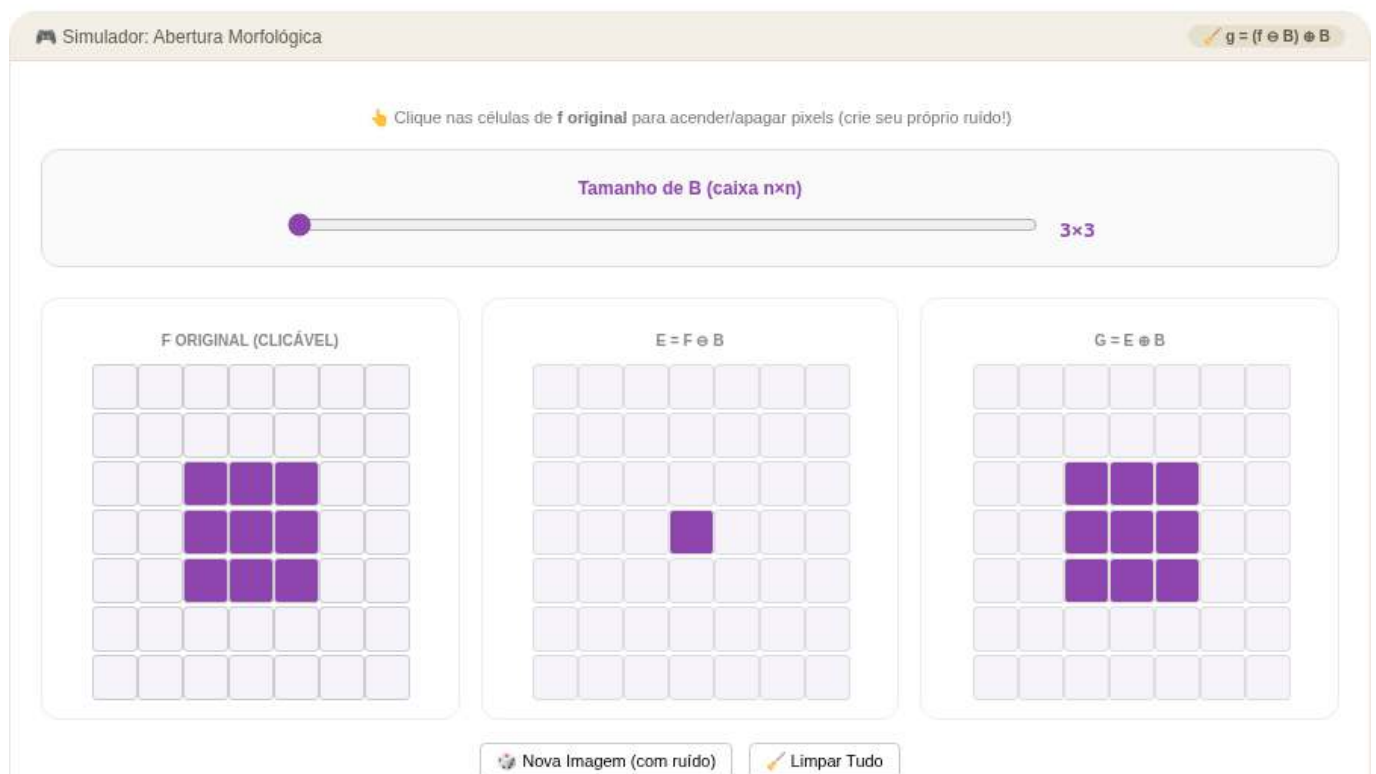


Figura 4.34: Simulador: Abertura Morfológica (erosão + dilatação)

```
%%writefile EP04_05.py
# Código Python
```

Writing EP04_05.py

```
TestSuite("EP04_05.py").run()
```

4.9.6 EP04_06 ✖ Fechamento Morfológico (Preenchimento de Falhas)

Em **digitalização de impressões digitais**, sulcos da pele às vezes ficam interrompidos por sujeira ou ressecamento, criando pequenas falhas na curva contínua que deveria existir. O **fechamento** — dilatação

seguida de erosão com o mesmo elemento estruturante — é o operador dual da abertura: ele **preenche buracos pequenos e reentrâncias estreitas**, sem alterar significativamente o contorno externo do objeto. É a etapa padrão antes de extrair o esqueleto de uma impressão digital. Ver na Figure 4.35 uma simulação deste EP.

4.9.6.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Dilatação:** Calcular $d = f \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding).
6. **Erosão:** Calcular $g = d \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding) — agora aplicado sobre d , não sobre f .
7. **Saída:** Exibir a matriz resultante g (o **fechamento** de f por B) com dimensões $L \times C$.

4.9.6.2 Restrições Computacionais

- **Ordem fixa:** É sempre dilatação primeiro, depois erosão — a ordem inversa é a abertura do EP04_05.
- **Mesmo B :** O elemento estruturante usado na dilatação e na erosão deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.6.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Extensividade	$g \supseteq f$ sempre	O fechamento nunca remove pixel, só adiciona
Idempotência	$\text{fecha}(\text{fecha}(f)) = \text{fecha}(f)$	Aplicar de novo não muda mais nada
Buracos pequenos	Menores que $\overline{B}$	São completamente preenchidos
Dualidade	$\text{fecha}(f) = \overline{\text{abre}(\overline{f})}$	É a abertura aplicada ao “negativo” da imagem

4.9.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.6.5 Exemplos

Entrada	Saída	Observação
8	0 0 1 1 1 1 0 0	Os dois buracos internos não-adjacentes são totalmente preenchidos
8	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
0 0 0 0 0 0 0 0	0 0 1 1 1 1 0 0	
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 0 1 1 0 0		
0 0 1 1 0 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 0 0 0 0 0 0		

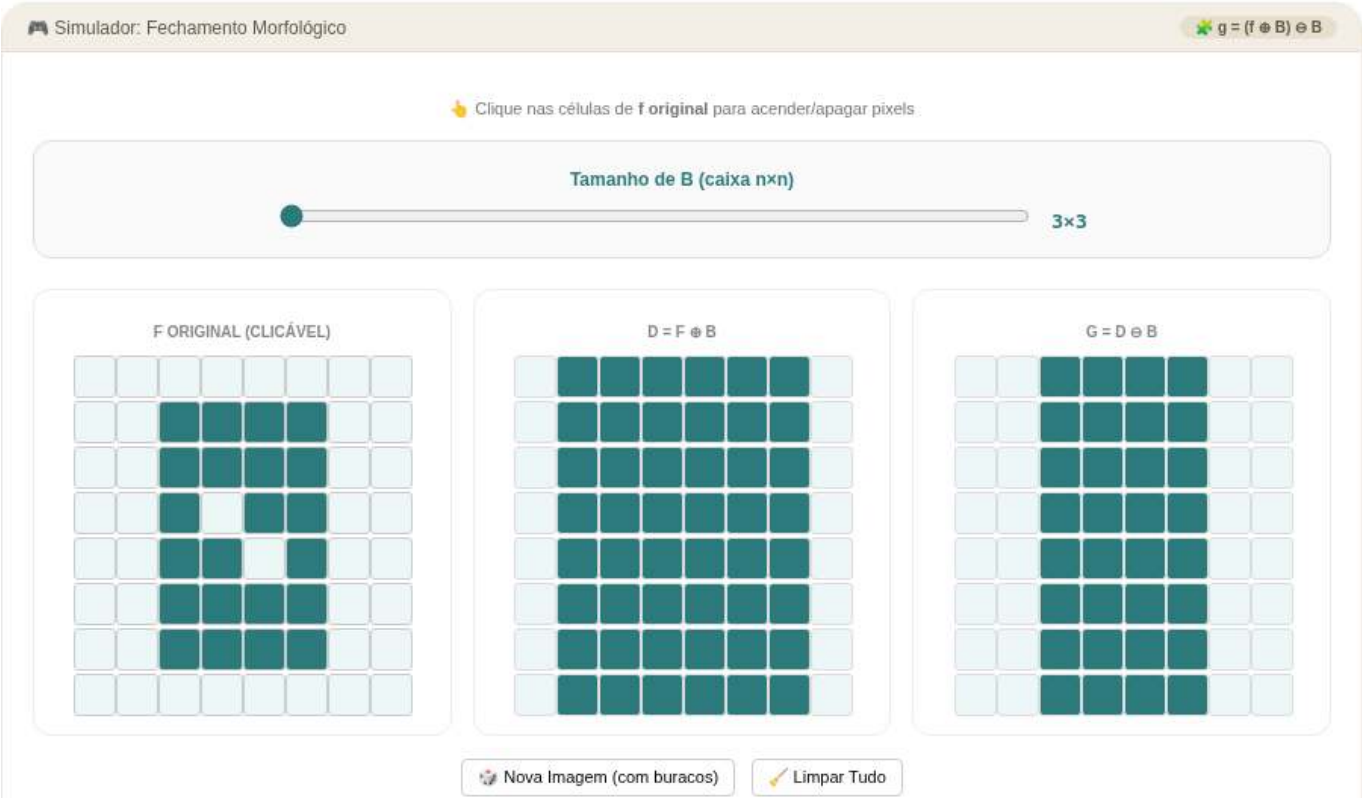


Figura 4.35: Simulador: Fechamento Morfológico (dilatação + erosão)

```
%writefile EP04_06.py
# Código Python

Writing EP04_06.py

TestSuite("EP04_06.py").run()
```

4.9.7 EP04_07 Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)

Até agora, o elemento estruturante só dizia “este vizinho conta” ou “não conta” — mas em **modelos digitais de elevação** (usados em SIG e em planejamento de drenagem urbana), cada vizinho deveria ter um **peso diferente** dependendo da distância ou da direção do relevo. As versões **ponderadas** da dilatação e da erosão, implementadas em `morph.py` como `mm.dil1(f, b)` e `mm.ero1(f, b)`, somam (ou subtraem) o peso de cada vizinho antes de tomar o máximo (ou mínimo) — generalizando tudo o que foi feito nos EPs anteriores. Ver na Figure 4.36 uma simulação deste EP.

4.9.7.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de b :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante ponderado.
3. **Pesos:** Ler a matriz b de pesos **inteiros** (podem ser negativos, zero ou positivos), linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer **todas** as posições (by, bx) de b (não apenas onde valeria 1 — aqui **todo** peso participa), usando o mesmo deslocamento dos EPs anteriores:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$.

6. **Dilatação ponderada:** Calcular

$$g_{dil}(y, x) = \max \left(f(y, x), \max_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) + b(by, bx)) \right)$$

7. **Erosão ponderada:** Calcular, **usando o mesmo b e sem refletir**:

$$g_{ero}(y, x) = \min \left(f(y, x), \min_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) - b(by, bx)) \right)$$

8. **Saída:** Exibir **primeiro** a matriz g_{dil} completa, e **depois** a matriz g_{ero} completa.

4.9.7.2 Restrições Computacionais

- **Nenhuma das duas reflete b** — a versão ponderada não usa reflexão, mesmo na dilatação (diferente de `mm.dil0`).
- **Todos os pesos participam:** Não existe aqui o filtro “ $B = 1$ ”; mesmo peso 0 entra na conta.
- **Sem padding:** vizinhos fora da imagem são ignorados, nunca virtualmente preenchidos.
- **Tipo:** A saída pode conter valores negativos ou maiores que 255 — **não** há *clipping* neste EP.
- **Dica:** Para remover mensagens de overflow ao ultrapassar limites do tipo `uint8`, incluir no início do código:

```
import warnings
warnings.filterwarnings("ignore")
```

4.9.7.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Peso positivo	“Puxa” o valor do vizinho para cima na dilatação	Simula relevo que sobe naquela direção
Peso negativo	Reduz a contribuição do vizinho	Simula distância ou atenuação direcional
Dualidade ponderada	$ero1(f, b) = -dil1(-f, b)$	A simetria entre as duas operações se mantém mesmo com pesos

4.9.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (podem ser negativos) da matriz b .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Primeiro a matriz g_{dil} em L linhas e C colunas.
- Em seguida a matriz g_{ero} em L linhas e C colunas.

4.9.7.5 Exemplos

Entrada	Saída	Observação
3	50 60 61	Peso central 2 acelera o crescimento na dilatação e o encolhimento na erosão
3	80 90 91	
3	81 91 92	
3	8 9 19	
0 1 0	9 10 20	
1 2 1	39 40 50	
0 1 0		
10 20 30		
40 50 60		
70 80 90		

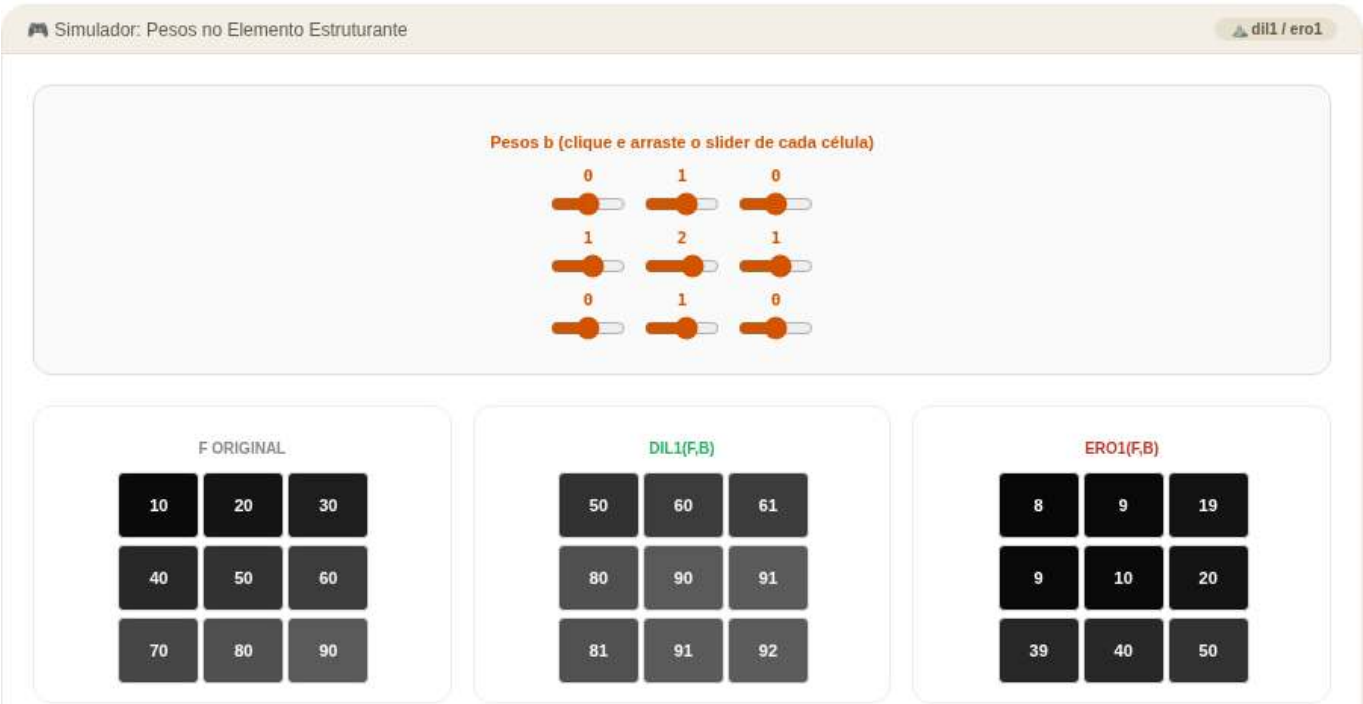


Figura 4.36: Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)


```
import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite
```

```
%%writefile EP04_07.py
# Código Python
```

```
Writing EP04_07.py
```

```
TestSuite("EP04_07.py").run()
```

4.9.8 EP04_08 Gradiente Morfológico, Top-hat e Black-hat

Em **inspeção automática de placas de circuito**, três perguntas aparecem o tempo todo: onde estão as **bordas** dos componentes? Quais **detalhes claros e pequenos** (como pontos de solda) se destacam do fundo? Quais **reentrâncias escuras** (como fissuras) o fundo esconde? Um único par erosão/dilatação responde às três: o **gradiente morfológico** evidencia contornos, o **top-hat** revela picos estreitos, e o **black-hat** revela vales estreitos — três ferramentas, uma só vizinhança. Ver na Figure 4.37 uma simulação deste EP.

4.9.8.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original, em tons de cinza), linha a linha.
5. **Operadores de base:** Calcular, exatamente como nos EPs 04_03 a 04_06:
 - $d = f \oplus B$ (dilatação),
 - $e = f \ominus B$ (erosão),
 - $\text{abertura} = e \oplus B$,
 - $\text{fechamento} = d \ominus B$.
6. **Gradiente morfológico:** $\text{grad}(y, x) = d(y, x) - e(y, x)$.
7. **Top-hat:** $\text{tophat}(y, x) = f(y, x) - \text{abertura}(y, x)$.
8. **Black-hat:** $\text{blackhat}(y, x) = \text{fechamento}(y, x) - f(y, x)$.
9. **Saída:** Exibir, **nesta ordem**, as três matrizes completas: gradiente, top-hat, black-hat.

4.9.8.2 Restrições Computacionais

- **Sem padding em nenhuma etapa intermediária** — dilatação, erosão, abertura e fechamento seguem as mesmas regras de vizinhança dos EPs anteriores.
- **Não há clipping:** as três saídas podem conter qualquer valor inteiro (o gradiente é sempre ≥ 0 , mas top-hat e black-hat também).
- **Reaproveitamento:** d e e devem ser calculados **uma única vez** e reaproveitados para montar abertura, fechamento e gradiente.

4.9.8.3 Fundamentação Teórica

Operador	Fórmula	O que revela
Gradiente	$d - e$	Bordas: zero em regiões planas, alto nas transições
Top-hat	$f - \text{abertura}(f)$	Elementos claros e finos , menores que B
Black-hat	$\text{fechamento}(f) - f$	Elementos escuros e finos , menores que B

4.9.8.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz gradiente em L linhas e C colunas.
- Matriz top-hat em L linhas e C colunas.
- Matriz black-hat em L linhas e C colunas.

4.9.8.5 Exemplos

Entrada	Saída	Observação
9	(gradiente: halo	Pico isolado vira top-hat; vale isolado vira black-hat; ambos aparecem no gradiente
9	$3 \times 3 = 70$ em torno de	
3	(2, 2) e halo $3 \times 3 = 8$ em	
3	torno de (6, 6), resto 0)	
1 1 1	(top-hat: único 70 em	
1 1 1	(2, 2), resto 0)	
1 1 1	(black-hat: único 8 em	
10 10 10 10 10 10 10 10 10	(6, 6), resto 0)	
10 10 10 10 10 10 10 10 10		
10 10 80 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 2 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		

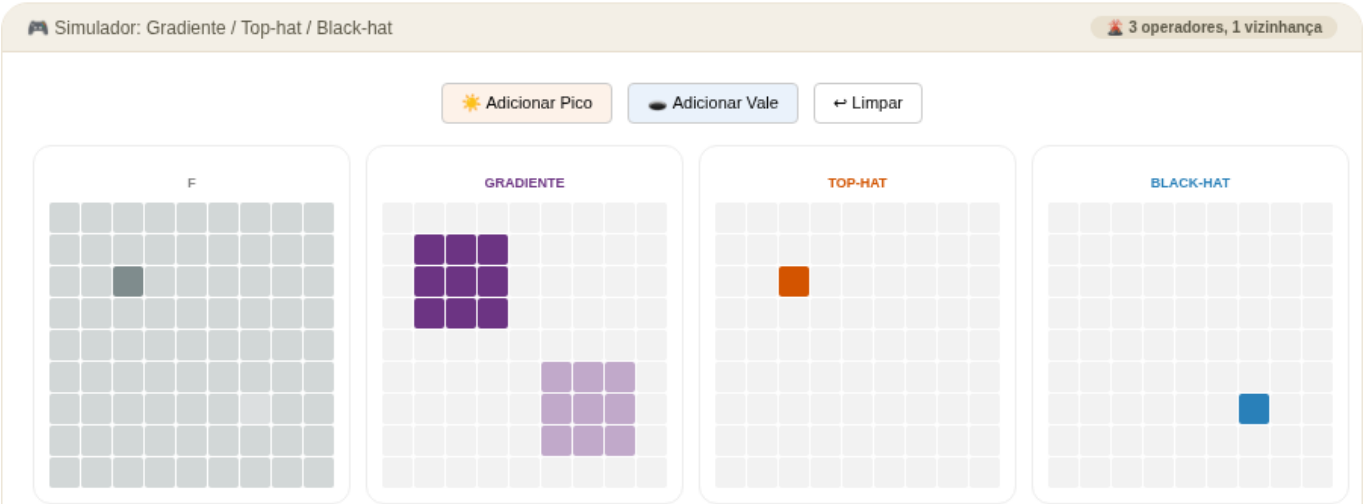


Figura 4.37: Simulador: Gradiente Morfológico, Top-hat e Black-hat

```
%%writefile EP04_08.py
# Código Python
```

```
Writing EP04_08.py
```

```
TestSuite("EP04_08.py").run()
```

4.9.9 EP04_09 Transformada de Distância e o “Miolo” do Objeto

Em **robótica móvel**, ao planejar uma rota dentro de um corredor, o robô quer saber não apenas *onde* existe espaço livre, mas também **quão longe** cada ponto livre está da parede mais próxima. Os caminhos mais seguros tendem a passar pelo “miolo” do corredor, longe dos obstáculos.

A **transformada de distância morfológica** atribui a cada pixel um valor que representa sua distância até a borda mais próxima, segundo a métrica definida pelo elemento estruturante. Pixels próximos à borda recebem valores baixos, enquanto pixels mais internos recebem valores maiores. O pixel de valor máximo corresponde à região mais protegida do objeto, frequentemente associada ao seu centro morfológico.

Ver na Figure 4.38 uma simulação deste EP.

4.9.9.1 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) da imagem f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz b , contendo valor 0 no centro e valores negativos nas demais posições.
4. **Imagem:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Preparação:** multiplicar a imagem por $L \times C$, garantindo que os pixels internos tenham valor inicial suficientemente alto para a propagação das distâncias.
6. **Transformada de distância:** calcular a matriz de distâncias utilizando o método `mm.dist1(f,b)`.
7. **Saída:** exibir a matriz resultante da transformada de distância.

4.9.9.2 Restrições Computacionais

- Utilizar a implementação de erosão ponderada fornecida pela biblioteca.
- O elemento estruturante pode conter valores negativos arbitrários.
- A transformada deve ser obtida pela aplicação iterativa de erosões ponderadas até atingir um ponto fixo.

 **Nota Crucial sobre Leitura de Matrizes:** Como o elemento estruturante pode conter valores inteiros negativos (por exemplo, -1 e -99), **não utilize a função `mm.readImg` para ler a matriz b** . Essa função converte os dados para o tipo `uint8`, provocando *underflow* e corrompendo os valores negativos. Leia as L_B linhas de b manualmente utilizando o tipo padrão `int`. A imagem f pode continuar sendo lida normalmente por `mm.readImg`.

4.9.9.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
$\text{dist}(y, x)$	Distância morfológica até a borda mais próxima segundo a métrica definida por b	Pixels mais internos recebem valores maiores
Valor máximo	Pixel mais distante da borda	Aproxima o centro morfológico do objeto
Elemento estruturante ponderado	Define os custos de deslocamento entre pixels vizinhos	Determina a métrica de distância utilizada

Conceito	Significado	Impacto Visual
Objetos finos	Regiões estreitas do objeto	Produzem valores baixos de distância

4.9.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L .
- Linha 2: inteiro C .
- Linha 3: inteiro L_B .
- Linha 4: inteiro C_B .
- Próximas L_B linhas: elementos inteiros da matriz b .
- Próximas L linhas: elementos binários (0 ou 1) da matriz f .

 **Nota de implementação:** Os elementos da matriz f (0 ou 1) devem ser multiplicados por **255** para gerar uma imagem binária adequada (0 e 255) antes de aplicar a Transformada de Distância (TD).

Saída:

- Matriz da transformada de distância em L linhas e C colunas.

4.9.9.5 Exemplo

Entrada	Saída	Observação
5	0 0 0 0 0 0 0 0	Resultado da transformada de distância.
9	0 1 1 1 1 1 1 0	
3	0 1 2 2 2 2 2 0	
3	0 1 1 1 1 1 1 0	
-99 -1 -99	0 0 0 0 0 0 0 0	
-1 0 -1		
-99 -1 -99		
0 0 0 0 0 0 0 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 0 0 0 0 0 0 0		

Nota: o valor -99 atua como uma aproximação prática de $-\infty$, impedindo a propagação pelas diagonais. Dessa forma, apenas os vizinhos horizontal e vertical contribuem para a distância, produzindo a distância de Manhattan.

```
%%writefile EP04_09.py
# Código Python
```

```
Writing EP04_09.py
```

```
TestSuite("EP04_09.py").run()
```

4.9.10 EP04_10 Separação de *Blobs*, Rotulação e Descritores

Em uma **linha de produção de moedas**, é comum que peças encostem umas nas outras na esteira, formando uma única mancha conectada na imagem — uma contagem ingênua erraria o total. A solução

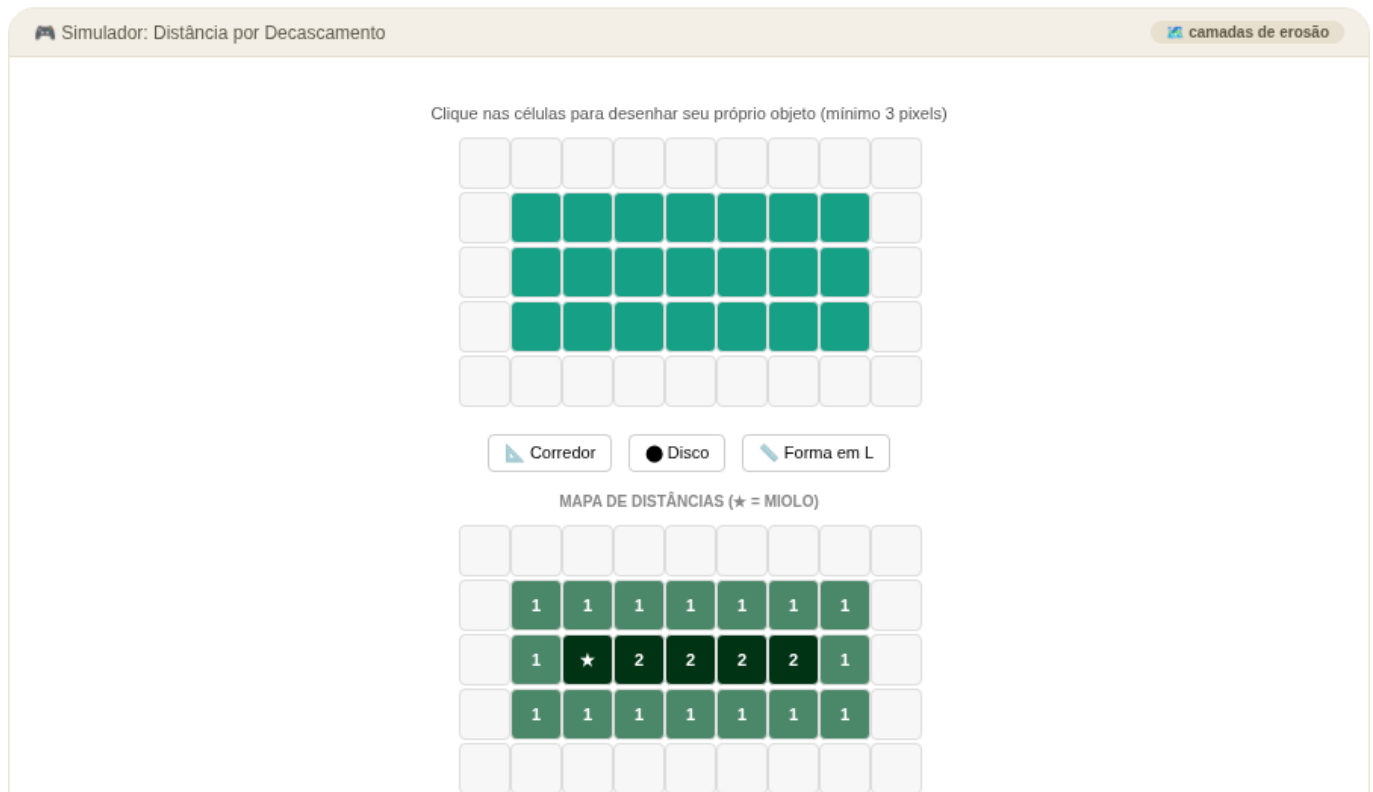


Figura 4.38: Simulador: Transformada de Distância

clássica combina operações morfológicas e análise de conectividade: primeiro uma **erosão** reduz ou rompe conexões frágeis entre objetos, e depois a **rotulação de componentes conectados** separa cada objeto em uma região distinta. Por fim, **descritores geométricos** (área e caixa delimitadora) resumem cada componente encontrado.

Ver na Figure 4.39 uma simulação deste EP.

4.9.10.1 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz B , contendo valores 0 ou 1, linha a linha.
4. **Dados:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Separação:** calcular

$$f_{ero} = f \ominus B$$

usando erosão binária plana (como no EP04_04), eliminando conexões frágeis entre objetos.

6. **Rotulação:** sobre f_{ero} , identificar componentes conectados usando conectividade definida pela vizinhança B . A rotulação deve seguir varredura *raster*: ao encontrar um pixel 1 ainda não rotulado, atribuir um novo rótulo inteiro crescente a partir de 1 e propagar esse rótulo a toda a região conectada.
7. **Descritores:** para cada rótulo k , calcular:
 - **Área:** número de pixels pertencentes ao rótulo;
 - **Caixa delimitadora:**

$$(y_{min}, x_{min}, y_{max}, x_{max})$$

8. **Saída:** exibir o número total de rótulos e, em seguida, uma linha por rótulo no formato:

$$k, \text{ área}, y_{\min}, x_{\min}, y_{\max}, x_{\max}$$

4.9.10.2 📌 Restrições Computacionais

- A erosão deve ser aplicada antes da rotulação.
- A conectividade é fixa e definida pela vizinhança acima.
- O elemento estruturante B não interfere na conectividade da rotulação.
- Sem padding em qualquer etapa.
- A ordem dos rótulos segue a primeira descoberta em varredura *raster*.

4.9.10.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto
Ponte fina	Conexão estreita entre objetos	Pode ser removida pela erosão morfológica
Conectividade	Definida pelo conjunto $\mathcal{N}(y, x)$	Determina quais pixels pertencem ao mesmo componente
Área	Número de pixels por componente	Estimativa direta do tamanho do objeto
Caixa delimitadora	Extensão espacial do rótulo	Resumo geométrico do componente

4.9.10.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L
- Linha 2: inteiro C
- Linha 3: inteiro L_B
- Linha 4: inteiro C_B
- Próximas L_B linhas: matriz B
- Próximas L linhas: matriz f

Saída:

- Linha 1: número total de rótulos encontrados
- Linhas seguintes:

$$k, \text{ área}, y_{\min}, x_{\min}, y_{\max}, x_{\max}$$

```
%%writefile EP04_10.py
# Código Python
```

```
Writing EP04_10.py
```

```
TestSuite("EP04_10.py").run()
```



Figura 4.39: Simulador: Separação de Blobs, Rotulação e Descritores

Capítulo 5

Transformadas e Compressão



Em construção!

Nos capítulos anteriores, todas as operações foram realizadas **no domínio espacial**, em que os algoritmos atuam diretamente sobre os valores de intensidade dos pixels.

Neste capítulo será apresentada uma abordagem complementar: o **domínio da frequência**, no qual a imagem é representada pelas variações espaciais de intensidade, e não apenas pelos valores individuais dos pixels.

O conceito de **frequência espacial** descreve a rapidez com que a intensidade varia ao longo da imagem. Variações lentas correspondem a **baixas frequências**, enquanto bordas, detalhes finos e ruídos correspondem a **altas frequências**.

Essa representação baseia-se no fato de que qualquer imagem digital discreta pode ser decomposta em uma combinação de **funções ortogonais**. A **Transformada de Fourier** utiliza uma **base de exponenciais complexas bidimensionais** (equivalentes a senoides com orientação e frequência específicas). Outras transformadas, como a **Transformada de Cossenos (DCT)** e a **Transformada Wavelet (DWT)**, utilizam diferentes famílias de funções de base — cossenos bidimensionais no caso da DCT, e funções com suporte compacto no caso das *wavelets*.

Entre as principais aplicações dessa representação destacam-se:

1. **Filtragem no domínio da frequência**, para atenuar ou realçar determinadas faixas de frequência;
2. **Análise multirresolução por meio de transformadas wavelet**, que representa estruturas em diferentes escalas;
3. **Compressão de imagens**, pela redução do número de coeficientes necessários para representar a imagem.

5.1 Objetivos

Ao concluir este capítulo, você será capaz de:

- **Interpretar o espectro de Fourier** de uma imagem, distinguindo magnitude, fase e componentes de frequência;
- **Aplicar o Teorema da Convolução** para realizar filtragem no domínio da frequência utilizando a Transformada Rápida de Fourier (FFT);
- **Projetar e analisar filtros no domínio da frequência**, compreendendo o funcionamento de filtros passa-baixa, passa-alta e *notch*;
- **Compreender a análise multirresolução por transformadas wavelet** e sua aplicação na representação hierárquica de imagens;
- **Descrever o processo de compressão de imagens**, incluindo a Transformada Discreta do Cosseno (DCT) e a quantização dos coeficientes;
- **Selecionar formatos de armazenamento de imagens**, como JPEG, PNG e WebP, de acordo com os requisitos da aplicação.

5.2 Configuração do Ambiente

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt
import cv2

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.2).

5.3 Transformada de Fourier Discreta 2D

A análise de Fourier baseia-se no princípio de que qualquer sinal periódico pode ser representado como uma soma de funções senoidais com diferentes frequências, amplitudes e fases. Esse conceito também se aplica às imagens digitais, permitindo representá-las no **domínio da frequência** em vez do domínio espacial.

A Figure 5.1 ilustra essa decomposição para um sinal unidimensional. No caso de uma imagem, a Transformada Discreta de Fourier (DFT) converte a matriz de intensidades $f(x, y)$ em um conjunto de coeficientes que descreve a contribuição das diferentes frequências espaciais presentes na imagem.

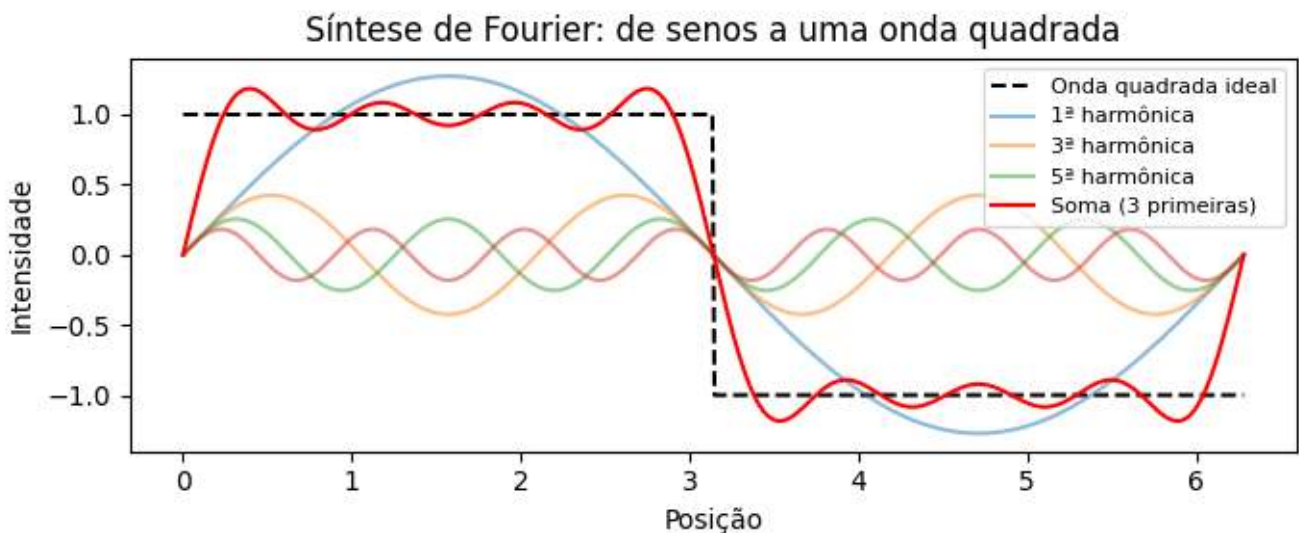


Figura 5.1: Decomposição de Fourier 1D: uma onda quadrada (linha tracejada) é aproximada pela soma das primeiras senoides (linhas coloridas). Quanto mais termos, melhor a aproximação.

5.3.1 Simulador: Reconstruindo Sinais com Senoides

Antes de estudar imagens bidimensionais, o simulador da Figure 5.2 ilustra o princípio da análise de Fourier para sinais unidimensionais: **uma forma de onda pode ser aproximada pela soma de senoides com diferentes frequências e amplitudes.**

À medida que novos termos são adicionados, a soma das senoides (curva preta) aproxima-se da forma de onda de referência (tracejada). O gráfico inferior apresenta o espectro de amplitudes, indicando a contribuição de cada frequência para a reconstrução do sinal.

💡 Atividade

Explore o simulador e responda:

1. Quantos termos são necessários para obter uma boa aproximação da onda quadrada?
2. Qual das três formas de onda converge mais rapidamente? Justifique sua resposta.
3. Como o espectro de amplitudes se altera ao trocar a onda quadrada pela triangular?

i Respostas

1. Quantos termos são necessários para uma boa aproximação da onda quadrada?

Com aproximadamente 15 a 20 termos, a forma da onda já se aproxima bem da referência. Entretanto, próximo às descontinuidades permanece uma pequena oscilação, conhecida como **fenômeno de Gibbs**, que não desaparece mesmo com a adição de mais termos.

2. Qual forma converge mais rapidamente? Por quê?

A **onda triangular** converge mais rapidamente, pois as amplitudes de seus harmônicos decaem mais rápido que as da onda quadrada e da onda dente-de-serra. Como consequência, poucos termos já produzem uma boa aproximação.

3. Como o espectro muda entre a onda quadrada e a triangular?

Ambas possuem apenas **harmônicos ímpares**, mas, na onda triangular, as amplitudes diminuem muito mais rapidamente. Assim, poucos harmônicos são suficientes para reconstruir o sinal com boa precisão.



Figura 5.2: Simulador interativo da decomposição de Fourier 1D: visualização da soma de senoides com diferentes frequências, amplitudes e fases. Adicione termos e observe a convergência para formas de onda arbitrárias.

5.3.2 Interpretação do espectro de frequência

Ao aplicar a Transformada Discreta de Fourier (DFT) a uma imagem e visualizar o módulo de seus coeficientes (ver Figure 5.5), obtém-se o **espectro de magnitude**, que mostra a distribuição das frequências espaciais presentes na imagem.

O coeficiente localizado na origem da DFT, denominado **componente DC** (*Direct Current*), corresponde à frequência nula e representa a intensidade média da imagem. Por convenção, esse coeficiente é armazenado no canto superior esquerdo do espectro. Para facilitar sua interpretação, aplica-se a operação **FFT Shift**, que desloca a componente DC para o centro da imagem. Após esse deslocamento, as baixas frequências concentram-se na região central, enquanto as altas frequências ficam próximas às bordas, como resume a Table 5.1.

Tabela 5.1: Correspondência entre as regiões do espectro de magnitude após a aplicação do FFT Shift.

Região do espectro	Componentes predominantes	Exemplos na imagem
Centro (baixas frequências)	Variações espaciais lentas	Iluminação, regiões homogêneas e formas globais
Região intermediária (médias frequências)	Variações de escala intermediária	Texturas e padrões repetitivos
Bordas (altas frequências)	Variações espaciais rápidas	Contornos, detalhes finos e ruído

Essa organização facilita a interpretação do espectro e o projeto de filtros. A atenuação das baixas frequências reduz as variações globais de intensidade, enquanto a atenuação das altas frequências suaviza a imagem ao reduzir detalhes finos e parte do ruído.

5.3.3 O Experimento da Grade: Construindo uma Imagem a partir de um Único Coeficiente

Antes de apresentar a formulação matemática da Transformada Discreta de Fourier (DFT), é útil analisar sua inversa, denominada Transformada Discreta Inversa de Fourier (IDFT). Considere um espectro em que todos os coeficientes sejam nulos, exceto um. Um exemplo dessa construção é apresentado no código da Figure 5.3 e pode ser explorado interativamente no simulador da Figure 5.4.

A imagem reconstruída é uma senoide bidimensional. A posição do coeficiente no espectro determina sua **orientação** e sua **frequência espacial**, enquanto sua magnitude e sua fase definem, respectivamente, sua amplitude e seu deslocamento espacial. Assim, cada coeficiente da DFT representa uma componente senoidal, e a imagem original pode ser reconstruída pela soma de todas essas componentes.

```
N_grid = 100
espectro_vazio = np.zeros((N_grid, N_grid), dtype=complex)

# Acendendo um único ponto (frequência) fora do centro
u0, v0 = 10, 5
espectro_vazio[N_grid//2 - v0, N_grid//2 - u0] = 1000

# Retornando para o domínio espacial (IDFT)
onda_2d = np.real(np.fft.ifft2(np.fft.ifftshift(espectro_vazio)))

onda_vis = cv2.normalize(onda_2d, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
espectro_vis = cv2.normalize(
    np.abs(espectro_vazio), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Destaque visual do ponto
espectro_color = cv2.cvtColor(espectro_vis, cv2.COLOR_GRAY2BGR)
cv2.circle(espectro_color, (N_grid//2 - u0, N_grid//2 - v0), 2, (0, 0, 255), -1)
```

```
mm.show([espectro_color, onda_vis],
        titles=["Espectro (1 ponto ativo)", "Onda 2D Resultante (IDFT)"],
        cols=2, figsize=(10, 4))
```

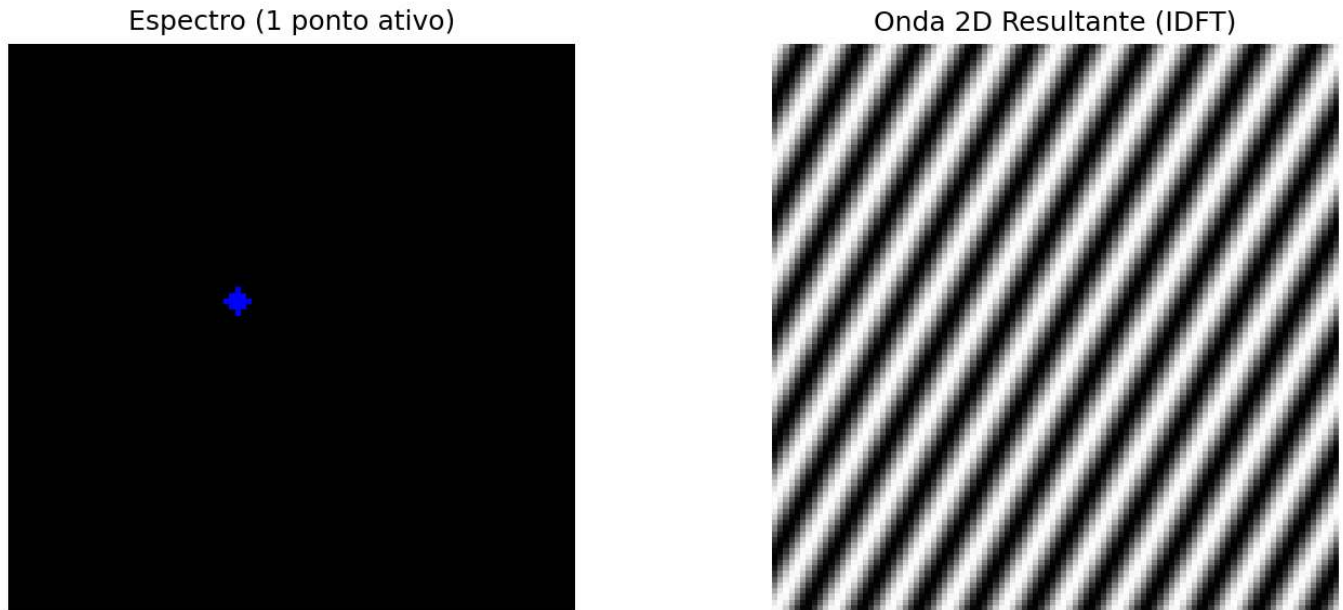


Figura 5.3: Toda frequência no espectro (ponto isolado) corresponde a uma onda senoidal 2D rotacionada no domínio espacial.

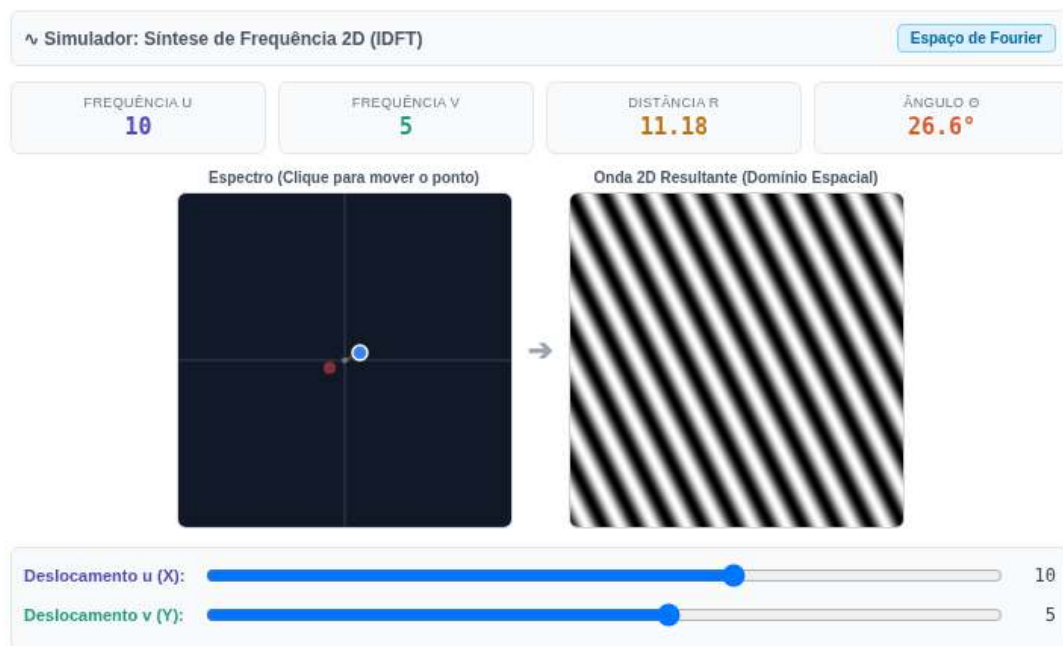


Figura 5.4: Simulador interativo da síntese de Fourier 2D. Altere a posição horizontal (u) e vertical (v) do coeficiente no espectro de frequências centrado e observe como a distância em relação ao centro dita a frequência espacial (espessura) e o ângulo dita a orientação da onda senoidal gerada.

5.3.4 Definição Matemática

Considere uma imagem $f(x, y)$ com dimensões $M \times N$. Sua **Transformada Discreta de Fourier 2D** (DFT) é definida por:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (5.1)$$

em que $u = 0, 1, \dots, M-1$ e $v = 0, 1, \dots, N-1$ representam as frequências discretas nas direções horizontal e vertical, respectivamente. O termo exponencial corresponde a uma senoide bidimensional, cuja frequência e orientação são determinadas pelos índices (u, v) .

A **Transformada Discreta Inversa de Fourier 2D** (IDFT) reconstrói a imagem original a partir de seus coeficientes:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (5.2)$$

As Equações Equation 5.1 e Equation 5.2 mostram que a DFT e a IDFT formam um par de transformações: a primeira converte a imagem para o domínio da frequência, enquanto a segunda reconstrói exatamente a imagem original a partir de seus coeficientes.

i Sobre o símbolo j

O termo j denota a **unidade imaginária**, definida por $j^2 = -1$. Em engenharia e processamento de sinais, adota-se j em vez de i para evitar conflito com a notação de corrente elétrica. Sua utilização na exponencial complexa, regida pela fórmula de Euler ($e^{j\theta} = \cos \theta + j \sin \theta$), permite representar de forma compacta a amplitude e a fase de cada frequência espacial presente na imagem.

i O que é o componente DC?

O coeficiente $F(0, 0)$, denominado **componente DC** (*Direct Current*), é igual à soma das intensidades de todos os pixels da imagem (ver Figure 5.5):

$$F(0, 0) = MN \bar{f},$$

em que $\bar{f}$ é a intensidade média da imagem. Por isso, o componente DC representa o nível médio de intensidade e, na maioria das imagens naturais, possui a maior magnitude do espectro.

Os demais coeficientes representam variações em torno dessa média. Após a aplicação do **FFT Shift**, o componente DC é deslocado para o centro do espectro, concentrando as baixas frequências na região central e as altas frequências nas bordas.

5.3.5 Magnitude e Fase

Cada coeficiente da Transformada Discreta de Fourier (DFT) é um número complexo e pode ser escrito como

$$F(u, v) = R(u, v) + j I(u, v),$$

em que $R(u, v)$ e $I(u, v)$ correspondem, respectivamente, às partes **real** e **imaginária** do coeficiente. Da Equação Equation 5.1, obtêm-se

$$R(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)\right),$$

e



Figura 5.5: Diagrama conceitual do espectro de Fourier 2D centrado.

$$I(u, v) = - \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \sin\left(2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)\right).$$

A partir dessa representação, definem-se duas grandezas fundamentais:

- **Magnitude**, que indica a intensidade da componente de frequência,

$$|F(u, v)| = \sqrt{R(u, v)^2 + I(u, v)^2};$$

- **Fase**, que determina o alinhamento (ou deslocamento) espacial da componente,

$$\phi(u, v) = \text{atan2}(I(u, v), R(u, v)).$$

Assim, cada coeficiente também pode ser escrito em sua forma polar,

$$F(u, v) = |F(u, v)| e^{j\phi(u, v)}.$$

O espectro de Fourier pode, portanto, ser visualizado por meio de duas imagens distintas: o **espectro de magnitude**, normalmente utilizado para analisar a distribuição das frequências, e o **espectro de fase**, que descreve a organização espacial das componentes senoidais.

Embora o espectro de magnitude seja o mais utilizado para inspeção visual, a fase contém grande parte das informações estruturais da imagem. A combinação de magnitude e fase permite reconstruir exatamente a imagem original por meio da IDFT.

5.3.6 O que a Magnitude e a Fase carregam?

Uma demonstração clássica consiste em combinar a magnitude de uma imagem com a fase de outra e reconstruir o resultado. Esse experimento evidencia que:

- **A fase** preserva a estrutura espacial da imagem, incluindo a posição dos objetos, seus contornos e sua geometria. Pequenas alterações na fase podem provocar grandes mudanças visuais.
- **A magnitude** controla como a energia é distribuída entre as frequências espaciais, influenciando principalmente o contraste e a textura.

Quando uma imagem é reconstruída com a magnitude de A e a fase de B, o resultado tende a **assemelhar-se mais a B do que a A**, evidenciando que a fase é o principal componente responsável pela organização espacial da cena. Entretanto, a magnitude continua sendo importante, pois modula o contraste das estruturas reconstruídas. Assim, uma reconstrução fiel depende da combinação consistente entre magnitude e fase.

Um exemplo desse comportamento é apresentado na Figure 5.6.

i Analogia com Áudio: Limitações e Cuidados

A fase de um sinal desempenha papéis distintos em áudio e imagens:

- **Áudio estéreo ou multicanal:** a fase relativa entre os canais é fundamental para a percepção da posição das fontes sonoras, por meio das diferenças interaurais de tempo (ITD, *Interaural Time Differences*).
- **Áudio monaural:** a fase absoluta exerce pouca influência perceptual direta.
- **Imagens (DFT):** a fase é o principal fator responsável pela organização espacial da cena, enquanto a magnitude modula o contraste e a distribuição da energia entre as frequências.

Em ambos os domínios, a **magnitude** está relacionada à intensidade das componentes de frequência: em áudio, influencia o timbre e a intensidade percebida; em imagens, influencia o contraste e a textura.

```
# Experimento: A Importância da Fase
# Carregamento da imagem
url = "https://upload.wikimedia.org/wikipedia/commons/2/25/GAZI.MD.AHAD_11.jpg"
caminho = "imagens/coins.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_color = np.array(img_obj)
img_gray = mm.gray(img_color)

img_a = cv2.resize(img_gray, (400, 400))

# Criar uma imagem B sintética (padrão geométrico)
img_b = np.zeros((400, 400), dtype=np.uint8)
cv2.rectangle(img_b, (100, 100), (300, 300), 255, -1)
cv2.circle(img_b, (200, 200), 150, 128, 10)

FA = np.fft.fft2(img_a)
FB = np.fft.fft2(img_b)

# Troca de Fase
rec_A_mag_B_fase = np.real(np.fft.ifft2(np.abs(FA) * np.exp(1j * np.angle(FB))))
rec_B_mag_A_fase = np.real(np.fft.ifft2(np.abs(FB) * np.exp(1j * np.angle(FA))))

mm.show(
    [img_a, img_b, rec_A_mag_B_fase, rec_B_mag_A_fase],
    titles=["Imagem A", "Imagem B", "Mag(A) + Fase(B)", "Mag(B) + Fase(A)"],
    cols=4, figsize=(16, 4)
)
```

```
print(" A fase preserva bordas e contornos; a magnitude controla contraste e")
print("textura. Em áudio estéreo, a fase afeta a localização espacial; em")
print("imagens, determina a organização da cena.")
```

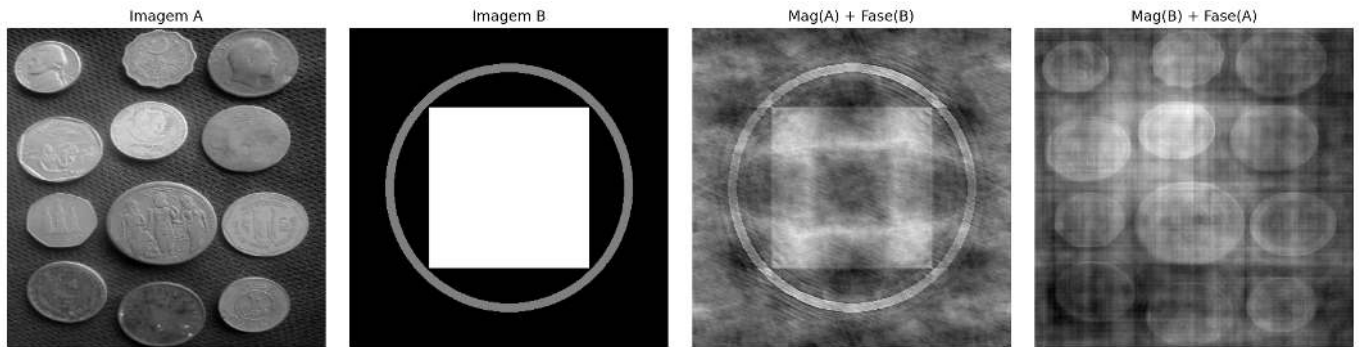


Figura 5.6: Experimento de troca de fase: Imagem A (moedas) e Imagem B (padrão geométrico) reconstruídas com magnitudes e fases trocadas. O resultado mostra que a estrutura visual é **muito mais sensível à fase** do que à magnitude: quando a fase de B é mantida, a imagem resultante preserva a organização espacial de B, mesmo com a magnitude de A. A magnitude, por sua vez, influencia principalmente o contraste e a textura. Observe que a qualidade da reconstrução não é perfeita — há artefatos visíveis —, evidenciando a interdependência entre fase e magnitude para uma representação fiel da imagem.

A fase preserva bordas e contornos; a magnitude controla contraste e textura. Em áudio estéreo, a fase afeta a localização espacial; em imagens, determina a organização da cena.

5.4 Teorema da Convolução e Estratégias de Filtragem

O **Teorema da Convolução** estabelece uma relação fundamental entre os domínios espacial e da frequência:

$$f(x, y) \otimes h(x, y) \xleftrightarrow{\mathcal{F}} F(u, v) H(u, v) \quad (5.3)$$

em que $\otimes$ representa a **convolução circular discreta**. Assim, a convolução entre uma imagem $f(x, y)$ e um filtro $h(x, y)$ pode ser substituída pela multiplicação de seus espectros.

Na prática, para obter o mesmo resultado da convolução linear realizada no domínio espacial, aplica-se **zero-padding** antes da Transformada Rápida de Fourier (FFT), evitando artefatos nas bordas da imagem.

Entretanto, nem sempre a filtragem no domínio da frequência é a alternativa mais eficiente. Para filtros como o Gaussiano e o filtro da média (*Box Filter*), a propriedade de **separabilidade** permite reduzir significativamente o custo computacional da convolução no domínio espacial.

5.4.1 Kernel Separável vs. Não Separável

Um **kernel separável** pode ser escrito como o produto externo de dois vetores unidimensionais,

$$H = v h^T,$$

permitindo que a convolução bidimensional seja substituída por duas convoluções unidimensionais consecutivas: uma na direção horizontal e outra na vertical.

Já um **kernel não separável** não admite essa decomposição e, portanto, sua convolução deve ser realizada diretamente sobre a vizinhança bidimensional.

Na prática, para um *kernel* de dimensão $K \times K$, a convolução direta exige K^2 multiplicações por pixel, enquanto um *kernel* separável requer apenas $2K$ multiplicações, reduzindo significativamente o custo computacional.

5.4.2 Análise de Eficiência Computacional

Considere uma imagem de dimensões $M \times N$ e um filtro quadrado de tamanho $K \times K$. A Table 5.2 compara a complexidade das principais estratégias de filtragem.

Tabela 5.2: Comparação da complexidade da convolução direta, separável e via Transformada Rápida de Fourier (FFT).

Método de Filtragem	Complexidade Assintótica	Dependência de K	Aplicação típica
Espacial não separável	$\mathcal{O}(MNK^2)$	Quadrática	<i>Kernels</i> pequenos e não separáveis
Espacial separável	$\mathcal{O}(MNK)$	Linear	Filtros Gaussiano e da média
Via FFT	$\mathcal{O}(MN \log(MN))$	Independente de K	<i>Kernels</i> grandes

Para *kernels* pequenos, a convolução espacial, especialmente quando o filtro é separável, costuma ser mais eficiente devido ao baixo custo das operações. À medida que o tamanho do *kernel* aumenta, a filtragem via FFT torna-se mais vantajosa, pois seu custo praticamente independe da dimensão do filtro.

5.4.3 Discussão dos resultados experimentais

O gráfico obtido no ensaio com a imagem das moedas (2560×1920), apresentado na Figure 5.7, confirma o comportamento previsto pela análise de complexidade computacional.

1. Convolução não separável ($\mathcal{O}(MNK^2)$)

A convolução direta apresenta crescimento quadrático com o tamanho do *kernel*. Para valores pequenos de K , o custo é baixo, mas aumenta rapidamente à medida que o *kernel* cresce, tornando-se inviável para aplicações em tempo real.

2. Filtragem via FFT ($\mathcal{O}(MN \log(MN))$)

O custo da FFT depende apenas do tamanho da imagem, sendo independente de K . Por isso, seu desempenho permanece aproximadamente constante ao variar o *kernel*, tornando-a vantajosa para filtros grandes ou não separáveis.

3. Convolução separável ($\mathcal{O}(MNK)$)

A decomposição do *kernel* em dois filtros unidimensionais reduz significativamente o custo computacional. Na prática, essa abordagem tende a ser a mais eficiente para filtros separáveis, especialmente em implementações otimizadas.

Em geral, a escolha do método depende do tamanho e da estrutura do *kernel*. Filtros separáveis são mais eficientes no domínio espacial, enquanto a FFT se torna mais vantajosa para *kernels* grandes ou múltiplas convoluções no domínio da frequência.

$$g = \mathcal{F}^{-1}[\mathcal{F}(f) \cdot \mathcal{F}(h)] \quad (\text{FFT}) \quad g = f \otimes h \quad (\text{convolução direta}) \quad g = (f \otimes v) \otimes h^T \quad (\text{separável}) \quad (5.4)$$

onde:

- $f(x, y)$ representa a imagem de entrada;
- $h(x, y)$ é o *kernel* bidimensional do filtro;

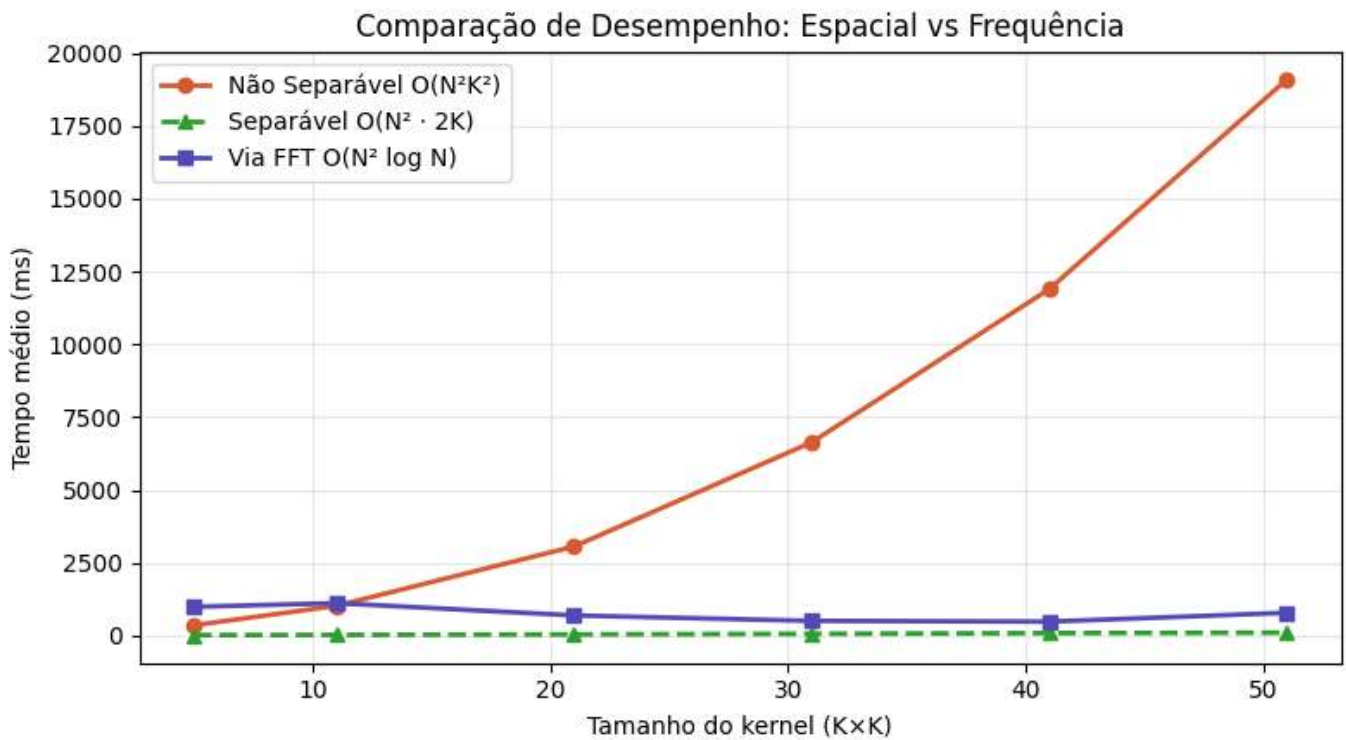


Figura 5.7: Comparação de eficiência: Convolução Não Separável (Espacial 2D), Separável (Espacial 1D) e via FFT.

- v e h^T são, respectivamente, os vetores vertical e horizontal que compõem o *kernel* separável.

! O problema da convolução circular (*wrap-around*)

A Transformada Discreta de Fourier (DFT) assume que a imagem é **periodicamente estendida no espaço**, isto é, que suas bordas se repetem indefinidamente.

Nessa condição, a multiplicação no domínio da frequência corresponde a uma **convolução circular** no domínio espacial. Como consequência, regiões opostas da imagem (topo e base, esquerda e direita) passam a interagir artificialmente, conforme ilustrado na Figure 5.8.

A aplicação de *zero-padding* antes da FFT reduz esse efeito ao estender a imagem com valores nulos nas bordas, aproximando o resultado da convolução linear. Esse comportamento pode ser interpretado à luz do Teorema da Convolução, apresentado na Figure 5.9.

```
# Definir M e N
M, N = img_gray.shape # linha adicionada

# Simulação de um filtro de deslocamento brutal
H_shift = np.zeros_like(img_gray, dtype=complex)
for u in range(M):
    for v in range(N):
        H_shift[u, v] = np.exp(-1j * 2 * np.pi * (u*120/M + v*120/N))

# Filtragem SEM padding (causa o wrap-around)
F_img = np.fft.fft2(img_gray)
img_vazada = np.real(np.fft.ifft2(F_img * H_shift))

img_vazada_vis = cv2.normalize(img_vazada, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
mm.show([img_gray, img_vazada_vis],
        titles=["Original", "Filtragem s/ Padding (Vazamento)"], cols=2, figsize=(10, 4))
```



Figura 5.8: Sem *padding*, um deslocamento severo faz a imagem vazar para o lado oposto (convolução circular).

```
# Kernel Gaussiano 11×11
sigma = 3.0
K = 11
ks = np.arange(K) - K // 2
gauss1d = np.exp(-ks**2 / (2 * sigma**2))
gauss1d /= gauss1d.sum()
kernel = np.outer(gauss1d, gauss1d) # kernel 2D separável

# Método 1: Convolução espacial direta
f_float = img_gray.astype(np.float64)
conv_esp = cv2.filter2D(f_float, -1, kernel, borderType=cv2.BORDER_CONSTANT)

# Método 2: Multiplicação em frequência (via FFT)
M, N = f_float.shape
# Padding para convolução linear (evita aliasing circular)
Mpad = 2 ** int(np.ceil(np.log2(M + K - 1)))
Npad = 2 ** int(np.ceil(np.log2(N + K - 1)))

# Posiciona o kernel com a origem no (0,0) e padding com zeros
kernel_pad = np.zeros((Mpad, Npad))
kh, kw = kernel.shape
kernel_pad[kh:kh+kw, :kw] = kernel

F_img = np.fft.fft2(f_float, (Mpad, Npad))
F_kern = np.fft.fft2(kernel_pad)
conv_freq = np.real(np.fft.ifft2(F_img * F_kern))

# Recorte para compensar o deslocamento introduzido pelo posicionamento do kernel
offset = K // 2
conv_freq_crop = conv_freq[offset:offset+M, offset:offset+N]

# Verificação numérica
```



```

diff = np.abs(conv_esp - conv_freq_crop)
print(f"Diferença máxima (|conv_esp - conv_freq|): {diff.max():.2e}")
print(f"Diferença média (|conv_esp - conv_freq|): {diff.mean():.2e}")
print(f"→ Teorema da Convolução verificado numericamente.")

conv_esp_vis = cv2.normalize(conv_esp, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
conv_freq_vis = cv2.normalize(conv_freq_crop, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
diff_vis = cv2.normalize(diff, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

mm.show(
    [img_gray, conv_esp_vis, conv_freq_vis, diff_vis],
    titles=[
        "Original",
        "Convolução espacial",
        "Multiplicação em frequência",
        f"Diferença (máx={diff.max():.1e})"
    ],
    cols=4, figsize=(16, 5)
)

```

Diferença máxima (|conv_esp - conv_freq|): 2.56e-13
 Diferença média (|conv_esp - conv_freq|): 2.76e-14
 → Teorema da Convolução verificado numericamente.



Figura 5.9: Verificação do Teorema da Convolução: a diferença pixel a pixel entre a convolução espacial (`cv2.filter2D`) e a multiplicação em frequência (FFT) é numericamente nula — confirmando a equivalência teórica.

i Sobre a diferença numérica

A diferença residual da ordem de 10^{-13} não viola o Teorema da Convolução, mas reflete **limitações computacionais** inerentes à aritmética de ponto flutuante (precisão dupla, $\sim 10^{-16}$) e à **ordem de operações** entre os dois métodos:

- **Convolução espacial:** soma ponderada de vizinhos com arredondamentos sucessivos.
- **Convolução em frequência:** envolve três transformadas FFT e uma multiplicação complexa, sujeita a erros de truncamento e quantização.

Portanto, a igualdade teórica é exata, mas a implementação numérica produz uma diferença praticamente nula (erro relativo $< 10^{-12}$), confirmando o teorema dentro da precisão da máquina.

5.5 Filtros no Domínio da Frequência

Um filtro no domínio da frequência pode ser interpretado como uma **função de transferência aplicada ao espectro da imagem**. Nessa representação, cada coeficiente de frequência é multiplicado por um valor entre 0 e 1, que determina sua atenuação ou preservação. A forma dessa função define o efeito visual do filtro.

Corte abrupto e *ringing*. Filtros ideais com transição instantânea em uma frequência de corte D_0 produzem discontinuidades no domínio da frequência. Essa discontinuidade se reflete no domínio espacial como oscilações próximas a bordas, conhecidas como *ringing*. Esse efeito está associado à convolução com funções de suporte infinito no espaço, como a função *sinc*, conforme ilustrado na Figure 5.10.

Filtros com transição suave. Alternativas como os filtros Gaussiano e Butterworth suavizam a transição entre regiões de passagem e rejeição, reduzindo o *ringing*. Em contrapartida, essa suavização implica uma fronteira de separação menos definida entre frequências preservadas e atenuadas.

```
# Simulando o Filtro Ideal na Frequência (Cilindro) e sua representação Espacial (Sinc)
N_grid = 2**7
u = np.arange(-N_grid//2, N_grid//2)
U, V = np.meshgrid(u, u)
D = np.sqrt(U**2 + V**2)

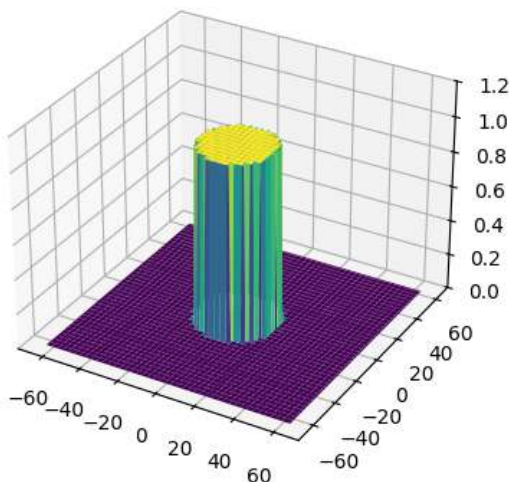
# Frequência: Cilindro Ideal (1 no centro, 0 fora do raio 20)
H_freq = np.zeros((N_grid, N_grid))
H_freq[D <= 20] = 1

# Espaço: A inversa resulta na famigerada Sinc 2D
h_space = np.fft.fftshift(np.real(np.fft.ifft2(np.fft.ifftshift(H_freq))))

fig, ax = plt.subplots(1, 2, subplot_kw={'projection': '3d'}, figsize=(12, 4))
ax[0].plot_surface(U, V, H_freq, cmap='viridis', edgecolor='none')
ax[0].set_title("Frequência: Filtro Ideal (Cilindro)")
ax[0].set_zlim(0, 1.2)

ax[1].plot_surface(U, V, h_space, cmap='plasma', edgecolor='none')
ax[1].set_title("Espaço Real: Ondulações da Sinc (Causa do Ringing)")
plt.tight_layout(); plt.show()
```

Frequência: Filtro Ideal (Cilindro)



Espaço Real: Ondulações da Sinc (Causa do Ringing)

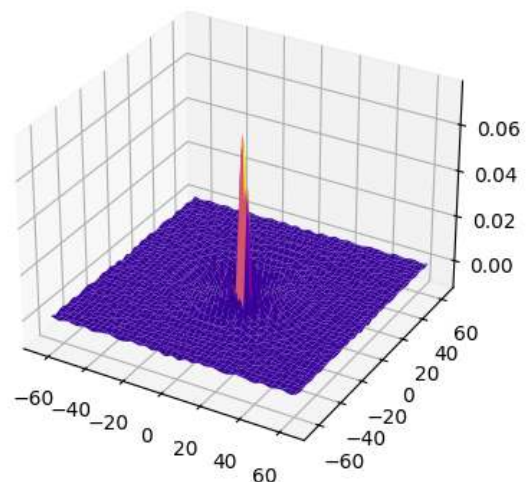


Figura 5.10: A Dualidade Perigosa: O corte abrupto na Frequência (Cilindro) transforma-se obrigatoriamente numa Sinc espacial. Suas ondulações causam o *ringing* fantasma nas bordas da imagem.

5.5.1 Filtros Passa-Baixa

Filtros passa-baixa atenuam componentes de alta frequência, resultando em suavização da imagem e redução de ruído. Após a centralização do espectro (FFT Shift), a distância de cada ponto ao centro é dada por:

$$D(u, v) = \sqrt{\left(u - \frac{M}{2}\right)^2 + \left(v - \frac{N}{2}\right)^2} \quad (5.5)$$

Filtro Ideal (LPFI):

$$H_{\text{ideal}}(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases} \quad (5.6)$$

O corte abrupto em D_0 introduz descontinuidades no domínio da frequência, resultando em oscilações no domínio espacial conhecidas como *ringing*. Esse efeito está associado à convolução com funções de suporte infinito.

Filtro Gaussiano (LPFG):

$$H_{\text{gauss}}(u, v) = e^{-D^2(u, v)/(2\sigma^2)} \quad (5.7)$$

A suavidade da função Gaussiana no domínio da frequência evita descontinuidades, o que elimina o *ringing* e produz uma transição gradual entre frequências preservadas e atenuadas.

Filtro Butterworth (LPFB) de ordem n :

$$H_{\text{BW}}(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}} \quad (5.8)$$

O parâmetro n controla a suavidade da transição entre passagem e rejeição de frequências. Valores pequenos produzem transições suaves, enquanto valores grandes aproximam o comportamento do filtro ideal, com maior risco de *ringing*. Um exemplo comparativo é apresentado na Figure 5.11.



Figura 5.11: Filtros passa-baixa.

5.5.2 Filtros Passa-Alta e Passa-Banda

Filtros passa-alta podem ser obtidos a partir de um filtro passa-baixa complementar, definido como:

$$H_{\text{HP}}(u, v) = 1 - H_{\text{LP}}(u, v)$$

Esse tipo de filtro preserva componentes de alta frequência, realçando bordas e detalhes, enquanto atenua regiões de variação suave.

Filtros passa-banda preservam apenas uma faixa intermediária de frequências, limitada por dois raios D_L e D_H :

$$H_{BP}(u, v) = H_{LP}^{(D_H)}(u, v) \cdot [1 - H_{LP}^{(D_L)}(u, v)]$$

Esse tipo de filtragem é útil quando se deseja remover simultaneamente componentes de baixa e alta frequência, preservando apenas estruturas de escala intermediária.

Uma aplicação importante é a remoção de **ruído periódico**, no qual padrões regulares aparecem como picos localizados no espectro de magnitude. Esses picos podem ser atenuados por meio de filtros *notch* (rejeita-banda), posicionados especificamente nas frequências indesejadas.

Exemplos de filtros no domínio da frequência são apresentados no simulador da Figure 5.12, Figure 5.13 e Figure 5.14.



Figura 5.12: Simulador interativo de filtros no domínio da frequência.

```
import io

def distancia_centro(M, N):
    """Matriz de distâncias ao centro do espectro."""
    u = np.arange(M) - M // 2
    v = np.arange(N) - N // 2
    V, U = np.meshgrid(v, u)
    return np.sqrt(U**2 + V**2)

def aplicar_filtro_freq(img, H):
```

```

"""Aplica filtro H (centrado) a imagem via FFT."""
F = np.fft.fftshift(np.fft.fft2(img.astype(np.float64)))
Fg = F * H
g = np.real(np.fft.ifft2(np.fft.ifftshift(Fg)))
return cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

M, N = img_gray.shape
D = distancia_centro(M, N)
D0 = 30 # frequência de corte
n_bw = 2 # ordem do Butterworth

# Funções de transferência
H_ideal = (D <= D0).astype(np.float64)
H_gauss = np.exp(-D**2 / (2 * D0**2))
H_bw = 1.0 / (1.0 + (D / D0)**(2 * n_bw))

# Imagens filtradas
img_ideal = aplicar_filtro_freq(img_gray, H_ideal)
img_gauss = aplicar_filtro_freq(img_gray, H_gauss)
img_bw = aplicar_filtro_freq(img_gray, H_bw)

# Perfis de H(u,v)
def fig2img(fig):
    b = io.BytesIO(); fig.savefig(b, format='png', dpi=100); plt.close(fig); b.seek(0)
    return (plt.imread(b)[:,:,:3]*255).astype(np.uint8)

fig, ax = plt.subplots(figsize=(6, 3))
linha = M // 2
ax.plot(H_ideal[linha, :], label="Ideal", color="#D85A30", lw=1.5, ls="--")
ax.plot(H_gauss[linha, :], label="Gaussiano", color="#1D9E75", lw=1.5)
ax.plot(H_bw[linha, :], label="Butterworth", color="#534AB7", lw=1.5)
ax.axvline(N//2-D0, color="#aaa", lw=0.8, ls=":")
ax.axvline(N//2+D0, color="#aaa", lw=0.8, ls=":")
ax.set(title="Perfis H(u,v) - linha central", xlabel="v", ylabel="H(u,v)")
ax.legend(fontsize=8); plt.tight_layout()
perfil_img = fig2img(fig)

# Filtros H visualizados
def H_vis(H):
    return cv2.normalize((H*255).astype(np.uint8), None, 0, 255, cv2.NORM_MINMAX)

mm.show(
    [img_gray, img_ideal, img_gauss, img_bw,
     H_vis(H_ideal), H_vis(H_gauss), H_vis(H_bw), perfil_img],
    titles=[
        "Original", "LPF Ideal", "LPF Gaussiano", "LPF Butterworth (n=2)",
        "H Ideal", "H Gaussiano", "H Butterworth", "Perfis H(u,v)"
    ],
    cols=4, figsize=(16, 9)
)

```

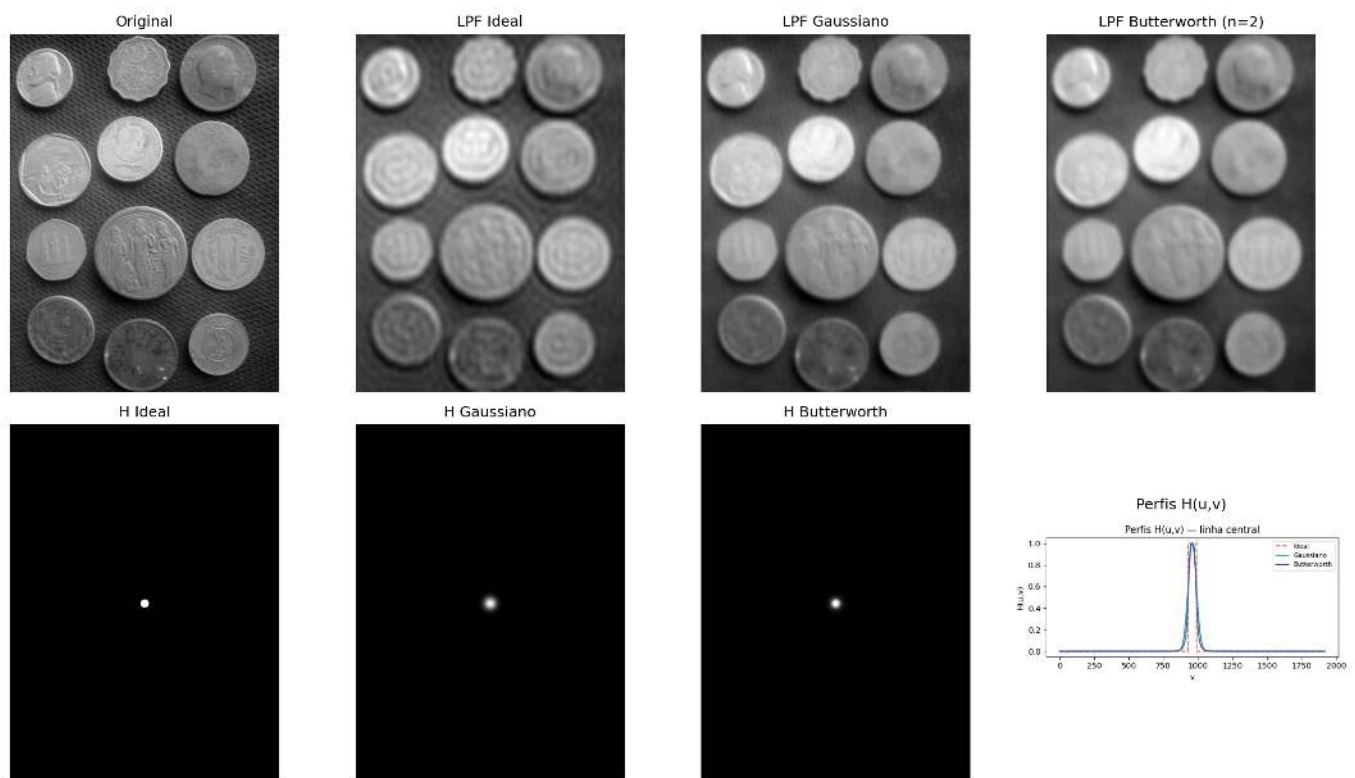


Figura 5.13: Comparação entre filtros passa-baixa: Ideal ($D = 30$), Gaussiano ($D = 30$) e Butterworth ($D = 30$, $n=2$). Perfis de $H(u,v)$ ao longo de uma linha central e imagens filtradas correspondentes.

```
# Filtro passa-alta: complemento do passa-baixa Gaussiano
# Reutiliza aplicar_filtro_freq() definida na célula anterior
H_alta = 1 - H_gauss
img_alta = aplicar_filtro_freq(img_gray, H_alta)

mm.show(
    [img_gray, img_alta],
    titles=["Original", "Passa-alta Gaussiano ( $D_0=30$ )"],
    cols=2
)
```

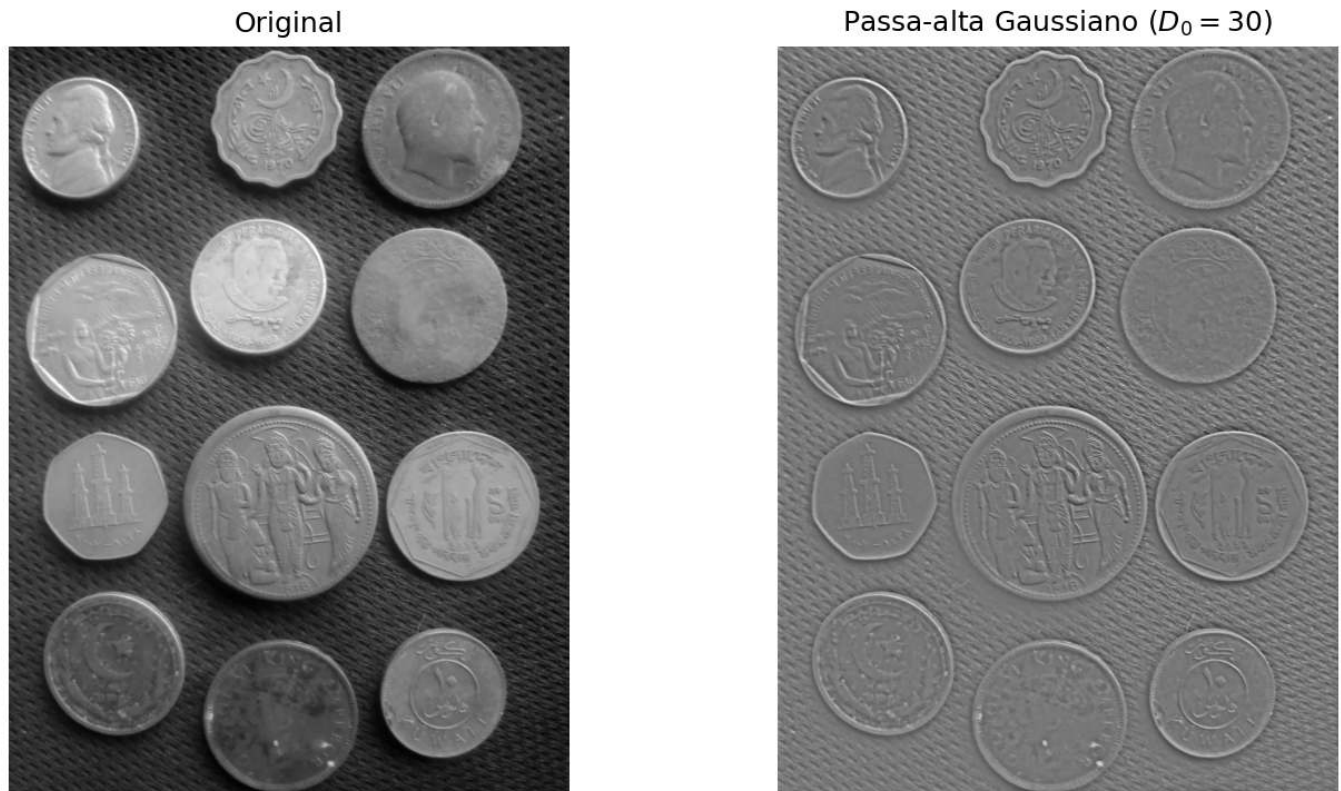


Figura 5.14: Filtro passa-alta Gaussiano. (a) Original; (b) Filtro passa-alta ($D = 30$) - as bordas das moedas e fundo texturizado são realçados.

5.5.3 Remoção de Ruído Periódico

Ruído periódico — associado a interferências elétricas, padrões regulares de sensores ou artefatos de digitalização — aparece no espectro de Fourier como **picos pontuais simétricos em torno do centro**.

O filtro **rejeita-banda** (*notch*) atenua seletivamente essas frequências, preservando as demais componentes da imagem. Um exemplo de aplicação é apresentado na Figure 5.15.

```
# Imagem com ruído periódico sintético
h_img, w_img = img_gray.shape
x = np.arange(w_img)
y = np.arange(h_img)
X, Y = np.meshgrid(x, y)

# Usar frequências inteiras e consistentes (importante para restauração perfeita)
u0, v0 = 20, 20 # frequências exatas do ruído

ruído = 40 * np.sin(2 * np.pi * (u0 * X / w_img + v0 * Y / h_img))
img_ruidosa = np.clip(img_gray.astype(np.float64) + ruído, 0, 255).astype(np.uint8)

# Espectro da imagem ruidosa
F_r = np.fft.fftshift(np.fft.fft2(img_ruidosa.astype(np.float64)))
mag_r = np.log1p(np.abs(F_r))
mag_vis = cv2.normalize(mag_r, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Máscara notch
mascara = np.ones((h_img, w_img), dtype=np.float64)
r_notch = 8 # raio do notch (ajuste fino se necessário)

def suprimir_pico(mask, cy, cx, r):
```

```

    """Zera um disco de raio r centrado em (cy, cx)"""
    yy, xx = np.ogrid[:mask.shape[0], :mask.shape[1]]
    dist = np.sqrt((yy - cy)**2 + (xx - cx)**2)
    mask[dist <= r] = 0
    return mask

# Coordenadas centrais
cy, cx = h_img // 2, w_img // 2

# Suprimir os 4 picos simétricos (importante!)
for dy, dx in [(v0, u0), (-v0, -u0), (v0, -u0), (-v0, u0)]:
    mascara = suprimir_pico(mascara, cy + dy, cx + dx, r_notch)

mascara_vis = (mascara * 255).astype(np.uint8)

# Filtragem e reconstrução
F_filtrada = F_r * mascara
img_rest = np.real(np.fft.ifft2(np.fft.ifftshift(F_filtrada)))
img_rest_vis = cv2.normalize(img_rest, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Avaliação
psnr = cv2.PSNR(img_gray, img_rest_vis)
ssim = cv2.SSIM(img_gray, img_rest_vis) if hasattr(cv2, 'SSIM') else "N/A"
#ssim = ssim_sk(img_gray, img_rest_vis, data_range=255)

print(f"PSNR (original vs restaurada): {psnr:.2f} dB")

# Visualização
mm.show(
    [img_ruidosa, mag_vis, mascara_vis, img_rest_vis],
    titles=[
        "Com ruído periódico",
        "Espectro (log)",
        "Máscara notch",
        f"Restaurada (PSNR={psnr:.1f} dB)"
    ],
    cols=4,
    figsize=(16, 4)
)

```

PSNR (original vs restaurada): 32.93 dB

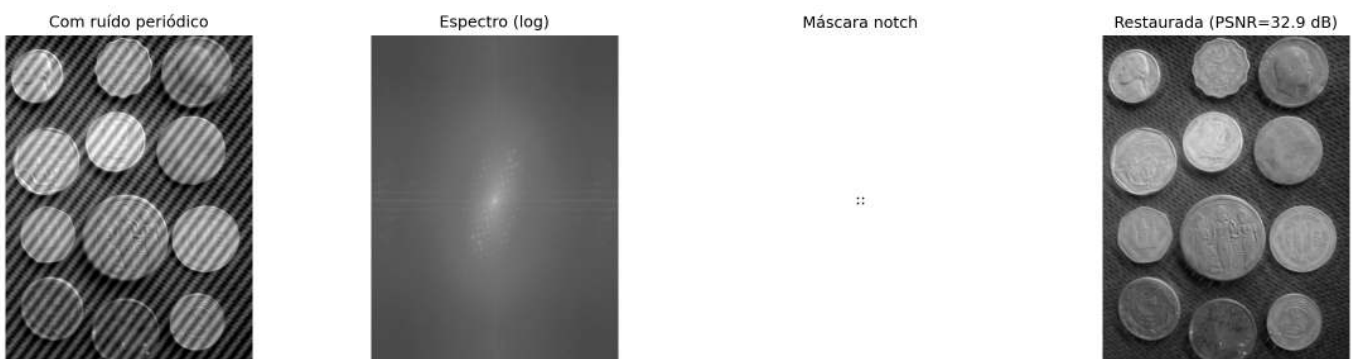


Figura 5.15: Remoção de ruído periódico via filtro *notch* no domínio da frequência: (a) imagem com ruído senoidal, (b) espectro mostrando os picos do ruído, (c) máscara *notch* centrada nos picos, (d) imagem restaurada.

Síntese — Filtros Espectrais

Filtro	Efeito visual	Artefato	Uso
Passa-baixa ideal	Suavização intensa	<i>Ringing</i>	Ilustrativo
Passa-baixa Gaussiano	Suavização suave	Não apresenta <i>ringing</i>	Suavização geral
Passa-baixa Butterworth	Suavização controlada	<i>Ringing</i> (ordens altas)	Compromisso entre suavização e seletividade
Passa-alta	Realce de bordas	Amplificação de ruído	Deteção de contornos
<i>Notch</i>	Remoção seletiva de frequências	Possíveis distorções locais	Remoção de ruído periódico

O projeto de filtros no domínio da frequência consiste na definição de máscaras espectrais. Entretanto, efeitos no domínio espacial, como *ringing* e borrimento, emergem diretamente dessas escolhas no espectro.

5.6 Wavelets e Multirresolução

A Transformada de Fourier decompõe o sinal em frequências **globais**: cada coeficiente $F(u, v)$ recebe contribuições de toda a imagem, sem informação explícita sobre a localização espacial dessas frequências. Assim, estruturas localizadas, como bordas, são representadas de forma distribuída no espectro.

As **wavelets (ondaletas)** superam essa limitação ao utilizar funções base **localizadas no espaço**, que podem ser deslocadas e escaladas. Essas funções possuem **suporte compacto**, isto é, são diferentes de zero apenas em uma região finita do domínio, permitindo uma representação simultânea em termos de **frequência e localização espacial**.

5.6.1 O Limite da Transformada de Fourier: localização espacial

A Transformada de Fourier descreve com precisão **quais frequências estão presentes** em um sinal, mas não representa explicitamente **onde essas frequências ocorrem no espaço**.

No experimento apresentado na Figure 5.16, duas imagens com estruturas localizadas em posições diferentes produzem espectros de magnitude praticamente idênticos. Isso ocorre porque a representação de Fourier é global: cada coeficiente recebe contribuição de toda a imagem.

Como consequência, o espectro de magnitude não representa explicitamente a localização de bordas ou outras estruturas, apenas a distribuição das frequências presentes. Essa limitação motivou o desenvolvimento de representações multirresolução, como a Transformada *Wavelet* Discreta (DWT), capazes de descrever simultaneamente a frequência e a localização espacial das estruturas da imagem.

```
img_sinal1 = np.zeros((128, 128)); img_sinal1[:, 20:25] = 1; img_sinal1[100:105, :] = 1
img_sinal2 = np.zeros((128, 128)); img_sinal2[:, 90:95] = 1; img_sinal2[30:35, :] = 1

mag1 = np.log1p(np.abs(np.fft.fftshift(np.fft.fft2(img_sinal1))))
mag2 = np.log1p(np.abs(np.fft.fftshift(np.fft.fft2(img_sinal2))))

mm.show([img_sinal1, mag1, img_sinal2, mag2],
        titles=["Sinal A", "Espectro A", "Sinal B (Deslocado)", "Espectro B"],
        cols=4, figsize=(14, 4))
```

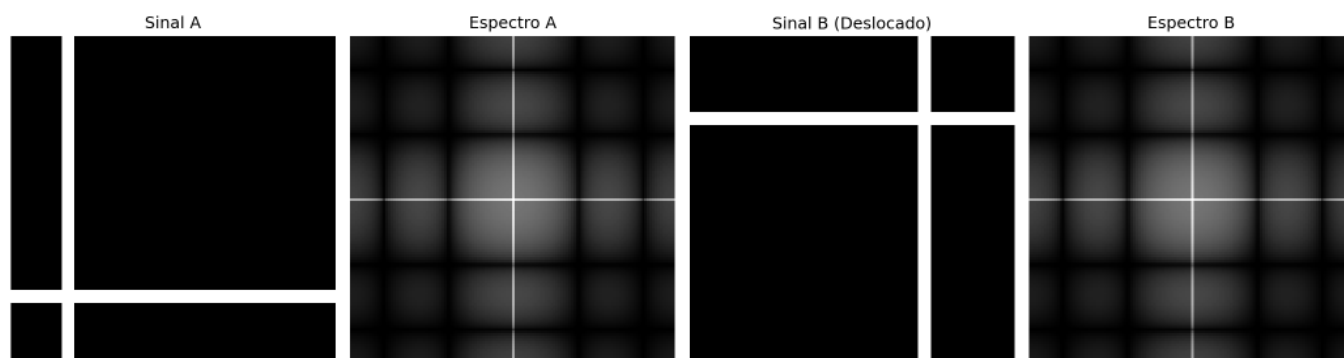



Figura 5.16: Fourier global é cego para a posição. Os espectros não dizem onde as bordas estão.

5.6.2 Transformada *Wavelet* Discreta 2D

A Transformada *Wavelet* Discreta (DWT) aplica, separadamente nas direções horizontal e vertical, dois filtros complementares: um **passa-baixa** h (aproximação) e um **passa-alta** g (detalhes), seguidos de subamostragem por um fator de 2 em cada dimensão. Esse processo produz quatro subbandas, cujos nomes indicam a combinação dos filtros aplicados em cada direção (L = *Low-pass*, passa-baixa; H = *High-pass*, passa-alta). As características de cada subbanda são resumidas na Table 5.3.

$$\text{DWT}(f) = \left\{ \underbrace{\text{LL}}_{\text{aprox.}}, \underbrace{\text{LH}}_{\text{detalhes horizontais}}, \underbrace{\text{HL}}_{\text{detalhes verticais}}, \underbrace{\text{HH}}_{\text{detalhes diagonais}} \right\}.$$

Tabela 5.3: Subbandas produzidas pela Transformada *Wavelet* Discreta 2D (DWT), indicando os filtros aplicados em cada direção e o conteúdo predominante de cada componente.

Subbanda	Filtros aplicados	Conteúdo visual
LL	baixa $\times$ baixa	Aproximação da imagem (versão suavizada e reduzida)
LH	baixa $\times$ alta	Bordas horizontais e variações verticais
HL	alta $\times$ baixa	Bordas verticais e variações horizontais
HH	alta $\times$ alta	Detalhes diagonais e texturas

A decomposição pode ser aplicada recursivamente sobre a subbanda LL, gerando uma representação multirresolução. Após J níveis, obtém-se uma estrutura com $3J + 1$ subbandas, em que cada novo nível reduz a resolução da componente de aproximação.

i Conexão com CNNs

A decomposição multirresolução das *wavelets* possui uma relação conceitual com as representações hierárquicas utilizadas em redes neurais convolucionais (CNNs). Em ambos os casos, sucessivas etapas de filtragem e redução de resolução produzem descrições cada vez mais abstratas da imagem. Entretanto, as *wavelets* utilizam filtros matematicamente definidos e reconstruíveis, enquanto as CNNs aprendem seus filtros durante o treinamento.

5.6.3 Famílias de *Wavelets*

Diferentes famílias de *wavelets* apresentam compromissos distintos entre **suporte espacial**, suavidade e capacidade de compressão. O **suporte** corresponde à extensão da função *wavelet* no domínio espacial: quanto

menor o suporte, mais localizada é a função; quanto maior, mais suave tende a ser sua representação, porém com maior custo computacional. A Table 5.4 compara algumas das famílias mais utilizadas.

Tabela 5.4: Comparação entre famílias de *wavelets*, destacando o comprimento do suporte, o número de momentos nulos, a simetria e aplicações típicas.

<i>Wavelet</i>	Comprimento do suporte	Momentos nulos	Simetria	Uso típico
Haar	2	1	Assimétrica	Introdução e análise básica
Daubechies db4	8	4	Assimétrica	Compressão e análise geral
Symlet sym4	8	4	Quase simétrica	Reconstrução de sinais
Biortogonal 5/3	5/3	2/2	Simétrica	JPEG 2000 sem perda
Biortogonal 9/7	9/7	4/4	Simétrica	JPEG 2000 com perda

Os **momentos nulos** medem a capacidade da *wavelet* de representar regiões suaves da imagem com poucos coeficientes diferentes de zero. Uma *wavelet* com p momentos nulos anula exatamente polinômios de grau até $p - 1$. Em consequência, quanto maior o número de momentos nulos, maior tende a ser a eficiência de compressão em regiões homogêneas, embora isso geralmente implique funções com suporte mais longo.

A Figure 5.17 apresenta as funções de base (*wavelets*) $\psi(t)$ no domínio espacial. Essas funções possuem **suporte compacto**, isto é, são diferentes de zero apenas em uma região finita do domínio, ao contrário das senoides da Transformada de Fourier, que se estendem por todo o domínio.

```
# Import pywt
try:
    import pywt
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "PyWavelets", "-q"])
    import pywt

wavelet_haar = pywt.Wavelet('haar')
wavelet_db4 = pywt.Wavelet('db4')

phi_h, psi_h, x_h = wavelet_haar.wavefun(level=4)
phi_d, psi_d, x_d = wavelet_db4.wavefun(level=4)

fig, ax = plt.subplots(1, 2, figsize=(10, 3))
ax[0].plot(x_h, psi_h, 'b', lw=2); ax[0].set_title("Ondaleta Haar ( )")
ax[1].plot(x_d, psi_d, 'g', lw=2); ax[1].set_title("Ondaleta Daubechies 4 ( )")
plt.tight_layout(); plt.show()
```

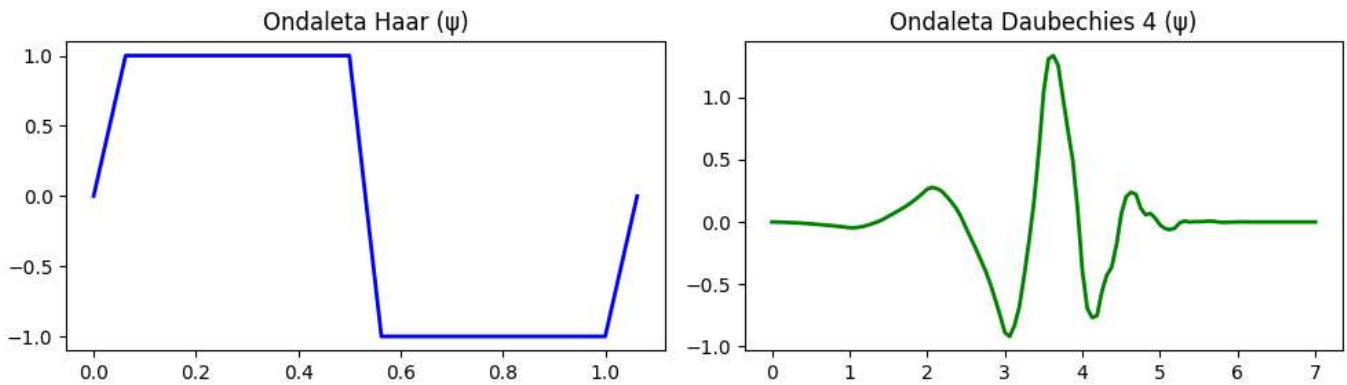


Figura 5.17: Funções da *Wavelet* (ψ). Note como elas rapidamente decaem para zero (suporte compacto), ao contrário das senoides infinitas de Fourier.

O diagrama da Figure 5.18 ilustra a análise multirresolução realizada pela DWT, na qual a subbanda de aproximação (LL) é sucessivamente decomposta, formando uma representação hierárquica com dois níveis.

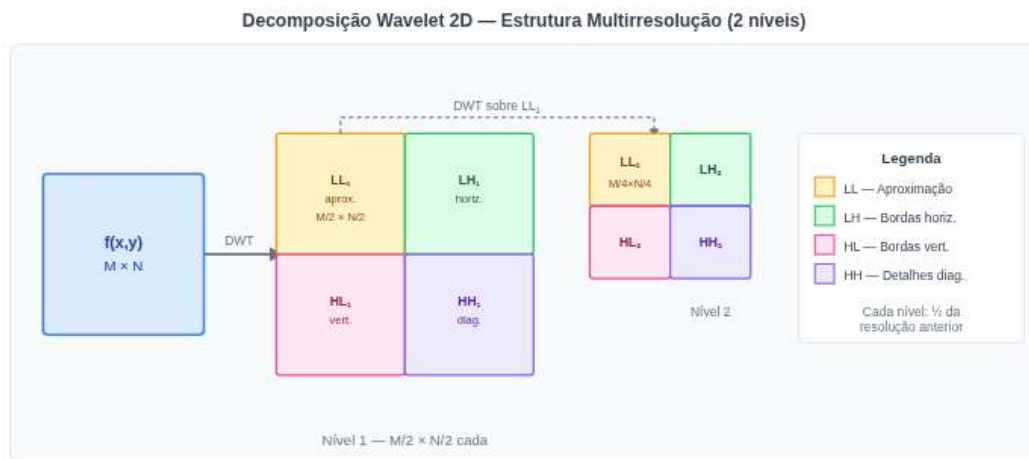


Figura 5.18: Diagrama da decomposição *wavelet* 2D em dois níveis.

O simulador da Figure 5.19 permite explorar interativamente a Transformada *Wavelet* Discreta 2D (DWT) utilizando a *wavelet* de Haar. A decomposição em subbandas evidencia a separação entre a componente de aproximação e as componentes de detalhe da imagem.

Os diferentes padrões de entrada permitem observar o comportamento direcional dos filtros. Em imagens com bordas horizontais e verticais, as subbandas LH e HL destacam, respectivamente, as variações verticais e horizontais da intensidade. Em regiões de variação suave, a maior parte da energia concentra-se na subbanda de aproximação LL, enquanto as subbandas de detalhe apresentam coeficientes próximos de zero.

A análise multirresolução também pode ser observada ao aumentar o número de níveis de decomposição. Nesse caso, apenas a subbanda LL_1 é novamente decomposta, originando as subbandas LL_2 , LH_2 , HL_2 e HH_2 , que formam o segundo nível da representação hierárquica.

Em padrões formados por regiões homogêneas de grande extensão, como um degradê suave ou um tabuleiro composto por blocos grandes, a energia permanece predominantemente concentrada na subbanda LL. No degradê, isso ocorre porque as diferenças entre pixels vizinhos são pequenas. No tabuleiro, por sua vez, os pixels possuem praticamente a mesma intensidade no interior de cada bloco, de modo que apenas as fronteiras entre blocos produzem coeficientes não nulos nas subbandas de detalhe. Como essas fronteiras ocupam apenas uma pequena fração da imagem, sua contribuição para a energia total permanece reduzida.

Para viabilizar a análise visual dessas variações sutis, o simulador incorpora um controle de ganho de contraste

dos detalhes (variando de 1 a 8). Esse parâmetro funciona como um fator de amplificação linear aplicado exclusivamente aos coeficientes das subbandas de detalhe (LH, HL e HH) antes de sua renderização em tela. Em cenários de transição suave (como o gradiente) ou de uniformidade local (como o interior dos blocos do tabuleiro), as diferenças numéricas calculadas pelo filtro passa-altas de Haar resultam em coeficientes muito próximos de zero, o que tornaria os quadrantes correspondentes escuros e imperceptíveis a olho nu. Ao multiplicar esses valores pelo ganho, o simulador resgata visualmente as estruturas de alta frequência ocultas e realça a orientação das bordas remanescentes.

O gráfico de energia por subbanda quantifica essa distribuição entre a componente de aproximação e as componentes de detalhe, demonstrando que o ganho visual não altera a métrica original da energia. Em imagens naturais, a maior parte da energia concentra-se na subbanda LL, enquanto as subbandas LH, HL e HH representam principalmente bordas, texturas e outras variações locais da intensidade.

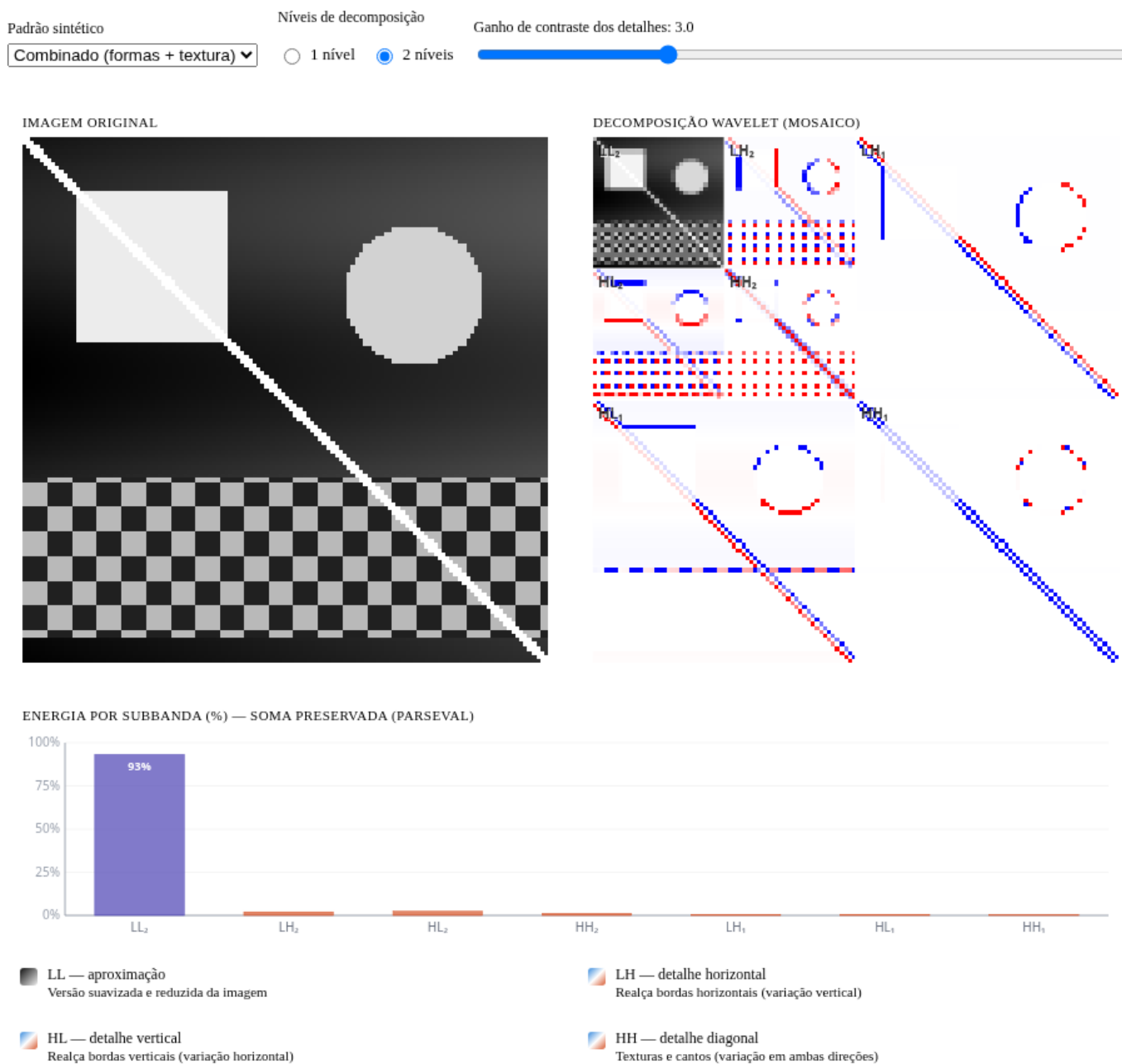


Figura 5.19: Simulação da decomposição *wavelet* 2D.

5.6.4 Análise Multirresolução com a DWT 2D

A Transformada *Wavelet* Discreta 2D (DWT) decompõe uma imagem em componentes de aproximação e detalhe, organizadas de forma hierárquica em diferentes escalas e orientações. Como as subbandas de detalhe em imagens naturais frequentemente apresentam coeficientes de baixo contraste, os exemplos práticos a seguir utilizam um padrão geométrico sintético gerado em Python. Essa abordagem replica o comportamento do simulador da Figure 5.19, tornando visualmente explícitos os efeitos da filtragem espacial e da decomposição multirresolução.

5.6.4.1 Decomposição em Mosaico de Múltiplos Níveis

A Figure 5.20 ilustra a estrutura hierárquica da DWT em dois níveis utilizando a *wavelet* de Haar. O processo baseia-se na aplicação combinada de filtros passa-baixa e passa-alta nas direções horizontal e vertical, seguidos por subamostragem por um fator de 2.

No primeiro nível, a imagem original origina a subbanda de aproximação (LL_1) e as componentes de detalhe horizontal (LH_1), vertical (HL_1) e diagonal (HH_1). Na análise multirresolução, a subbanda LL_1 é novamente filtrada e subamostrada, gerando o segundo nível de decomposição (LL_2 , LH_2 , HL_2 e HH_2).

Para viabilizar a interpretação visual das componentes de detalhe, o código extrai o valor absoluto de seus coeficientes e aplica uma normalização linear (*min-max*) para ocupar toda a faixa dinâmica de tons de cinza [0, 255]. Essa operação transforma regiões homogêneas (coeficientes nulos) em preto e destaca em branco as bordas e texturas extraídas em cada escala e orientação.

```
try:
    import pywt
    HAS_PYWT = True
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "PyWavelets", "-q"])
    import pywt
    HAS_PYWT = True

import numpy as np
import cv2

# Geração da Imagem Sintética (Mesmo padrão 'combined' do simulador)
def gerar_imagem_sintetica(N=256):
    img = np.zeros((N, N), dtype=np.float64)
    for y in range(N):
        for x in range(N):
            v = 55 + 35 * (x / N) + 15 * np.sin(y / 24)
            # Quadrado
            if 24 < x < 100 and 24 < y < 100:
                v = 225
            # Círculo
            cx, cy, r = 190, 76, 34
            if (x - cx)**2 + (y - cy)**2 < r**2:
                v = 205
            # Textura periódica (inferior)
            if y > 164 and y < 244:
                p = 12
                v = 185 if ((x // p + y // p) % 2 == 0) else 65
            # Linha diagonal
            if abs(x - y) < 4:
                v = 240
            img[y, x] = np.clip(v, 0, 255)
    return img.astype(np.uint8)
```

```

# Substitui a imagem escura de moedas pelo padrão sintético claro
img_gray = gerar_imagem_sintetica(256)

# Decomposição wavelet 2 níveis
wavelet = "haar"
img_float = img_gray.astype(np.float64)

# Nível 1
coefs1 = pywt.dwt2(img_float, wavelet)
LL1, (LH1, HL1, HH1) = coefs1

# Nível 2 (aplicado sobre LL1)
coefs2 = pywt.dwt2(LL1, wavelet)
LL2, (LH2, HL2, HH2) = coefs2

def sb_vis(sb):
    """Normaliza subbanda para visualização [0,255]."""
    return cv2.normalize(np.abs(sb), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

print(f"Forma original      : {img_gray.shape}")
print(f"LL1 (nível 1)       : {LL1.shape} | LH1/HL1/HH1: {LH1.shape}")
print(f"LL2 (nível 2)       : {LL2.shape} | LH2/HL2/HH2: {LH2.shape}")

imgs_dwt = [img_gray, sb_vis(LL1), sb_vis(LH1), sb_vis(HL1), sb_vis(HH1),
              sb_vis(LL2), sb_vis(LH2), sb_vis(HL2), sb_vis(HH2)]
titles_dwt = ["Original",
              "LL (aprox.)", "LH (horiz.)", "HL (vert.)", "HH (diag.)",
              "LL (aprox.)", "LH (horiz.)", "HL (vert.)", "HH (diag.)"]

mm.show(imgs_dwt, titles=titles_dwt, cols=5, figsize=(16, 7))

```

```

Forma original      : (256, 256)
LL1 (nível 1)       : (128, 128) | LH1/HL1/HH1: (128, 128)
LL2 (nível 2)       : (64, 64)  | LH2/HL2/HH2: (64, 64)

```

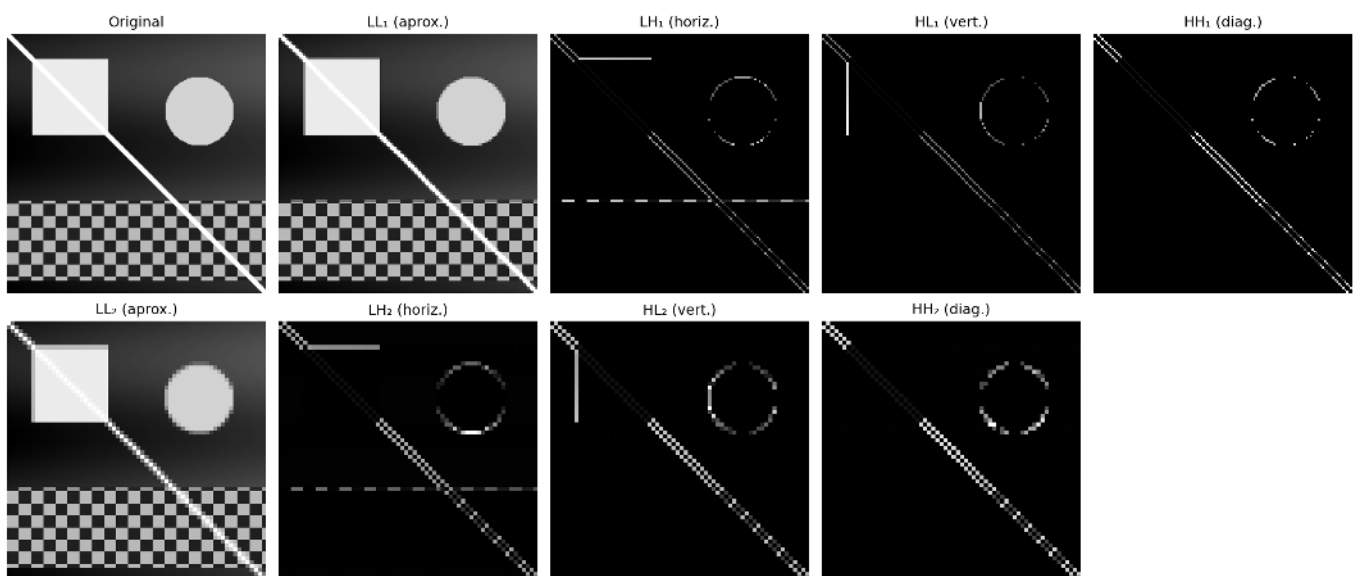


Figura 5.20: Decomposição *wavelet* 2D de 2 níveis com *wavelet* Haar: subbandas LL, LH, HL, HH em cada nível. As subbandas de detalhe revelam estruturas orientadas em diferentes escalas utilizando um padrão sintético.

5.6.4.2 O Compromisso entre Localização e Suavidade

A escolha da função de base (*wavelet*) influencia diretamente a forma como as feições da imagem são distribuídas e codificadas pelos coeficientes da DWT. A Figure 5.21 compara os resultados práticos obtidos ao aplicar quatro famílias distintas sobre o padrão geométrico sintético: *haar*, *db4*, *sym4* e *bior2.2*.

Por possuir suporte curto e formato de função degrau, a *wavelet* de Haar produz coeficientes altamente localizados nas descontinuidades espaciais, gerando bordas finas e nítidas nas subbandas de detalhe. Em contrapartida, famílias como Daubechies (*db4*) e Symlets (*sym4*), que apresentam maior suporte (filtros mais longos) e maior número de momentos nulos, geram respostas mais suaves e distribuídas ao redor das transições, o que pode introduzir leves oscilações ou borramentos nas fronteiras abruptas.

Esse comportamento evidencia o clássico compromisso (*trade-off*) da análise de multirresolução: suportes menores favorecem a localização espacial exata das bordas, enquanto suportes maiores e maior número de momentos nulos tendem a produzir representações mais esparsas e suaves. Essa suavidade e capacidade de atenuação de altas frequências garantem maior eficiência na compactação da energia, características fundamentais para aplicações de compressão de dados e remoção de ruído (*denoising*).

```
import numpy as np
import cv2
import pywt

# Garante que img_gray e img_float utilizem o mesmo padrão sintético claro
if 'gerar_imagem_sintetica' in globals():
    img_gray = gerar_imagem_sintetica(256)
else:
    # Fallback caso o bloco anterior não tenha sido executado na mesma sessão
    def gerar_imagem_sintetica(N=256):
        img = np.zeros((N, N), dtype=np.float64)
        for y in range(N):
            for x in range(N):
                v = 55 + 35 * (x / N) + 15 * np.sin(y / 24)
                if 24 < x < 100 and 24 < y < 100: v = 225
                cx, cy, r = 190, 76, 34
                if (x - cx)**2 + (y - cy)**2 < r**2: v = 205
                if y > 164 and y < 244:
                    p = 12
                    v = 185 if ((x // p + y // p) % 2 == 0) else 65
                if abs(x - y) < 4: v = 240
                img[y, x] = np.clip(v, 0, 255)
            return img.astype(np.uint8)
        img_gray = gerar_imagem_sintetica(256)

img_float = img_gray.astype(np.float64)

wavelets_comp = ["haar", "db4", "sym4", "bior2.2"]
imgs_comp, titles_comp = [], []

for wname in wavelets_comp:
    LL, (LH, HL, HH) = pywt.dwt2(img_float, wname)
    imgs_comp += [sb_vis(LL), sb_vis(HH)]
    titles_comp += [f"{wname} - LL ", f"{wname} - HH "]

mm.show(imgs_comp, titles=titles_comp, cols=4, figsize=(14, 8))
```

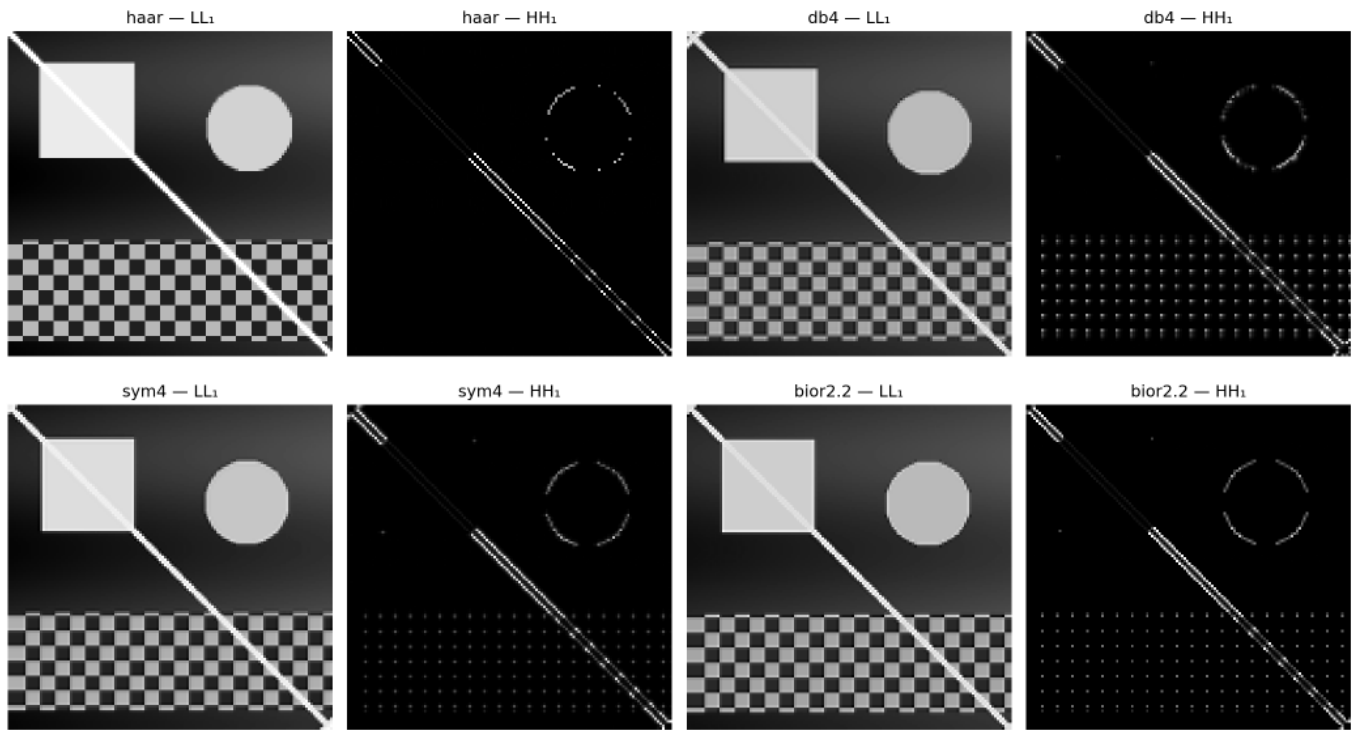



Figura 5.21: Comparação entre famílias de *wavelets*: Haar, db4, sym4 e bior2.2. Subbanda LL (aproximação) e HH (diagonal) para cada escolha, ilustrando o compromisso entre compactação e suavidade com base no padrão sintético.

5.6.4.3 Limiarização de Coeficientes e Compressão

Uma das principais aplicações da Transformada *Wavelet* Discreta (DWT) é a compressão de dados, impulsionada pela capacidade de representação **esparças** dos coeficientes. A Figure 5.22 ilustra o efeito da limiarização abrupta (*hard thresholding*), técnica na qual coeficientes de detalhe com magnitude inferior a um limiar T são integralmente anulados antes do processo de síntese realizado pela Transformada *Wavelet* Discreta Inversa (IDWT).

À medida que o limiar T é elevado, um volume crescente de coeficientes de alta frequência é zerado. Por concentrarem menor energia, a remoção dessas componentes reduz consideravelmente a quantidade de informação necessária para representar a imagem, mantendo a componente de aproximação global (a subbanda LL mais profunda) intacta para preservar a estrutura macro. Visualmente, esse descarte de coeficientes manifesta-se através do desaparecimento progressivo de texturas finas e da suavização de transições abruptas de intensidade.

A fidelidade da imagem reconstruída frente à original é quantificada pela métrica de **Pico da Relação Sinal-Ruído (PSNR, *Peak Signal-to-Noise Ratio*)**, expressa em decibéis (dB). Valores mais altos de PSNR indicam menor distorção e maior proximidade matemática com o sinal original. O experimento prático evidencia o decaimento gradual do PSNR conforme a agressividade da limiarização aumenta, permitindo avaliar numericamente o limiar ótimo para o balanço entre compressão e degradação visual.

```
import numpy as np
import cv2
import pywt

# Garante que img_gray utilize o mesmo padrão sintético claro
if 'gerar_imagem_sintetica' in globals():
    img_gray = gerar_imagem_sintetica(256)
else:
    # Fallback caso o bloco anterior não tenha sido executado na mesma sessão
```

```

def gerar_imagem_sintetica(N=256):
    img = np.zeros((N, N), dtype=np.float64)
    for y in range(N):
        for x in range(N):
            v = 55 + 35 * (x / N) + 15 * np.sin(y / 24)
            if 24 < x < 100 and 24 < y < 100: v = 225
            cx, cy, r = 190, 76, 34
            if (x - cx)**2 + (y - cy)**2 < r**2: v = 205
            if y > 164 and y < 244:
                p = 12
                v = 185 if ((x // p + y // p) % 2 == 0) else 65
            if abs(x - y) < 4: v = 240
            img[y, x] = np.clip(v, 0, 255)
    return img.astype(np.uint8)
img_gray = gerar_imagem_sintetica(256)

def dwt_threshold_reconstruct(img, wavelet='db4', nivel=2, threshold=0.0):
    """Decompõe, aplica limiar e reconstrói via IDWT."""
    coefs = pywt.wavedec2(img.astype(np.float64), wavelet, level=nivel)
    # Copia e aplica hard thresholding em todos os detalhes
    coefs_t = [coefs[0]] # LL final não é limiarizado
    for detalhe in coefs[1:]:
        coefs_t.append(tuple(pywt.threshold(sb, threshold, mode='hard') for sb in detalhe))
    rec = pywt.waverec2(coefs_t, wavelet)
    # Recorte para dimensão original
    rec = rec[:img.shape[0], :img.shape[1]]
    return np.clip(rec, 0, 255).astype(np.uint8)

thresholds = [0, 10, 30, 60, 100]
imgs_thr = [img_gray]
titles_thr = ["Original"]

for t in thresholds:
    rec = dwt_threshold_reconstruct(img_gray, threshold=t)
    psnr = cv2.PSNR(img_gray, rec)
    imgs_thr.append(rec)
    titles_thr.append(f"T={t} PSNR={psnr:.1f} dB")

mm.show(imgs_thr, titles=titles_thr, cols=3, figsize=(14, 10))

```

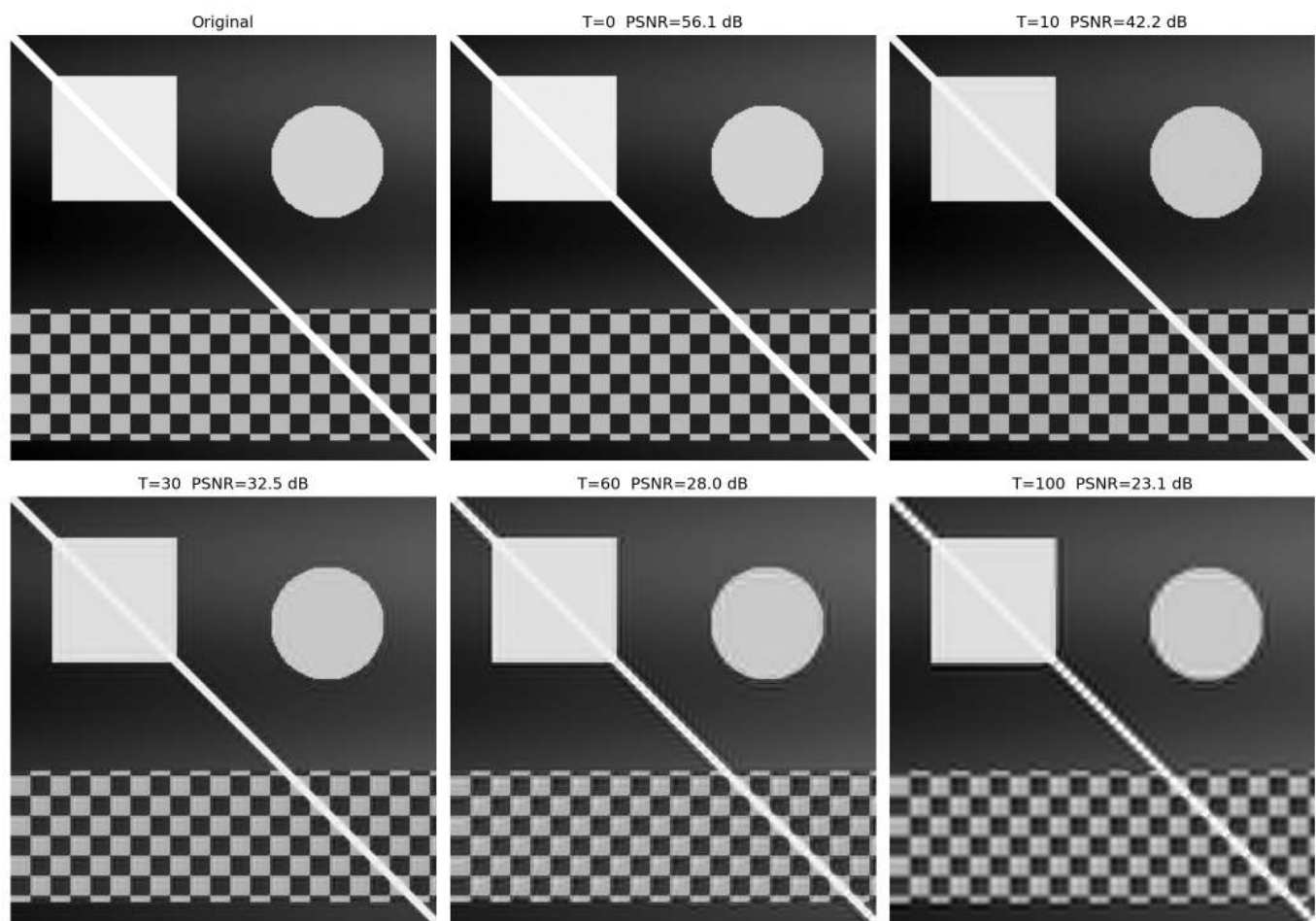


Figura 5.22: Reconstrução *wavelet* com limiarização de coeficientes (*hard thresholding*): à medida que o limiar aumenta, mais detalhes são zerados, produzindo imagens progressivamente mais suaves. Métrica PSNR quantifica a perda de qualidade sobre o padrão sintético.

Síntese — Fourier vs. *Wavelets*: quando utilizar cada abordagem?

A Table 5.5 sintetiza as principais diferenças estruturais e operacionais entre a Transformada Discreta de Fourier (DFT) e a Transformada *Wavelet* Discreta (DWT).

Tabela 5.5: Comparação entre a Transformada Discreta de Fourier (DFT) e a Transformada *Wavelet* Discreta (DWT), destacando suas principais características e aplicações.

Critério	Fourier (DFT)	<i>Wavelet</i> (DWT)
Funções de base	Senoides de suporte infinito	Funções de suporte compacto
Localização espacial	Não explícita (global)	Explícita (local)
Filtragem espectral	Excelente para controle fino de frequências	Baseada em subbandas (escalas)
Compressão de imagens	Base da DCT (JPEG tradicional)	Base da DWT (JPEG 2000)
Análise multiescala	Não	Sim
Remoção de ruído periódico	Altamente eficiente	Pouco indicada
Sinais não estacionários	Limitada	Altamente eficiente

Em termos práticos, a DFT consolida-se como a ferramenta ideal para análise espectral pura, projeto de filtros seletivos no domínio da frequência e atenuação de ruídos periódicos e harmônicos. Por outro lado, a DWT sobressai-se em cenários que exigem a preservação rigorosa da localização espacial das feições associada ao seu conteúdo frequencial, destacando-se em compressão de dados, análise multirresolução e processamento

de transições abruptas. Desse modo, ambas as transformadas devem ser compreendidas como técnicas perfeitamente complementares, mapeando caminhos distintos e específicos para a resolução de problemas em PDI-VC.

i Analogias com Áudio: Limitações e Cuidados

Ao fazer analogias entre processamento de imagens e áudio, é importante considerar as diferenças fundamentais:

- Em sistemas de áudio estéreo/multicanais, a fase entre canais é crucial para a percepção de localização espacial (diferenças interaurais de fase e tempo).
- Em sistemas monaurais, a fase tem influência perceptual limitada — o ouvido humano é relativamente insensível à fase absoluta de componentes senoidais isolados.
- Em imagens, a fase da DFT é sempre fundamental para a localização espacial de estruturas, independentemente de ser uma imagem monocromática ou colorida.

A analogia entre fase em áudio e fase em imagens deve ser usada com cautela, destacando que, embora ambas carreguem informações sobre a organização espacial/temporal do sinal, os mecanismos perceptuais são fundamentalmente diferentes.

5.7 Compressão de Imagens

Enquanto as *wavelets* estabelecem a fundação teórica do padrão JPEG 2000, o padrão JPEG tradicional baseia-se na **Transformada Discreta de Cossenos (DCT, *Discrete Cosine Transform*)**. Apesar das diferenças estruturais, ambas as abordagens compartilham o mesmo princípio fundamental: compactar a energia da imagem em um número reduzido de coeficientes e descartar as componentes de menor relevância com impacto visual mínimo.

O objetivo central da compressão é reduzir o volume de dados necessário para o armazenamento ou transmissão de uma imagem. Esse processo é viabilizado pela identificação e eliminação de **redundâncias** estruturais e perceptuais.

5.7.1 Taxonomia das Redundâncias

O desenvolvimento de algoritmos de compressão fundamenta-se na identificação e eliminação de três categorias principais de redundância, sintetizadas na Table 5.6.

Tabela 5.6: Categorias de redundância em imagens digitais e seus respectivos mecanismos de exploração.

Tipo	Definição	Abordagem de Exploração
Espacial (interpixel)	Alta correlação e dependência estatística entre pixels vizinhos.	DCT, DWT e codificação preditiva.
Espectral (intercanal)	Correlação estatística entre os canais de cor de uma mesma imagem.	Transformações de espaço de cor (ex: RGB para YC_bC_r).
Psicovisual	Insensibilidade do sistema visual humano (SVH) a variações de alta frequência e baixo contraste.	Processos de quantização seletiva de coeficientes.

A depender da preservação da informação original após o processo de decodificação, os métodos de compressão dividem-se em duas classes fundamentais:

- **Sem perda (*lossless*):** Garante uma reconstrução bit a bit idêntica à imagem original. É empregada em cenários onde a integridade dos dados é estritamente crítica, como em imagens médicas, diagnósticos por imagem e armazenamento de documentos textuais.

- **Com perda (*lossy*):** Admite a introdução de uma distorção controlada no sinal em troca de taxas de compressão substancialmente mais elevadas. É a abordagem padrão para fotografias de consumo e *streaming* de vídeo, ecossistemas nos quais o SVH tolera pequenas atenuações de alta frequência sem percepção de degradação da qualidade visual.

5.7.2 Transformada de Cossenos Discreta (DCT-II 2D)

A **Transformada de Cossenos Discreta** (DCT) constitui a operação central do padrão JPEG. Diferentemente da DFT, que utiliza uma base complexa, a DCT baseia-se em funções trigonométricas puramente reais. Para um bloco de imagem $f(x, y)$ de dimensões $N \times N$, a **DCT-II 2D** mapeia o sinal espacial para o domínio das frequências espaciais, gerando a matriz de coeficientes $C(u, v)$ por meio de:

$$C(u, v) = \alpha(u) \alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right] \quad (5.9)$$

onde os fatores de normalização ortogonal são dados por $\alpha(0) = \sqrt{1/N}$ e $\alpha(k) = \sqrt{2/N}$ para $k > 0$.

Cada coeficiente $C(u, v)$ quantifica a contribuição — ou “peso” — de uma frequência espacial específica dentro daquele bloco. O termo $C(0, 0)$ é denominado **componente DC** e representa a intensidade média do bloco (frequência nula). Os demais coeficientes, chamados de **componentes AC** (*Alternating Current*), correspondem às frequências espaciais progressivamente maiores.

5.7.3 As Funções de Base da DCT

Sob uma perspectiva geométrica, a Equation 5.9 realiza a projeção do bloco de pixels sobre um conjunto de funções ortogonais. Para o caso padrão do JPEG ($N = 8$), o bloco espacial é decomposto em uma combinação linear de **64 funções de base** bidimensionais, denotadas por $B_{u,v}(x, y)$ e geradas pelo produto de funções cossenoidais:

$$B_{u,v}(x, y) = \cos \left[\frac{\pi(2x+1)u}{16} \right] \cos \left[\frac{\pi(2y+1)v}{16} \right]$$

Dessa forma, a operação inversa pode ser interpretada como a reconstrução exata do bloco original por meio da soma ponderada dessas 64 matrizes de base, onde cada coeficiente $C(u, v)$ atua como o peso analítico de sua respectiva componente harmônica.

A **frequência espacial** indicada pelos índices (u, v) determina o número de ciclos de oscilação ao longo das dimensões horizontais e verticais do bloco. Como ilustrado na Figure 5.23 — cujo código isola cada base aplicando a transformação inversa sobre impulsos unitários —, essas 64 funções são organizadas em uma matriz 8×8 . O canto superior esquerdo ($u = 0, v = 0$) exhibe o padrão uniforme de frequência nula (DC), enquanto o avanço para a direita (eixo u) ou para baixo (eixo v) mapeia variações harmônicas progressivamente maiores, representando transições rápidas, bordas e texturas nas orientações horizontais, verticais e diagonais.

i DCT vs DFT: Vantagem da Compactação de Energia

Tanto a DCT quanto a DFT mapeiam um bloco $N \times N$ espacial em uma matriz de coeficientes de mesma dimensão. Contudo, para imagens naturais, a DCT apresenta maior eficiência na **compactação de energia** nas baixas frequências. Isso ocorre porque a DCT assume implicitamente uma simetria par do sinal nas fronteiras do bloco, o que equivale a uma extensão periódica contínua, minimizando o efeito de espalhamento espectral (*ringing*). Como consequência, a maioria dos coeficientes AC decai rapidamente para valores próximos de zero, otimizando o *pipeline* de compressão sem introduzir degradação visual perceptível.

```

from scipy.fft import dct, idct # linha adicionada

fig, axes = plt.subplots(8, 8, figsize=(6, 6))
fig.subplots_adjust(hspace=0.05, wspace=0.05)
for i in range(8):
    for j in range(8):
        coef = np.zeros((8, 8)); coef[i, j] = 1
        b = idct(idct(coef.T, norm='ortho').T, norm='ortho')
        axes[i, j].imshow(b, cmap='gray')
        axes[i, j].axis('off')
plt.suptitle("As 64 Bases da DCT 8x8", y=0.92, fontsize=12, fontweight='bold')
plt.show()

```

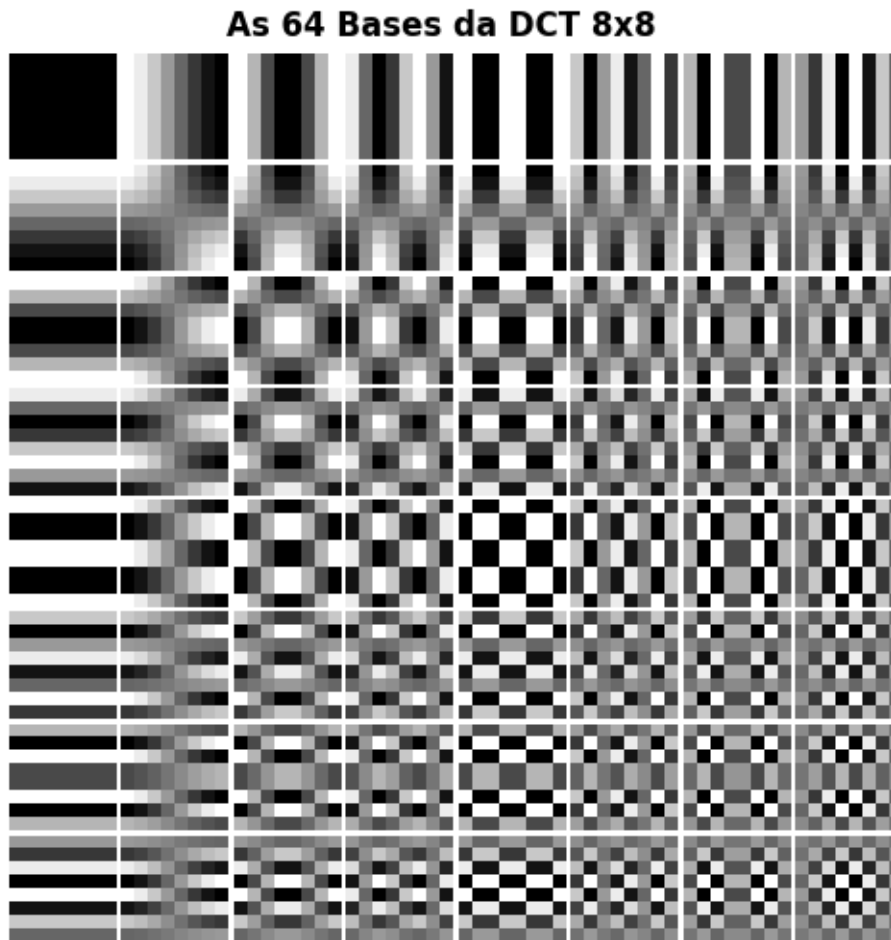


Figura 5.23: O Alfabeto Visual do JPEG: As 64 funções de base da DCT-II. O coeficiente DC fica no topo esquerdo (suave). Ao descer e avançar à direita, a oscilação espacial aumenta drasticamente.

5.7.4 Concentração de Energia e Reconstrução Progressiva

Antes da aplicação da DCT, os pixels do bloco de intensidade são rotineiramente transladados (subtraindo-se 128 para imagens de 8 bits) a fim de centralizar o sinal em torno de zero, eliminando componentes contínuas desnecessárias. Ao computar a DCT sobre o bloco resultante, a propriedade de **compactação de energia** torna-se evidente: a quase totalidade da variância e da informação da imagem original concentra-se no coeficiente DC ($C(0,0)$) e nos primeiros harmônicos AC de baixa frequência.

A Figure 5.24 demonstra esse fenômeno por meio de uma reconstrução progressiva por truncamento abrupto. Em vez de utilizar todos os 64 coeficientes, o algoritmo preserva apenas os k primeiros componentes — selecionados com base em uma varredura que prioriza as baixas frequências espaciais — e anula os demais.

A síntese inversa (**IDCT**) realizada com apenas uma fração dos coeficientes (como 15% ou 30%) já é capaz de recuperar as estruturas e a iluminação macro do bloco original de pixels. À medida que harmônicos de frequências mais altas são progressivamente reincorporados, os detalhes finos e as transições rápidas são restaurados. Esse comportamento valida o princípio da compressão perceptual: as altas frequências descartadas possuem pouca energia e sua ausência, em condições normais, gera um impacto visual secundário na percepção do observador.

```
from scipy.fft import dct, idct

def dct2(bloco):
    """DCT-II 2D ortogonal (separável)."""
    return dct(dct(bloco.T, norm='ortho').T, norm='ortho')

def idct2(coefs):
    """IDCT-II 2D ortogonal."""
    return idct(idct(coefs.T, norm='ortho').T, norm='ortho')

# Bloco 8x8 centralizado da imagem
cy, cx = img_gray.shape[0]//2, img_gray.shape[1]//2
bloco = img_gray[cy:cy+8, cx:cx+8].astype(np.float64) - 128.0

C = dct2(bloco)

print("Coeficientes DCT do bloco 8x8:")
print(np.round(C).astype(int))
print(f"\nEnergia DC      : {C[0,0]**2:.1f}")
print(f"Energia total    : {(C**2).sum():.1f}")
print(f"Fração no DC      : {C[0,0]**2 / (C**2).sum():.1%} ← concentração de energia")

# Reconstrução progressiva
imgs_rec = [cv2.normalize((bloco+128).astype(np.uint8), None, 0, 255, cv2.NORM_MINMAX)]
titles_rec = ["Bloco original\n(8x8 pixels)"]

for keep in [1, 4, 10, 20, 40, 64]:
    C_trunc = np.zeros_like(C)
    indices = sorted([(u,v) for u in range(8) for v in range(8)], key=lambda p: p[0]+p[1])
    for u, v in indices[:keep]:
        C_trunc[u, v] = C[u, v]
    rec = np.clip(idct2(C_trunc) + 128, 0, 255).astype(np.uint8)
    imgs_rec.append(rec)
    titles_rec.append(f"{keep} coef.\n({keep}/64:.0%} do total)")

mm.show(imgs_rec, titles=titles_rec, cols=4, figsize=(12, 7))
```

```
Coeficientes DCT do bloco 8x8:
[[ 450   2 -199   -1   0   0 -14   0]
 [  -2  506   0 -58   0  24   0 -24]
 [-199   2  89   -1  82   1   0   0]
 [  -1 -58   0 -158   0  59   0 -24]
 [   0   0  83   0 -89   0 -34   0]
 [   0  24   0  58   0  84   0 -58]
 [-14   0   0   0 -34   0  89  -1]
 [   0 -24   0 -24   0 -58   0 -76]]

Energia DC      : 202725.1
Energia total   : 639868.0
Fração no DC    : 31.7% ← concentração de energia
```

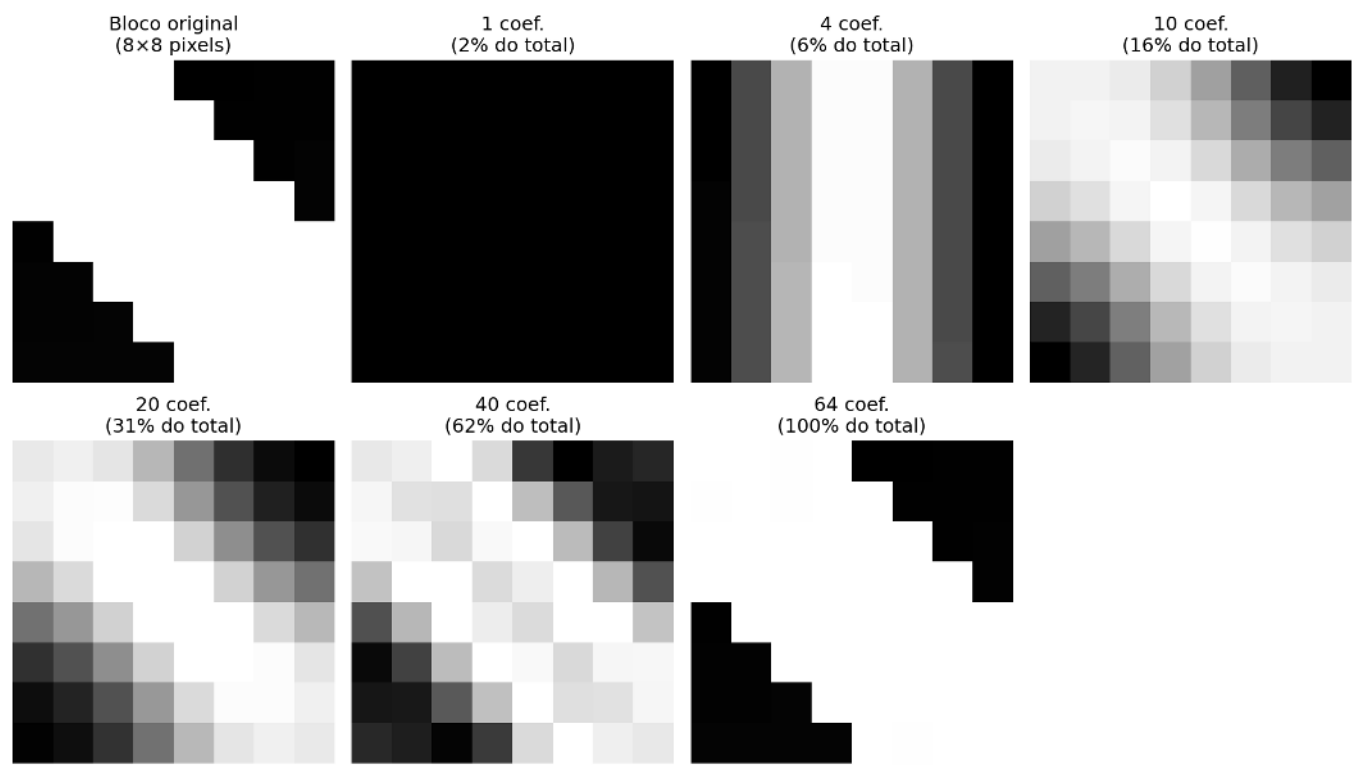
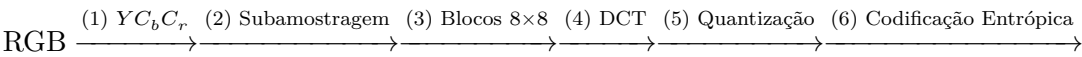



Figura 5.24: DCT 2D em bloco 8×8: coeficientes e reconstrução progressiva.

5.7.5 O *Pipeline* de Compressão JPEG

O padrão JPEG opera dividindo a imagem em blocos disjuntos de 8 × 8 pixels, processados por meio de uma sequência de transformações espaciais, perceptuais e estatísticas. O *pipeline* completo de codificação é estruturado em seis etapas principais:



A Table 5.7 detalha a função analítica e o fundamento perceptual que justifica cada uma dessas etapas.

Tabela 5.7: Etapas do *pipeline* de compressão JPEG e seus respectivos fundamentos de projeto.

Etapa	Operação	Fundamento Perceptual e Estatístico
1	Conversão $RGB \rightarrow YC_bC_r$	Separa a luminância (Y) da cromaticidade (C_b, C_r). O sistema visual humano (SVH) apresenta maior sensibilidade a variações de brilho do que de cor.
2	Subamostragem de cromaticidade (ex: 4:2:0)	Reduz a resolução espacial dos canais de cor pela metade, descartando dados redundantes com impacto visual desprezível.

Etapa	Operação	Fundamento Perceptual e Estatístico
3–4	Centralização e aplicação da DCT 8×8	Translada os pixels para o intervalo $[-128, 127]$ e compacta a energia espectral do bloco nos coeficientes de baixa frequência.
5	Quantização linear seletiva	Divide cada coeficiente $C(u, v)$ pelo elemento correspondente da matriz $Q(u, v)$, aplicando arredondamento inteiro. Constitui a principal fonte de compressão com perda.
6	Varredura em ziguezague e codificação	Ordena os coeficientes quantizados para maximizar sequências nulas consecutivas, otimizando a codificação por comprimento de corrida (RLE) e a codificação de Huffman.

A **matriz de quantização** $Q(u, v)$ é o mecanismo central de controle do compromisso entre taxa de compressão e qualidade visual. No algoritmo prático da Figure 5.25, o fator de qualidade estipulado pelo usuário (escala de 1 a 100) é convertido em um escalar que parametriza a severidade da matriz Q . Valores reduzidos de qualidade expandem os divisores de $Q(u, v)$, forçando o truncamento em massa dos coeficientes AC para zero. Quando essa eliminação é excessiva, a descontinuidade nas fronteiras dos blocos adjacentes não é atenuada na reconstrução, gerando os denominados **artefatos de bloco** (*blocking artifacts*).

A Lógica da Varredura em Ziguezague

A eficiência do codificador entrópico subsequente à quantização depende diretamente da ordenação dos dados. Como a DCT concentra a energia vital no vértice superior esquerdo da matriz (baixas frequências) e empurra os coeficientes nulos para as extremidades opostas, a leitura linear por linhas ou colunas fragmentaria as sequências de zeros.

A ordenação em ziguezague soluciona essa limitação ao percorrer a matriz diagonalmente em ordem crescente de frequência espacial. Esse mapeamento agrupa os coeficientes significativos no início do vetor e concentra os coeficientes nulos em uma única sequência contínua ao final do arranjo, permitindo que o algoritmo RLE codifique grandes blocos de dados de forma compacta e eficiente.

i O que é RLE?

RLE (*Run-Length Encoding*) é uma técnica de compressão sem perdas que codifica sequências consecutivas de valores idênticos — especialmente **zeros** — como um par (contagem, valor). No JPEG, após a varredura em ziguezague, os coeficientes quantizados são organizados de modo que os zeros se concentrem ao final do vetor. O RLE então comprime essa longa corrida de zeros com extrema eficiência, otimizando o armazenamento e a transmissão da imagem comprimida.

```
import numpy as np
import cv2
from scipy.fft import dct, idct

# Carregamento Seguro da Imagem da Câmera (skimage)
try:
    from skimage import data
```

```

    img_gray = data.camera()
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "scikit-image", "-q"])
    from skimage import data
    img_gray = data.camera()

# Redimensiona levemente para 256x256 para manter o padrão e velocidade dos testes anteriores
img_gray = cv2.resize(img_gray, (256, 256))

# Tabela de quantização luminância (padrão JPEG)
Q_luma = np.array([
    [16,11,10,16,24,40,51,61],
    [12,12,14,19,26,58,60,55],
    [14,13,16,24,40,57,69,56],
    [14,17,22,29,51,87,80,62],
    [18,22,37,56,68,109,103,77],
    [24,35,55,64,81,104,113,92],
    [49,64,78,87,103,121,120,101],
    [72,92,95,98,112,100,103,99]
], dtype=np.float64)

def dct2(bloco):
    """DCT-II 2D ortogonal (separável)."""
    return dct(dct(bloco.T, norm='ortho').T, norm='ortho')

def idct2(coefs):
    """IDCT-II 2D ortogonal."""
    return idct(idct(coefs.T, norm='ortho').T, norm='ortho')

def jpeg_compress_block(bloco, Q_table):
    """DCT → quantização → dequantização → IDCT em bloco 8×8."""
    C = dct2(bloco.astype(np.float64) - 128)
    Cq = np.round(C / Q_table) * Q_table # quantiza e dequantiza
    return np.clip(idct2(Cq) + 128, 0, 255)

def jpeg_quality_compress(img, qualidade=50):
    """JPEG simplificado: comprime imagem inteira por blocos 8×8."""
    if qualidade < 50:
        escala = 5000 / qualidade
    else:
        escala = 200 - 2 * qualidade
    # Corrigido de 'scala' para 'escala'
    Q = np.clip(np.round(Q_luma * escala / 100), 1, 255)

    h, w = img.shape
    result = np.zeros_like(img, dtype=np.float64)
    for r in range(0, h-7, 8):
        for c in range(0, w-7, 8):
            result[r:r+8, c:c+8] = jpeg_compress_block(img[r:r+8, c:c+8], Q)
    return result.astype(np.uint8)

# Comparação de fatores de qualidade
qualidades = [10, 25, 50, 75, 90]
imgs_jpeg = [img_gray]
titles_jpeg = ["Original\n(Cameraman)"]

for q in qualidades:
    rec = jpeg_quality_compress(img_gray, qualidade=q)
    psnr = cv2.PSNR(img_gray, rec)

```

```

imgs_jpeg.append(rec)
titles_jpeg.append(f"Q={q}\nPSNR={psnr:.1f}dB")

mm.show(imgs_jpeg, titles=titles_jpeg, cols=3, figsize=(14, 10))

```



Figura 5.25: *Pipeline JPEG* simplificado aplicado à imagem clássica do *Cameraman*: DCT em blocos 8×8 , quantização com diferentes fatores de qualidade e reconstrução via IDCT. Os artefatos de bloco (*blocking artifacts*) tornam-se visualmente evidentes em fatores de qualidade reduzidos ($Q = 10$ e $Q = 25$).

5.7.6 Simulador Interativo: Quantização DCT

O simulador da Figure 5.26 permite explorar o impacto do processo de quantização sobre um bloco 8×8 extraído de uma imagem real, sintetizando em tempo real as seguintes componentes:

- **Bloco original e reconstruído:** Representação direta dos pixels no domínio espacial em escala de cinza $[0, 255]$.
- **Coefficientes DCT:** Distribuição da energia mapeada de forma logarítmica em um gradiente cromático, evidenciando a concentração de intensidade no vértice superior esquerdo (baixas frequências).
- **Coefficientes quantizados:** Exibição dos valores inteiros resultantes da divisão pela matriz $Q(u, v)$, tornando visualmente explícito o surgimento em massa de coeficientes nulos (em tons escuros) conforme o fator de qualidade é reduzido.
- **Métricas de compressão:** Painel de monitoramento que quantifica o Erro Quadrático Médio (MSE), o número de coeficientes preservados e o volume de zeros gerados para a codificação entrópica.



Figura 5.26: Simulador interativo de compressão DCT-JPEG: ajuste o fator de qualidade e visualize em tempo real os coeficientes zerados, o bloco reconstruído e o erro de quantização.

5.8 Comparação de Formatos de Imagem

A escolha de um formato de armazenamento digital impacta diretamente o compromisso entre qualidade visual, tamanho de arquivo e custo computacional de decodificação. Os três formatos de maior relevância para arquiteturas *web* e sistemas de computação visual são o JPEG, o PNG e o WebP.

5.8.1 Características dos Formatos

A Table 5.8 sintetiza as propriedades estruturais dos principais formatos de imagem rasterizados.

Tabela 5.8: Comparação estrutural entre os principais formatos de imagem rasterizados.

Característica	JPEG	PNG	WebP
Compressão	Com perda	Sem perda	Com e sem perda.
Transparência (canal alfa)	Não	Sim	Sim.
Suporte a animação	Não	Limitado (APNG)	Sim.
Algoritmo base	DCT + Huffman	DEFLATE (LZ77 + Huffman)	VP8 / VP8L.
Melhor para	Fotografia	Gráficos, texto e ícones	Uso universal em ambiente Web.
Pior para	Texto e bordas nítidas	Imagens fotográficas complexas	Compatibilidade legada.

5.8.2 Métricas de Avaliação de Qualidade

Duas métricas objetivas são amplamente adotadas para quantificar a distorção introduzida por processos de compressão:

Pico da Relação Sinal-Ruído (PSNR, *Peak Signal-to-Noise Ratio*):

$$\text{PSNR} = 10 \log_{10} \left(\frac{L^2}{\text{MSE}} \right) \quad [\text{dB}]$$

(5.10)

onde $L = 255$ para imagens quantizadas em 8 bits e MSE representa o **Erro Quadrático Médio** (*Mean Squared Error*). Valores de PSNR acima de 40 dB indicam excelente fidelidade; entre 30 dB e 40 dB representam boa qualidade; e valores inferiores a 30 dB correspondem a degradações visuais facilmente perceptíveis.

Índice de Similaridade Estrutural (SSIM, *Structural Similarity Index*):

$$\text{SSIM}(f, g) = \frac{(2\mu_f\mu_g + c_1)(2\sigma_{fg} + c_2)}{(\mu_f^2 + \mu_g^2 + c_1)(\sigma_f^2 + \sigma_g^2 + c_2)} \quad (5.11)$$

O SSIM avalia janelas locais da imagem com base em três componentes complementares: **luminância** (μ_f, μ_g), **contraste** (σ_f, σ_g) e **estrutura** (σ_{fg}), ponderados por constantes de estabilidade c_1 e c_2 . O índice varia no intervalo $[-1, 1]$, onde a unidade representa a identidade perfeita. Ao contrário do PSNR, o SSIM considera a organização espacial dos erros, alinhando-se à percepção do sistema visual humano (SVH).

i PSNR vs SSIM: Aplicação de Métricas Perceptuais

O PSNR possui formulação matemática simples e baixo custo computacional, contudo, tende a superestimar a qualidade em imagens com distorções localizadas ou subestimá-la em variações globais de brilho toleradas pelo observador. O SSIM modela com maior fidelidade a percepção biológica, mas exige maior esforço de processamento. Para análises rigorosas de codificadores, recomenda-se reportar ambas as métricas estatísticas em caráter complementar.

5.8.3 Inspeção Visual: Natureza dos Artefatos de Compressão

A natureza matemática do codificador dita o tipo de degradação introduzida em taxas de bits reduzidas. Conforme ilustrado na Figure 5.27, a compressão agressiva via DCT no padrão JPEG segmenta a imagem em malhas rígidas, gerando os **artefatos de bloco** (*blocking artifacts*). Em contrapartida, algoritmos baseados em codificação preditiva ou representações submetidas a transformadas espaciais avançadas (como o WebP e o JPEG 2000) eliminam as descontinuidades de bloco, mas introduzem perda de textura fina e borramentos característicos ao redor de bordas de alto contraste.

```
import os
import cv2

# Garante a existência do diretório e grava os arquivos comprimidos
os.makedirs("imagens/comp_test", exist_ok=True)
cv2.imwrite("imagens/comp_test/camera_q10.jpg", img_gray, [cv2.IMWRITE_JPEG_QUALITY, 10])
cv2.imwrite("imagens/comp_test/camera_q10.webp", img_gray, [cv2.IMWRITE_WEBP_QUALITY, 10])

# Extração de região de interesse para visualização de artefatos (Zoom de 4x)
zoom_original = cv2.resize(img_gray[120:200, 150:230], (320, 320),
                           interpolation=cv2.INTER_NEAREST)

rec_jpeg = cv2.imread("imagens/comp_test/camera_q10.jpg", cv2.IMREAD_GRAYSCALE)
zoom_jpeg = cv2.resize(rec_jpeg[120:200, 150:230], (320, 320),
                      interpolation=cv2.INTER_NEAREST)

rec_webp = cv2.imread("imagens/comp_test/camera_q10.webp", cv2.IMREAD_GRAYSCALE)
zoom_webp = cv2.resize(rec_webp[120:200, 150:230], (320, 320),
                      interpolation=cv2.INTER_NEAREST)

mm.show([zoom_original, zoom_jpeg, zoom_webp],
        titles=["Zoom Original", "JPEG Q=10 (Artefato de Bloco)", "WebP Q=10 (Suavização)"],
        cols=3, figsize=(14, 5))
```




Figura 5.27: Análise comparativa de artefatos de compressão sob fator de qualidade reduzido ($Q = 10$). À esquerda, observa-se o artefato de bloco característico da discretização por DCT no JPEG. À direita, evidencia-se o efeito de atenuação e suavização de bordas intrínseco ao padrão WebP.

5.8.4 Avaliação Quantitativa e Espacial da Compressão

A validação dos algoritmos de compressão com perda exige uma análise que correlacione o custo de armazenamento à fidelidade do sinal reconstruído. Essa avaliação é realizada de forma complementar através de curvas de desempenho global e pelo mapeamento local das distorções induzidas pelos codificadores.

5.8.4.1 Curvas de Taxa-Distorção

A Figure 5.28 apresenta a avaliação empírica do *pipeline* JPEG e WebP por meio de **curvas de taxa-distorção**, que monitoram o ganho de compressão (tamanho do arquivo em KB) em função do PSNR. O formato PNG atua como linha de base ideal ($\text{PSNR} = \infty$), pois sua natureza *lossless* impede qualquer degradação, embora demande um volume de dados substancialmente maior.

A análise das curvas demonstra a superioridade e a eficiência do padrão WebP sobre o JPEG tradicional: para atingir um mesmo patamar de fidelidade matemática (como a faixa de excelente qualidade, onde $\text{PSNR} > 40$ dB), o codificador WebP gera arquivos significativamente menores. Esse comportamento traduz o impacto prático da evolução dos algoritmos na otimização de sistemas de transmissão e armazenamento digital.

```
import os
import cv2
import matplotlib.pyplot as plt

# Garante a existência do diretório de testes
os.makedirs("imagens/comp_test", exist_ok=True)
resultados = []

# JPEG
for q in [10, 20, 30, 40, 50, 60, 70, 80, 90, 95]:
    path = f"imagens/comp_test/camera_q{q}.jpg"
    cv2.imwrite(path, img_gray, [cv2.IMWRITE_JPEG_QUALITY, q])
    rec = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    resultados.append({"formato": "JPEG", "qualidade": q,
                      "PSNR": cv2.PSNR(img_gray, rec),
                      "KB": os.path.getsize(path)/1024})

# PNG
path_png = "imagens/comp_test/camera.png"
cv2.imwrite(path_png, img_gray, [cv2.IMWRITE_PNG_COMPRESSION, 9])
```



```

resultados.append({"formato": "PNG", "qualidade": "lossless",
                  "PSNR": float('inf'), "KB": os.path.getsize(path_png)/1024})

# WebP
for q in [50, 75, 90]:
    path_w = f"imagens/comp_test/camera_q{q}.webp"
    cv2.imwrite(path_w, img_gray, [cv2.IMWRITE_WEBP_QUALITY, q])
    rec_w = cv2.imread(path_w, cv2.IMREAD_GRAYSCALE)
    resultados.append(
        {"formato": "WebP", "qualidade": q,
         "PSNR": cv2.PSUB_VAL if 'cv2.PSNR' in globals() else cv2.PSNR(img_gray, rec_w),
         "KB": os.path.getsize(path_w)/1024})

# Geração da Curva Taxa-Distorção
jpeg_r = [r for r in resultados if r["formato"]=="JPEG"]
webp_r = [r for r in resultados if r["formato"]=="WebP"]
png_r = [r for r in resultados if r["formato"]=="PNG"]

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot([r["KB"] for r in jpeg_r], [r["PSNR"] for r in jpeg_r],
        "o-", label="JPEG", color="#D85A30", lw=2, ms=5)
ax.plot([r["KB"] for r in webp_r], [r["PSNR"] for r in webp_r],
        "s-", label="WebP", color="#534AB7", lw=2, ms=5)
ax.axhline(50, color="#1D9E75", lw=2, ls="--",
           label=f"PNG sem perda ({png_r[0]['KB']:.1f} KB)")
ax.axhspan(40, 60, alpha=0.05, color="#1D9E75", label="Qualidade excelente (PSNR>40)")
ax.set(xlabel="Tamanho do arquivo (KB)", ylabel="PSNR (dB)",
       title="Curva Taxa-Distorção: JPEG × WebP × PNG")
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"\nTamanho bruto (sem compressão): {img_gray.nbytes/1024:.0f} KB")
print(f"\n{'Formato':>8} {'Qual.':>6} {'KB':>7} {'PSNR (dB)':>11}")
print("-"*38)
for r in resultados:
    psnr_s = f"{r['PSNR']:>11.2f}" if r['PSNR']!=float('inf') else f"∞ (lossless)':>11}"
    print(f"{r['formato']:>8} {str(r['qualidade']):>6} {r['KB']:>7.1f} {psnr_s}")

```

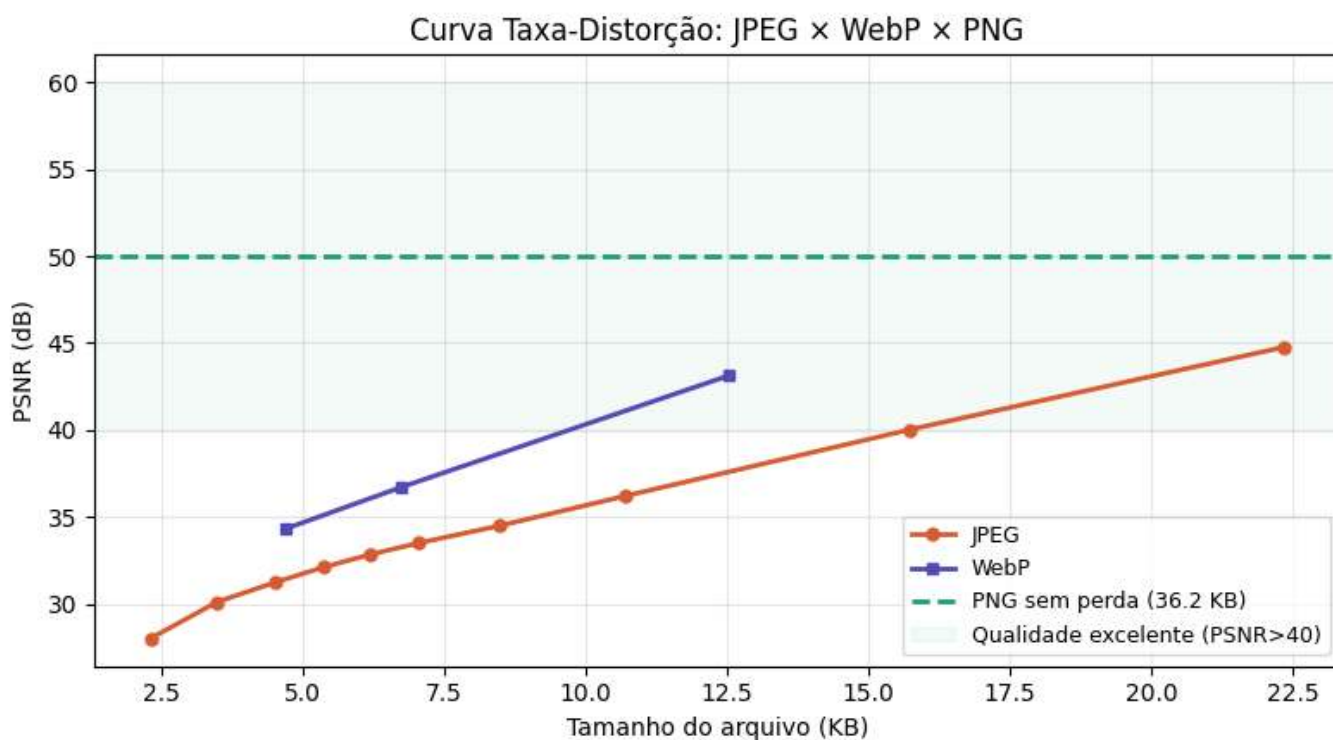


Figura 5.28: Curva taxa-distorção: PSNR vs tamanho de arquivo para JPEG, WebP e PNG aplicada à imagem do *Cameraman*.

Tamanho bruto (sem compressão): 64 KB

Formato	Qual.	KB	PSNR (dB)
JPEG	10	2.3	28.00
JPEG	20	3.5	30.09
JPEG	30	4.5	31.23
JPEG	40	5.4	32.10
JPEG	50	6.2	32.81
JPEG	60	7.0	33.49
JPEG	70	8.5	34.48
JPEG	80	10.7	36.19
JPEG	90	15.7	40.02
JPEG	95	22.3	44.77
PNG lossless	-	36.2	50.00 (lossless)
WebP	50	4.7	34.29
WebP	75	6.7	36.69
WebP	90	12.5	43.14

i Tamanho original da imagem

A imagem *Cameraman* (256×256 pixels em escala de cinza) ocupa **64 KB** em formato bruto (sem compressão). Como referência, o PNG *lossless* comprime esse volume para **36,2 KB** — evidenciando que a compressão sem perdas já reduz significativamente o armazenamento para imagens com regiões homogêneas. Em contrapartida, os formatos com perda (JPEG e WebP) atingem tamanhos ainda menores: o JPEG com qualidade 95 ocupa 22,3 KB (PSNR 45 dB), enquanto o WebP com qualidade 90 atinge 12,5 KB com PSNR equivalente, demonstrando sua superioridade em eficiência de compressão.

5.8.4.2 Mapeamento Espacial de Erros e Correlação Perceptual

Embora o PSNR ofereça um indicativo numérico rápido, métricas globais falham em discriminar como a perda de informação se distribui geometricamente sobre a imagem. A Figure 5.29 soluciona essa limitação ao associar as reconstruções em diferentes qualidades aos seus respectivos mapas de erro absoluto e ao SSIM.

Os mapas residuais — obtidos pela diferença absoluta normalizada entre a imagem original e a comprimida — revelam a assinatura espacial intrínseca de cada arquitetura de codificação:

- **Em altas qualidades ($Q = 95$ a $Q = 75$):** As distorções concentram-se predominantemente ao redor de transições abruptas de intensidade (bordas), fruto do espelhamento espectral decorrente do descarte de altas frequências. O índice SSIM permanece próximo à unidade, atestando a integridade das estruturas originais.
- **Em qualidades agressivas ($Q = 50$ a $Q = 25$):** O erro assume uma estrutura de malha ortogonal regularizada. Esse padrão geométrico evidencia o surgimento dos **artefatos de bloco** (*blocking artifacts*), indicando que a quantização severa corrompeu a correlação espacial entre blocos adjacentes de 8×8 pixels.

O SSIM captura essa degradação morfológica de forma muito mais sensível que o PSNR, penalizando o escore final à medida que a organização estrutural e as texturas finas — às quais o sistema visual humano é altamente responsivo — são eliminadas pelo codificador.

```
import os
import numpy as np
import cv2

try:
    from skimage.metrics import structural_similarity as ssim
except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "scikit-image", "-q"])
    from skimage.metrics import structural_similarity as ssim

# Garante a existência do diretório de testes
os.makedirs("imagens/comp_test", exist_ok=True)

qualidades_ssim = [25, 50, 75, 95]
imgs_ssim = [img_gray]
titles_ssim = ["Original"]

for q in qualidades_ssim:
    path = f"imagens/comp_test/camera_ssim_q{q}.jpg"

    # GRAVAÇÃO FORÇADA: Gera e grava o JPEG com a qualidade atual no caminho correto
    img_compactada = jpeg_quality_compress(img_gray, qualidade=q)
    cv2.imwrite(path, img_compactada)

    # Leitura segura do arquivo recém-gravado
    rec = cv2.imread(path, cv2.IMREAD_GRAYSCALE)

    if rec is None:
        continue

    if rec.shape != img_gray.shape:
        rec = cv2.resize(rec, (img_gray.shape[1], img_gray.shape[0]))

    psnr_v = cv2.PSNR(img_gray, rec)
    ssim_v, _ = ssim(img_gray, rec, full=True)

    # Diferença absoluta normalizada para evidenciar a estrutura espacial do erro
```

```
diff_vis = cv2.normalize(np.abs(img_gray.astype(float) - rec.astype(float)),
                        None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

imgs_ssim += [rec, diff_vis]
titles_ssim += [f"Q={q}\nPSNR={psnr_v:.1f}dB | SSIM={ssim_v:.3f}",
               f"Mapa de erro (Q={q})\n(Bordas e blocagem)"]

mm.show(imgs_ssim, titles=titles_ssim, cols=3, figsize=(14, 14))
```

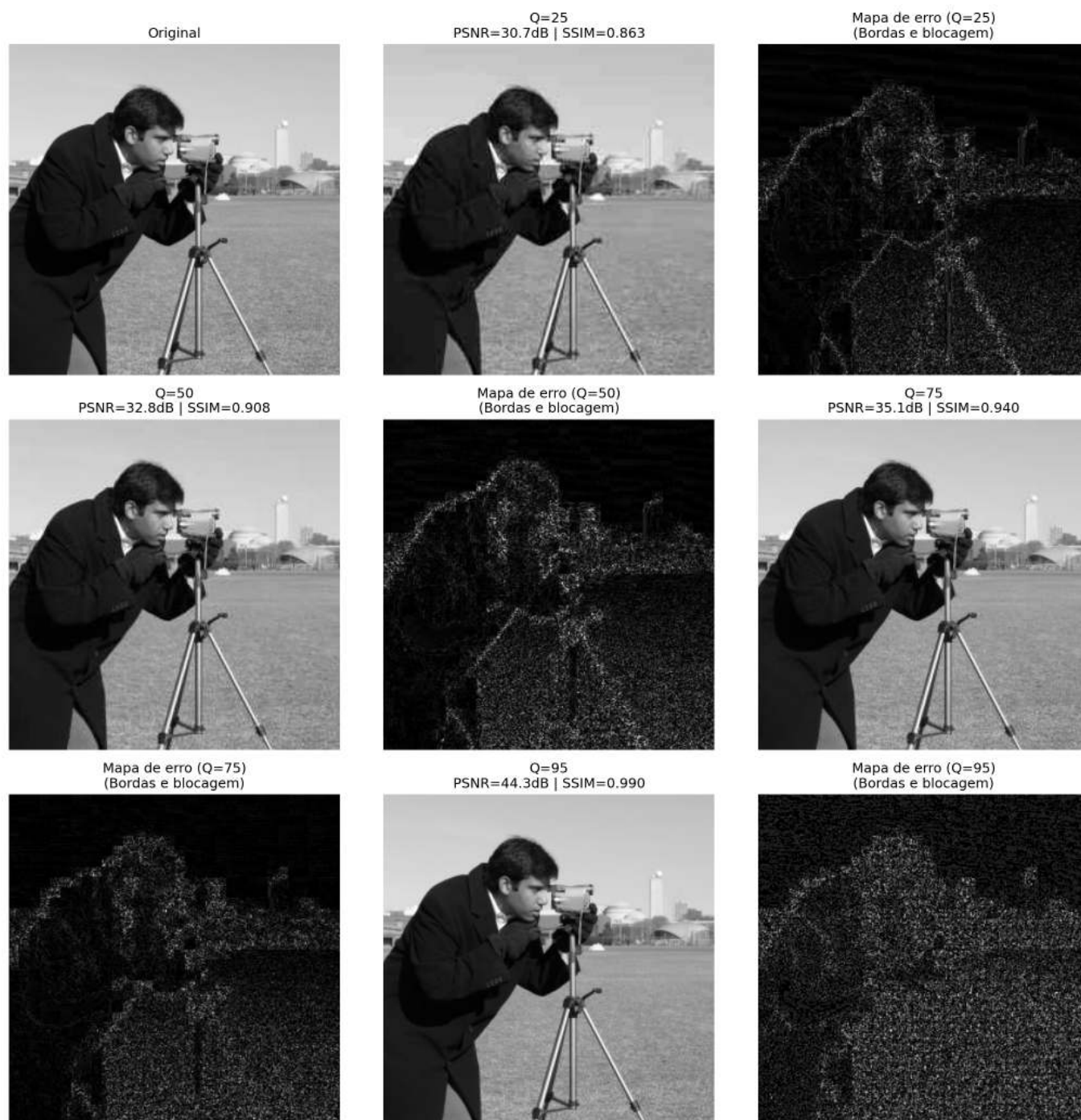


Figura 5.29: Análise espacial de degradação: imagens reconstruídas e respectivos mapas de erro absoluto normalizados para diferentes fatores de qualidade JPEG.

i Interpretando os mapas de erro

Os mapas de erro apresentados foram **normalizados individualmente** (`cv2.NORM_MINMAX`) para maximizar o contraste visual e revelar a estrutura espacial das distorções. Isso significa que:

- Em **Q=95**, o erro absoluto é da ordem de **0.5–1.5 níveis de cinza** (imperceptível visualmente), mas a normalização o amplifica para preto-branco para evidenciar sua localização em bordas e transições.
- Em **Q=25**, o erro absoluto é **10–20 vezes maior** (5–15 níveis de cinza), mas a normalização também o leva ao mesmo intervalo [0, 255].

Portanto, **a intensidade do branco nos mapas NÃO é comparável entre diferentes qualidades** — os mapas servem apenas para revelar a **assinatura espacial** do erro (bordas vs blocos), não sua magnitude. A magnitude correta é dada pelos valores de PSNR e SSIM, que mostram claramente que Q=95 tem erro muito menor que Q=25.

Síntese — Compressão JPEG

O processo de compressão no padrão JPEG baseia-se na aplicação combinada de transformações espaciais, perceptuais e estatísticas para reduzir as redundâncias de uma imagem. A Table 5.9 resume o papel de cada etapa no *pipeline* e seu respectivo impacto na redução de dados.

Tabela 5.9: Síntese das etapas do *pipeline* de compressão JPEG e seus respectivos impactos.

Etapa	Operação Analítica	Mecanismo de Ganho / Compressão
Conversão YC_bC_r	Isolamento dos canais de luminância e crominância.	Modela a percepção do SVH, permitindo tratar cor e brilho de forma independente.
Subamostragem 4:2:0	Redução da resolução espacial dos canais de cor (C_b e C_r).	Elimina aproximadamente 50% dos dados brutos com impacto visual mínimo.
DCT 8×8	Mapeamento do domínio espacial para o domínio de frequências espaciais.	Compactação de energia, concentrando a informação vital nos primeiros coeficientes.
Quantização Linear	Divisão inteira dos coeficientes por uma matriz de ponderação $Q(u, v)$.	Principal fonte de compressão com perda; elimina altas frequências imperceptíveis.
Codificação Entrópica	Aplicação de algoritmos RLE e codificação de Huffman.	Compressão estatística sem perda, otimizada pelas longas corridas de coeficientes nulos.

Artefatos de Degradação Característicos

A aplicação de taxas de compressão excessivamente agressivas (fatores de qualidade reduzidos) introduz distorções previsíveis na imagem reconstruída, decorrentes das limitações matemáticas do modelo:

- **Artefatos de bloco (*blocking artifacts*):** Descontinuidades geométricas visíveis nas fronteiras dos blocos de 8×8 pixels, causadas pela perda de correlação espacial após a quantização severa das componentes AC.
- **Efeito de espalhamento (*ringing*):** Oscilações fantasmas ou distorções de “fumaça” ao redor de bordas nítidas e de alto contraste, provocadas pela eliminação abrupta de harmônicos de alta frequência necessários para reconstruir funções degrau.
- **Perda de textura fina:** Atenuação de detalhes de alta frequência e baixo contraste (como gramados, tecidos ou porosidade), fazendo com que regiões originalmente texturizadas assumam um aspecto excessivamente liso ou homogeneizado.

5.9 Aplicação Prática: Remoção de Ruído por Filtragem Híbrida

Reunindo as técnicas consolidadas ao longo deste capítulo, apresenta-se um *pipeline* completo de **restauração de imagens** que combina a análise espectral no domínio da frequência com a filtragem adaptativa no domínio espacial. O objetivo é atenuar um ruído misto (composto por degradação Gaussiana e interferência periódica) preservando ao máximo os detalhes estruturais da imagem original.

Imagem Ruidosa $\xrightarrow{\text{FFT2}}$ $\xrightarrow{\text{Filtro Notch Gaussiano}}$ $\xrightarrow{\text{IFFT2}}$ $\xrightarrow{\text{Filtro Bilateral}}$ Imagem Restaurada

i Avaliação Complementar: PSNR vs. SSIM

O par de métricas estatísticas PSNR e SSIM fornece uma avaliação qualitativa e morfológica complementar do processo de restauração:

- **PSNR:** Penaliza uniformemente o desvio quadrático médio pixel a pixel.
- **SSIM:** Avalia a preservação de estruturas locais perceptualmente relevantes (luminância, contraste e contornos).

Na prática, existe um compromisso analítico (*trade-off*) entre **redução de ruído** e **preservação de detalhes**: filtros espaciais excessivamente agressivos atenuam bem o ruído de alta frequência, mas degradam texturas finas e suavizam bordas nítidas — o que **reduz simultaneamente** tanto o PSNR quanto o SSIM em relação à imagem original. O desafio do projeto de filtros é encontrar o ponto de equilíbrio que maximize ambas as métricas, garantindo uma restauração fiel e visualmente agradável.

5.9.1 Análise de Desempenho e Conclusão do Capítulo

Os resultados numéricos e visuais gerados pela Figure 5.30 demonstram a relevância prática de associar diferentes domínios de processamento. A inserção simultânea de ruído periódico e estocástico corrompe as propriedades morfológicas do sinal, reduzindo severamente os índices de similaridade e a relação sinal-ruído da imagem de referência.

O isolamento e a supressão dos picos harmônicos no domínio da frequência por meio da máscara *notch* removem as franjas de interferência senoidais espalhadas sobre o espaço bi-dimensional. Como evidenciado nos dados impressos da Figure 5.30, essa filtragem cirúrgica promove um salto imediato e substancial na métrica PSNR. Contudo, o ruído Gaussiano de alta frequência permanece ativo de forma homogênea no espectro, exigindo uma abordagem complementar.

A restauração final é consolidada no domínio espacial com a introdução do filtro bilateral. Diferentemente de operadores passa-baixas convencionais (como o Gaussiano ou de média), que suavizariam indiscriminadamente o ruído e os contornos estruturais, a filtragem bilateral calcula pesos ponderados pela proximidade geométrica e pela diferença de intensidade radiométrica. Esse comportamento adaptativo atenua as flutuações estocásticas remanescentes nas regiões de transição suave e preserva a nitidez das bordas espaciais.

A convergência de ambas as abordagens resulta em uma **melhoria substancial e simultânea** do PSNR e do SSIM em relação à imagem ruidosa — embora os valores finais permaneçam inferiores aos da imagem original (PSNR = ∞ , SSIM = 1,0), devido à perda inevitável de informações espectrais e texturais durante os processos de filtragem. A atenuação suave (gaussiana) dos picos no espectro evita artefatos de *ringing*, enquanto o filtro bilateral elimina o ruído estocástico residual sem comprometer a nitidez das bordas. Os resultados comprovam a eficácia e a complementaridade prática das ferramentas de análise de frequência apresentadas neste capítulo, demonstrando que a filtragem híbrida (frequência + espacial) é superior a qualquer abordagem isolada para a restauração de imagens degradadas por ruído misto.

```
import numpy as np
import cv2

try:
    from skimage.metrics import structural_similarity as ssim_sk
```

```

except ImportError:
    import subprocess
    subprocess.run(["pip", "install", "scikit-image", "-q"])
    from skimage.metrics import structural_similarity as ssim_sk

def suprimir_pico_gaussiano(mask, cy, cx, sigma=3.0):
    yy, xx = np.ogrid[:mask.shape[0], :mask.shape[1]]
    dist = np.sqrt((yy - cy)**2 + (xx - cx)**2)
    notch = np.exp(-dist**2 / (2 * sigma**2))
    mask *= (1 - notch)
    return mask

# 1. Construção do ruído misto
np.random.seed(42)
h_img, w_img = img_gray.shape
X2, Y2 = np.meshgrid(np.arange(w_img), np.arange(h_img))

u0, v0 = 15, 10
ruído_gauss = np.random.normal(0, 15, img_gray.shape)
ruído_period = 30 * np.sin(2 * np.pi * (u0 * X2 / w_img + v0 * Y2 / h_img))
img_noisy = np.clip(img_gray.astype(float) +
                    ruído_gauss + ruído_period, 0, 255).astype(np.uint8)

# 2. Espectro e identificação dos picos
F_n = np.fft.fftshift(np.fft.fft2(img_noisy.astype(np.float64)))
mag_n = cv2.normalize(np.log1p(np.abs(F_n)), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# 3. Notch gaussiano nos picos periódicos
cy0, cx0 = h_img // 2, w_img // 2
mascara_notch = np.ones((h_img, w_img), dtype=np.float64)
for dy, dx in [(+v0, +u0), (-v0, -u0), (+v0, -u0), (-v0, +u0)]:
    mascara_notch = suprimir_pico_gaussiano(mascara_notch, cy0 + dy, cx0 + dx, sigma=3.0)

img_notch = np.real(np.fft.ifft2(np.fft.ifftshift(F_n * mascara_notch)))
img_notch = np.clip(img_notch, 0, 255).astype(np.uint8)

# 4. Filtro Bilateral: remoção do ruído gaussiano residual
img_den = cv2.bilateralFilter(img_notch, d=7, sigmaColor=25, sigmaSpace=7)

# Cálculo das Métricas de Validação
psnr_n, ssim_n = cv2.PSNR(img_gray, img_noisy), ssim_sk(img_gray, img_noisy)
psnr_no, ssim_no = cv2.PSNR(img_gray, img_notch), ssim_sk(img_gray, img_notch)
psnr_d, ssim_d = cv2.PSNR(img_gray, img_den), ssim_sk(img_gray, img_den)

print(f"{'Etapa':>20} | {'PSNR (dB)':>9} | {'SSIM':>6}")
print("-" * 42)
print(f"{'Ruidosa (gauss+per)':>20} | {psnr_n:>9.2f} | {ssim_n:>6.4f}")
print(f"{'Após notch':>20} | {psnr_no:>9.2f} | {ssim_no:>6.4f}")
print(f"{'Notch + bilateral':>20} | {psnr_d:>9.2f} | {ssim_d:>6.4f}")

mascara_vis = (mascara_notch * 255).astype(np.uint8)
mm.show(
    [img_gray, img_noisy, mag_n, mascara_vis, img_notch, img_den],
    titles=[
        "Original",
        f"Ruidosa\nPSNR={psnr_n:.1f} dB",
        "Espectro\n(picos visíveis)",
        "Máscara notch\n(gaussiana suave)",
        f"Após notch\nPSNR={psnr_no:.1f} dB",
        f"Notch + bilateral\nPSNR={psnr_d:.1f} dB SSIM={ssim_d:.3f}"
    ]
)

```



```
],
cols=6, figsize=(20, 4)
)
```

Etapa	PSNR (dB)	SSIM
Ruidosa (gauss+per)	20.19	0.3486
Após notch	24.47	0.4715
Notch + bilateral	28.93	0.7426

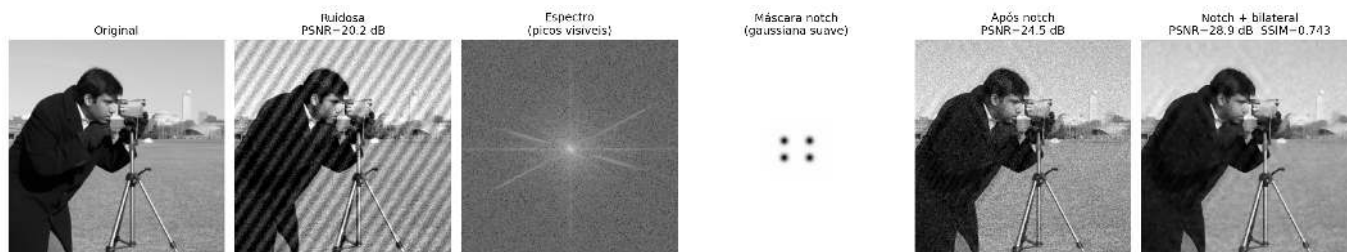


Figura 5.30: *Pipeline* completo de remoção de ruído misto: (1) adição de ruído gaussiano e periódico; (2) identificação de picos de interferência no espectro de frequências; (3) aplicação de máscara *notch* com atenuação gaussiana suave; (4) pós-processamento via filtro bilateral para eliminação do ruído estocástico residual.

5.10 Resumo do Capítulo

A transição do **domínio espacial** para o **domínio da frequência** revela a distribuição espectral de energia da imagem, estabelecendo a base analítica para a filtragem avançada, restauração e compressão de dados. A articulação estrutural desses conceitos é sintetizada no mapa conceitual da Figure 5.31.

Fundamentos Essenciais

- **DFT e Percepção Visual:** O espectro decompõe a imagem em componentes harmônicas. A **fase** retém a inteligibilidade geométrica da cena e a localização de contornos, enquanto a **magnitude** dita a distribuição de contraste e amplitudes globais.
- **Eficiência Algorítmica:** O Teorema da Convolução viabiliza o processamento de máscaras de grande escala no domínio da frequência via FFT, reduzindo a complexidade computacional assintótica de $O(N^2K^2)$ no espaço para $O(N^2 \log N)$.
- **Fenômeno de *Ringíng*:** Cortes abruptos no espectro (Filtros Ideais) geram oscilações espaciais indesejadas (fenômeno de Gibbs). A atenuação suave por filtros de **Butterworth** ou **Gaussianos** elimina essas discontinuidades.
- **Análise Multirresolução via *Wavelets*:** Superando o caráter puramente global de Fourier, a DWT captura a frequência e a localização espacial simultaneamente, fundamentando o padrão JPEG 2000 e subsidiando representações hierárquicas análogas às extrações de feições em Redes Neurais Convolucionais (CNNs).
- **Compressão Perceptual (DCT):** O *pipeline* JPEG explora as limitações de contraste do sistema visual humano em altas frequências espaciais. A DCT isola a energia de blocos 8×8 , permitindo que a quantização descarte coeficientes AC de detalhes finos sem prejuízo perceptual severo.

Próximos Passos: O Capítulo 6 inaugura a Parte II da obra, aplicando as ferramentas de processamento de imagens na resolução de problemas reais de inspeção industrial. Serão exploradas técnicas de **segmentação e análise de formas** para a detecção automática de falhas em linhas de produção — desde a identificação de defeitos superficiais em peças até a leitura *QRCode* em provas, consolidando a ponte entre a teoria apresentada na Parte I e as demandas práticas da visão computacional.

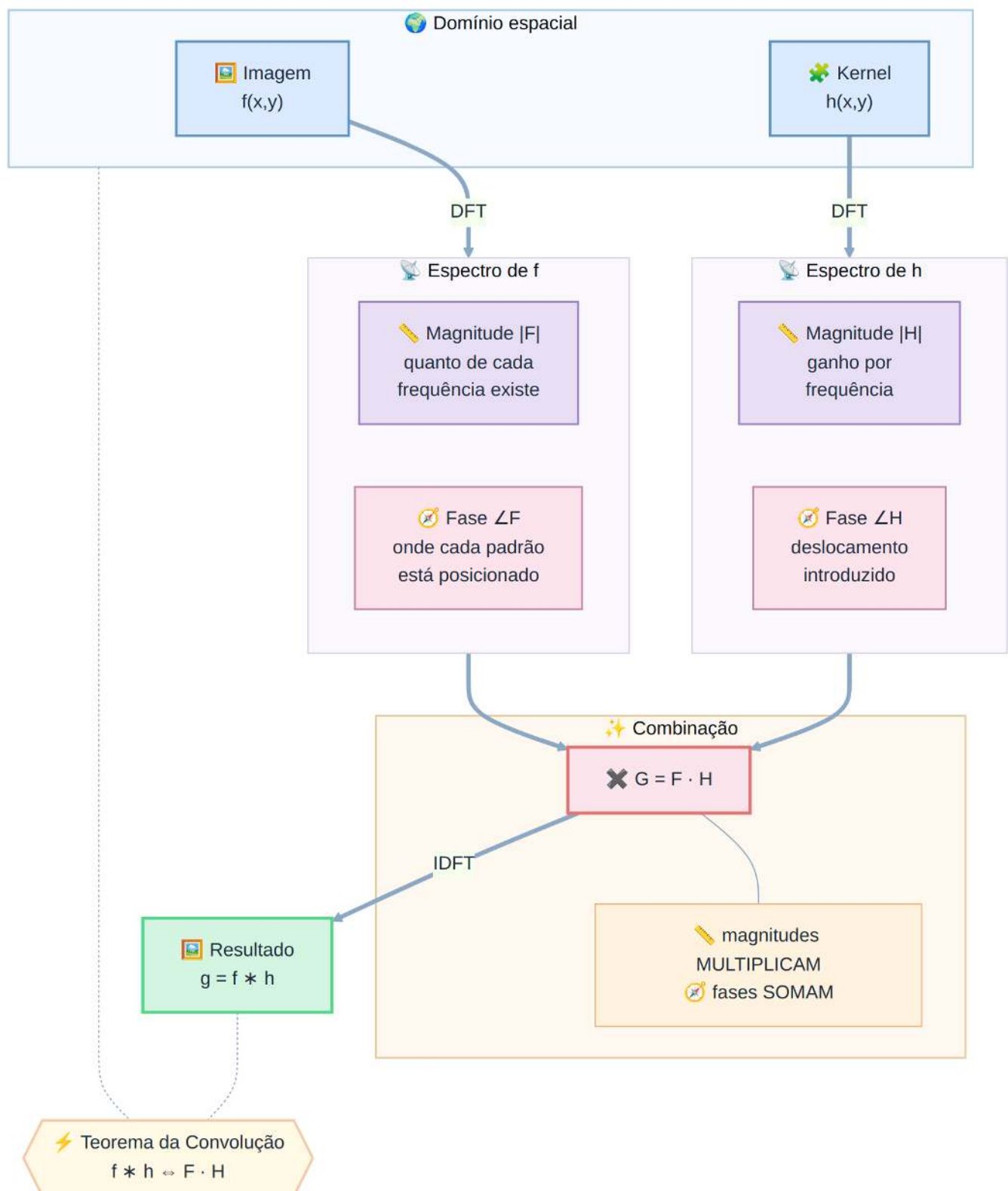


Figura 5.31: Mapa conceitual das transformações e propriedades no domínio da frequência.

5.11 Uso do NotebookLM como Tutor Complementar

Nesta edição, incentiva-se o uso da plataforma **NotebookLM** como ferramenta complementar de aprendizagem — **não como substituta** da leitura atenta, da resolução de exercícios ou da experimentação prática. Baseado em arquiteturas de inteligência artificial, o sistema utiliza exclusivamente o material didático e os documentos fornecidos pelo autor como base de conhecimento, assegurando que as respostas geradas estejam conceitualmente alinhadas ao conteúdo programático e à abordagem pedagógica adotada ao longo desta obra.

Acesso ao Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 05](#)

Diretrizes sobre o Conteúdo Gerado por Inteligência Artificial

Embora as ferramentas de inteligência artificial constituam aliados eficientes no processo de aprendizagem e revisão, o conteúdo gerado está sujeito a inconsistências ou imprecisões técnicas. Desse modo, é indispensável a consulta sistemática a livros-texto, artigos científicos e fontes acadêmicas indexadas para a validação rigorosa das informações. Recomenda-se veementemente a execução e a modificação dos exemplos práticos em Python fornecidos neste capítulo como método primário de verificação experimental dos resultados.

5.12 Lista de Exercícios

1. **(10%) Implementação Direta da DFT 2D:** Implemente analiticamente a Transformada Discreta de Fourier 2D (DFT) sem o auxílio de funções nativas de bibliotecas (como `np.fft.fft2`), utilizando estritamente a formulação matemática definida na Equation 5.1 para uma matriz de dimensões 16×16 . Realize a validação numérica comparando os coeficientes gerados com os resultados da função `np.fft.fft2`, certificando-se de que o desvio absoluto máximo seja inferior a 10^{-8} . Mensure os tempos de execução de ambos os métodos e apresente uma justificativa teórica para a disparidade observada em termos de complexidade assintótica.
2. **(15%) Supressão de Ruído Periódico:** Adicione interferências senoidais com frequências espaciais $(u_0, v_0) \in \{(5, 10), (20, 5), (30, 30)\}$ à imagem de teste do *Cameraman*. Para cada cenário de degradação, projete uma máscara de filtragem *notch* específica no domínio da frequência para isolar e atenuar os picos harmônicos indesejados. Avalie quantitativamente a eficácia do processo de restauração por meio do cálculo das métricas de PSNR e SSIM. Discuta analiticamente o compromisso (*trade-off*) entre a atenuação do ruído senoidal e a indesejada atenuação de feições estruturais legítimas da imagem.
3. **(15%) Análise Comparativa de Operadores Passa-Baixa:** Realize um estudo comparativo entre os filtros passa-baixa Ideal, Gaussiano e Butterworth (com ordens harmônicas $n = 1, 2, 4$), parametrizados com frequências de corte $D_0 = 20, 40, 60$ pixels. Para cada combinação estrutural, calcule os índices PSNR e SSIM da imagem resultante frente ao sinal original de referência. Organize os dados quantitativos em uma tabela estruturada e plote os gráficos unidimensionais das funções de transferência correspondentes ao longo do perfil horizontal $H(u, 0)$.
4. **(15%) Banco de Filtros Multirresolução de Haar:** Desenvolva um script para executar manualmente a decomposição *wavelet* discreta 2D de primeiro nível utilizando a família Haar. O algoritmo deve calcular os coeficientes dos filtros correspondentes passa-baixa (h) e passa-alta (g), aplicando-os de forma separável sobre as linhas e colunas da matriz, seguidos pela operação de decimação (subamostragem espacial por um fator de 2). Valide numericamente a exatidão da sua implementação confrontando as subbandas obtidas com a saída da função `pywt.dwt2(img, 'haar')`.
5. **(15%) Compressão Esparsa por Limiarização Wavelet:** Aplique a técnica de filtragem por limiarização abrupta (*hard thresholding*) sobre os coeficientes de detalhe da decomposição *wavelet*, adotando os limiares numéricos $T \in \{5, 10, 20, 40, 80\}$ para as famílias Haar, Daubechies (`db4`) e Symlets

- (`sym4`). Após realizar o processo de síntese por meio da transformada inversa (`pywt.waverec2`), compute os valores de PSNR e SSIM de cada imagem reconstruída. Identifique e justifique qual combinação de família *wavelet* e limiar T maximiza a similaridade estrutural.
6. **(15%) Construção de Codificador JPEG Simplificado:** Implemente o pipeline completo de compressão de dados simulando o padrão JPEG. O fluxo deve englobar: conversão espacial $RGB \rightarrow YC_bC_r$, subamostragem cromática na proporção 4:2:0, segmentação da luminância em blocos disjuntos de 8×8 pixels, aplicação da DCT-II 2D ortogonal, e quantização linear baseada na matriz normalizada de luminância escalada por fatores de qualidade desejados. Realize a decodificação inversa e compare quantitativamente as reconstruções com os arquivos gerados pela função `cv2.imencode` para os fatores de qualidade de 20, 50 e 80.
7. **(15%) Análise Perceptual em Conteúdos Heterogêneos:** Desenvolva uma imagem sintética composta por três regiões distintas e de características espectrais contrastantes: uma textura fotográfica complexa (representando altas frequências estocásticas), uma área de texto vetorizado com bordas nítidas (representando transições degrau puras) e um gradiente linear contínuo (representando baixas frequências homogêneas). Submeta essa imagem mista aos processos de compressão sob os formatos JPEG, PNG e WebP. Avalie e interprete os resultados correlacionando o tamanho final do arquivo em disco às métricas PSNR e SSIM obtidas, justificando qual formato exibe o melhor desempenho para sinais de natureza heterogênea e por que essa vantagem ocorre em termos de compactação de energia e preservação perceptual.

Referências do Capítulo

A fundamentação teórica e o desenvolvimento analítico dos conceitos abordados neste capítulo fundamentam-se nas seguintes obras de referência:

- **Gonzalez; Woods (2018)** — Formulações clássicas de Transformadas Discretas de Fourier 2D (DFT), projeto de filtros analíticos no domínio da frequência, Transformada Discreta de Cossenos (DCT) e princípios fundamentais de sistemas de compressão de imagens.
- **Oppenheim; Schaffer (2010)** — Teoria formal de sinais e sistemas aplicados no domínio discreto, cobrindo as propriedades matemáticas da DFT e a modelagem analítica do Teorema da Convolução.
- **Mallat (1999)** — Fundamentação matemática da teoria de *wavelets*, formalização da análise multirresolução (MRA) e arquitetura de bancos de filtros diádicos.
- **Wallace (1991)** — Especificação original e aspectos de engenharia do padrão de compressão ISO/IEC JPEG, com ênfase nos critérios psicovisuais para o projeto de matrizes de quantização DCT.
- **Szeliski (2022)** — Modelagem computacional e caracterização de métricas modernas de fidelidade e qualidade perceptual (PSNR e SSIM), bem como a análise comparativa de formatos de imagem rasterizados de alto desempenho.



5.13 Parte Prática com Exercícios de Programação

Em construção!

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 5 — Transformadas e Compressão — por meio de uma trilha prática aplicada. Os exercícios são estruturados a partir de matrizes de dimensões reduzidas, viabilizando a validação analítica e a inspeção manual de cada coeficiente, mantendo a consistência metodológica adotada nos capítulos anteriores.

O encadeamento dos exercícios reproduz rigorosamente o fluxo conceitual do capítulo: inicia-se com a implementação explícita da Transformada Discreta de Fourier (DFT) a partir de sua definição matemática fundamental; avança-se para o projeto de filtros passa-baixa e máscaras *notch* no domínio da frequência;

aplica-se a quantização de coeficientes (núcleo da compressão com perda); e conclui-se com a integração dessas etapas na construção de um *pipeline* de compressão JPEG simplificado e na análise perceptual de formatos de imagem.

! Diretrizes para a Resolução dos Exercícios de Programação

Em todos os exercícios deste capítulo, as coordenadas do **centro do espectro** (origem das frequências espaciais pós-aplicação do deslocamento `fftshift`) devem ser determinadas via divisão inteira. Para uma matriz com L linhas e C colunas, a componente de frequência nula localiza-se na posição:

$$(c_y, c_x) = \left(\left\lfloor \frac{L}{2} \right\rfloor, \left\lfloor \frac{C}{2} \right\rfloor \right)$$

Esta convenção é rigorosamente idêntica à adotada pela função `np.fft.fftshift`. Ademais, em todas as etapas que exijam discretização ou arredondamento numérico (seja na quantização de coeficientes AC ou na reconstrução final de pixels), deve-se empregar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*), mitigando ambiguidades em valores com fração exatamente igual a 0.5.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para rodar os testes, execute `TestSuite("EP05_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
```

"""

TestSuite("EP05_01").run_code(codigo)

5.13.1 EP05_01 Filtro Passa-Baixa Ideal por Distância no Espectro

Em um *scanner* de documentos antigo, o sensor capta papel amassado e textura de fibra junto com o texto — ruído de alta frequência que “polui” o espectro nas bordas. O técnico de manutenção não tem acesso à imagem original, apenas ao **espectro de magnitude já calculado** pelo software do *scanner*. Seu trabalho é simples e cirúrgico: manter apenas o **círculo central** de baixas frequências (a estrutura global do documento) e apagar tudo que estiver fora do raio D_0 , eliminando a textura fina sem nem precisar tocar na imagem espacial.

Este é o **Filtro Passa-Baixa Ideal (LPFI)**: a operação espectral mais direta do capítulo, mas também a que melhor revela a anatomia de um espectro centrado.

5.13.1.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas) do espectro de magnitude — já fornecido **centrado** (equivalente à saída de `np.fft.fftshift`).
2. **Frequência de corte:** Ler o inteiro D_0 .
3. **Dados:** Ler os valores inteiros da matriz de magnitude, linha a linha.
4. **Centro do espectro:** Calcular $(c_y, c_x) = (L // 2, C // 2)$.
5. **Distância:** Para cada posição (u, v) , calcular

$$D(u, v) = \sqrt{(u - c_y)^2 + (v - c_x)^2}$$

6. **Máscara ideal:** Aplicar

$$H(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases}$$

7. **Filtragem:** O valor de saída é $\text{mag}'(u, v) = \text{mag}(u, v) \cdot H(u, v)$.
8. **Saída:** Exibir a matriz filtrada com dimensões $L \times C$.

5.13.1.2 Restrições Computacionais

- **Comparação não estrita:** o critério usa $D(u, v) \leq D_0$ (a fronteira pertence ao filtro, ou seja, é mantida).
- **Tipo:** todos os valores de entrada e saída são inteiros; a distância é calculada em ponto flutuante apenas internamente.
- **Sem arredondamento de magnitude:** como a entrada já é inteira e a máscara é binária (0 ou 1), a saída nunca precisa de arredondamento.

5.13.1.3 Fundamentação Teórica

Região	Distância ao centro	Efeito do filtro
Centro ($D \leq D_0$)	Baixas frequências	Preservadas — estrutura global mantida
Bordas ($D > D_0$)	Altas frequências	Zeradas — textura e ruído removidos
D_0 pequeno	—	Imagem reconstruída ficaria muito borrada
D_0 grande	—	Pouca filtragem; quase toda energia preservada

5.13.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro D_0 .
- Linhas seguintes: Elementos inteiros da matriz de magnitude (centrada).

Saída:

- Matriz filtrada em L linhas e C colunas, separados por espaço.

5.13.1.5 Exemplos

Entrada	Saída	Observação
3	0 20 0	Centro (1, 1). Cantos têm $D = \sqrt{2} \approx 1.41 > 1$, logo são zerados; vizinhos ortogonais têm $D = 1 \leq 1$ e são mantidos.
3	40 50 60	
1	0 80 0	
10 20 30		
40 50 60		
70 80 90		
1	0 9 0	$L = 1, C = 3$: centro em (0, 1). Apenas a própria posição central ($D = 0$) sobrevive a $D_0 = 0$.
3		
0		
5 9 7		

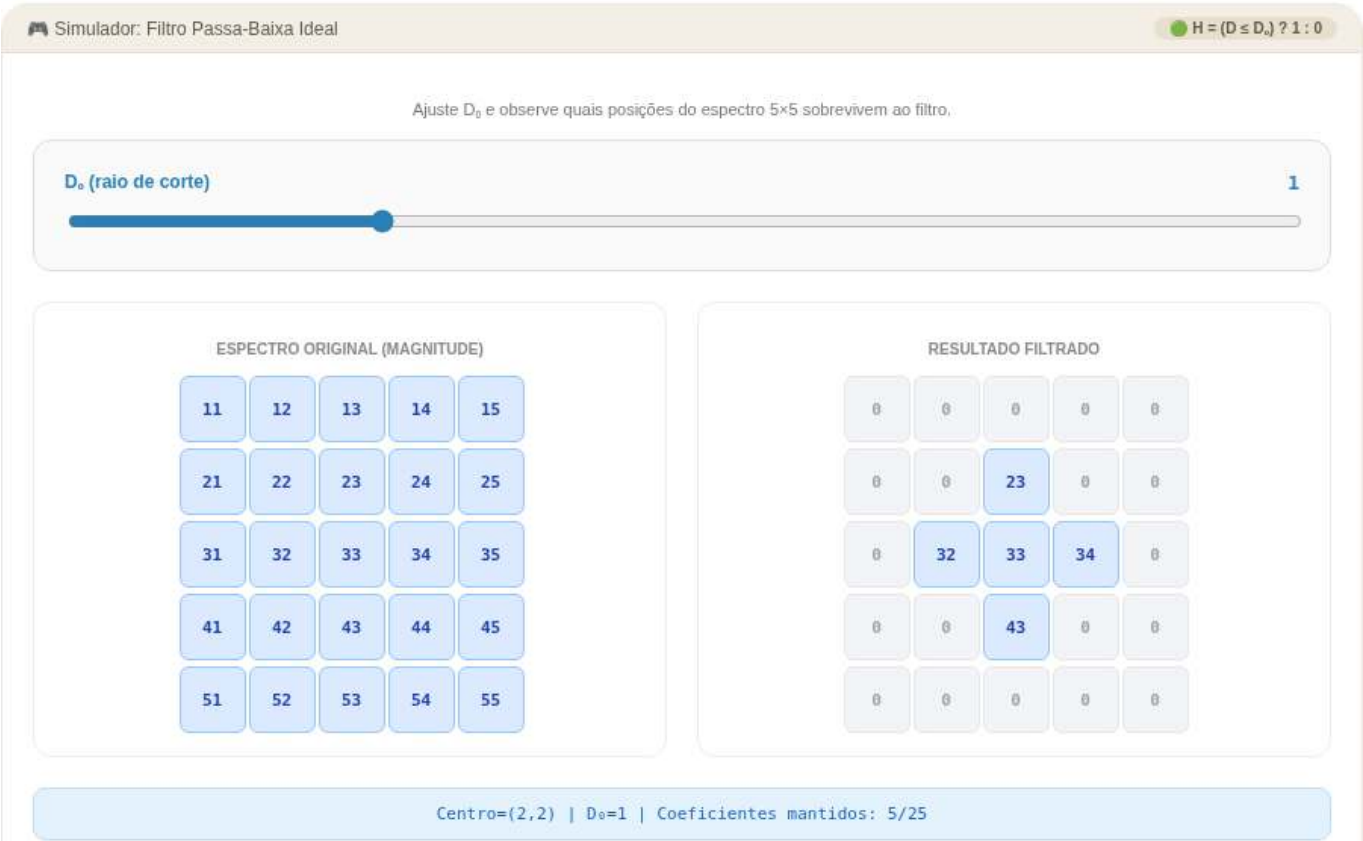


Figura 5.32: Simulador: Filtro Passa-Baixa Ideal no Espectro


```
%%writefile EP05_01.py
# Código Python
```

```
Overwriting EP05_01.py
```

```
TestSuite("EP05_01.py").run()
```

5.13.2 EP05_02 Filtro *Notch*: Removendo Picos Periódicos

Uma câmera de **inspeção industrial** captura imagens de placas de circuito, mas a fonte de alimentação da linha de produção introduz uma **interferência elétrica periódica** — um padrão de listras quase imperceptível a olho nu, mas que aparece no espectro de Fourier como **pares de picos brilhantes** simetricamente posicionados em torno do centro. A equipe de visão computacional não pode reprocessar a captura: precisa **localizar e apagar cirurgicamente** esses pares de picos no espectro, preservando todo o resto da informação útil da imagem.

Esse é o papel do **filtro rejeita-banda *notch***: diferente do passa-baixa (que afeta uma região contínua), ele ataca **pontos específicos e seus simétricos**, deixando o restante do espectro intocado.

5.13.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas) do espectro de magnitude centrado.
2. **Dados:** Ler os valores inteiros da matriz de magnitude, linha a linha.
3. **Picos:** Ler o inteiro K (quantidade de pares de picos a remover).
4. **Para cada um dos K picos:** ler três inteiros Δv , Δu , r — deslocamento vertical, deslocamento horizontal e raio do *notch*.
5. **Centro do espectro:** $(c_y, c_x) = (L // 2, C // 2)$.
6. **Supressão simétrica:** para cada pico, zerar **todas** as posições (u, v) tais que a distância ao ponto $(c_y + \Delta v, c_x + \Delta u)$ for $\leq r$, e **também** todas as posições com distância $\leq r$ ao ponto simétrico $(c_y - \Delta v, c_x - \Delta u)$.
7. **Saída:** Exibir a matriz resultante com dimensões $L \times C$.

5.13.2.2 Restrições Computacionais

- **Simetria obrigatória:** cada pico informado gera **dois** discos zerados (o ponto e seu simétrico em relação ao centro) — esquecer o simétrico é o erro mais comum.
- **Sobreposição:** se dois discos se sobrepõem, a posição permanece zerada (não há “soma” ou restauração).
- **Comparação não estrita:** uma posição é zerada se distância $\leq r$.
- **Ordem de leitura:** os K picos devem ser processados na ordem em que aparecem na entrada, mas o resultado final independe da ordem (operações de zerar são comutativas).

5.13.2.3 Fundamentação Teórica

Conceito	Papel no filtro <i>notch</i>
Pico em $(\Delta v, \Delta u)$	Frequência da interferência periódica detectada visualmente no espectro
Ponto simétrico $(-\Delta v, -\Delta u)$	Toda DFT de sinal real é hermitiana: picos sempre aparecem em pares simétricos ao centro
Raio r	Controla a “largura” da rejeição — r grande remove mais energia ao redor do pico, mas também informação útil

5.13.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz de magnitude (centrada), L linhas.
- Próxima linha: Inteiro K .
- K linhas seguintes: três inteiros Δv , Δu , r (separados por espaço).

Saída:

- Matriz resultante em L linhas e C colunas, separados por espaço.

5.13.2.5 Exemplos

Entrada	Saída	Observação
5	1 2 3 4 5	Centro $(c_y, c_x) = (2, 2)$. Pico informado $(\Delta v, \Delta u) = (1, 1)$ gera o ponto $(3, 3)$ (valor 19) e seu simétrico $(1, 1)$ (valor 7), ambos zerados com $r = 0$ (apenas os pontos exatos).
5	6 0 8 9 10	
1 2 3 4 5	11 12 13 14 15	
6 7 8 9 10	16 17 18 0 20	
11 12 13 14 15	21 22 23 24 25	
16 17 18 19 20		
21 22 23 24 25		
1		
1 1 0		



Figura 5.33: Simulador: Filtro Notch

```
%%writefile EP05_02.py
# Código Python
```

```
Overwriting EP05_02.py
```

```
TestSuite("EP05_02.py").run()
```

5.13.3 EP05_03 ● Quantização DCT: a Verdadeira Fonte de Compressão

Um aplicativo de **galeria de fotos** precisa reduzir o tamanho de milhares de imagens antes de fazer *upload* para a nuvem, sem recodificar tudo do zero. O engenheiro responsável já tem os **coeficientes DCT** de cada bloco 4×4 calculados (a etapa cara computacionalmente já foi feita) — falta apenas aplicar a **tabela de quantização**, a etapa que realmente descarta informação e gera compressão. Coeficientes de alta frequência, menos perceptíveis ao olho humano, recebem divisores grandes e tendem a virar **zero**; coeficientes de baixa frequência, mais perceptíveis, recebem divisores pequenos e sobrevivem quase intactos.

Você vai implementar exatamente essa etapa: **quantizar e desquantizar** (dividir, arredondar, multiplicar de volta) — o coração da compressão *lossy* do JPEG.

5.13.3.1 📋 Diretrizes de Implementação

1. **Dimensão do bloco:** Ler o inteiro N (bloco $N \times N$).
2. **Coeficientes:** Ler a matriz C de coeficientes DCT, N linhas com N inteiros cada (podem ser negativos).
3. **Tabela de quantização:** Ler a matriz Q , N linhas com N inteiros positivos cada.
4. **Quantização:** Para cada posição (u, v) , calcular o índice quantizado

$$\tilde{C}(u, v) = \text{round}\left(\frac{C(u, v)}{Q(u, v)}\right)$$

usando arredondamento padrão para o inteiro mais próximo (valores intermediários .5 nunca ocorrem nos casos de teste).

5. **Desquantização (reconstrução):** Calcular

$$C'(u, v) = \tilde{C}(u, v) \times Q(u, v)$$

6. **Saída:** Exibir a matriz reconstruída C' , $N \times N$, inteiros.

5.13.3.2 📌 Restrições Computacionais

- **Round-trip completo:** a saída é o coeficiente **reconstruído** ($\tilde{C} \times Q$), não o índice quantizado isolado.
- **Divisão em ponto flutuante:** a divisão $C(u, v)/Q(u, v)$ deve ser feita em ponto flutuante antes do arredondamento — divisão inteira truncada produzirá resultado incorreto.
- **Sinal preservado:** coeficientes negativos mantêm o sinal após quantização e reconstrução.
- $Q(u, v) > 0$ **sempre:** não há necessidade de tratar divisão por zero.

5.13.3.3 🧠 Fundamentação Teórica

Coeficiente	Frequência	Valor típico de Q	Efeito da quantização
$C(0, 0)$	DC (média do bloco)	Pequeno	Quase sempre sobrevive — domina a energia
$C(u, v)$ baixo $u + v$	Baixa frequência	Pequeno/médio	Parcialmente preservado
$C(u, v)$ alto $u + v$	Alta frequência	Grande	Frequentemente vira zero — fonte da compressão

5.13.3.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- N linhas seguintes: matriz C (coeficientes DCT, inteiros, podem ser negativos).
- N linhas seguintes: matriz Q (tabela de quantização, inteiros positivos).

Saída:

- Matriz reconstruída C' , $N \times N$, inteiros separados por espaço.

5.13.3.5 📌 Exemplos

Entrada	Saída	Observação
4	50 10 -7 0	$C(0,0) = 50/2 = 25 \rightarrow 25 \times 2 = 50$
50 10 -5 0	8 0 0 0	(preservado). $C(0,2) = -5/7 \approx$
8 -3 2 1	0 0 0 0	$-0.71 \rightarrow -1 \rightarrow -1 \times 7 = -7$. Já
0 1 0 0	0 0 0 0	$C(1,1) = -3/7 \approx -0.43 \rightarrow 0$:
2 0 0 -1		zerado pela quantização — a
2 5 7 8		maior parte do bloco vira zero,
4 7 8 11		ilustrando a compactação de
6 8 11 12		energia no canto superior
9 11 12 14		esquerdo.

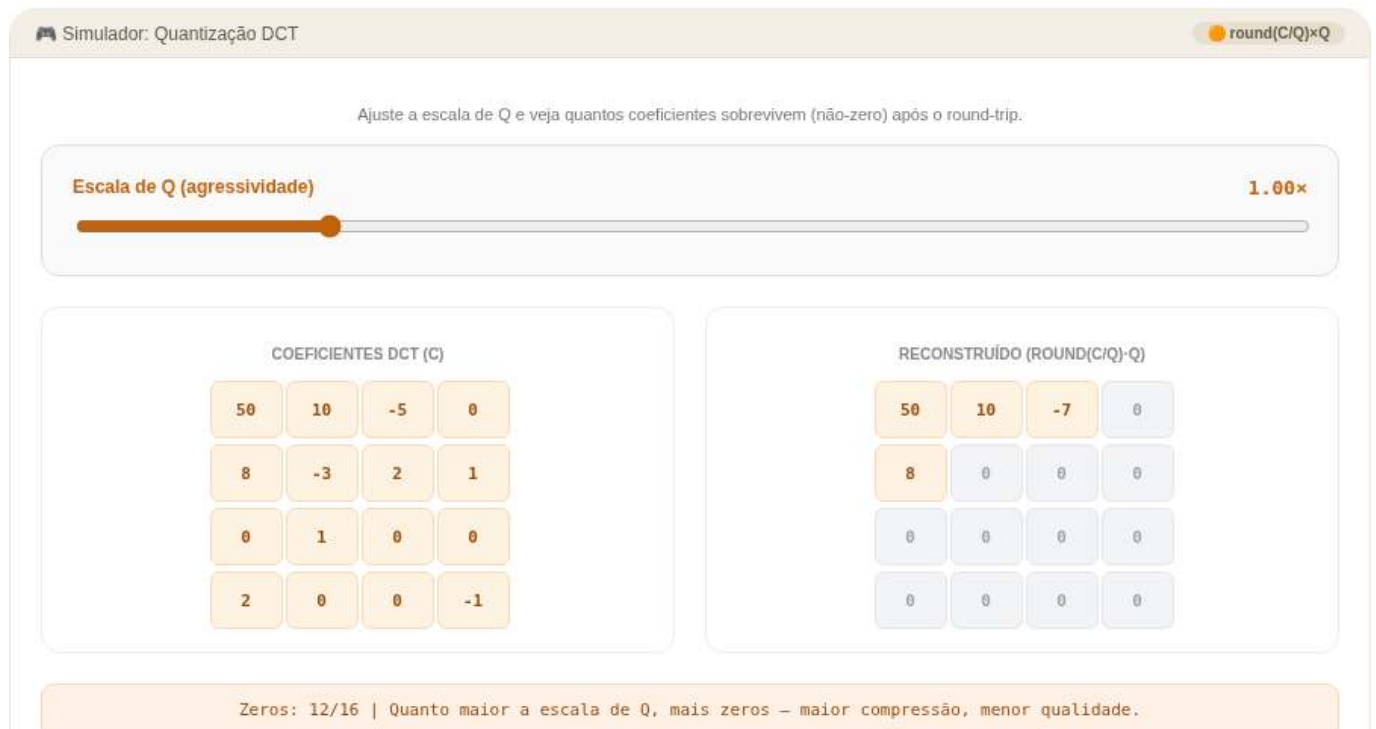


Figura 5.34: Simulador: Quantização DCT (round-trip)

```
%writefile EP05_03.py
# Código Python
```

Overwriting EP05_03.py

```
TestSuite("EP05_03.py").run()
```

5.13.4 EP05_04 Implementando a DFT 2D a Partir da Definição

Um laboratório de pesquisa em **astronomia computacional** recebeu, de uma missão antiga, um pequeno sensor experimental cujos dados brutos não podem ser processados por bibliotecas modernas de FFT — o ambiente de validação é isolado e só permite operações aritméticas básicas. A equipe precisa **reimplementar a Transformada de Fourier Discreta 2D a partir da própria definição matemática**, célula por célula, para depois comparar bit a bit com `np.fft.fft2` em outro ambiente.

Este é o exercício mais conceitual da lista: não há atalhos. Você vai implementar o duplo somatório da Equation 5.1 diretamente, evidenciando *por que* a FFT existe — e o custo computacional que ela evita.

5.13.4.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros M (linhas) e N (colunas) da imagem $f(x, y)$.
2. **Dados:** Ler os valores inteiros de $f(x, y)$, linha a linha.
3. **DFT 2D:** Para cada par de frequências (u, v) com $u = 0, \dots, M - 1$ e $v = 0, \dots, N - 1$, calcular

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

usando a identidade de Euler $e^{-j\theta} = \cos(\theta) - j\sin(\theta)$ para separar parte real e imaginária — **não utilize nenhuma função de FFT pronta**.

4. **Magnitude:** Calcular $|F(u, v)| = \sqrt{\text{Re}(F)^2 + \text{Im}(F)^2}$ e arredondar para o inteiro mais próximo.
5. **Saída:** Exibir a matriz de magnitudes arredondadas, $M \times N$, na mesma ordem (sem `fftshift` — o DC permanece em $(0, 0)$).

5.13.4.2 Restrições Computacionais

- **Proibido usar bibliotecas de FFT:** a implementação deve calcular os somatórios duplos explicitamente (laços aninhados), mesmo que mais lenta.
- **Sem `fftshift`:** a saída mantém a convenção crua da DFT, com o componente DC em $F(0, 0)$ (canto superior esquerdo).
- **Arredondamento:** a magnitude final deve ser arredondada para o inteiro mais próximo; nos casos de teste não há ambiguidade .5.
- **Precisão:** pequenos erros de ponto flutuante (ordem de 10^{-6}) antes do arredondamento são esperados e não afetam o resultado inteiro final.

5.13.4.3 Fundamentação Teórica

Elemento	Significado
$F(0, 0)$	Componente DC — soma de todos os pixels, $F(0, 0) = \sum f(x, y)$
Parte real $\text{Re}(F)$	Projeção do sinal sobre cossenos
Parte imaginária $\text{Im}(F)$	Projeção do sinal sobre senos
Complexidade desta implementação	$\mathcal{O}((MN)^2)$ — por isso a FFT, com $\mathcal{O}(MN \log(MN))$, é indispensável em imagens reais

5.13.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro M .

- Linha 2: Inteiro N .
- Linhas seguintes: Elementos inteiros de $f(x, y)$, M linhas.

Saída:

- Matriz de magnitudes $|F(u, v)|$ arredondadas, $M \times N$, separadas por espaço.

5.13.4.5 📌 Exemplos

Entrada	Saída	Observação
2	10 2	$F(0, 0) = 1 + 2 + 3 + 4 = 10$ (DC = soma total). $F(0, 1) = (1 - 2) + (3 - 4) = -2 \rightarrow F = 2$. $F(1, 0) = (1 + 2) - (3 + 4) = -4 \rightarrow F = 4$. $F(1, 1) = (1 - 2) - (3 - 4) = 0$.
2	4 0	
1 2		
3 4		



Figura 5.35: Simulador: DFT 2D manual

```
%%writefile EP05_04.py
# Código Python

Overwriting EP05_04.py

TestSuite("EP05_04.py").run()
```

5.13.5 EP05_05 🏆 Pipeline JPEG Completo: DCT, Quantização e Reconstrução

Você foi contratado para criar, do zero, um **codec JPEG didático** em ambiente embarcado, sem qualquer biblioteca de imagem disponível — apenas operações matemáticas básicas. O cliente quer entender exatamente onde a qualidade é perdida e onde ela é recuperada, bloco por bloco. Este é o desafio final do capítulo: integrar **tudo** o que foi estudado — a DCT-II ortonormal, a quantização perceptual e a reconstrução via IDCT — em um único *pipeline* de ponta a ponta, processando um bloco $N \times N$ do início ao fim, exatamente como o padrão JPEG faz internamente, 8×8 pixels de cada vez.

5.13.5.1 📋 Diretrizes de Implementação

1. **Dimensão do bloco:** Ler o inteiro N .

2. **Bloco original:** Ler a matriz de pixels $f(x, y)$, N linhas com N inteiros em $[0, 255]$.
3. **Tabela de quantização:** Ler a matriz Q , $N \times N$ inteiros positivos.
4. **Centralização:** Subtrair 128 de cada pixel: $g(x, y) = f(x, y) - 128$.
5. **DCT-II 2D ortonormal:** Calcular

$$C(u, v) = \alpha(u) \alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} g(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right]$$

com $\alpha(0) = \sqrt{1/N}$ e $\alpha(k) = \sqrt{2/N}$ para $k > 0$.

6. **Quantização:** $\tilde{C}(u, v) = \text{round}(C(u, v)/Q(u, v))$.
7. **Desquantização:** $C'(u, v) = \tilde{C}(u, v) \times Q(u, v)$.
8. **IDCT-II 2D (inversa ortonormal):** Calcular $g'(x, y)$ a partir de $C'(u, v)$ usando a transformada inversa correspondente (mesma base, somatório sobre u, v).
9. **Reversão da centralização e arredondamento:** $f'(x, y) = \text{round}(g'(x, y) + 128)$, restrito ao intervalo $[0, 255]$ (*clipping*).
10. **Saída:** Exibir o bloco reconstruído f' , $N \times N$, inteiros.

5.13.5.2 Restrições Computacionais

- **Pipeline completo obrigatório:** todas as seis etapas (centralizar, DCT, quantizar, desquantizar, IDCT, reverter) devem ser implementadas — pular a quantização não passa nos testes, pois o resultado seria idêntico ao original.
- **Clipping:** valores reconstruídos fora de $[0, 255]$ devem ser truncados (0 se negativo, 255 se maior que 255).
- **Arredondamento:** tanto na quantização quanto na reconstrução final dos pixels, use arredondamento padrão; os casos de teste evitam ambiguidade .5.
- **Base ortonormal:** a normalização $\alpha(u)$ e $\alpha(v)$ deve ser aplicada exatamente como especificado — sem ela, a IDCT não reconstrói corretamente.

5.13.5.3 Fundamentação Teórica

Etapas	Análoga no padrão JPEG real	Onde a qualidade é perdida
Centralização	Mesma — DCT assume sinal centrado em zero	Nenhuma perda
DCT-II	Etapas 3–4 do <i>pipeline</i> (Table 5.7)	Nenhuma perda (transformação exata e reversível)
Quantização	Etapas 5 — divisão por $Q(u, v)$	Principal fonte de perda — coeficientes de alta frequência viram zero
IDCT	Reconstrução final	Reconstrói exatamente os coeficientes <i>quantizados</i> , não os originais

5.13.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- N linhas seguintes: bloco original $f(x, y)$, inteiros em $[0, 255]$.
- N linhas seguintes: tabela de quantização Q , inteiros positivos.

Saída:

- Bloco reconstruído $f'(x, y)$, $N \times N$, inteiros em $[0, 255]$, separados por espaço.

5.13.5.5 📌 Exemplos

Entrada	Saída	Observação
4	118 126 119 131	Após DCT, quantização agressiva nas altas frequências (valores grandes de Q no canto inferior direito) e reconstrução via IDCT, o bloco fica próximo do original, mas não idêntico — a diferença é o custo da compressão <i>lossy</i> .
120 130 125 128	114 143 140 119	
115 140 135 122	117 149 159 130	
118 150 160 130	107 121 146 139	
110 120 145 138		
4 6 8 10		
6 8 10 12		
8 10 12 16		
10 12 16 20		

5.13.5.6 💡 Dica de Depuração

Se o resultado não bater, verifique nesta ordem: (1) os coeficientes DCT brutos (antes da quantização) — eles devem reconstruir o original **exatamente** via IDCT se você pular a etapa 6–7; (2) a tabela $\alpha(u)$ — erro comum é aplicar $\sqrt{2/N}$ também para $u = 0$; (3) o arredondamento da quantização, que deve ocorrer **antes** de multiplicar de volta por Q .



Figura 5.36: Simulador: *Pipeline* JPEG completo em bloco

```
%%writefile EP05_05.py
# Código Python
```

```
Overwriting EP05_05.py
```

```
TestSuite("EP05_05.py").run()
```

Parte II

Parte II — Visão Computacional

Capítulo 6

Análise de Documentos e Inspeção Industrial



Na **Parte I — Processamento Digital de Imagens (PDI)**, foram estudadas técnicas para transformação e aprimoramento de imagens, como operações morfológicas, filtragem espacial, convoluções, limiarização, segmentação e processamento no domínio da frequência.

A **Parte II — Visão Computacional (VC)** amplia esse escopo ao tratar da interpretação automática do conteúdo visual, envolvendo a extração de informações, o reconhecimento de padrões e a tomada de decisões a partir de imagens.

Este capítulo apresenta essa transição por meio de duas aplicações representativas:

1. **Análise Automatizada de Documentos**, aplicada ao processamento de formulários, avaliações e outros documentos estruturados por meio de sistemas de reconhecimento óptico de marcas (*Optical Mark Recognition* – OMR);
2. **Inspeção Industrial Automatizada**, voltada ao controle de qualidade e à detecção de defeitos em linhas de produção.

Essas aplicações integram técnicas de detecção de estruturas geométricas, extração de descritores invariantes, reconhecimento de padrões e classificação de objetos, constituindo a base de diversos sistemas modernos de inspeção visual e automação.

6.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Aplicar técnicas de **alinhamento e retificação geométrica de documentos** utilizando a Transformada de Hough e transformações projetivas;
- **Detectar e segmentar regiões de interesse** com base em operações morfológicas, contornos e propriedades geométricas;
- **Decodificar marcadores bidimensionais e códigos de barras**, integrando bibliotecas de Visão Computacional a *pipelines* de processamento documental;
- **Implementar sistemas de reconhecimento óptico de marcas (OMR)** para leitura automatizada de avaliações e formulários;
- **Realizar reconhecimento óptico de caracteres (OCR)** para converter documentos digitalizados em texto codificado;
- **Aplicar técnicas de processamento de linguagem natural**, incluindo tradução automática, ao texto obtido por OCR;
- **Avaliar a influência do pré-processamento** na qualidade do reconhecimento automático de informações em documentos;
- **Desenvolver *pipelines* de Visão Computacional** para análise automatizada de documentos.

Este capítulo marca a transição do **Processamento Digital de Imagens**, voltado à transformação de imagens, para a **Visão Computacional**, cujo objetivo é interpretar o conteúdo visual, extrair informações e apoiar processos automatizados de análise e tomada de decisão.

6.2 Configuração do Ambiente

Os exemplos deste capítulo utilizam bibliotecas amplamente empregadas em PDI-VC. O bloco abaixo instala os pacotes necessários; em ambientes que já os possuam, a execução pode ser ignorada.

```
import sys, subprocess, importlib

# 1. Dependências de Sistema (Apenas no Google Colab)
if 'google.colab' in sys.modules:
    print("[AMBIENTE] Google Colab. Configurando dependências do sistema...")
    subprocess.run(
        "apt-get update && apt-get install -y poppler-utils "
        "libzbar0 tesseract-ocr tesseract-ocr-por",
        shell=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
    )

# 2. Dependências do Python (Instala apenas as ausentes)
pkgs = {
    "cv2": "opencv-python", "skimage": "scikit-image", "numpy": "numpy",
    "pdf2image": "pdf2image", "pandas": "pandas", "tabulate": "tabulate",
    "PyPDF2": "PyPDF2", "bcrypt": "bcrypt", "pyarrow": "pyarrow",
    "pyzbar": "pyzbar", "pytesseract": "pytesseract", "deep_translator": "deep-translator"
}
for mod, pkg in pkgs.items():
    if importlib.util.find_spec(mod) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg])

# 3. Imports Globais do Pipeline
import cv2, numpy as np, matplotlib.pyplot as plt
from skimage import io, data, color
```

Além dessas bibliotecas, será utilizado o módulo didático `morph.py`, desenvolvido para simplificar operações de leitura, visualização e processamento de imagens ao longo deste livro. O código a seguir verifica sua disponibilidade, realiza o *download* quando necessário e confirma a versão carregada.

```
import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.2

6.3 Bases de Imagens para Experimentação

Os exemplos apresentados nesta parte do livro utilizam, sempre que possível, documentos digitalizados, folhas de respostas, códigos de barras, *QR Codes* e outras imagens provenientes de aplicações reais. Para tornar os experimentos integralmente reproduzíveis — inclusive em ambientes sem acesso à internet ou aos arquivos originais do MCTest —, também são empregadas imagens públicas amplamente utilizadas no ensino e na pesquisa em Processamento Digital de Imagens e Visão Computacional (PDI-VC).

💡 Por que utilizar um banco de imagens de *benchmark*?

Imagens como `camera()` e `coins()` são empregadas há décadas em livros-texto, artigos científicos e materiais didáticos de PDI-VC. Sua utilização oferece importantes vantagens:

- **Reprodutibilidade:** qualquer leitor obtém exatamente as mesmas imagens, independentemente do computador ou sistema operacional utilizado, sem necessidade de *downloads* externos ou de arquivos específicos deste livro;
- **Comparabilidade:** os resultados produzidos podem ser comparados diretamente com aqueles reportados na literatura, uma vez que os mesmos conjuntos de imagens são amplamente adotados como referência;
- **Foco nos algoritmos:** por serem imagens compactas, bem documentadas e distribuídas sem restrições de uso para fins educacionais e científicos, permitem concentrar a atenção nas técnicas de processamento, reduzindo interferências relacionadas à aquisição ou ao gerenciamento dos dados.

6.3.1 Imagens Públicas com `skimage.data`

O módulo `skimage.data` disponibiliza uma coleção de imagens de referência amplamente utilizada em atividades de ensino, pesquisa e validação de algoritmos em PDI-VC. A inspeção do atributo `data.__all__` mostra que a versão atual reúne **42 itens**, incluindo fotografias naturais, documentos digitalizados, imagens médicas, microscopia, texturas, padrões sintéticos, modelos tridimensionais e sequências temporais. Convém observar que alguns desses itens correspondem a funções utilitárias, como `data_dir()` e `download_all()`, e não a imagens propriamente ditas.

Como o objetivo deste capítulo é apresentar aplicações de análise documental e inspeção visual, a Table 6.2 reúne uma **seleção representativa** das imagens mais relevantes, organizada de acordo com suas principais aplicações em Visão Computacional. A Figure 6.1 apresenta uma amostra dessas imagens, agrupadas conforme a mesma classificação adotada na tabela.

i Imagens utilizadas neste capítulo

Embora o `skimage.data` disponibilize dezenas de imagens de referência, apenas quatro são empregadas diretamente nos experimentos deste capítulo. Elas foram selecionadas por reproduzirem, de forma controlada, características frequentemente encontradas em documentos digitalizados e em sistemas de inspeção visual industrial. A Table 6.1 resume o papel de cada uma delas ao longo deste capítulo.

Tabela 6.1: Imagens do `skimage.data` utilizadas nos experimentos deste capítulo.

Imagem	Aplicação no capítulo
<code>data.page()</code>	Página digitalizada utilizada nos experimentos de correção de iluminação, realce local (CLAHE) e limiarização automática por Otsu.
<code>data.text()</code>	Documento contendo texto impresso, empregado para ilustrar segmentação, extração de contornos e etapas típicas de OCR e OMR.

<code>data.coffee()</code>	Fotografia colorida com variações naturais de iluminação, utilizada para exemplificar técnicas aplicáveis a cenas reais não documentais.
<code>data.brick()</code>	Textura de referência empregada em exemplos de inspeção superficial e detecção de defeitos por análise de variância local.

Tabela 6.2: Seleção de imagens públicas representativas disponíveis no módulo *skimage.data*, organizadas por área de aplicação.

Categoria	Função	Descrição
 Documentos & OCR/OMR	<code>data.page()</code>	Página digitalizada de documento — normalização de fundo, CLAHE e Otsu.
 Documentos & OCR/OMR	<code>data.text()</code>	Texto impresso — segmentação e extração de contornos em cenários de OCR/OMR.
 Segmentação, Morfologia & Contornos	<code>data.coins()</code>	Conjunto de moedas — referência clássica para segmentação e <i>watershed</i> .
 Segmentação, Morfologia & Contornos	<code>data.clock()</code>	Relógio analógico — detecção de formas e contornos.
 Segmentação, Morfologia & Contornos	<code>data.binary_blobs()</code>	Blobs binários sintéticos — conectividade e morfologia matemática.
 Segmentação, Morfologia & Contornos	<code>data.moon()</code>	Superfície lunar — segmentação de crateras por relevo de intensidade.
Textura & Inspeção Industrial	<code>data.brick()</code>	Textura uniforme de tijolos — detecção de defeitos por variância local.
Textura & Inspeção Industrial	<code>data.checkerboard()</code>	Padrão xadrez — calibração de câmera e transformações geométricas.
 Fotografias Clássicas de PDI/VC	<code>data.camera()</code>	Fotógrafo com tripé — imagem de referência mais citada na literatura de PDI.
 Fotografias Clássicas de PDI/VC	<code>data.astronaut()</code>	Retrato colorido de astronauta — filtragem e realce em cor.
 Fotografias Clássicas de PDI/VC	<code>data.coffee()</code>	Xícara de café — cena real com variação de iluminação e cor.
 Fotografias Clássicas de PDI/VC	<code>data.cat()</code> / <code>data.chelsea()</code>	Fotografias coloridas de gatos — detecção de bordas e realce.
 Fotografias Clássicas de PDI/VC	<code>data.horse()</code>	Silhueta binária de cavalo — descritores de forma e contorno.

```
import matplotlib.pyplot as plt
from skimage import data

# Imagens ordenadas por categoria; a cor do título reproduz a cor da categoria na tab. anterior
imgs = [
    ("page",      data.page(),      "#2563eb"),    # Documentos & OCR/OMR
    ("text",      data.text(),      "#2563eb"),
    ("coins",     data.coins(),     "#16a34a"),    # Segmentação & morfologia
    ("binary_blobs", data.binary_blobs(), "#16a34a"),
    ("brick",     data.brick(),     "#ea580c"),    # Textura & inspeção industrial
    ("checkerboard", data.checkerboard(), "#ea580c"),
    ("camera",    data.camera(),    "#7c3aed"),    # Fotografias clássicas de PDI/VC
    ("coffee",   data.coffee(),    "#7c3aed"),
]
```

```
fig, ax = plt.subplots(2, 4, figsize=(11, 5.5))
for a, (nome, img, cor) in zip(ax.ravel(), imgs):
    a.imshow(img, cmap="gray")
    a.set_title(nome, color=cor, fontweight="bold")
    a.axis("off")
plt.tight_layout()
```

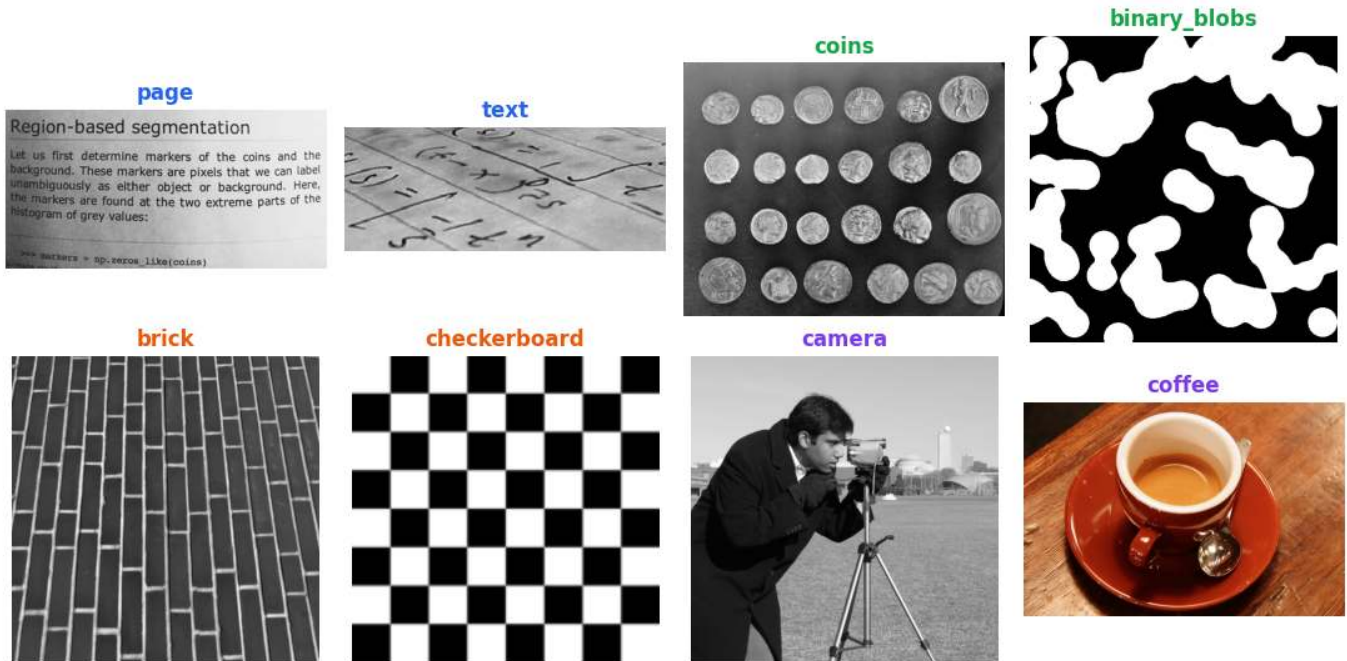


Figura 6.1: Amostra de imagens públicas do *skimage.data*, agrupadas por área de aplicação.

6.4 Fundamentos de OMR e Inspeção Industrial

O **Reconhecimento Óptico de Marcas** (*Optical Mark Recognition* — OMR) é uma técnica de Visão Computacional destinada à identificação automática de marcações em posições previamente definidas de um formulário. Suas aplicações incluem folhas de respostas, questionários, formulários administrativos e outros documentos estruturados.

Diferentemente do OCR (*Optical Character Recognition*), que reconhece caracteres e palavras, o OMR determina a presença, a ausência ou a intensidade de marcas em regiões previamente conhecidas. Em vez de interpretar texto, explora propriedades geométricas e estatísticas associadas ao preenchimento dessas regiões.

Os sistemas modernos de OMR processam imagens obtidas por *scanners*, câmeras ou dispositivos móveis, automatizando tarefas que anteriormente dependiam de equipamentos especializados.

De forma geral, um sistema de OMR compreende as seguintes etapas:

1. **Aquisição:** conversão do documento físico em formato digital;
2. **Pré-processamento:** correção geométrica, redução de ruídos e binarização;
3. **Localização das regiões de interesse:** identificação das áreas destinadas às marcações;
4. **Análise das marcações:** avaliação do preenchimento das regiões candidatas;
5. **Interpretação:** conversão das marcações em respostas ou dados estruturados.

Esses princípios se estendem naturalmente à **Inspeção Industrial Automatizada**. Em linhas de produção, o mesmo encadeamento — aquisição, pré-processamento, segmentação, extração de características e decisão — é empregado para detectar defeitos superficiais, verificar a integridade de componentes e medir dimensões com precisão subpixel. A diferença reside no domínio de aplicação: enquanto o OMR opera sobre documentos

com estrutura predefinida, a inspeção industrial lida com objetos cujas variações geométricas e radiométricas devem ser modeladas de forma mais flexível.

Nas seções seguintes, ambas as aplicações são desenvolvidas por meio de projetos práticos que reproduzem etapas típicas de sistemas reais.

6.5 Projetos Práticos: Construção de um *Pipeline* de Análise Documental

Os conceitos deste capítulo serão desenvolvidos por meio de projetos que reproduzem etapas típicas de sistemas reais de análise documental, introduzindo técnicas reutilizáveis em aplicações de OCR, OMR, inspeção visual e processamento de formulários.

6.5.1 Alinhamento Automático de Documentos (*OCR/OMR Pre-processing*)

A correção de inclinação (*deskew*) é uma etapa fundamental no processamento de documentos. Rotações introduzidas durante a digitalização ou captura comprometem a localização de regiões de interesse e reduzem a precisão das etapas subsequentes.

Neste projeto será desenvolvido um sistema para estimar automaticamente a orientação predominante do documento e corrigir sua inclinação. Para isso, serão empregadas técnicas clássicas de detecção de bordas com o operador de Canny e detecção de retas pela Transformada de Hough. A partir das linhas identificadas, será estimado o ângulo de rotação e aplicada uma transformação afim para produzir uma versão alinhada do documento.

Como formulários e folhas de resposta são frequentemente distribuídos em formato PDF, o *pipeline* inicia-se com a rasterização de cada página, convertendo-a em uma imagem matricial. Neste capítulo, essa etapa será realizada com a biblioteca `pdf2image`, gerando imagens PNG com resolução de 300 DPI (*dots per inch*). A partir delas, poderão ser aplicadas as técnicas de detecção de bordas, Transformada de Hough, segmentação, extração de contornos e reconhecimento automático de padrões estudadas ao longo do capítulo.

```
import os
import urllib.request
from pdf2image import convert_from_path
from skimage import data
import cv2

# Diretório dos microdados e folhas de respostas do exame institucional
file_path = "dados/provas_qrcode_EP.pdf"
url_github = (
    "https://raw.githubusercontent.com/fzampirolli/"
    "pdi-vc/master/all/cap06/dados/provas_qrcode_EP.pdf"
)

# Se o arquivo não existir localmente, baixa automaticamente do GitHub
if not os.path.exists(file_path):
    print(f"[DOWNLOAD] Baixando PDF do GitHub: {url_github}")
    try:
        # Garante que a pasta 'dados' existe antes de salvar
        os.makedirs(os.path.dirname(file_path), exist_ok=True)
        urllib.request.urlretrieve(url_github, file_path)
        print("[DOWNLOAD] PDF baixado com sucesso!")
    except Exception as e:
        print(f"[DOWNLOAD] Falha ao baixar o arquivo: {e}")

print(f"PDF de folhas de prova digitalizadas: {file_path}")

if os.path.exists(file_path):
```

```
# Rasterização das páginas com resolução otimizada de 300 DPI
pages = convert_from_path(file_path, dpi=300)
for i, page in enumerate(pages):
    saida = f"test{i+1:02d}.png"
    page.save(saida)
    print(f"[INGESTÃO] Página PDF convertida com sucesso: {saida}")
else:
    print("[AVISO] Arquivo PDF não localizado no path. Ativando fallback via skimage.data.")
    # Injeta matriz de texto pública para garantir a execução contínua do pipeline
    img_fallback = data.text()
    cv2.imwrite("test02.png", img_fallback)
    print("[INGESTÃO] Imagem de fallback estruturada: test02.png")

# Carrega e exibe a imagem rasterizada inicial utilizando o ecossistema morph
img_original = mm.read('test02.png')
mm.show(img_original, figsize=(4, 3))
```

PDF de folhas de prova digitalizadas: dados/provas_qrcode_EP.pdf

[INGESTÃO] Página PDF convertida com sucesso: test01.png

[INGESTÃO] Página PDF convertida com sucesso: test02.png

[INGESTÃO] Página PDF convertida com sucesso: test03.png

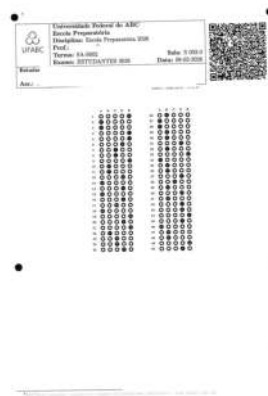


Figura 6.2: *Pipeline* de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.

6.5.2 Algoritmo de Retificação de Inclinação (*Deskew*)

A etapa de *deskew* tem como objetivo estimar e corrigir a inclinação global de um documento digitalizado, alinhando seu conteúdo aos eixos da imagem. A Figure 6.4 apresenta o fluxo completo de processamento, desde a imagem original até o resultado após a correção geométrica. Complementarmente, o simulador da Figure 6.3 permite visualizar o funcionamento da Transformada de Hough e compreender como a orientação predominante é estimada.

O procedimento é composto por três etapas principais:

1. detecção de bordas pelo operador de Canny;
2. estimativa da orientação predominante por meio da Transformada de Hough Linear;
3. correção da inclinação utilizando uma transformação afim de rotação.

Após a retificação, o documento passa a apresentar orientação aproximadamente horizontal, favorecendo as etapas subsequentes de segmentação, rotulagem de componentes conexos e reconhecimento de caracteres e marcas.

6.5.3 Modelagem Matemática

Esta seção descreve os fundamentos matemáticos das três etapas que compõem o processo de retificação: detecção de bordas, estimação da orientação predominante e correção geométrica.

6.5.3.1 Detecção de Bordas

Inicialmente, a imagem é suavizada por um filtro Gaussiano, reduzindo o efeito de ruídos de alta frequência que podem gerar bordas espúrias. Os conceitos de filtragem espacial e convolução foram apresentados no **Capítulo 3**.

Em seguida, o operador de Canny estima o gradiente da imagem. Seja $f(x, y)$ a intensidade da imagem e α o ângulo de rotação.

A magnitude do gradiente é dada por

$$|\nabla f(x, y)| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}.$$

em que:

- $f(x, y)$ representa a intensidade da imagem na posição (x, y) ;
- $\frac{\partial f}{\partial x}$ e $\frac{\partial f}{\partial y}$ são as derivadas parciais nas direções horizontal e vertical;
- $|\nabla f(x, y)|$ é a magnitude do gradiente.

Após o cálculo do gradiente, o algoritmo aplica a supressão de não máximos (*non-maximum suppression*) e a limiarização por histerese, produzindo uma imagem binária contendo as principais bordas do documento.

6.5.3.2 Transformada de Hough

A imagem binária de bordas é processada pela Transformada de Hough Linear, cujo objetivo é detectar estruturas aproximadamente retilíneas. Em vez da representação cartesiana da reta, $y = ax + b$, utiliza-se a representação na forma normal,

$$\rho = x \cos \theta + y \sin \theta,$$

em que:

- x e y são as coordenadas de um ponto pertencente à reta;
- ρ é a distância perpendicular entre a reta e a origem do sistema de coordenadas da imagem;
- θ é o ângulo formado entre a normal à reta e o eixo horizontal da imagem.

Nessa representação, cada ponto de borda (x, y) gera uma curva no espaço de parâmetros (ρ, θ) . A interseção das curvas produzidas por pontos pertencentes à mesma reta origina máximos em uma matriz bidimensional denominada **acumulador**. Assim, os picos do acumulador correspondem às retas predominantes da imagem, como bordas do documento, linhas de formulários ou linhas de texto.

Para estimar a inclinação global do documento, consideram-se apenas as retas cujos ângulos satisfazem

$$-45^\circ \leq \theta \leq 45^\circ.$$

Essa restrição elimina orientações incompatíveis com a disposição esperada do documento e reduz a influência de retas verticais ou de estruturas irrelevantes. Seja $\theta_1, \theta_2, \dots, \theta_n$ o conjunto dos ângulos das retas selecionadas. A estimativa da inclinação global é obtida pela mediana,

$$\hat{\theta} = \text{med}(\theta_1, \theta_2, \dots, \theta_n),$$

em que:

- θ_i é o ângulo da i -ésima reta detectada pela Transformada de Hough;
- n é o número de retas consideradas após a filtragem angular;
- $\hat{\theta}$ é a estimativa da inclinação global do documento.

A mediana é adotada por ser menos sensível à presença de detecções isoladas (*outliers*) do que a média aritmética, produzindo uma estimativa mais estável da orientação predominante.

6.5.3.3 Rotação Afim

Seja $\hat{\theta}$ a inclinação estimada na etapa anterior. A correção geométrica consiste em aplicar uma transformação afim de rotação em torno do centro da imagem, de modo que a orientação predominante passe a coincidir com o eixo horizontal. As transformações afins foram estudadas no **Capítulo 2**, juntamente com as operações de translação, escala, cisalhamento e rotação.

Seja (x, y) a posição de um pixel em relação ao centro da imagem e (x', y') sua posição após a rotação. A transformação é descrita por

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\alpha) \begin{bmatrix} x \\ y \end{bmatrix},$$

em que

$$R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix},$$

sendo:

- (x, y) as coordenadas originais do pixel em relação ao centro da imagem;
- (x', y') as coordenadas do pixel após a rotação;
- α o ângulo de rotação aplicado para compensar a inclinação estimada do documento;
- $R(\alpha)$ a matriz de rotação.

Na prática, o ângulo aplicado corresponde ao oposto da inclinação estimada,

$$\alpha = -\hat{\theta},$$

em que $\hat{\theta}$ representa a orientação predominante obtida pela Transformada de Hough.

Como as coordenadas transformadas nem sempre coincidem com posições inteiras da malha de pixels, é necessário reamostrar a imagem para determinar os novos valores de intensidade. Na implementação apresentada neste capítulo, a função `mm.rotate` realiza essa operação utilizando interpolação bicúbica, reduzindo os artefatos de reamostramento e preservando a continuidade visual de bordas e caracteres.

```
def retificar_inclinacao_documento(img):

    gray = mm.gray(img) if img.ndim == 3 else img
    edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
    lines = cv2.HoughLines(edges, 1, np.pi/180, 200)

    if lines is None:
        return edges, img

    angulos = []
    for line in lines:
        angulo = np.rad2deg(line[0][1]) - 90
        if -45 < angulo < 45:
            angulos.append(angulo)
```



Figura 6.3: Simulador interativo de correção de inclinação (*deskew*): mova o *slider* para inclinar o documento e observe as três etapas do *pipeline* — imagem inclinada, bordas Canny e resultado corrigido.

```

if not angulos:
    return edges, img

return edges, mm.rotate(img, np.median(angulos), interp="bicubic")

# Execução do pipeline de deskew
img_edges, img_final = retificar_inclinacao_documento(img_original)

# Exibição múltipla padronizada com o formato nativo do livro
mm.show(
    [img_original, img_edges, img_final],
    titles=["Imagem Original", "Bordas de Canny", "Documento Retificado"],
    cols=3,
    figsize=(12, 4)
)

```

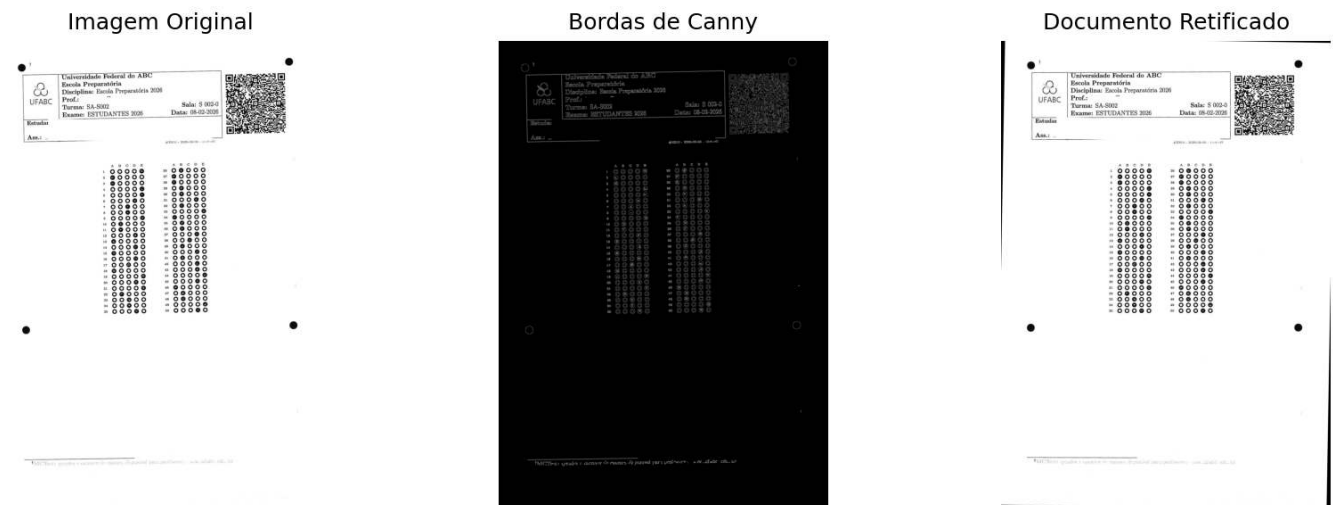


Figura 6.4: *Pipeline* de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.

Por que funciona? — Transformada de Hough

Na Transformada de Hough, cada pixel de borda contribui com votos para todas as retas que podem passar por sua posição. Em vez de selecionar apenas a reta com maior número de votos, o algoritmo considera todas as retas cuja quantidade de votos excede um limiar mínimo e calcula seus respectivos ângulos. A inclinação global do documento é então estimada pela mediana desses ângulos, uma medida robusta a valores discrepantes. Assim, retas espúrias produzidas por sombras, ruídos ou outros elementos da imagem exercem pouca influência sobre a estimativa final, desde que a maioria das retas detectadas corresponda às bordas do documento.

6.5.4 Limitações Práticas

Embora apresente bom desempenho em condições usuais de digitalização, o método depende da existência de estruturas lineares suficientemente definidas para serem detectadas pela Transformada de Hough, como bordas da página, linhas de formulários ou linhas de texto. Sua precisão pode ser reduzida em imagens com baixa resolução, ruído excessivo, sombras intensas ou grandes inclinações. Em geral, documentos digitalizados com resolução próxima de 300 DPI e iluminação homogênea fornecem resultados adequados para aplicações de OCR e OMR.

A implementação apresentada neste capítulo possui finalidade didática, ilustrando os princípios da correção automática de inclinação por meio da detecção de bordas, da Transformada de Hough e da rotação afim. Por

utilizar apenas a orientação das estruturas lineares predominantes, o método pode ser aplicado a diferentes tipos de documentos, sem depender de marcadores específicos.

Em sistemas reais de análise documental, entretanto, o alinhamento normalmente utiliza marcadores geométricos previamente conhecidos. No modelo de folha de respostas empregado pelo ecossistema MCTest, por exemplo, são utilizados quatro discos pretos de referência, além das regiões correspondentes ao cabeçalho, ao *QRCode* e aos quadros de respostas. A localização desses elementos permite estimar simultaneamente a rotação, a escala e a translação da folha, tornando o registro menos sensível à quantidade de texto, à ausência de linhas estruturais e às variações de impressão ou digitalização.

Por esse motivo, a abordagem baseada na Transformada de Hough é utilizada neste capítulo para introduzir os fundamentos do problema, enquanto as etapas posteriores adotam o alinhamento por marcadores geométricos, estratégia predominante em sistemas de OMR e análise documental.

6.5.5 Normalização de Fundo e Equalização Local de Contraste

A qualidade da segmentação depende diretamente das características da imagem de entrada. Em documentos digitalizados, variações de iluminação, sombras, regiões superexpostas e diferenças de tonalidade do papel reduzem o contraste entre o primeiro plano e o fundo, dificultando a aplicação de métodos de limiarização global, como o algoritmo de Otsu.

Para minimizar esses efeitos, são empregadas duas técnicas complementares de pré-processamento:

- **Normalização de fundo:** estima a componente de baixa frequência da imagem por meio de uma forte suavização e, em seguida, normaliza a imagem original em relação a esse fundo estimado. Esse procedimento reduz gradientes de iluminação e compensa variações lentas de intensidade, preservando as estruturas de interesse.
- **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*), apresentado no **Capítulo 4**: divide a imagem em pequenas regiões (*tiles*) e realiza a equalização do histograma de cada região de forma independente. O contraste é limitado para evitar a amplificação excessiva do ruído, tornando a técnica especialmente adequada para imagens com variações locais de iluminação.

A Figure 6.5 compara essas estratégias utilizando a imagem `page()` da biblioteca `skimage.data`. São apresentados seis resultados: (a) a imagem original; (b) a binarização direta pelo método de Otsu, utilizada como referência; (c) o fundo estimado por filtragem Gaussiana; (d) a imagem após a normalização de fundo; (e) a binarização obtida após a aplicação do CLAHE seguida do método de Otsu; e (f) a binarização obtida após a normalização de fundo seguida da aplicação do método de Otsu.

A comparação permite observar o efeito produzido por cada etapa do pré-processamento e sua influência na qualidade da segmentação. Em particular, a normalização de fundo reduz as variações globais de iluminação, enquanto o CLAHE aumenta o contraste local entre caracteres e fundo. Dependendo das características da imagem, uma ou outra estratégia pode produzir resultados superiores, não existindo uma técnica universalmente mais adequada.

No exemplo de retificação apresentado na seção anterior, a folha de respostas foi digitalizada em condições controladas de iluminação, permitindo a aplicação direta da limiarização. Em aplicações práticas, entretanto, documentos frequentemente são capturados por *scanners*, câmeras ou dispositivos móveis sob condições de iluminação não uniforme. Nesses casos, técnicas de pré-processamento como a normalização de fundo e o CLAHE tendem a produzir imagens com contraste mais uniforme, aumentando a robustez das etapas subsequentes de segmentação e análise documental.

```
import cv2
from skimage import data
from morph import mm

img = data.page() # ou: img = mm.gray(img_final) - com imagem da folha de prova

# Método 1: Normalização de fundo + Otsu
```



```
# Estima o fundo com um filtro Gaussiano de sigma grande (variações lentas de luz)
# e divide pixel a pixel para cancelar o gradiente de iluminação
bg = cv2.GaussianBlur(img, (0, 0), sigmaX=25)
img_norm = cv2.divide(img, bg, scale=255)
img_norm_otsu = mm.threshold(img_norm)

# Método 2: CLAHE + Otsu
# tileGridSize define o tamanho de cada região local (tile)
# clipLimit controla o teto de amplificação - valores altos aumentam contraste
# mas também amplificam ruído
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img)
img_clahe_otsu = mm.threshold(img_clahe)

# Método 0: Otsu direto (sem pré-processamento) - referência
img_otsu = mm.threshold(img)

mm.show(
    [img, img_otsu, bg, img_norm, img_clahe_otsu, img_norm_otsu],
    titles=["(a) Original", "(b) Otsu direto", "(c) Gaussiano", \
           "(d) Fundo normalizado (img/bg)", "(e) CLAHE + Otsu", \
           "(f) Normaliz. fundo + Otsu"],
    cols=3,
    figsize=(14, 8)
)
```

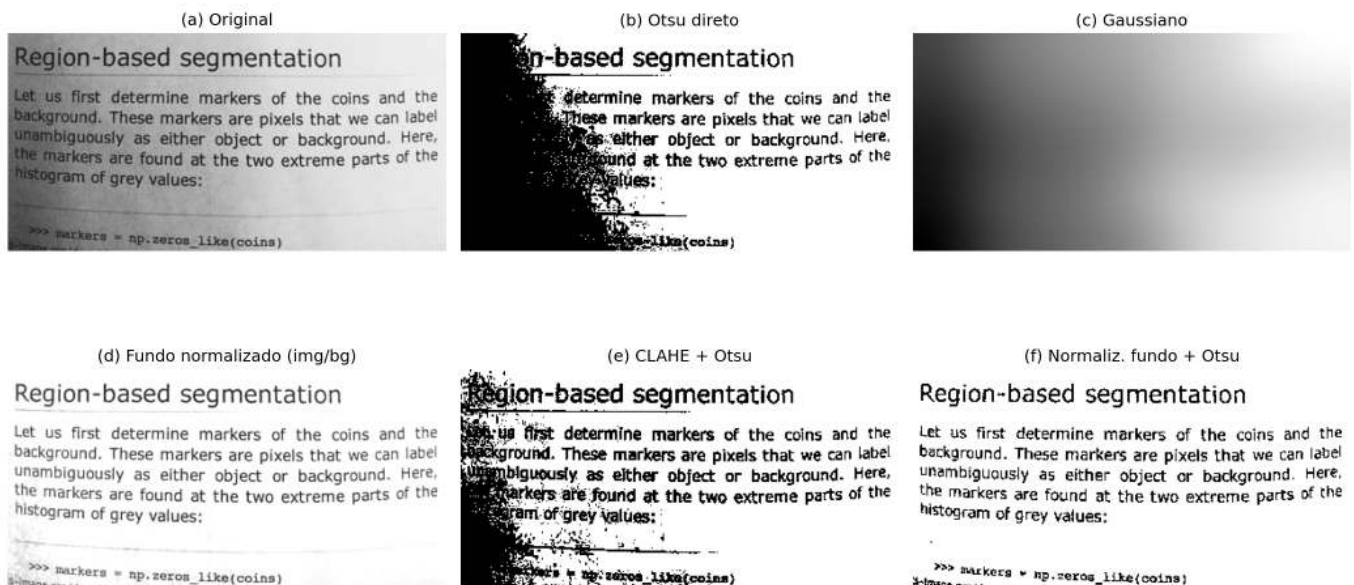


Figura 6.5: Comparação entre estratégias de pré-processamento para binarização da imagem de texto: (a) imagem original; (b) limiarização direta pelo método de Otsu; (c) fundo estimado por filtragem Gaussiana; (d) imagem após normalização de fundo (divisão pela imagem suavizada); (e) CLAHE seguido de limiarização por Otsu; e (f) normalização de fundo seguida de limiarização por Otsu. As imagens ilustram o efeito de cada técnica na compensação de variações de iluminação e na qualidade da segmentação.

i 🧠 Por que funciona? — Normalização de fundo vs. CLAHE

Normalização de fundo: ao dividir a imagem por uma versão fortemente suavizada de si mesma, eliminam-se as variações lentas de luminosidade (gradiente de luz, sombra de borda) sem afetar os detalhes finos — texto, linhas, bolhas. O resultado é uma imagem com iluminação aproximadamente uniforme, onde o limiar global de Otsu passa a funcionar bem em toda a página.

CLAHE: um histograma global equalizado “estica” os tons de toda a imagem de uma vez — útil quando a iluminação é uniforme, mas problemático quando não é. O CLAHE divide a imagem em pequenos blocos (*tiles*) e equaliza cada um separadamente, com um limite máximo de amplificação (`clipLimit`) para não explodir o ruído. É especialmente eficaz para realçar regiões subexpostas localmente, mas não elimina gradientes globais — por isso, aplicá-lo após a normalização de fundo tende a produzir resultados mais consistentes.

6.5.6 Reconhecimento Óptico de Caracteres (OCR)

Após a binarização, a etapa seguinte do processamento documental consiste na conversão da representação visual dos caracteres em texto codificado digitalmente, processo denominado **Reconhecimento Óptico de Caracteres** (*Optical Character Recognition* — **OCR**).

De forma geral, um sistema de OCR compreende três etapas:

1. **Segmentação:** identifica linhas, palavras e caracteres na imagem, utilizando projeções horizontais e verticais ou detecção de componentes conexos.
2. **Extração de características:** representa cada caractere por atributos visuais, como bordas, curvaturas e padrões de traço.
3. **Reconhecimento:** associa os atributos extraídos ao caractere mais provável. Sistemas atuais utilizam predominantemente redes neurais recorrentes (LSTM) ou arquiteturas baseadas em *transformers*.

Neste livro, utiliza-se o **Tesseract OCR**, acessado por meio da biblioteca `pytesseract` (instalação: `pip install pytesseract`). Desenvolvido originalmente pela Hewlett-Packard entre 1985 e 1995 e atualmente mantido pelo Google, o Tesseract é descrito em Smith (2007) e Smith (2013). Nas versões recentes, o reconhecimento textual é realizado por redes neurais LSTM.

O desempenho do OCR depende da qualidade da imagem de entrada. Ruído, baixo contraste, distorções geométricas e iluminação irregular reduzem a taxa de reconhecimento. Por esse motivo, etapas como retificação, normalização de fundo e equalização adaptativa (CLAHE) integram o pré-processamento da imagem.

O Tesseract precisa de uma imagem binarizada?

O Tesseract incorpora internamente uma etapa de binarização adaptativa antes do reconhecimento dos caracteres. Por esse motivo, fornecer ao OCR uma imagem previamente binarizada nem sempre produz os melhores resultados.

Como a limiarização é uma operação irreversível, ela pode eliminar variações sutis de intensidade nas bordas dos caracteres, como o *antialiasing*, que podem auxiliar o mecanismo de reconhecimento. Em muitos casos, uma imagem em tons de cinza, com boa iluminação e contraste, produz uma transcrição mais fiel do que sua versão binarizada.

Para investigar esse efeito, compara-se o texto extraído pelo Tesseract a partir de quatro versões da mesma imagem, apresentadas na Figure 6.6: (a) imagem original; (b) imagem submetida à equalização adaptativa (CLAHE) seguida da limiarização pelo método de Otsu; (c) imagem submetida à normalização de fundo seguida da limiarização pelo método de Otsu; e (d) imagem submetida apenas à normalização de fundo, preservando os tons de cinza.

A comparação entre as versões (c) e (d) mostra que, em trechos contendo caracteres visualmente semelhantes, a versão em tons de cinza (d) produziu uma transcrição mais fiel ao texto original do que a versão binarizada (c). Esse resultado indica que a limiarização aplicada no pré-processamento pode eliminar informações úteis ao reconhecimento. Assim, embora a binarização seja essencial para diversas operações de processamento de imagens, ela não constitui necessariamente a melhor entrada para o OCR. A escolha da técnica de pré-processamento deve considerar a etapa subsequente do *pipeline* documental.

```

import pytesseract
from skimage import data
import cv2
from morph import mm

img = data.page()

# Reaproveitando os resultados da seção anterior
img_otsu = mm.threshold(img)

bg = cv2.GaussianBlur(img, (0, 0), sigmaX=25)
img_norm = cv2.divide(img, bg, scale=255)      # tons de cinza, sem Otsu
img_norm_otsu = mm.threshold(img_norm)        # binarizada

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img)
img_clahe_otsu = mm.threshold(img_clahe)

# Configuração do Tesseract
# --psm 6: assume um único bloco uniforme de texto (adequado à imagem `page`)
config = "--psm 6"

texto_original = pytesseract.image_to_string(img, config=config)
texto_clahe_otsu = pytesseract.image_to_string(img_clahe_otsu, config=config)
texto_norm_otsu = pytesseract.image_to_string(img_norm_otsu, config=config)
texto_norm_gray = pytesseract.image_to_string(img_norm, config=config)

for nome, texto in zip(
    ["(a) Original", "(b) CLAHE + Otsu", "(c) Normaliz. fundo + Otsu",
     "(d) Normaliz. fundo (tons de cinza)"],
    [texto_original, texto_clahe_otsu, texto_norm_otsu, texto_norm_gray]
):
    print(f"--- {nome} ---")
    print(texto.strip(), "\n")

mm.show(
    [img, img_clahe_otsu, img_norm_otsu, img_norm],
    titles=["(a) Original", "(b) CLAHE + Otsu", "(c) Normaliz. fundo + Otsu",
           "(d) Normaliz. fundo (cinza)"],
    cols=4,
    figsize=(16, 4)
)

```

```

--- (a) Original ---
ion-based segmentation
Fst determine markers of the coins and the
These markers are pixels that we can label
ly a5 either object or background. Here,
are found at the two extreme parts of the
eS
Be iatetcoins)

--- (b) CLAHE + Otsu ---
Rédion-based segmentation
hia ha
Menus fist determine markers of the coins and the
yee ground. These markers are pixels that we can label
Sere Epbloueously. aS either object or background. Here,
Reverkers are fourid at the two extreme parts of the
Bsesern.of Grey Values:

```

```
Bee
eessere i neo zdtoa lixstcoins)

--- (c) Normaliz. fundo + Otsu ---
Region-based segmentation

Let us first determine markers of the coins and the
background. These markers are pixels that we can label
unambiguously as either object or background. Here,
the markers are found at the two extreme parts of the
histogram of grey values:

sa" Mathers © np.ceros_Like(coins)

--- (d) Normaliz. fundo (tons de cinza) ---
Region-based segmentation

Let us first determine markers of the coins and the
background. These markers are pixels that we can label
unambiguously as either object or background. Here,
the markers are found at the two extreme parts of the
histogram of grey values:

4 2? Barkers = np. zeros_like(coins)
```

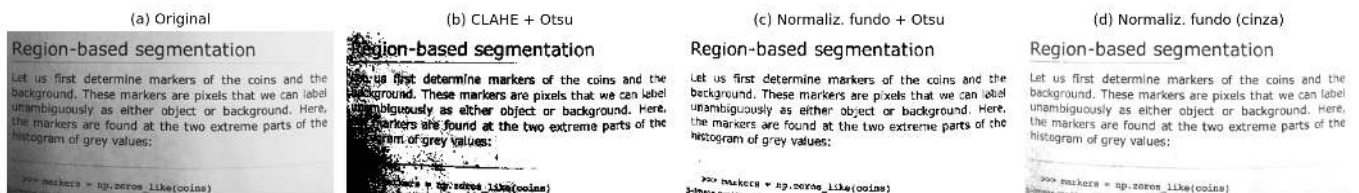


Figura 6.6: Comparação do texto extraído pelo Tesseract a partir de quatro versões da mesma imagem: (a) imagem original; (b) CLAHE seguido de limiarização por Otsu; (c) normalização de fundo seguida de limiarização por Otsu; e (d) normalização de fundo em tons de cinza, sem limiarização. A comparação evidencia que a binarização externa nem sempre favorece o reconhecimento, uma vez que o Tesseract já realiza sua própria binarização adaptativa internamente.

Por que o pré-processamento melhora o OCR?

O desempenho do OCR depende diretamente da qualidade da imagem de entrada. Baixo contraste, iluminação não uniforme, ruído e distorções geométricas dificultam a separação entre texto e fundo e aumentam a probabilidade de erros de reconhecimento.

Técnicas como a normalização de fundo e a equalização adaptativa (CLAHE) corrigem gradientes de iluminação e realçam o contraste local entre os caracteres e o fundo, produzindo imagens mais adequadas ao reconhecimento automático. A limiarização, por sua vez, deve ser aplicada com cautela: por tratar-se de uma operação irreversível, pode eliminar variações sutis de intensidade nas bordas dos caracteres — como o *antialiasing* — que o próprio motor de OCR utiliza internamente para resolver ambiguidades entre símbolos visualmente semelhantes. Por esse motivo, imagens em tons de cinza, corrigidas apenas quanto à iluminação, frequentemente produzem transcrições mais fiéis do que suas versões binarizadas. Em documentos capturados por câmeras de dispositivos móveis, o pré-processamento tende a proporcionar maior ganho de desempenho do que em documentos digitalizados por *scanner*, nos quais a iluminação costuma ser mais uniforme.

6.5.7 Tradução Automática do Texto Reconhecido

Após o reconhecimento óptico de caracteres, o texto obtido pode ser submetido a técnicas de processamento de linguagem natural, como correção ortográfica, indexação, sumarização e tradução automática.

A tradução automática constitui uma etapa independente do OCR. Enquanto o OCR converte os caracteres presentes na imagem em texto codificado, a tradução opera sobre esse texto no idioma original do documento. Dessa forma, erros de reconhecimento podem ser propagados para a tradução, comprometendo a qualidade do resultado. Sistemas atuais de tradução automática utilizam predominantemente arquiteturas neurais baseadas em mecanismos de atenção e *transformers* (Bahdanau; Cho; Bengio, 2015; Vaswani et al., 2017).

Neste exemplo, utiliza-se o texto obtido a partir da imagem submetida apenas à normalização de fundo, sem limiarização (item **d** da Figure 6.6), por apresentar a transcrição mais fiel entre as estratégias avaliadas na seção anterior.

A tradução é realizada por meio da biblioteca `deep-translator` (instalação: `pip install deep-translator`), que fornece uma interface para diferentes serviços de tradução automática, incluindo o Google Translate.

```
from deep_translator import GoogleTranslator

# Texto obtido pelo OCR a partir da imagem com normalização de fundo (tons de cinza)
texto_en = texto_norm_gray

texto_pt = GoogleTranslator(source="en", target="pt").translate(texto_en)

print("--- Texto original gerado pela imagem normalizada em tons de cinza (OCR, EN) ---")
print(texto_en)

print("\n--- Texto traduzido (PT-BR) ---")
print(texto_pt)

mm.show(
    [img_norm],
    titles=["Imagem com normalização de fundo (tons de cinza)"],
    cols=1,
    figsize=(6, 4)
)
```

```
--- Texto original gerado pela imagem normalizada em tons de cinza (OCR, EN) ---
Region-based segmentation
```

Let us first determine markers of the coins and the background. These markers are pixels that we can label unambiguously as either object or background. Here, the markers are found at the two extreme parts of the histogram of grey values:

```
4 2? Barkers = np. zeros_like(coins)
```

```
--- Texto traduzido (PT-BR) ---
Segmentação baseada em região
```

Vamos primeiro determinar os marcadores das moedas e o fundo. Esses marcadores são pixels que podemos rotular inequivocamente como objeto ou plano de fundo. Aqui, os marcadores são encontrados nas duas partes extremas do histograma de valores de cinza:


```
4 2? Ladradores = np. zeros_like(moedas)
```

Imagem com normalização de fundo (tons de cinza)

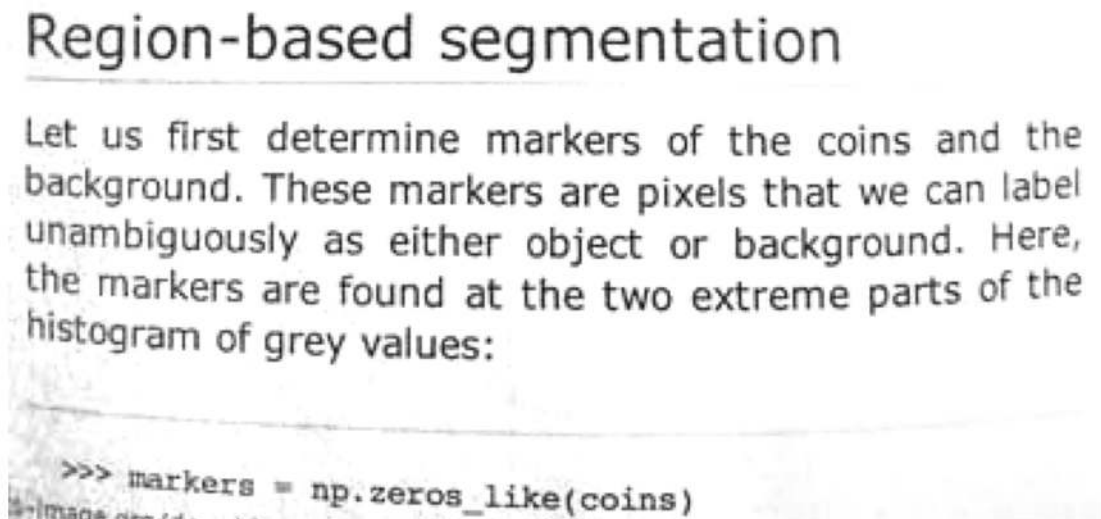


Figura 6.7: Fluxo de reconhecimento e tradução automática. A imagem pré-processada por normalização de fundo, sem limiarização, é utilizada como entrada para o Tesseract OCR, e o texto reconhecido é traduzido do inglês para o português por meio da biblioteca *deep-translator*.

i 🧠 Por que a qualidade do OCR influencia a tradução?

A tradução automática utiliza como entrada o texto produzido pelo OCR. Erros de reconhecimento, como caracteres incorretos, palavras incompletas ou fragmentadas, são propagados para a etapa de tradução e podem alterar o significado do texto.

Consequentemente, a qualidade da tradução depende diretamente da fidelidade da transcrição obtida pelo OCR. Como discutido anteriormente, essa fidelidade nem sempre é maximizada por uma binarização externa: imagens em tons de cinza, corrigidas apenas quanto à iluminação, podem preservar informações relevantes para a distinção entre caracteres visualmente semelhantes. Assim, o pré-processamento da imagem — e a escolha adequada de suas etapas em função da tarefa subsequente — contribui para melhorar não apenas o reconhecimento dos caracteres, mas também o desempenho de etapas posteriores de processamento de linguagem natural, como tradução, indexação e sumarização.

6.5.8 Detecção de Bordas e Contornos

A localização precisa das regiões de interesse é uma etapa essencial em sistemas de OMR. No modelo de folha de respostas utilizado neste capítulo, o cabeçalho e o quadro de respostas estão contidos em um retângulo virtual delimitado por quatro discos pretos posicionados nos cantos. A identificação desses marcadores permite localizar a região de interesse e corrigir distorções geométricas introduzidas durante a aquisição da imagem.

O procedimento é composto por cinco etapas. Inicialmente, aplica-se um **fechamento morfológico** (dilatação seguida de erosão), operação estudada no **Capítulo 4**, utilizando um elemento estruturante em disco (`mm.sedisk(33)`). Essa operação reduz pequenas discontinuidades e preserva os discos de referência, tornando-os mais homogêneos. Em seguida, a imagem é invertida (`mm.neg`), de modo que os discos passem a constituir componentes claros sobre fundo escuro.

Na etapa seguinte, aplica-se a operação `mm.edgeoff` (**Capítulo 4**), que remove componentes conectados às bordas da imagem, eliminando artefatos como sombras de digitalização, marcas de corte e outros objetos espúrios nas margens. Os componentes remanescentes são então analisados a partir de seus contornos e

filtrados por propriedades geométricas, como área e circularidade, para identificar os discos candidatos. A implementação do MCTest torna esse processo mais robusto ao selecionar, entre todos os candidatos, os quatro cujos centros formam um retângulo com largura compatível com a da imagem, reduzindo a ocorrência de falsos positivos.

Por fim, os centros dos quatro discos são ordenados espacialmente (superior esquerdo, superior direito, inferior esquerdo e inferior direito) e utilizados como pontos de controle em uma transformação de perspectiva (*perspective warp*). Essa transformação retifica a imagem, produzindo uma representação alinhada e com dimensões conhecidas, adequada às etapas subsequentes de segmentação e reconhecimento.

As principais etapas desse *pipeline*, desde o processamento morfológico até a imagem retificada, são ilustradas a seguir.

```
import cv2
import numpy as np
from morph import mm

# img: imagem em escala de cinza da folha de prova
if img_final.ndim == 2:
    img = img_final
else:
    img = mm.gray(img_final)

# 1. Fechamento morfológico: preserva os discos escuros, removendo tudo menor que o disco
img_close = mm.close(img, mm.sedisk(41))

# 2. Inversão: discos escuros tornam-se componentes claros sobre fundo escuro
img_neg = mm.neg(img_close)

# 3. Remove componentes conectados que tocam a borda da imagem
img_edgeoff = mm.edgeoff(img_neg)

# 4. Extração dos contornos externos
contornos, _ = cv2.findContours(img_edgeoff, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 5. Filtragem por área e circularidade, mantendo apenas os 4 discos
centros = []
for c in contornos:
    area = cv2.contourArea(c)
    perimetro = cv2.arcLength(c, True)
    if area < 50 or perimetro == 0:
        continue
    circularidade = 4 * np.pi * area / (perimetro ** 2)
    if circularidade > 0.8:
        M = cv2.moments(c)
        cx, cy = M["m10"] / M["m00"], M["m01"] / M["m00"]
        centros.append((cx, cy))

# Verificação robusta: interrompe o pipeline com mensagem clara em vez de AssertionError
if len(centros) != 4:
    print(f"[AVISO] Esperado 4 discos marcadores, encontrado {len(centros)}.")
    print("  Verifique se a imagem é uma folha de respostas MCTest válida")
    print("  ou ajuste os parâmetros de circularidade e área mínima.")
    img_retificada = img # fallback: preserva a imagem sem retificação
else:
    # 6. Ordenação dos centros: superior-esquerdo, superior-direito,
    # inferior-esquerdo, inferior-direito
    pts = np.array(centros, dtype=np.float32)
    soma = pts.sum(axis=1)
    diff = pts[:, 0] - pts[:, 1]
```



```

t1 = pts[np.argmin(soma)]
br = pts[np.argmax(soma)]
tr = pts[np.argmax(diff)]
bl = pts[np.argmin(diff)]
pts_ordenados = np.array([t1, tr, bl, br], dtype=np.float32)

# 7. Retificação por transformação de perspectiva (warp)
largura, altura = 800, 800
destino = np.array(
    [[0, 0], [largura, 0], [0, altura], [largura, altura]], dtype=np.float32
)
M_persp = cv2.getPerspectiveTransform(pts_ordenados, destino)
img_retificada = cv2.warpPerspective(img, M_persp, (largura, altura))

mm.show(
    [img_close, img_edgeoff, img_retificada],
    titles=["Fechamento (sedisk 41)", "edgeoff", "Retificada (warp)"],
    cols=3,
    figsize=(12, 4)
)

mm.write(img_retificada, "img_beetween_disks.png")

```

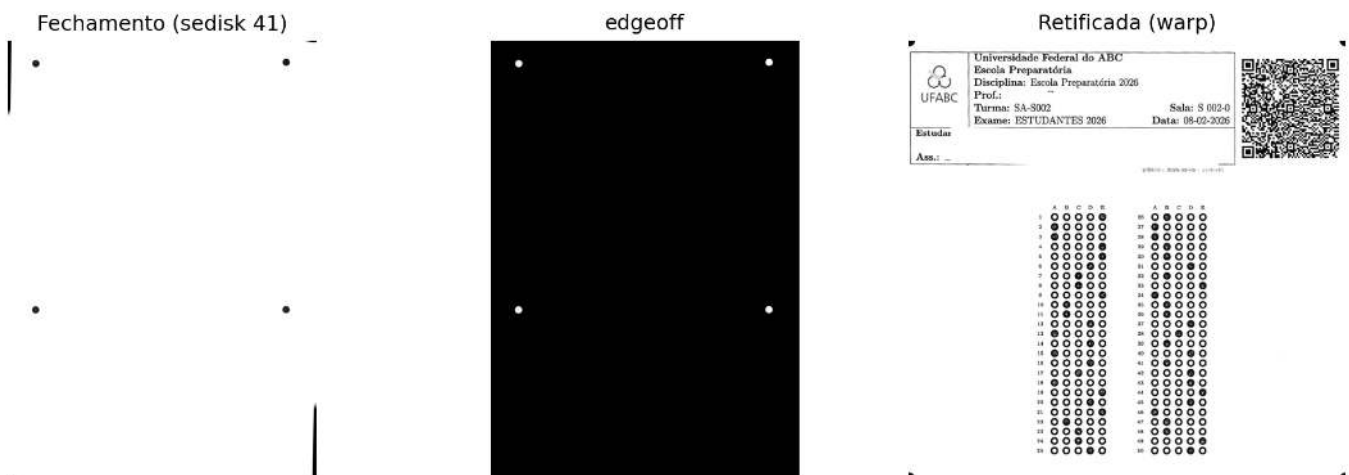


Figura 6.8: Detecção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.

i Por que funciona? — Do fechamento morfológico à retificação

Fechamento morfológico: a dilatação seguida de erosão preenche pequenas descontinuidades e suaviza os contornos dos objetos sem alterar significativamente sua forma global. Ao utilizar um elemento estruturante grande (`sedisk(41)`), detalhes finos, como textos, linhas do formulário e pequenos ruídos, tendem a ser incorporados ao fundo durante o processamento, enquanto objetos de maior escala, como os discos de referência, permanecem preservados e tornam-se mais homogêneos.

Circularidade: após o isolamento dos componentes candidatos, a métrica $C = \frac{4\pi A}{P^2}$ quantifica o quanto sua forma se aproxima de um círculo. Seu valor é igual a 1 para um círculo perfeito e diminui à medida que o contorno se torna mais irregular. Assim, um limiar como $C > 0,8$ permite descartar a maior parte dos falsos positivos sem recorrer a modelos de aprendizado. Um simulador dessa métrica é apresentado na Figure 6.9.

Transformação de perspectiva (*perspective warp*): uma vez identificados os quatro discos de referência, seus centros são utilizados como pontos de controle para estimar a transformação projetiva

que mapeia a imagem capturada para o plano do documento. Essa transformação corrige as distorções introduzidas pela perspectiva durante a aquisição da imagem, produzindo uma representação frontal com dimensões conhecidas e adequada às etapas subsequentes de segmentação e reconhecimento.

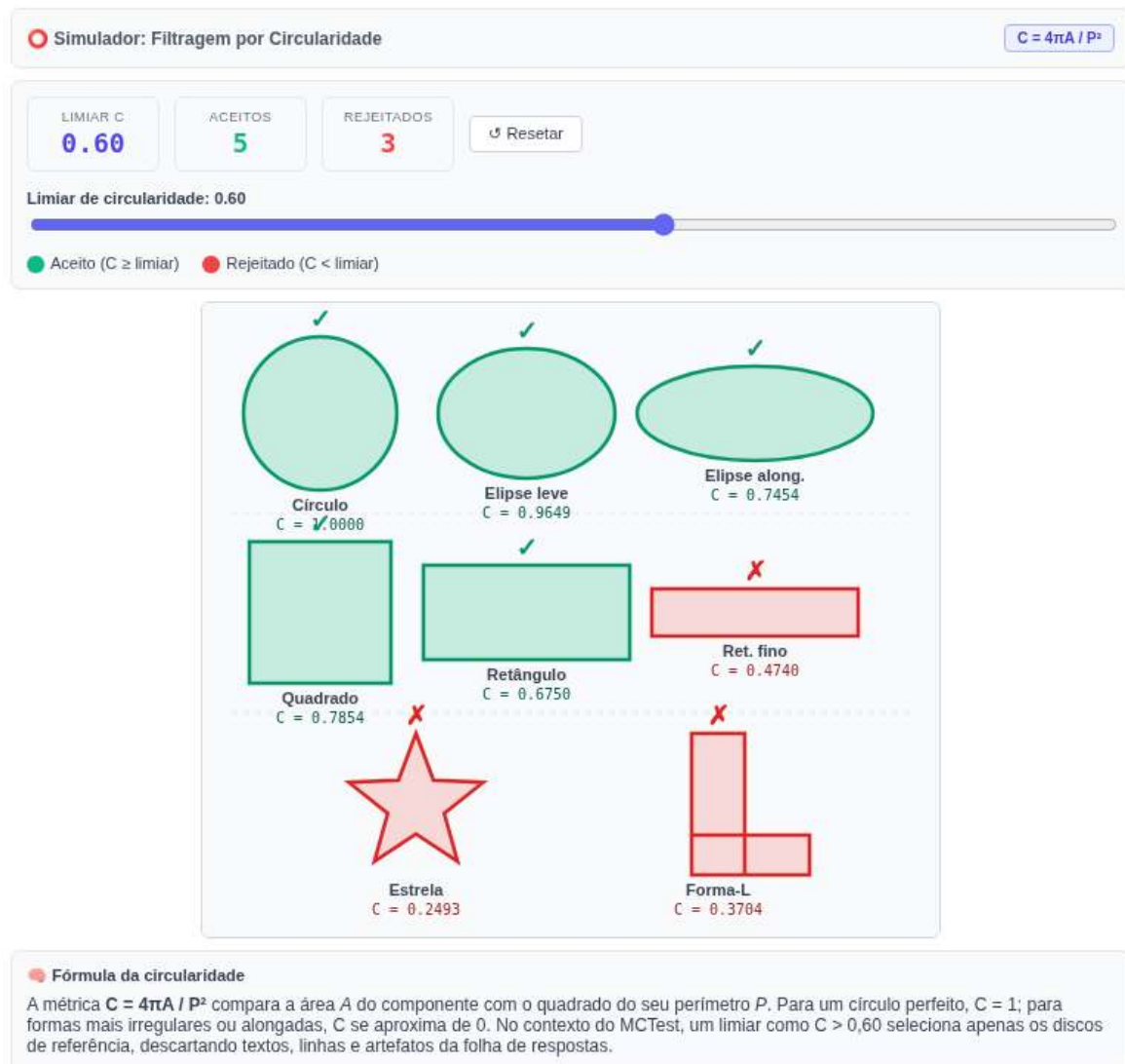


Figura 6.9: Simulador interativo de filtragem por circularidade: mova o *slider* para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).

6.5.9 Isolamento, Segmentação e Decodificação do *QRCode*

Após a retificação geométrica da folha de respostas, realiza-se a detecção e a decodificação do *QRCode* presente no formulário. Esse marcador armazena informações utilizadas pelo sistema de OMR (*Optical Mark Recognition*), como a identificação do aluno, o código da prova e sua variação, possibilitando a recuperação do gabarito correspondente no banco de dados. Por segurança, essas informações são criptografadas antes da geração do *QRCode*. Assim, a sequência decodificada corresponde a uma *string* hexadecimal, cuja interpretação é realizada exclusivamente pelo sistema MCTest. O procedimento é composto por três etapas: pré-processamento morfológico, isolamento da região do *QRCode* e decodificação de seu conteúdo.

Inicialmente, a imagem retificada em escala de cinza é binarizada por meio da operação `mm.threshold`. Em seguida, aplica-se uma **abertura morfológica** (erosão seguida de dilatação), estudada no **Capítulo 4**, utilizando um elemento estruturante quadrado (`mm.sebox(2)`). Essa operação remove pequenos ruídos e suaviza imperfeições sem comprometer a estrutura do marcador. Por fim, a imagem é invertida (`mm.neg`), de

modo que o *QRCode* passe a constituir um componente claro sobre fundo escuro, facilitando a extração de seus contornos.

A localização do *QRCode* é realizada pela análise dos contornos externos da imagem binarizada. Entre os componentes detectados, seleciona-se aquele com maior área e geometria aproximadamente quadrada, descartando os demais elementos impressos da folha. Em seguida, a região correspondente é expandida por uma pequena margem de segurança, garantindo a preservação integral do marcador.

O *QRCode* é então extraído diretamente da imagem retificada em escala de cinza, preservando sua qualidade radiométrica. Como essa região geralmente apresenta dimensões reduzidas, aplica-se um redimensionamento com interpolação cúbica, aumentando a resolução espacial e favorecendo a identificação de seus módulos. A leitura é realizada pelo detector de *QRCode* do OpenCV (`cv2.QRCodeDetector`), que recupera a sequência de caracteres originalmente codificada.

O fluxo completo desse processamento, desde o pré-processamento morfológico até a decodificação do *QRCode*, é ilustrado na Figure 6.10. O trecho exibido na saída corresponde apenas ao início da *string* hexadecimal criptografada; sua interpretação completa é realizada internamente pelo MCTest após a decodificação.

```
import cv2
import numpy as np
from morph import mm

# f: imagem retificada convertida para o tipo correto de 8 bits (0-255)
f = img_retificada.astype('uint8')

# 1. Limiarização: conversão da imagem em tons de cinza para binária
f_thresh = mm.threshold(f)

# 2. Abertura morfológica: elimina pequenos ruídos e suaviza o contorno dos blocos
f_open = mm.open(f_thresh, mm.sebox(2))

# 3. Inversão morfológica: módulos escuros tornam-se componentes claros sobre fundo escuro
f_inv = mm.neg(f_open)

# 4. Conversão segura para uint8 com escala 0-255
img_uint8 = (f_inv.astype(np.uint8) * 255) if f_inv.max() == 1 else f_inv.astype(np.uint8)

# 5. Detecção de contornos externos
contornos, _ = cv2.findContours(img_uint8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if not contornos:
    raise ValueError("Nenhum contorno encontrado. Verifique o limiar ou a imagem de entrada.")

# 6. Filtragem pelo maior contorno com proporção aproximadamente quadrada
# (aspect ratio entre 0.7 e 1.3 descarta retângulos alongados da folha)
def is_square_like(contorno, tol=0.3):
    x, y, w, h = cv2.boundingRect(contorno)
    ratio = w / h if h > 0 else 0
    return (1 - tol) <= ratio <= (1 + tol)

candidatos = [c for c in contornos if is_square_like(c)]

if not candidatos:
    raise ValueError(
        "Nenhum contorno quadrado encontrado. "
        "Verifique se o QRCode está presente na imagem ou ajuste a tolerância."
    )

# Seleciona o maior candidato quadrado por área de bounding box
maior_contorno = max(candidatos, key=lambda c: cv2.boundingRect(c)[2] * cv2.boundingRect(c)[3])
```

```
x, y, w, h = cv2.boundingRect(maior_contorno)

# 7. Expansão da bounding box com margem de segurança (evita truncamento do QRCode)
margem = 5
h_img, w_img = img_uint8.shape[:2]
x1 = max(x - margem, 0)
y1 = max(y - margem, 0)
x2 = min(x + w + margem, w_img)
y2 = min(y + h + margem, h_img)

# 8. Recorte da região de interesse a partir da imagem original (nítida, em cinza)
img_qrcode_final = img_retificada[y1:y2, x1:x2]

# 9. Ampliação para resolução mínima de decodificação (400 px no lado maior)
# cv2.QRCodeDetector requer módulos com ao menos 3-4 px de largura para decodificar
# com segurança; imagens menores que ~400 px tendem a falhar.
lado = max(img_qrcode_final.shape[:2])
escala = max(400 / lado, 1.0)
img_para_leitura = cv2.resize(
    img_qrcode_final, None,
    fx=escala, fy=escala,
    interpolation=cv2.INTER_CUBIC
)

# 10. Inicialização do detector nativo de QRCode do OpenCV
detector = cv2.QRCodeDetector()

# 11. Detecção geométrica e decodificação dos dados textuais
dados, pontos, qrcode_reto = detector.detectAndDecode(img_para_leitura)

# Visualização intermediária: progressão da binarização ao isolamento do QRCode
mm.show(
    [f_thresh, f_open, f_inv, img_qrcode_final],
    titles=["1. Limiarização", "2. Abertura morfológica", "3. Inversão", "Imagem final"],
    cols=3, figsize=(12, 4)
)

# Validação e saída dos metadados extraídos
if dados:
    print(f"QRCode decodificado com sucesso: \n{dados[:50]}...")
else:
    raise ValueError(
        "Falha na decodificação do QRCode. "
        "Verifique o limiar, as margens da região ou a qualidade da imagem."
    )
```

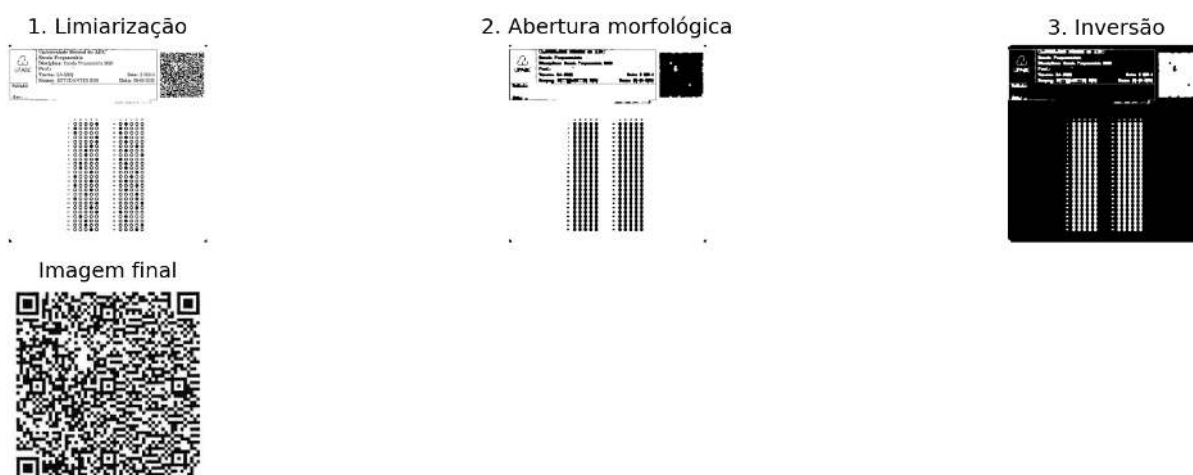


Figura 6.10: *Pipeline* de processamento do *QRCode*: limiarização, abertura morfológica, inversão e recorte final para decodificação.

QRCode decodificado com sucesso:

325a356b71367266556955646b7233454149624a694f417730...

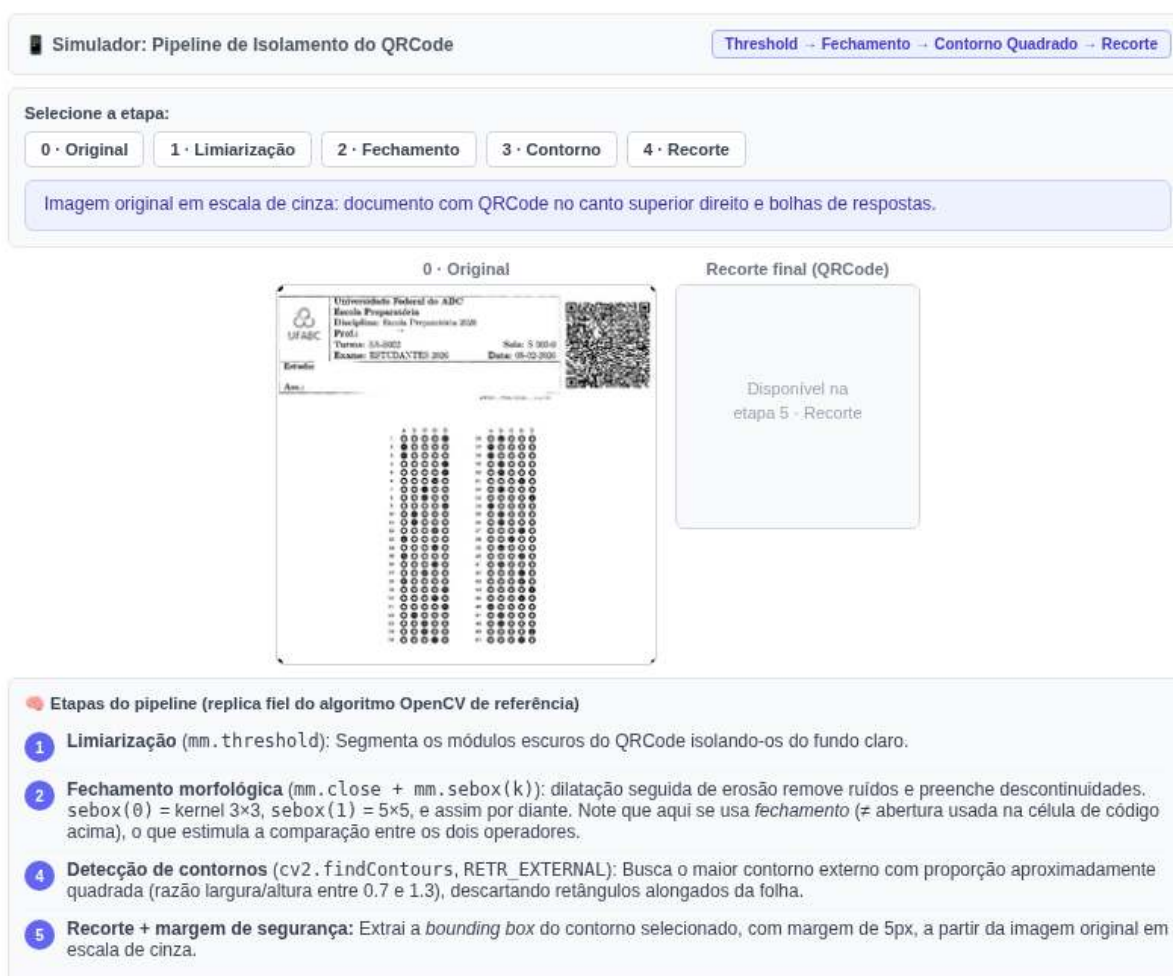


Figura 6.11: Simulador interativo do *pipeline* de isolamento do *QRCode*: navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a detecção do contorno quadrado sobre a imagem original do gabarito.

Por que funciona? — Isolamento e decodificação do *QRCode*

Abertura morfológica: diferentemente do fechamento, a abertura (erosão seguida de dilatação) remove pequenos ruídos e protuberâncias sem alterar significativamente a geometria dos objetos maiores. Assim, preserva a estrutura do *QRCode* enquanto elimina componentes espúrios que poderiam dificultar sua localização.

Seleção por geometria: o *QRCode* possui formato aproximadamente quadrado ($w/h \approx 1$). A combinação desse critério com a seleção do componente de maior área descarta linhas do formulário, textos e outros elementos impressos, permitindo isolar o marcador sem o uso de modelos de aprendizado.

Redimensionamento antes da decodificação: quando o *QRCode* ocupa poucos pixels na imagem, seus módulos tornam-se difíceis de distinguir. O redimensionamento com interpolação cúbica aumenta a resolução espacial da região de interesse, facilitando a identificação dos padrões do código pelo `cv2.QRCodeDetector` e tornando a decodificação mais robusta.

6.5.10 Decodificação de Códigos de Barras

Na seção anterior, a decodificação de *QRCodes* foi realizada utilizando o detector nativo do OpenCV (`cv2.QRCodeDetector`), que integra em uma única interface a detecção geométrica do símbolo, sua retificação e a extração da informação codificada.

Para códigos de barras lineares (1D), como EAN-13, Code 39 e Code 128, uma alternativa amplamente utilizada é a biblioteca `pyzbar`. Diferentemente do `QRCodeDetector`, ela suporta diversas simbologias de códigos de barras e também pode ser empregada na leitura de *QRCodes*.

O procedimento consiste em localizar automaticamente cada símbolo presente na imagem e interpretar a sequência de barras e espaços correspondente, produzindo a cadeia de caracteres codificada. Além dos dados decodificados, a biblioteca fornece informações como a simbologia identificada e a posição do código na imagem, permitindo sua posterior validação ou processamento.

A Figure 6.12 apresenta um exemplo de código de barras e o resultado de sua decodificação utilizando a biblioteca `pyzbar`.

```
import os
import urllib.request
import numpy as np
from pyzbar.pyzbar import decode
from morph import mm

barcode_path = 'dados/barcode.png'
url_github = (
    "https://raw.githubusercontent.com/fzampirolli/"
    "pdi-vc/master/all/cap06/dados/barcode.png"
)

# Se o arquivo não existir localmente, baixa automaticamente do GitHub
if not os.path.exists(barcode_path):
    print(f"[DOWNLOAD] Baixando código de barras do GitHub: {url_github}")
    try:
        os.makedirs(os.path.dirname(barcode_path), exist_ok=True)
        urllib.request.urlretrieve(url_github, barcode_path)
        print("[DOWNLOAD] Imagem baixada com sucesso!")
    except Exception as e:
        print(f"[DOWNLOAD] Falha ao baixar o arquivo: {e}")

if os.path.exists(barcode_path):
    image = mm.read(barcode_path)
else:
    # Fallback sintético: gera um padrão de barras verticais que simula um Code-128
```



```

print("[AVISO] Arquivo 'dados/barcode.png' não encontrado.")
print("      Usando imagem sintética para demonstração do pipeline.")
h, w = 100, 400
img_synt = np.ones((h, w), dtype=np.uint8) * 255
# Barras escuras em posições regulares (padrão simplificado)
for x in range(20, w - 20, 8):
    if (x // 8) % 3 != 0:
        img_synt[:, x:x+4] = 0
image = img_synt

mm.show(image)

# Executa o decode do pyzbar
try:
    barcodes = decode(image)
except ImportError:
    print("[ERRO] zbar do sistema não encontrada. Rode: !apt-get install -y libzbar0")
    barcodes = []

if barcodes:
    dados_bc = barcodes[0].data.decode("utf-8")
    tipo = barcodes[0].type
    print(f"Código de barras decodificado com sucesso [{tipo}]:\n{dados_bc}")
else:
    print("[INFO] Nenhum código de barras detectado na imagem.")
    print(
        "Em imagem sintética isso é esperado - "
        "substitua pelo arquivo real para decodificar."
    )

```



Figura 6.12: Decodificação de código de barras linear com *pyzbar*: imagem de entrada e dados extraídos.

Código de barras decodificado com sucesso [EAN13]:
0000000000055

6.6 O MCTest como Estudo de Caso: do Protótipo ao Sistema em Produção

Até este ponto, as principais etapas do *pipeline* de processamento foram apresentadas e analisadas individualmente, incluindo a correção da inclinação (*deskew*), a detecção de marcadores, a retificação por perspectiva e a leitura de *QR Codes*. Embora essa abordagem facilite a compreensão de cada técnica, aplicações reais exigem a integração dessas etapas em um fluxo de processamento único e consistente.

O **MCTest** constitui um exemplo dessa integração. Desenvolvido na UFABC e disponibilizado como software de código aberto, o sistema é utilizado desde 2012 na correção automatizada de avaliações, oferecendo suporte a diferentes modelos de folhas de resposta, gabaritos individualizados e geração automática de relatórios de desempenho (Zampirolli, 2023).

A partir deste ponto, o foco deixa de ser a implementação isolada de algoritmos e passa a ser a organização desses algoritmos em uma aplicação completa. Além da qualidade dos métodos de processamento de imagens, um sistema dessa natureza deve atender a requisitos como robustez diante de diferentes condições de aquisição, facilidade de manutenção e capacidade de evolução para novas funcionalidades.

Nas próximas seções, será analisado o módulo de Visão Computacional do MCTest, implementado no arquivo `CVMCTest.py`. O objetivo é mostrar como os conceitos apresentados ao longo deste capítulo são combinados em um *pipeline* de processamento empregado em uma aplicação real.

6.6.1 Obtenção e Preparação do Módulo

O arquivo `CVMCTest.py` integra o sistema MCTest e, em sua versão original, depende de modelos, configurações e outros componentes do *framework* Django. Como essas dependências não estão disponíveis no ambiente utilizado neste capítulo, o módulo precisa ser adaptado para execução de forma independente.

Para isso, o arquivo é obtido diretamente do repositório do projeto por meio da biblioteca `requests`. Em seguida, comandos `sed` são empregados para remover as importações e dependências específicas do ambiente Web, produzindo uma versão autocontida do módulo. Essa adaptação preserva a implementação dos algoritmos de Visão Computacional, permitindo sua execução e análise sem a necessidade de instalar ou configurar toda a infraestrutura do sistema MCTest.

```
import requests
CVMCTest = requests.get(
    "https://raw.githubusercontent.com/fzampirolli/mctest/master/exam/CVMCTest.py"
)
with open('CVMCTest.py', 'w') as writefile:
    writefile.write(CVMCTest.text)
```

Cada comando `sed` remove importações específicas do ambiente Django, tornando o arquivo `CVMCTest.py` utilizável de forma independente neste capítulo.

```
# remove linhas with "from django.", ...
!sed --in-place '/from django./d' CVMCTest.py
!sed --in-place '/from exam./d' CVMCTest.py
!sed --in-place '/from mctest./d' CVMCTest.py
!sed --in-place '/from student./d' CVMCTest.py
!sed --in-place '/from topic./d' CVMCTest.py
!sed --in-place '/from .models import VariationExam/d' CVMCTest.py
```

Por que remover as dependências do Django?

Na implementação original, o arquivo `CVMCTest.py` faz parte de uma aplicação desenvolvida com o *framework* Django e, por isso, importa modelos, configurações e outros componentes específicos desse ambiente. Como esses elementos não estão disponíveis neste capítulo, o módulo não pode ser importado diretamente.

Os comandos `sed` removem apenas essas dependências, sem alterar as rotinas de Visão Computacional implementadas no arquivo. Dessa forma, o módulo pode ser executado de maneira independente, preservando o comportamento dos algoritmos apresentados.

Esse procedimento ilustra um princípio importante de engenharia de software: separar a lógica da aplicação da infraestrutura em que ela está inserida, facilitando a reutilização, os testes e o estudo de componentes específicos.

6.6.2 Extração da Área de Respostas

Após a leitura da folha, a função `getAnswerArea` executa automaticamente as etapas de detecção dos marcadores de referência e retificação por perspectiva apresentadas nas seções anteriores. Como resultado, é obtida uma imagem contendo apenas a região destinada às respostas, alinhada e com dimensões padronizadas.

Essa padronização simplifica as etapas subsequentes de processamento, pois a localização dos campos de marcação passa a ser conhecida e independente da posição original da folha durante a digitalização.

A Figure 6.13 apresenta a folha de respostas original em escala de cinza, enquanto a Figure 6.14 mostra a região de respostas obtida após a aplicação da função `getAnswerArea`.

```
import os
from pdf2image import convert_from_path
from skimage import data as skdata
import cv2
from morph import mm

file = "dados/provas_qrcode_EP.pdf"
MYFILES = 'extra02.qrcode'

if os.path.exists(file):
    pages = convert_from_path(file, 200) # dpi 100=min 500=max
    numPAGES = 0
    for page in pages:
        myfile0 = MYFILES + '_p' + str(numPAGES) + '.png'
        page.save(myfile0)
        numPAGES += 1
        print(f"[INGESTÃO] Página convertida: {myfile0}")
    pages.clear()
    img_color = mm.read(myfile0)
    img_inicial = mm.gray(img_color)
else:
    print("[AVISO] Arquivo 'dados/provas_qrcode.pdf' não encontrado.")
    print("      Usando imagem pública skimage.data.page() como substituto.")
    img_inicial = skdata.page()

mm.show(img_inicial)
```

```
[INGESTÃO] Página convertida: extra02.qrcode_p0.png
[INGESTÃO] Página convertida: extra02.qrcode_p1.png
[INGESTÃO] Página convertida: extra02.qrcode_p2.png
```

Universidade Federal do ABC
Escola Preparatória
Disciplina: Escola Preparatória 2026
Prof.:
Turma: SA-S002 Sala: S 002-0
Exame: ESTUDANTES 2026 Data: 08-02-2026

Estuda
Ass.: 4

#DA10 - 2026-02-08 - 15:45:41

	A	B	C	D	E
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					
22					
23					
24					
25					
26					
27					
28					
29					
30					
31					
32					
33					
34					
35					
36					
37					
38					
39					
40					
41					
42					
43					
44					
45					
46					
47					
48					
49					
50					

UFABC Testa: gerador e corretor de exames disponível para professores - www.ufabc.edu.br

Figura 6.13: Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado.

```
import CVMCTest
countPage = 0
img_getAnswerArea = CVMCTest.cvMCTest.getAnswerArea(img_inicial, countPage)
mm.show(img_getAnswerArea)
```

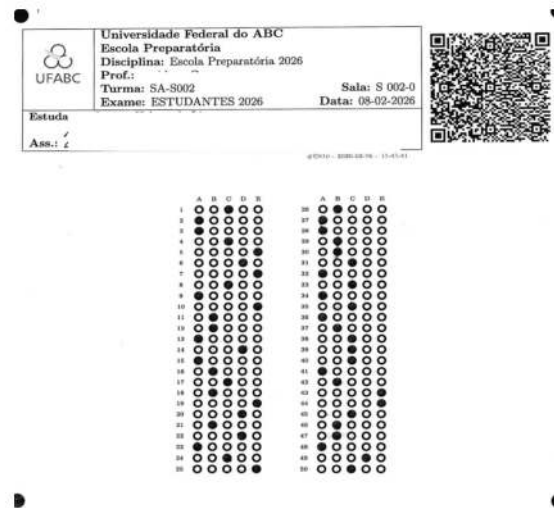


Figura 6.14: Área de respostas extraída por *getAnswerArea*: região retificada contendo os quadros de marcação.

Nota de compatibilidade: versões recentes do NumPy (2.0) removeram o alias `np.int0`. Caso `CVMCTest.py` utilize esse tipo, o comando abaixo aplica a correção diretamente no arquivo antes de recarregá-lo:

```
!sed -i 's/box = np.int0(cv2.boxPoints(rect))/box = cv2.boxPoints(rect).astype(np.intp)/' \
./CVMCTest.py
```

```
import importlib
import CVMCTest

importlib.reload(CVMCTest)
```

```
<module 'CVMCTest' from '/home/fz/VSCoDe/pdi-vc/all/cap06/CVMCTest.py'>
```

6.6.3 Segmentação do *QRCode*

Após a extração da área de respostas, o `MCTest` realiza duas etapas preparatórias para a leitura das marcações: a segmentação do *QRCode* e a localização dos quadros que contêm as questões.

A função `segmentQRcode` isola a região da imagem correspondente ao *QRCode*, apresentada na Figure 6.15. Em seguida, essa região é processada pela função `getQRCode`, responsável por sua decodificação e pela extração dos metadados da prova.

Caso o *QRCode* não possa ser decodificado, o processamento da folha continua normalmente. As respostas do estudante ainda são lidas e registradas no arquivo CSV de saída; apenas as informações obtidas a partir do *QRCode*, como a identificação da prova ou do estudante, permanecem indisponíveis.

```
import CVMCTest
imgQRcode = CVMCTest.cvMCTest.segmentQRcode(img_getAnswerArea, countPage)
mm.show(imgQRcode)
```



Figura 6.15: Região do *QRCode* isolada por *segmentQRcode* dentro da área de respostas retificada.

A função `CVMCTest.cvMCTest.getQRcode(img, countPage)` integra as etapas de segmentação e decodificação do *QRCode*. Internamente, ela utiliza `CVMCTest.cvMCTest.decodeQRcode(imgQRcode)` para interpretar a *string* hexadecimal codificada no símbolo e construir o dicionário `qr`, além de retornar o indicador lógico `myFlagArea`, que informa se a leitura foi realizada com sucesso.

O dicionário `qr` reúne os metadados da prova utilizados nas etapas subsequentes de processamento. Seus principais campos são:

- **date:** identificador temporal da prova, composto pela data de geração e por um *timestamp* interno do `MCTest`.
- **idClassroom, idExam e idStudent:** identificadores da turma, da prova e do estudante.
- **term:** período letivo.
- **stylesheet:** folha de estilo utilizada na geração do formulário.
- **var1 a var5:** quantidade de questões em cada nível de dificuldade.
- **text:** número de questões dissertativas.
- **answer:** número de alternativas por questão.
- **numquest:** número total de questões.
- **correct e dbtext:** campos preenchidos durante a correção, contendo o gabarito e informações adicionais.
- **variations e variant:** informações sobre as versões da prova.

Esses metadados identificam a prova e o estudante, permitindo selecionar o gabarito correspondente e parametrizar as etapas seguintes de leitura e correção das respostas.

```
myFlagArea, qr = CVMCTest.cvMCTest.getQRcode(img_inicial, countPage)
myFlagArea, qr
```

```
(True,
{'date': '260208-1770403541363',
 'idClassroom': '955',
 'idExam': '810',
 'idStudent': '448898',
 'term': '0',
 'stylesheet': '1',
 'var1': '50',
 'var2': '0',
 'var3': '0',
 'var4': '0',
 'var5': '0',
 'text': '0',
 'answer': '5',
```

```
'numquest': 50,
'correct': '',
'dbtext': '',
'variations': '0',
'variant': '0'})
```

Esses metadados permitem identificar a prova, recuperar o gabarito correspondente e parametrizar as etapas subsequentes de processamento.

Após a decodificação do *QRCode*, o processamento retorna à área de respostas para localizar os quadros que contêm as marcações do estudante.

6.6.4 Localização dos Quadros de Respostas

A imagem produzida por `getAnswerArea` contém toda a região útil da folha, incluindo o cabeçalho, onde se encontra o *QRCode*, e os quadros destinados às respostas. Como a leitura das marcações utiliza apenas esses quadros, a região correspondente ao cabeçalho é descartada por meio do recorte `img_getAnswerArea[300:, :]`.

A Figura 6.16 apresenta essa região de interesse. Embora esse recorte seja posteriormente utilizado pela função `findSquares` para localizar os quadros de respostas, o `MCTest` realiza inicialmente a leitura do *QRCode*, pois ele contém os metadados necessários para identificar a prova e configurar as etapas subsequentes do processamento.

```
img_getAnswerArea_aux = img_getAnswerArea[300:, :]
mm.show(img_getAnswerArea_aux)
```

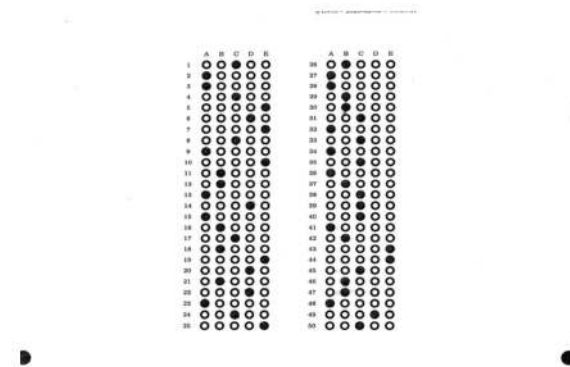


Figura 6.16: Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.

A função `findSquares` recebe a imagem da área de respostas e os metadados armazenados em `qr`, retornando as coordenadas dos quadros que delimitam os grupos de questões.

Cada elemento de `rectSquares` contém as coordenadas dos vértices superior esquerdo e inferior direito de um quadro de respostas, que serão utilizadas na etapa de segmentação das bolhas.

```
rectSquares = CVMCTest.cvMCTest.findSquares(qr, img_getAnswerArea, countPage)
rectSquares
```

```
[[[np.int64(395), np.int64(350)], [np.int64(950), np.int64(516)]],
 [[np.int64(394), np.int64(602)], [np.int64(951), np.int64(767)]]]
```

6.6.5 Leitura Automática das Respostas

Conhecidos os metadados da prova (`qr`) e as coordenadas dos quadros de respostas (`rectSquares`), o `MCTest` identifica automaticamente as alternativas marcadas pelo estudante.

O código a seguir integra as etapas apresentadas anteriormente. Para cada quadro delimitado em `rectSquares`, as funções `setColumns` e `setLines` estimam, respectivamente, o número de alternativas por questão e o número de questões a partir da distribuição espacial das bolhas. Em seguida, `segmentAnswers` determina a alternativa assinalada em cada questão e `setAnswersOneLine` reúne os resultados de todos os quadros no campo `qr['answers']`.

No modo de operação adotado neste capítulo, em que o `MCTest` é executado de forma independente de seu banco de dados, o conteúdo de `qr['answers']` é comparado ao gabarito armazenado na primeira página do arquivo PDF, correspondente ao modelo de prova sem enunciados utilizado nos exemplos.

A Figure 6.17 apresenta os quadros de respostas processados pelo algoritmo, enquanto a saída do programa exibe o conteúdo final de `qr['answers']`.

```
testAnswers = []
if myFlagArea:

    imgQ_all = []

    for countSquare in range(len(rectSquares)):
        p1, p2 = rectSquares[countSquare]

        if True:
            imgQi = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
            [NUM_COLUMNS, img] = CVMCTest.cvMCTest.setColumns(imgQi, countPage, countSquare)
            [NUM_LINES, img] = CVMCTest.cvMCTest.setLines(imgQi, countPage, countSquare)
            NUM_RESPOSTAS = NUM_COLUMNS
            NUM_QUESTOES = NUM_LINES

            imgQiNC = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
            testAnswers.append(CVMCTest.cvMCTest.segmentAnswers(
                [imgQi, imgQiNC], countPage, countSquare, NUM_QUESTOES, qr

            ))

            imgQ_all.append(imgQiNC)

    qr = CVMCTest.cvMCTest.setAnswersOneLine(testAnswers, qr)
    # deixa as respostas de cada quadro em uma linha

mm.show(imgQ_all)
print(f"Respostas lidas das {len(qr['answers'].split(","))} questões: \
      \n{qr['answers'][:-19]}...")
```

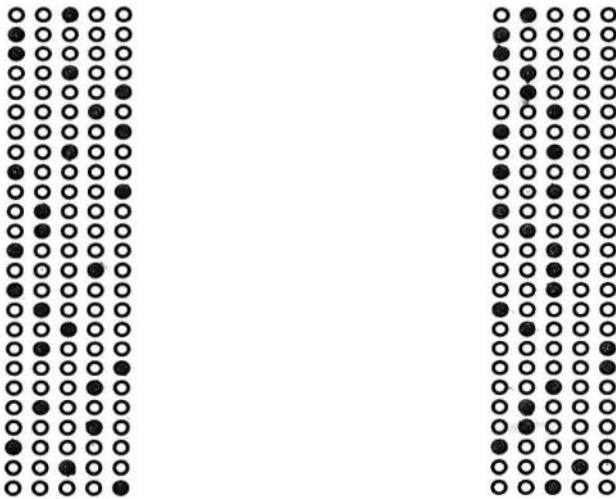


Figura 6.17: Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.

Respostas lidas das 50 questões:

C,A,A,C,E,D,E,C,A,E,B,B,A,D,A,B,C,B,E,D,B,D,A,C,E,B,A,A,B,B,C,A,C,A,C,A,B,C,C,C,...

💡 Conectando os Pontos

O campo `qr['answers']` representa o resultado final do *pipeline* de Visão Computacional apresentado neste capítulo. Sua obtenção integra todas as etapas estudadas, desde a rasterização do documento e a retificação geométrica até a extração da área de respostas, a decodificação do *QRCode*, a localização dos quadros e a identificação das alternativas marcadas.

Esse *pipeline* ilustra a transição de um protótipo para um sistema em produção. No MCTest, os algoritmos de Visão Computacional permanecem essencialmente os mesmos; as principais diferenças concentram-se em aspectos de engenharia de software, como tratamento de exceções, suporte a diferentes modelos de formulários, integração com banco de dados, interface Web e mecanismos de auditoria e manutenção.

Nos experimentos deste capítulo, a primeira página do arquivo PDF contém o gabarito da prova, enquanto as páginas subsequentes correspondem às folhas de respostas dos estudantes. Após a obtenção de `qr['answers']`, o MCTest compara automaticamente as respostas lidas com o gabarito para calcular a pontuação de cada estudante.

Na utilização completa do sistema, os resultados da correção são consolidados em um arquivo CSV e enviados ao professor juntamente com um arquivo compactado contendo informações auxiliares para auditoria. Entre esses arquivos estão os recortes das questões em que foram detectadas múltiplas marcações ou outras situações que exigem revisão manual. O professor pode, então, inspecionar essas imagens, decidir a interpretação mais adequada e, se necessário, atualizar o arquivo CSV antes da importação definitiva das notas.

6.7 Inspeção Industrial Automatizada

A inspeção visual automatizada é uma aplicação da Visão Computacional em que imagens de peças ou produtos são analisadas para verificar o atendimento a critérios de qualidade previamente definidos. Em uma linha de produção, as imagens podem ser obtidas por câmeras ou outros dispositivos de aquisição e processadas automaticamente para identificar defeitos, medir dimensões ou verificar a presença de componentes.

A estratégia de inspeção depende das características do produto, do tipo de defeito de interesse e da disponibilidade de uma imagem de referência. Neste capítulo são apresentadas duas abordagens clássicas:

- **Subtração de imagens:** compara a imagem da peça inspecionada com uma imagem de referência

considerada livre de defeitos. Regiões em que a diferença de intensidade excede um limiar são classificadas como possíveis defeitos. Essa abordagem pressupõe que as imagens estejam geometricamente alinhadas e tenham sido adquiridas sob condições semelhantes de iluminação.

- **Análise de textura:** utiliza características da textura da superfície para identificar regiões cuja aparência difere do padrão esperado, sem a necessidade de uma imagem de referência. Essa abordagem é adequada para materiais que apresentam textura aproximadamente homogênea, como tecidos, papéis e superfícies metálicas.

Nas próximas seções, essas duas estratégias são ilustradas por meio de exemplos construídos a partir de imagens da biblioteca `skimage.data`. O objetivo é apresentar os princípios de funcionamento de cada abordagem em experimentos que possam ser reproduzidos integralmente pelo leitor.

6.7.1 Referências e *Datasets* Públicos

Em aplicações de inspeção industrial, o desempenho de algoritmos de detecção de defeitos é frequentemente avaliado em *datasets* públicos, que fornecem imagens representativas e, em muitos casos, anotações de referência (*ground truth*). Neste capítulo, entretanto, os exemplos utilizam imagens sintéticas derivadas de `skimage.data` (ver Figure 6.18), permitindo reproduzir todos os experimentos sem dependência de bases de dados externas.

Para estudos mais abrangentes e comparação entre algoritmos, destacam-se os seguintes *datasets* públicos:

- **MVTec *Anomaly Detection Dataset* (MVTec AD):** conjunto de imagens de objetos e texturas, contendo amostras sem defeitos e com defeitos, acompanhadas de máscaras de segmentação em nível de pixel para as imagens anômalas (Bergmann et al., 2019). Disponível em: <https://www.mvtec.com/company/research/datasets/mvtec-ad>.
- **Kolektor *Surface-Defect Dataset* (KolektorSDD):** conjunto de imagens de componentes industriais com defeitos superficiais anotados, utilizado em estudos de detecção e segmentação de defeitos (Tabernik et al., 2020). Disponível em: <https://www.vicos.si/resources/kolektorsdd/>.
- **NEU *Surface Defect Database*:** conjunto de imagens de superfícies de aço laminado, organizado em seis categorias de defeitos superficiais, frequentemente empregado na avaliação de métodos de classificação e detecção (Song; Yan, 2013). Disponível em: http://faculty.neu.edu.cn/songkechen/zh_CN/zdylm/263270/list/index.htm.

```
import numpy as np
from skimage import data, color
from morph import mm

# Imagem de referência (produto sem defeito)
product_color = data.coffee()
product_gray = color.rgb2gray(product_color)

# Inserção de defeito simulado: arranhão escuro de 10×100 px
defect_image = np.copy(product_gray)
defect_image[100:110, 200:300] = 0.1

# Detecção por subtração e limiarização
difference = np.abs(product_gray - defect_image)
defect_threshold = 0.15 # ajustável conforme a aplicação
detected_defect = (difference > defect_threshold).astype(np.uint8) * 255

mm.show(
    [product_gray, defect_image, detected_defect],
    titles=["Referência", "Com defeito", "Defeito detectado"],
    cols=3,
    figsize=(12, 4)
```

)

```
status = "Defeito detectado." if detected_defect.any() else "Produto conforme."  
print(status)
```



Figura 6.18: Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.

Defeito detectado.

i Subtração de Imagens: Registro Geométrico e Princípio de Funcionamento

A subtração de imagens pressupõe que a imagem de inspeção esteja geometricamente alinhada à imagem de referência. Diferenças de posicionamento, rotação, escala ou perspectiva produzem regiões de diferença que podem ser confundidas com defeitos.

Em aplicações com aquisição controlada, esse alinhamento é obtido durante a captura por meio de gabaritos mecânicos, esteiras e câmeras fixas, permitindo comparar diretamente imagens sucessivas. Um exemplo é a inspeção de uma caixa de ferramentas sempre posicionada na mesma orientação para verificar a ausência de algum item.

Quando esse controle não é possível, emprega-se o **registro de imagens**, que estima uma transformação geométrica para compensar diferenças de translação, rotação, escala e, quando necessário, perspectiva. Após o registro, realiza-se a comparação pixel a pixel entre as duas imagens. Em regiões sem alterações, as diferenças de intensidade tendem a ser próximas de zero; onde existe um defeito, surgem diferenças locais que podem ser destacadas por limiarização. A utilização da diferença absoluta permite detectar tanto defeitos mais claros quanto mais escuros que a referência.

O desempenho do método depende principalmente da qualidade do alinhamento geométrico e da escolha do limiar utilizado para separar pequenas variações de aquisição das diferenças associadas aos defeitos. A Figure 6.19 ilustra o efeito do desalinhamento entre as imagens e a importância do registro geométrico antes da aplicação da subtração.

6.7.2 Detecção de Defeitos por Análise de Textura

Em aplicações nas quais não existe uma imagem de referência, a detecção de defeitos pode ser baseada nas características da textura da superfície. Nesse caso, busca-se identificar regiões cuja aparência difere do padrão predominante do material.

Neste exemplo, utiliza-se a **variância local** como medida de heterogeneidade. Para cada posição da imagem, calcula-se a variância dos níveis de intensidade em uma vizinhança de dimensões fixas. Regiões com baixa variância tendem a apresentar textura mais uniforme, enquanto alterações locais, como riscos, manchas ou imperfeições, podem produzir valores mais elevados dessa medida.

A Figure 6.20 ilustra esse procedimento utilizando a imagem `skimage.data.brick()`. Inicialmente, calcula-se o mapa de variância local por meio de uma janela deslizante. Em seguida, aplica-se uma limiarização para destacar as regiões cuja variância excede o valor especificado, identificando possíveis áreas de interesse para inspeção.



Figura 6.19: Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.

Modelagem Matemática

Considere uma imagem em níveis de cinza representada por

$$f : \Omega \subset \mathbb{Z}^2 \rightarrow \mathbb{R},$$

em que Ω é o domínio da imagem e $f(x, y)$ representa a intensidade do pixel nas coordenadas (x, y) . Em imagens de 8 bits, essas intensidades pertencem ao intervalo $[0, 255]$. Neste exemplo, entretanto, elas foram normalizadas para o intervalo $[0, 1]$, sem alterar o funcionamento do algoritmo.

Para cada posição da imagem, considera-se uma vizinhança quadrada $W_{x,y}$ de dimensão 15×15 pixels.

A média local é dada por

$$\mu(x, y) = \frac{1}{|W_{x,y}|} \sum_{(u,v) \in W_{x,y}} f(u, v),$$

e a variância local é calculada por

$$\sigma^2(x, y) = \frac{1}{|W_{x,y}|} \sum_{(u,v) \in W_{x,y}} f(u, v)^2 - \mu(x, y)^2.$$

Na implementação a seguir, essas duas médias são obtidas pela função `cv2.blur`,

```
mean = cv2.blur(img, (15,15))
mean2 = cv2.blur(img**2, (15,15))
var = mean2 - mean**2
```

Em seguida, calcula-se a diferença entre os mapas de variância da imagem de referência e da imagem inspecionada,

$$D(x, y) = |\sigma_d^2(x, y) - \sigma_r^2(x, y)|,$$

em que $\sigma_r^2(x, y)$ e $\sigma_d^2(x, y)$ são, respectivamente, as variâncias locais da imagem de referência e da imagem contendo o defeito. Após a normalização do mapa $D(x, y)$, aplica-se uma limiarização para obter a máscara das possíveis anomalias.

```
import numpy as np
import cv2
from skimage import data as skdata
from morph import mm

# Imagem de textura uniforme (tijolo)
texture = skdata.brick().astype(np.float32) / 255.0

# Inserção de defeito sintético: mancha clara 20x80 px
texture_defect = np.copy(texture)
texture_defect[60:80, 80:160] = 0.95

# Mapa de variância local (janela 15x15)
def variancia_local(img, ksize=15):
    img_f = img.astype(np.float32)
    mean = cv2.blur(img_f, (ksize, ksize))
    mean2 = cv2.blur(img_f ** 2, (ksize, ksize))
    return np.clip(mean2 - mean ** 2, 0, None)

var_ref = variancia_local(texture)
```



```

var_defect = variancia_local(texture_defect)
diff_var   = np.abs(var_defect - var_ref)

# Normaliza e limiariza
diff_norm = (diff_var / diff_var.max() * 255).astype(np.uint8)
_, mask    = cv2.threshold(diff_norm, 30, 255, cv2.THRESH_BINARY)

mm.show(
    [texture, texture_defect, diff_norm, mask],
    titles=["Textura original", "Com defeito", "Δ variância local", "Anomalia detectada"],
    cols=4,
    figsize=(16, 4)
)

status = "Defeito de textura detectado." if mask.any() else "Superfície conforme."
print(status)

```

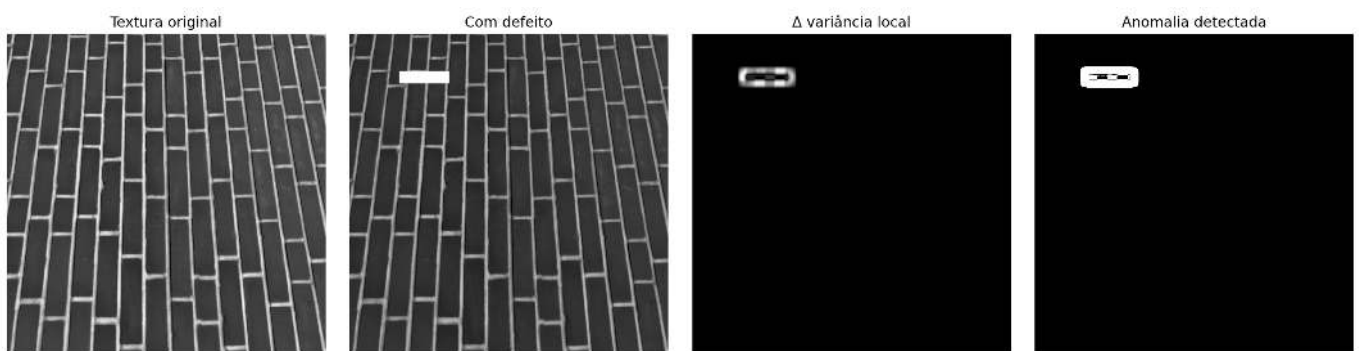


Figura 6.20: Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia.

Defeito de textura detectado.

Por que funciona? — Análise de Textura

A variância local mede a dispersão das intensidades em uma vizinhança da imagem. Em regiões cuja textura permanece uniforme, essa medida tende a variar pouco. Quando um defeito modifica o padrão da superfície, a distribuição das intensidades também se altera, produzindo diferenças na variância local.

Neste capítulo, a detecção é realizada comparando os mapas de variância da imagem de referência e da imagem com defeito. Após a normalização, aplica-se uma limiarização para destacar as regiões em que essa diferença excede um valor especificado.

Os principais parâmetros do método são o tamanho da janela utilizada no cálculo da variância e o limiar empregado na segmentação. Janelas menores são mais sensíveis a detalhes finos, enquanto janelas maiores produzem mapas mais suaves e podem reduzir a resposta a defeitos de pequenas dimensões.

6.8 Resumo

Neste capítulo foram apresentados métodos de Visão Computacional aplicados à análise de documentos e à inspeção visual automatizada. As principais técnicas estudadas foram:

- **Retificação geométrica de documentos:** utilização do detector de bordas de Canny, da Transformada de Hough e de transformações projetivas para corrigir a perspectiva de documentos digitalizados.
- **Pré-processamento de documentos:** aplicação de normalização de fundo, equalização adaptativa (CLAHE) e limiarização por Otsu para reduzir os efeitos de iluminação não uniforme e melhorar a segmentação do texto.

- **Reconhecimento óptico de caracteres (OCR):** conversão de imagens de documentos em texto codificado por meio do Tesseract OCR, evidenciando a influência do pré-processamento na qualidade do reconhecimento.
- **Tradução automática:** aplicação de técnicas de processamento de linguagem natural para traduzir o texto obtido pelo OCR.
- **Localização e retificação de formulários:** emprego de operações morfológicas, análise de contornos e transformação de perspectiva para identificar marcadores de referência e extrair automaticamente regiões de interesse.
- **Leitura de códigos bidimensionais e unidimensionais:** detecção e decodificação de *QR Codes* e códigos de barras para identificação automática de documentos e metadados.
- **Reconhecimento óptico de marcas (OMR):** leitura automatizada de formulários e folhas de respostas, ilustrada por um estudo de caso do sistema MCTest.
- **Inspeção industrial:** detecção de defeitos por comparação com uma imagem de referência e por análise da variância local.

Ao longo do capítulo, os algoritmos foram implementados e avaliados com imagens da biblioteca `skimage.data` e com documentos reais, permitindo reproduzir os experimentos apresentados.

Próximos Passos

Os métodos apresentados neste capítulo mostram como técnicas de Processamento Digital de Imagens, Visão Computacional e Processamento de Linguagem Natural podem ser integradas em *pipelines* para análise automatizada de documentos.

No próximo capítulo serão estudadas técnicas de **extração de características e reconhecimento de padrões**, com ênfase em descritores capazes de representar imagens por atributos numéricos para comparação, classificação e reconhecimento automático. Esses conceitos constituem a base para os capítulos dedicados ao aprendizado de máquina e ao aprendizado profundo aplicados à Visão Computacional.

6.9 Uso do NotebookLM como Tutor Complementar

Como apoio ao estudo deste capítulo, recomenda-se a utilização do **NotebookLM** como tutor complementar. A ferramenta emprega modelos de inteligência artificial para responder perguntas, elaborar resumos e explicar conceitos com base nos documentos fornecidos como fonte de consulta, permitindo ao estudante revisar o conteúdo de forma interativa.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 06](#)

 **Uso Crítico das Respostas**

As respostas geradas pelo NotebookLM são produzidas automaticamente por um modelo de inteligência artificial e podem conter omissões ou imprecisões. Por esse motivo, elas devem ser utilizadas como material de apoio, e não como substitutas do estudo do capítulo.

Sempre que houver dúvidas, consulte o texto deste livro, execute os exemplos apresentados e, quando necessário, complemente a consulta com livros, artigos científicos e outras fontes acadêmicas confiáveis.

6.10 Lista de Exercícios

Os exercícios a seguir consolidam os conceitos apresentados neste capítulo por meio de adaptações, experimentos e extensões dos algoritmos desenvolvidos ao longo do texto.

1. **(10%)** Investigue a influência do ângulo de inclinação na etapa de *deskew*. Gere versões rotacionadas da imagem `skimage.data.page()` para ângulos entre -10° e 10° , aplique o algoritmo apresentado no capítulo e compare o ângulo estimado com o ângulo utilizado na rotação. Apresente os resultados em uma tabela e discuta a precisão do método.
2. **(15%)** Aplique normalização de fundo e CLAHE (utilizando pelo menos três combinações de `clipLimit` e `tileGridSize`) à imagem `skimage.data.page()` degradada artificialmente com gradiente de iluminação e sombra lateral. Segmente cada versão por meio do método de Otsu e compare os resultados utilizando o número de componentes conexos espúrios e a métrica IoU em relação a uma máscara de referência construída manualmente.
3. **(15%)** Investigue a sensibilidade da filtragem por circularidade, $C = \frac{4\pi A}{P^2}$, na detecção dos marcadores circulares. Utilizando o simulador da Figure 6.9, gere discos sintéticos com ruído geométrico crescente e avalie os limiares $C \in \{0,5, 0,6, 0,7, 0,8\}$. Apresente uma tabela relacionando o limiar ao número de falsos positivos e falsos negativos e discuta o compromisso entre sensibilidade e especificidade.
4. **(15%)** A partir dos quatro marcadores detectados, implemente a retificação por perspectiva utilizando `cv2.getPerspectiveTransform` e `cv2.warpPerspective`. Em seguida, perturbe artificialmente as coordenadas dos pontos de controle com ruído gaussiano de desvio-padrão $\sigma \in \{1, 3, 5\}$ pixels e avalie o erro de reprojeção obtido após a homografia inversa.
5. **(15%)** Adapte o *pipeline* de aquisição e retificação desenvolvido neste capítulo para processar documentos contendo códigos de barras lineares em substituição aos *QR Codes*. Rasterize o PDF com `pdf2image` a 300 DPI, aplique o *deskew*, decodifique o símbolo com `pyzbar` e apresente a imagem retificada juntamente com a sequência de caracteres obtida.
6. **(15%)** Estenda a leitura de bolhas do MCTest para identificar três situações: **OK**, **BRANCO** (nenhuma alternativa marcada) e **DUPLA MARCAÇÃO** (duas ou mais alternativas acima de um limiar de preenchimento). Avalie pelo menos três valores desse limiar, apresente os resultados em um `pandas.DataFrame` e discuta sua influência na classificação das respostas.
7. **(15%)** Construa um *pipeline* de inspeção industrial combinando subtração de imagens e análise de variância local da textura sobre um conjunto de imagens sintéticas contendo defeitos simulados. Para cada imagem, gere uma máscara de referência (*ground truth*), calcule a métrica IoU (*Intersection over Union*) das duas abordagens para diferentes limiares de decisão e apresente os resultados em tabelas e visualizações produzidas com `mm.show`.
8. **(Bônus – 10%)** Implemente manualmente a estimação do ângulo de inclinação sem utilizar `cv2.HoughLines` ou `cv2.HoughLinesP`. A partir do mapa de bordas obtido pelo detector de Canny, construa o acumulador da Transformada de Hough, $\rho = x \cos \theta + y \sin \theta$, para $\theta \in [-45^\circ, 45^\circ]$, identifique os máximos do acumulador e estime a inclinação pela mediana das retas detectadas. Compare os resultados com a implementação do OpenCV e discuta a influência de retas espúrias na estimativa final.

Referências do Capítulo

A fundamentação teórica e os estudos de caso apresentados neste capítulo apoiam-se nas seguintes referências:

- Gonzalez; Woods (2018), para os fundamentos de detecção de bordas, limiarização, segmentação, operações morfológicas, reconhecimento óptico de caracteres e transformações geométricas aplicadas à análise de documentos.
- Szeliski (2022), para a Transformada de Hough, o registro e alinhamento de imagens, as transformações projetivas (homografias) e os fundamentos da inspeção visual automatizada.
- Bradski; Kaehler (2008), para a utilização da biblioteca OpenCV nas etapas de detecção de bordas, Transformada de Hough, transformações geométricas, análise de contornos e decodificação de *QR Codes*.
- Smith (2007) e Smith (2013), para a arquitetura, o funcionamento e a evolução do mecanismo de reconhecimento óptico de caracteres **Tesseract OCR**, empregado nos exemplos de OCR apresentados

neste capítulo.

- Bahdanau; Cho; Bengio (2015) e Vaswani et al. (2017), para os fundamentos da tradução automática baseada em redes neurais, incluindo mecanismos de atenção e arquiteturas *transformer*.
- Zampirolli (2023), para a descrição do sistema MCTest, utilizado como estudo de caso de um *pipeline* completo para leitura e correção automatizada de folhas de respostas.
- Bergmann et al. (2019), Tabernik et al. (2020) e Song; Yan (2013), para os *datasets* públicos de inspeção industrial **MVTec AD**, **KolektorSDD** e **NEU Surface Defect Database**, utilizados como referência para avaliação e comparação de algoritmos de detecção de defeitos.



6.11 Parte Prática com Exercícios de Programação

Os exercícios de programação (EP) desta seção complementam os conceitos apresentados ao longo do Capítulo 6 por meio da implementação de algoritmos relacionados à inspeção industrial e à análise de documentos. O objetivo é consolidar os fundamentos estudados, reproduzindo, em escala reduzida, etapas de um *pipeline* típico de Visão Computacional.

Diferentemente dos capítulos anteriores, cujos exercícios enfatizavam operações mais diretamente relacionadas aos dados de imagem, os EPs deste capítulo concentram-se nas **grandezas intermediárias** produzidas durante o processamento, como áreas, perímetros, circularidade, ângulos de retas, graus de preenchimento de bolhas, mapas de variância e mapas de diferença. Essa abordagem permite compreender e validar cada etapa do *pipeline* de forma independente, sem depender de bibliotecas especializadas para aquisição de imagens, detecção de marcadores ou decodificação de códigos — com exceção do exercício de encerramento do capítulo (EP06_08), que propositalmente introduz o uso do OpenCV para a segmentação e a decodificação real de um *QRCode*, fechando o ciclo entre os conceitos teóricos e as ferramentas empregadas na prática.

Os exercícios seguem a mesma sequência conceitual do capítulo, em ordem crescente de complexidade. Inicialmente, são abordadas métricas de avaliação de segmentação, utilizadas para quantificar a qualidade de máscaras binárias. Em seguida, estudam-se critérios geométricos para seleção de marcadores, classificação de marcações em formulários e estimação da inclinação de documentos por meio da Transformada de Hough. Na parte final, os exercícios exploram a normalização de iluminação, a detecção de defeitos por análise de textura e a integração entre registro geométrico e subtração de imagens em um *pipeline* simplificado de inspeção industrial.

Cada exercício representa uma etapa isolada de um sistema real de Visão Computacional, permitindo validar individualmente conceitos que, em aplicações industriais, são combinados em um único *pipeline* de inspeção.

Legenda de Dificuldade

Nível	Significado	EPs
	Muito fácil / fácil — implementação de um único conceito ou algoritmo simples	EP06_01, EP06_02
	Fácil-médio — tratamento de múltiplos casos ou utilização de critérios estatísticos simples	EP06_03, EP06_04
	Médio — processamento matricial ponto a ponto	EP06_05

Nível	Significado	EPs
	Difícil — processamento matricial com operações em vizinhança (janela deslizante)	EP06_06
	Muito difícil — integração de múltiplas etapas de um <i>pipeline</i> de Visão Computacional	EP06_07
	Especial — uso de biblioteca especializada (cv2) para segmentação geométrica e decodificação real de código de barras/QRCode	EP06_08

! Diretrizes para a Resolução dos Exercícios de Programação

Salvo indicação em contrário, todos os exercícios utilizam a convenção de coordenadas matriciais [linha][coluna], com origem em (0,0) no canto superior esquerdo da imagem. Quando houver necessidade de arredondamento numérico, deve-se utilizar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*, com `np.floor(img + 0.5)`). Comparações com limiares (por exemplo, circularidade, variância, diferença de intensidade ou grau de preenchimento) devem ser consideradas **estritas** ($>$), exceto quando o enunciado especificar explicitamente outro critério. Cada exercício foi elaborado para enfatizar um conceito específico apresentado no capítulo. Recomenda-se implementar inicialmente a solução de forma direta e, somente após sua validação, buscar alternativas mais eficientes ou mais gerais.

🎯 Objetivo deste Caderno

Este caderno foi elaborado para apoiar o desenvolvimento, a validação e os testes das soluções dos **Exercícios de Programação (EPs)** em um ambiente interativo, como o Google Colab ou o Jupyter Notebook. Após verificar o funcionamento da implementação com os casos de teste apresentados, o código pode ser submetido ao Moodle para a avaliação oficial.

Download

Execute a célula a seguir para obter os arquivos `morph.py` e `testsuite.py`, utilizados pelos exercícios deste capítulo.

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Após implementar a solução, execute `TestSuite("EP06_01.extensão").run()` em uma nova célula, substituindo `extensão` pela linguagem utilizada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema obtém automaticamente os casos de teste do repositório do curso, executa o programa e apresenta o resultado da avaliação.

Em Python, também é possível testar a solução diretamente a partir de uma *string*, sem a necessidade de salvar o código em um arquivo. Para isso, armazene o programa em uma variável e utilize o método `run_code`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP06_01").run_code(codigo)
```

6.11.1 EP06_01 Avaliação de Segmentação por IoU (*Intersection over Union*)

Ao longo deste capítulo, diversas etapas do *pipeline* produzem **máscaras binárias**, como na segmentação de documentos, na localização de *QR Codes* e na detecção de defeitos. Para avaliar objetivamente a qualidade dessas segmentações, é necessário compará-las com uma máscara de referência (*ground truth*).

Uma das métricas mais utilizadas para esse fim é a **IoU** (*Intersection over Union*, ou Interseção sobre União), definida como a razão entre a área de interseção e a área de união de duas máscaras binárias. Quanto maior o valor da IoU, maior a concordância entre a segmentação produzida pelo algoritmo e a referência.

6.11.1.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (número de linhas) e C (número de colunas).
2. **Máscara de referência:** Ler os $L \times C$ elementos binários (0 ou 1) da matriz `ref`.
3. **Máscara predita:** Ler os $L \times C$ elementos binários (0 ou 1) da matriz `pred`.
4. **Interseção:** Contar o número de posições (i, j) para as quais `ref[i][j] = 1` e `pred[i][j] = 1`.
5. **União:** Contar o número de posições (i, j) para as quais `ref[i][j] = 1` ou `pred[i][j] = 1`.
6. **Caso degenerado:** Se a união for igual a 0, definir $\text{IoU} = 1,0$, pois ambas as máscaras são vazias.
7. **Cálculo:** Caso a união seja maior que zero, calcular

$$\text{IoU} = \frac{|\text{Intersecao}|}{|\text{Uniao}|}.$$

8. **Classificação:** Determinar a classificação qualitativa utilizando o valor de IoU **antes** do arredondamento.
9. **Arredondamento:** Exibir a IoU com quatro casas decimais.
10. **Saída:** Imprimir, nessa ordem, a interseção, a união, a IoU e a classificação.

6.11.1.2 Restrições Computacionais

- Se a união for igual a 0, não deve ser realizada a divisão; a IoU deve ser definida como 1,0.
- As faixas de classificação utilizam comparações não estritas ($\geq$).
- A classificação deve ser realizada utilizando o valor da IoU em precisão completa, antes do arredondamento para exibição.

6.11.1.3 Fundamentação Teórica

A IoU é definida por

$$\text{IoU} = \frac{|R \cap P|}{|R \cup P|},$$

em que:

- R representa o conjunto de pixels pertencentes à máscara de referência;
- P representa o conjunto de pixels pertencentes à máscara predita;
- $|R \cap P|$ corresponde ao número de pixels pertencentes simultaneamente às duas máscaras;
- $|R \cup P|$ corresponde ao número de pixels pertencentes a pelo menos uma das máscaras.

Faixa de IoU	Classificação	Interpretação
$\text{IoU} \geq 0,90$	EXCELENTE	Concordância muito elevada entre as máscaras.
$0,70 \leq \text{IoU} < 0,90$	BOM	Pequenas diferenças entre as máscaras.
$0,50 \leq \text{IoU} < 0,70$	ACEITAVEL	Concordância parcial entre as máscaras.
$\text{IoU} < 0,50$	RUIM	Baixa concordância entre as máscaras.

A IoU depende apenas da sobreposição entre as máscaras e, portanto, é independente do tamanho da imagem.

6.11.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L .
- Linha 2: inteiro C .
- Próximas L linhas: elementos binários (0 ou 1) da matriz **ref**.
- Próximas L linhas: elementos binários (0 ou 1) da matriz **pred**.

Saída:

- Linha 1: **Intersecao:** X
- Linha 2: **Uniao:** Y
- Linha 3: **IoU:** Z
- Linha 4: **Classificacao:** NOME

O valor de IoU deve ser impresso com quatro casas decimais.

6.11.1.5 Exemplos

Entrada	Saída	Observação
2	Intersecao: 1	A metade da região de referência foi corretamente segmentada.
2	Uniao: 2	
1 1	IoU: 0.5000	
0 0	Classificacao: ACEITAVEL	
1 0		
0 0		

Entrada	Saída	Observação
2	Intersecao: 0	Ambas as máscaras são vazias; por convenção, IoU = 1,0.
2	Uniao: 0	
0 0	IoU: 1.0000	
0 0	Classificacao: EXCELENTE	
0 0		



Figura 6.21: Simulador: IoU entre máscara de referência e máscara predita

```
%%writefile EP06_01.py
# Código Python
```

```
Overwriting EP06_01.py
```

```
TestSuite("EP06_01.py").run()
```

6.11.2 EP06_02 ● Filtro de Marcadores por Circularidade

Após a segmentação de uma imagem, é comum que diversos componentes conexos sejam identificados. Em aplicações como a retificação de documentos, apenas alguns desses componentes correspondem aos marcadores de referência utilizados para o alinhamento da imagem. Um critério frequentemente empregado para selecionar esses marcadores é a **circularidade**, que mede o quão próxima a forma de um componente está de um círculo.

Neste exercício, cada componente é descrito por sua área A e seu perímetro P . O objetivo é calcular sua circularidade e decidir, a partir de um limiar fornecido, se o componente deve ser aceito ou rejeitado como candidato a marcador.

6.11.2.1 📄 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro N (número de candidatos) e o limiar de circularidade C_{limiar} (número real).
2. **Dados dos candidatos:** Para cada um dos N candidatos, ler a área A (inteiro) e o perímetro P (número real).
3. **Circularidade:** Calcular $C = \frac{4\pi A}{P^2}$, em que:
 - A é a área do componente;

- P é o perímetro do componente;
 - C é a circularidade.
4. **Caso degenerado:** Se $P = 0$, considerar $C = 0$ e classificar diretamente o candidato como REJEITADO.
 5. **Classificação:** Se $C > C_{\text{limiar}}$, classificar o candidato como ACEITO; caso contrário, classificá-lo como REJEITADO.
 6. **Arredondamento:** Exibir o valor de C com quatro casas decimais.
 7. **Saída:** Para cada candidato, imprimir o valor de C seguido da classificação. Ao final, imprimir o número total de candidatos aceitos.

6.11.2.2 Restrições Computacionais

- Utilizar a constante π da biblioteca padrão da linguagem (por exemplo, `math.pi`), sem aproximações.
- A comparação deve ser realizada com o valor de C em precisão completa, antes do arredondamento para exibição.
- O critério de aceitação é estrito ($C > C_{\text{limiar}}$).
- Se $P = 0$, a divisão não deve ser realizada.

6.11.2.3 Fundamentação Teórica

A circularidade é um descritor geométrico definido por $C = \frac{4\pi A}{P^2}$, em que:

- A é a área do componente;
- P é o perímetro do componente;
- C é a circularidade.

Para um círculo perfeito, $C = 1$. À medida que a forma se torna mais alongada ou irregular, o perímetro cresce mais rapidamente que a área, reduzindo o valor de C .

Forma	Circularidade aproximada	Interpretação
Círculo	1,0000	Forma circular.
Quadrado	0,7854	Forma aproximadamente compacta.
Forma alongada ou irregular	$C \ll 1$	Baixa circularidade.
$P = 0$	0 (convenção adotada)	Contorno degenerado.

A circularidade é invariante à translação, à rotação e à escala, sendo amplamente utilizada para distinguir componentes aproximadamente circulares de outros formatos.

6.11.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro N .
- Linha 2: número real C_{limiar} .
- Próximas N linhas: área A (inteiro) e perímetro P (real), separados por espaço.

Saída:

- Uma linha para cada candidato, no formato `C ACEITO` ou `C REJEITADO`, com C apresentado com quatro casas decimais.
- Última linha: `Total aceitos: X`.

6.11.2.5 Exemplos

Entrada	Saída	Observação
3	0.9941 ACEITO	Candidato aproximadamente circular, forma compacta e forma alongada.
0.6	0.7854 ACEITO	
78 31.4	0.1745 REJEITADO	
100 40	Total aceitos: 2	
50 60		
1	0.0000 REJEITADO	Perímetro nulo: contorno degenerado.
0.9	Total aceitos: 0	
10 0		

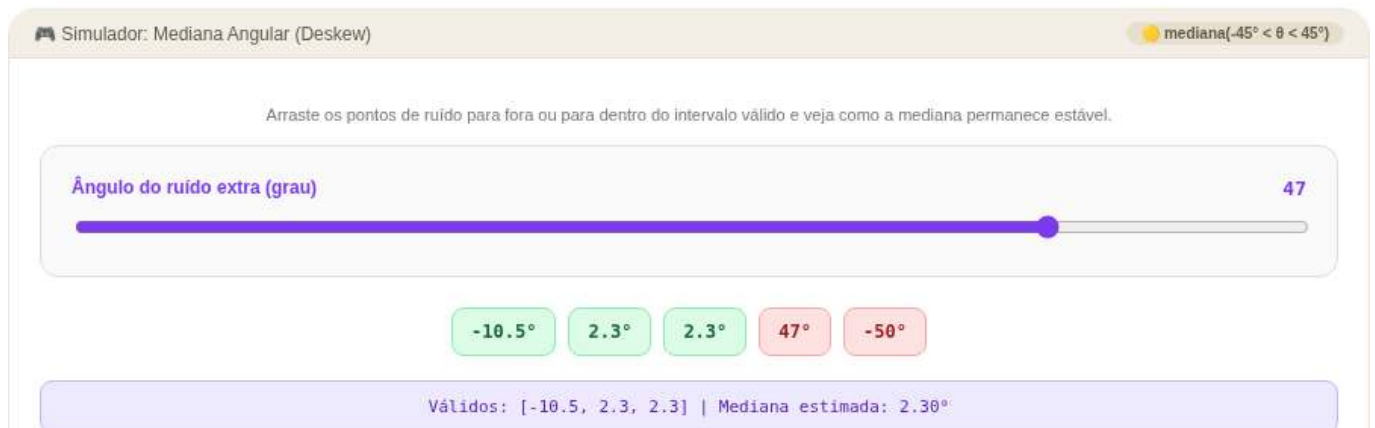


Figura 6.22: Simulador: Filtro de Marcadores por Circularidade

```
%%writefile EP06_02.py
# Código Python
```

```
Overwriting EP06_02.py
```

```
TestSuite("EP06_02.py").run()
```

6.11.3 EP06_03 ● Classificação de Marcações em Folhas de Resposta (OMR)

Após a retificação da folha e a segmentação dos quadros de respostas, o MCTest estima, para cada bolha, um **grau de preenchimento**, representado por um valor entre 0 e 100. A partir desses valores, o sistema deve determinar automaticamente a alternativa marcada, identificando também questões em branco e casos de múltiplas marcações.

Neste exercício, você implementará essa etapa de decisão do *pipeline* de OMR. A classificação depende de um limiar de preenchimento: pequenas variações nesse valor podem alterar o resultado da leitura automática.

6.11.3.1 📋 Diretrizes de Implementação

1. **Parâmetros:** Ler os inteiros Q (número de questões) e K (número de alternativas por questão, com $2 \leq K \leq 26$) e o limiar de preenchimento Th (número real entre 0 e 100).
2. **Graus de preenchimento:** Para cada uma das Q questões, ler os K valores reais correspondentes às alternativas A, B, C, ..., na ordem de entrada.
3. **Contagem de marcações:** Para cada questão, contar quantas alternativas possuem grau de preenchimento **estritamente maior** que Th .
4. **Classificação:**
 - Se nenhuma alternativa exceder Th , classificar a questão como BRANCO.

- Se exatamente uma alternativa exceder Th , imprimir a letra correspondente (A, B, C, ...).
 - Se duas ou mais alternativas excederem Th , classificar a questão como DUPLA_MARCACAO.
5. **Saída por questão:** Imprimir, na ordem de leitura, a classificação de cada questão.
6. **Totais:** Ao final, imprimir o número de questões OK (uma única marcação), BRANCO e DUPLA_MARCACAO.

6.11.3.2 Restrições Computacionais

- **Comparação estrita:** apenas valores maiores que Th são considerados marcações válidas; valores exatamente iguais ao limiar não devem ser contabilizados.
- **Letras das alternativas:** o índice 0 corresponde à alternativa A, o índice 1 à alternativa B e assim sucessivamente.
- **Múltiplas marcações:** sempre que duas ou mais alternativas excederem o limiar, a classificação deve ser DUPLA_MARCACAO, independentemente dos respectivos graus de preenchimento.

6.11.3.3 Fundamentação Teórica

Situação	Classificação	Interpretação
Exatamente uma alternativa acima do limiar	Letra da alternativa	Resposta válida
Nenhuma alternativa acima do limiar	BRANCO	Questão não respondida
Duas ou mais alternativas acima do limiar	DUPLA_MARCACAO	Resposta ambígua

O limiar de preenchimento controla a sensibilidade do algoritmo. Valores muito baixos tendem a aumentar o número de DUPLA_MARCACAO, enquanto valores muito altos podem aumentar a quantidade de questões classificadas como BRANCO.

6.11.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro Q .
- Linha 2: Inteiro K .
- Linha 3: Número real Th .
- Próximas Q linhas: K números reais, correspondentes aos graus de preenchimento das alternativas.
- Linha 1: Inteiro Q e K .

Saída:

- Q linhas, cada uma contendo a classificação da respectiva questão.
- Linha final: OK: x BRANCO: y DUPLA_MARCACAO: z .

6.11.3.5 Exemplos

Entrada	Saída	Observação
3	B	Na primeira questão apenas B supera o limiar; na segunda nenhuma alternativa o supera; na terceira, A e B excedem o limiar.
4	BRANCO	
50	DUPLA_MARCACAO	
10 85 5 12	OK: 1 BRANCO: 1	
20 15 18 22	DUPLA_MARCACAO: 1	
90 88 10 5		

Entrada	Saída	Observação
1	BRANCO	Valores iguais ao limiar não são considerados marcações válidas.
2	OK: 0 BRANCO: 1	
50.0	DUPLA_MARCACAO: 0	
50 50		



Figura 6.23: Simulador: Classificação de Marcações OMR

```
%%writefile EP06_03.py
# Código Python
```

```
Overwriting EP06_03.py
```

```
TestSuite("EP06_03.py").run()
```

6.11.4 EP06_04 ● Estimador de Inclinação por Mediana Angular (*Deskew*)

Após a detecção de bordas e a aplicação da Transformada de Hough, obtém-se um conjunto de retas candidatas à orientação predominante do documento. Cada reta fornece uma estimativa do ângulo de inclinação, calculada por

$$\text{ângulo} = \text{rad2deg}(\theta) - 90.$$

Entretanto, nem todas as retas correspondem às linhas do documento: algumas resultam de ruídos, sombras ou outros elementos da imagem. Neste exercício, você implementará a etapa de estimação robusta do ângulo de inclinação, filtrando os valores plausíveis e calculando sua mediana.

6.11.4.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro M , correspondente ao número de ângulos estimados.
2. **Ângulos:** Ler os M valores reais, em graus.
3. **Filtragem:** Manter apenas os ângulos que satisfaçam **estritamente** $-45 < \text{ângulo} < 45$.
4. **Ausência de candidatos:** Se nenhum ângulo permanecer após a filtragem, imprimir exatamente SEM_CORRECAO.
5. **Mediana:** Caso existam ângulos válidos:

- se a quantidade for ímpar, a mediana é o elemento central da sequência ordenada;
 - se for par, a mediana é a média aritmética dos dois elementos centrais.
6. **Saída:** Imprimir a mediana arredondada para duas casas decimais (arredondamento padrão, *round half away from zero*, com `np.floor(img + 0.5)`).

6.11.4.2 Restrições Computacionais

- **Intervalo aberto:** ângulos iguais a -45 ou 45 não devem ser considerados.
- **Precisão:** calcular a mediana utilizando os valores originais; o arredondamento deve ser realizado apenas na saída.
- **Caso vazio:** se não houver ângulos válidos, nenhuma mediana deve ser calculada.

6.11.4.3 Fundamentação Teórica

Situação	Resultado
Maioria dos ângulos concentrada em torno da inclinação real	A mediana aproxima a orientação do documento.
Poucos ângulos discrepantes (<i>outliers</i>)	A mediana sofre pouca influência desses valores.
Ângulos fora do intervalo ($-45^\circ, 45^\circ$)	São descartados antes do cálculo.
Nenhum ângulo válido	Não é aplicada correção (<code>SEM_CORRECAO</code>).

A mediana é utilizada por ser mais robusta que a média na presença de poucos valores discrepantes, produzindo uma estimativa mais estável da inclinação predominante do documento.

6.11.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro M .
- Linha 2: M números reais, correspondentes aos ângulos em graus.

Saída:

- Uma única linha contendo o ângulo estimado, com duas casas decimais, ou a palavra `SEM_CORRECAO` caso nenhum ângulo seja válido.

6.11.4.5 Exemplos

Entrada	Saída	Observação
5 -50 -10.5 2.3 2.3 47	2.30	Apenas os ângulos no intervalo $(-45, 45)$ são considerados; a mediana é 2,3.
4 -46 50 45 -45	<code>SEM_CORRECAO</code>	Nenhum ângulo pertence ao intervalo aberto $(-45, 45)$.

```
%%writefile EP06_04.py
# Código Python
```

```
Overwriting EP06_04.py
```

```
TestSuite("EP06_04.py").run()
```

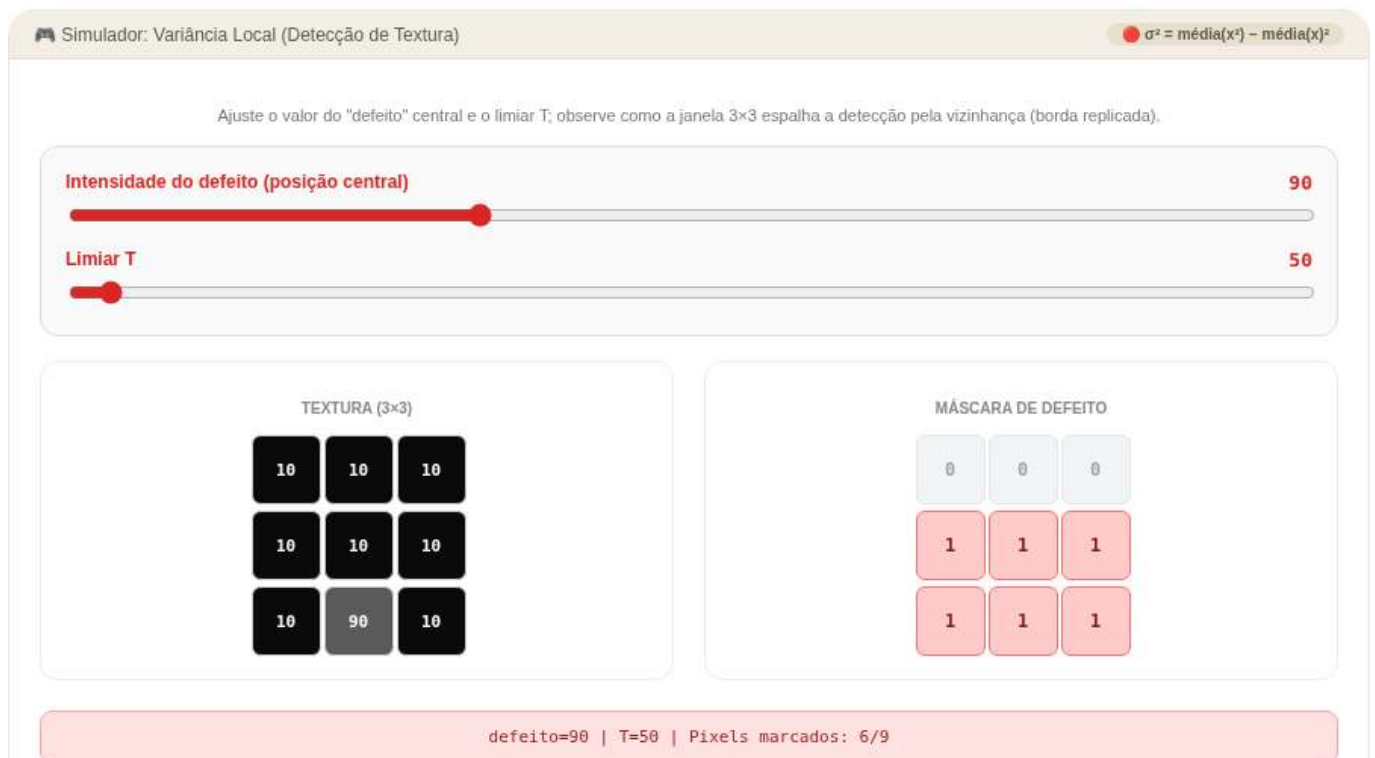


Figura 6.24: Simulador: Estimador de Inclinação por Mediana Angular

6.11.5 EP06_05 ● Normalização de Fundo por Divisão (Correção de Iluminação)

Um formulário foi fotografado sob iluminação não uniforme, fazendo com que um lado da folha apareça mais claro que o outro. Nessas condições, a limiarização global por Otsu pode produzir resultados insatisfatórios, pois um único limiar não separa adequadamente texto e fundo em toda a imagem. A solução apresentada no capítulo consiste em **normalizar o fundo**, dividindo a imagem original por uma versão fortemente suavizada de si mesma, que representa a iluminação de baixa frequência.

Neste exercício, a imagem original e o fundo suavizado (equivalente ao resultado de um `cv2.GaussianBlur` com σ elevado) já são fornecidos. Sua tarefa é implementar a etapa de normalização que produz a imagem corrigida.

6.11.5.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Imagem original:** Ler os $L \times C$ valores inteiros da matriz `img` (intensidades entre 0 e 255).
3. **Fundo estimado:** Ler os $L \times C$ valores inteiros da matriz `bg` (intensidades entre 0 e 255, sempre estritamente maiores que zero).
4. **Normalização:** Para cada posição (i, j) , calcular

$$\text{valor}(i, j) = \frac{\text{img}(i, j)}{\text{bg}(i, j)} \times 255.$$

5. **Arredondamento:** Arredondar o resultado para o inteiro mais próximo (*round half away from zero*, com `np.floor(img + 0.5)`).
6. **Saturação:** Limitar o valor obtido ao intervalo $[0, 255]$.
7. **Saída:** Imprimir a matriz `img_norm` resultante.

6.11.5.2 📌 Restrições Computacionais

- **Divisão por zero:** a entrada garante $\text{bg}(i, j) > 0$ em todas as posições.

- **Ordem das operações:** primeiro arredondar, depois aplicar a saturação.
- **Processamento independente:** cada pixel deve ser normalizado individualmente, sem utilizar informações dos pixels vizinhos.

6.11.5.3 Fundamentação Teórica

Situação	Efeito da normalização
$\text{img}(i, j) = \text{bg}(i, j)$	Resultado igual a 255, correspondente ao fundo normalizado.
$\text{img}(i, j) < \text{bg}(i, j)$	Resultado menor que 255, preservando regiões mais escuras, como texto.
$\text{img}(i, j) > \text{bg}(i, j)$	Resultado superior a 255, posteriormente saturado.
Fundo com iluminação não uniforme	A divisão reduz as variações lentas de iluminação, tornando a imagem mais homogênea.

A divisão pelo fundo estimado reduz os efeitos da iluminação não uniforme e preserva o contraste entre o primeiro plano e o fundo, facilitando as etapas posteriores de segmentação.

6.11.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Próximas L linhas: elementos da matriz `img`.
- Próximas L linhas: elementos da matriz `bg`.

Saída:

- Matriz `img_norm`, com L linhas e C colunas, contendo valores inteiros separados por espaço.

6.11.5.5 Exemplos

Entrada	Saída	Observação
2	153 255	Valores superiores a 255 devem ser saturados; $180/200 \times 255 = 229,5$ resulta em 230 após o arredondamento.
2	230 128	
60 120		
180 40		
100 100		
200 80		Apenas o primeiro valor permanece abaixo de 255 após a normalização.
1	128 255 255	
3		
30 60 90		
60 60 60		

```
%%writefile EP06_05.py
# Código Python
```

```
Overwriting EP06_05.py
```

```
TestSuite("EP06_05.py").run()
```




Figura 6.25: Simulador: Normalização de Fundo por Divisão

6.11.6 EP06_06 ● Mapa de Variância Local para Detecção de Textura

Uma fábrica de tecidos precisa inspecionar rolos de pano em tempo real, sem dispor de uma imagem de referência — cada rolo apresenta pequenas variações naturais. Nessa situação, a estratégia apresentada no capítulo consiste em analisar a **homogeneidade local da textura**: regiões uniformes apresentam baixa variância de intensidade em pequenas vizinhanças, enquanto riscos, manchas e falhas de fabricação produzem aumentos locais dessa variância.

Neste exercício, você implementará o núcleo desse método, calculando a variância local em uma janela deslizante e gerando uma máscara binária que identifica as regiões cuja variância excede um limiar.

6.11.6.1 📋 Diretrizes de Implementação

1. **Dimensões e parâmetros:** Ler os inteiros L , C , k (tamanho da janela, sempre ímpar) e T (limiar de variância).
2. **Imagem:** Ler os $L \times C$ valores inteiros da matriz de textura (intensidades entre 0 e 255).
3. **Tratamento das bordas:** Quando a janela ultrapassar os limites da imagem, utilizar **replicação de borda**, isto é, repetir o valor do pixel válido mais próximo.
4. **Média local:** Para cada posição (i, j) , calcular

$$\mu(i, j) = \frac{1}{k^2} \sum_{(p, q) \in \text{janela}} \text{textura}(p, q).$$

5. **Variância local:** Calcular a variância populacional da janela,

$$\sigma^2(i, j) = \frac{1}{k^2} \sum_{(p, q) \in \text{janela}} (\text{textura}(p, q) - \mu(i, j))^2,$$

ou, de forma equivalente,

$$\sigma^2(i, j) = \overline{x^2} - \mu(i, j)^2,$$

em que $\overline{x^2}$ representa a média dos quadrados das intensidades.

6. **Arredondamento:** Arredondar a variância para o inteiro mais próximo (*round half away from zero*, com `np.floor(res_norm + 0.5)`).
7. **Limiarização:** Definir $máscara(i, j) = 1$ se a variância arredondada for **estritamente maior** que T ; caso contrário, definir $máscara(i, j) = 0$.
8. **Saída:** Imprimir a máscara binária resultante.

6.11.6.2 Restrições Computacionais

- **Replicação de borda:** utilizar o valor do pixel válido mais próximo sempre que a janela ultrapassar os limites da imagem.
- **Variância populacional:** utilizar denominador k^2 , nunca $k^2 - 1$.
- **Comparação estrita:** a máscara deve ser calculada utilizando a condição $\sigma_{\text{arred}}^2 > T$.
- **Janela ímpar:** o valor de k é sempre ímpar, garantindo um pixel central.

6.11.6.3 Fundamentação Teórica

Situação	Variância local	Interpretação
Região uniforme	Baixa	Intensidades semelhantes na vizinhança.
Região contendo defeito	Alta	A presença de intensidades distintas aumenta a dispersão dos valores.
Janela pequena	Maior sensibilidade a detalhes e ruído	Detecta alterações localizadas.
Janela grande	Resposta mais suave	Evidencia defeitos maiores, porém reduz a precisão de sua localização.

A variância local mede a dispersão das intensidades em uma vizinhança. Regiões homogêneas apresentam baixa variância, enquanto alterações na textura aumentam essa medida, permitindo identificar possíveis defeitos por meio de uma simples limiarização.

6.11.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro k (ímpar).
- Linha 4: Inteiro T .
- Próximas L linhas: elementos inteiros da matriz de textura.

Saída:

- Máscara binária (valores 0 ou 1), com L linhas e C colunas.

6.11.6.5 Exemplos

Entrada	Saída	Observação
3	0 0 0	O defeito aumenta a variância em todas as janelas que o contêm.
3	1 1 1	
3	1 1 1	
50		
10 10 10		
10 10 10		A textura é uniforme; a variância é nula em toda a imagem.
10 90 10		
2	0 0	
2	0 0	
3		
5		
100 100		
100 100		



Figura 6.26: Simulador: Mapa de Variância Local para Detecção de Textura

```
%%writefile EP06_06.py
# Código Python

Overwriting EP06_06.py

TestSuite("EP06_06.py").run()
```

6.11.7 EP06_07 Pipeline de Inspeção Industrial: Registro por Translação e Subtração

Em uma linha de produção, uma câmera fixa fotografa cada peça que passa pela esteira, comparando-a a uma imagem de referência sem defeitos. O problema: pequenas vibrações da esteira deslocam a peça em relação à posição de referência a cada captura. Se a subtração de imagens for aplicada diretamente, sem correção, o deslocamento por si só já gera diferenças enormes — **falsos positivos** que mascaram os defeitos reais.

Este é o exercício mais completo do capítulo: você deve **primeiro registrar** (alinhar geometricamente) a imagem capturada usando um deslocamento conhecido (dx, dy) , fornecido por um sensor de posição da esteira, e **só então aplicar a subtração** com limiarização, exatamente como descrito na seção de inspeção industrial.

6.11.7.1 Diretrizes de Implementação

1. **Dimensões e parâmetros:** Ler L, C (dimensões das imagens), o deslocamento inteiro conhecido dx, dy (podendo ser negativos) e o limiar de detecção T (inteiro).
2. **Imagens:** Ler a matriz de referência (**ref**, $L \times C$, sem defeitos) e a matriz capturada (**cap**, $L \times C$, possivelmente deslocada e com defeito).
3. **Registro por translação:** Construir a imagem alinhada **alin** aplicando o deslocamento (dx, dy) recebido:

$$\text{alin}(i, j) = \begin{cases} \text{cap}(i + dy, j + dx), & \text{se } (i + dy, j + dx) \in [0, L) \times [0, C) \\ 0, & \text{caso contrário} \end{cases}$$

4. **Preenchimento de borda:** As posições que “saem” da imagem capturada após o deslocamento recebem o valor **0** (*zero-padding* — fora do campo de visão da câmera; **note que este exercício usa zero, diferente da replicação de borda do EP06_06**).
5. **Diferença absoluta:** Calcular, pixel a pixel,

$$\text{diff}(i, j) = |\text{ref}(i, j) - \text{alin}(i, j)|$$

6. **Limiarização:** Definir $\text{máscara}(i, j) = 1$ se $\text{diff}(i, j) > T$; caso contrário, $\text{máscara}(i, j) = 0$.
7. **Saída:** Nesta ordem — (a) a matriz **alin** ($L \times C$); (b) a máscara de defeito ($L \times C$); (c) uma última linha com o total de pixels classificados como defeituosos.

6.11.7.2 Restrições Computacionais

- **Zero-padding, não replicação:** posições fora dos limites da imagem capturada, após o deslocamento, valem exatamente 0 — este é o ponto que mais diferencia este exercício do EP06_06.
- **Comparação estrita:** $\text{diff}(i, j) > T$.
- **Sinal de (dx, dy) :** o deslocamento pode ser positivo ou negativo; a fórmula do passo 3 deve ser aplicada literalmente, sem inverter os sinais.
- **Todos os valores são inteiros:** não há arredondamento nesta etapa.

6.11.7.3 Fundamentação Teórica

Etapa omitida	Consequência
Pular o registro geométrico	A borda inteira da imagem (introduzida pelo deslocamento) é marcada como “defeito” — falso positivo sistemático
Registro com (dx, dy) incorreto	Peça e referência ficam desalinhadas; a subtração detecta contornos deslocados, não defeitos reais
Limiar T muito baixo	Ruído de captura (variações de 1–2 níveis de cinza) é confundido com defeito
Limiar T muito alto	Defeitos sutis deixam de ser detectados

O registro geométrico e a subtração são etapas complementares: o primeiro garante que ambas as imagens representem exatamente a mesma cena no mesmo referencial espacial; o segundo isola o que realmente mudou entre elas — idealmente, apenas os defeitos.

6.11.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Dois inteiros dx e dy , separados por espaço.
- Linha 4: Inteiro T .
- Próximas L linhas: elementos inteiros da matriz `ref`.
- Próximas L linhas: elementos inteiros da matriz `cap`.

Saída:

- L linhas com a matriz `alin`.
- L linhas com a máscara de defeito (0/1).
- Última linha: Total de pixels defeituosos: X .

6.11.7.5 Exemplos

Entrada	Saída	Observação
3	50 50 0	$dx = 1$ desloca a leitura uma coluna à direita; a última coluna de <code>alin</code> fica sem correspondência (vira 0) e é sistematicamente marcada; o defeito real (90) também é detectado.
3	50 90 0	
1 0	50 50 0	
30	0 0 1	
50 50 50	0 1 1	
50 50 50	0 0 1	
50 50 50	Total de pixels defeituosos: 4	
0 50 50		
0 50 90		
0 50 50		
2	10 10	Sem deslocamento ($dx = dy = 0$): <code>alin</code> é idêntica a <code>cap</code> ; apenas o defeito real (60) é detectado.
2	10 60	
0 0	0 0	
20	0 1	
10 10	Total de pixels defeituosos: 1	
10 10		
10 10		
10 60		

```
%%writefile EP06_07.py
# Código Python
```

```
Overwriting EP06_07.py
```

```
TestSuite("EP06_07.py").run()
```

6.11.8 EP06_08 Segmentação e Decodificação Real de *QRCode* com OpenCV

Nos exercícios anteriores, as grandezas intermediárias do *pipeline* de processamento de imagens — como áreas, perímetros, variâncias e deslocamentos — foram fornecidas diretamente ou calculadas a partir de

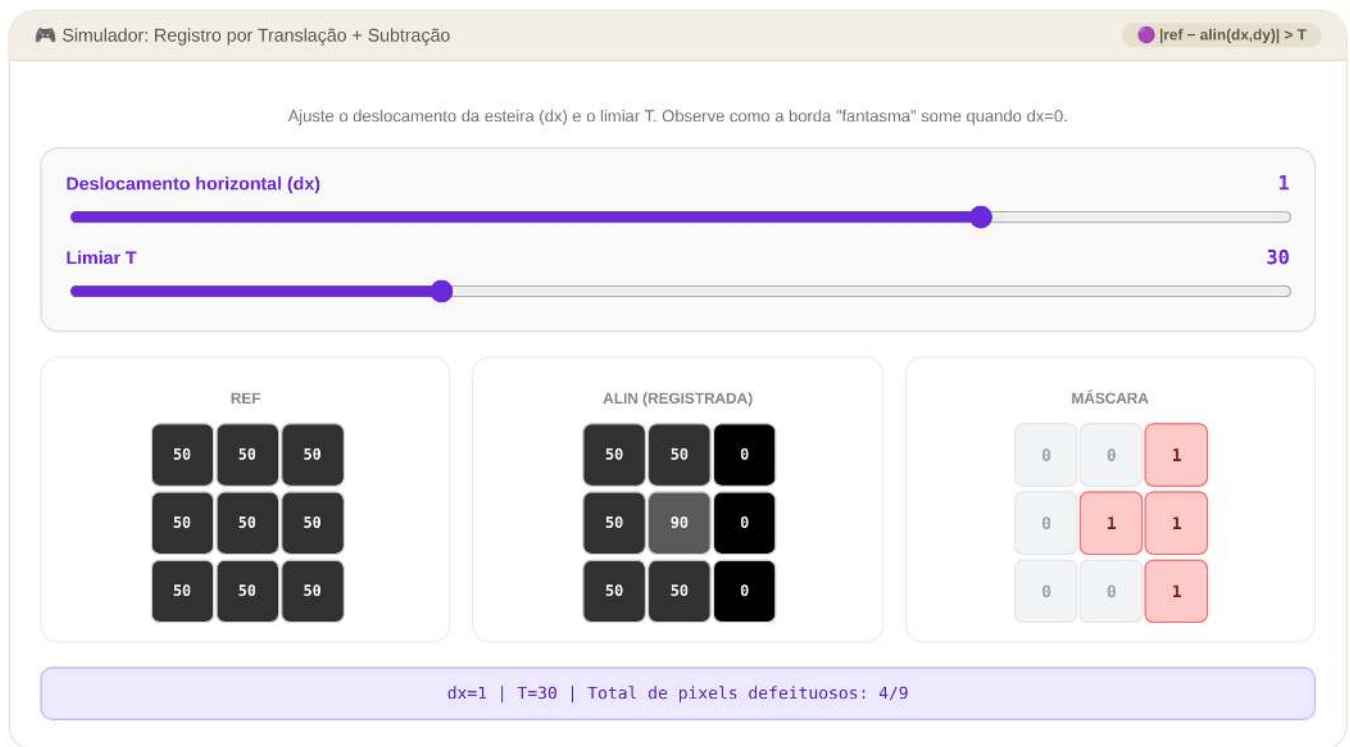


Figura 6.27: Simulador: *Pipeline* de Inspeção — Registro por Translação e Subtração

matrizes numéricas, sem a necessidade de bibliotecas especializadas de Visão Computacional. Neste exercício de encerramento do capítulo, essa restrição é removida de forma intencional: será utilizada a biblioteca **OpenCV** (`cv2`) para localizar e decodificar um *QRCode* real presente em uma cena.

A proposta reproduz um fluxo simplificado de sistemas empregados em inspeção visual, automação industrial e leitura automática de documentos. Para manter a entrada de dados acessível ao contexto educacional, o carregamento da imagem será integrado à biblioteca didática **morph**, por meio da função `mm.readImg`.

A cena é fornecida no formato **PGM ASCII (P2)** e contém um único *QRCode* válido, além de diversos **objetos distratores**, como retângulos, regiões de ruído texturizado e blocos isolados. A segmentação baseada apenas em propriedades geométricas — como área e formato aproximadamente quadrado — é necessária para reduzir o espaço de busca, mas não é suficiente para identificar o código correto. A confirmação final será realizada exclusivamente pela tentativa de decodificação utilizando `cv2.QRCodeDetector`, procedimento compatível com aplicações reais de reconhecimento automático.

6.11.8.1 Diretrizes de Implementação

1. Leitura das dimensões e parâmetros

Ler, nesta ordem, por meio da entrada padrão:

- uma linha contendo o número de linhas L ;
- uma linha contendo o número de colunas C ;
- uma linha contendo os quatro parâmetros do algoritmo separados por espaço:
 - limiar de binarização T (inteiro);
 - área mínima $A_{\min}$ (inteiro);
 - tolerância de aspecto tol (real);
 - margem M (inteiro, em pixels).

2. Carregamento da imagem

Utilizar a função `cv2.imread`

```
f = mm.readImg(L, C)
```

para ler os $L \times C$ valores da imagem em tons de cinza, obtendo um *array* NumPy do tipo `uint8`.

3. Binarização

Aplicar limiarização binária invertida utilizando o limiar T . Todo pixel da imagem original com intensidade estritamente maior que T deve ser convertido para 255, enquanto os demais devem assumir o valor 0.

4. Detecção de contornos

Extrair os componentes conectados externos utilizando

```
cv2.findContours(...)
```

com os parâmetros:

- `cv2.RETR_EXTERNAL`;
- `cv2.CHAIN_APPROX_SIMPLE`.

5. Filtragem geométrica

Para cada contorno encontrado:

- calcular o retângulo delimitador (x, y, w, h) por meio de `cv2.boundingRect`;
- manter apenas os candidatos que satisfaçam simultaneamente:

Área mínima

$$w \times h > A_{\min}$$

Razão de aspecto

$$\left| \frac{w}{h} - 1 \right| \leq \text{tol}$$

6. Ordenação dos candidatos

Ordenar os candidatos pela área do retângulo delimitador

$$w \times h$$

em ordem decrescente.

Em caso de empate, preservar a ordem originalmente retornada por `cv2.findContours`.

7. Verificação por decodificação

Para cada candidato, seguindo a ordem estabelecida:

- expandir o retângulo em M pixels nas quatro direções;
- limitar os índices para permanecerem dentro da imagem;
- extrair o recorte diretamente da imagem original f ;
- aplicar


```
cv2.QRCodeDetector().detectAndDecode(...)
```

sobre esse recorte.

8. Critério de parada

Interromper imediatamente o processamento quando o primeiro candidato produzir uma *string* decodificada não vazia.

9. Caso não encontrado

Se nenhum candidato for decodificado com sucesso, imprimir exatamente:

```
QRCODE_NAO_ENCONTRADO
```

10. Saída (caso encontrado)

Imprimir duas linhas.

Primeira linha:

```
linha coluna altura largura
```

utilizando o retângulo delimitador **original**, antes da expansão pela margem M .

Segunda linha:

```
texto_decodificado
```

6.11.8.2 📌 Restrições Computacionais

- Utilizar funções do OpenCV para realizar a binarização, a detecção de contornos, o cálculo do retângulo delimitador e a decodificação do QRCode.
- A filtragem geométrica deve ocorrer obrigatoriamente antes da etapa de decodificação.
- Utilizar exclusivamente o limiar fixo T fornecido na entrada. Não é permitido utilizar métodos automáticos de limiarização, como Otsu ou limiarização adaptativa.
- Garantir que os recortes enviados ao decodificador permaneçam dentro dos limites da imagem.

6.11.8.3 🧠 Fundamentação Teórica

Etapas	Papel no pipeline	Consequência se omitida
Filtragem geométrica	Reduz o espaço de busca selecionando apenas regiões compatíveis com a geometria esperada de um QRCode.	O decodificador processaria todos os contornos, incluindo ruídos e objetos distratores.
Decodificação	Confirma semanticamente se o candidato contém um QRCode válido.	Objetos geometricamente semelhantes poderiam ser classificados incorretamente como QRCode.
Margem M	Preserva a <i>quiet zone</i> ao redor do código, facilitando sua detecção.	A ausência dessa margem pode impedir o alinhamento e a leitura correta do código.

Este exercício integra conceitos estudados ao longo do capítulo em um único *pipeline* de Visão Computacional. A segmentação reduz o conjunto de regiões candidatas por meio de características geométricas, enquanto a etapa de decodificação valida o conteúdo da região utilizando um algoritmo especializado de reconhecimento.

6.11.8.4 Especificação de Entrada e Saída (VPL)

Estrutura de Entrada

```
L
C
T A_min tol M
[matriz da imagem]
```

Estrutura de Saída (Sucesso)

```
linha coluna altura largura
texto_decodificado
```

Estrutura de Saída (Falha)

```
QRCODE_NAO_ENCONTRADO
```

6.11.8.5 Arquivos de Referência (.pgm)

Para fins de validação, depuração local e análise de matrizes reais de pixels, os arquivos de imagem gerados no padrão ASCII P2 encontram-se disponíveis no diretório do projeto. Você pode utilizá-los para testar em decodificados do seu celular a aderência do seu código (salvar *.pgm localmente para visualizar):

-  **Caso 1: Padrão Normal** – Contém um único código perfeitamente centralizado com distratores geométricos simples na periferia.
-  **Caso 2: Cenário Complexo** – Apresenta maior densidade de ruído texturizado e múltiplos distratores candidatos que testam os limites da filtragem por aspecto.
-  **Caso 3: Mensagem Expandida** – Contém um QRCode estruturado a partir de uma cadeia de caracteres de maior comprimento, gerando maior densidade de módulos internos.
-  **Caso 4: Geometria Compacta** – Avalia o comportamento do pipeline sob condições otimizadas de contraste e posicionamento limítrofe.
-  **Caso 5: Cenário de Exclusão** – Imagem composta puramente por elementos distratores de alta área, projetada para validar o comportamento de falha controlada do programa.

```
%%writefile EP06_08.py
# Código Python
```

```
Overwriting EP06_08.py
```

```
TestSuite("EP06_08.py").run()
```

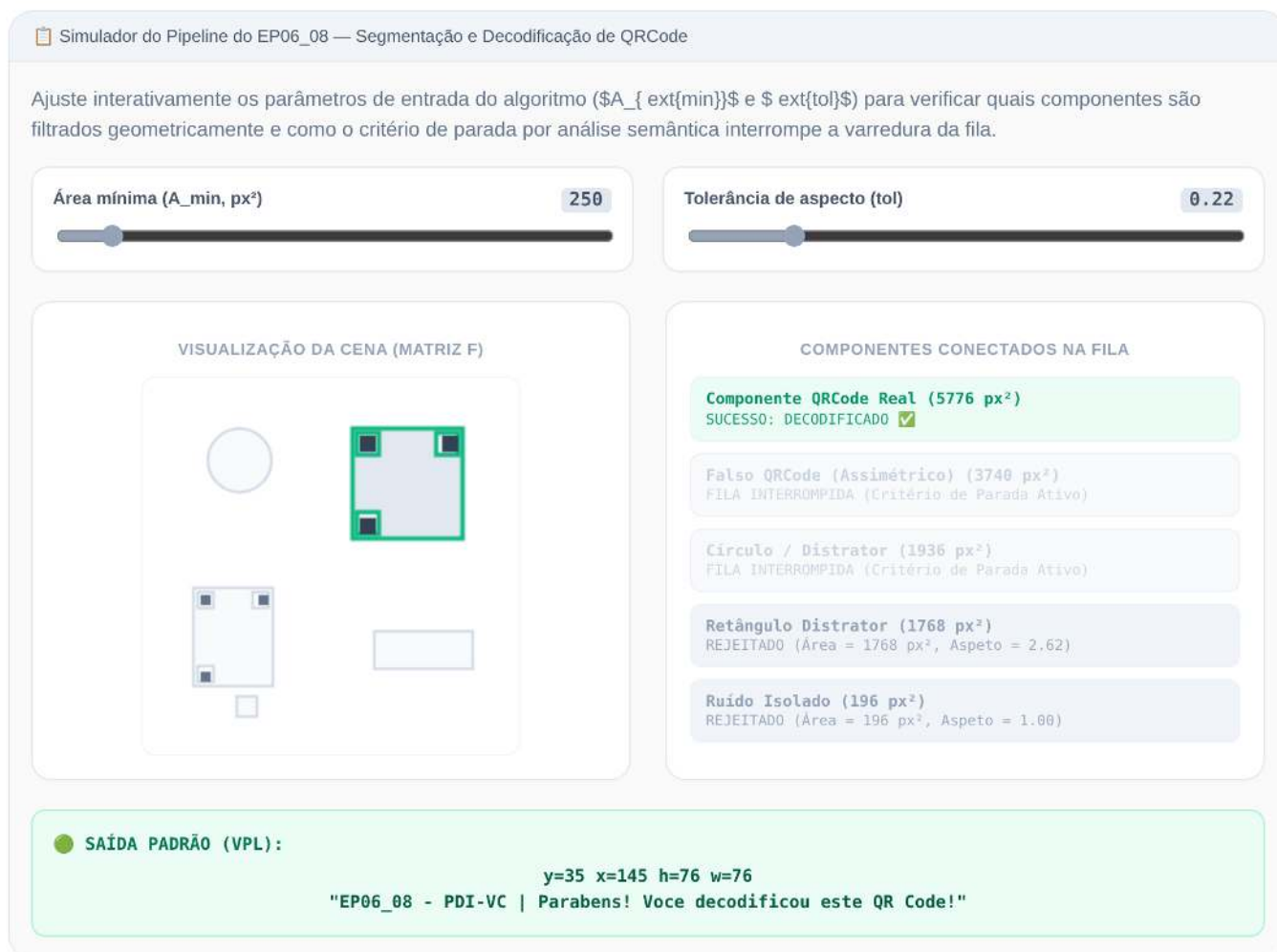


Figura 6.28: Simulador: Segmentação Geométrica + Verificação por Decodificação de QRCode

Capítulo 7

Classificação de Imagens e Reconhecimento de Padrões



Em construção!

No **Capítulo 6**, a transição da Parte I para a Parte II foi apresentada por meio de duas aplicações que já exigiam decisões automatizadas: o reconhecimento de marcas em folhas de resposta (OMR) e a detecção de defeitos em inspeção industrial. Em ambos os casos, no entanto, as decisões dependiam de regras geométricas e de limiares definidos manualmente, como determinar se um disco era suficientemente circular ou se uma região era escura o bastante.

Este capítulo formaliza o problema mais geral subjacente a essas aplicações: dado um conjunto de exemplos rotulados, como treinar um sistema para **classificar automaticamente** novas imagens ou regiões de interesse? Essa questão está no núcleo do **Reconhecimento de Padrões**, disciplina que fundamenta grande parte das tarefas modernas de Visão Computacional, desde a classificação de imagens até a detecção de objetos e a segmentação semântica, exploradas nos próximos capítulos.

Serão estudados os principais **descritores clássicos de imagem** (cor, textura e forma/gradiente) e o classificador **k-Vizinhos mais Próximos** (*k-Nearest Neighbors* — k-NN), escolhido por sua simplicidade conceitual e por evidenciar, de forma direta, a relação entre o espaço de características, as métricas de distância e as fronteiras de decisão — conceitos que permanecem centrais mesmo em classificadores baseados em redes neurais profundas, estudados no capítulo final desta parte.

7.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Compreender o **pipeline clássico de reconhecimento de padrões**: aquisição, pré-processamento, extração de descritores, classificação e avaliação;
- **Extraair e interpretar descritores clássicos** de cor, textura (*Local Binary Patterns* — LBP) e forma/gradiente (*Histogram of Oriented Gradients* — HOG);
- **Implementar e treinar um classificador k-NN** para tarefas de classificação de imagens;
- **Avaliar classificadores** por meio de métricas como acurácia, matriz de confusão, precisão e revocação;
- **Analisar o efeito do parâmetro k** e da dimensionalidade do espaço de características no desempenho do classificador;
- **Reconhecer as limitações dos descritores artesanais** (*hand-crafted features*) e compreender a motivação para a transição, nos próximos capítulos, para descritores aprendidos automaticamente.

7.2 Configuração do Ambiente

Os exemplos deste capítulo utilizam bibliotecas amplamente empregadas em Processamento Digital de Imagens, Visão Computacional e Aprendizado de Máquina. O bloco abaixo instala os pacotes necessários; em ambientes que já os possuam, a execução pode ser ignorada.

```
# Instalar apenas as bibliotecas ausentes
import importlib
import subprocess
import sys

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
    "pandas": "pandas",
    "tabulate": "tabulate",
    "kaleido": "kaleido",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

# Imports
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data as skdata
from skimage.feature import hog, local_binary_pattern
```

Assim como no capítulo anterior, será utilizado o módulo didático `morph.py`, responsável por padronizar a leitura, a exibição e o processamento de imagens ao longo do livro.

```
import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.2

7.3 Um Problema Concreto: Classificação de Frutas

Antes de apresentar os fundamentos teóricos, considere o seguinte problema, que será utilizado como exemplo ao longo deste capítulo para ilustrar os principais conceitos de reconhecimento de padrões.

O cenário: Uma fazenda automatizada utiliza um sistema de Visão Computacional para separar maçãs, bananas e laranjas em linhas de embalagem.

O desafio: As frutas chegam à esteira em diferentes posições e orientações, sob condições de iluminação que podem variar. Além disso, folhas, sombras e pequenas oclusões podem dificultar sua identificação. Como desenvolver um sistema capaz de classificá-las corretamente?

Uma possível abordagem:

1. Extrair descritores que representem características relevantes das frutas:
 - **Cor:** distribuição predominante das cores;
 - **Textura:** diferenças na superfície da casca;
 - **Forma:** características geométricas do contorno.
2. Treinar um classificador utilizando exemplos previamente rotulados.
3. Utilizar o modelo treinado para classificar automaticamente novas frutas.

A Figure 7.1 ilustra, de forma conceitual, como diferentes frutas podem ser representadas em um espaço de características tridimensional.

 Reflita antes de continuar

Se cada fruta fosse representada apenas pelos valores de intensidade de seus pixels, seria possível distingui-las de forma confiável? Que tipos de informação poderiam ser extraídos da imagem para facilitar essa tarefa?

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

centros = {
    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(projection="3d")

for fruta, centro, cor, marcador in zip(
    centros,
    centros.values(),
    ["red", "gold", "orange"],
    ["o", "s", "^"],
):
    X = np.clip(np.random.normal(centro, 0.08, (70, 3)), 0, 1)
    ax.scatter(X[:, 0], X[:, 1], X[:, 2],
               c=cor, marker=marcador, s=35,
               alpha=0.7, label=fruta)

ax.set(
    xlim=(0,1), ylim=(0,1), zlim=(0,1),
    xlabel="Intensidade de cor",
    ylabel="Textura",
    zlabel="Forma",
    title="Espaço de Características"
)
ax.view_init(elev=25, azim=-60)
ax.legend(title="Frutas")
ax.zaxis.labelpad = 0.01

plt.tight_layout()
plt.show()
```

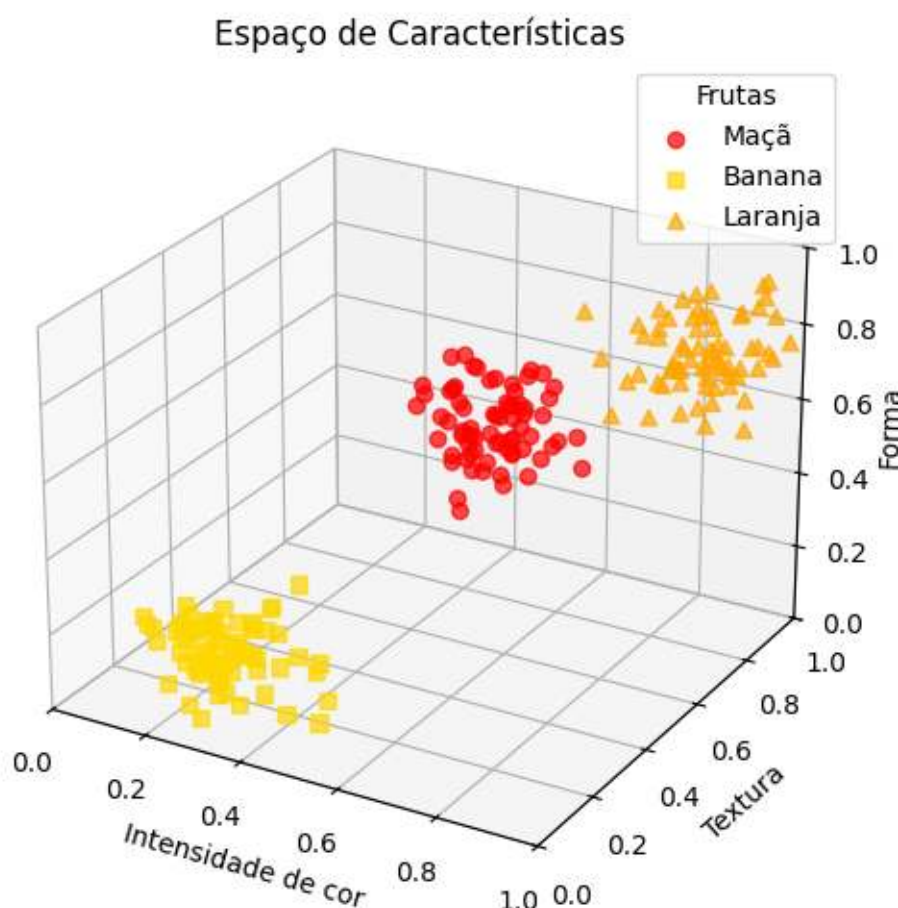


Figura 7.1: Exemplo motivacional: diferentes frutas formando agrupamentos distintos em um espaço de características.

7.4 Panorama do Capítulo: O *Pipeline* Clássico de Classificação de Imagens

Antes de prosseguir, é útil apresentar uma visão integrada do que será estudado. A Figure 7.2 mostra o fluxo geral de um sistema clássico de classificação de imagens, desde a imagem de entrada até a etapa de atribuição do rótulo final. Nas próximas seções, cada uma das etapas desse processo será estudada em detalhes.

i Escopo deste capítulo

Neste capítulo, o foco recai exclusivamente sobre o classificador k-NN, por sua simplicidade didática e por ilustrar de forma intuitiva o conceito de espaço de características. Outros classificadores tradicionais amplamente utilizados em Reconhecimento de Padrões — como Árvores de Decisão, Regras de Classificação e Máquinas de Vetores de Suporte (SVM) — são discutidos em profundidade em Quilici-Gonzalez; Zampirolli (2014), especialmente em sua 2ª edição, atualmente em produção (Quilici-Gonzalez; Zampirolli; Souza, 2026).

7.5 Fundamentos de Reconhecimento de Padrões

Um sistema de **reconhecimento de padrões** tem como objetivo atribuir uma categoria (rótulo) a uma observação — uma imagem inteira, uma região de interesse ou um sinal — com base em exemplos previamente rotulados. De forma geral, esse processo é organizado nas seguintes etapas:

1. **Aquisição:** obtenção da imagem ou do sinal a ser classificado;

Classificação de Imagens e Reconhecimento de Padrões: O Pipeline Clássico

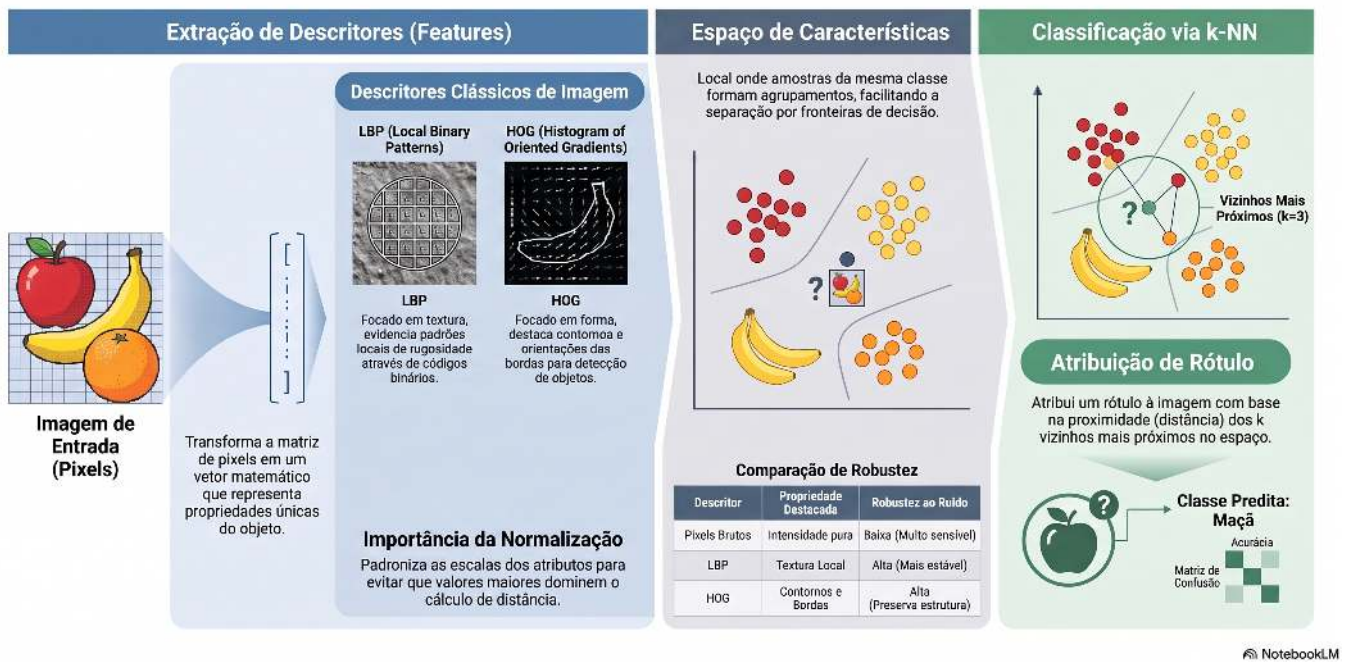


Figura 7.2: Panorama do *pipeline* clássico de classificação de imagens: extração de descritores (LBP, HOG), formação do espaço de características e classificação via k-NN. Não se aplica a modelos de *deep learning* (CNNs, YOLO), que aprendem *features end-to-end* diretamente dos pixels. **Fonte:** elaborado com auxílio do NotebookLM (Google, 2025).

2. **Pré-processamento:** normalização, remoção de ruído, correção geométrica ou de iluminação — etapas já estudadas nos capítulos anteriores;
3. **Extração de descritores (*features*):** transformação da imagem em um **vetor de características** de dimensão fixa, que representa as propriedades relevantes para a tarefa de classificação;
4. **Classificação:** aplicação de um modelo que associa o vetor de características a uma classe;
5. **Avaliação:** análise do desempenho do modelo em um conjunto de dados independente daquele utilizado para o treinamento.

O conjunto de todos os vetores de características possíveis constitui o **espaço de características** (*feature space*). Um bom descritor produz representações que aproximam, nesse espaço, observações da mesma classe e afastam observações de classes distintas. Essa propriedade favorece métodos de classificação baseados em proximidade, como o **k-NN**, e também beneficia diversos outros classificadores.

A Figure 7.1 ilustra esse conceito de forma esquemática: cada fruta é representada por um ponto em um espaço de características de três dimensões (cor, textura e forma). Embora esse espaço seja apenas uma simplificação didática, ele mostra como amostras da mesma classe tendem a formar agrupamentos, enquanto classes diferentes ocupam regiões distintas, facilitando a tarefa de classificação.

7.6 Extração de Descritores Clássicos

Antes da popularização das redes neurais profundas, os descritores de imagem eram, em sua maioria, projetados manualmente por especialistas (*hand-crafted features*), com base em propriedades estatísticas ou geométricas conhecidas. Três famílias clássicas são particularmente relevantes:

- **Descritores de cor:** histogramas de intensidade ou de matiz, que capturam a distribuição dos valores de cor de uma região, já introduzidos no **Capítulo 3** por meio da função `mm.hist`;
- **Descritores de textura:** capturam padrões locais de repetição, rugosidade ou orientação, como o

Local Binary Patterns (LBP), estudado a seguir;

- **Descritores de forma/gradiente:** descrevem a distribuição das bordas e das orientações do gradiente, como o *Histogram of Oriented Gradients* (HOG), amplamente empregado na detecção de pessoas e outros objetos.

Para entender por que descritores são tão poderosos, a Figure 7.3 mostra como diferentes técnicas “enxergam” a mesma imagem.

```
import matplotlib.cm as cm
from skimage import data
from skimage.feature import hog, local_binary_pattern

# Carregar imagem de exemplo
imagem = data.camera()

# Aplicar descritores
lbp_img = local_binary_pattern(
    imagem,
    P=8,
    R=1,
    method="uniform"
)

# Converter o LBP para RGB apenas para facilitar a visualização
lbp_norm = (lbp_img - lbp_img.min()) / (lbp_img.max() - lbp_img.min() + 1e-8)
lbp_rgb = (cm.nipy_spectral(lbp_norm)[..., :3] * 255).astype("uint8")

_, hog_img = hog(
    imagem,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
)

# Exibição padronizada
mm.show(
    [imagem, lbp_rgb, hog_img],
    titles=[
        "Imagem Original\n(como o humano vê)",
        "LBP: Textura\n(cada cor = um código LBP)",
        "HOG: Gradientes e Contornos\n(regiões claras = maior intensidade)",
    ],
    cols=3,
    figsize=(12, 4),
)

print("Observe como cada descritor destaca propriedades diferentes:")
print("• LBP: evidencia padrões locais de textura.")
print("• HOG: evidencia contornos e orientações das bordas.")
print("• Imagem original: contém apenas os valores de intensidade.")
```

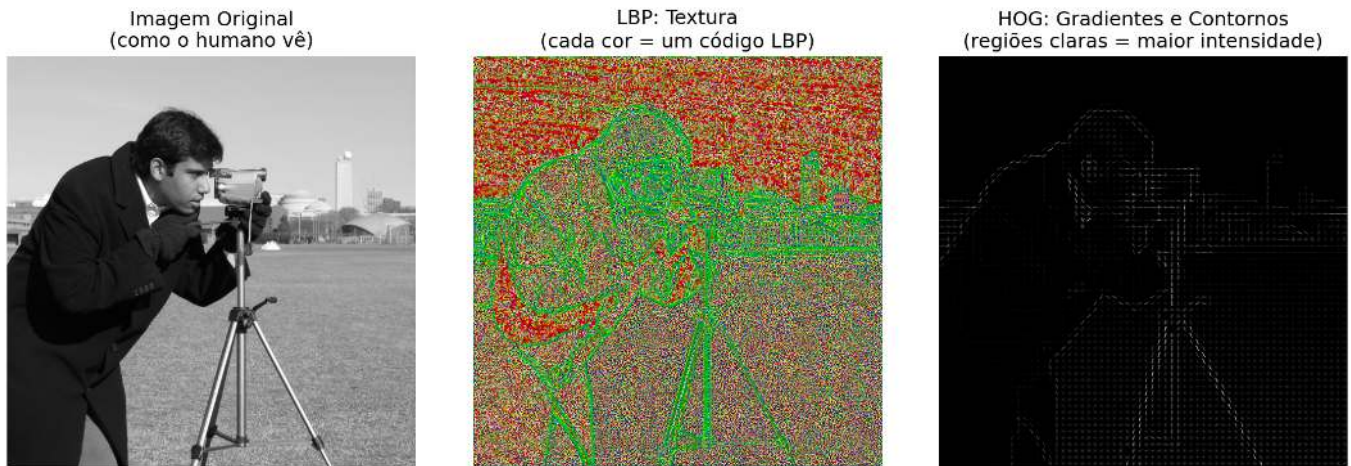


Figura 7.3: Comparação visual de diferentes descritores aplicados à mesma imagem. Cada descritor revela aspectos distintos da cena.

Observe como cada descritor destaca propriedades diferentes:

- LBP: evidencia padrões locais de textura.
- HOG: evidencia contornos e orientações das bordas.
- Imagem original: contém apenas os valores de intensidade.

7.6.1 Local Binary Patterns (LBP)

O LBP é um descritor de textura que codifica, para cada pixel central g_c , a relação entre sua intensidade e a dos P vizinhos dispostos em uma vizinhança circular de raio R :

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p, \quad s(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

em que:

- (x_c, y_c) são as coordenadas do pixel central;
- g_c é a intensidade do pixel central;
- g_p é a intensidade do p -ésimo pixel vizinho;
- P é o número de vizinhos considerados;
- R é o raio da vizinhança circular;
- p é o índice do vizinho, com $p = 0, 1, \dots, P-1$;
- $s(z)$ é a função limiar definida na equação, em que $z = g_p - g_c$; ela assume valor 1 quando $z \geq 0$ e 0 quando $z < 0$;
- 2^p corresponde ao peso binário associado ao p -ésimo vizinho.

O código LBP obtido descreve o padrão local de contraste ao redor do pixel central. O histograma dos códigos LBP calculados em uma região constitui um vetor de características compacto para representar sua textura. Neste capítulo utiliza-se a variante *uniforme*, que reduz o número de padrões possíveis ao agrupar padrões não uniformes, produzindo descritores mais compactos e robustos.

💡 Função `local_binary_pattern`

A implementação utilizada neste capítulo é fornecida pela biblioteca `scikit-image`:

```
local_binary_pattern(
    imagem,
    P=8,
    R=1,
    method="uniform"
)
```

onde:

- **image**: imagem em escala de cinza;
- **P**: número de vizinhos igualmente espaçados na vizinhança circular;
- **R**: raio da vizinhança, em pixels;
- **method**: estratégia de codificação. Neste capítulo utiliza-se o valor "uniform".

A equação apresentada anteriormente descreve o **LBP original**. Na implementação adotada neste capítulo, a opção `method="uniform"` calcula inicialmente esse código e, em seguida, remapeia os padrões não uniformes para uma única categoria, reduzindo a dimensionalidade do descritor e tornando-o mais robusto a pequenas variações locais.

A Figure 7.3 ilustra o processo de codificação dos padrões locais de textura pelo descritor LBP.

O **Projeto Prático 2** (seção Classificação de Texturas com Descritores LBP) emprega o LBP na classificação de diferentes tipos de textura sintética.

7.6.2 Histogram of Oriented Gradients (HOG)

O HOG é um descritor que representa a forma de um objeto por meio da distribuição das orientações do gradiente local. Assim como no operador de Canny (**Capítulo 6**), calcula-se inicialmente o gradiente:

$$|\nabla f(x, y)| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}, \quad \theta(x, y) = \text{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right).$$

em que:

- $f(x, y)$ é a intensidade da imagem no pixel (x, y) ;
- $\frac{\partial f}{\partial x}$ e $\frac{\partial f}{\partial y}$ são, respectivamente, as derivadas parciais da imagem nas direções horizontal e vertical;
- $|\nabla f(x, y)|$ é a magnitude do vetor gradiente no pixel (x, y) , indicando a intensidade da variação local da imagem;
- $\theta(x, y)$ é a orientação do vetor gradiente no pixel (x, y) , calculada pela função `atan2`, cujo resultado pertence ao intervalo $(-\pi, \pi]$.

Embora $\theta(x, y)$, como calculado pela função `atan2`, pertença ao intervalo $(-\pi, \pi]$, a implementação padrão do HOG utiliza o **gradiente não sinalizado** (*unsigned*): orientações opostas (por exemplo, 0 e π) são tratadas como equivalentes, e os ângulos são mapeados para o intervalo $[0, \pi)$ antes da construção do histograma. Essa escolha torna o descritor invariante à direção do contraste (por exemplo, uma borda clara-escuro e uma borda escuro-clara produzem a mesma orientação).

A imagem é então dividida em **células** (*cells*). Para cada célula, constrói-se um histograma das orientações do gradiente, ponderado pela magnitude correspondente. A concatenação dos histogramas de todas as células forma o vetor de características HOG, que representa a distribuição espacial das orientações do gradiente e captura informações sobre a forma e os contornos do objeto.

Função hog

A extração do descritor HOG é realizada pela função:

```
hog(  
    image,  
    orientations=9,  
    pixels_per_cell=(8, 8),  
    cells_per_block=(2, 2),  
    visualize=True,  
)
```

Os principais parâmetros são:

- **image**: imagem de entrada;
- **orientations**: número de divisões angulares do histograma de orientações em cada célula;
- **pixels_per_cell**: tamanho, em pixels, de cada célula onde o histograma é calculado;
- **cells_per_block**: número de células utilizadas na normalização do descritor;
- **visualize**: quando **True**, retorna também uma imagem ilustrando os gradientes utilizados pelo HOG.

A equação apresentada anteriormente descreve o cálculo da magnitude e da orientação do gradiente, que constituem a base do descritor HOG. Na implementação adotada neste capítulo, a função `hog()` utiliza essas informações para construir histogramas de orientações em cada célula da imagem e, em seguida, realiza a normalização em blocos (`cells_per_block`), reduzindo a sensibilidade do descritor a variações de iluminação e contraste.

A Figure 7.3 ilustra a representação produzida pelo descritor HOG a partir da imagem original.

O Projeto Prático 1 (seção Classificação de Dígitos Manuscritos com k-NN) compara o desempenho de descritores HOG com o uso direto das intensidades dos pixels como vetor de características.

7.6.3 O Impacto da Escala e a Normalização de Características

O classificador *k*-NN toma suas decisões com base na distância entre os vetores de características. Por isso, a escala de cada característica influencia diretamente o resultado da classificação. Se uma variável apresentar valores muito maiores do que as demais (por exemplo, uma intensidade de cor variando de 0 a 255, enquanto um índice de circularidade varia de 0 a 1), ela tende a dominar o cálculo da distância, reduzindo a influência dos outros descritores.

Para evitar esse problema, aplica-se uma etapa de **normalização das características**, geralmente por meio da padronização (*Z-score standardization*). Nesse procedimento, cada característica passa a ter média igual a zero e desvio-padrão igual a um, tornando comparáveis grandezas originalmente medidas em escalas diferentes.

A padronização é realizada pela transformação

$$z = \frac{x - \mu}{\sigma},$$

em que:

- x é o valor original da característica;
- μ é a média dessa característica calculada sobre o conjunto de treinamento;
- σ é o desvio-padrão da característica;
- z é o valor padronizado.

Após essa transformação, todas as características passam a possuir média igual a zero e desvio-padrão igual a um, permitindo que contribuam de forma equilibrada para o cálculo das distâncias.

💡 Classe StandardScaler

A padronização utilizada neste capítulo é realizada pela classe `StandardScaler`, da biblioteca `scikit-learn`:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_norm = scaler.fit_transform(X)
```

em que:

- `StandardScaler()`: cria o objeto responsável pela padronização;
- `fit_transform(X)`: calcula a média e o desvio-padrão de cada característica do conjunto `X` e retorna a matriz padronizada.

Na prática, o método `fit_transform()` executa duas etapas: primeiro (`fit`), estima a média (μ) e o desvio-padrão (σ) de cada característica; em seguida (`transform`), aplica a transformação de padronização apresentada anteriormente a todos os valores da matriz de entrada.

A Figure 7.4 mostra o efeito da normalização. Visualmente, a distribuição dos pontos permanece a mesma; o que muda é a escala dos eixos. Sem a normalização, a característica de maior magnitude domina o cálculo das distâncias entre as amostras. Após a padronização, todas as características passam a contribuir de forma equilibrada para o cálculo das distâncias utilizadas pelo classificador *k*-NN.

```
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Dados sintéticos com escalas muito diferentes
np.random.seed(42)
X_demo = np.random.randn(20, 2) * [100, 1]
y_demo = np.array([0] * 10 + [1] * 10)

print("Efeito da normalização:")
print("  Característica 1: escala 100")
print("  Característica 2: escala 1")
print("\nSem normalização, a primeira característica domina o cálculo das distâncias.")
print("Com normalização, ambas contribuem de forma equilibrada.")
print("\nA normalização é essencial quando as características possuem escalas diferentes.")
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Sem normalização
axes[0].scatter(
    X_demo[y_demo == 0, 0], X_demo[y_demo == 0, 1],
    c="blue", label="Classe 0"
)
axes[0].scatter(
    X_demo[y_demo == 1, 0], X_demo[y_demo == 1, 1],
    c="red", label="Classe 1"
)
axes[0].set_title("Sem Normalização\n(escalas diferentes)")
axes[0].set_xlabel("Característica 1 (escala 100)")
axes[0].set_ylabel("Característica 2 (escala 1)")
axes[0].legend()

# Com normalização
scaler = StandardScaler()
X_norm = scaler.fit_transform(X_demo)

axes[1].scatter(
```

```

X_norm[y_demo == 0, 0], X_norm[y_demo == 0, 1],
c="blue", label="Classe 0"
)
axes[1].scatter(
    X_norm[y_demo == 1, 0], X_norm[y_demo == 1, 1],
    c="red", label="Classe 1"
)
axes[1].set_title("Com Normalização\n(características balanceadas)")
axes[1].set_xlabel("Característica 1")
axes[1].set_ylabel("Característica 2")
axes[1].legend()

plt.tight_layout()
plt.show()

```

Efeito da normalização:

Característica 1: escala 100

Característica 2: escala 1

Sem normalização, a primeira característica domina o cálculo das distâncias.

Com normalização, ambas contribuem de forma equilibrada.

A normalização é essencial quando as características possuem escalas diferentes.

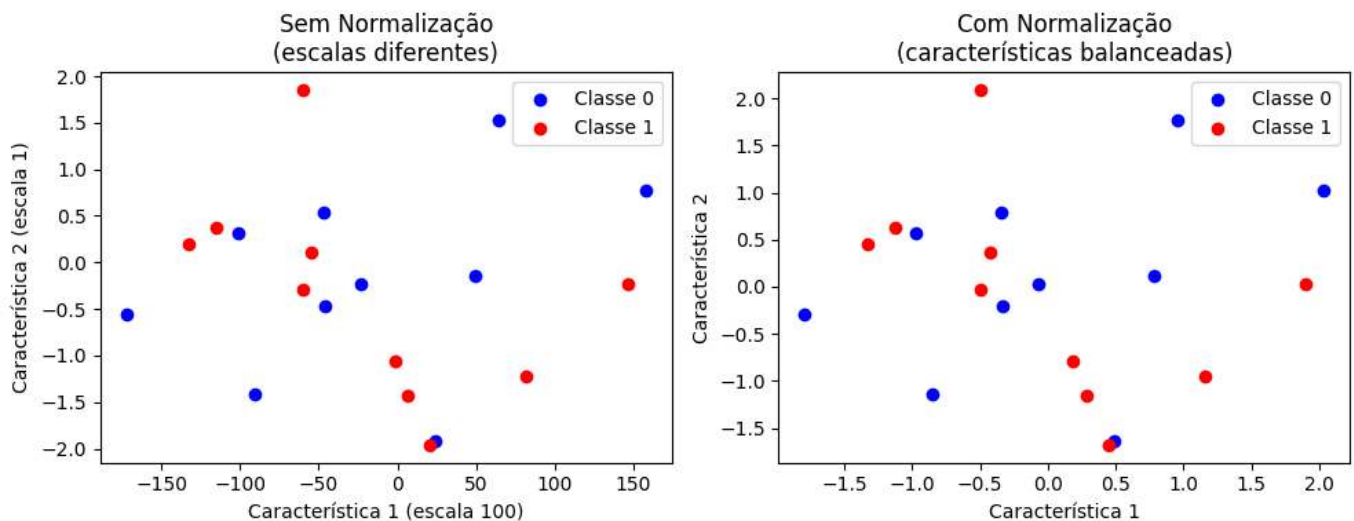


Figura 7.4: Importância da normalização das características para o classificador k-NN.

7.7 Mapa Conceitual

Até aqui, foram apresentados os descritores clássicos (LBP, HOG, pixels brutos) e a forma como eles organizam as amostras em um espaço de características. A Figure 7.5 sintetiza esse percurso e situa essas etapas dentro do fluxo geral de um sistema clássico de classificação de imagens, indicando também as etapas seguintes — classificação (k-NN) e avaliação dos resultados — que serão formalizadas nas próximas seções.

7.8 Descritores na Prática

Após conhecer os principais descritores clássicos, é natural perguntar como eles influenciam o desempenho de um classificador em situações próximas às encontradas na prática.

Nesta seção, compara-se o uso de três representações distintas das mesmas imagens: intensidades dos pixels, descritores LBP e descritores HOG. Para tornar o experimento mais realista, adiciona-se ruído sintético aos

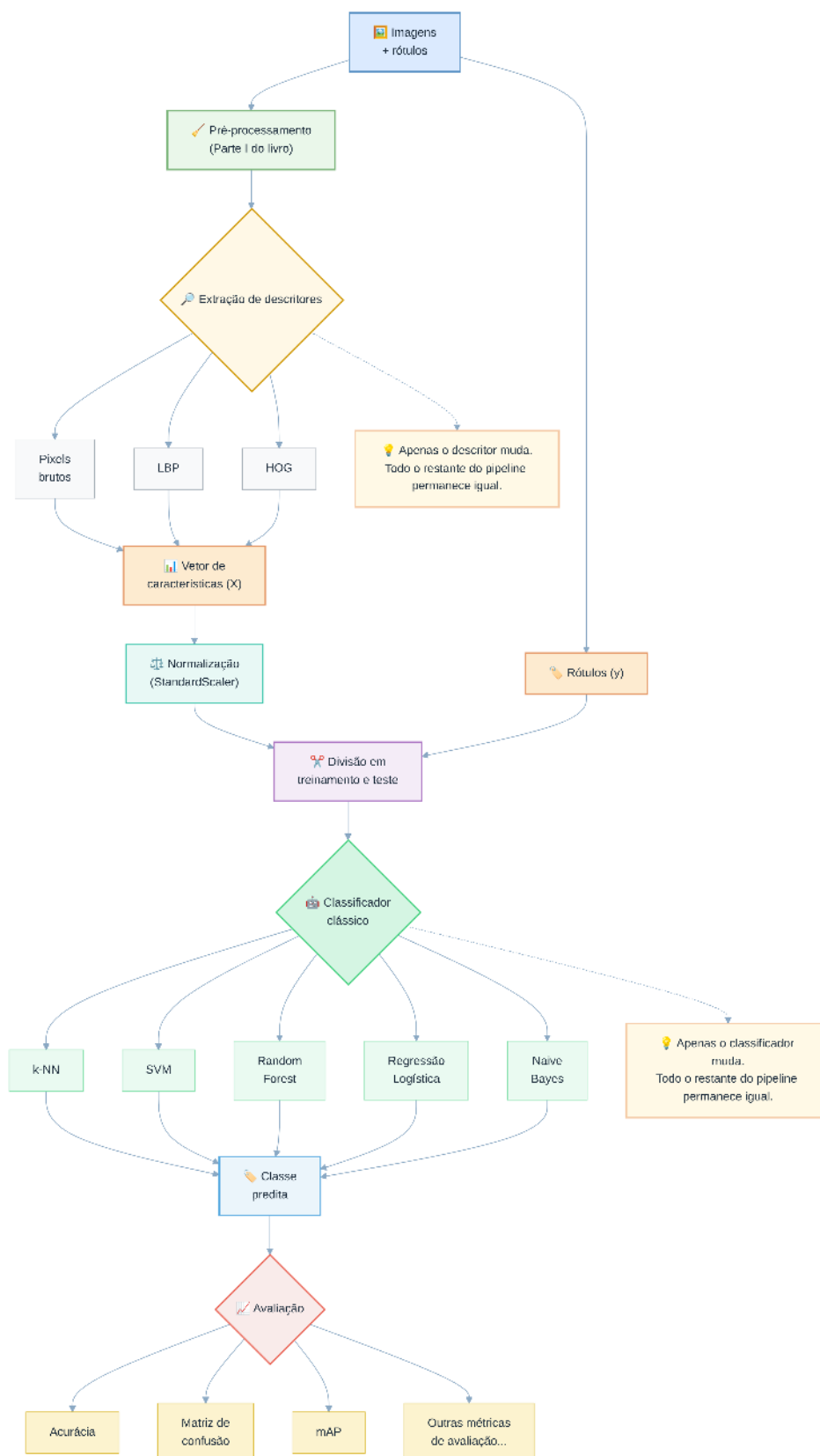


Figura 7.5: Mapa conceitual do processo de classificação de imagens utilizando descritores clássicos (LBP, HOG, pixels brutos) e classificador tradicional (k-NN). Não se aplica a modelos de *deep learning* (CNNs, YOLO), que aprendem *features end-to-end* diretamente dos pixels.

dados, em intensidades diferentes para cada descritor — uma forma simplificada de simular o fato de que, na prática, diferentes representações toleram de forma desigual as imperfeições da captura (ruído do sensor, pequenas variações de posição, etc.).

A Figure 7.6 apresenta as matrizes de confusão obtidas para cada descritor, permitindo identificar em quais classes ocorrem os principais erros de classificação. A interpretação dessas matrizes foi introduzida no **Capítulo 1**, quando foram apresentados os conceitos de **Verdadeiro Positivo (VP)**, **Falso Positivo (FP)**, **Verdadeiro Negativo (VN)** e **Falso Negativo (FN)**. Esses conceitos foram explorados nos EPs **01_02** (métricas de classificação) e **01_03** (*mean Average Precision* – mAP), disponíveis em:

- https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_02.html
- https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_03.html

Neste capítulo, as matrizes de confusão são empregadas para analisar como diferentes descritores influenciam o desempenho do classificador.

Em seguida, a Figure 7.7 resume a acurácia global obtida por cada descritor.

Os resultados mostram que o desempenho do classificador depende diretamente da representação escolhida para descrever as imagens. Enquanto o uso direto das intensidades dos pixels é mais sensível às degradações introduzidas, os descritores LBP e HOG preservam melhor as informações relevantes para a classificação, resultando em maior desempenho nesse cenário. É importante ressaltar que os níveis de ruído aplicados a cada descritor foram escolhidos apenas para fins didáticos, de modo a ilustrar o princípio geral de que descritores mais elaborados *podem* ser mais robustos a degradações — o que não significa que essa relação se verifique sempre, como o estudo de caso da próxima seção demonstrará.

💡 Como o experimento é realizado

Como o objetivo desta seção é comparar apenas o efeito dos descritores, gera-se um conjunto de dados sintético simples: três nuvens de pontos gaussianas, centradas nos mesmos valores de “cor, textura e forma” já utilizados na Figure 7.1 — o mesmo padrão empregado desde o início do capítulo para representar as três classes de frutas.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
centros = {
    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

n_por_classe = 100
X = np.vstack([
    np.random.normal(centro, 0.12, (n_por_classe, 3))
    for centro in centros.values()
])
y = np.repeat(list(centros.keys()), n_por_classe)
```

em que:

- **centros**: dicionário com o ponto médio de cada classe no espaço de características (cor, textura, forma);
- **n_por_classe**: número de amostras geradas por classe;
- **np.random.normal(centro, 0.12, (n_por_classe, 3))**: gera **n_por_classe** amostras ao redor de cada centro, com desvio-padrão 0,12 em cada dimensão;
- **np.repeat(list(centros.keys()), n_por_classe)**: gera o vetor de rótulos correspondente,

na mesma ordem dos centros.

Em seguida, utiliza-se o fluxo de treinamento e avaliação:

- `train_test_split(X, y, test_size=0.3)`: divide os dados em treinamento (70%) e teste (30%);
- `KNeighborsClassifier(n_neighbors=5)`: cria um classificador k -NN com $k = 5$ vizinhos;
- `fit(X_train, y_train)`: ajusta o modelo aos dados de treinamento;
- `predict(X_test)`: classifica as amostras de teste;
- `accuracy_score(y_test, y_pred)`: calcula a acurácia;
- `confusion_matrix(y_test, y_pred)`: gera a matriz de confusão.

Neste experimento, o conjunto de treinamento, o classificador e o método de avaliação permanecem exatamente os mesmos. A única diferença entre os experimentos é a representação utilizada para cada imagem (pixels brutos, LBP ou HOG), permitindo avaliar exclusivamente a influência do descritor sobre o desempenho do classificador.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns

classes = ["Maçã", "Banana", "Laranja"]

np.random.seed(42)

# Mesmos centros de classe (cor, textura, forma) utilizados na
# @fig-07-frutas-motivacao, agora reaproveitados para gerar os dados
# sintéticos de treino e teste deste experimento.
centros = {
    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

n_por_classe = 100
X = np.vstack([
    np.random.normal(centro, 0.12, (n_por_classe, 3))
    for centro in centros.values()
])
y = np.repeat(list(centros.keys()), n_por_classe)

# Simular descritores com diferentes níveis de sensibilidade ao ruído.
# Quanto maior o ruído adicionado, pior tende a ser a representação.
descritores = {
    "Pixels Brutos": X + 0.5 * np.random.randn(*X.shape),
    "LBP": X + 0.3 * np.random.randn(*X.shape),
    "HOG": X + 0.2 * np.random.randn(*X.shape),
}

# Criar uma única figura com 3 subplots lado a lado para as matrizes
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
resultados = {}

for idx, (nome, Xd) in enumerate(descritores.items()):
    X_train, X_test, y_train, y_test = train_test_split(Xd, y, test_size=0.3, random_state=42)

    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_train, y_train)
```

```

y_pred = knn.predict(X_test)

acc = accuracy_score(y_test, y_pred)
resultados[nome] = acc

# Matriz de confusão no subplot correspondente

cm = confusion_matrix(y_test, y_pred, labels=classes)

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False,
    xticklabels=classes,
    yticklabels=classes,
    ax=axes[idx],
)

axes[idx].set_title(
    f"{nome}\nAcurácia: {acc:.3f}",
    fontsize=11,
    fontweight="bold"
)

axes[idx].set_xlabel("Classe Predita")
axes[idx].set_ylabel("Classe Real")

plt.tight_layout()
plt.show()

```

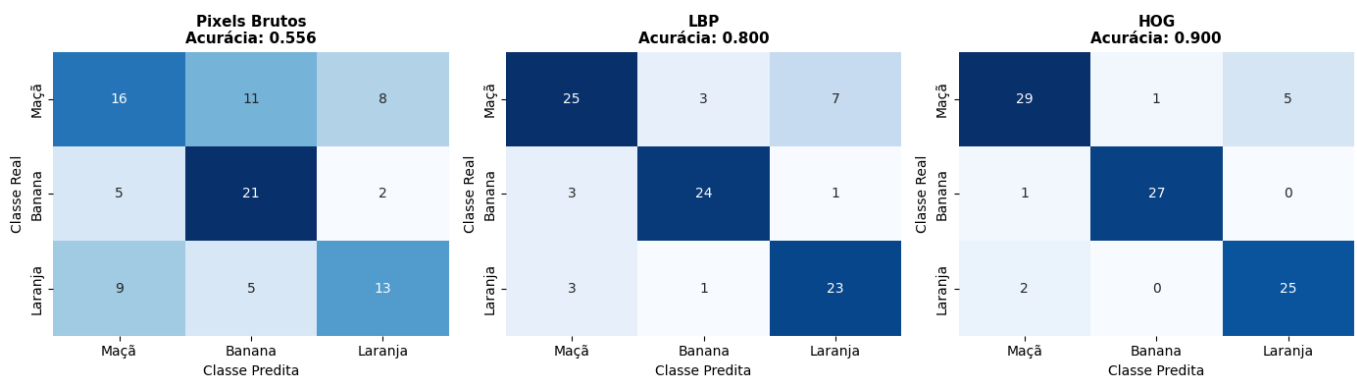


Figura 7.6: Matrizes de confusão obtidas pelo classificador k-NN utilizando três descritores diferentes. As linhas representam a classe real (Maçã, Banana e Laranja) e as colunas a classe predita. Quanto maior a concentração de valores na diagonal principal, melhor o desempenho do descritor.

```

# Comparação visual em uma figura isolada
plt.figure(figsize=(6, 3.5))
nomes = list(resultados.keys())
acuracia = list(resultados.values())
colors = ['#6366f1', '#f97316', '#22c55e']

bars = plt.bar(nomes, acuracia, color=colors, width=0.5)
plt.ylabel('Acurácia Global')
plt.title('Desempenho Geral dos Descritores sob Ruído Realista', fontsize=12, fontweight='bold')
plt.ylim(0.5, 1.0)

```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Adicionar os valores acima das barras usando round padrão para exibição
for bar, val in zip(bars, acuracia):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{val:.3f}', ha='center', fontweight='bold', fontsize=10)

plt.tight_layout()
plt.show()

print("Análise dos resultados:")
print("- Pixels Brutos: Sensíveis a variações locais de iluminação e ruído.")
print("- LBP: Boa tolerância para variações monotônicas de iluminação global.")
print("- HOG: Excelente para contornos e formas estáveis sob pequenas flutuações geométricas.")
```

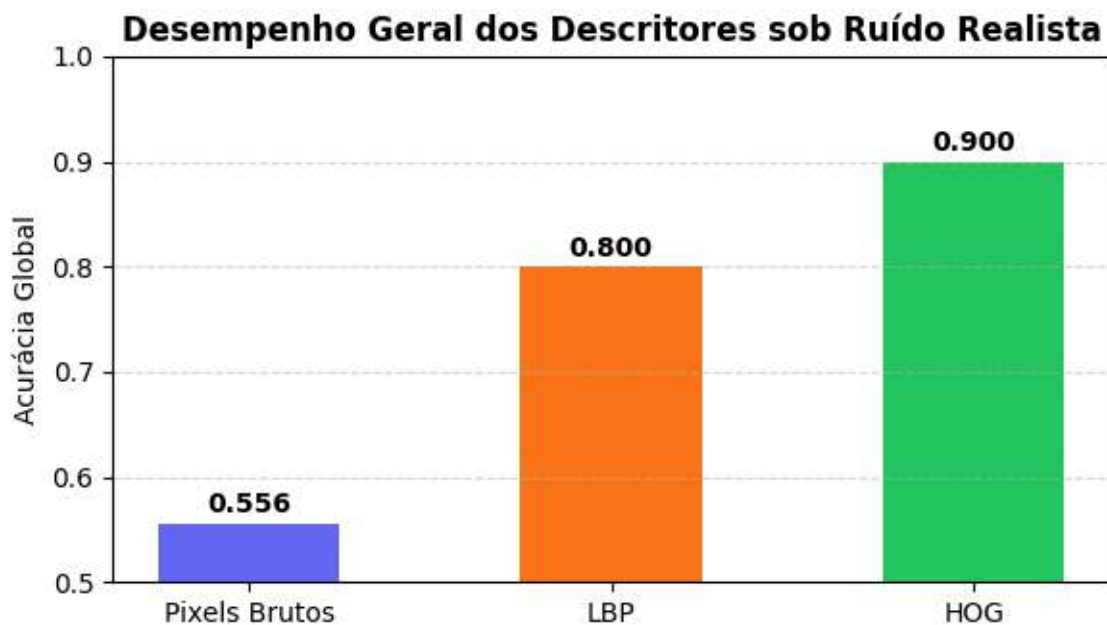


Figura 7.7: Comparação detalhada de acurácia global em cenário realista. Observe como os descritores extraídos estruturalmente (LBP e HOG) superam o uso de intensidades puras de pixels brutos.

Análise dos resultados:

- Pixels Brutos: Sensíveis a variações locais de iluminação e ruído.
- LBP: Boa tolerância para variações monotônicas de iluminação global.
- HOG: Excelente para contornos e formas estáveis sob pequenas flutuações geométricas.

7.9 Um Problema Concreto: Simulando Descritores de Frutas

Retomando o problema de classificação de frutas apresentado no início do capítulo, cada imagem pode ser representada por um vetor de características (*feature vector*), obtido a partir da extração de descritores de cor, textura e forma. A Table 7.1 apresenta um conjunto de descritores frequentemente utilizados em aplicações de Visão Computacional, incluindo alguns já introduzidos no Capítulo 3 e no início deste capítulo.

Tabela 7.1: Conjunto de descritores cromáticos, texturais e geométricos utilizados para representar imagens de frutas.

Característica	Descrição
R, G, B	intensidade média dos canais vermelho, verde e azul

Característica	Descrição
NC	intensidade média em níveis de cinza (<i>grayscale</i>)
LBP	descritor de textura (<i>Local Binary Pattern</i>)
HOG	descritor de forma (<i>Histogram of Oriented Gradients</i>)
Área	número de pixels do objeto
Perímetro	comprimento do contorno
Circularidade	medida de quão circular é o objeto
Razão largura/altura	proporção entre largura e altura da região

Nesse exemplo, cada imagem é representada pelo vetor

$$X = (R, G, B, NC, LBP, HOG, \text{Área}, \text{Perímetro}, \text{Circularidade}, \text{Razão}).$$

i Simplificação adotada nesta tabela

Na prática, LBP e HOG não são valores escalares únicos, mas **histogramas** com dezenas ou centenas de componentes (por exemplo, o LBP uniforme utilizado neste capítulo produz um histograma com $P + 2$ posições). Na tabela e no vetor X , cada um deles é representado, por simplicidade didática, por um único valor-resumo. Em uma aplicação real, essas duas posições seriam substituídas pelas componentes do histograma correspondente.

Em aplicações reais, nem todas as características possuem o mesmo poder discriminativo. Algumas contribuem de forma significativa para distinguir as classes, enquanto outras fornecem pouca informação ou são redundantes. Essa situação é comum em Visão Computacional e frequentemente motiva o uso de técnicas de seleção de características.

Diferentemente do experimento da seção anterior, em que bastava gerar três nuvens de pontos separáveis, o objetivo agora é simular também a presença de características redundantes ou pouco informativas. Para isso, utiliza-se a função `make_classification()`, da biblioteca `scikit-learn`, que permite controlar quantas das características geradas são efetivamente relevantes para a classificação:

```
X, y = make_classification(
    n_samples=300,
    n_features=10,
    n_informative=7,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)
```

Cada uma das dez características sintéticas é associada a um dos descritores da Table 7.1. O parâmetro `n_informative=7` indica que apenas sete dessas características contêm informação útil para separar as três classes, enquanto as demais simulam atributos redundantes ou pouco discriminativos. Esse controle sobre a quantidade de características informativas é a principal razão para utilizar `make_classification()`, pois não é obtido diretamente por geradores aleatórios simples, como `np.random.normal`.

7.10 Classificador k-NN: Como Funciona por Dentro

O **k-Nearest Neighbors** (k-NN) é um dos algoritmos de classificação mais simples e intuitivos da Aprendizagem de Máquina. Diferentemente de muitos classificadores, ele não constrói explicitamente um modelo durante a etapa de treinamento. Em vez disso, armazena as amostras rotuladas e, quando uma nova amostra precisa ser classificada, procura aquelas que mais se assemelham a ela.

O princípio do algoritmo baseia-se na hipótese de que amostras com características semelhantes tendem a pertencer à mesma classe. Para quantificar essa proximidade, o k-NN utiliza uma medida de distância entre os vetores de características.

Como exemplo, considere a Table 7.2, que apresenta uma versão simplificada do problema de classificação de frutas utilizando apenas duas características: intensidade de cor e circularidade, ambas normalizadas no intervalo de 0 a 1.

Tabela 7.2: Exemplo simplificado de classificação de frutas utilizando duas características normalizadas.

Amostra	Cor	Circularidade	Classe
Fruta 1	0,82	0,88	Maçã
Fruta 2	0,30	0,20	Banana
Fruta 3	0,88	0,85	Maçã
Fruta ? (teste)	0,80	0,90	?

Observando apenas essas duas características, percebe-se que a fruta de teste está muito mais próxima das amostras rotuladas como **Maçã** do que da amostra rotulada como **Banana**. Na seção seguinte, essa noção intuitiva de proximidade será formalizada por meio de uma métrica de distância, utilizada pelo algoritmo para identificar os vizinhos mais próximos e decidir a classe da nova amostra.

7.10.1 Métrica de Distância

A proximidade entre duas amostras é normalmente quantificada pela **distância euclidiana**, definida por

$$d(x, x_i) = \|x - x_i\|_2 = \sqrt{\sum_{j=1}^n (x_j - x_{i,j})^2},$$

em que:

- x é a amostra de teste;
- x_i é uma amostra do conjunto de treinamento;
- n é o número de características;
- x_j e $x_{i,j}$ representam a j -ésima característica.

Na implementação deste capítulo, x corresponde a uma linha de `X_test` e x_i a uma linha de `X_train`. O método `predict()` calcula automaticamente a distância entre x e todas as amostras de treinamento.

No exemplo da Table 7.2:

$$d(\text{teste}, \text{Fruta 1}) \approx 0,028, \quad d(\text{teste}, \text{Fruta 2}) \approx 0,860, \quad d(\text{teste}, \text{Fruta 3}) \approx 0,094.$$

Como as menores distâncias correspondem às Frutas 1 e 3, essas amostras serão utilizadas na etapa de decisão.

7.10.2 Regra de Decisão

Após ordenar as distâncias, o algoritmo seleciona os k vizinhos mais próximos. Seja $N_k(x)$ esse conjunto. A classe predita é dada por

$$\hat{y} = \text{moda}\{y_i : x_i \in N_k(x)\},$$

em que y_i é o rótulo da amostra x_i e $\hat{y}$ é a classe atribuída à amostra de teste.

No exemplo, para $k = 3$, os vizinhos são Fruta 1 (Maçã), Fruta 3 (Maçã) e Fruta 2 (Banana). Como **Maçã** recebe dois votos, essa é a classe predita.

💡 Classe KNeighborsClassifier

Neste capítulo, o algoritmo é implementado com a classe `KNeighborsClassifier`, da biblioteca `scikit-learn`:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
```

em que:

- `KNeighborsClassifier(n_neighbors=3)`: define o valor de k ;
- `fit(X_train, y_train)`: armazena as amostras de treinamento (`X_train`) e seus rótulos (`y_train`);
- `predict(X_test)`: retorna as classes preditas para as amostras de `X_test`.

Internamente, `predict()` executa as etapas descritas anteriormente: calcula as distâncias, identifica os k vizinhos mais próximos e determina a classe por votação majoritária.

A Figure 7.8 ilustra esse procedimento em um conjunto bidimensional. A figura destaca os vizinhos utilizados na classificação, enquanto o console apresenta as etapas do algoritmo: cálculo das distâncias, ordenação, seleção dos vizinhos, votação e predição da classe.

```
import numpy as np
import matplotlib.pyplot as plt

def knn_passo_a_passo(X, y, x, k=3):
    """Executa as cinco etapas do algoritmo k-NN.
    Parâmetros: X (treino), y (rótulos), x (teste) e k (número de vizinhos).
    """
    # 1. Distâncias
    dist = [(np.linalg.norm(x-xi), yi, i) for i, (xi, yi) in enumerate(zip(X, y))]
    print(f"1. Distâncias calculadas: {len(dist)}")

    # 2. Ordenação
    dist.sort(key=lambda t: t[0])
    print("2. Distâncias ordenadas")

    # 3. Seleção
    vizinhos = dist[:k]
    print(f"3. {k} vizinhos mais próximos:")
    for d, c, _ in vizinhos:
        print(f"    {d:.4f} → {c}")

    # 4. Votação
    votos = {}
    for _, c, _ in vizinhos:
        votos[c] = votos.get(c, 0) + 1
    print("4. Votos:", votos)

    # 5. Decisão
    classe = max(votos, key=votos.get)
    print("5. Classe predita:", classe)

    return classe, vizinhos
```

```

# Dados de exemplo
np.random.seed(4)
X = np.r_[np.random.randn(15,2)+[2,2],
          np.random.randn(15,2)+[-2,-2]]
y = np.array(["Classe A"]*15 + ["Classe B"]*15)
x = np.array([0.5,0.5])

classe, vizinhos = knn_passo_a_passo(X, y, x)

# Visualização
plt.figure(figsize=(5,5))

for c, rotulo in [("Classe A","Classe A"), ("Classe B","Classe B")]:
    P = X[y==c]
    plt.scatter(P[:,0], P[:,1], s=80, label=rotulo)

plt.scatter(*x, marker="*", s=220, edgecolors="black", label="Teste")

for _, _, i in vizinhos:
    plt.scatter(*X[i], s=220, facecolors="none", edgecolors="black", linewidths=2)
    plt.plot([x[0], X[i,0]], [x[1], X[i,1]], "--", lw=1)

plt.xlabel("Característica 1")
plt.ylabel("Característica 2")
plt.title(f"k-NN ($k=3$): classe predita = {classe}")
plt.legend()
plt.grid(alpha=.3)
plt.axis("equal")
plt.tight_layout()
plt.show()

```

1. Distâncias calculadas: 30
2. Distâncias ordenadas
3. 3 vizinhos mais próximos:
 - 1.0850 → Classe A
 - 1.6379 → Classe B
 - 1.6382 → Classe A
4. Votos: {np.str_('Classe A'): 2, np.str_('Classe B'): 1}
5. Classe predita: Classe A

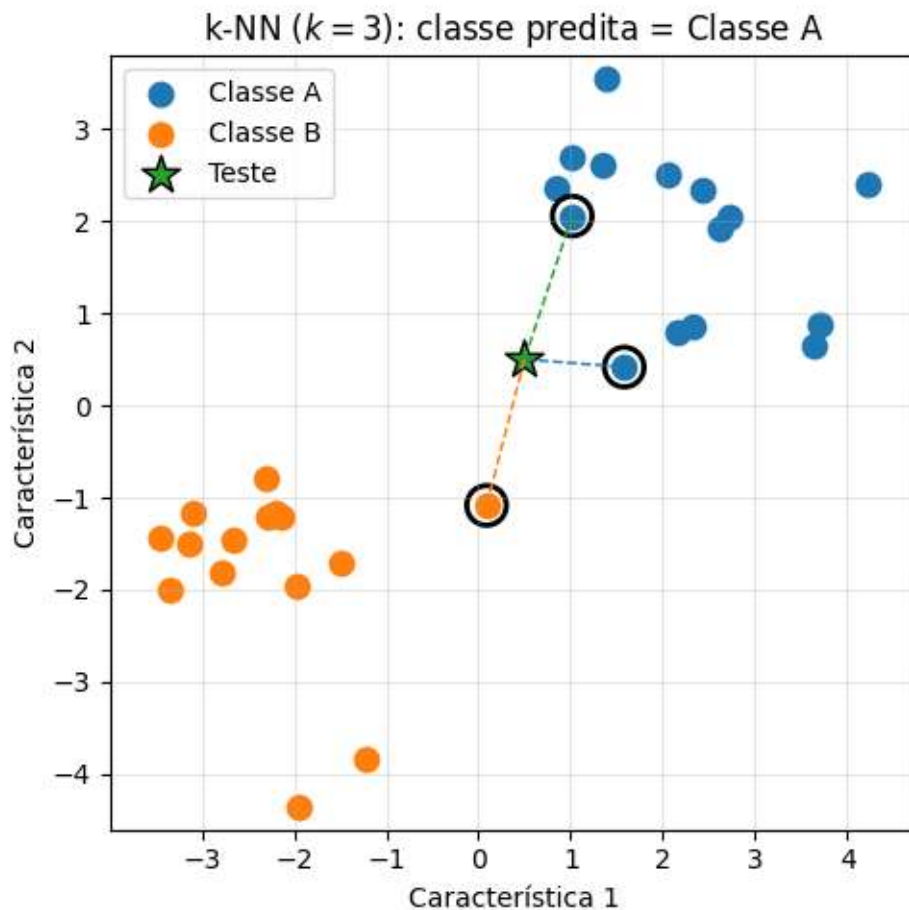


Figura 7.8: Classificação de uma nova amostra pelo algoritmo k-NN. O ponto de teste (estrela) é classificado a partir dos três vizinhos mais próximos, destacados por círculos.

7.10.3 O Papel do Parâmetro k

O parâmetro k determina quantos vizinhos participam da decisão de classificação.

- **Valores pequenos de k** (por exemplo, $k = 1$) tornam o classificador mais sensível a ruídos e variações locais, produzindo fronteiras de decisão mais irregulares e favorecendo o *overfitting*.
- **Valores maiores de k** produzem fronteiras de decisão mais suaves, porém podem reduzir a sensibilidade a estruturas locais, favorecendo o *underfitting*.

Em problemas com duas classes, é comum utilizar valores ímpares de k para reduzir a ocorrência de empates.

Outro aspecto importante é a **maldição da dimensionalidade** (*curse of dimensionality*). À medida que o número de características aumenta, as distâncias entre as amostras tendem a se tornar mais semelhantes, dificultando a identificação de vizinhos realmente representativos.

i Resumo

O algoritmo k-NN pode ser resumido em três etapas:

1. extrair o vetor de características da nova amostra;
2. identificar os k vizinhos mais próximos;
3. classificar a amostra pela classe mais frequente entre esses vizinhos.

O simulador da Figure 7.9 permite explorar visualmente o efeito do parâmetro k sobre a fronteira de decisão.



Figura 7.9: Simulador interativo da fronteira de decisão do k-NN: adicione pontos de treinamento e ajuste o valor de k para observar o efeito sobre a região de decisão.

7.11 Projeto Prático 1: Classificação de Dígitos Manuscritos com k-NN

As seções anteriores apresentaram o algoritmo k-NN por meio de um exemplo simplificado de classificação de frutas, utilizando apenas duas características. A seguir, o mesmo algoritmo é aplicado a um conjunto de dados de imagens, no qual cada amostra é representada por um vetor de maior dimensão.

Como estudo de caso, utiliza-se a base pública `load_digits`, disponibilizada pela biblioteca `scikit-learn`. Esse conjunto de dados contém 1797 imagens de dígitos manuscritos das classes de 0 a 9, cada uma com resolução de 8×8 pixels em níveis de cinza. Cada imagem é representada por um vetor com 64 características, correspondentes às intensidades dos pixels, e cada vetor possui um rótulo indicando o dígito correspondente.

A base `load_digits` é disponibilizada pela biblioteca `scikit-learn` e é utilizada neste capítulo para ilustrar a aplicação do algoritmo k-NN. Além de estar disponível diretamente no `scikit-learn`, ela dispensa etapas adicionais de obtenção e preparação dos dados, permitindo concentrar a atenção na implementação e na avaliação do classificador.

A Figure 7.10 apresenta uma amostra das imagens da base de dados.

```
from sklearn.datasets import load_digits

digits = load_digits()
print(f"Total de amostras: {digits.data.shape[0]}, dimensão do vetor: {digits.data.shape[1]}")
print(f"Classes: {[int(i) for i in sorted(set(digits.target))]}")

n_amstras = 16
imgs = list(digits.images[:n_amstras])
imgs_titles = [str(label) for label in digits.target[:n_amstras]]
mm.show(imgs, titles=imgs_titles, cols=8, figsize=(12, 4))
```

```
Total de amostras: 1797, dimensão do vetor: 64
Classes: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

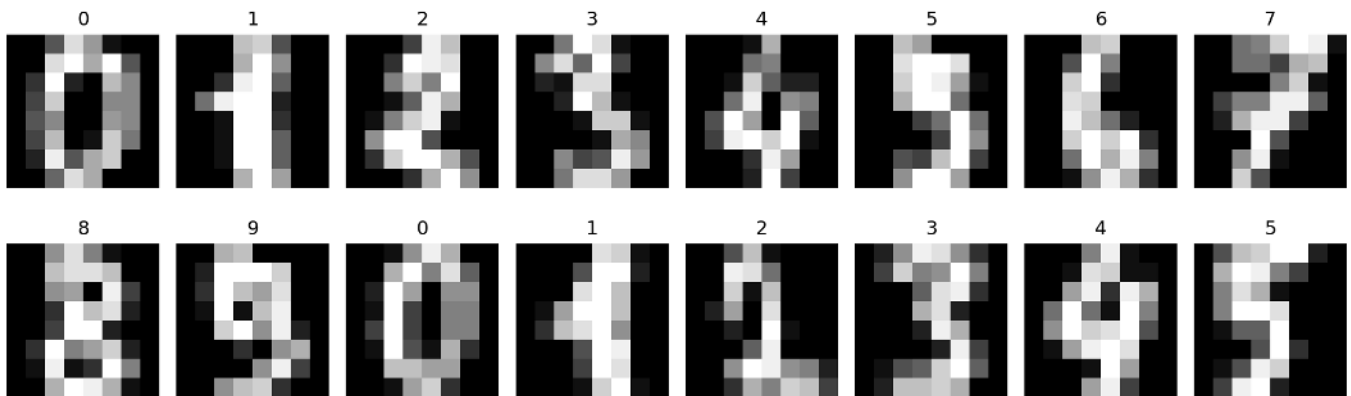


Figura 7.10: Amostra de dígitos manuscritos da base `load_digits`, utilizada como estudo de caso de classificação.

7.11.1 Classificação com Vetores de Intensidade

Neste primeiro experimento, cada imagem de dimensão 8×8 é representada diretamente pelas intensidades de seus 64 pixels, sem a extração de descritores adicionais. Assim, cada amostra corresponde a um vetor de 64 características, utilizado como entrada do classificador k-NN.

Em seguida, o conjunto de dados é dividido em subconjuntos de treinamento e teste, preservando a proporção das dez classes por meio do parâmetro `stratify=y`. O classificador é treinado com $k = 3$ e avaliado sobre o conjunto de teste utilizando a acurácia e a matriz de confusão apresentada na Figure 7.11.

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = digits.data, digits.target

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

n_neighbors = 3
knn_pixels = KNeighborsClassifier(n_neighbors=n_neighbors)
knn_pixels.fit(X_treino, y_treino)
pred_pixels = knn_pixels.predict(X_teste)

acc_pixels = accuracy_score(y_teste, pred_pixels)
print(f"Acurácia (vetores de intensidade, k={n_neighbors}): {acc_pixels:.4f}")

cm = confusion_matrix(y_teste, pred_pixels)

plt.figure(figsize=(5,4))
plt.imshow(cm, cmap="Blues")

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(
            j, i, cm[i, j],
            ha="center", va="center",
            color="white" if cm[i, j] > cm.max()/2 else "black",
            fontsize=9
        )

plt.title("Matriz de Confusão - Pixels Brutos")
plt.xlabel("Classe Predita")
plt.ylabel("Classe Real")
plt.xticks(range(10))
plt.yticks(range(10))
plt.colorbar(fraction=0.046)
plt.tight_layout()
plt.show()
```

Acurácia (vetores de intensidade, k=3): 0.9870

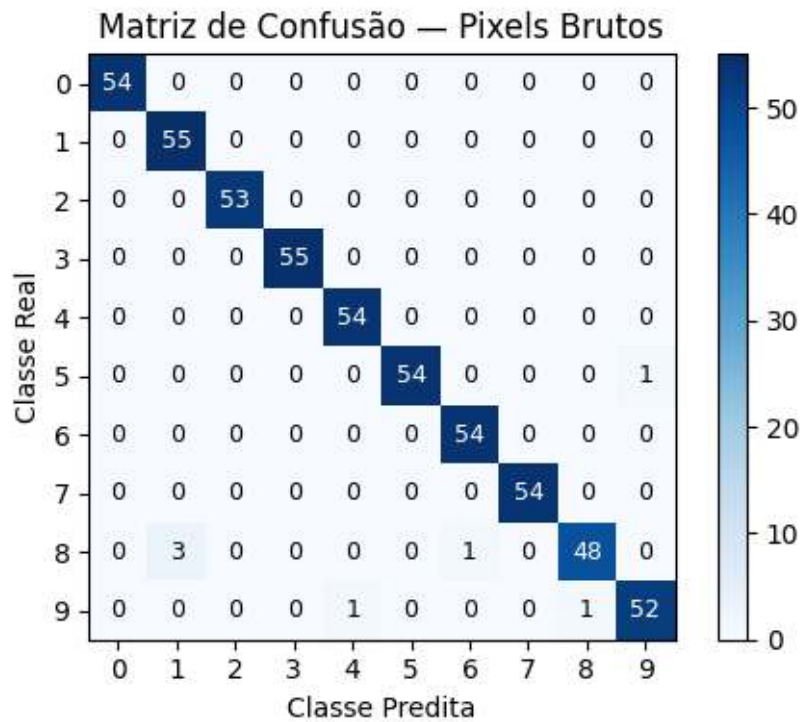


Figura 7.11: Matriz de confusão do classificador k-NN treinado com vetores de intensidade brutos (pixels).

7.11.2 Classificação com Descritores HOG

No experimento anterior, cada imagem foi representada diretamente pelas intensidades de seus pixels. Nesta seção, essa representação é substituída por descritores HOG (*Histogram of Oriented Gradients*), que codificam informações sobre a distribuição das orientações dos gradientes da imagem.

Mantêm-se o mesmo particionamento dos dados, o mesmo classificador k-NN e o mesmo protocolo de avaliação, alterando apenas a representação das imagens. A Figure 7.12 compara os resultados obtidos com vetores de intensidade e com descritores HOG.

```
descritores_hog = np.array([
    hog(img, orientations=8, pixels_per_cell=(4, 4), cells_per_block=(1, 1))
    for img in digits.images
])
print(f"Dimensão do vetor HOG: {descritores_hog.shape[1]}")

Xh_treino, Xh_teste, yh_treino, yh_teste = train_test_split(
    descritores_hog, y, test_size=0.3, random_state=42, stratify=y
)

n_neighbors = 3
knn_hog = KNeighborsClassifier(n_neighbors=n_neighbors)
knn_hog.fit(Xh_treino, yh_treino)
pred_hog = knn_hog.predict(Xh_teste)
acc_hog = accuracy_score(yh_teste, pred_hog)
print(f"Acurácia (descritores HOG, k={n_neighbors}): {acc_hog:.4f}")

plt.figure(figsize=(4, 3))
plt.bar(["Pixels brutos", "HOG"], [acc_pixels, acc_hog], color=["#6366f1", "#f97316"])
plt.ylim(0, 1.1) # Aumenta o limite superior para dar espaço
plt.ylabel("Acurácia")
plt.title("Comparação de Descritores")
for i, v in enumerate([acc_pixels, acc_hog]):
```



```
plt.text(i, v + 0.02, f"{v:.3f}", ha="center") # Aumenta o deslocamento vertical
plt.tight_layout()
```

Dimensão do vetor HOG: 32

Acurácia (descriptor HOG, k=3): 0.7593

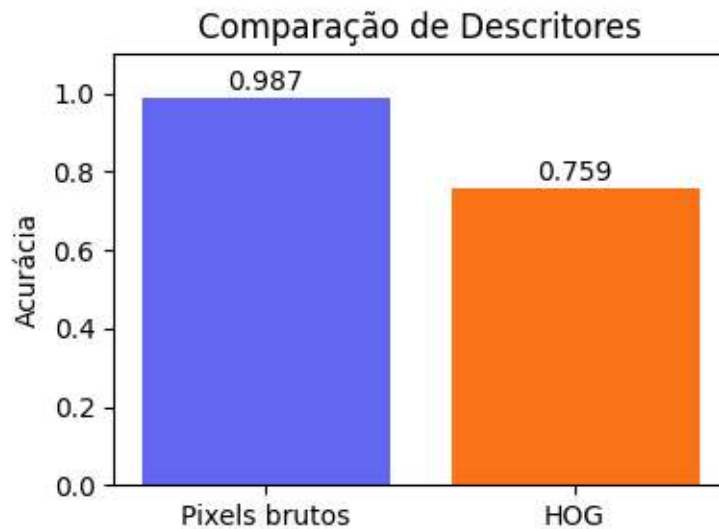


Figura 7.12: Comparação de acurácia entre descritores de pixels brutos e HOG para o classificador k-NN (k=3) na base de dígitos.

i Por que isso acontece?

Na Figure 7.12, o classificador treinado com vetores de **intensidade dos pixels** alcança maior acurácia (0.987) do que aquele baseado em descritores **HOG** (0.759). Esse resultado está relacionado às características da base `load_digits`.

As imagens possuem resolução de apenas 8×8 pixels, encontram-se aproximadamente centralizadas e apresentam pouca variação de iluminação, escala e orientação. Nesse cenário, as intensidades dos pixels preservam praticamente toda a informação necessária para distinguir as classes. Em contraste, o HOG resume a imagem em histogramas de orientações dos gradientes, reduzindo parte do detalhamento espacial disponível nos pixels originais.

Essa redução de informação pode dificultar a separação de dígitos visualmente semelhantes, como 3 e 8 ou 4 e 9, especialmente quando a resolução da imagem é baixa.

Em problemas com imagens de maior resolução ou sujeitas a variações de iluminação, posição, escala ou pequenas deformações, descritores como o HOG tendem a representar melhor a estrutura local da imagem do que os valores individuais dos pixels. Assim, este experimento ilustra um princípio importante da Aprendizagem de Máquina: **a representação dos dados deve ser escolhida de acordo com as características do problema e não pela complexidade do descritor.**

7.12 Avaliação de Classificadores

Nas seções anteriores, a qualidade do classificador foi analisada por meio da acurácia e da matriz de confusão. Nesta seção, essas ferramentas são complementadas por métricas utilizadas na avaliação de modelos e por um procedimento para selecionar o valor do parâmetro k .

A acurácia corresponde à proporção de amostras classificadas corretamente. Embora seja uma medida simples e amplamente utilizada, ela pode ser insuficiente quando as classes apresentam distribuições muito desbalanceadas.

A partir da matriz de confusão — introduzida no **Capítulo 1** e utilizada ao longo deste capítulo — podem ser calculadas métricas por classe, como precisão e revocação:

$$\text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Revocação} = \frac{VP}{VP + FN},$$

em que VP , FP e FN representam, respectivamente, o número de verdadeiros positivos, falsos positivos e falsos negativos da classe analisada. A precisão quantifica a proporção de predições positivas corretas, enquanto a revocação mede a capacidade do classificador de identificar os exemplos pertencentes à classe.

7.12.1 Escolha de k por Validação Cruzada

Nos experimentos anteriores, adotou-se $k = 3$ para ilustrar o funcionamento do algoritmo. Entretanto, esse parâmetro influencia diretamente o desempenho do classificador e, na prática, deve ser selecionado a partir dos dados.

Uma abordagem amplamente utilizada é a **validação cruzada** (*cross-validation*), na qual o conjunto de treinamento é dividido em partições sucessivas para estimar o desempenho do modelo em dados não utilizados durante o treinamento.

O código a seguir calcula a acurácia média obtida por validação cruzada de cinco partições (*5-fold cross-validation*) para diferentes valores de k . A Figure 7.13 apresenta os resultados, permitindo identificar a região em que o classificador atinge melhor desempenho.

```
from sklearn.model_selection import cross_val_score

valores_k = range(1, 16)
acuracias_medias = []

for k in valores_k:
    modelo = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(modelo, X, y, cv=5)
    acuracias_medias.append(scores.mean())

melhor_k = list(valores_k)[int(np.argmax(acuracias_medias))]
print(f"Melhor valor de k encontrado: {melhor_k} (acurácia média={max(acuracias_medias):.4f})")

plt.figure(figsize=(6, 4))
plt.plot(list(valores_k), acuracias_medias, marker="o", color="#4f46e5")
plt.axvline(melhor_k, color="#f97316", linestyle="--", label=f"melhor k = {melhor_k}")
plt.xlabel("k")
plt.ylabel("Acurácia média (validação cruzada)")
plt.title("Seleção de k por Validação Cruzada")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

Melhor valor de k encontrado: 2 (acurácia média=0.9672)



Figura 7.13: Acurácia média por validação cruzada (5 partições) em função do parâmetro k , para a base de dígitos com vetores de intensidade brutos.

7.12.2 O Compromisso entre Viés e Variância: Diagnóstico de *Overfitting* e *Underfitting*

O hiperparâmetro k influencia a complexidade da fronteira de decisão do classificador k -NN e, consequentemente, sua capacidade de generalização. Em termos gerais, valores pequenos de k tornam o modelo mais sensível às amostras de treinamento, enquanto valores maiores produzem fronteiras de decisão mais suaves.

Esses comportamentos estão associados ao compromisso entre **viés** (*bias*) e **variância** (*variance*). Valores muito pequenos de k tendem a aumentar o risco de **sobreajuste** (*overfitting*), especialmente em conjuntos de dados ruidosos, ao passo que valores muito grandes podem levar ao **subajuste** (*underfitting*), reduzindo a capacidade do modelo de capturar estruturas locais dos dados.

Enquanto a Figure 7.13 apresentou apenas a acurácia média obtida por validação cruzada, a Figure 7.14 compara as acurácias de treinamento e de teste para diferentes valores de k . As regiões destacadas no gráfico representam o comportamento esperado do algoritmo: maior risco de sobreajuste para valores pequenos de k , uma região intermediária que frequentemente produz bom equilíbrio entre viés e variância e maior risco de subajuste para valores elevados de k .

Entretanto, essas regiões devem ser interpretadas apenas como uma referência conceitual. O comportamento observado depende das características do conjunto de dados. Na base `load_digits`, por exemplo, as imagens apresentam pouca variabilidade e boa separação entre as classes, de modo que valores pequenos de k podem apresentar desempenho semelhante — ou até superior — aos demais, sem evidenciar um sobreajuste significativo.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

k_values = range(1, 16)
train_acc, test_acc = [], []
```

```

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_treino, y_treino)
    train_acc.append(accuracy_score(y_treino, knn.predict(X_treino)))
    test_acc.append(accuracy_score(y_teste, knn.predict(X_teste)))

plt.figure(figsize=(9,5))
plt.plot(k_values, train_acc, "o-", lw=2, label="Treinamento")
plt.plot(k_values, test_acc, "s-", lw=2, label="Teste")

plt.axvspan(1, 3, color="#fca5a5", alpha=.25, label="Maior risco de overfitting")
plt.axvspan(3,11, color="#86efac", alpha=.25, label="Compromisso entre viés e variância")
plt.axvspan(11,15, color="#93c5fd", alpha=.25, label="Maior risco de underfitting")

plt.xlabel("Número de vizinhos ($k$)")
plt.ylabel("Acurácia")
plt.xticks(k_values)
plt.grid(alpha=.3)
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()

maior_acc = max(test_acc)
melhores_k = [k for k, a in zip(k_values, test_acc) if np.isclose(a, maior_acc)]

print("Interpretação")
print("- Valores pequenos de k: maior risco de overfitting.")
print("- Valores intermediários: melhor compromisso entre viés e variância.")
print("- Valores grandes de k: maior risco de underfitting.")
print("\nAs regiões coloridas representam tendências gerais;")
print("o comportamento observado depende do conjunto de dados.")
print(f"\nMaior acurácia no teste: {maior_acc:.3f}")
print(f"Valores de k que atingiram essa acurácia: {melhores_k}")

```

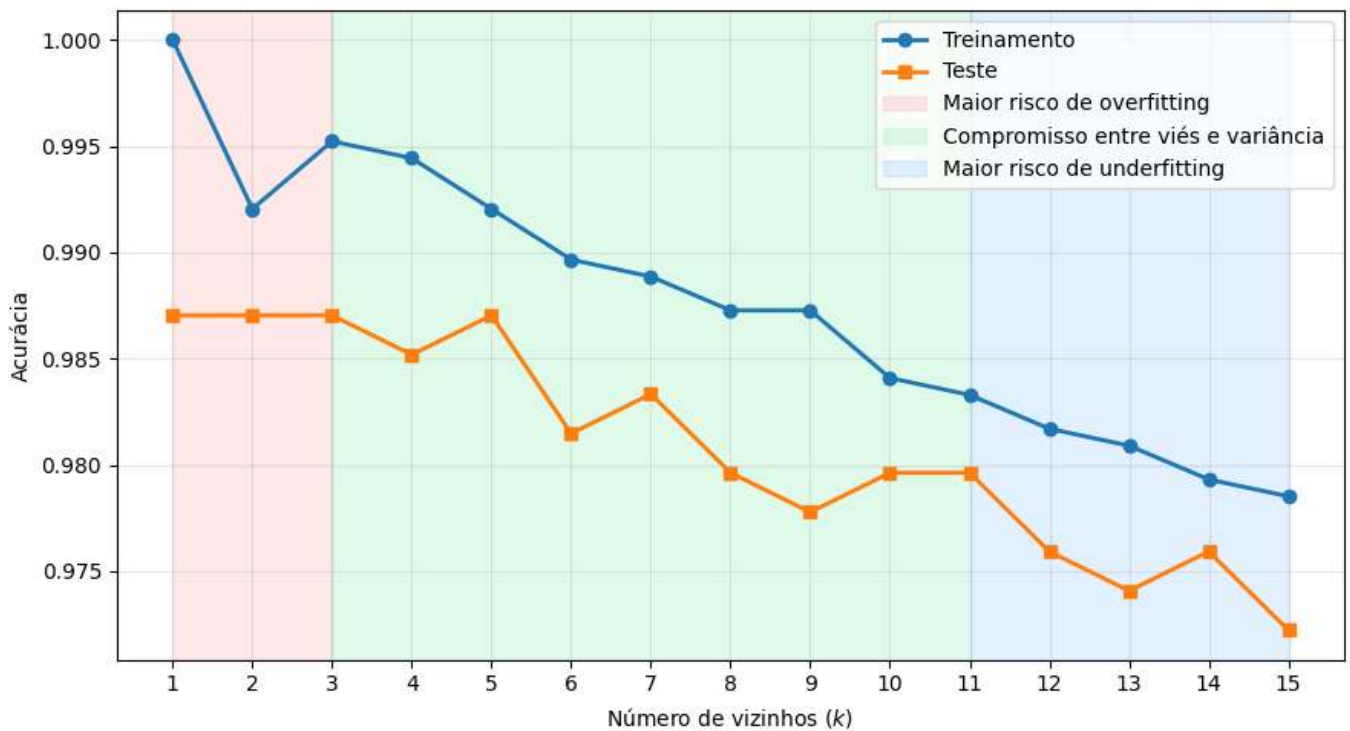


Figura 7.14: Acurácia nos conjuntos de treinamento e teste para diferentes valores de k . As regiões coloridas representam, de forma conceitual, tendências de comportamento do classificador: maior risco de sobreajuste (vermelho), compromisso entre viés e variância (verde) e maior risco de subajuste (azul).

Interpretação

- Valores pequenos de k : maior risco de overfitting.
- Valores intermediários: melhor compromisso entre viés e variância.
- Valores grandes de k : maior risco de underfitting.

As regiões coloridas representam tendências gerais;
o comportamento observado depende do conjunto de dados.

Maior acurácia no teste: 0.987

Valores de k que atingiram essa acurácia: [1, 2, 3, 5]

7.13 Projeto Prático 2: Classificação de Texturas com Descritores LBP

No **Capítulo 6**, a variância local de textura foi utilizada para **detectar** anomalias em superfícies industriais, distinguindo amostras **conformes** e **defeituosas**. Neste projeto, o mesmo contexto é ampliado para um problema de **classificação multiclasse**: dado um recorte de textura, determinar a qual categoria ele pertence utilizando o descritor LBP em conjunto com o classificador k -NN.

Para tornar a avaliação mais representativa, as texturas sintéticas geradas neste experimento incorporam ruído gaussiano e variabilidade intra-classe, produzindo diferenças de amplitude, frequência e contraste entre amostras de uma mesma categoria. Essas variações simulam, de forma controlada, fatores comuns na aquisição de imagens, como ruído do sensor, mudanças de iluminação e pequenas diferenças de foco.

Sem essa variabilidade, as classes tenderiam a ser perfeitamente separáveis, resultando em classificações praticamente sem erros. Embora esse cenário facilite a tarefa do classificador, ele pouco contribui para a análise de seu comportamento. Ao introduzir uma sobreposição parcial entre as classes, o experimento passa a produzir falsos positivos e falsos negativos, permitindo interpretar a matriz de confusão e analisar, de forma mais informativa, métricas como precisão, revocação e F1-score.

São geradas três classes de textura sintéticas: **granular** (ruído gaussiano suavizado), **listrada** (padrão

periódico) e **manchada** (regiões circulares de intensidade variável). A Figure 7.15 apresenta exemplos de cada uma delas.

```
rng = np.random.default_rng(42)

def gerar_textura(classe, tamanho=64, ruido=0.10):
    """Gera uma textura sintética 64x64 pertencente a uma das três classes."""
    if classe == "granular":
        escala = rng.uniform(0.14, 0.22)
        img = rng.normal(0.5, escala, (tamanho, tamanho))
        img = cv2.GaussianBlur(img.astype(np.float32), (3, 3), 0)

    elif classe == "listrada":
        n_periodos = rng.uniform(4, 8)
        amplitude = rng.uniform(0.22, 0.38)
        eixo_x = np.linspace(0, n_periodos * np.pi, tamanho)
        base = 0.5 + amplitude * np.sin(eixo_x)
        img = np.tile(base, (tamanho, 1)).astype(np.float32)
        img += rng.normal(0, 0.09, (tamanho, tamanho)).astype(np.float32)

    elif classe == "manchada":
        img = np.full((tamanho, tamanho), 0.5, dtype=np.float32)
        n_manchas = rng.integers(5, 11)
        for _ in range(n_manchas):
            cx, cy = rng.integers(0, tamanho, 2)
            raio = int(rng.integers(3, 11))
            intensidade = float(rng.uniform(0.15, 0.9))
            cv2.circle(img, (int(cx), int(cy)), raio, intensidade, -1)
        img = cv2.GaussianBlur(img, (5, 5), 0)

    else:
        raise ValueError(f"Classe desconhecida: {classe}")

    # Ruído gaussiano adicional, comum a todas as classes.
    img = img + rng.normal(0, ruido, (tamanho, tamanho)).astype(np.float32)
    img = np.clip(img, 0, 1)
    return (img * 255).astype(np.uint8)

classes_textura = ["granular", "listrada", "manchada"]
amostras = [gerar_textura(c) for c in classes_textura]

mm.show(amostras, titles=classes_textura, cols=3, figsize=(9, 3))
```

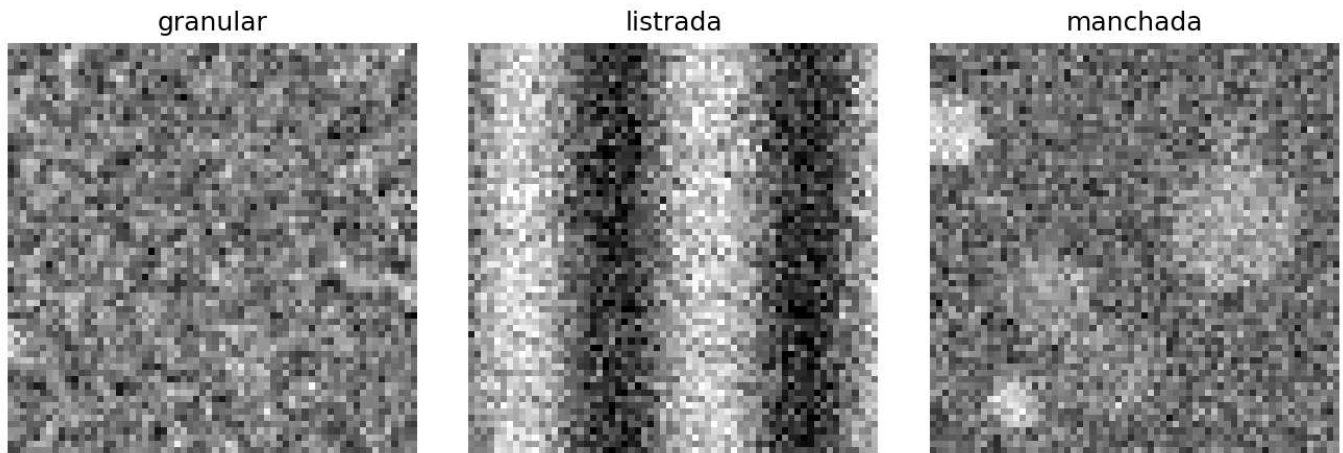



Figura 7.15: Amostras sintéticas das três classes de textura utilizadas no experimento de classificação, geradas com ruído gaussiano e variabilidade intra-classe.

7.13.1 Extração do Descritor LBP e Treinamento do Classificador

Cada imagem é representada por um descritor LBP uniforme. A partir dos códigos produzidos pelo LBP, constrói-se um histograma normalizado, que passa a representar numericamente a textura e constitui o vetor de características utilizado pelo classificador k -NN.

Neste experimento, são geradas 60 imagens para cada classe de textura. Após a extração dos descritores, o conjunto de dados é dividido em 70% para treinamento e 30% para teste, preservando a proporção entre as classes. Em seguida, um classificador k -NN com $k = 5$ é treinado e avaliado sobre o conjunto de teste.

Como as texturas apresentam ruído e variabilidade intra-classe, amostras de categorias diferentes podem produzir descritores semelhantes. A matriz de confusão apresentada na Figure 7.16 permite verificar como essas semelhanças afetam a classificação, evidenciando tanto os acertos quanto as confusões entre as classes.

```
def descritor_lbp(img, P=8, R=1, bins=10):
    lbp = local_binary_pattern(img, P=P, R=R, method="uniform")
    hist, _ = np.histogram(lbp, bins=bins, range=(0, P + 2), density=True)
    return hist

X_textura, y_textura = [], []
for classe in classes_textura:
    for _ in range(60):
        img = gerar_textura(classe, ruído=0.10)
        X_textura.append(descritor_lbp(img))
        y_textura.append(classe)

X_textura = np.array(X_textura)
y_textura = np.array(y_textura)

Xt_treino, Xt_teste, yt_treino, yt_teste = train_test_split(
    X_textura, y_textura, test_size=0.3, random_state=0, stratify=y_textura
)

knn_textura = KNeighborsClassifier(n_neighbors=5)
knn_textura.fit(Xt_treino, yt_treino)
pred_textura = knn_textura.predict(Xt_teste)

acc_textura = accuracy_score(yt_teste, pred_textura)
print(f"Acurácia (descritor LBP, k=5): {acc_textura:.4f}")

cm_textura = confusion_matrix(yt_teste, pred_textura, labels=classes_textura)
```



```
plt.figure(figsize=(4.5, 4))
plt.imshow(cm_textura, cmap="Purples")
plt.xticks(range(3), classes_textura, rotation=20)
plt.yticks(range(3), classes_textura)
plt.xlabel("Classe Predita")
plt.ylabel("Classe Real")
plt.title("Matriz de Confusão - Texturas (LBP)")
for i in range(3):
    for j in range(3):
        plt.text(j, i, cm_textura[i, j], ha="center", va="center",
                 color="white" if cm_textura[i, j] > cm_textura.max()/2 else "black")
plt.colorbar(fraction=0.046)
plt.tight_layout()
```

Acurácia (descriptor LBP, k=5): 0.7778

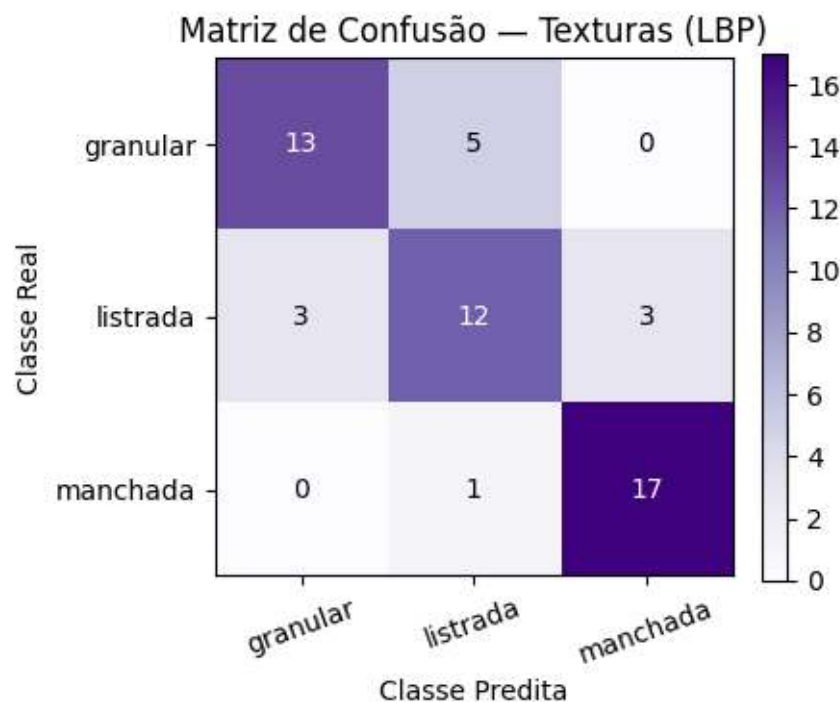


Figura 7.16: Matriz de confusão do classificador k-NN treinado com descritores LBP para as três classes de textura sintética, incluindo ruído e variabilidade intra-classe.

i Por que funciona? — LBP como assinatura de textura

O descriptor LBP resume a distribuição dos padrões locais de intensidade presentes na imagem por meio de um histograma normalizado. Em vez de armazenar os valores dos pixels ou suas posições, o histograma registra a frequência com que cada padrão local ocorre, produzindo uma representação compacta da textura.

Como cada classe de textura é gerada por um processo diferente (granular, listrado ou manchado), seus histogramas tendem a apresentar distribuições distintas. Entretanto, o ruído gaussiano e a variabilidade introduzida na geração das imagens tornam algumas amostras mais semelhantes entre si, reduzindo a separação entre as classes no espaço de características. Essa sobreposição explica por que o classificador pode confundir determinadas texturas, mesmo quando o desempenho global permanece elevado.

Outra característica importante do LBP é que o descriptor utiliza apenas a frequência dos padrões locais, descartando sua posição exata na imagem. Essa representação reduz a dimensionalidade dos dados e contribui para que pequenas translações e variações locais tenham impacto limitado sobre o vetor de

características, tornando o descritor adequado para tarefas de classificação de texturas.

7.13.2 Diagnóstico Fino do Classificador: Precisão, Revocação e F1-Score

A acurácia resume o desempenho do classificador em um único valor, mas não indica como esse desempenho se distribui entre as diferentes classes. Para uma análise mais detalhada, utilizam-se métricas calculadas individualmente para cada classe.

A **precisão** (*precision*) mede a proporção de amostras classificadas como pertencentes a uma classe que realmente pertencem a ela. A **revocação** (*recall*) mede a proporção de amostras da classe que foram corretamente identificadas pelo classificador. O **F1-score** corresponde à média harmônica entre precisão e revocação, fornecendo um indicador que equilibra ambas as medidas.

O relatório também informa o **suporte** (*support*), isto é, o número de amostras de cada classe presentes no conjunto de teste. Essa informação é importante para contextualizar as métricas, pois resultados obtidos sobre poucas amostras tendem a apresentar maior variabilidade.

A Figure 7.17 apresenta essas métricas para as três classes de textura. Em conjunto, elas permitem identificar diferenças de desempenho que não são evidentes apenas pela acurácia. Por exemplo, uma classe pode apresentar alta precisão e menor revocação, indicando que o classificador comete poucos falsos positivos, mas deixa de identificar parte das amostras que realmente pertencem àquela classe. Esse tipo de análise auxilia na compreensão das limitações do modelo e na identificação de possíveis estratégias para seu aprimoramento.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, precision_score, recall_score, f1_score

y_pred = knn_textura.predict(Xt_teste)

report = classification_report(
    yt_teste,
    y_pred,
    target_names=classes_textura,
    output_dict=True
)

print("=== RELATÓRIO DE CLASSIFICAÇÃO DETALHADO ===")
print(f"{'Classe':<12} {'Precisão':>10} {'Revocação':>12} {'F1-score':>10} {'Suporte':>10}")
for classe in classes_textura:
    r = report[classe]
    print(f"{'classe':<12} {r['precision']:>10.2f} {r['recall']:>12.2f} "
          f"{r['f1-score']:>10.2f} {r['support']:>10.0f}")

precision = precision_score(yt_teste, y_pred, average=None, labels=classes_textura)
recall = recall_score(yt_teste, y_pred, average=None, labels=classes_textura)
f1 = f1_score(yt_teste, y_pred, average=None, labels=classes_textura)

fig, ax = plt.subplots(figsize=(10, 5))
x = np.arange(len(classes_textura))
width = 0.25

bars1 = ax.bar(x - width, precision, width, label='Precisão', color='#6366f1', alpha=0.8)
bars2 = ax.bar(x, recall, width, label='Revocação', color='#f97316', alpha=0.8)
bars3 = ax.bar(x + width, f1, width, label='F1-Score', color='#22c55e', alpha=0.8)

ax.set_xlabel('Classe', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Métricas por Classe - Classificação de Texturas', fontsize=14, fontweight='bold')
ax.set_xticks(x)
```

```

ax.set_xticklabels(classes_textura)
ax.legend(loc='upper right')
ax.set_ylim(0, 1.35)
ax.grid(axis='y', alpha=0.3)

for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.02,
                f'{height:.2f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

print("Interpretação das métricas:")
print("- Precisão: entre as amostras classificadas como pertencentes à classe, ",
      "quantas estavam corretas?")
print("- Revocação: entre as amostras que realmente pertencem à classe, quantas ",
      "foram identificadas?")
print("- F1-Score: média harmônica entre precisão e revocação.")
print("- Suporte: número de amostras reais de cada classe presentes no conjunto de teste.")
print("\nO suporte não mede desempenho; ele apenas informa quantos exemplos de cada ",
      "classe foram utilizados na avaliação.")

```

=== RELATÓRIO DE CLASSIFICAÇÃO DETALHADO ===

Classe	Precisão	Revocação	F1-score	Suporte
granular	0.81	0.72	0.76	18
listrada	0.67	0.67	0.67	18
manchada	0.85	0.94	0.89	18

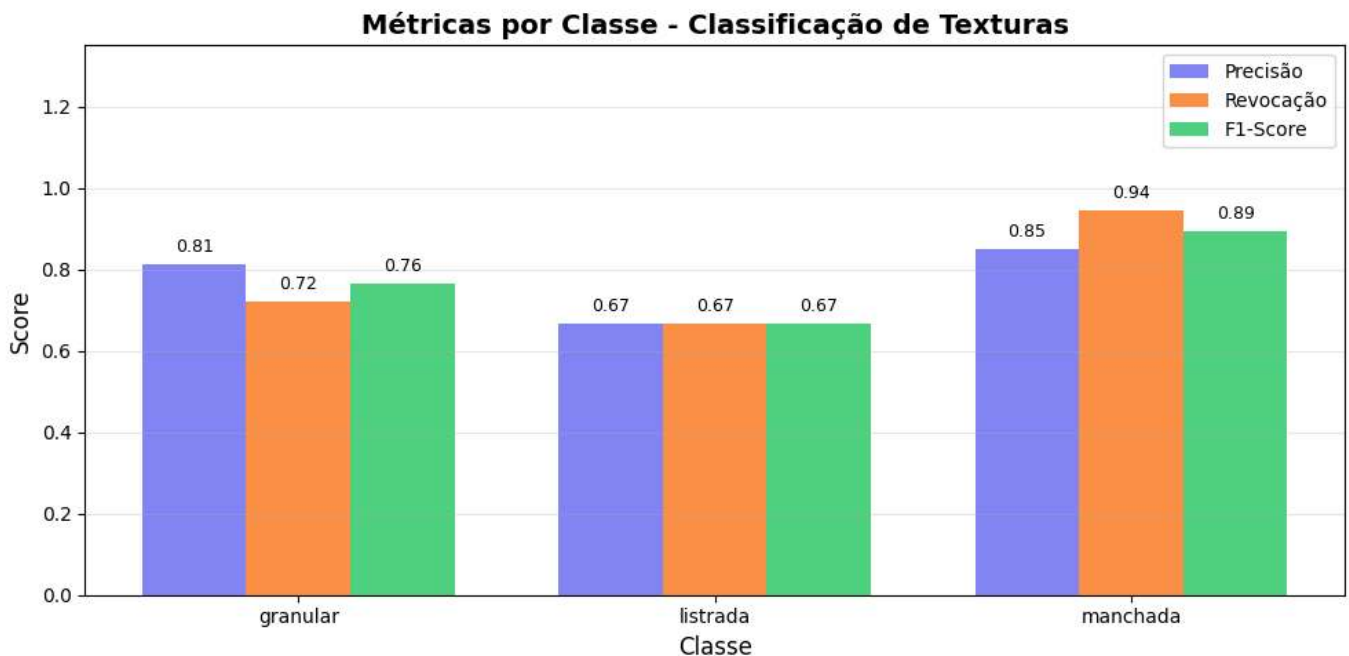


Figura 7.17: Métricas de avaliação detalhadas para o classificador k-NN com descritores LBP

Interpretação das métricas:

- Precisão: entre as amostras classificadas como pertencentes à classe, quantas estavam corretas?
- Revocação: entre as amostras que realmente pertencem à classe, quantas foram identificadas?
- F1-Score: média harmônica entre precisão e revocação.
- Suporte: número de amostras reais de cada classe presentes no conjunto de teste.

O suporte não mede desempenho; ele apenas informa quantos exemplos de cada classe foram utilizados na avaliação.

7.14 Limitações dos Descritores Artesanais

Os experimentos deste capítulo mostram que descritores clássicos podem ser bastante eficazes em tarefas de classificação, mas também apresentam limitações importantes:

- **Especificidade:** cada descritor foi desenvolvido para representar um determinado tipo de informação, como cor, textura ou forma. Assim, um descritor adequado para uma tarefa pode não ser o mais apropriado para outra.
- **Dependência de hiperparâmetros:** o desempenho de descritores como LBP e HOG depende da escolha de parâmetros, como raio de vizinhança, número de pontos amostrados, tamanho da célula e número de orientações, que precisam ser ajustados conforme a aplicação.
- **Representação limitada:** descritores de cor, textura e gradiente capturam propriedades de baixo nível da imagem, mas não representam diretamente conceitos semânticos mais complexos, como objetos ou cenas.
- **Maldição da dimensionalidade:** descritores muito extensos podem reduzir a eficácia de classificadores baseados em distância, como o k -NN.

Essas limitações motivam a evolução das técnicas estudadas nos próximos capítulos. O **Capítulo 8** apresenta métodos clássicos para detecção e correspondência de características em imagens, enquanto o **Capítulo 9** introduz as **Redes Neurais Convolucionais**, capazes de aprender automaticamente representações adequadas para cada tarefa a partir dos dados.

7.15 Resumo

Neste capítulo foram apresentados os fundamentos do reconhecimento de padrões aplicado a imagens. Os principais conceitos estudados foram:

- **Pipeline de reconhecimento de padrões:** aquisição, pré-processamento, extração de descritores, classificação e avaliação.
- **Descritores clássicos:** descritores de cor, LBP para textura e HOG para forma, utilizados para representar diferentes características das imagens.
- **Normalização de características:** padronização (Z -score) para evitar que atributos de maior magnitude dominem o cálculo das distâncias.
- **Classificador k -NN:** classificação baseada nos k vizinhos mais próximos no espaço de características.
- **Escolha do parâmetro k :** influência do valor de k sobre o desempenho do classificador e uso da validação cruzada para sua seleção.
- **Avaliação de classificadores:** acurácia, matriz de confusão, precisão, revocação e F1-score como métricas complementares de desempenho.
- **Limitações dos descritores artesanais:** especificidade, dependência de hiperparâmetros e dificuldade em representar informações de alto nível.

Os conceitos foram ilustrados por meio de experimentos com a base pública `load_digits`, texturas sintéticas geradas para fins didáticos e dados simulados de descritores de frutas.

7.16 Uso do NotebookLM como Tutor

Nesta edição, o **NotebookLM** é apresentado como uma ferramenta de apoio ao estudo. O sistema utiliza exclusivamente os documentos disponibilizados pelo autor como fonte de conhecimento, permitindo explorar os conceitos do capítulo por meio de perguntas, resumos e explicações relacionadas ao material estudado.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 07](#)

 Aviso sobre Conteúdo Gerado por IA

As respostas fornecidas pelo NotebookLM podem conter imprecisões ou omissões. Sempre que necessário, confirme as informações utilizando o material deste capítulo e outras fontes acadêmicas confiáveis. A execução dos exemplos práticos apresentados ao longo do texto continua sendo a melhor forma de consolidar os conceitos estudados.

7.17 Lista de Exercícios

Os exercícios a seguir exploram e estendem os conceitos apresentados neste capítulo por meio de adaptações dos algoritmos implementados, análises experimentais e comparações entre diferentes abordagens.

1. **(10%)** Implemente um descritor de cor (histograma RGB ou HSV, com pelo menos 16 *bins* por canal) para as três classes de frutas simuladas na Figure 7.1. Treine um classificador k -NN com esse descritor, compare sua acurácia com a obtida pelos descritores LBP e HOG (Figure 7.7) e discuta em quais situações a informação de cor é mais discriminativa.
2. **(15%)** Investigue o efeito da normalização de características (*Z-score*) sobre o desempenho do k -NN em um espaço de atributos heterogêneo, combinando descritores de cor, LBP e HOG em um único vetor. Compare os resultados obtidos com e sem normalização para pelo menos três valores de k .
3. **(15%)** Reproduza a análise de *overfitting* e *underfitting* da Figure 7.14 variando o tamanho do conjunto de treinamento (por exemplo, 20%, 50% e 80% da base `load_digits`). Discuta como a quantidade de exemplos influencia a escolha do valor de k .
4. **(15%)** Estenda o Projeto Prático 2 adicionando uma quarta classe sintética de textura. Avalie precisão, revocação e F1-score para cada classe, seguindo o padrão da Figure 7.17, e analise o impacto da nova classe na matriz de confusão.
5. **(15%)** Implemente manualmente o classificador k -NN, sem utilizar `sklearn`:

```
sklearn.neighbors.KNeighborsClassifier,
```

completando a função `knn_passo_a_passo` apresentada no capítulo. Compare a acurácia e o tempo de execução da implementação manual com a implementação do `scikit-learn` em conjuntos de dados de tamanhos crescentes e relacione os resultados à maldição da dimensionalidade.

6. **(15%)** Investigue a influência dos parâmetros `orientations`, `pixels_per_cell` e `cells_per_block` do descritor HOG na base `load_digits`. Avalie pelo menos quatro combinações de parâmetros e discuta o compromisso entre dimensionalidade do descritor e desempenho do classificador.
7. **(15%)** Avalie a influência dos parâmetros P (número de vizinhos) e R (raio) do descritor LBP na classificação das texturas sintéticas do Projeto Prático 2, considerando $P \in \{4, 8, 16\}$ e $R \in \{1, 2, 3\}$. Analise como esses parâmetros afetam a capacidade discriminativa do descritor.
8. **(Bônus – 10%)** Implemente manualmente a validação cruzada k -fold para o classificador k -NN na base `load_digits`, sem utilizar `cross_val_score`, e compare os resultados com os obtidos pela implementação do `scikit-learn` apresentada na Figure 7.13.

Referências do Capítulo

A fundamentação teórica e os experimentos apresentados neste capítulo baseiam-se nas seguintes referências:

- Gonzalez; Woods (2018), pelos fundamentos de descritores estatísticos de textura e das operações de pré-processamento aplicadas à extração de características.
- Szeliski (2022), pela apresentação do pipeline clássico de reconhecimento de padrões, da extração de descritores e da avaliação de classificadores em Visão Computacional.
- Duda; Hart; Stork (2001), pelos fundamentos teóricos do reconhecimento de padrões, do classificador k -NN e da relação entre viés e variância.
- Cover; Hart (1967), pela formulação original do algoritmo dos k vizinhos mais próximos.
- Ojala; Pietikäinen; Mäenpää (2002), pela formulação do descritor *Local Binary Patterns* (LBP) e de sua variante uniforme, utilizada neste capítulo.
- Dalal; Triggs (2005), pela formulação do descritor *Histogram of Oriented Gradients* (HOG), empregado na representação de forma e contorno.
- Pedregosa et al. (2011), pela implementação do classificador k -NN, das métricas de avaliação e da validação cruzada na biblioteca `scikit-learn`.
- Quilici-Gonzalez; Zampirolli (2014), pela apresentação didática de classificadores tradicionais de Reconhecimento de Padrões, como Árvores de Decisão, Regras de Classificação e Máquinas de Vetores de Suporte (SVM), complementares ao classificador k -NN explorado neste capítulo.
- Quilici-Gonzalez; Zampirolli; Souza (2026), pela atualização e ampliação desses conteúdos em sua 2ª edição, atualmente em produção.

 Executar  Colab  Abrir pdi-vc  GitHub

7.18 Parte Prática com Exercícios de Programação

Em construção!

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 7 — Classificação de Imagens e Reconhecimento de Padrões — por meio de uma trilha prática aplicada. Diferentemente da manipulação direta de pixels dos capítulos anteriores, os EPs deste capítulo trabalham com as **grandezas intermediárias** de um *pipeline* real de reconhecimento de padrões — vetores de características, distâncias, rótulos previstos e reais, códigos binários locais e histogramas de orientação — permitindo validar manualmente cada etapa do raciocínio sem depender de bibliotecas externas de aprendizado de máquina.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com a implementação manual da regra de decisão do classificador **k-NN** sobre um pequeno espaço de características; avança-se para o cálculo das métricas de **avaliação** (matriz de confusão, precisão e revocação) a partir de rótulos previstos e reais; prossegue-se com a codificação manual do descritor de textura **LBP** a partir de uma vizinhança 3×3 ; aprofunda-se no cálculo do histograma de orientações do descritor **HOG** para uma única célula; avança-se, em seguida, para a integração de **extração de descritores**, **classificação k-NN** e **avaliação multi-classe** em um *pipeline* completo de reconhecimento de texturas; e conclui-se com a aplicação desse mesmo *pipeline* sobre uma **imagem real** (formato PGM), em que o descritor LBP é calculado diretamente sobre os pixels de um mosaico de texturas.

! Diretrizes para a Resolução dos Exercícios de Programação

Em todos os exercícios deste capítulo, as etapas de discretização ou arredondamento numérico devem empregar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*), mitigando ambiguidades em valores com fração exatamente igual a 0,5. Salvo indicação explícita em contrário, distâncias utilizam a métrica euclidiana, empates em votações são resolvidos pela regra descrita em cada exercício, e vetores/matrizes seguem indexação a partir de 0, com a convenção [linha] [coluna] para estruturas bidimensionais.

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para rodar os testes, execute `TestSuite("EP07_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP07_01").run_code(codigo)
```

7.18.1 EP07_01 Classificador k-NN Passo a Passo

O `KNeighborsClassifier` do `scikit-learn`, usado ao longo do capítulo, esconde por trás de uma única chamada (`.fit` / `.predict`) uma regra de decisão bastante simples: para cada nova observação, calcular a distância a todos os exemplos de treinamento, selecionar os k mais próximos e votar pela classe majoritária entre eles.

Antes de confiar na biblioteca, você foi encarregado de implementar essa regra do zero, para um espaço de características bidimensional, exatamente como o simulador interativo de fronteira de decisão do capítulo faz internamente a cada clique do usuário.

7.18.1.1 Diretrizes de Implementação

1. **Quantidade e parâmetro:** Ler o inteiro N (número de exemplos de treinamento) e o inteiro ímpar k (número de vizinhos).
2. **Exemplos de treinamento:** Para cada um dos N exemplos, ler três valores: as coordenadas x e y (reais) e o rótulo r (inteiro, 0 ou 1).

3. **Consultas:** Ler o inteiro Q (número de pontos de consulta) e, em seguida, as coordenadas x_q, y_q (reais) de cada consulta.
4. **Distância:** Para cada consulta, calcular a distância euclidiana até **todos** os exemplos de treinamento:

$$d(x_q, x_i) = \sqrt{(x_q - x_i)^2 + (y_q - y_i)^2}.$$

5. **Seleção dos vizinhos:** Ordenar os exemplos por distância crescente e selecionar os k primeiros. Em caso de **empate de distância** na fronteira do k -ésimo vizinho, desempate pelo exemplo lido **primeiro** na entrada (ordem de leitura estável).
6. **Votação majoritária:** Contar os votos de cada classe entre os k vizinhos selecionados. Se houver **empate na votação** (apenas possível quando k é par, o que não deve ocorrer pela diretriz do item 1, mas trate defensivamente), atribua a classe do vizinho mais próximo entre as classes empatadas.
7. **Saída:** Para cada consulta, na ordem de entrada, imprimir a classe prevista. Ao final, imprimir o total de consultas classificadas como classe 1.

7.18.1.2 Restrições Computacionais

- **Métrica fixa:** utilize exclusivamente a distância euclidiana (não a squared distance) para a ordenação, embora o resultado da comparação seja o mesmo.
- **k sempre ímpar:** a entrada garante k ímpar e $k \leq N$; ainda assim, implemente o desempate do item 6 por robustez.
- **Estabilidade:** ao ordenar por distância, preserve a ordem relativa de exemplos com a mesma distância (ordenação estável).

7.18.1.3 Fundamentação Teórica

Elemento	Papel no k-NN
Espaço de características	Conjunto de todos os vetores (x, y) possíveis
Distância euclidiana	Medida de similaridade entre observações
k pequeno	Fronteira irregular, alta variância
k grande	Fronteira suave, alto viés
Votação majoritária	Regra de decisão $\hat{y} = \text{moda}\{y_i : x_i \in N_k(x)\}$

7.18.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros N e k , separados por espaço.
- Próximas N linhas: três valores por linha — x, y (reais) e r (inteiro $\in \{0, 1\}$), separados por espaço.
- Próxima linha: inteiro Q .
- Próximas Q linhas: dois valores por linha — x_q, y_q (reais), separados por espaço.

Saída:

- Q linhas, cada uma com a classe prevista (0 ou 1) para a respectiva consulta, na ordem de entrada.
- Última linha: **Total classe 1: X.**

7.18.1.5 Exemplos

Entrada	Saída	Observação
4 3	0	Consulta próxima do agrupamento de classe 0.
0 0 0	Total classe 1: 0	
1 0 0		
5 5 1		
6 5 1		
1		
1 1		
4 1	0	Com $k = 1$, cada consulta herda a classe do vizinho mais próximo.
0 0 0	1	
1 0 0	Total classe 1: 1	
5 5 1		
6 5 1		
2		
0.9 0.1		
5.5 5.1		

NotImplemented

Figura 7.18: Simulador: Classificador k-NN Passo a Passo

```
%%writefile EP07_01.py
# Código Python
```

Writing EP07_01.py

```
TestSuite("EP07_01.py").run()
```

7.18.2 EP07_02 ● Avaliação por Matriz de Confusão

Um classificador binário de qualidade de solda foi treinado e testado em uma linha de produção. Para cada peça inspecionada, o sistema registrou o rótulo **real** (obtido por um especialista) e o rótulo **previsto** pelo classificador, em que 1 representa “defeituosa” e 0 representa “conforme”.

A gerência de qualidade quer saber não apenas a acurácia do sistema, mas também sua **precisão** (quando o sistema aponta defeito, com que frequência ele está certo?) e sua **revocação** (de todas as peças realmente defeituosas, quantas o sistema conseguiu identificar?) — a distinção discutida na seção de avaliação de classificadores do capítulo.

7.18.2.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro N (número de peças inspecionadas).
2. **Dados de cada peça:** Para cada uma das N peças, ler dois inteiros — o rótulo real y e o rótulo previsto $\hat{y}$ (ambos $\in \{0, 1\}$).
3. **Matriz de confusão:** Considerando a classe 1 (defeituosa) como **positiva**, contar:
 - VP (Verdadeiro Positivo): $y = 1$ e $\hat{y} = 1$;
 - FP (Falso Positivo): $y = 0$ e $\hat{y} = 1$;
 - FN (Falso Negativo): $y = 1$ e $\hat{y} = 0$;
 - VN (Verdadeiro Negativo): $y = 0$ e $\hat{y} = 0$.
4. **Métricas:** Calcular

$$\text{Acurácia} = \frac{VP + VN}{N}, \quad \text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Revocação} = \frac{VP}{VP + FN}.$$

5. **Casos degenerados:** Se $VP + FP = 0$ (nenhuma predição positiva), imprima **Precisao: indefinida**. Se $VP + FN = 0$ (nenhum caso positivo real), imprima **Revocacao: indefinida**.
6. **Arredondamento:** Todas as métricas numéricas devem ser arredondadas para 4 casas decimais (*round half away from zero*) apenas na exibição.

7.18.2.2 Restrições Computacionais

- **Convenção de classe positiva fixa:** a classe 1 é sempre a classe positiva neste exercício, independentemente de sua frequência relativa.
- **Proteção de divisão por zero:** implemente os casos degenerados do item 5 antes de realizar a divisão.
- **Ordem de saída:** siga exatamente a ordem especificada na seção de saída, mesmo nos casos degenerados.

7.18.2.3 Fundamentação Teórica

Métrica	Pergunta que responde	Sensível a desbalanceamento?
Acurácia	Qual fração das peças foi classificada corretamente?	Sim — pode mascarar erros na classe minoritária
Precisão	Das peças apontadas como defeituosas, quantas realmente são?	Penaliza falsos positivos
Revocação	Das peças realmente defeituosas, quantas foram detectadas?	Penaliza falsos negativos

Em um contexto industrial, uma **revocação** baixa é frequentemente mais grave do que uma **precisão** baixa: deixar passar uma peça defeituosa (falso negativo) tende a ser mais custoso do que inspecionar manualmente uma peça boa apontada por engano (falso positivo).

7.18.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- Próximas N linhas: dois inteiros por linha — y e $\hat{y}$, separados por espaço.

Saída (nesta ordem exata):

```
VP=<int> FP=<int> FN=<int> VN=<int>
Acuracia: <valor ou métrica indefinida>
Precisao: <valor ou indefinida>
Revocacao: <valor ou indefinida>
```

7.18.2.5 Exemplos

Entrada	Saída	Observação
4	VP=1 FP=1 FN=1 VN=1	Um erro de cada tipo.
1 1	Acuracia: 0.5000	
0 1	Precisao: 0.5000	
1 0	Revocacao: 0.5000	
0 0		

Entrada	Saída	Observação
3	VP=0 FP=0 FN=0 VN=3	Nenhum caso positivo real nem previsto.
0 0	Acuracia: 1.0000	
0 0	Precisao: indefinida	
0 0	Revocacao: indefinida	

NotImplemented

Figura 7.19: Simulador: Precisão x Revocação

```
%%writefile EP07_02.py
# Código Python
```

```
Writing EP07_02.py
```

```
TestSuite("EP07_02.py").run()
```

7.18.3 EP07_03 ● Codificação Manual do Descritor LBP

A função `local_binary_pattern` do `scikit-image`, utilizada no projeto de classificação de texturas, calcula automaticamente o código LBP de cada pixel de uma imagem. Antes de utilizá-la como uma caixa-preta, você foi encarregado de implementar manualmente o cálculo do código LBP clássico ($P = 8$, $R = 1$) para o pixel central de uma vizinhança 3×3 , exatamente como definido na equação do capítulo.

Além do código, o sistema de inspeção de texturas também precisa saber se aquele padrão é **uniforme** — um padrão é uniforme quando o número de transições ($0 \rightarrow 1$ ou $1 \rightarrow 0$) ao percorrer os 8 bits **circularmente** (voltando do último bit ao primeiro) é **no máximo 2**, propriedade explorada pela variante *uniforme* do LBP mencionada no capítulo.

7.18.3.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro T (número de vizinhanças a processar).
2. **Dados de cada vizinhança:** Para cada uma das T vizinhanças, ler uma matriz 3×3 de inteiros (intensidades), fornecida em 3 linhas de 3 valores cada. O pixel central é a posição `[1][1]`.
3. **Ordem dos vizinhos:** Percorra os 8 vizinhos em sentido **horário**, iniciando no canto superior esquerdo, na seguinte ordem de posições `[linha][coluna]`: `[0][0]`, `[0][1]`, `[0][2]`, `[1][2]`, `[2][2]`, `[2][1]`, `[2][0]`, `[1][0]`. Esse é o índice $p = 0, 1, \dots, 7$ da equação do LBP.
4. **Função limiar:** Para cada vizinho p com intensidade g_p e centro g_c , calcule $s(g_p - g_c)$, que vale 1 se $g_p \geq g_c$ e 0 caso contrário.
5. **Código LBP:** Calcule

$$\text{LBP} = \sum_{p=0}^7 s(g_p - g_c) 2^p.$$

6. **Transições:** Considerando a sequência circular de bits $s_0, s_1, \dots, s_7$ (na ordem do item 3), conte quantos pares consecutivos **adjacentes na sequência circular** (incluindo o par s_7, s_0) diferem entre si.
7. **Classificação:** Se o número de transições for ≤ 2 , classifique como UNIFORME; caso contrário, NAO_UNIFORME.
8. **Saída:** Para cada vizinhança, na ordem de entrada, imprimir o código LBP (inteiro decimal, 0–255), o número de transições e a classificação.

7.18.3.2 📌 Restrições Computacionais

- **Ordem fixa dos vizinhos:** a ordem do item 3 é obrigatória — invertê-la produz um código numericamente diferente, mesmo representando o mesmo padrão visual.
- **Comparação não estrita:** $s(z) = 1$ quando $z \geq 0$ (o próprio capítulo define a igualdade como incluída no caso 1).
- **Contagem circular:** não esqueça o par que fecha o ciclo (s_7 com s_0); ignorar esse par é um erro comum que classifica incorretamente padrões uniformes.

7.18.3.3 🧠 Fundamentação Teórica

Padrão (bits $s_0 \dots s_7$)	Transições	Interpretação
00000000 ou 11111111	0	Região homogênea (mancha clara ou escura)
00001111	2	Borda simples entre duas regiões
01010101	8	Textura de contraste alternado — não uniforme

Padrões uniformes concentram-se em regiões de textura suave ou bordas simples; padrões não uniformes tendem a corresponder a ruído de alta frequência. Por isso, o histograma LBP *uniforme*, usado no projeto de classificação de texturas, agrupa todos os padrões não uniformes em um único compartimento, reduzindo a dimensionalidade do descritor.

7.18.3.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro T .
- Para cada vizinhança: 3 linhas com 3 inteiros cada (matriz 3×3).

Saída:

- T linhas, no formato `LBP=<int> transicoes=<int> <UNIFORME|NAO_UNIFORME>`.

7.18.3.5 📌 Exemplos

Entrada	Saída	Observação
1 10 10 10 10 50 10 10 10 10	LBP=0 transicoes=0 UNIFORME	Centro é o mais claro; todos os vizinhos geram bit 0.
1 90 90 90 10 50 10 90 90 90	LBP=119 transicoes=4 NAO_UNIFORME	Vizinhos claros e escuros alternados na vizinhança.

NotImplemented

Figura 7.20: Simulador: Código LBP

```
%%writefile EP07_03.py
# Código Python
```

Writing EP07_03.py

```
TestSuite("EP07_03.py").run()
```

7.18.4 EP07_04 ● Histograma de Orientações de uma Célula HOG

A função `hog` do `scikit-image`, empregada no projeto de classificação de dígitos, divide a imagem em pequenas **células** e, para cada uma, constrói um histograma das orientações do gradiente ponderado pela magnitude — exatamente a etapa central descrita na seção sobre o descritor HOG do capítulo.

Você foi encarregado de implementar esse cálculo para uma única célula, a partir dos valores de magnitude e orientação do gradiente **já calculados** para cada pixel da célula (dispensando o cálculo das derivadas parciais).

7.18.4.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros n (a célula tem $n \times n$ pixels) e B (número de compartimentos do histograma).
2. **Magnitudes:** Ler n linhas com n valores reais cada, representando $|\nabla f(x, y)|$ para cada pixel da célula.
3. **Orientações:** Ler mais n linhas com n valores reais cada, representando $\theta(x, y)$ em **graus**, já convertido para o intervalo **não sinalizado** $[0^\circ, 180^\circ)$, como convencionalmente utilizado pelo HOG.
4. **Compartimentos:** Os B compartimentos cobrem $[0^\circ, 180^\circ)$ em faixas iguais de largura $180/B$ graus. Um pixel com orientação θ pertence ao compartimento $\lfloor \theta / (180/B) \rfloor$; se esse índice for igual a B (possível apenas quando θ é exatamente 180° , o que não deve ocorrer pela diretriz do item 3), utilize o compartimento $B - 1$.
5. **Histograma bruto:** Para cada pixel, acumule sua **magnitude** (não sua contagem) no compartimento correspondente:

$$H[b] = \sum_{(x,y) : \text{bin}(\theta(x,y))=b} |\nabla f(x, y)|.$$

6. **Normalização L2:** Após construir H , normalize-o para obter $\hat{H}$:

$$\hat{H}[b] = \frac{H[b]}{\sqrt{\sum_{j=0}^{B-1} H[j]^2 + \epsilon}}, \quad \epsilon = 10^{-6}.$$

7. **Saída:** Imprimir o histograma bruto H (arredondado a 2 casas decimais) em uma linha, seguido do histograma normalizado $\hat{H}$ (arredondado a 4 casas decimais) em outra linha, ambos com os B valores separados por espaço, na ordem dos compartimentos.

7.18.4.2 🚫 Restrições Computacionais

- **Binning não sinalizado:** o intervalo de orientações é $[0, 180)$, não $[0, 360)$ — gradientes em direções opostas (diferença de 180°) contribuem para o **mesmo** compartimento, convenção padrão do HOG para detecção de objetos.
- **Acumulação por magnitude, não por contagem:** o histograma pondera cada pixel por sua magnitude de gradiente, não simplesmente conta quantos pixels caem em cada compartimento.
- **Constante de estabilização:** o $\epsilon = 10^{-6}$ no denominador da normalização evita divisão por zero quando a célula é completamente homogênea (todas as magnitudes nulas).

7.18.4.3 🧠 Fundamentação Teórica

Etapa	Papel
Magnitude do gradiente	Pondera a contribuição de cada pixel — bordas fortes pesam mais que ruído fraco

Etapa	Papel
Orientação não sinalizada	Torna o descritor invariante à polaridade do contraste (claro→escuro vs. escuro→claro)
Histograma por célula	Resume a distribuição local de bordas em um vetor compacto
Normalização L2	Reduz a sensibilidade do descritor a variações globais de iluminação e contraste

A concatenação dos histogramas normalizados de todas as células da imagem — não implementada neste exercício — forma o vetor de características HOG completo, utilizado como entrada do classificador k-NN no projeto do capítulo.

7.18.4.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros n e B .
- Próximas n linhas: n magnitudes reais cada.
- Próximas n linhas: n orientações reais (graus, $[0, 180)$) cada.

Saída:

- Linha 1: os B valores do histograma bruto, arredondados a 2 casas decimais.
- Linha 2: os B valores do histograma normalizado, arredondados a 4 casas decimais.

7.18.4.5 📌 Exemplos

Entrada	Saída	Observação
2 2 1.0 2.0 3.0 4.0 10 100 170 20	5.00 5.00 0.7071 0.7071	Bin de largura 90°: $[0, 90)$ e $[90, 180)$; magnitudes 1 e 4 caem no bin 0, 2 e 3 no bin 1.
2 4 0.0 0.0 0.0 0.0 0 0 0 0	0.00 0.00 0.00 0.00 0.0000 0.0000 0.0000 0.0000	
		Célula homogênea: ϵ evita divisão por zero.

NotImplemented

Figura 7.21: Simulador: Histograma HOG de uma Celula

```
%%writefile EP07_04.py
# Código Python
```

```
Writing EP07_04.py
```

```
TestSuite("EP07_04.py").run()
```


7.18.5 EP07_05 Pipeline Completo: Descritores + k-NN + Avaliação Multi-Classe

Este último exercício integra as três etapas centrais do capítulo em um único *pipeline*, reproduzindo em miniatura o **Projeto Prático 2** (classificação de texturas sintéticas por LBP): um conjunto de histogramas de descritores **já extraídos** (como se fossem histogramas LBP) é utilizado para treinar um classificador k-NN, que por sua vez é avaliado sobre um conjunto de teste independente por meio de uma matriz de confusão multi-classe.

Diferentemente do EP07_01, aqui o espaço de características tem dimensão arbitrária H (o tamanho do histograma), existem mais de duas classes, e a métrica de distância é um parâmetro de entrada — permitindo reproduzir o experimento de comparação de métricas discutido no capítulo.

7.18.5.1 Diretrizes de Implementação

1. **Classes:** Ler o inteiro C (número de classes) seguido de C nomes de classe (*strings* sem espaço), na ordem em que devem aparecer na matriz de confusão.
2. **Configuração:** Ler o inteiro H (dimensão dos histogramas), a *string* M (métrica: **euclidiana** ou **manhattan**) e o inteiro ímpar k .
3. **Treinamento:** Ler o inteiro N e, em seguida, N linhas, cada uma contendo o nome da classe seguido de H valores reais (o histograma de descritor).
4. **Teste:** Ler o inteiro Q e, em seguida, Q linhas, cada uma contendo o nome da classe **real** seguido de H valores reais (o histograma de descritor da amostra de teste).
5. **Distância:** Para cada amostra de teste, calcule a distância a cada exemplo de treinamento usando a métrica M :

$$d_{\text{euclidiana}}(u, v) = \sqrt{\sum_{j=1}^H (u_j - v_j)^2}, \quad d_{\text{manhattan}}(u, v) = \sum_{j=1}^H |u_j - v_j|.$$

6. **Classificação k-NN:** Selecione os k exemplos de treinamento mais próximos (desempate de distância pela ordem de leitura, como no EP07_01) e classifique pela classe majoritária entre eles. Em caso de **empate de votação** entre duas ou mais classes, escolha a que aparece **primeiro** na lista de classes do item 1.
7. **Matriz de confusão:** Construa uma matriz $C \times C$ em que a linha corresponde à classe real e a coluna à classe prevista, seguindo a ordem de classes do item 1.
8. **Acurácia:** Calcule a acurácia global como a razão entre acertos e Q .
9. **Saída:** Para cada amostra de teste, na ordem de entrada, imprimir a classe prevista. Em seguida, imprimir a matriz de confusão (uma linha por classe real, valores separados por espaço, na ordem das classes). Por fim, imprimir a acurácia arredondada a 4 casas decimais.

7.18.5.2 Restrições Computacionais

- **Métrica selecionável:** implemente ambas as distâncias; a métrica M define qual é utilizada em toda a execução (não é possível misturar métricas na mesma chamada).
- **Desempate de votação determinístico:** o critério do item 6 (ordem da lista de classes) deve ser seguido mesmo quando o empate envolve mais de duas classes.
- **Independência de treino e teste:** não há necessidade de validar que as amostras de teste não aparecem no treino — assuma que a entrada é válida.

7.18.5.3 Fundamentação Teórica

Etapas do exercício	Etapas correspondentes no capítulo
Histogramas de treino/teste já extraídos	<code>descriptor_lbp</code> aplicado às texturas sintéticas
Distância euclidiana ou Manhattan	Parâmetro <code>metric</code> do <code>KNeighborsClassifier</code>
Votação majoritária com k vizinhos	<code>KNeighborsClassifier.predict</code>
Matriz de confusão $C \times C$	<code>confusion_matrix</code> do <code>scikit-learn</code>
Acurácia global	<code>accuracy_score</code> do <code>scikit-learn</code>

Etapa do exercício

Etapa correspondente no capítulo

Este exercício evidencia, de forma controlada, um resultado discutido no capítulo: a **escolha da métrica de distância** e do **valor de k** pode alterar a classe prevista para uma mesma amostra, mesmo mantendo fixo o descritor utilizado — reforçando que, no reconhecimento de padrões clássico, o descritor, a métrica e o classificador formam um sistema interdependente, e não peças isoladas.

7.18.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro C seguido de C nomes de classe.
- Linha 2: inteiro H , *string* M e inteiro k .
- Linha 3: inteiro N .
- Próximas N linhas: nome da classe seguido de H reais.
- Próxima linha: inteiro Q .
- Próximas Q linhas: nome da classe real seguido de H reais.

Saída:

- Q linhas com a classe prevista de cada amostra de teste, na ordem de entrada.
- C linhas com a matriz de confusão (uma linha por classe real).
- Última linha: **Acuracia:** <valor>.

7.18.5.5 Exemplos

Entrada (resumida)	Saída	Observação
2 granular listrada	granular	Com $k = 1$, cada teste é classificado pelo vizinho de treino mais próximo.
2 euclidiana 1	listrada	
4	1 0	
granular 0.9 0.1	0 1	
granular 0.8 0.2	Acuracia: 1.0000	
listrada 0.1 0.9		
listrada 0.2 0.8		
2		
granular 0.85 0.15		
listrada 0.15 0.85		

NotImplemented

Figura 7.22: Simulador: *Pipeline* k-NN com escolha de métrica

```
%%writefile EP07_05.py
# Código Python
```

Writing EP07_05.py

```
TestSuite("EP07_05.py").run()
```

7.18.6 EP07_06 ● Classificação Real de um Mosaico de Texturas via LBP + k-NN

Nos exercícios anteriores, o descritor LBP (EP07_03) e o classificador k-NN multi-classe (EP07_05) foram tratados separadamente, sempre a partir de valores numéricos já fornecidos na entrada — vizinhanças 3×3 isoladas ou histogramas já extraídos. Neste exercício de encerramento do capítulo, essa separação é removida de forma intencional: o programa deverá **ler uma imagem real**, no formato **PGM ASCII (P2)**, calcular o descritor LBP diretamente a partir dos pixels e só então classificar cada região por k-NN — reproduzindo, em miniatura, o **Projeto Prático 2** (classificação de texturas sintéticas) combinado com a ideia de **classificação por mosaico de regiões**, que antecipa a segmentação semântica estudada em um capítulo posterior desta parte.

A imagem de entrada é um **mosaico**: uma grade $G \times G$ de blocos quadrados de $S \times S$ pixels, em que cada bloco contém uma amostra de uma das classes de textura sintética do capítulo — **granular**, **listrada**, **manchada** — acrescida de uma quarta classe, **xadrez** (blocos de intensidade alternada em padrão de tabuleiro), análoga à extensão proposta ao final do capítulo. Para manter a entrada acessível ao contexto educacional, o carregamento da imagem será feito, como no restante do capítulo, por meio da função didática `mm.readImg`.

7.18.6.1 📋 Diretrizes de Implementação

1. Leitura das dimensões da imagem

Ler, por meio da entrada padrão, duas linhas contendo, respectivamente, o número de linhas L e o número de colunas C do mosaico (ambos múltiplos do tamanho de bloco S , com $L = C$).

2. Carregamento da imagem

Utilizar a função didática

```
f = mm.readImg(L, C)
```

para ler os $L \times C$ valores de intensidade (tons de cinza, `uint8`) do mosaico.

3. Parâmetros da grade

Ler o inteiro G (número de blocos por lado) e o inteiro S (tamanho do lado de cada bloco, em pixels), satisfazendo $L = C = G \times S$.

4. Cálculo do código LBP por pixel

Para cada pixel **interior** da imagem (ou seja, que não esteja na borda global de `f` — linha ou coluna 0 ou $L - 1 / C - 1$), calcule o código LBP com $P = 8$ vizinhos e raio $R = 1$, percorrendo os vizinhos em sentido **horário** a partir do canto superior esquerdo, exatamente como no EP07_03: `[lin-1][col-1]`, `[lin-1][col]`, `[lin-1][col+1]`, `[lin][col+1]`, `[lin+1][col+1]`, `[lin+1][col]`, `[lin+1][col-1]`, `[lin][col-1]`.

Pixels na borda global da imagem **não** possuem vizinhança completa e devem ser **ignorados** (não contribuem para nenhum histograma). Isso inclui pixels de borda que caem no interior de um bloco (a exclusão é sempre em relação à borda da imagem inteira, não à borda de cada bloco).

5. Histograma LBP uniforme por bloco (10 compartimentos)

Para cada bloco (i, j) da grade ($i, j = 0, \dots, G - 1$), acumule, entre seus pixels válidos (item 4), um histograma $H^{(i,j)}$ de 10 compartimentos:

- Considerando a sequência circular de bits $s_0, \dots, s_7$ do pixel (mesma regra de transições do EP07_03): se o número de transições for ≤ 2 (padrão **uniforme**), o pixel contribui para o compartimento `popcount( $s_0, \dots, s_7$ )  $\in \{0, \dots, 8\}$` (número de bits iguais a 1);
- Caso contrário (padrão **não uniforme**), o pixel contribui para o compartimento 9.

Ao final, normalize o histograma de cada bloco dividindo pelo número de pixels válidos nele contidos, obtendo $\hat{H}^{(i,j)}$, com $\sum_{b=0}^9 \hat{H}^{(i,j)}[b] = 1$.

6. Protótipos de treinamento

Ler o inteiro Ncl (número de classes) seguido de Ncl nomes de classe (ordem que define a matriz de confusão e o desempate de votação, como no EP07_05); em seguida, ler a *string* M (métrica: euclidiana ou manhattan) e o inteiro ímpar k ; por fim, ler o inteiro N (número de protótipos) e, para cada um, o nome da classe seguido de 10 valores reais (histograma protótipo já normalizado).

7. Classificação k-NN de cada bloco

Para cada bloco, calcule a distância de $\hat{H}^{(i,j)}$ a cada um dos N protótipos, usando a métrica M (mesmas fórmulas do EP07_05). Selecione os k protótipos mais próximos (desempate de distância pela ordem de leitura dos protótipos) e classifique pela classe majoritária (desempate de votação pela ordem das classes do item 6).

8. Rótulos reais e avaliação

Ler, em uma única linha, os $G \times G$ nomes de classe **reais** de cada bloco, em ordem de leitura por linha da grade (bloco $(0,0)$, $(0,1)$, ..., $(0,G-1)$, $(1,0)$, ...). Construa a matriz de confusão $Ncl \times Ncl$ (linha = classe real, coluna = classe prevista) e a acurácia global.

9. Saída

Imprimir, para cada bloco (na mesma ordem de leitura dos rótulos reais do item 8), a classe prevista. Em seguida, imprimir a matriz de confusão (uma linha por classe real, na ordem do item 6). Por fim, imprimir a acurácia, arredondada a 4 casas decimais.

7.18.6.2 Restrições Computacionais

- **Descritor fixo:** $P = 8$, $R = 1$ e 10 compartimentos (conforme item 5) são fixos neste exercício — não são lidos da entrada.
- **Exclusão de borda global, não de bloco:** um pixel no limite entre dois blocos, mas no interior da imagem, é válido e contribui normalmente para o histograma do bloco ao qual pertence.
- **Ordem de leitura como critério de desempate:** tanto o desempate de distância (item 7) quanto o de votação (item 7) seguem exatamente as mesmas convenções do EP07_01 e do EP07_05.
- **Protótipos como entrada, não aprendidos:** diferentemente do Projeto Prático 2, os histogramas de treinamento são fornecidos diretamente na entrada; o programa não deve gerar texturas sintéticas.

7.18.6.3 Fundamentação Teórica

Etapa do exercício	Etapa correspondente no capítulo
Leitura da imagem via <code>mm.readImg</code>	Aquisição da imagem no <i>pipeline</i> de reconhecimento de padrões
Código LBP por pixel (EP07_03)	<code>local_binary_pattern(imagem, P=8, R=1, method="uniform")</code>
Histograma de 10 compartimentos por bloco	Função <code>descritor_lbp</code> do Projeto Prático 2 (<code>bins=10, range=(0, P+2)</code>)
Classificação k-NN com métrica selecionável (EP07_05)	<code>KNeighborsClassifier</code> treinado sobre <code>X_textura</code>
Matriz de confusão $Ncl \times Ncl$ e acurácia	<code>confusion_matrix</code> e <code>accuracy_score</code> sobre <code>yt_teste</code>

Este exercício evidencia, com pixels reais em vez de valores sintéticos, uma limitação discutida na seção final do capítulo: classes de textura visualmente distintas para um observador humano — como **granular** e **manchada** — podem produzir histogramas LBP semelhantes quando a vizinhança considerada é pequena ($R = 1$), pois ambas apresentam alta frequência de padrões não uniformes na escala de um único pixel.

Já a classe **xadrez**, por possuir bordas regulares e repetitivas, tende a ser separada com maior facilidade. Espera-se que a matriz de confusão produzida reflita exatamente esse padrão de confusão parcial.

7.18.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

```
L
C
[matriz L x C da imagem]
G S
Ncl nome_classe_1 ... nome_classe_Ncl
M k
N
nome_classe h0 h1 ... h9      (repetida N vezes)
rotulo(0,0) rotulo(0,1) ... rotulo(G-1,G-1)
```

Saída:

- $G \times G$ linhas com a classe prevista de cada bloco, na ordem de leitura da grade.
- Ncl linhas com a matriz de confusão (uma linha por classe real, valores separados por espaço).
- Última linha: Acuracia: <valor>.

7.18.6.5 Exemplo (verificação manual)

Para conferir a implementação do descritor antes de testá-la sobre um mosaico completo, considere uma imagem 6×6 **homogênea**, com todos os pixels de intensidade 100, tratada como um único bloco ($G = 1$, $S = 6$). Como todo pixel interior tem os 8 vizinhos com intensidade igual à do centro ($g_p \geq g_c$ em todos os casos), todos os bits s_p valem 1, o número de transições é 0 (uniforme) e o compartimento é $\text{popcount}(11111111) = 8$. O histograma do único bloco é, portanto, 0 0 0 0 0 0 0 0 1 0.

Entrada (resumida)	Saída	Observação
6	uniforme	Distância do bloco ao protótipo
6	1 0	uniforme é exatamente 0; a classe
[36 valores iguais a 100]	0 0	outra não aparece no rótulo real,
1 6	Acuracia: 1.0000	por isso sua linha na matriz de
2 uniforme outra		confusão é nula.
euclidiana 1		
2		
uniforme 0 0 0 0 0 0 0 0 1 0		
outra 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1		
0.1 0.1		
uniforme		

7.18.6.6 Arquivos de Referência (.pgm)

Para depuração local, dois mosaicos de teste no padrão ASCII P2 são disponibilizados (anexados a esta entrega; ao integrá-los ao repositório do capítulo, salve-os em `all/cap07/dados/EP06/`):

-  **Caso 1 — Mosaico simples (Caso1_Mosaico_Simples.pgm)**: grade 2×2 de blocos de 24×24 pixels, uma amostra de cada uma das quatro classes, com baixo ruído — útil para validar a leitura da imagem e a lógica de classificação em um cenário controlado.
-  **Caso 2 — Mosaico misto (Caso2_Mosaico_Misto.pgm)**: grade 3×3 de blocos de 16×16 pixels, com classes repetidas e maior variabilidade — cenário em que a confusão entre **granular** e **manchada** discutida na Fundamentação Teórica tende a se manifestar.

A Figure 7.23 exibe os dois mosaicos, para inspeção visual antes da implementação.

```
import os
import urllib.request
import numpy as np

def garantir_e_baixar_arquivo(nome_arquivo):
    diretorio_local = "dados/EP06"
    caminho_local = os.path.join(diretorio_local, nome_arquivo)

    # Criar o diretório local se ele não existir
    if not os.path.exists(diretorio_local):
        os.makedirs(diretorio_local)

    # Se o arquivo não existir localmente, baixa do repositório remoto
    if not os.path.exists(caminho_local):
        url_base = "https://raw.githubusercontent.com/fzampirolli/"
        url_base += "pdi-vc/master/all/cap07/dados/EP06"
        url_arquivo = f"{url_base}/{nome_arquivo}"
        print(f"Baixando {nome_arquivo} do GitHub...")
        try:
            urllib.request.urlretrieve(url_arquivo, caminho_local)
        except Exception as e:
            raise IOError(f"Erro ao baixar {nome_arquivo} do GitHub. ",
                          "Verifique a conexão ou a URL. Detalhes: {e}")

    return caminho_local

def ler_pgm_p2(caminho):
    with open(caminho) as f:
        linhas = [l for l in f.read().split() if l]
        assert linhas[0] == "P2"
        C, L = int(linhas[1]), int(linhas[2])
        maxv = int(linhas[3])
        valores = list(map(int, linhas[4:4 + L * C]))
        return np.array(valores, dtype=np.uint8).reshape(L, C)

# Garante o download e obtém o caminho correto
arq_caso1 = garantir_e_baixar_arquivo("Caso1_Mosaico_Simples.pgm")
arq_caso2 = garantir_e_baixar_arquivo("Caso2_Mosaico_Misto.pgm")

# Lê as matrizes PGM
caso1 = ler_pgm_p2(arq_caso1)
caso2 = ler_pgm_p2(arq_caso2)

mm.show(
    [caso1, caso2],
    titles=[
        "Caso 1: Mosaico Simples\n(2x2 blocos, 1 amostra/classe)",
        "Caso 2: Mosaico Misto\n(3x3 blocos, classes repetidas)",
    ],
    cols=2,
    figsize=(8, 4),
)
```

Baixando Caso1_Mosaico_Simples.pgm do GitHub...
 Baixando Caso2_Mosaico_Misto.pgm do GitHub...

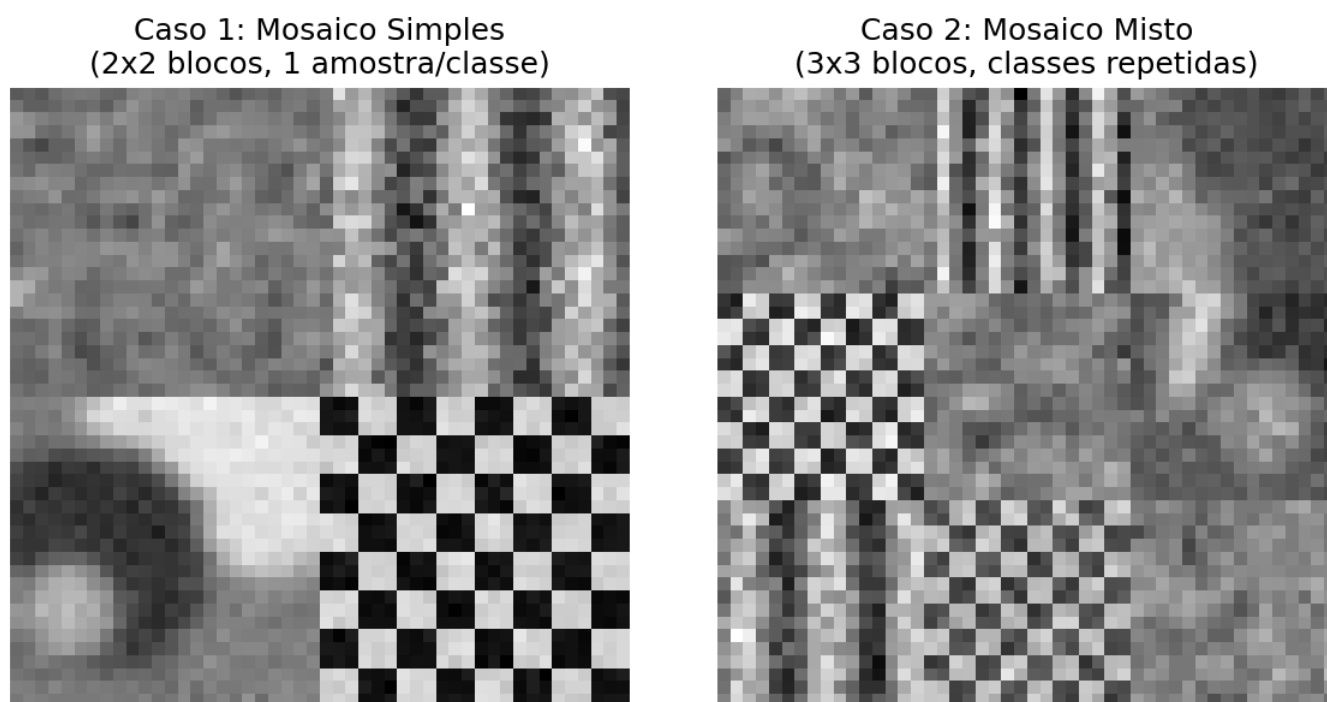


Figura 7.23: Mosaicos de referência (formato PGM ASCII) utilizados no EP07_06. Caso 1: grade 2x2 com uma amostra de cada classe. Caso 2: grade 3x3 com classes repetidas e maior variabilidade.

Simulador: Classificacao de Mosaico via LBP + k-NN ● pipeline completo

Ajuste k e observe como cada bloco do mosaico e classificado com base na distancia (euclidiana) do seu histograma LBP aos protótipos de cada classe.

k (numero de vizinhos) 1

Real: granular Predita: granular ✓	Real: listrada Predita: listrada ✓	Real: manchada Predita: manchada ✓
Real: xadrez Predita: xadrez ✓	Real: granular Predita: manchada ✗	Real: manchada Predita: granular ✗

k=1 | Acertos: 4/6 | Acuracia: 0.6667

Figura 7.24: Simulador: Classificacao de um Mosaico de Texturas via LBP + k-NN


```
%%writefile EP07_06.py  
# Código Python
```

```
TestSuite("EP07_06.py").run()
```

Capítulo 8

Compreendendo Cenas: Correspondência de Características, Detecção e Segmentação



Em construção!

No **Capítulo 7**, cada imagem — ou cada recorte de textura — era tratada como uma unidade isolada, já devidamente enquadrada, a ser rotulada com uma única classe: “este dígito é um 7”, “esta textura é granular”. Essa é uma simplificação poderosa para introduzir os fundamentos do reconhecimento de padrões, mas está longe de como a visão computacional é exigida na prática.

Uma cena real raramente chega pronta e enquadrada. Ela é bagunçada: contém múltiplos objetos, em posições, escalas e orientações desconhecidas, muitas vezes parcialmente sobrepostos, e a tarefa não é apenas “que classe é essa imagem?”, mas perguntas bem mais ricas — “*este objeto que vejo aqui é o mesmo que vi naquela outra foto, só que girado e mais longe?*”, “*onde exatamente, nesta imagem, está cada rosto?*”, “*quais pixels pertencem a cada objeto individual da cena?*”.

Este capítulo caminha por essas três perguntas, na ordem em que historicamente foram resolvidas por técnicas clássicas de visão computacional:

1. **Correspondência de características** — como reconhecer o *mesmo* padrão local em duas imagens diferentes, mesmo sob rotação, escala e mudança de ponto de vista, usando detectores e descritores locais (como o **ORB**) combinados a modelos geométricos robustos (**homografia** via **RANSAC**);
2. **Detecção de objetos** — como localizar, e não apenas classificar, um padrão de interesse dentro de uma imagem maior, partindo do clássico algoritmo de **Haar Cascade** (Viola-Jones) até os conceitos gerais de *bounding box*, *Intersection over Union* (IoU) e supressão de não-máximos (NMS), comuns a praticamente todo detector moderno;
3. **Segmentação visual** — como ir além da caixa delimitadora e atribuir uma classe a **cada pixel** da imagem, distinguindo os três principais paradigmas: segmentação **semântica**, de **instâncias** e **panóptica**.

Ao final, o capítulo apresenta uma visão geral dos principais modelos modernos que resolvem essas tarefas com redes neurais profundas — sem, no entanto, aprofundar seu treinamento, o que será feito no **Capítulo 9**, capítulo final desta parte.

8.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Compreender o papel de **detectores e descritores locais de características** na correspondência entre imagens;
- Implementar um pipeline de correspondência de pontos com **ORB** e estimar uma **homografia robusta** com **RANSAC**;

- Compreender a intuição e as limitações do algoritmo de **Haar Cascade** para detecção de objetos (faces e olhos);
- Definir e calcular **Intersection over Union (IoU)** e aplicar **supressão de não-máximos (NMS)** para refinar detecções redundantes;
- Diferenciar conceitualmente **segmentação semântica, de instâncias e panóptica**, e implementar uma versão clássica (não baseada em aprendizado profundo) de segmentação semântica e de instâncias;
- Reconhecer, em linhas gerais, os principais modelos modernos de detecção e segmentação (YOLO, Faster R-CNN, SSD, U-Net, Mask R-CNN e Segment Anything) e o papel do aprendizado profundo em cada um deles.

8.2 Configuração do Ambiente

```
import importlib
import subprocess
import sys

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from skimage import data as skdata
from skimage.filters import threshold_otsu
from skimage.measure import label
from skimage.morphology import remove_small_objects, opening, disk

import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}
      | OpenCV {cv2.__version__}")
```

8.3 Detectores e Descritores Locais de Características

Retomando a distinção já estabelecida no **Capítulo 6**: um **detector** localiza pontos de interesse (*keypoints*) em uma imagem — regiões com alto contraste local, cantos, ou padrões distintivos, que tendem a ser reencontrados de forma estável mesmo sob pequenas transformações. Um **descritor**, por sua vez, resume a

vizinhança de cada ponto detectado em um vetor numérico, de forma análoga aos descritores de textura e forma do Capítulo 7 — a diferença crucial é que aqui o descritor é calculado em torno de um ponto específico, e não sobre a imagem inteira.

Com um conjunto de pares (ponto, descritor) extraído de duas imagens, o problema de **correspondência** (*matching*) consiste em, para cada ponto da primeira imagem, encontrar o ponto da segunda cujo descritor é mais similar — permitindo responder à pergunta “este padrão local também aparece na outra imagem, e onde?”.

8.3.1 O Descritor ORB

Este capítulo utiliza o **ORB** (*Oriented FAST and Rotated BRIEF*), um descritor binário, rápido e de uso livre (sem restrições de patente), amplamente utilizado em aplicações de tempo real. O ORB combina:

- o detector **FAST** (*Features from Accelerated Segment Test*), que localiza cantos comparando a intensidade de um pixel central a um círculo de pixels vizinhos;
- o descritor **BRIEF** (*Binary Robust Independent Elementary Features*), que codifica a vizinhança de cada ponto como uma sequência de bits, resultante de comparações de intensidade entre pares de pixels amostrados;
- uma componente de **orientação**, que torna o descritor robusto a rotações da imagem.

Por ser um descritor **binário**, a comparação entre dois descritores ORB é feita pela **distância de Hamming** (número de bits diferentes) em vez da distância euclidiana utilizada no k-NN do Capítulo 7 — uma operação muito mais rápida de calcular.

8.4 Projeto Prático 1: Correspondência de Características e Homografia com ORB

Para demonstrar a correspondência de características sem depender de imagens externas, uma “cena” sintética é criada a partir da imagem **astronaut** da base **scikit-image**, aplicando uma rotação, uma escala e uma translação conhecidas — simulando a mesma cena fotografada de um ângulo e distância diferentes.

```
img_original = cv2.cvtColor(skdata.astronaut(), cv2.COLOR_RGB2GRAY)

altura, largura = img_original.shape
M = cv2.getRotationMatrix2D((largura/2, altura/2), 25, 0.8)
M[0, 2] += 40
M[1, 2] += 20
img_cena = cv2.warpAffine(img_original, M, (largura, altura))

mm.show([img_original, img_cena], titles=["Imagem Original", "Cena Sintética (rotação + escala + transla"]
```

Figura 8.1

Em seguida, o detector/descritor ORB é aplicado a ambas as imagens, e os descritores são comparados por força bruta (**BFMatcher**) com distância de Hamming, mantendo apenas correspondências mútuas (**crossCheck=True**).

8.4.1 Estimando a Homografia com RANSAC

Mesmo com a distância de Hamming, algumas correspondências estarão incorretas (falsos positivos de *matching*). A **homografia** — a mesma transformação de perspectiva utilizada no Capítulo 6 para retificar documentos — pode ser estimada de forma robusta a essas correspondências espúrias por meio do algoritmo **RANSAC** (*Random Sample Consensus*), detalhado na próxima seção.

```

orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(img_original, None)
kp2, des2 = orb.detectAndCompute(img_cena, None)
print(f"Pontos de interesse detectados: {len(kp1)} (original), {len(kp2)} (cena)")

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)
print(f"Correspondências encontradas: {len(matches)}")

img_matches = cv2.drawMatches(
    img_original, kp1, img_cena, kp2, matches[:30], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
mm.show([img_matches], titles=["Top 30 Correspondências ORB"], cols=1, figsize=(10, 5))

```

Figura 8.2

```

pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

H, mascara_inliers = cv2.findHomography(pts1, pts2, cv2.RANSAC, ransacReprojThreshold=5.0)
n_inliers = int(mascara_inliers.sum())

print(f"Matriz de homografia estimada:\n{H}\n")
print(f"Inliers: {n_inliers} de {len(matches)} correspondências ({100*n_inliers/len(matches):.1f}%)")

img_inliers = cv2.drawMatches(
    img_original, kp1, img_cena, kp2, matches, None,
    matchesMask=mascara_inliers.ravel().tolist(),
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

mm.show([img_inliers], titles=[f"Correspondências Inliers (RANSAC) - {n_inliers}/{len(matches)}"], cols=

```

Figura 8.3

8.5 Modelagem Matemática: Homografia e RANSAC

8.5.1 Homografia

Uma homografia H é uma matriz 3×3 que mapeia pontos de um plano projetivo a outro:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad \left(\frac{x'}{w'}, \frac{y'}{w'} \right) \text{ é o ponto correspondente.}$$

Como H possui 8 graus de liberdade (a matriz é definida a menos de escala), são necessários, no mínimo, **4 pares de pontos correspondentes** para estimá-la — porém, na prática, o conjunto de correspondências obtido pelo ORB contém múltiplos pares incorretos.

8.5.2 RANSAC

O **RANSAC** estima um modelo robusto a essas correspondências espúrias (*outliers*) por meio de um processo iterativo:

1. Sorteia-se aleatoriamente um subconjunto mínimo de correspondências (4 pares, no caso da homografia);
2. Estima-se o modelo (a homografia) a partir desse subconjunto;
3. Conta-se quantas correspondências do conjunto total são consistentes com esse modelo — os **inliers** — dentro de uma margem de erro (`ransacReprojThreshold`);
4. Repete-se o processo por um número de iterações, mantendo o modelo com o **maior número de inliers**;
5. Ao final, o modelo é **refinado** utilizando apenas os inliers do melhor conjunto encontrado.

Esse procedimento é bastante geral e não se restringe à homografia — o mesmo princípio pode ajustar retas, círculos ou qualquer modelo paramétrico na presença de dados ruidosos, como demonstrado no simulador a seguir.

O simulador a seguir ilustra o funcionamento do RANSAC em um cenário simplificado — o ajuste de uma **reta** a um conjunto de pontos com aproximadamente 25% de *outliers*. Ajuste o limiar de distância e clique em “Rodar RANSAC” para observar como o algoritmo separa inliers de outliers e ajusta o modelo apenas com base nos primeiros.

Por que funciona? — Robustez por consenso

O RANSAC não tenta usar todos os dados de uma vez — ao contrário da regressão por mínimos quadrados tradicional, que é fortemente distorcida por *outliers*, o RANSAC ajusta modelos a partir de pequenas amostras aleatórias e **confia no consenso**: o modelo correto tende a ser consistente com uma grande fração dos dados (os inliers), enquanto um modelo ajustado a uma amostra “azarada” (contendo *outliers*) raramente será consistente com muitos outros pontos.

Parâmetros críticos: o limiar de distância define o quão “rigorosa” é a definição de inlier — um limiar muito pequeno pode descartar inliers legítimos (dados com ruído natural); um limiar muito grande pode aceitar outliers como se fossem inliers, contaminando o modelo final. O número de iterações deve ser grande o suficiente para que, estatisticamente, pelo menos uma amostra aleatória livre de outliers seja sorteada.

Limitações: o RANSAC assume que existe **um único modelo dominante** nos dados; funciona mal se os inliers representam menos da metade do conjunto, ou se há múltiplas estruturas igualmente relevantes (por exemplo, dois planos distintos em uma cena 3D).



Figura 8.4: Simulador interativo do algoritmo RANSAC: ajuste o limiar de distância e execute o algoritmo para observar a separação entre inliers e outliers.

8.6 Detecção de Objetos: Haar Cascade (Viola-Jones)

A correspondência de características responde “onde está o mesmo padrão que já conheço?”. A **detecção de objetos** propõe um problema diferente: “onde está um padrão de uma **categoria genérica** (por exemplo, “rosto humano”), mesmo que eu nunca tenha visto esta pessoa específica antes?”.

O algoritmo de **Haar Cascade**, proposto por Viola e Jones em 2001, foi o primeiro método capaz de realizar detecção de faces em tempo real, e permanece disponível no OpenCV para fins didáticos e aplicações leves. Seu funcionamento combina três ideias:

1. **Características do tipo Haar:** filtros retangulares simples (por exemplo, a diferença de intensidade entre uma região clara e uma região escura adjacente) que capturam padrões grosseiros, mas informativos, como “a região dos olhos costuma ser mais escura que a da testa”;
2. **Imagem integral:** uma estrutura de dados que permite calcular a soma de intensidades de qualquer região retangular em tempo constante, tornando viável avaliar milhares de características Haar em diferentes posições e escalas rapidamente:

$$I_{\text{integral}}(x, y) = \sum_{x' \leq x, y' \leq y} I(x', y');$$

3. **Cascata de classificadores fracos (AdaBoost):** em vez de um único classificador complexo, uma sequência de classificadores simples é aplicada em cascata — janelas que claramente não contêm o objeto são descartadas nos primeiros estágios (baratos), e apenas as candidatas promissoras avançam para estágios mais rigorosos, tornando a busca por toda a imagem, em múltiplas escalas (*sliding window* multi-escala), computacionalmente viável.


```

# Carregar imagem
img_rgb = skdata.astronaut()
img_gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)

# Pré-processamento
img_gray_equalizado = cv2.equalizeHist(img_gray)
img_gray_suave = cv2.GaussianBlur(img_gray_equalizado, (3, 3), 0)

# Carregar classificadores
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")
face_cascade_alt = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_alt.xml")
eye_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_eye.xml")

# Detecção com parâmetros otimizados
faces = face_cascade.detectMultiScale(
    img_gray_suave,
    scaleFactor=1.03,
    minNeighbors=8,
    minSize=(70, 70),
    maxSize=(300, 300)
)

# Validação com segundo classificador
faces2 = face_cascade_alt.detectMultiScale(
    img_gray_suave,
    scaleFactor=1.03,
    minNeighbors=6,
    minSize=(70, 70),
    maxSize=(300, 300)
)

# Combinar detecções (simplificado)
faces_validas = []

for (x, y, w, h) in faces:
    # Filtrar por proporção
    if 0.7 <= w/h <= 1.1:
        # Verificar olhos
        regioao_face = img_gray[y:y+h, x:x+w]
        olhos = eye_cascade.detectMultiScale(
            regioao_face,
            scaleFactor=1.03,
            minNeighbors=5,
            minSize=(15, 15)
        )
        if len(olhos) >= 1:
            # Verificar se está nas detecções do segundo classificador
            for (x2, y2, w2, h2) in faces2:
                if abs(x - x2) < w/2 and abs(y - y2) < h/2:
                    faces_validas.append((x, y, w, h))
                    break

# Visualização
img_annotada = img_rgb.copy()
for (x, y, w, h) in faces_validas:
    cv2.rectangle(img_annotada, (x, y), (x+w, y+h), (0, 255, 0), 3)

    regioao_face = img_gray[y:y+h, x:x+w]
    olhos = eye_cascade.detectMultiScale(regioao_face, scaleFactor=1.03, minNeighbors=5)
    for (ex, ey, ew, eh) in olhos:
        cv2.rectangle(img_annotada, (x+ex, y+ey), (x+ex+ew, y+ey+eh), (255, 0, 0), 2)

```

  Por que funciona? — E por que também falha

Ao executar a célula acima, o classificador de fato localiza corretamente o rosto na imagem — e, dentro dele, os dois olhos. Mas repare: **uma segunda região também foi marcada como “face”**, sobre um trecho do traje espacial. Trata-se de um **falso positivo** genuíno: o padrão de contraste local daquela região do tecido, por coincidência, é suficiente para satisfazer os filtros Haar em todos os estágios da cascata.

Esse resultado, embora indesejado, ilustra exatamente a principal **limitação** do Haar Cascade: por depender de características extremamente genéricas (diferenças de intensidade entre blocos retangulares), o algoritmo é sensível a texturas e padrões que “parecem” o suficiente com um rosto do ponto de vista desses filtros, mesmo sem qualquer semântica real. Além disso, o Haar Cascade tende a funcionar bem apenas para **faces frontais**, sendo degradado por variações de pose, oclusão parcial ou iluminação muito diferente daquela presente nas imagens de treinamento originais.

Quando utilizar: Haar Cascade continua sendo uma escolha razoável quando há restrição severa de poder computacional (ex.: dispositivos embarcados) e a aplicação tolera uma taxa moderada de falsos positivos. Para aplicações mais exigentes, os detectores modernos baseados em redes neurais profundas — apresentados adiante — reduzem drasticamente esse tipo de erro.

8.7 Detecção de Objetos: Caixas Delimitadoras, IoU e NMS

Praticamente todo detector de objetos — do Haar Cascade aos modelos modernos baseados em redes neurais profundas — expressa suas detecções como **caixas delimitadoras** (*bounding boxes*), tipicamente representadas por quatro números: $(x_{min}, y_{min}, x_{max}, y_{max})$.

8.7.1 Intersection over Union (IoU)

Para avaliar o quão bem uma caixa detectada coincide com a localização real de um objeto — ou o quanto duas detecções se sobrepõem — utiliza-se a métrica **IoU**:

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A \cup B)}, \quad \text{IoU} \in [0, 1].$$

Um IoU de 1 indica sobreposição perfeita; um IoU de 0 indica caixas completamente disjuntas. Na prática, é comum considerar uma detecção “correta” quando seu IoU com a localização real (*ground truth*) excede um limiar, tipicamente 0,5.

8.7.2 Supressão de Não-Máximos (NMS)

Detectores do tipo *sliding window* (como o Haar Cascade) tipicamente produzem **múltiplas detecções sobrepostas** para o mesmo objeto — em posições e escalas ligeiramente diferentes, todas com alta confiança. A **supressão de não-máximos** elimina essa redundância:

1. Ordena-se as detecções por confiança (pontuação), da maior para a menor;
2. Seleciona-se a detecção de maior confiança e descarta-se todas as demais cujo IoU com ela exceda um limiar;
3. Repete-se o processo com as detecções restantes, até que nenhuma sobre.

  Por que funciona? — NMS como “faxina” pós-deteção

O NMS não melhora a qualidade individual de cada detecção — ele apenas remove redundância. Sua eficácia depende diretamente do limiar de IoU escolhido: um limiar muito baixo pode eliminar detecções de objetos genuinamente próximos entre si (dois rostos lado a lado, por exemplo); um limiar muito alto pode deixar passar múltiplas caixas para o mesmo objeto. Esse é exatamente o tipo de ajuste fino

```

def calcular_iou(caixa_a, caixa_b):
    xA = max(caixa_a[0], caixa_b[0]); yA = max(caixa_a[1], caixa_b[1])
    xB = min(caixa_a[2], caixa_b[2]); yB = min(caixa_a[3], caixa_b[3])
    intersecao = max(0, xB - xA) * max(0, yB - yA)
    area_a = (caixa_a[2] - caixa_a[0]) * (caixa_a[3] - caixa_a[1])
    area_b = (caixa_b[2] - caixa_b[0]) * (caixa_b[3] - caixa_b[1])
    return intersecao / float(area_a + area_b - intersecao + 1e-9)

def supressao_nao_maximos(caixas, pontuacoes, limiar_iou=0.4):
    ordem = np.argsort(pontuacoes)[::-1]
    mantidas = []
    ordem = list(ordem)
    while len(ordem) > 0:
        atual = ordem.pop(0)
        mantidas.append(atual)
        ordem = [i for i in ordem if calcular_iou(caixas[atual], caixas[i]) < limiar_iou]
    return mantidas

caixas = np.array([
    [50, 50, 150, 150], [60, 55, 155, 145], [58, 60, 160, 150],
    [300, 300, 400, 420], [310, 305, 395, 415],
])
pontuacoes = np.array([0.90, 0.75, 0.60, 0.95, 0.70])

mantidas = supressao_nao_maximos(caixas, pontuacoes, limiar_iou=0.4)
print(f"Caixas antes do NMS: {len(caixas)} | Caixas após o NMS: {len(mantidas)}")
print(f"IoU entre a 1ª e a 2ª caixa: {calcular_iou(caixas[0], caixas[1]):.3f}")

fig, ax = plt.subplots(1, 2, figsize=(9, 4.5))
for i, c in enumerate(caixas):
    ax[0].add_patch(patches.Rectangle((c[0], c[1]), c[2]-c[0], c[3]-c[1], fill=False, edgecolor="#dc2626"))
    ax[0].text(c[0], c[1]-5, f"{pontuacoes[i]:.2f}", color="#dc2626", fontsize=9)
ax[0].set_xlim(0, 450); ax[0].set_ylim(450, 0); ax[0].set_title("Antes do NMS")

for i in mantidas:
    c = caixas[i]
    ax[1].add_patch(patches.Rectangle((c[0], c[1]), c[2]-c[0], c[3]-c[1], fill=False, edgecolor="#16a34a"))
    ax[1].text(c[0], c[1]-5, f"{pontuacoes[i]:.2f}", color="#16a34a", fontsize=9)
ax[1].set_xlim(0, 450); ax[1].set_ylim(450, 0); ax[1].set_title("Depois do NMS")
plt.tight_layout()

```

Figura 8.6

que, no Haar Cascade, ajudaria a reduzir — mas não eliminaria — o falso positivo observado na seção anterior, caso ele fosse detectado repetidamente em escalas próximas.

8.8 Segmentação Visual: Semântica, de Instâncias e Panóptica

Uma caixa delimitadora localiza um objeto de forma aproximada — mas não diz exatamente quais pixels pertencem a ele. A **segmentação visual** resolve essa limitação, atribuindo um rótulo a **cada pixel** da imagem. Três paradigmas são amplamente utilizados, e a distinção entre eles é uma fonte comum de confusão:

Paradigma	Pergunta que responde	Distingue objetos individuais da mesma classe?
Semântica	“A que classe pertence cada pixel?”	Não — todos os pixels de “pessoa” recebem o mesmo rótulo, mesmo que sejam pessoas diferentes
De instâncias	“Quais pixels pertencem a cada objeto individual?”	Sim — mas tipicamente apenas para classes “contáveis” (objetos), ignorando o fundo
Panóptica	Combinação das duas anteriores	Sim — cada pixel recebe uma classe semântica e, quando aplicável, um identificador de instância

A segmentação semântica trata a imagem inteira, incluindo regiões de fundo não contáveis (céu, grama, estrada); a segmentação de instâncias foca em objetos discretos e contáveis, distinguindo cada ocorrência; a segmentação panóptica, formalizada em 2019, unifica os dois paradigmas em uma única representação.

Antes do amplo uso de redes neurais profundas — que serão apresentadas no Capítulo 9 — versões simplificadas de segmentação semântica e de instâncias já eram possíveis com técnicas clássicas, como demonstrado a seguir.

```
img_moedas = skdata.coins()

limiar = threshold_otsu(img_moedas)
mascara_bin = img_moedas > limiar

# Limpeza morfológica: remove ruído e pequenos artefatos da limiarização
mascara_limpa = opening(mascara_bin, disk(3))
mascara_limpa = remove_small_objects(mascara_limpa, max_size=200)

# Segmentação semântica: apenas "moeda" x "fundo"
mascara_semantica = (mascara_limpa.astype(np.uint8)) * 255

# Segmentação de instâncias: cada componente conectado recebe um rótulo distinto
rotulos_instancias = label(mascara_limpa)
print(f"Instâncias (moedas individuais) identificadas: {rotulos_instancias.max()}")

mm.show(
    [img_moedas, mascara_semantica, rotulos_instancias],
    titles=["Original", "Segmentação Semântica\n(moeda x fundo)", "Segmentação de Instâncias\n(cada moeda)"],
    cols=3, figsize=(10, 3.5)
)
```

Figura 8.7

Por que funciona? — E onde a abordagem clássica esbarra

A limiarização de Otsu (Capítulo 4) mais limpeza morfológica funciona bem aqui porque as moedas têm contraste consistente com o fundo e não se tocam de forma significativa — condições favoráveis. A rotulagem de componentes conectados então separa “instâncias” apenas por estarem espacialmente desconectadas na máscara binária.

Essa abordagem, no entanto, **não escala**: ela falha imediatamente diante de objetos da mesma classe que se sobrepõem parcialmente (a rotulagem de componentes os fundiria em uma única instância), de classes com aparência muito variável (não há um único limiar de intensidade capaz de separar “gato” de “não-gato” em fotos naturais), ou de cenas com múltiplas classes semanticamente distintas simultaneamente. É exatamente essa lacuna que os modelos de segmentação baseados em redes neurais profundas — capazes de aprender a noção de “objeto” e “classe” diretamente dos dados, em vez de depender de contraste de intensidade — vêm preencher, como será estudado no Capítulo 9.

A **segmentação panóptica** completa o quadro: ela aplicaria a segmentação semântica ao fundo (regiões não contáveis, se houvesse alguma nesta imagem) e a segmentação de instâncias às moedas, combinando ambos os resultados em um único mapa consistente.

8.9 Visão Geral de Modelos Modernos

As técnicas clássicas apresentadas neste capítulo — Haar Cascade, IoU/NMS e segmentação por limiarização — foram, em grande parte, substituídas ou complementadas por modelos baseados em redes neurais profundas, capazes de aprender diretamente dos dados quais características são relevantes para cada tarefa. A tabela a seguir situa os principais modelos, sem aprofundar seu treinamento — o que será feito no **Capítulo 9**.

Modelo	Tarefa	Ideia Central
YOLO (<i>You Only Look Once</i>)	Detecção de objetos	Trata a detecção como um único problema de regressão: uma rede prevê, em uma única passada, todas as caixas delimitadoras e classes da imagem — priorizando velocidade
Faster R-CNN	Detecção de objetos	Utiliza uma sub-rede (<i>Region Proposal Network</i>) para propor regiões candidatas, refinadas por uma segunda rede — priorizando precisão em detrimento de velocidade
SSD (<i>Single Shot Detector</i>)	Detecção de objetos	Similar ao YOLO em filosofia (detecção em passada única), avaliando caixas em múltiplas escalas de um mapa de características
U-Net	Segmentação semântica	Arquitetura em “U”, com um caminho de codificação (reduz resolução, extrai contexto) e um caminho de decodificação simétrico (recupera resolução espacial), popular especialmente em imagens médicas

Modelo	Tarefa	Ideia Central
Mask R-CNN	Segmentação de instâncias	Estende o Faster R-CNN adicionando um ramo que prediz, para cada caixa detectada, uma máscara de pixels precisa do objeto
Segment Anything (SAM)	Segmentação promptable	Modelo de segmentação de propósito geral, capaz de segmentar qualquer objeto indicado por um ponto, caixa ou texto, sem re-treinamento específico para cada nova classe

Note o padrão histórico: **Faster R-CNN** → **Mask R-CNN** ilustra como um detector de caixas é naturalmente estendido para produzir máscaras de segmentação — a mesma progressão conceitual, de caixa para pixel, que motivou a seção de segmentação deste capítulo.

8.10 Limitações das Abordagens Clássicas e Motivação para o Deep Learning

Os três problemas explorados neste capítulo — correspondência, detecção e segmentação — foram resolvidos, historicamente, por técnicas que dependem de padrões projetados manualmente:

- O **ORB** depende de descritores binários pré-definidos, eficazes para encontrar o *mesmo* padrão sob transformações geométricas simples, mas não para reconhecer objetos de uma *categoria* nunca antes vista;
- O **Haar Cascade** depende de filtros retangulares genéricos, o que o torna sensível a falsos positivos (como observado experimentalmente neste capítulo) e restrito a categorias de objetos com aparência razoavelmente rígida (faces frontais);
- A **segmentação clássica** por limiarização depende de contraste de intensidade favorável, e não generaliza para classes semanticamente complexas ou cenas com múltiplas categorias sobrepostas.

Em todos os três casos, o “gargalo” é o mesmo já identificado ao final do **Capítulo 7**: descritores artesanais carregam suposições fixas sobre o que é relevante em uma imagem, suposições que não se sustentam diante da enorme variabilidade do mundo real — pose, iluminação, oclusão, escala e, sobretudo, a diversidade semântica de milhares de categorias de objetos.

O **Capítulo 9**, capítulo final desta parte, apresenta as **Redes Neurais Convolucionais (CNNs)** — a mudança de paradigma que permite a um sistema **aprender**, diretamente dos dados de treinamento, quais características extrair em cada nível de abstração, eliminando a necessidade de desenhar manualmente filtros como os do ORB ou do Haar Cascade. Os próprios modelos modernos apresentados na tabela anterior — YOLO, Faster R-CNN, U-Net, Mask R-CNN e SAM — são, em sua essência, diferentes arquiteturas construídas sobre esse mesmo princípio.

8.11 Resumo

Neste capítulo, o problema de reconhecimento de padrões do Capítulo 7 foi estendido de imagens isoladas para cenas completas, ao longo de três frentes:

- **Correspondência de características**: detectores e descritores locais (como o **ORB**) permitem encontrar o mesmo padrão em imagens diferentes; a **homografia**, estimada de forma robusta com **RANSAC**, modela a transformação de perspectiva entre as duas vistas, mesmo na presença de correspondências incorretas;

- **Detecção de objetos:** o **Haar Cascade** localiza objetos de uma categoria genérica combinando características retangulares simples, imagem integral e uma cascata de classificadores fracos — com limitações evidentes de falsos positivos e restrição a poses frontais; **IoU** e **NMS** são ferramentas essenciais, comuns a praticamente todo detector, para avaliar e refinar caixas delimitadoras;
- **Segmentação visual:** os paradigmas **semântico**, **de instâncias** e **panóptico** respondem perguntas complementares sobre a atribuição de classes a nível de pixel; uma versão clássica (limiarização + rotulagem de componentes conectados) foi demonstrada, evidenciando suas limitações diante de cenas mais complexas;
- **Modelos modernos:** YOLO, Faster R-CNN, SSD, U-Net, Mask R-CNN e SAM resolvem essas mesmas tarefas aprendendo características diretamente dos dados, superando as limitações das abordagens artesanais apresentadas neste capítulo.

8.12 Uso do NotebookLM como Tutor Complementar

Nesta edição, o uso do **NotebookLM** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 08](#)

 **Aviso sobre Conteúdo Gerado por IA**

Embora seja uma ferramenta valiosa de apoio aos estudos, o NotebookLM pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

8.13 Lista de Exercícios

8.13.1 Exercício 1: Robustez do ORB a Transformações (Básico)

Contexto: O Projeto Prático 1 aplicou uma rotação de 25° , escala de 0,8 e uma pequena translação à imagem original para gerar a “cena sintética”.

Desafio: Repita o experimento variando **apenas o ângulo de rotação** para os valores $\{0^\circ, 45^\circ, 90^\circ, 135^\circ, 180^\circ\}$, mantendo escala e translação fixas. Para cada ângulo, registre o número de correspondências totais e o número de inliers após o RANSAC.

Saída esperada: Uma tabela ou gráfico relacionando o ângulo de rotação ao número de inliers, com uma frase indicando se o ORB manteve-se robusto em todos os ângulos testados.

Dica: Reutilize a função `cv2.getRotationMatrix2D`, alterando apenas o segundo argumento (ângulo); o restante do pipeline (ORB, `BFMatcher`, `findHomography`) permanece igual.

8.13.2 Exercício 2: Ajustando o Haar Cascade (Intermediário)

Contexto: A detecção de faces produziu um falso positivo sobre uma região do traje espacial, além da detecção correta do rosto.

Desafio: Experimente diferentes combinações dos parâmetros `scaleFactor` (ex.: 1.05, 1.1, 1.2, 1.3) e `minNeighbors` (ex.: 3, 5, 8, 12) do método `detectMultiScale`, registrando, para cada combinação, quantas regiões foram detectadas e se o falso positivo persiste.

Saída esperada: Uma tabela com as combinações testadas e o número de detecções obtidas, e uma conclusão sobre qual combinação eliminou o falso positivo sem descartar a detecção correta do rosto — se isso for possível.

Dica: Aumentar `minNeighbors` tende a reduzir falsos positivos, mas também pode eliminar detecções corretas; existe um compromisso entre as duas taxas de erro, similar ao já discutido em avaliação de classificadores no Capítulo 7.

8.13.3 Exercício 3: IoU e NMS em um Cenário Mais Denso (Intermediário/Desafio)

Contexto: O exemplo de NMS deste capítulo utilizou apenas 5 caixas sintéticas, agrupadas em 2 objetos.

Desafio: Gere um conjunto de 20 caixas sintéticas, distribuídas aleatoriamente em 4 a 5 “objetos” (grupos de caixas sobrepostas, com pontuações de confiança aleatórias entre 0,5 e 1,0). Aplique a função `supressao_nao_maximos` com pelo menos três limiares de IoU diferentes (por exemplo, 0,2, 0,4, 0,6) e compare visualmente o resultado.

Saída esperada: Três figuras (uma por limiar de IoU testado), mostrando as caixas mantidas após o NMS em cada caso, e uma breve análise de como o limiar afeta o número final de detecções.

Dica: Para gerar caixas sobrepostas de forma controlada, sorteie um “centro” para cada objeto e, a partir dele, gere de 3 a 5 caixas com pequenas perturbações de posição e tamanho.

8.13.4 Exercício 4: Do Mosaico de Texturas à Segmentação (Desafio)

Contexto: No Exercício 4 do Capítulo 7, um mosaico de texturas foi classificado bloco a bloco com o classificador `knn_textura`, antecipando a ideia de segmentação.

Desafio: Utilizando o mesmo mosaico de texturas (ou gerando um novo), produza um mapa de segmentação **semântica** (um rótulo de classe por bloco, como no Capítulo 7) e, em seguida, aplique a função `label` do `scikit-image` sobre esse mapa para obter uma segmentação de **instâncias** (blocos vizinhos da mesma classe agrupados formam uma única instância; blocos não-adjacentes da mesma classe formam instâncias separadas).

Saída esperada: Três imagens lado a lado: o mosaico original, o mapa de segmentação semântica (uma cor por classe de textura) e o mapa de segmentação de instâncias (uma cor por instância conectada), exibidas com `mm.show`.

Dica: A função `label` do `skimage.measure` aceita diretamente uma matriz de rótulos inteiros (não apenas máscaras binárias) e atribuirá um identificador distinto a cada componente conectado de mesmo valor.

8.14 Próximos Passos

Este capítulo percorreu três problemas centrais de Visão Computacional — correspondência de características, detecção de objetos e segmentação — utilizando exclusivamente técnicas clássicas: descritores binários projetados manualmente (ORB), filtros retangulares e cascatas de classificadores fracos (Haar Cascade), e limiarização de intensidade (segmentação clássica). Em cada caso, experimentos concretos revelaram as limitações dessas abordagens: sensibilidade a falsos positivos, restrição a categorias rígidas de objetos, e incapacidade de generalizar para cenas semanticamente complexas.

O **Capítulo 9**, que encerra esta parte do livro, apresenta a solução que a área encontrou para essas limitações: as **Redes Neurais Convolucionais**. Em vez de projetar manualmente características — como os filtros Haar ou os pares de pixels comparados pelo BRIEF — uma CNN **aprende automaticamente**, a partir de milhares de exemplos, quais padrões visuais, em cada nível de abstração, são relevantes para a tarefa. O capítulo explica os blocos fundamentais de uma CNN (convolução, pooling, treinamento e transferência de aprendizado) e mostra como esses mesmos três problemas — classificação, detecção e segmentação — são resolvidos por modelos pré-treinados, antes de encerrar o livro integrando esses conceitos a aplicações de realidade aumentada, fotogrametria e visão estereoscópica.



8.15 Parte Prática com Exercícios de Programação

Em construção!

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 8 — Correspondência de Características, Detecção de Objetos e Segmentação Clássica — por meio de uma trilha prática aplicada. Assim como no capítulo anterior, os EPs isolam as **grandezas intermediárias** de cada técnica — a distância entre descritores binários, os termos de uma imagem integral, a contagem de *inliers* de um modelo candidato, a sobreposição entre caixas delimitadoras e o rótulo de cada componente conectado — permitindo validar manualmente cada etapa do raciocínio sem depender do OpenCV nem de imagens externas.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com a **distância de Hamming**, coração da correspondência de descritores binários como o ORB; avança-se para a contagem de *inliers* que sustenta o **RANSAC** na estimação robusta de uma homografia; prossegue-se com a **imagem integral**, o truque computacional que torna o Haar Cascade viável em tempo real; aprofunda-se em **IoU e Supressão de Não-Máximos**, o pós-processamento comum a praticamente todo detector de objetos; e conclui-se com a **rotulagem de componentes conectados**, a abordagem clássica — e suas limitações — para segmentar instâncias individuais em uma máscara binária.

! Diretrizes para a Resolução dos Exercícios de Programação

Em todos os exercícios deste capítulo, as etapas de discretização ou arredondamento numérico devem empregar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*), mitigando ambiguidades em valores com fração exatamente igual a 0,5. Salvo indicação explícita em contrário: (i) caixas delimitadoras são especificadas no formato canto-a-canto $(x_{min}, y_{min}, x_{max}, y_{max})$; (ii) matrizes e imagens usam indexação a partir de 0, com a convenção [linha][coluna]; e (iii) comparações de limiar seguem exatamente a convenção descrita em cada exercício — preste atenção especial a se o limiar é excludente ($<$) ou inclusivo ($\leq$), pois isso varia entre os exercícios, tal como no código de referência do próprio capítulo.

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
```

```
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

```
Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2
```

Executando os Testes

Para rodar os testes, execute `TestSuite("EP08_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""
TestSuite("EP08_01").run_code(codigo)
```

8.15.1 EP08_01 Distância de Hamming e Correspondência de Descritores Binários

O ORB, usado no Projeto Prático 1 deste capítulo, descreve a vizinhança de cada ponto de interesse como uma sequência de bits — e, por isso, a comparação entre dois descritores não usa a distância euclidiana do k-NN do Capítulo 7, e sim a **distância de Hamming**: o número de posições em que os bits diferem. Antes de chamar `cv2.BFMatcher(cv2.NORM_HAMMING)`, você foi encarregado de implementar manualmente essa correspondência (*matching*) por força bruta — a mesma etapa que, executada internamente pelo OpenCV, precede a estimação robusta da homografia por RANSAC.

8.15.1.1 Diretrizes de Implementação

1. **Quantidades:** Ler os inteiros N e M — número de descritores extraídos da imagem A e da imagem B, respectivamente.
2. **Descritores de A:** Ler N linhas, cada uma contendo um descritor binário (uma *string* de caracteres 0 e 1, todos do mesmo comprimento).
3. **Descritores de B:** Ler M linhas, no mesmo formato.
4. **Limiar:** Ler o inteiro τ — distância de Hamming máxima aceitável para considerar uma correspondência válida.
5. **Distância de Hamming:** Para dois descritores binários a e b de mesmo comprimento,

$$d_H(a, b) = \sum_k \mathbb{1}[a_k \neq b_k],$$

ou seja, a contagem de posições em que os bits diferem.

6. **Correspondência por vizinho mais próximo:** Para cada descritor a_i de A (i na ordem de leitura, começando em 0), calcule sua distância de Hamming a **todos** os descritores de B e encontre o de menor distância. Em caso de empate entre dois ou mais descritores de B com a mesma distância mínima, escolha o de **menor índice**.
7. **Filtragem pelo limiar:** Se a menor distância encontrada for $\leq \tau$, a correspondência é válida; caso contrário, a_i não possui correspondência.
8. **Saída:** Para cada i de 0 a $N - 1$, na ordem de leitura, imprimir uma linha: `i j d` se houver correspondência válida (onde j é o índice do descritor de B escolhido e d sua distância), ou `i -1` caso contrário. Ao final, imprimir `Total correspondências válidas: X`.

8.15.1.2 Restrições Computacionais

- **Mesmo comprimento:** todos os descritores (de A e de B) têm exatamente o mesmo número de bits.

- **Força bruta:** compare cada descritor de A a **todos** os de B — não é necessário nenhum tipo de indexação ou estrutura de aceleração.
- **Desempate por menor índice em B**, e **nunca** por ordem de leitura de A (que já é natural, pois cada a_i é tratado de forma independente).

8.15.1.3 🧠 Fundamentação Teórica

Elemento	Papel na correspondência ORB
Descritor binário (BRIEF)	Cada bit é o resultado de uma comparação de intensidade entre dois pixels da vizinhança
Distância de Hamming	Métrica de dissimetria entre <i>strings</i> binárias; muito mais rápida de calcular que a distância euclidiana (operação XOR + contagem de bits)
Vizinho mais próximo	Critério de correspondência: cada ponto de A é pareado ao ponto de B com descritor mais similar
Limiar τ	Filtra correspondências pouco confiáveis antes mesmo do RANSAC — mas, como discutido no capítulo, algumas correspondências incorretas ainda passam, exigindo a robustez do RANSAC

8.15.1.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros N e M .
- Próximas N linhas: um descritor binário por linha (*string* de 0s e 1s).
- Próximas M linhas: um descritor binário por linha, no mesmo formato.
- Última linha: Inteiro τ .

Saída:

- N linhas, uma por descritor de A, no formato $i\ j\ d$ ou $i\ -1$.
- Última linha: Total correspondências válidas: X .

8.15.1.5 📌 Exemplos

Entrada	Saída	Observação
3 3	0 0 1	O descritor 11110000 não encontra correspondência: seu vizinho mais próximo está a distância 4, acima do limiar $\tau = 2$.
10101010	1 -1	
11110000	2 1 1	
00001111	Total correspondências válidas: 2	
10101011		
00001110		
11111111		
2		

```
%%writefile EP08_01.py
# Código Python
```

```
Overwriting EP08_01.py
```

```
TestSuite("EP08_01.py").run()
```

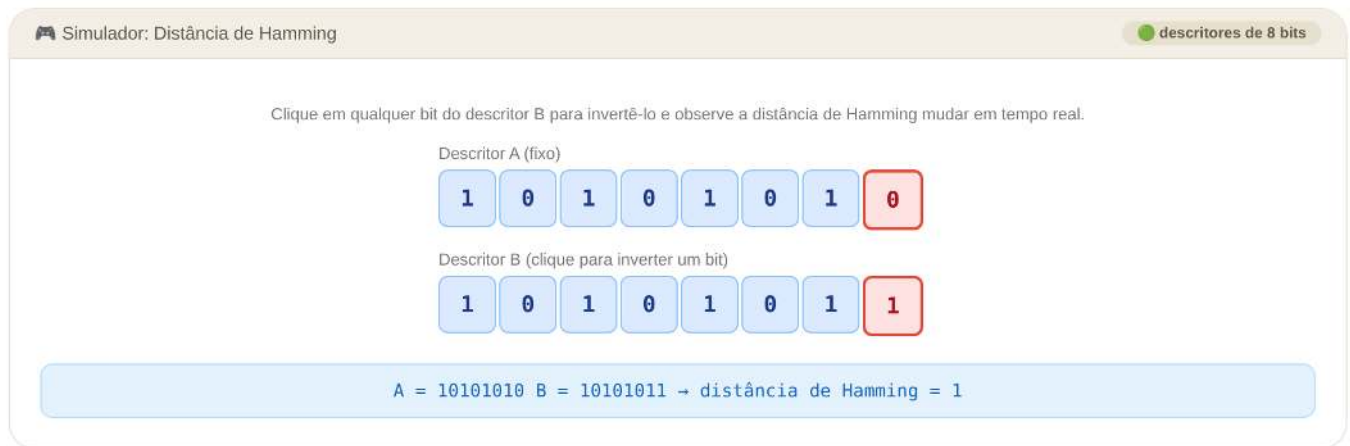


Figura 8.8: Simulador: Distância de Hamming entre Dois Descritores Binários

8.15.2 EP08_02 ● Homografia e RANSAC: A Votação por *Inliers*

O RANSAC, apresentado na seção “Modelagem Matemática: Homografia e RANSAC”, repete um ciclo de três passos — sortear uma amostra mínima, estimar um modelo candidato, e contar quantas correspondências são consistentes com ele (os *inliers*) — mantendo ao final o modelo mais votado. A etapa de estimação do modelo a partir de 4 pontos (passo 2) envolve álgebra linear que foge ao escopo deste EP; aqui, você recebe diretamente um conjunto de homografias **já candidatas** — como se cada uma tivesse sido estimada a partir de uma amostra aleatória diferente — e é encarregado de reproduzir exatamente o passo decisivo do algoritmo: **aplicar cada modelo a todas as correspondências e contar seus *inliers***, escolhendo o vencedor.

8.15.2.1 📋 Diretrizes de Implementação

1. **Correspondências:** Ler o inteiro N e, em seguida, N linhas com quatro reais cada, $x \ y \ x' \ y'$ — um ponto da imagem A e seu correspondente (possivelmente incorreto) na imagem B, exatamente como produzido pela etapa de *matching* do EP08_01.
2. **Modelos candidatos:** Ler o inteiro K (número de homografias candidatas) e o real ε (limiar de erro de reprojeção). Em seguida, ler K linhas, cada uma com nove reais $h_{11} \ h_{12} \ h_{13} \ h_{21} \ h_{22} \ h_{23} \ h_{31} \ h_{32} \ h_{33}$ — os elementos da matriz H candidata, em ordem de leitura por linha (*row-major*).
3. **Reprojeção:** Para cada correspondência (x, y, x', y') e cada modelo candidato H_k , calcular o ponto projetado

$$\begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{w} \end{bmatrix} = H_k \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (\hat{x}/\hat{w}, \hat{y}/\hat{w}) \text{ é o ponto projetado.}$$

4. **Erro de reprojeção:** $e = \sqrt{(\hat{x}/\hat{w} - x')^2 + (\hat{y}/\hat{w} - y')^2}$.
5. **Contagem de *inliers*:** Uma correspondência é um *inlier* do modelo H_k se $e \leq \varepsilon$.

6. **Seleção do melhor modelo:** O modelo vencedor é o de maior número de *inliers*; em caso de empate, escolha o de **menor índice** k (o primeiro encontrado durante o ciclo iterativo do RANSAC).
7. **Saída:** Para cada modelo k de 0 a $K - 1$, na ordem de leitura, imprimir `Modelo k: I inliers`. Ao final, imprimir `Melhor modelo: k_best com I_best inliers`.

8.15.2.2 Restrições Computacionais

- **Comparação inclusiva:** um erro de reprojeção **exatamente igual** a ε conta como *inlier* ($e \leq \varepsilon$).
- **Sem estimação de H :** as matrizes já são fornecidas prontas — não é necessário (nem esperado) resolver nenhum sistema linear.
- **Empate resolvido pelo menor índice**, refletindo o comportamento natural de um algoritmo iterativo que percorre os modelos em ordem e só substitui o melhor encontrado até então quando um novo modelo o **supera estritamente**.

8.15.2.3 Fundamentação Teórica

Elemento	Papel no RANSAC
Amostra mínima (4 pares)	Suficiente para determinar os 8 graus de liberdade de uma homografia
Modelo candidato H_k	Estimado a partir de uma amostra mínima; pode ser bom ou ruim, dependendo se a amostra continha <i>outliers</i>
Erro de reprojeção	Mede o quão bem o modelo “prevê” cada correspondência observada
<i>Inlier</i> vs. <i>outlier</i>	Correspondências consistentes com o modelo vencedor (<i>inliers</i>) vs. as demais, tipicamente correspondências incorretas do <i>matching</i>
Refinamento final	Na prática, após escolher o melhor modelo, o RANSAC o recalcula usando apenas seus <i>inliers</i> — passo não exigido neste EP

8.15.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- Próximas N linhas: quatro reais $x \ y \ x' \ y'$.
- Próxima linha: Inteiro K e real ε .
- Próximas K linhas: nove reais (elementos de H_k , *row-major*).

Saída:

- K linhas no formato `Modelo k: I inliers`.
- Última linha: `Melhor modelo: k_best com I_best inliers`.

8.15.2.5 Exemplos

Entrada	Saída	Observação
5	Modelo 0: 4 inliers	O Modelo 0 (escala $\times 2$) explica corretamente 4 das 5 correspondências; a 5ª, $(5, 5) \rightarrow (1, 1)$, é um <i>outlier</i> que nenhum dos dois modelos explica bem.
0 0 0 0	Modelo 1: 1 inliers	
1 1 2 2	Melhor modelo: 0 com 4 inliers	
2 0 4 0		
0 2 0 4		
5 5 1 1		
2 0.5		
2 0 0 0 2 0 0 0 1		
1 0 0 0 1 0 0 0 1		

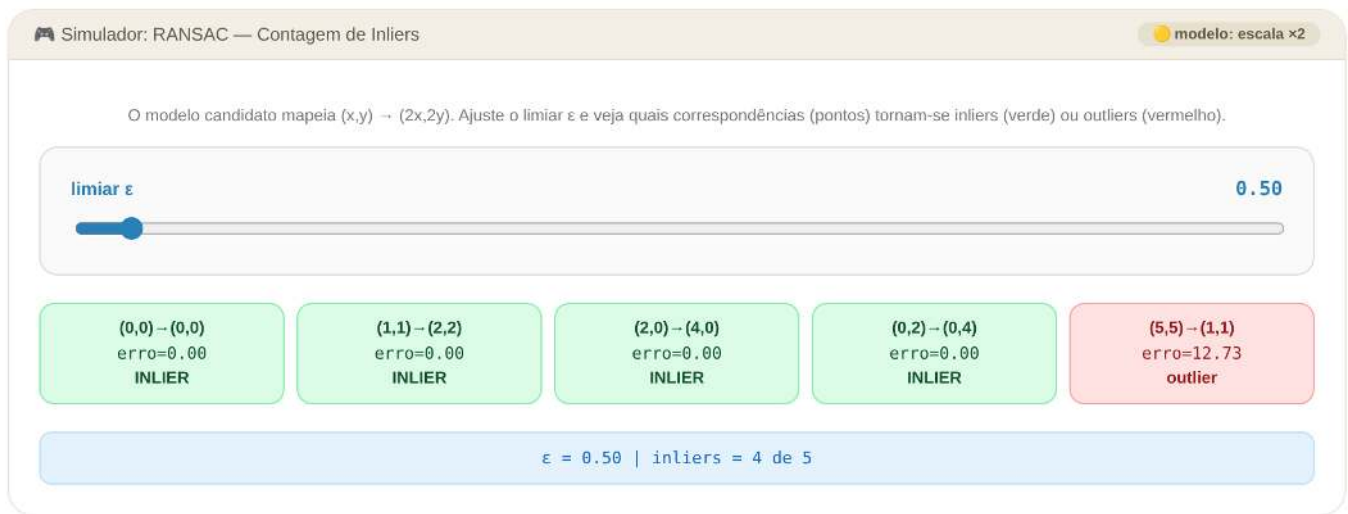


Figura 8.9: Simulador: RANSAC — Votação por Inliers entre Modelos Candidatos

```
%%writefile EP08_02.py
# Código Python
```

```
Overwriting EP08_02.py
```

```
TestSuite("EP08_02.py").run()
```

8.15.3 EP08_03 ● Imagem Integral: Somas Retangulares em Tempo Constante

O Haar Cascade avalia milhares de características retangulares por janela, em múltiplas posições e escalas — algo inviável em tempo real se cada retângulo exigisse somar seus pixels um a um. A **imagem integral**, definida na seção sobre Haar Cascade, resolve esse problema: uma vez pré-computada, a soma de intensi-

dades de **qualquer** região retangular é obtida com apenas quatro consultas e três operações aritméticas, independentemente do tamanho do retângulo.

Você foi encarregado de implementar essa estrutura do zero: primeiro, calcular a imagem integral a partir da imagem original; em seguida, respondê-la para consultas retangulares arbitrárias.

8.15.3.1 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ da imagem e seus $H \times W$ valores inteiros de intensidade.
2. **Imagem integral:** Calcular, para cada posição (i, j) (indexação a partir de 0, [linha] [coluna]),

$$II(i, j) = \sum_{i' \leq i, j' \leq j} I(i', j'),$$

ou seja, a soma de todos os pixels acima e à esquerda de (i, j) , incluindo a própria posição.

3. **Consultas:** Ler o inteiro Q e, em seguida, Q linhas, cada uma com quatro inteiros $x_1 \ y_1 \ x_2 \ y_2$ — os cantos superior-esquerdo e inferior-direito de um retângulo, **ambos inclusivos**, com $0 \leq x_1 \leq x_2 < W$ e $0 \leq y_1 \leq y_2 < H$.
4. **Soma retangular em $O(1)$:** Para cada consulta, calcular a soma de intensidades dentro do retângulo usando exclusivamente valores já presentes em II (sem percorrer os pixels originais):

$$S(x_1, y_1, x_2, y_2) = II(y_2, x_2) - II(y_2, x_1 - 1) - II(y_1 - 1, x_2) + II(y_1 - 1, x_1 - 1),$$

tratando qualquer termo com índice de linha ou coluna igual a -1 como 0.

5. **Saída:** Primeiro, imprimir a imagem integral completa — H linhas com W inteiros cada. Em seguida, para cada consulta, imprimir um único inteiro: a soma da região correspondente.

8.15.3.2 Restrições Computacionais

- **Não recalcule por força bruta:** a resposta a cada consulta deve usar a fórmula de quatro termos sobre II , não uma soma direta dos pixels do retângulo (ainda que o resultado numérico seja o mesmo, o objetivo do exercício é justamente essa técnica).
- **Retângulos com coordenadas inclusivas:** (x_1, y_1) e (x_2, y_2) pertencem à região somada.
- **Tratamento de borda:** ao consultar II com índice -1 (quando $x_1 = 0$ ou $y_1 = 0$), utilize o valor 0.

8.15.3.3 Fundamentação Teórica

Elemento	Papel no Haar Cascade
Imagem integral II	Pré-computada uma única vez por imagem, em tempo $O(HW)$
Consulta em $O(1)$	Cada característica Haar (diferença entre somas de regiões retangulares) é avaliada com poucas operações, independentemente da área do retângulo
Escalabilidade	É essa constância que viabiliza avaliar milhares de características, em múltiplas posições e escalas, em tempo real
Princípio de inclusão-exclusão	Os quatro termos da fórmula somam a região desejada e subtraem exatamente as áreas contadas em excesso

8.15.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W inteiros cada (imagem original).

- Próxima linha: Inteiro Q .
- Próximas Q linhas: quatro inteiros $x_1\ y_1\ x_2\ y_2$.

Saída:

- H linhas com W inteiros cada (a imagem integral).
- Q linhas, uma por consulta, com a soma da região correspondente.

8.15.3.5 Exemplos

Entrada	Saída	Observação
3 3	1 3 6	A consulta cobre a imagem inteira; a soma coincide com $II(2, 2)$ e com a soma de todos os 9 valores.
1 2 3	5 12 21	
4 5 6	12 27 45	
7 8 9	45	
1		
0 0 2 2		A primeira consulta usa os quatro termos da fórmula; a segunda coincide diretamente com $II(1, 1)$, pois começa na origem.
3 3	1 3 6	
1 2 3	5 12 21	
4 5 6	12 27 45	
7 8 9	28	
2	12	
1 1 2 2		
0 0 1 1		



Figura 8.10: Simulador: Imagem Integral e Consulta Retangular em O(1)

```
%%writefile EP08_03.py
# Código Python
```

Overwriting EP08_03.py

```
TestSuite("EP08_03.py").run()
```

8.15.4 EP08_04 IoU e Supressão de Não-Máximos (NMS)

A figura desta seção mostrou o efeito da Supressão de Não-Máximos sobre um conjunto de caixas produzidas por um detector do tipo *sliding window*: múltiplas detecções redundantes por objeto foram reduzidas a uma única caixa por objeto. Você foi encarregado de reimplementar, byte a byte, as duas funções que produziram aquele resultado — `calcular_iou` e `supressao_ao_maximos` — para confirmar, com suas próprias mãos, exatamente os números que o capítulo apresentou.

8.15.4.1 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro N (número de caixas) e o real τ (limiar de IoU). Em seguida, ler N linhas, cada uma com cinco reais x_{min} y_{min} x_{max} y_{max} `score`.
2. **Interseção sobre União:** Para duas caixas A e B ,

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A) + \text{área}(B) - \text{área}(A \cap B)},$$

com área de interseção nula quando as caixas não se sobrepõem.

3. **Algoritmo de NMS** (exatamente como descrito no capítulo):
 - a. Ordene as caixas por `score` decrescente (empates mantêm a ordem de leitura original).
 - b. Selecione a caixa de maior pontuação entre as restantes; adicione-a à saída e remova-a da lista.
 - c. Descarte, da lista restante, **todas** as caixas cujo IoU com a caixa selecionada seja **maior ou igual** a τ — apenas as caixas com $\text{IoU} < \tau$ permanecem candidatas.
 - d. Repita (b)–(c) até que a lista de restantes esteja vazia.
4. **Saída:** Para cada caixa mantida, na ordem em que foi selecionada, imprimir seu índice original (posição de leitura, a partir de 0) e seu `score`, com 2 casas decimais. Ao final, imprimir **Total mantidas: X**.

8.15.4.2 Restrições Computacionais

- **Atenção ao sentido do limiar:** ao contrário do que se poderia supor, uma caixa é **suprimida** quando $\text{IoU} \geq \tau$ (não apenas quando $\text{IoU} > \tau$) — siga exatamente esse critério, o mesmo do código de referência do capítulo.
- **Índices originais:** a saída referencia a posição de leitura de cada caixa na entrada, não sua posição após a ordenação por `score`.
- **Área sem soma de 1 pixel:** use $\text{área} = (x_{max} - x_{min}) \times (y_{max} - y_{min})$, exatamente como no capítulo (sem o ajuste “+1” às vezes usado em outras convenções).

8.15.4.3 Fundamentação Teórica

Elemento	Papel no pós-processamento
IoU	Quantifica a sobreposição espacial entre duas caixas delimitadoras
<i>Sliding window</i> (Haar Cascade)	Produz tipicamente várias detecções sobrepostas para o mesmo objeto, em posições e escalas próximas
Limiar τ	Controla a agressividade da supressão: baixo demais funde objetos próximos; alto demais deixa passar redundâncias

Elemento	Papel no pós-processamento
Ordenação por score	Garante que, entre caixas redundantes, a de maior confiança sempre sobrevive

8.15.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N e real τ .
- Próximas N linhas: cinco reais x_{min} y_{min} x_{max} y_{max} **score**.

Saída:

- Uma linha por caixa mantida, na ordem de seleção: **índice** **score** (score com 2 casas decimais).
- Última linha: **Total mantidas: X**.

8.15.4.5 Exemplos

Entrada	Saída	Observação
5 0.4	3 0.95	Exatamente o exemplo da figura do capítulo: 5 caixas redundantes (2 objetos) tornam-se 2 detecções finais. O IoU entre a 1ª e a 2ª caixas é $\approx 0,775$, bem acima de $\tau = 0,4$.
50 50 150 150 0.90	0 0.90	
60 55 155 145 0.75	Total mantidas: 2	
58 60 160 150 0.60		
300 300 400 420 0.95		
310 305 395 415 0.70		

```
%%writefile EP08_04.py
# Código Python
```

```
Overwriting EP08_04.py
```

```
TestSuite("EP08_04.py").run()
```

8.15.5 EP08_05 Rotulagem de Componentes Conectados: Segmentação Clássica de Instâncias

O exemplo de segmentação clássica deste capítulo separou “instâncias” de moedas simplesmente pela sua desconexão espacial na máscara binária resultante da limiarização de Otsu. Essa etapa final — rotular cada componente conectado com um identificador de instância — é exatamente o que você foi encarregado de implementar aqui, do zero, sobre uma máscara binária já pronta (0 = fundo, 1 = objeto), como se fosse uma reimplementação manual de `cv2.connectedComponents`.

Este exercício também expõe, de forma muito concreta, a limitação discutida no capítulo: o resultado depende inteiramente de como se define “vizinhança” entre pixels — e, como você verá no segundo exemplo, dois pixels em diagonal podem ser considerados a mesma instância ou instâncias diferentes, dependendo exclusivamente da **conectividade** escolhida, não de qualquer noção semântica de objeto.

8.15.5.1 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ da máscara binária e seus $H \times W$ valores (0 ou 1).
2. **Conectividade:** Ler o inteiro $c \in \{4, 8\}$. Na conectividade 4, os vizinhos de (i, j) são $(i-1, j)$, $(i+1, j)$, $(i, j-1)$ e $(i, j+1)$. Na conectividade 8, somam-se as quatro diagonais: $(i-1, j-1)$, $(i-1, j+1)$, $(i+1, j-1)$ e $(i+1, j+1)$.

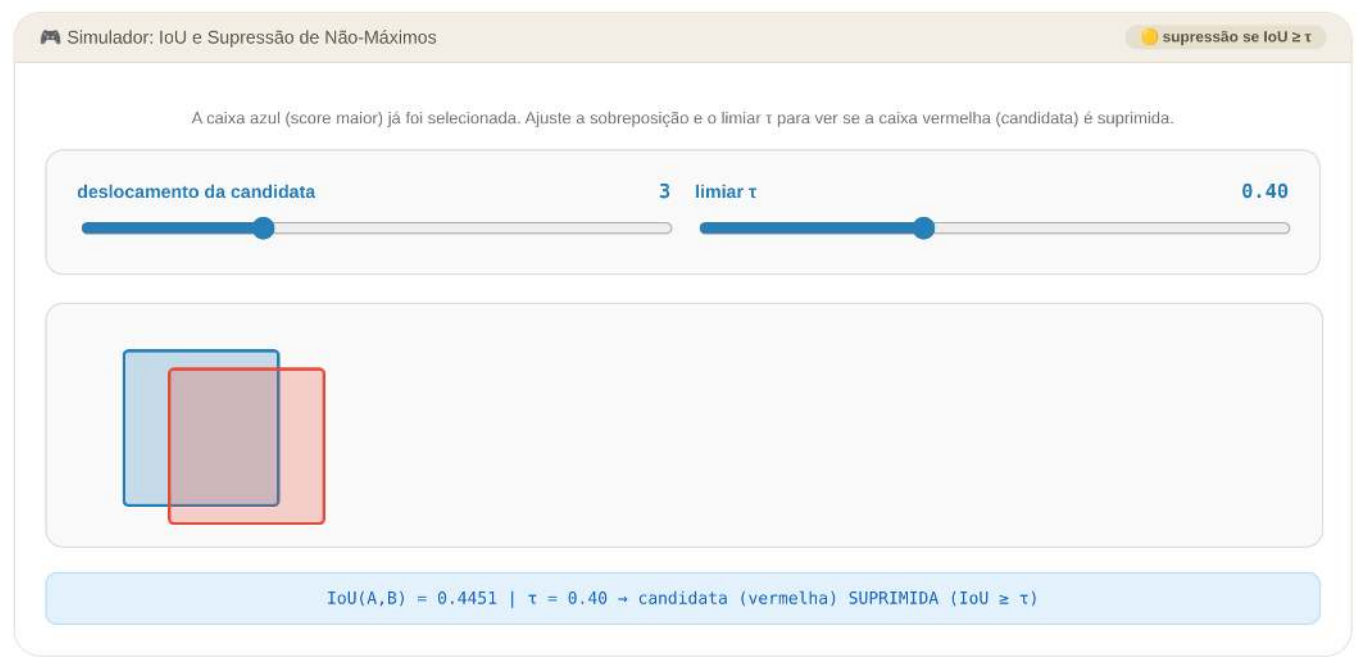


Figura 8.11: Simulador: IoU e Supressão de Não-Máximos

3. **Descoberta de componentes:** Percorrendo a máscara em varredura linha a linha, da esquerda para a direita e de cima para baixo, sempre que um pixel de valor 1 ainda sem rótulo for encontrado, ele inicia um **novo componente**: atribua a ele o próximo rótulo disponível (o primeiro componente descoberto recebe o rótulo 1, o segundo o rótulo 2, e assim por diante) e propague esse mesmo rótulo a todos os pixels de valor 1 alcançáveis a partir dele por uma cadeia de vizinhos (de acordo com a conectividade escolhida) — por busca em largura, profundidade, ou *union-find*, à sua escolha.
4. **Pixels de fundo:** permanecem com rótulo 0 e não pertencem a nenhuma instância.
5. **Saída:** Primeiro, imprimir o mapa de rótulos completo — H linhas com W inteiros cada. Em seguida, para cada rótulo ℓ de 1 a K (na ordem de descoberta), imprimir **Instância 1: A pixels**, onde A é a quantidade de pixels com aquele rótulo. Por fim, imprimir **Total de instâncias: K**.

8.15.5.2 📌 Restrições Computacionais

- **Ordem de descoberta = ordem de varredura:** os rótulos são numerados na ordem em que cada novo componente é encontrado pela varredura linha a linha, não por tamanho ou posição.
- **Conectividade explícita:** dois pixels de valor 1 só pertencem à mesma instância se existir uma cadeia de vizinhos **de acordo com c** ligando um ao outro — não use a conectividade oposta por engano.
- **Máscara binária pura:** todos os valores de entrada são exatamente 0 ou 1.

8.15.5.3 🧠 Fundamentação Teórica

Elemento	Papel na segmentação clássica de instâncias
Limiarização (Otsu, Cap. 4)	Etapas anteriores que produzem a máscara binária a partir da imagem de intensidade

Elemento	Papel na segmentação clássica de instâncias
Componente conectado	Cada instância é definida apenas por conectividade espacial dos pixels de objeto, sem qualquer noção de forma, classe ou aparência
Conectividade 4 vs. 8	Parâmetro que altera o resultado: sob conectividade 8, dois blobs unidos apenas na diagonal tornam-se uma única instância
Limitação central	A técnica funde instâncias que se tocam ou se sobrepõem (mesmo que sejam objetos claramente distintos), pois não há noção de “objeto” — apenas de “região conectada”

8.15.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W inteiros (0 ou 1) cada.
- Última linha: Inteiro c (4 ou 8).

Saída:

- H linhas com W inteiros cada (o mapa de rótulos).
- Uma linha por instância, na ordem de descoberta: **Instância 1: A pixels**.
- Última linha: **Total de instâncias: K**.

8.15.5.5 Exemplos

Entrada	Saída	Observação
6 6 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 8	0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 2 2 0 0 0 0 2 2 Instância 1: 4 pixels Instância 2: 4 pixels Total de instâncias: 2	Dois blocos 2×2 claramente separados: o resultado é o mesmo sob conectividade 4 ou 8.
2 2 1 0 0 1 8	1 0 0 1 Instância 1: 2 pixels Total de instâncias: 1	Sob conectividade 8, os dois pixels em diagonal pertencem à mesma instância. Repita este exemplo com $c = 4$: o resultado passa a ser 2 instâncias de 1 pixel cada — puramente pela mudança de conectividade, sem qualquer diferença na máscara.

```
%%writefile EP08_05.py
# Código Python
```

```
Overwriting EP08_05.py
```

```
TestSuite("EP08_05.py").run()
```

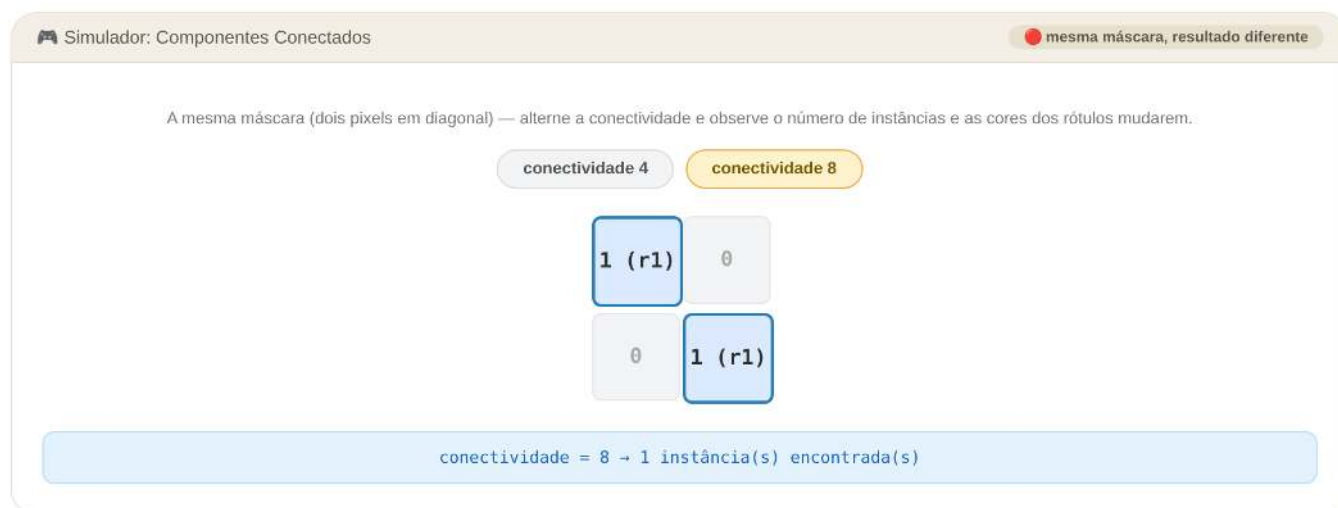


Figura 8.12: Simulador: Rotulagem de Componentes Conectados — Conectividade 4 vs. 8

Capítulo 9

Deep Learning para Visão Computacional



Em construção!

Ao longo desta parte do livro, um padrão se repetiu com insistência. No **Capítulo 7**, descritores de cor, textura (LBP) e forma (HOG) foram projetados manualmente, e suas limitações — especificidade, dependência de hiperparâmetros, incapacidade de capturar semântica de alto nível — foram discutidas abertamente. No **Capítulo 8**, o mesmo padrão se repetiu com o **ORB** (descritor binário fixo) e o **Haar Cascade** (filtros retangulares fixos), cujo falso positivo real, observado experimentalmente, expôs de forma concreta o custo de depender de características genéricas e não-aprendidas.

Este capítulo final da Parte II apresenta a resposta da área a essa limitação recorrente: as **Redes Neurais Convolucionais** (*Convolutional Neural Networks* — CNNs). A mudança de paradigma é conceitualmente simples de enunciar, ainda que profunda em suas consequências: em vez de um especialista humano decidir, a priori, quais filtros aplicar a uma imagem (bordas de Sobel no Capítulo 3, blocos de contraste do LBP, pares de pixels do BRIEF, retângulos do Haar Cascade), **os próprios filtros se tornam parâmetros aprendidos**, ajustados automaticamente a partir de milhares de exemplos, para minimizar o erro na tarefa de interesse.

O capítulo caminha por três etapas: primeiro, os fundamentos de convolução e pooling **aprendidos**, e como eles se conectam à convolução **fixa** já estudada no Capítulo 3; segundo, o treinamento de uma CNN do zero e a técnica de **transferência de aprendizado**, que reaproveita características já aprendidas para acelerar o aprendizado de novas tarefas; por fim, o livro se encerra integrando tudo o que foi estudado — geometria computacional (homografia, correspondência, calibração) e aprendizado profundo — em três aplicações reais de Visão Computacional: **realidade aumentada**, **fotogrametria** e **visão estereoscópica**.

9.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Compreender a convolução e o *pooling* como operações **aprendidas**, relacionando-as à convolução de kernels fixos já estudada no Capítulo 3;
- Descrever a arquitetura típica de uma CNN (blocos de convolução, ativação e *pooling*, seguidos de camadas totalmente conectadas);
- **Implementar e treinar uma CNN do zero** para uma tarefa de classificação de imagens, comparando seu desempenho ao dos classificadores clássicos do Capítulo 7;
- Compreender e aplicar a técnica de **transferência de aprendizado**, identificando quando ela é vantajosa e quais são suas limitações;
- Reconhecer, em linhas gerais, como modelos pré-treinados de grande escala são utilizados em classificação, detecção e segmentação;
- Integrar geometria computacional e aprendizado profundo em aplicações reais: **realidade aumentada**, **fotogrametria** (medição a partir de imagens) e **visão estereoscópica** (estimativa de profundidade).

9.2 Configuração do Ambiente

Além das bibliotecas utilizadas nos capítulos anteriores, este capítulo introduz o **PyTorch**, um dos principais *frameworks* para aprendizado profundo, utilizado na construção e no treinamento de redes neurais convolucionais. O código a seguir verifica a disponibilidade das dependências necessárias e, caso o PyTorch ainda não esteja instalado, instala automaticamente a versão mais adequada ao hardware do computador: com suporte à GPU NVIDIA quando disponível ou, caso contrário, a versão otimizada para execução em CPU.

```
import importlib, subprocess, sys, shutil, os, urllib.request

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

try:
    import torch
except ImportError:
    args = (
        ["torch", "torchvision"]
        if shutil.which("nvidia-smi")
        else ["--index-url", "https://download.pytorch.org/whl/cpu",
            "torch", "torchvision"]
    )
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *args], check=True)
    import torch

import cv2
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from skimage import data as skdata

torch.manual_seed(42)

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

device = "cuda" if torch.cuda.is_available() else "cpu"
gpu = f" ({torch.cuda.get_device_name(0)})" if device == "cuda" else ""

print(
    f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')} | "
```

```
f"PyTorch {torch.__version__} | {device}{gpu}"
)
```

Ambiente pronto. morph 1.1.2 | PyTorch 2.12.1+cu130 | cpu

9.3 Da Convolução Fixa à Convolução Aprendida

O **Capítulo 3** introduziu a convolução espacial com *kernels* fixos, como os operadores de Sobel, definidos manualmente para realçar bordas em uma direção específica. Os Capítulos 7 e 8 estenderam essa mesma lógica de “padrão fixo, definido por um especialista” aos descritores HOG, LBP, ORB e aos filtros retangulares do Haar Cascade. Em todos os casos, um humano decidiu, antecipadamente, **o que procurar** na imagem.

Uma **Rede Neural Convolutacional** aplica a mesma operação matemática de convolução — a soma ponderada de uma vizinhança de pixels por um *kernel* — mas com uma diferença decisiva: os valores do *kernel* deixam de ser fixados manualmente e passam a ser **parâmetros aprendidos**, ajustados automaticamente por um algoritmo de otimização (descida de gradiente) de forma a minimizar o erro do modelo em uma tarefa específica, a partir de exemplos de treinamento rotulados.

9.3.1 Convolução como Camada Aprendida

Formalmente, a saída de uma camada convolutacional na posição (i, j) , para um *kernel* K de tamanho $k \times k$, é:

$$F(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K(u, v) \cdot I(i + u, j + v),$$

exatamente a mesma operação do Capítulo 3 — a diferença está em **como K é obtido**: em vez de ser definido a priori, seus valores começam aleatórios e são ajustados a cada iteração de treinamento. Uma camada convolutacional tipicamente aprende **múltiplos kernels** simultaneamente (múltiplos “filtros”), cada um produzindo um **mapa de características** (*feature map*) diferente — um pode aprender a responder a bordas verticais, outro a texturas específicas, outro a manchas de cor, sem que nenhum humano tenha especificado isso explicitamente.

Duas propriedades tornam essa ideia particularmente eficiente:

- **Compartilhamento de pesos** (*weight sharing*): o mesmo *kernel* é aplicado a todas as posições da imagem, o que reduz drasticamente o número de parâmetros em comparação a uma camada totalmente conectada, e garante **equivariância à translação** — se o padrão se deslocar na imagem de entrada, sua detecção se desloca correspondentemente no mapa de características;
- **Hierarquia de características**: camadas convolutacionais empilhadas aprendem representações progressivamente mais abstratas — as primeiras camadas tendem a responder a bordas e texturas simples (de forma análoga aos filtros de Sobel e LBP), enquanto camadas mais profundas combinam essas respostas em padrões cada vez mais complexos e específicos da tarefa.

9.3.2 Pooling

Entre blocos de convolução, camadas de **pooling** reduzem a resolução espacial dos mapas de características, tipicamente pela operação de **max-pooling**, que retém o maior valor em cada janela local:

$$P(i, j) = \max_{(u, v) \in \text{janela}(i, j)} F(u, v).$$

O *pooling* cumpre dois papéis: reduz o custo computacional das camadas seguintes, e confere certa **invariância a pequenas translações** — um padrão detectado ligeiramente deslocado ainda ativa a mesma região após o agrupamento.

9.3.3 Arquitetura Típica

Uma CNN de classificação organiza esses blocos em uma pilha:

Entrada → [Convolução → Ativação (ReLU) → Pooling] × N → Flatten → Camada(s) Totalmente Conectada(s) → S

As camadas convolucionais iniciais extraem características cada vez mais abstratas da imagem; ao final, essas características são “achatadas” (*flatten*) em um vetor e passadas a camadas totalmente conectadas — a mesma estrutura de classificador com a qual o k-NN do Capítulo 7 operava, mas agora recebendo características **aprendidas**, e não artesanais.

O simulador a seguir tornar concreta a operação de convolução: uma imagem 8×8 (com uma borda vertical e um bloco mais claro, propositalmente construída para tornar visíveis os diferentes efeitos de cada *kernel*) é percorrida por uma janela deslizante 3×3 . Experimente os diferentes *kernels* e avance passo a passo para observar como cada posição do mapa de características é calculada.

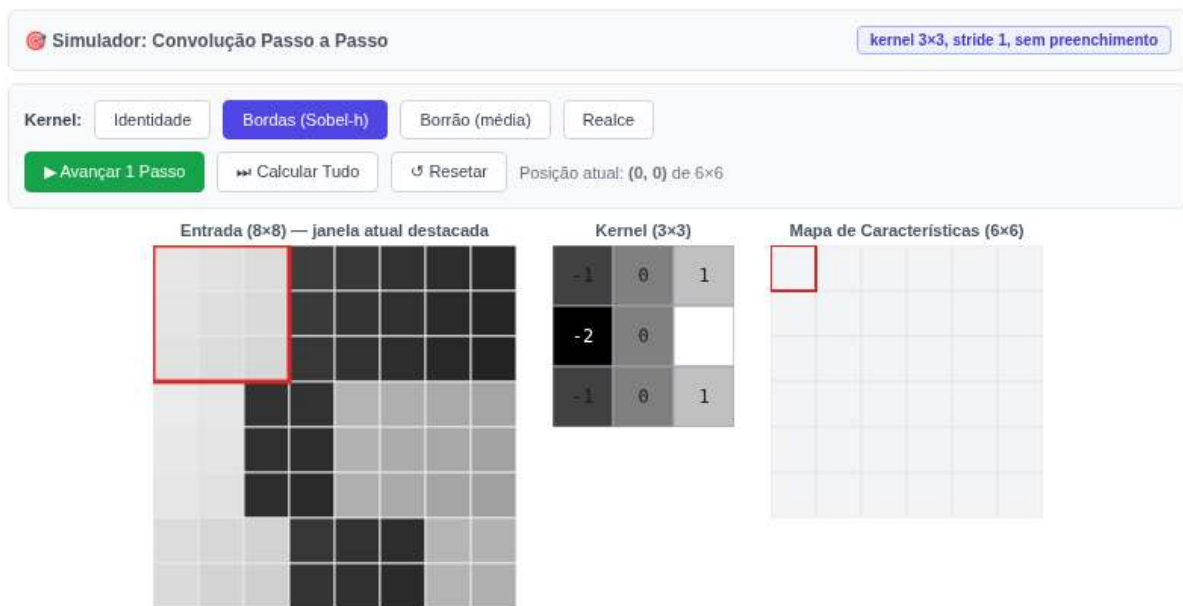


Figura 9.1: Simulador interativo de convolução: escolha um kernel e avance passo a passo (ou calcule tudo de uma vez) para observar a construção do mapa de características.

i 🧠 Por que funciona? — Compartilhamento de pesos e hierarquia

Observe, no simulador, que o **mesmo kernel** de bordas é aplicado a todas as 36 posições da imagem — é exatamente essa reutilização (*compartilhamento de pesos*) que torna as CNNs muito mais econômicas em parâmetros do que seria uma camada totalmente conectada equivalente, e que garante que uma borda seja detectada da mesma forma independentemente de sua posição na imagem.

Também vale notar a diferença entre o kernel de **bordas** e o de **realce**: o primeiro tem resposta próxima de zero em regiões homogêneas e alta magnitude (positiva ou negativa) em transições de intensidade — o mesmo comportamento do operador de Sobel do Capítulo 3. O de **realce**, por sua vez, amplia o contraste local sem eliminar a intensidade de base. Em uma CNN real, esses *kernels* não são escolhidos manualmente como aqui — eles emergem do treinamento, mas frequentemente as primeiras camadas aprendem, de forma independente, filtros notavelmente semelhantes a detectores de borda e realce como estes.

9.4 Projeto Prático 1: Uma CNN do Zero para os Dígitos do Capítulo 7

Para tornar a comparação com o Capítulo 7 direta, esta CNN é treinada sobre a **mesma base load_digits** utilizada naquele capítulo — a diferença está inteiramente na forma como as características são obtidas: antes, pixels brutos ou HOG, calculados manualmente; agora, filtros convolucionais, aprendidos durante o treinamento.

```

digits = load_digits()
X = digits.images.astype(np.float32) / 16.0
y = digits.target

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

X_treino_t = torch.tensor(X_treino).unsqueeze(1)  # (N, 1, 8, 8)
y_treino_t = torch.tensor(y_treino, dtype=torch.long)
X_teste_t = torch.tensor(X_teste).unsqueeze(1)
y_teste_t = torch.tensor(y_teste, dtype=torch.long)

class CNNDigitos(nn.Module):
    # CNN simples: duas camadas convolucionais + pooling, seguidas de camadas densas.
    def __init__(self, n_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)  # 8x8 -> 8x8, 8 filtros
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)  # 4x4 -> 4x4, 16 filtros
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 2 * 2, 32)
        self.fc2 = nn.Linear(32, n_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))  # 8x8 -> 4x4
        x = self.pool(self.relu(self.conv2(x)))  # 4x4 -> 2x2
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        return self.fc2(x)

modelo_cnn = CNNDigitos()
print(f"Parâmetros treináveis: {sum(p.numel() for p in modelo_cnn.parameters())}")

otimizador = optim.Adam(modelo_cnn.parameters(), lr=1e-2)
criterio = nn.CrossEntropyLoss()

n = X_treino_t.size(0)
tam_lote = 32
epocas = 60
historico_perda, historico_acc = [], []

for epoca in range(epocas):
    modelo_cnn.train()
    perm = torch.randperm(n)
    perda_epoca = 0.0
    for i in range(0, n, tam_lote):
        idx = perm[i:i + tam_lote]
        otimizador.zero_grad()
        saida = modelo_cnn(X_treino_t[idx])
        perda = criterio(saida, y_treino_t[idx])
        perda.backward()
        otimizador.step()

```

```

perda_epoca += perda.item() * len(idx)

modelo_cnn.eval()
with torch.no_grad():
    acc_teste = (modelo_cnn(X_teste_t).argmax(dim=1) == y_teste_t).float().mean().item()
    historico_perda.append(perda_epoca / n)
    historico_acc.append(acc_teste)

acc_final_cnn = historico_acc[-1]
print(f"Acurácia final da CNN no conjunto de teste: {acc_final_cnn:.4f}")

fig, ax1 = plt.subplots(figsize=(6, 4))
ax1.plot(historico_perda, color="#dc2626", label="Perda (treino)")
ax1.set_xlabel("Época")
ax1.set_ylabel("Perda", color="#dc2626")
ax2 = ax1.twinx()
ax2.plot(historico_acc, color="#2563eb", label="Acurácia (teste)")
ax2.set_ylabel("Acurácia (teste)", color="#2563eb")
plt.title("Treinamento da CNN - Base de Dígitos")
fig.tight_layout()

```

Parâmetros treináveis: 3658

Acurácia final da CNN no conjunto de teste: 0.9644

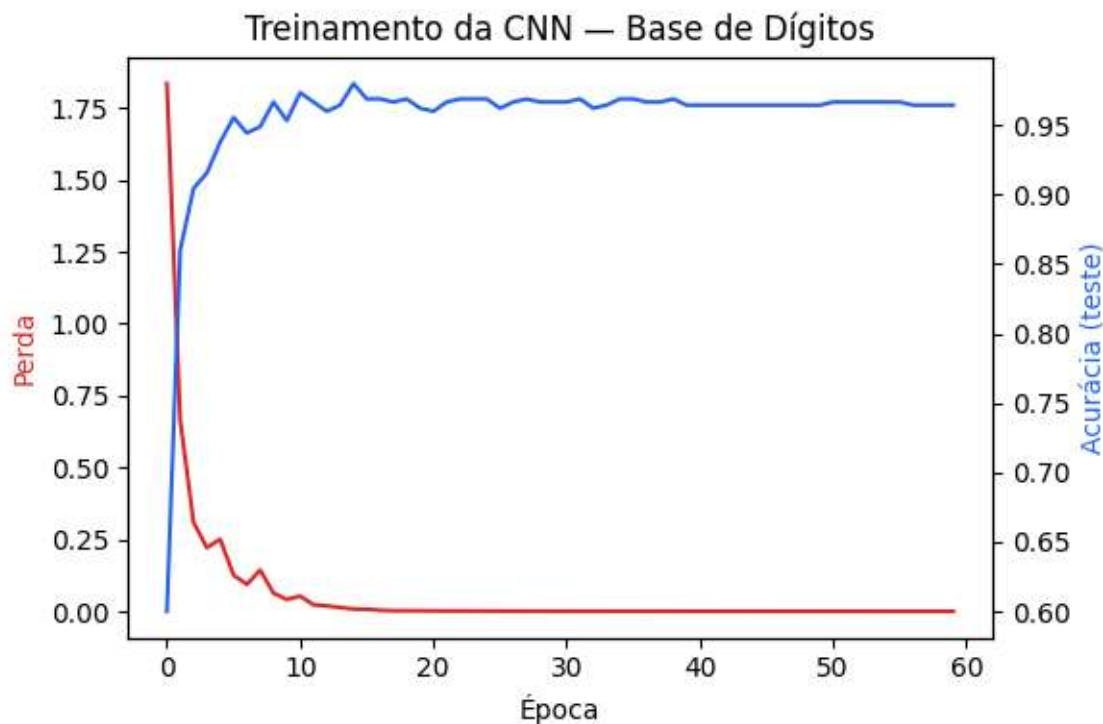


Figura 9.2: Curva de treinamento de uma CNN simples na base de dígitos: perda de treinamento e acurácia no conjunto de teste ao longo das épocas.

9.4.1 Comparando com o Capítulo 7

A tabela a seguir reúne os resultados obtidos na mesma base de dados, com as três abordagens estudadas ao longo do livro:

```

# Valores obtidos no Capítulo 7 (k-NN, k=3), reproduzidos aqui para comparação direta
ACC_KNN_PIXELS_CAP7 = 0.9844
ACC_KNN_HOG_CAP7 = 0.7578

```

```
metodos = ["k-NN\n(pixels brutos)", "k-NN\n(HOG)", "CNN\n(esteste capítulo)"]
acuracias = [ACC_KNN_PIXELS_CAP7, ACC_KNN_HOG_CAP7, acc_final_cnn]

plt.figure(figsize=(5, 4))
cores = ["#6366f1", "#f97316", "#16a34a"]
plt.bar(metodos, acuracias, color=cores)
plt.ylim(0, max(acuracias) + 0.08)
plt.ylabel("Acurácia (conjunto de teste)")
plt.title("Cap. 7 vs. Cap. 9 - Base de Dígitos")
for i, v in enumerate(acuracias):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center")
plt.tight_layout()
```

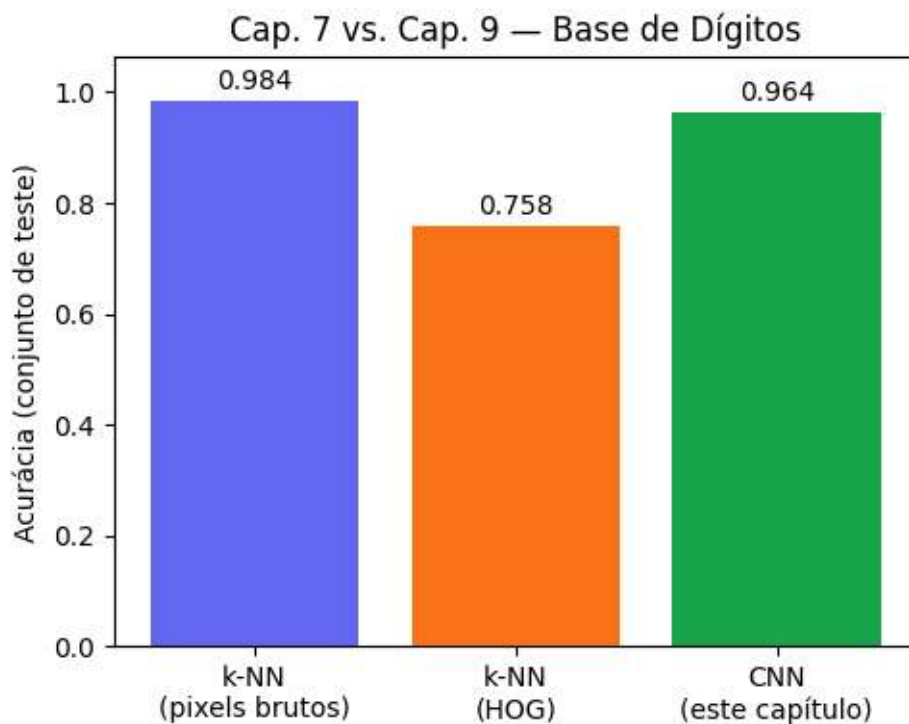


Figura 9.3: Comparação de acurácia entre os classificadores clássicos do Capítulo 7 (pixels brutos e HOG com k-NN) e a CNN treinada neste capítulo, na mesma base de dígitos.

i 🧠 Por que funciona? — E por que a CNN nem sempre “ganha”

O resultado observado aqui **repete o padrão já visto no Capítulo 7**: a CNN, apesar de aprender automaticamente suas características, não supera necessariamente o k-NN com pixels brutos nesta base específica. A explicação é a mesma: `load_digits` é uma base pequena (menos de 1800 exemplos), com imagens já centralizadas, normalizadas e de baixíssima resolução (8×8) — condições em que a comparação direta de intensidades já é altamente informativa, e há poucos dados para que a rede aprenda filtros verdadeiramente superiores aos descritores simples.

O verdadeiro diferencial das CNNs aparece em cenários que descritores artesanais e classificadores simples não conseguem endereçar: imagens maiores e mais realistas, com milhares de categorias, variação substancial de pose, iluminação e fundo, e conjuntos de treinamento massivos — exatamente o regime em que os modelos apresentados na seção “Aplicações em Larga Escala”, mais adiante, foram treinados. A lição pedagógica que atravessa os Capítulos 7, 8 e 9 deste livro é consistente: **a sofisticação de um método deve ser proporcional à complexidade do problema** — usar uma CNN para um problema que um k-NN resolve igualmente bem é desperdício de recursos computacionais, não uma virtude.

9.5 Transferência de Aprendizado

Treinar uma CNN do zero, como no projeto anterior, exige dados suficientes para ajustar todos os seus parâmetros de forma confiável — uma limitação severa quando se dispõe de poucos exemplos rotulados para a tarefa de interesse. A **transferência de aprendizado** (*transfer learning*) contorna essa limitação reaproveitando um extrator de características já treinado em uma tarefa relacionada:

1. Treina-se (ou obtém-se já treinada) uma CNN em uma tarefa de origem, tipicamente com muitos dados disponíveis;
2. As camadas convolucionais iniciais — que tendem a aprender características gerais, como bordas e texturas, pouco específicas da tarefa de origem — são **congeladas** (seus pesos deixam de ser atualizados);
3. Apenas as camadas finais (geralmente a(s) camada(s) totalmente conectada(s)) são substituídas e **treinadas do zero** para a nova tarefa, com muito menos dados e muito menos tempo de treinamento do que seriam necessários para treinar a rede inteira.

Esse princípio é o motivo pelo qual modelos pré-treinados em bases massivas, como a ImageNet (mais de um milhão de imagens em mil categorias), tornaram-se o ponto de partida padrão de praticamente qualquer projeto moderno de visão computacional, mesmo quando a tarefa final é bastante diferente da classificação de objetos genéricos.

```
# Divide as classes de dígitos em dois domínios disjuntos:
# Domínio A (0-4): usado para treinar o extrator de características "pré-treinado"
# Domínio B (5-9): tarefa de destino, com poucos dados disponíveis
classes_A, classes_B = [0, 1, 2, 3, 4], [5, 6, 7, 8, 9]
mask_A, mask_B = np.isin(y, classes_A), np.isin(y, classes_B)
XA, yA = X[mask_A], y[mask_A]
XB, yB = X[mask_B], y[mask_B] - 5 # reindexa para 0..4

XA_tr, XA_te, yA_tr, yA_te = train_test_split(XA, yA, test_size=0.25, random_state=42, stratify=yA)
XB_tr, XB_te, yB_tr, yB_te = train_test_split(XB, yB, test_size=0.25, random_state=42, stratify=yB)

# Simula escassez de dados no domínio de destino: apenas 20 exemplos rotulados
rng = np.random.default_rng(0)
idx_poucos = rng.choice(len(XB_tr), size=20, replace=False)
XB_tr_poucos, yB_tr_poucos = XB_tr[idx_poucos], yB_tr[idx_poucos]

def para_tensor(Ximg, yarr):
    return torch.tensor(Ximg).unsqueeze(1), torch.tensor(yarr, dtype=torch.long)

XA_tr_t, yA_tr_t = para_tensor(XA_tr, yA_tr)
XA_te_t, yA_te_t = para_tensor(XA_te, yA_te)
XB_tr_t, yB_tr_t = para_tensor(XB_tr_poucos, yB_tr_poucos)
XB_te_t, yB_te_t = para_tensor(XB_te, yB_te)

class ExtratorConv(nn.Module):
    # Bloco convolucional reaproveitável entre tarefas.
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        return x.view(x.size(0), -1)
```

```

class CNNCompleta(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.extrator = ExtratorConv()
        self.fc1 = nn.Linear(64, 32)
        self.fc2 = nn.Linear(32, n_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.extrator(x)
        x = self.relu(self.fc1(x))
        return self.fc2(x)

def treinar(modelo, X_t, y_t, epocas, lr, tam_lote=16, parametros=None):
    params = parametros if parametros is not None else modelo.parameters()
    otim = optim.Adam(params, lr=lr)
    crit = nn.CrossEntropyLoss()
    n_amstras = X_t.size(0)
    for _ in range(epocas):
        perm = torch.randperm(n_amstras)
        for i in range(0, n_amstras, tam_lote):
            idx = perm[i:i + tam_lote]
            otim.zero_grad()
            perda = crit(modelo(X_t[idx]), y_t[idx])
            perda.backward()
            otim.step()

def calcular_acuracia(modelo, X_t, y_t):
    modelo.eval()
    with torch.no_grad():
        pred = modelo(X_t).argmax(dim=1)
    return (pred == y_t).float().mean().item()

# 1) Treina o modelo "de origem" no domínio A
modelo_origem = CNNCompleta(n_classes=5)
treinar(modelo_origem, XA_tr_t, yA_tr_t, epocas=40, lr=1e-2)
print(f"Acurácia no domínio de origem A (dígitos 0-4): {calcular_acuracia(modelo_origem, XA_te_t, yA_te_t)}")

# 2) Transferência: reaproveita o extrator (congelado), treina apenas a nova cabeça
modelo_transferencia = CNNCompleta(n_classes=5)
modelo_transferencia.extrator.load_state_dict(modelo_origem.extrator.state_dict())
for p in modelo_transferencia.extrator.parameters():
    p.requires_grad = False

treinar(
    modelo_transferencia, XB_tr_t, yB_tr_t, epocas=40, lr=1e-2,
    parametros=list(modelo_transferencia.fc1.parameters()) + list(modelo_transferencia.fc2.parameters())
)
acc_transferencia = calcular_acuracia(modelo_transferencia, XB_te_t, yB_te_t)

# 3) Do zero: mesmo pouco dado, mesmas épocas, mas sem reaproveitar nada
modelo_do_zero = CNNCompleta(n_classes=5)
treinar(modelo_do_zero, XB_tr_t, yB_tr_t, epocas=40, lr=1e-2)
acc_do_zero = calcular_acuracia(modelo_do_zero, XB_te_t, yB_te_t)

print(f"Domínio de destino B (dígitos 5-9), apenas {len(XB_tr_poucos)} exemplos de treino:")
print(f"  Com transferência (extrator congelado): {acc_transferencia:.4f}")
print(f"  Treinando do zero (mesmos dados/épocas): {acc_do_zero:.4f}")

plt.figure(figsize=(4.5, 4))

```

```
plt.bar(["Do zero", "Transferência"], [acc_do_zero, acc_transferencia], color=["#dc2626", "#16a34a"])
plt.ylim(0, max([acc_do_zero, acc_transferencia]) + 0.08)
plt.ylabel("Acurácia no domínio B (teste)")
plt.title(f"Efeito da Transferência ({len(XB_tr_poucos)} exemplos de treino)")
for i, v in enumerate([acc_do_zero, acc_transferencia]):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center")
plt.tight_layout()
```

Acurácia no domínio de origem A (dígitos 0-4): 1.0000
 Domínio de destino B (dígitos 5-9), apenas 20 exemplos de treino:
 Com transferência (extrator congelado): 0.7277
 Treinando do zero (mesmos dados/épocas): 0.6786

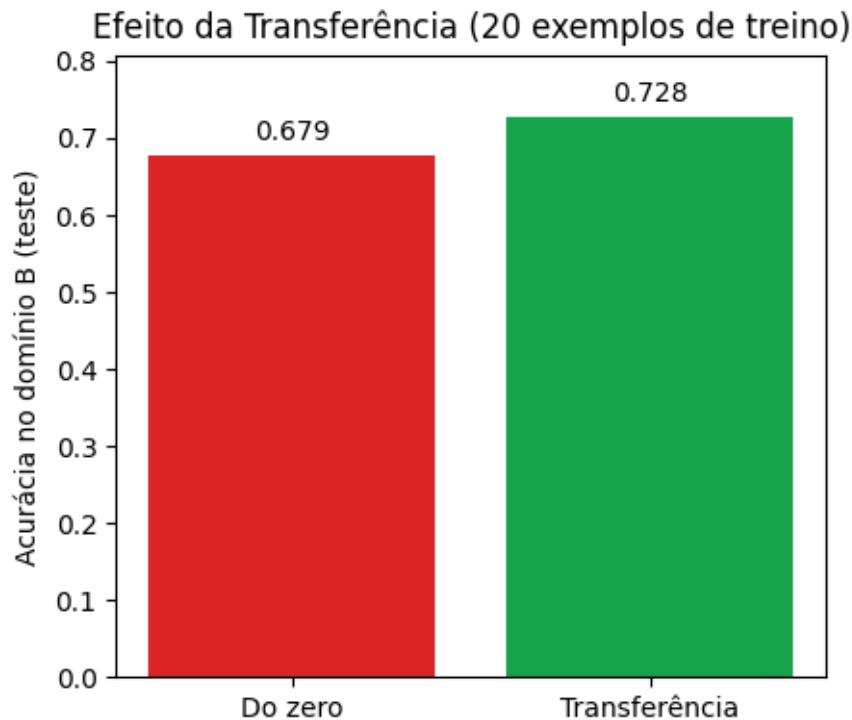


Figura 9.4: Comparação de acurácia entre treinar do zero e reaproveitar (transferir) um extrator de características já treinado, ambos com o mesmo pequeno número de exemplos rotulados na tarefa de destino.

i 🧠 Por que funciona? — E quando não funciona

Com apenas 20 exemplos rotulados no domínio de destino, o modelo treinado do zero mal consegue superar um classificador aleatório — não há dados suficientes para ajustar todos os parâmetros da rede de forma confiável. O modelo com transferência de aprendizado, por outro lado, já parte de um extrator de características capaz de identificar bordas, curvas e traços característicos de dígitos manuscritos em geral (aprendidos a partir dos dígitos 0-4) — restando à nova cabeça apenas a tarefa, muito mais simples, de aprender a combinar essas características já úteis para distinguir os dígitos 5-9.

Quando a transferência ajuda: quando a tarefa de origem e a de destino compartilham estatísticas visuais de baixo e médio nível — o que é razoável esperar entre dígitos manuscritos diferentes, como neste experimento, ou, mais amplamente, entre a maioria das imagens naturais e uma tarefa de nicho.

Quando a transferência falha ou até prejudica (fenômeno conhecido como *transferência negativa*): quando o domínio de origem é excessivamente diferente do domínio de destino — por exemplo, transferir um extrator treinado em fotografias naturais para radiografias médicas ou imagens de satélite pode não trazer benefício algum, e em alguns casos até atrapalhar, se as características de baixo nível relevantes forem muito distintas entre os dois domínios. Nesses casos, um ajuste fino (*fine-tuning*) parcial das

camadas convolucionais — em vez de congelá-las por completo — costuma produzir melhores resultados.

9.6 Aplicações em Larga Escala com Modelos Pré-treinados

Os experimentos anteriores utilizaram CNNs pequenas, treinadas em minutos em uma base de poucas centenas de exemplos. Na prática profissional, é extremamente comum utilizar diretamente modelos **já treinados** em bases massivas — como a ImageNet, com mais de um milhão de imagens em mil categorias — como ponto de partida (via transferência de aprendizado) ou mesmo diretamente, sem qualquer re-treinamento.

! Important

As células desta seção **não são executadas automaticamente** neste notebook: o carregamento desses modelos exige o download de pesos pré-treinados (centenas de megabytes) diretamente dos servidores do PyTorch, o que requer conexão à internet no ambiente de execução. O código é apresentado tal como seria executado em um ambiente com conectividade, para fins de referência.

9.6.1 Classificação de Imagens

O padrão de uso é sempre o mesmo: carregar a arquitetura com pesos pré-treinados e, para uma nova tarefa, substituir apenas a camada final, como discutido na seção de transferência de aprendizado.

```
import torchvision.models as models

# Carrega uma ResNet-18 pré-treinada na ImageNet (requer internet)
modelo_resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Transferência de aprendizado: congela o extrator de características
for parametro in modelo_resnet.parameters():
    parametro.requires_grad = False

# Substitui a camada final (originalmente 1000 classes da ImageNet)
# por uma nova camada adequada à tarefa de destino (ex.: 3 classes)
n_classes_destino = 3
modelo_resnet.fc = nn.Linear(modelo_resnet.fc.in_features, n_classes_destino)

# A partir daqui, apenas `modelo_resnet.fc` seria treinada com os dados da nova tarefa
```

9.6.2 Detecção de Objetos

Modelos como o **Faster R-CNN**, apresentado no Capítulo 8, estão disponíveis prontos para uso, já treinados para detectar até 91 categorias de objetos comuns (base COCO):

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.transforms.functional import to_tensor

from PIL import Image
from urllib.request import urlopen

img = Image.open(urlopen(
    "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"
)).convert("RGB")
modelo = fasterrcnn_resnet50_fpn(weights="DEFAULT").eval()
```

```

with torch.no_grad():
    pred = modelo([to_tensor(img)])[0]

fig, ax = plt.subplots(figsize=(6, 4))
ax.imshow(img)

for box, score in zip(pred["boxes"], pred["scores"]):
    if score < 0.8:
        continue
    x1, y1, x2, y2 = box.numpy()
    ax.add_patch(patches.Rectangle((x1, y1), x2-x1, y2-y1,
                                   fill=False, edgecolor="red", linewidth=2))

ax.set_axis_off()
plt.show()

```



9.6.3 Segmentação Semântica

De forma análoga, modelos de segmentação semântica pré-treinados retornam diretamente um mapa de classes por pixel:

```

import torch
import matplotlib.pyplot as plt
from PIL import Image
from urllib.request import urlopen
from torchvision.transforms.functional import to_tensor
from torchvision.models.segmentation import deeplabv3_resnet50

img = Image.open(urlopen(
    "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"
)).convert("RGB")

modelo = deeplabv3_resnet50(weights="DEFAULT").eval()

with torch.no_grad():
    mapa = modelo(to_tensor(img).unsqueeze(0))["out"].argmax(1).squeeze()

plt.imshow(mapa)
plt.axis("off")

```

```
plt.show()
```



9.7 Projeto Prático 2: Detecção de Objetos com YOLO e Transferência de Aprendizado

A subseção anterior mostrou, por meio do **Faster R-CNN**, um modelo de detecção pré-treinado sendo usado “como está”, sem qualquer re-treinamento. O Faster R-CNN pertence à família de detectores **de dois estágios**: uma primeira rede propõe regiões candidatas a conter um objeto (*region proposals*) e uma segunda rede classifica e refina cada região — a mesma lógica de “propor, depois verificar” já discutida no Capítulo 8 a propósito do Haar Cascade em cascata.

A família **YOLO** (*You Only Look Once*) segue uma filosofia distinta, de **um único estágio**: a imagem é dividida conceitualmente em uma grade, e uma única rede convolucional prevê, em uma única passagem direta, as caixas delimitadoras, a confiança de haver um objeto e a classe de cada região — sem uma etapa separada de proposta de regiões. Essa simplificação é o que torna a família YOLO especialmente adequada à detecção **em tempo real**, citada na seção de encerramento deste capítulo como uma das direções naturais de aprofundamento.

Nesta seção, o princípio de **transferência de aprendizado** — já demonstrado para classificação, com uma CNN pequena treinada nos dígitos do Capítulo 7 — é aplicado a uma **tarefa de detecção**: partindo de um YOLO pré-treinado na base COCO (fotografias naturais, 80 categorias), o modelo é ajustado (*fine-tuned*) para uma tarefa de detecção bastante diferente — localizar e classificar **objetos geométricos sintéticos** (triângulos, quadrados, estrelas, etc.), variando em cor, tamanho e rotação.

9.7.1 Gerando um Conjunto de Dados Sintético de Objetos Geométricos

O cenário reproduzido aqui é inspirado em uma avaliação prática comum em disciplinas de Visão Computacional: dado um conjunto de imagens contendo objetos geométricos gerados aleatoriamente, treinar um detector capaz de localizar e classificar cada objeto em nove classes:

```
classes = ['Triangle', 'Square', 'Pentagon', 'Hexagon',  
           'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross']
```

Cada objeto varia aleatoriamente em **cor**, **tamanho** e **rotação**, e cada imagem pode conter um ou mais objetos. Para cada imagem, um arquivo TXT associado descreve, em uma linha por objeto, a classe e a caixa delimitadora no **formato YOLO**: a classe seguida de quatro valores normalizados entre 0 e 1 — centro em x , centro em y , largura e altura, todos expressos como fração da largura ou da altura da imagem, por exemplo:

```
0 0.2895 0.2007 0.3750 0.2862
```

Essa é exatamente a convenção de anotação esperada pelas bibliotecas de treinamento de YOLO (como a **ultralytics**, usada a seguir) — de modo que, caso o leitor tenha acesso a um conjunto de imagens reais anotado neste mesmo formato, basta organizá-lo na estrutura de pastas descrita adiante para reutilizar todo o restante do código sem qualquer alteração.

Como não há, neste notebook, acesso a um conjunto de imagens reais, o código a seguir **gera sinteticamente** um pequeno conjunto de dados com as mesmas características do cenário descrito: formas geométricas com cor, tamanho e rotação aleatórios, desenhadas com o OpenCV, já anotadas no formato YOLO.

```
import os, random

CLASSES = ['Triangle', 'Square', 'Pentagon', 'Hexagon',
            'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross']
N_LADOS = {'Triangle': 3, 'Square': 4, 'Pentagon': 5, 'Hexagon': 6, 'Heptagon': 7}

def poligono_regular(cx, cy, r, n_lados, rot_graus):
    ang0 = np.deg2rad(rot_graus - 90)
    angs = ang0 + 2 * np.pi * np.arange(n_lados) / n_lados
    return np.stack([cx + r * np.cos(angs), cy + r * np.sin(angs)], axis=1)

def poligono_estrela(cx, cy, r_externo, rot_graus, n_pontas=5):
    r_interno = r_externo * 0.45
    ang0 = np.deg2rad(rot_graus - 90)
    angs = ang0 + np.pi * np.arange(2 * n_pontas) / n_pontas
    raios = np.where(np.arange(2 * n_pontas) % 2 == 0, r_externo, r_interno)
    return np.stack([cx + raios * np.cos(angs), cy + raios * np.sin(angs)], axis=1)

def poligono_cruz(cx, cy, r, rot_graus, espessura_rel=0.35):
    w = r * espessura_rel
    base = np.array([
        (-w, -r), (w, -r), (w, -w), (r, -w), (r, w), (w, w),
        (w, r), (-w, r), (-w, w), (-r, w), (-r, -w), (-w, -w),
    ])
    theta = np.deg2rad(rot_graus)
    R = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
    return base @ R.T + np.array([cx, cy])

def desenha_objeto(img, classe_idx, cx, cy, tamanho, rotacao, cor):
    nome = CLASSES[classe_idx]
    if nome in N_LADOS:
        pts = poligono_regular(cx, cy, tamanho, N_LADOS[nome], rotacao)
        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Star':
        pts = poligono_estrela(cx, cy, tamanho, rotacao)
        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Cross':
        pts = poligono_cruz(cx, cy, tamanho, rotacao)
```



```

        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Circle':
        cv2.circle(img, (int(cx), int(cy)), int(tamanho), cor, -1)
        xs, ys = np.array([cx - tamanho, cx + tamanho]), np.array([cy - tamanho, cy + tamanho])
    else: # Ellipse
        eixo = (int(tamanho), int(tamanho * 0.6))
        cv2.ellipse(img, (int(cx), int(cy)), eixo, rotacao, 0, 360, cor, -1)
        ang = np.deg2rad(rotacao)
        dx = np.hypot(eixo[0] * np.cos(ang), eixo[1] * np.sin(ang))
        dy = np.hypot(eixo[0] * np.sin(ang), eixo[1] * np.cos(ang))
        xs, ys = np.array([cx - dx, cx + dx]), np.array([cy - dy, cy + dy])
    return xs.min(), ys.min(), xs.max(), ys.max()

def gera_imagem(tam_img=160, n_objetos=(1, 3), rng=None):
    rng = rng or random.Random()
    img = np.full((tam_img, tam_img, 3), 255, dtype=np.uint8)
    anotacoes = []
    for _ in range(rng.randint(*n_objetos)):
        classe_idx = rng.randrange(len(CLASSES))
        tamanho = rng.randint(tam_img // 10, tam_img // 5)
        cx = rng.randint(tamanho + 2, tam_img - tamanho - 2)
        cy = rng.randint(tamanho + 2, tam_img - tamanho - 2)
        rotacao = rng.uniform(0, 360)
        cor = tuple(rng.sample(range(30, 226), 3))
        x0, y0, x1, y1 = desenha_objeto(img, classe_idx, cx, cy, tamanho, rotacao, cor)
        x0, y0 = max(x0, 0), max(y0, 0)
        x1, y1 = min(x1, tam_img), min(y1, tam_img)
        xc, yc = (x0 + x1) / 2 / tam_img, (y0 + y1) / 2 / tam_img
        w, h = (x1 - x0) / tam_img, (y1 - y0) / tam_img
        anotacoes.append((classe_idx, xc, yc, w, h))
    return img, anotacoes

# Monta o conjunto de dados no formato esperado pela biblioteca `ultralitics`:
# shapes_dataset/{images,labels}/{train,val}/*.{jpg,txt} + data.yaml
base_dir = "shapes_dataset"
rng_global = random.Random(42)
for split, n_imgs in [("train", 90), ("val", 20)]:
    os.makedirs(f"{base_dir}/images/{split}", exist_ok=True)
    os.makedirs(f"{base_dir}/labels/{split}", exist_ok=True)
    for i in range(n_imgs):
        img, anotacoes = gera_imagem(rng=rng_global)
        cv2.imwrite(f"{base_dir}/images/{split}/{i:04d}.jpg", img)
        with open(f"{base_dir}/labels/{split}/{i:04d}.txt", "w") as f:
            for c, xc, yc, w, h in anotacoes:
                f.write(f"{c} {xc:.4f} {yc:.4f} {w:.4f} {h:.4f}\n")

with open(f"{base_dir}/data.yaml", "w") as f:
    f.write(f"path: {os.path.abspath(base_dir)}\ntrain: images/train\nval: images/val\nnames:\n")
    for i, nome in enumerate(CLASSES):
        f.write(f" {i}: {nome}\n")

print("Dataset sintético gerado: 90 imagens de treino, 20 de validação.")

# Visualiza três amostras com as caixas delimitadoras sobrepostas
fig, axs = plt.subplots(1, 3, figsize=(10, 3.5))
cores_plot = plt.cm.tab10(np.linspace(0, 1, len(CLASSES)))
for ax, i in zip(axs, [0, 1, 2]):

```

```

img = cv2.cvtColor(cv2.imread(f"{base_dir}/images/train/{i:04d}.jpg"), cv2.COLOR_BGR2RGB)
ax.imshow(img)
tam_img = img.shape[0]
with open(f"{base_dir}/labels/train/{i:04d}.txt") as f:
    for linha in f:
        c, xc, yc, w, h = map(float, linha.split())
        c = int(c)
        x0, y0 = (xc - w / 2) * tam_img, (yc - h / 2) * tam_img
        ax.add_patch(plt.Rectangle((x0, y0), w * tam_img, h * tam_img,
                                   fill=False, edgecolor=cores_plot[c], linewidth=2))
        ax.text(x0, max(y0 - 3, 0), CLASSES[c], color=cores_plot[c], fontsize=8, weight="bold")
ax.set_axis_off()
plt.tight_layout()
plt.show()

```

Dataset sintético gerado: 90 imagens de treino, 20 de validação.



Figura 9.5: Amostras do conjunto de dados sintético de objetos geométricos, com as caixas delimitadoras no formato YOLO sobrepostas.

9.7.2 Ajuste Fino de um YOLO Pré-treinado

A biblioteca *ultralytics* fornece uma interface de alto nível para carregar arquiteturas YOLO com pesos pré-treinados na base COCO e ajustá-las (*fine-tuning*) a uma nova tarefa — exatamente o padrão de transferência de aprendizado já discutido para classificação, mas agora aplicado à detecção: em vez de apenas substituir a última camada, a cabeça de detecção é reconfigurada para as novas classes (nove formas geométricas em vez de 80 categorias da COCO), e o treinamento é conduzido por poucas épocas sobre o conjunto de dados sintético.

! Important

A célula a seguir **não é executada automaticamente** neste notebook: o carregamento do modelo pré-treinado exige o download de seus pesos a partir dos servidores da Ultralytics/GitHub, o que requer conexão à internet no ambiente de execução, além da instalação da biblioteca *ultralytics*. Em um ambiente com GPU, o treinamento abaixo (15 épocas, ~110 imagens de 160×160) leva menos de um minuto; em CPU, alguns minutos.

```

try:
    from ultralytics import YOLO
except ImportError:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "ultralytics"], check=True)
    from ultralytics import YOLO

```

```
# Carrega o YOLOv8-nano com pesos pré-treinados na base COCO (transferência de aprendizado)
modelo_yolo = YOLO("yolov8n.pt")

# Ajuste fino: `freeze=10` mantém congeladas as 10 primeiras camadas do backbone
# (características genéricas de baixo nível, como bordas e contornos), treinando
# apenas as camadas finais e a cabeça de detecção para as novas 9 classes -
# o mesmo princípio de congelamento parcial discutido no Exercício 3.
resultados = modelo_yolo.train(
    data="shapes_dataset/data.yaml",
    epochs=15,
    imgsz=160,
    batch=8,
    freeze=10,
    verbose=False,
    plots=False,
)

metricas = modelo_yolo.val()
print(f"mAP50 no conjunto de validação: {metricas.box.map50:.3f}")
```

Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cu130 CPU (Intel Core i7-4870HQ 2.50GHz)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=1
Overriding model.yaml nc=80 with nc=9

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	7360	ultralytics.nn.modules.block.C2f	[32, 32, 1, True]
3	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
4	-1	2	49664	ultralytics.nn.modules.block.C2f	[64, 64, 2, True]
5	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
6	-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	460288	ultralytics.nn.modules.block.C2f	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12	-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15	-1	1	37248	ultralytics.nn.modules.block.C2f	[192, 64, 1]
16	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18	-1	1	123648	ultralytics.nn.modules.block.C2f	[192, 128, 1]
19	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21	-1	1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
22	[15, 18, 21]	1	753067	ultralytics.nn.modules.head.Detect	[9, 16, None, [64, 128, 160, 320, 640, 800, 1000]]

Model summary: 130 layers, 3,012,603 parameters, 3,012,587 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights

Freezing layer 'model.0.conv.weight'

Freezing layer 'model.0.bn.weight'

Freezing layer 'model.0.bn.bias'

Freezing layer 'model.1.conv.weight'

Freezing layer 'model.1.bn.weight'

Freezing layer 'model.1.bn.bias'

Freezing layer 'model.2.cv1.conv.weight'

Freezing layer 'model.2.cv1.bn.weight'

```

Freezing layer 'model.2.cv1.bn.bias'
Freezing layer 'model.2.cv2.conv.weight'
Freezing layer 'model.2.cv2.bn.weight'
Freezing layer 'model.2.cv2.bn.bias'
Freezing layer 'model.2.m.0.cv1.conv.weight'
Freezing layer 'model.2.m.0.cv1.bn.weight'
Freezing layer 'model.2.m.0.cv1.bn.bias'
Freezing layer 'model.2.m.0.cv2.conv.weight'
Freezing layer 'model.2.m.0.cv2.bn.weight'
Freezing layer 'model.2.m.0.cv2.bn.bias'
Freezing layer 'model.3.conv.weight'
Freezing layer 'model.3.bn.weight'
Freezing layer 'model.3.bn.bias'
Freezing layer 'model.4.cv1.conv.weight'
Freezing layer 'model.4.cv1.bn.weight'
Freezing layer 'model.4.cv1.bn.bias'
Freezing layer 'model.4.cv2.conv.weight'
Freezing layer 'model.4.cv2.bn.weight'
Freezing layer 'model.4.cv2.bn.bias'
Freezing layer 'model.4.m.0.cv1.conv.weight'
Freezing layer 'model.4.m.0.cv1.bn.weight'
Freezing layer 'model.4.m.0.cv1.bn.bias'
Freezing layer 'model.4.m.0.cv2.conv.weight'
Freezing layer 'model.4.m.0.cv2.bn.weight'
Freezing layer 'model.4.m.0.cv2.bn.bias'
Freezing layer 'model.4.m.1.cv1.conv.weight'
Freezing layer 'model.4.m.1.cv1.bn.weight'
Freezing layer 'model.4.m.1.cv1.bn.bias'
Freezing layer 'model.4.m.1.cv2.conv.weight'
Freezing layer 'model.4.m.1.cv2.bn.weight'
Freezing layer 'model.4.m.1.cv2.bn.bias'
Freezing layer 'model.5.conv.weight'
Freezing layer 'model.5.bn.weight'
Freezing layer 'model.5.bn.bias'
Freezing layer 'model.6.cv1.conv.weight'
Freezing layer 'model.6.cv1.bn.weight'
Freezing layer 'model.6.cv1.bn.bias'
Freezing layer 'model.6.cv2.conv.weight'
Freezing layer 'model.6.cv2.bn.weight'
Freezing layer 'model.6.cv2.bn.bias'
Freezing layer 'model.6.m.0.cv1.conv.weight'
Freezing layer 'model.6.m.0.cv1.bn.weight'
Freezing layer 'model.6.m.0.cv1.bn.bias'
Freezing layer 'model.6.m.0.cv2.conv.weight'
Freezing layer 'model.6.m.0.cv2.bn.weight'
Freezing layer 'model.6.m.0.cv2.bn.bias'
Freezing layer 'model.6.m.1.cv1.conv.weight'
Freezing layer 'model.6.m.1.cv1.bn.weight'
Freezing layer 'model.6.m.1.cv1.bn.bias'
Freezing layer 'model.6.m.1.cv2.conv.weight'
Freezing layer 'model.6.m.1.cv2.bn.weight'
Freezing layer 'model.6.m.1.cv2.bn.bias'
Freezing layer 'model.7.conv.weight'
Freezing layer 'model.7.bn.weight'
Freezing layer 'model.7.bn.bias'
Freezing layer 'model.8.cv1.conv.weight'
Freezing layer 'model.8.cv1.bn.weight'
Freezing layer 'model.8.cv1.bn.bias'
Freezing layer 'model.8.cv2.conv.weight'
Freezing layer 'model.8.cv2.bn.weight'

```

```

Freezing layer 'model.8.cv2.bn.bias'
Freezing layer 'model.8.m.0.cv1.conv.weight'
Freezing layer 'model.8.m.0.cv1.bn.weight'
Freezing layer 'model.8.m.0.cv1.bn.bias'
Freezing layer 'model.8.m.0.cv2.conv.weight'
Freezing layer 'model.8.m.0.cv2.bn.weight'
Freezing layer 'model.8.m.0.cv2.bn.bias'
Freezing layer 'model.9.cv1.conv.weight'
Freezing layer 'model.9.cv1.bn.weight'
Freezing layer 'model.9.cv1.bn.bias'
Freezing layer 'model.9.cv2.conv.weight'
Freezing layer 'model.9.cv2.bn.weight'
Freezing layer 'model.9.cv2.bn.bias'
Freezing layer 'model.22.dfl.conv.weight'
train: Fast image access (ping: 0.0±0.0 ms, read: 109.5±39.1 MB/s, size: 3.3 KB)
train: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/train.cache... 90 images, 0 backgr
val: Fast image access (ping: 0.0±0.0 ms, read: 157.9±82.3 MB/s, size: 2.9 KB)
val: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val.cache... 20 images, 0 backgr
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimi
optimizer: AdamW(lr=0.000769, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=
Image sizes 160 train, 160 val
Using 0 dataloader workers
Logging results to /home/fz/fz/VSCode/pdi-vc/runs/detect/train-3
Starting training for 15 epochs...

```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
1/15	OG	1.097	4.44	1.05	9	160: 100%	12/12	2.6it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.00112	0.0509	0.00198	0.000495		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
2/15	OG	0.9467	4.195	0.998	7	160: 100%	12/12	3.9it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.00109	0.0509	0.00171	0.000417		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
3/15	OG	0.8689	3.992	0.9577	14	160: 100%	12/12	3.7it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.00205	0.145	0.00989	0.0044		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
4/15	OG	0.8853	3.787	0.9762	7	160: 100%	12/12	3.6it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.00711	0.648	0.162	0.124		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
5/15	OG	0.8703	3.61	0.9696	8	160: 100%	12/12	3.6it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.0103	0.882	0.293	0.203		

Closing dataloader mosaic

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
6/15	OG	0.7441	3.006	0.9087	5	160: 100%	12/12	3.4it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.716	0.111	0.37	0.307		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
7/15	OG	0.6964	2.895	0.8826	4	160: 100%	12/12	3.4it/s	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		2/
	all	20	42	0.509	0.194	0.395	0.334		

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
8/15	OG	0.7232	2.701	0.9014	3	160:	100%	12/12	1.8it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.502	0.426	0.417	0.353		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
9/15	OG	0.6997	2.514	0.9183	4	160:	100%	12/12	2.5it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.447	0.542	0.418	0.35		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
10/15	OG	0.705	2.297	0.8892	5	160:	100%	12/12	4.7it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.411	0.588	0.422	0.348		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
11/15	OG	0.6989	2.299	0.8962	6	160:	100%	12/12	5.5it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.501	0.45	0.536	0.453		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
12/15	OG	0.6627	2.155	0.8908	6	160:	100%	12/12	4.5it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.618	0.466	0.538	0.461		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
13/15	OG	0.661	2.154	0.9063	4	160:	100%	12/12	4.6it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.666	0.477	0.495	0.422		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
14/15	OG	0.7004	2.09	0.9239	4	160:	100%	12/12	5.9it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.431	0.575	0.496	0.44		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
15/15	OG	0.7029	2.023	0.9067	5	160:	100%	12/12	6.0it/s
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
	all	20	42	0.569	0.581	0.502	0.447		

15 epochs completed in 0.017 hours.

Optimizer stripped from /home/fz/fz/VSCode/pdi-vc/runs/detect/train-3/weights/last.pt, 6.2MB

Optimizer stripped from /home/fz/fz/VSCode/pdi-vc/runs/detect/train-3/weights/best.pt, 6.2MB

Validating /home/fz/fz/VSCode/pdi-vc/runs/detect/train-3/weights/best.pt...

Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cu130 CPU (Intel Core i7-4870HQ 2.50GHz)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
all	20	42	0.62	0.466	0.538	0.461		

Speed: 0.1ms preprocess, 11.3ms inference, 0.0ms loss, 0.7ms postprocess per image

Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cu130 CPU (Intel Core i7-4870HQ 2.50GHz)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.0±0.0 ms, read: 214.5±94.6 MB/s, size: 3.8 KB)

val: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val.cache... 20 images, 0 backgr

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	2/
all	20	42	0.62	0.466	0.538	0.461		
Triangle	2	2	0	0	0.195	0.179		
Square	6	6	0.943	0.333	0.526	0.475		
Pentagon	4	5	0	0	0.21	0.189		
Hexagon	2	2	1	0	0.221	0.0601		

Heptagon	1	1	0.66	1	0.995	0.895
Circle	7	7	0.862	0.286	0.53	0.451
Ellipse	2	2	0.397	1	0.398	0.368
Star	8	9	0.895	1	0.984	0.853
Cross	7	8	0.819	0.572	0.78	0.678

Speed: 0.6ms preprocess, 11.8ms inference, 0.0ms loss, 0.7ms postprocess per image

Results saved to /home/fz/fz/VSCode/pdi-vc/runs/detect/val-2

mAP50 no conjunto de validação: 0.538

```

resultado = modelo_yolo.predict("shapes_dataset/images/val/0000.jpg", conf=0.4, verbose=False)[0]

fig, ax = plt.subplots(figsize=(4, 4))
ax.imshow(cv2.cvtColor(resultado.orig_img, cv2.COLOR_BGR2RGB))
for box in resultado.bboxes:
    x0, y0, x1, y1 = box.xyxy[0].tolist()
    classe, conf = CLASSES[int(box.cls)], float(box.conf)
    ax.add_patch(plt.Rectangle((x0, y0), x1 - x0, y1 - y0,
                              fill=False, edgecolor="red", linewidth=2))
    ax.text(x0, max(y0 - 3, 0), f"{classe} {conf:.2f}", color="red", fontsize=8, weight="bold")
ax.set_axis_off()
plt.show()

```



Figura 9.6: Detecções do YOLO ajustado por transferência de aprendizado em uma imagem de validação não vista durante o treinamento.

```

import os
from ultralytics import YOLO

# 1. Definição das classes do problema geométrico
CLASSES_GEOMETRICAS = [
    'Triangle', 'Square', 'Pentagon', 'Hexagon',
    'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross'
]

# 2. Configuração do arquivo de dados (dataset.yaml) exigido pelo YOLO
yaml_content = """
train: ./shapes_dataset/images/train
val: ./shapes_dataset/images/val

```



```

nc: 9
names: ['Triangle', 'Square', 'Pentagon', 'Hexagon', 'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross']
"""

with open("dataset.yaml", "w") as f:
    f.write(yaml_content)

# 3. Carrega um modelo YOLO nano pré-treinado (Extrator de características congelado/inicializado)
# 0 sufixo 'n' indica a versão mais leve, ideal para restrições computacionais
modelo_yolo = YOLO("yolov8n.pt")

# 4. Transferência de Aprendizado / Ajuste Fino (Fine-tuning)
# 0 framework adapta automaticamente a camada de saída para 9 classes
resultados = modelo_yolo.train(
    data="dataset.yaml",
    epochs=4,
    imgsz=640,
    batch=16,
    device=device # Reaproveita a variável 'device' configurada no início do capítulo
)

```

```

# 5. Validação e Inferência em uma imagem de teste
# Exemplo de URL de teste enviada em exames: https://dropbox.com/s/ekjbzp14jt90bfq/00004.jpg
imagem_teste = "./shapes_dataset/images/val/0001.jpg"
predicoes = modelo_yolo(imagem_teste)

# Exibe o resultado visual com as caixas sobrepostas
predicoes[0].show()

```

```

image 1/1 /home/fz/fz/VSCoDe/pdi-vc/all/cap09/shapes_dataset/images/val/0001.jpg: 640x640 (no detections)
Speed: 6.9ms preprocess, 213.0ms inference, 0.6ms postprocess per image at shape (1, 3, 640, 640)

```

```

# --- 1. Instalação e Importação das Bibliotecas ---
# 0 Ultralytics YOLO é uma biblioteca popular para treino e inferência.
# !pip install ultralytics -q

from ultralytics import YOLO
import matplotlib.pyplot as plt
import cv2
import numpy as np
from pathlib import Path
import torch

# Verificar dispositivo
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Dispositivo usado: {device}")

# --- 2. Carregar o Modelo YOLO Pré-treinado ---
# 'yolov8n.pt' é a versão "nano" (mais leve e rápida), pré-treinada no COCO.
# Usaremos seus pesos como ponto de partida (transferência de aprendizado).
modelo = YOLO('yolov8n.pt')
print("Modelo YOLOv8n carregado com sucesso!")

# --- 3. Preparar o Dataset de Objetos Geométricos ---
# Assumimos que você tem uma estrutura de pastas pronta para o YOLO:
# dataset_geometrico/
#   images/
#     00004.jpg, 00005.jpg, ...
#   labels/

```

```

# 00004.txt, 00005.txt, ...
#
# 0 arquivo de label deve estar no formato YOLO:
# <class_id> <x_center> <y_center> <width> <height>
# (todos normalizados entre 0 e 1)

# Exemplo de caminho para o dataset (substitua pelo seu caminho)
caminho_dataset = Path("./shapes_dataset")
caminho_imagens = caminho_dataset / "images"
caminho_rotulos = caminho_dataset / "labels"

# Crie um arquivo data.yaml para descrever o dataset
data_yaml_content = f"""
path: {caminho_dataset.resolve()}
train: images
val: images # Se você tiver uma pasta separada, use 'images/val'

nc: 9 # Número de classes
names: ['Triangle', 'Square', 'Pentagon', 'Hexagon', 'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross']
"""

# Cria o arquivo data.yaml (se não existir)
data_yaml_path = caminho_dataset / "data.yaml"
if not data_yaml_path.exists():
    data_yaml_path.write_text(data_yaml_content)
    print(f"Arquivo data.yaml criado em: {data_yaml_path}")
else:
    print("Arquivo data.yaml já existe.")

# --- 4. Treinamento com Transferência de Aprendizado ---
# A chave aqui é que os pesos da cabeça de detecção (camadas finais) serão
# re-inicializados, enquanto o extrator de características (backbone) é ajustado.
resultados = modelo.train(
    data=str(data_yaml_path), # Caminho para o arquivo data.yaml
    epochs=5, # Número de épocas (ajuste fino com poucos dados)
    imsz=640, # Tamanho da imagem de entrada
    batch=16, # Tamanho do lote
    device=device, # 'cpu' ou 'cuda'
    project='runs/detect', # Pasta para salvar resultados
    name='geometricos_yolov8n', # Nome do experimento
    exist_ok=True, # Sobrescreve se existir
    pretrained=True, # Usar pesos pré-treinados (já é padrão)
)

print("Treinamento concluído!")
melhor_modelo_path = Path(resultados.save_dir) / "weights" / "best.pt"

# --- 5. Avaliação e Inferência em uma Imagem de Exemplo ---
if melhor_modelo_path.exists():
    # Carrega o melhor modelo treinado
    melhor_modelo = YOLO(str(melhor_modelo_path))

    # Realiza inferência em uma imagem de teste (ex: '00004.jpg')
    caminho_teste = caminho_imagens / "00004.jpg"
    if caminho_teste.exists():
        resultados_pred = melhor_modelo(caminho_teste, conf=0.25)

        # Plota a imagem com as detecções
        for r in resultados_pred:
            # A imagem com as caixas é renderizada no OpenCV

```

```

img_plot = r.plot() # Retorna um array BGR
img_plot_rgb = cv2.cvtColor(img_plot, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8, 8))
plt.imshow(img_plot_rgb)
plt.axis('off')
plt.title("Detecções YOLO (Transferência)")
plt.show()

# Exibe as detecções como texto
print("\nObjetos Detectados:")
for box in r.bboxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    nome_classe = modelo.names[cls]
    print(f" - Classe: {nome_classe} (Confiança: {conf:.2f})")
else:
    print(f"Imagem de teste '{caminho_teste}' não encontrada.")
else:
    print("Modelo treinado não encontrado. Verifique o diretório de saída do treinamento.")

```

Dispositivo usado: cpu

Modelo YOLOv8n carregado com sucesso!

Arquivo data.yaml já existe.

Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cu130 CPU (Intel Core i7-4870HQ 2.50GHz)

engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=

Overriding model.yaml nc=80 with nc=9

	from	n	params	module	arguments
0		-1 1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1		-1 1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2		-1 1	7360	ultralytics.nn.modules.block.C2f	[32, 32, 1, True]
3		-1 1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
4		-1 2	49664	ultralytics.nn.modules.block.C2f	[64, 64, 2, True]
5		-1 1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
6		-1 2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
7		-1 1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8		-1 1	460288	ultralytics.nn.modules.block.C2f	[256, 256, 1, True]
9		-1 1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10		-1 1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12		-1 1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
13		-1 1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15		-1 1	37248	ultralytics.nn.modules.block.C2f	[192, 64, 1]
16		-1 1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18		-1 1	123648	ultralytics.nn.modules.block.C2f	[192, 128, 1]
19		-1 1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21		-1 1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
22	[15, 18, 21]	1	753067	ultralytics.nn.modules.head.Detect	[9, 16, None, [64, 128, 256, 384]]

Model summary: 130 layers, 3,012,603 parameters, 3,012,587 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

train: Fast image access (ping: 0.0±0.0 ms, read: 147.1±60.4 MB/s, size: 3.3 KB)

train: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/train.cache... 90 images, 0 backgr

val: Fast image access (ping: 0.0±0.0 ms, read: 100.7±44.3 MB/s, size: 2.9 KB)

val: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val.cache... 20 images, 0 backgr

```
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimi
optimizer: AdamW(lr=0.000769, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=
Plotting labels to /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n/labels.jpg...
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n
Starting training for 5 epochs...
```

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size		
1/5	OG	0.8689	3.848	1.116	49	640: 100%	6/6	5.2s/it 3
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/
	all	20	42	0.0137	0.959	0.0943	0.0783	
2/5	OG	0.7538	3.61	1.074	45	640: 100%	6/6	5.5s/it 3
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/
	all	20	42	0.0109	1	0.165	0.144	
3/5	OG	0.7169	3.336	1.082	38	640: 100%	6/6	5.9s/it 3
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/
	all	20	42	0.0109	0.984	0.197	0.175	
4/5	OG	0.6537	3.156	1.026	34	640: 100%	6/6	4.6s/it 2
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/
	all	20	42	0.0111	0.984	0.206	0.183	
5/5	OG	0.6495	3.073	1.067	47	640: 100%	6/6	5.6s/it 3
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	1/
	all	20	42	0.0113	0.984	0.223	0.2	

5 epochs completed in 0.049 hours.

Optimizer stripped from /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n/weights/la

Optimizer stripped from /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n/weights/be

Validating /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n/weights/best.pt...

Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cu130 CPU (Intel Core i7-4870HQ 2.50GHz)

Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	
all	20	42	0.0113	0.984	0.221	0.196	1/
Triangle	2	2	0.0104	1	0.117	0.108	
Square	6	6	0.00852	1	0.168	0.157	
Pentagon	4	5	0.00245	1	0.359	0.35	
Hexagon	2	2	0.00222	1	0.0492	0.0356	
Heptagon	1	1	0.00353	1	0.0433	0.0433	
Circle	7	7	0.0293	0.857	0.548	0.494	
Ellipse	2	2	0.00438	1	0.13	0.125	
Star	8	9	0.0319	1	0.361	0.275	
Cross	7	8	0.00856	1	0.214	0.18	

Speed: 1.7ms preprocess, 81.4ms inference, 0.0ms loss, 25.0ms postprocess per image

Results saved to /home/fz/fz/VSCoDe/pdi-vc/runs/detect/runs/detect/geometricos_yolov8n

Treinamento concluído!

Imagem de teste 'shapes_dataset/images/00004.jpg' não encontrada.

Um domínio muito mais distante que o dos dígitos

No experimento de transferência de aprendizado entre dígitos manuscritos (domínios A e B), a tarefa de origem e a de destino compartilhavam estatísticas visuais muito próximas: ambas eram traços em tons de cinza sobre fundo uniforme. Aqui, a distância entre domínios é bem maior — o YOLO foi pré-treinado em **fotografias naturais coloridas** da COCO (pessoas, animais, veículos, objetos do cotidiano), e a tarefa de destino consiste em **formas geométricas sintéticas, de cor sólida e contorno bem definido**, sem textura, iluminação ou fundo complexo.

Ainda assim, a transferência de aprendizado tende a ajudar: as camadas iniciais de um detector treinado na COCO aprendem filtros muito genéricos — detectores de bordas, cantos e regiões de contraste — que continuam úteis para delimitar o contorno de um triângulo ou de uma estrela, mesmo que o conteúdo visual final seja muito diferente. É por isso que o congelamento (`freeze=10`) é aplicado apenas às primeiras camadas do backbone: as camadas mais profundas, que na COCO aprenderam a reconhecer partes de objetos complexos do mundo real (rodas, focinhos, faces), têm pouca serventia direta para formas geométricas simples e se beneficiam de serem reajustadas.

Este é o mesmo compromisso discutido no Exercício 3 deste capítulo — entre congelar totalmente o extrator, ajustar parcialmente algumas camadas, ou treinar tudo do zero — agora observado em uma tarefa de detecção, e com um domínio de origem e destino visivelmente mais distantes do que o observado entre os dois grupos de dígitos.

9.7.3 Usando um Conjunto de Dados Real

O pipeline acima foi construído inteiramente em torno do **formato de anotação YOLO** (classe, x_{centro} , y_{centro} , largura, altura, normalizados pela largura e altura da imagem) exatamente para que ele possa ser reaproveitado sem alterações caso o leitor tenha acesso a um conjunto de imagens reais anotado da mesma forma — por exemplo, um conjunto de imagens de objetos geométricos fotografados ou renderizados, cada um com um arquivo `.txt` correspondente no mesmo formato usado aqui. Para isso, bastaria:

1. Organizar as imagens reais em `shapes_dataset/images/train` e `shapes_dataset/images/val`, e os arquivos `.txt` de anotação correspondentes nas pastas `labels/train` e `labels/val` (um arquivo de anotação por imagem, mesmo nome-base, extensão `.txt`);
2. Ajustar o arquivo `data.yaml` caso o número ou os nomes das classes sejam diferentes;
3. Executar as mesmas células de treinamento, ajuste fino e visualização já apresentadas, sem qualquer outra modificação de código.

Essa separação entre **geração/organização dos dados** e **treinamento do modelo** é, na prática, o motivo pelo qual formatos de anotação padronizados (como o do YOLO) são tão amplamente adotados: eles permitem trocar o conjunto de dados de entrada — sintético por real, um domínio por outro — mantendo inalterado todo o restante do pipeline de transferência de aprendizado.

9.8 Integrando Geometria e Aprendizado Profundo

O livro se encerra retomando um fio condutor que atravessou toda a Parte II: a **geometria projetiva** (transformações de perspectiva e homografia, estudadas nos Capítulos 6 e 8) e, agora, o **aprendizado profundo**, combinam-se para resolver problemas de Visão Computacional com aplicação direta no mundo real. Três aplicações ilustram essa integração.

9.8.1 Realidade Aumentada

A realidade aumentada sobrepõe conteúdo virtual a uma cena real, alinhado geometricamente com ela. O princípio é exatamente a **homografia** do Capítulo 8: detecta-se um marcador (ou, em sistemas modernos, características naturais da cena, aprendidas por uma rede), estima-se a transformação de perspectiva entre suas coordenadas conhecidas e sua posição na imagem, e essa mesma transformação é usada para deformar e posicionar um conteúdo virtual sobre a cena.

```

dic = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
aruco = cv2.cvtColor(cv2.aruco.generateImageMarker(dic, 7, 200), cv2.COLOR_GRAY2BGR)

src = np.float32([[0,0],[200,0],[200,200],[0,200]])
dst = np.float32([[150,120],[430,90],[460,350],[140,380]])

cena = np.full((480,640,3), 200, np.uint8)
H, _ = cv2.findHomography(src, dst)

mask = cv2.warpPerspective(np.full((200,200),255,np.uint8), H, (640,480))
cena[mask>0] = cv2.warpPerspective(aruco, H, (640,480))[mask>0]

det = cv2.aruco.ArucoDetector(dic, cv2.aruco.DetectorParameters())
corners, ids, _ = det.detectMarkers(cena)

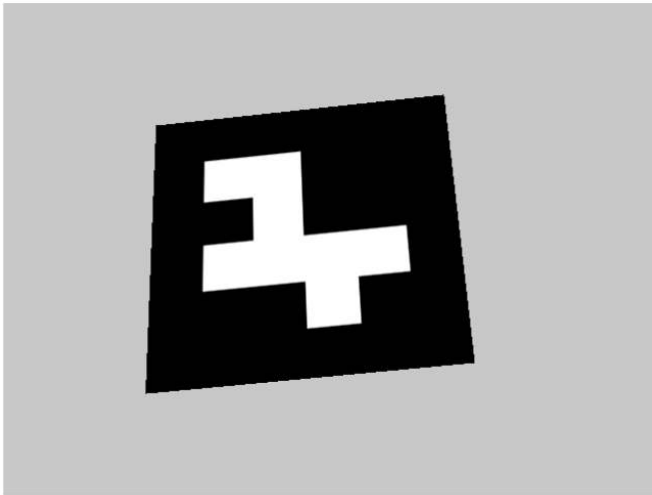
virtual = cv2.resize(cv2.cvtColor(skdata.astronaut(), cv2.COLOR_RGB2BGR), (200,200))
H, _ = cv2.findHomography(src, corners[0][0])

ra = cena.copy()
mask = cv2.warpPerspective(np.full((200,200),255,np.uint8), H, (640,480))
ra[mask>0] = cv2.warpPerspective(virtual, H, (640,480))[mask>0]

mm.show(
    [cv2.cvtColor(cena, cv2.COLOR_BGR2RGB),
     cv2.cvtColor(ra, cv2.COLOR_BGR2RGB)],
    titles=["Marcador detectado", "Overlay via homografia"],
    cols=2, figsize=(9,4)
)

```

Marcador detectado



Overlay via homografia



Figura 9.7: Realidade aumentada baseada em marcador: um marcador ArUco é detectado na cena e sua homografia é reaproveitada para posicionar uma imagem virtual sobre ele.

Nota: sistemas modernos de realidade aumentada (como o ARKit da Apple ou o ARCore do Google) substituem marcadores explícitos por **odometria visual-inercial** baseada em aprendizado profundo, capaz de rastrear características naturais da cena (sem a necessidade de um marcador impresso) e estimar a pose da câmera em tempo real — mas o princípio geométrico subjacente de alinhar conteúdo virtual à cena real por meio de uma transformação estimada permanece o mesmo.

9.8.2 Fotogrametria e Referência de Escala

Uma pergunta recorrente em aplicações práticas é: dado um objeto medido apenas em pixels em uma fotografia, qual é seu tamanho real? A resposta clássica utiliza um **objeto de referência de tamanho conhecido** presente na mesma imagem, na mesma distância aproximada da câmera:

```
COR_REFERENCIA = (200, 200, 200) # cartão de referência: cinza claro
COR_OBJETO = (60, 60, 220) # objeto a medir: vermelho

# Cena original: os objetos são desenhados PREENCHIDOS, como apareceriam de fato
# em uma foto -- e não apenas como contornos de caixa delimitadora.
cena_medicao = np.full((300, 500, 3), 255, dtype=np.uint8)
cv2.rectangle(cena_medicao, (30, 200), (30 + 140, 200 + 88), COR_REFERENCIA, -1) # referência conhecido
cv2.rectangle(cena_medicao, (250, 100), (250 + 220, 100 + 150), COR_OBJETO, -1) # objeto a medir

def caixa_delimitadora_por_cor(imagem_bgr, cor_bgr, tolerancia=40):
    diferenca = np.abs(imagem_bgr.astype(int) - np.array(cor_bgr)).sum(axis=2)
    mascara = (diferenca < tolerancia).astype(np.uint8) * 255
    contornos, _ = cv2.findContours(mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    maior_contorno = max(contornos, key=cv2.contourArea)
    return cv2.boundingRect(maior_contorno) # (x, y, largura, altura) em pixels

x_ref, y_ref, w_ref_px, h_ref_px = caixa_delimitadora_por_cor(cena_medicao, COR_REFERENCIA)
x_obj, y_obj, w_obj_px, h_obj_px = caixa_delimitadora_por_cor(cena_medicao, COR_OBJETO)

LARGURA_REFERENCIA_CM = 8.56 # largura padrão de um cartão (ISO/IEC 7810)
razao_cm_por_px = LARGURA_REFERENCIA_CM / w_ref_px
largura_obj_cm = w_obj_px * razao_cm_por_px
altura_obj_cm = h_obj_px * razao_cm_por_px

# Painel de resultado: a MESMA cena, agora com as caixas delimitadoras detectadas
# (em verde, obtidas por segmentação de cor) e as medições estimadas sobrepostas.
resultado = cena_medicao.copy()
cv2.rectangle(resultado, (x_ref, y_ref), (x_ref + w_ref_px, y_ref + h_ref_px), (0, 180, 0), 2)
cv2.rectangle(resultado, (x_obj, y_obj), (x_obj + w_obj_px, y_obj + h_obj_px), (0, 180, 0), 2)

cv2.putText(resultado, f"{LARGURA_REFERENCIA_CM:.2f} cm",
             (x_ref, y_ref - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 120, 0), 2)

cv2.putText(resultado,
             f"{largura_obj_cm:.1f} x {altura_obj_cm:.1f} cm",
             (x_obj, y_obj - 8), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 120, 0), 2)

mm.show(
    [cv2.cvtColor(cena_medicao, cv2.COLOR_BGR2RGB),
     cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB)],
    titles=[
        "Cena original",
        "Caixas delimitadoras detectadas + medição por referência de escala"
    ],
    cols=2, figsize=(9,4)
)
```

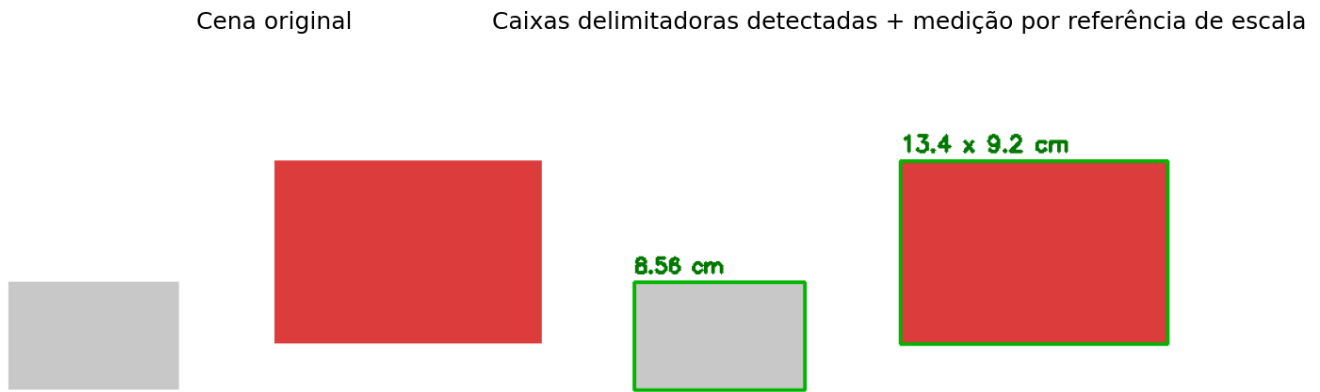



Figura 9.8: Medição de um objeto a partir de uma referência de escala conhecida (um cartão de 8,56 cm de largura).

Essa é a base da **fotogrametria** clássica — a mesma lógica se estende, com maior sofisticação geométrica (calibração de câmera, correção de distorção de lente, múltiplas vistas), a aplicações como levantamento topográfico por drones e reconstrução 3D de objetos a partir de fotografias.

9.8.3 Visão Estereoscópica

Enquanto a fotogrametria acima assume uma única imagem e um objeto de referência conhecido, a **visão estereoscópica** estima profundidade diretamente a partir de **duas câmeras** (ou duas fotografias da mesma cena, tiradas de posições ligeiramente diferentes) — o mesmo princípio da visão binocular humana. A ideia central é a **disparidade**: um ponto próximo à câmera se desloca mais, entre as duas imagens, do que um ponto distante.

Formalmente, a profundidade Z de um ponto se relaciona à disparidade d (a diferença de posição horizontal do mesmo ponto nas duas imagens) por:

$$Z = \frac{f \cdot B}{d},$$

em que f é a distância focal da câmera e B é a **linha de base** (*baseline*) — a distância entre os dois pontos de captura. Quanto maior a disparidade, menor a profundidade (objeto mais próximo).

```
img_base = cv2.cvtColor(skdata.astronaut(), cv2.COLOR_RGB2GRAY)

DESLOC_FUNDO = 4    # pequeno deslocamento: objeto distante
DESLOC_OBJETO = 22  # grande deslocamento: objeto próximo à câmera

def deslocar(imagem, dx):
    M = np.float32([[1, 0, dx], [0, 1, 0]])
    return cv2.warpAffine(imagem, M, (imagem.shape[1], imagem.shape[0]))

img_esquerda = img_base.copy()
img_direita = deslocar(img_base, -DESLOC_FUNDO)

# Simula um objeto em primeiro plano com deslocamento (disparidade) maior
y0, y1, x0, x1 = 180, 340, 200, 360
img_direita[y0:y1, x0:x1] = deslocar(img_base, -DESLOC_OBJETO)[y0:y1, x0:x1]

correspondencia_estereo = cv2.StereoBM_create(numDisparities=32, blockSize=15)
mapa_disparidade = correspondencia_estereo.compute(img_esquerda, img_direita).astype(np.float32) / 16.0

disp_fundo = mapa_disparidade[:,150, :150].mean()
```

```

disp_objeto = mapa_disparidade[200:320, 220:340].mean()
print(f"Disparidade média no fundo: {disp_fundo:.2f} px")
print(f"Disparidade média no objeto em primeiro plano: {disp_objeto:.2f} px")
print("→ Disparidade maior = objeto mais próximo da câmera, como previsto pela equação  $Z = f \cdot B/d$ .")

mm.show(
    [img_esquerda, img_direita, mapa_disparidade],
    titles=["Imagem Esquerda", "Imagem Direita", "Mapa de Disparidade"],
    cols=3, figsize=(10, 3.5)
)

```

Disparidade média no fundo: 2.58 px
 Disparidade média no objeto em primeiro plano: 21.63 px
 → Disparidade maior = objeto mais próximo da câmera, como previsto pela equação $Z = f \cdot B/d$.



Figura 9.9: Par estéreo sintético (um objeto em primeiro plano deslocado mais do que o fundo) e o mapa de disparidade calculado — regiões mais claras indicam maior disparidade (objetos mais próximos).

Nota: métodos clássicos de correspondência estéreo, como o utilizado acima, comparam diretamente blocos de pixels entre as duas imagens — uma tarefa que sofre em regiões de textura pouco distintiva (paredes lisas, céu). Redes neurais de **estimativa de profundidade monocular** (a partir de uma única imagem) e de **correspondência estéreo aprendida** superam essa limitação aprendendo priors visuais sobre a estrutura de cenas do mundo real a partir de grandes volumes de dados — um caso de uso direto das CNNs estudadas neste capítulo aplicado diretamente ao problema geométrico com o qual o livro se encerra.

9.9 Resumo

Este capítulo, que encerra a Parte II do livro, apresentou:

- **Convolução e pooling aprendidos:** a mesma operação matemática de convolução do Capítulo 3, mas com *kernels* tratados como parâmetros ajustados por treinamento, em vez de definidos manualmente;
- **Compartilhamento de pesos e hierarquia de características** como as propriedades que tornam as CNNs eficientes e capazes de aprender representações cada vez mais abstratas em camadas sucessivas;
- **Treinamento de uma CNN do zero**, com desempenho comparável — e não necessariamente superior — aos classificadores clássicos do Capítulo 7 em uma base pequena e simples, reforçando que a escolha do método deve ser proporcional à complexidade real do problema;
- **Transferência de aprendizado**, demonstrada experimentalmente como uma estratégia eficaz para tarefas com poucos dados rotulados, reaproveitando um extrator de características já treinado em uma tarefa relacionada — e suas limitações, na forma da transferência negativa entre domínios muito distintos;

- **Aplicações em larga escala** com modelos pré-treinados de classificação, detecção (Faster R-CNN) e segmentação (DeepLabV3), seguindo o mesmo princípio de transferência de aprendizado em escala industrial;
- **Transferência de aprendizado aplicada à detecção de objetos**, ajustando um YOLO pré-treinado na COCO para localizar e classificar objetos geométricos sintéticos, ilustrando o mesmo princípio de congelamento parcial em um domínio de origem e destino ainda mais distantes entre si;
- A integração de **geometria computacional** (homografia, calibração) e **aprendizado profundo** em três aplicações reais que encerram o livro: **realidade aumentada**, **fotogrametria** e **visão estereoscópica**.

9.10 Uso do NotebookLM como Tutor Complementar

Nesta edição, o uso do **NotebookLM** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR NOTEBOOKLM: CAPÍTULO 09](#)

 **Aviso sobre Conteúdo Gerado por IA**

Embora seja uma ferramenta valiosa de apoio aos estudos, o NotebookLM pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

9.11 Exercícios Práticos

9.11.1 Exercício 1: Profundidade da Rede (Básico)

Contexto: A CNN do Projeto Prático 1 utiliza duas camadas convolucionais (8 e 16 filtros, respectivamente).

Desafio: Adicione uma terceira camada convolucional (por exemplo, 32 filtros) antes da camada totalmente conectada, ajustando as dimensões necessárias. Treine a nova rede nas mesmas condições (60 épocas, mesma divisão treino/teste) e compare sua acurácia final e seu número de parâmetros com a rede original de duas camadas.

Saída esperada: O número de parâmetros de cada rede, suas acurácias finais, e uma frase concluindo se a camada adicional trouxe benefício mensurável nesta base de dados específica.

Dica: Como a imagem de entrada é pequena (8×8), o *pooling* repetido pode reduzir demais a resolução espacial antes da terceira convolução; considere remover um dos *poolings* ou usar *padding* para manter dimensões compatíveis.

9.11.2 Exercício 2: Limiar de Transferência de Dados (Intermediário)

Contexto: O experimento de transferência de aprendizado utilizou apenas 20 exemplos de treino no domínio de destino (dígitos 5–9).

Desafio: Repita o experimento variando o número de exemplos de treino disponíveis no domínio B para {5, 10, 20, 40, 80}, comparando, para cada quantidade, a acurácia obtida com transferência de aprendizado e treinando do zero.

Saída esperada: Um gráfico com duas curvas (transferência e do zero), mostrando a acurácia em função do número de exemplos de treino, e uma conclusão sobre em que ponto (se houver) treinar do zero se torna tão eficaz quanto a transferência de aprendizado.

Dica: Espera-se que a vantagem da transferência de aprendizado diminua à medida que mais dados se tornam disponíveis no domínio de destino — eventualmente, dados suficientes tornam o treinamento do zero competitivo.

9.11.3 Exercício 3: Ajuste Fino Parcial (*Fine-Tuning*) (Intermediário/Desafio)

Contexto: O experimento de transferência de aprendizado **congelou totalmente** o extrator de características, treinando apenas a cabeça de classificação.

Desafio: Implemente uma terceira estratégia: descongele apenas a **segunda** camada convolucional (`conv2`) do extrator (mantendo `conv1` congelada), permitindo que ela seja ajustada durante o treinamento no domínio B, junto com a nova cabeça de classificação. Compare a acurácia resultante às duas estratégias já implementadas (transferência totalmente congelada e treinamento do zero).

Saída esperada: Uma tabela ou gráfico de barras com as três estratégias (congelado, parcialmente ajustado, do zero) e suas respectivas acurácias no domínio B.

Dica: Para descongelar apenas `conv2`, itere sobre `modelo.extrator.conv2.parameters()` e defina `requires_grad = True` antes de incluí-los na lista de parâmetros passada ao otimizador.

9.11.4 Exercício 5: Congelamento do Backbone no YOLO (Intermediário/Desafio)

Contexto: No Projeto Prático 2, o ajuste fino do YOLO pré-treinado congelou as 10 primeiras camadas do backbone (`freeze=10`), deixando as camadas finais e a cabeça de detecção livres para se adaptar às 9 classes de objetos geométricos.

Desafio: Repita o treinamento variando o número de camadas congeladas para $\{0, 5, 10, 15\}$ (`freeze=0` corresponde a um ajuste fino completo, sem nenhuma camada congelada), mantendo fixos o número de épocas e o conjunto de dados. Para cada configuração, registre o `mAP50` obtido no conjunto de validação.

Saída esperada: Uma tabela ou gráfico relacionando o número de camadas congeladas ao `mAP50` final, e uma conclusão sobre se, para esta tarefa específica (domínio de origem e destino bastante distintos), congelar mais ou menos camadas produziu melhores resultados.

Dica: Como o conjunto de dados sintético é pequeno, espera-se alguma variância entre execuções; se possível, repita cada configuração com mais de uma semente aleatória na geração dos dados e reporte a média.

9.11.5 Exercício 4: Disparidade e Distância de Base (Desafio)

Contexto: O exemplo de visão estereoscópica utilizou deslocamentos fixos (4 e 22 pixels) para simular um objeto distante e um objeto próximo, respectivamente.

Desafio: Utilizando a fórmula $Z = f \cdot B/d$, e assumindo uma distância focal fictícia $f = 700$ pixels e uma linha de base $B = 6$ cm (aproximando a distância entre os olhos humanos), calcule a profundidade estimada Z , em centímetros, para o fundo e para o objeto em primeiro plano do experimento, a partir das disparidades médias já calculadas (`disp_fundo` e `disp_objeto`). Em seguida, gere um **novo par estéreo** com um terceiro deslocamento intermediário (por exemplo, 12 pixels) para uma segunda região da imagem, e calcule sua profundidade estimada.

Saída esperada: As três profundidades estimadas (fundo, objeto original, novo objeto intermediário), em centímetros, e uma verificação de que a relação entre disparidade e profundidade estimada é inversa, como previsto pela fórmula.

Dica: Tome cuidado com disparidades muito próximas de zero — a fórmula $Z = f \cdot B/d$ produz valores extremamente grandes (ou indefinidos) nesse caso, refletindo a dificuldade real de estimar profundidade para objetos muito distantes por triangulação estéreo.

9.12 Encerramento da Parte II

Este capítulo conclui a Parte II do livro — e, com ela, a jornada que começou no Capítulo 6 com a formalização de “detectar, descrever e corresponder” características em imagens. Ao longo do caminho, um mesmo tema conceitual se repetiu sob formas cada vez mais sofisticadas: a tensão entre **características projetadas manualmente** (Sobel, LBP, HOG, ORB, Haar Cascade) e **características aprendidas automaticamente** (as CNNs deste capítulo) — e a constatação experimental, repetida em três capítulos consecutivos, de que nenhuma das duas abordagens é universalmente superior: a escolha correta depende da complexidade real do problema, da quantidade de dados disponível e das restrições computacionais da aplicação.

Para o leitor interessado em aprofundar os tópicos aqui apresentados apenas em suas ideias centrais, algumas direções naturais de estudo incluem:

- **Arquiteturas modernas de CNN** (ResNet, EfficientNet, Vision Transformers) e as inovações que permitiram treinar redes cada vez mais profundas;
- **Detecção e segmentação em tempo real**, com foco nas famílias YOLO e nos modelos de segmentação promptable como o Segment Anything;
- **Reconstrução 3D e SLAM** (*Simultaneous Localization and Mapping*), que estendem a visão estereoscópica deste capítulo a cenários dinâmicos, com câmeras em movimento;
- **Modelos generativos** para imagens (GANs, modelos de difusão), capazes de sintetizar, e não apenas interpretar, conteúdo visual.

Os fundamentos consolidados nesta Parte II — desde a extração de características mais simples até as redes neurais convolucionais — são a base sobre a qual todas essas direções mais avançadas se apoiam.



9.13 Parte Prática com Exercícios de Programação

Em construção!

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 9 — *Deep Learning* para Visão Computacional — por meio de uma trilha prática aplicada. Diferentemente do treinamento de redes neurais completas com PyTorch, que exige tempo de execução e, por vezes, GPU, os EPs deste capítulo isolam as **grandezas intermediárias** de um *pipeline* real de aprendizado profundo — a saída de uma única camada convolucional, o resultado de uma operação de *pooling*, a contagem de parâmetros treináveis de uma arquitetura, a sobreposição entre caixas delimitadoras candidatas e o filtro de supressão de não-máximos — permitindo validar manualmente cada etapa do raciocínio sem depender de bibliotecas de aprendizado de máquina nem de treinamento real.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com o cálculo manual da saída de uma **camada convolucional aprendida**, a partir de um *kernel* e um viés já treinados; avança-se para a operação de ***pooling*** (máximo e média), que reduz a resolução espacial entre blocos convolucionais; prossegue-se com a **contagem de parâmetros treináveis** de uma arquitetura CNN completa, evidenciando por que o compartilhamento de pesos torna essas redes tão mais econômicas que uma camada totalmente conectada equivalente; aprofunda-se no cálculo de **Interseção sobre União (IoU)** e na **Supressão de Não-Máximos (NMS)**, etapa de pós-processamento comum a detectores como o Faster R-CNN e o YOLO; e conclui-se com um ***pipeline* integrado**, unindo a saída de um detector de objetos (após NMS) a uma medição do mundo real por referência de escala — o mesmo princípio da fotogrametria estudada na integração final do capítulo.

! Diretrizes para a Resolução dos Exercícios de Programação

Em todos os exercícios deste capítulo, as etapas de discretização ou arredondamento numérico devem empregar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*), mitigando ambiguidades em valores com fração exatamente igual a 0,5. Salvo indicação explícita em contrário: (i) a operação de “convolução” segue a convenção adotada pelos *frameworks* de aprendizado profundo — **correlação cruzada**, sem inversão espacial do *kernel*, exatamente como apresentado na Seção “Convolução como Camada Aprendida”; (ii) o preenchimento (*padding*) é feito com zeros; (iii) caixas delimitadoras são especificadas no formato canto-a-canto (x_1, y_1, x_2, y_2) , com $x_1 < x_2$ e $y_1 < y_2$; e (iv) vetores/matrizes seguem indexação a partir de 0, com a convenção [linha] [coluna] para estruturas bidimensionais.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para rodar os testes, execute `TestSuite("EP09_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como string numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP09_01").run_code(codigo)
```

9.13.1 EP09_01 ● Convolução 2D Manual (Forward de uma Camada Aprendida)

O PyTorch, apresentado neste capítulo, executa `nn.Conv2d(x)` em uma única chamada — mas por trás dessa chamada está apenas a operação de correlação cruzada entre um *kernel* (já treinado) e uma vizinhança da

entrada, seguida da soma de um viés e de uma ativação, exatamente como formalizado na Seção “Convolução como Camada Aprendida”. A diferença essencial em relação à convolução fixa do Capítulo 3 é que, aqui, os valores do *kernel* e do viés **já vêm prontos** (como se tivessem sido aprendidos por gradiente), e cabe a você reproduzir manualmente a passagem direta (*forward pass*) que o *framework* executa internamente.

Antes de treinar uma CNN de verdade, você foi encarregado de implementar essa passagem direta do zero, para uma única camada convolucional com um único canal de entrada e um único filtro de saída, incluindo suporte a *padding* e *stride* arbitrários.

9.13.1.1 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ do mapa de características de entrada e, em seguida, seus $H \times W$ valores reais.
2. **Kernel e viés:** Ler as dimensões $k_h \times k_w$ do *kernel* (já treinado), seus valores reais, e o viés b (real, escalar).
3. **Hiperparâmetros:** Ler o *padding* p (inteiro, número de zeros adicionados em cada borda) e o *stride* s (inteiro, passo do deslizamento).
4. **Preenchimento:** Adicionar p zeros em cada uma das quatro bordas do mapa de entrada antes da correlação.
5. **Correlação cruzada:** Para cada posição de saída (i, j) , calcular

$$z(i, j) = b + \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v) \cdot X_{pad}(i \cdot s + u, j \cdot s + v),$$

percorrendo a entrada **sem** inverter o *kernel* (convenção dos *frameworks* de aprendizado profundo, diferente da convolução matemática clássica).

6. **Ativação:** Aplicar ReLU a cada valor: $a(i, j) = \max(0, z(i, j))$.
7. **Dimensões de saída:** $O_h = \lfloor (H + 2p - k_h)/s \rfloor + 1$ e $O_w = \lfloor (W + 2p - k_w)/s \rfloor + 1$.
8. **Saída:** Imprimir O_h e O_w na primeira linha, seguidos de O_h linhas com O_w valores reais cada (o mapa de características de saída, já com ReLU aplicada), formatados com 4 casas decimais.

9.13.1.2 Restrições Computacionais

- **Um canal de entrada, um filtro de saída:** não é necessário lidar com múltiplos canais ou múltiplos filtros nesta versão simplificada.
- **Sem inversão do *kernel*:** implemente correlação cruzada, não a convolução matemática clássica com *kernel* invertido — é essa a operação que o PyTorch (e a maioria dos *frameworks*) chama de “convolução”.
- **Preenchimento por zeros:** os p pixels adicionados em cada borda valem sempre 0.
- **Formatação:** todos os valores de saída devem ter exatamente 4 casas decimais, mesmo quando o valor é inteiro (ex.: 2.0000).

9.13.1.3 Fundamentação Teórica

Elemento	Papel na camada convolucional
<i>Kernel</i> K	Parâmetros aprendidos por gradiente, análogos aos filtros de Sobel do Capítulo 3, mas ajustados a partir de dados
Viés b	Parâmetro aprendido, somado à saída de cada janela
<i>Padding</i> p	Controla a dimensão espacial da saída; $p = 0$ (“ <i>valid</i> ”) reduz a resolução, p maior preserva-a
<i>Stride</i> s	Passo do deslizamento; $s > 1$ sub-amostra a saída, reduzindo custo computacional

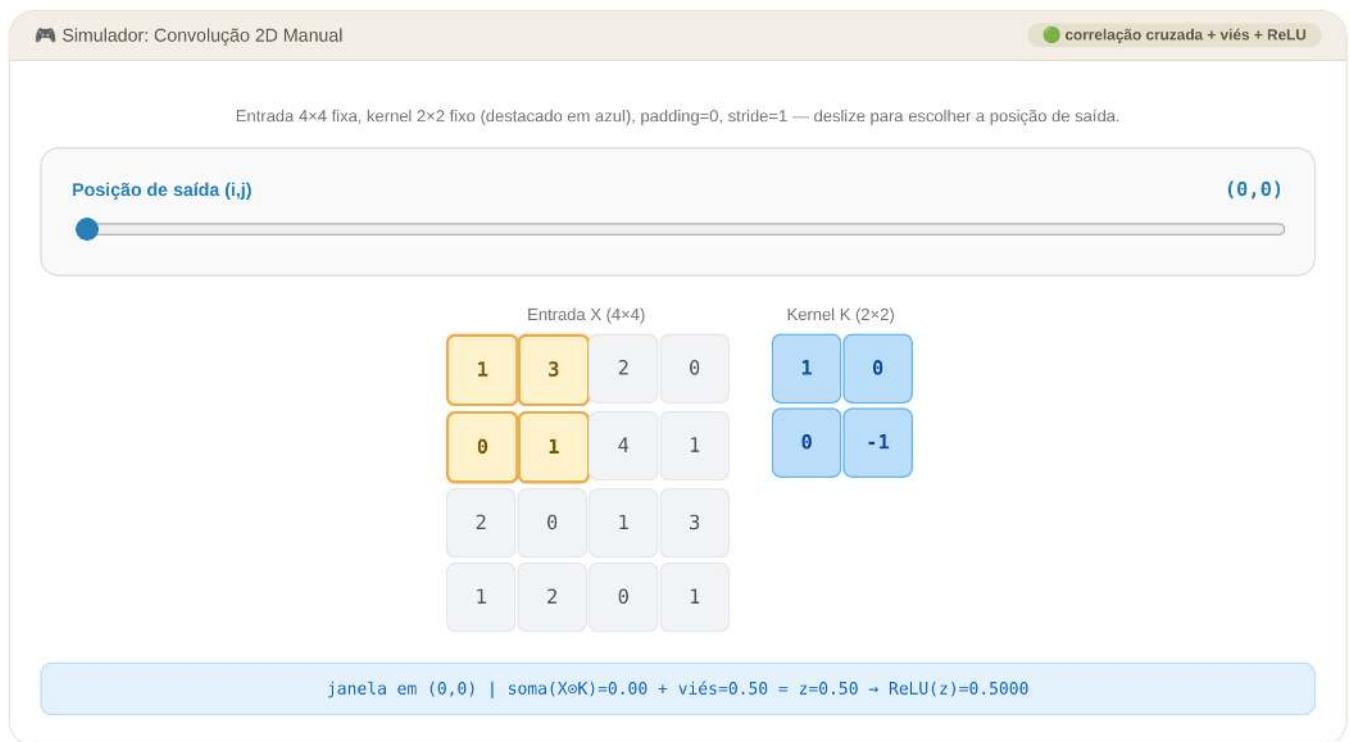


Figura 9.10: Simulador: Convolução 2D Manual (correlação cruzada + viés + ReLU)

9.13.2 EP09_02 ● *Pooling* Manual (Máximo e Média)

Entre blocos convolucionais, a arquitetura típica de uma CNN intercala camadas de *pooling*, que reduzem a resolução espacial do mapa de características sem introduzir novos parâmetros treináveis — ao contrário da convolução, o *pooling* não tem pesos: ele apenas resume cada janela da entrada a um único valor, por um máximo ou por uma média.

Você foi encarregado de implementar essa operação a partir de uma janela deslizante quadrada, sem sobreposição parcial nas bordas (apenas janelas completas), suportando os dois tipos mais comuns: **max** (preserva o valor mais saliente, tipicamente usado para reter bordas e texturas fortes) e **avg** (suaviza a região, preservando informação de intensidade média).

9.13.2.1 📋 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ do mapa de características de entrada e seus $H \times W$ valores reais.
2. **Janela:** Ler os inteiros k (tamanho da janela quadrada $k \times k$) e s (*stride*).
3. **Tipo:** Ler uma *string*, **max** ou **avg**, indicando o tipo de *pooling*.
4. **Sem preenchimento:** Esta operação **não** utiliza *padding*; janelas que ultrapassariam a borda da entrada são descartadas.
5. **Cálculo:** Para cada posição de saída (i, j) , calcular o máximo ou a média dos $k \times k$ valores da janela correspondente, iniciando em $(i \cdot s, j \cdot s)$.
6. **Dimensões de saída:** $O_h = \lfloor (H - k) / s \rfloor + 1$ e $O_w = \lfloor (W - k) / s \rfloor + 1$.
7. **Saída:** Imprimir O_h e O_w na primeira linha, seguidos de O_h linhas com O_w valores reais cada, formatados com 4 casas decimais.

9.13.2.2 📌 Restrições Computacionais

- **Janela quadrada:** $k \times k$, sem suporte a janelas retangulares nesta versão.
- **Sem *padding*:** apenas janelas inteiramente contidas na entrada são consideradas.
- **avg usa divisão real:** a média é sempre soma/ k^2 , mesmo quando o resultado tem muitas casas decimais — arredonde apenas na formatação final, conforme a diretriz geral do capítulo.
- **Formatação:** todos os valores de saída com exatamente 4 casas decimais.

9.13.2.3 🧠 Fundamentação Teórica

Elemento	Papel na arquitetura
<i>Pooling</i> máximo	Preserva a ativação mais forte da janela; comum após camadas convolucionais para reter bordas e texturas salientes
<i>Pooling</i> médio	Suaviza a região, preservando a intensidade média; comum em camadas finais (<i>global average pooling</i>)
Ausência de parâmetros	Diferencia o <i>pooling</i> da convolução: reduz resolução espacial sem custo adicional de treinamento
Redução de resolução	Contribui para a invariância a pequenas translações e para a redução do custo computacional das camadas seguintes

9.13.2.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W valores reais cada.
- Próxima linha: Inteiros k e s .
- Próxima linha: **max** ou **avg**.

Saída:

- Linha 1: Inteiros O_h e O_w .
- Próximas O_h linhas: O_w valores reais cada, formatados com 4 casas decimais.

9.13.2.5 📌 Exemplos

Entrada	Saída	Observação
4 4 1 3 2 4 5 6 1 2 2 1 0 3 4 2 5 1 2 2 max	2 2 6.0000 4.0000 4.0000 5.0000	<i>Pooling</i> máximo, janela 2×2 , <i>stride</i> 2.
4 4 1 3 2 4 5 6 1 2 2 1 0 3 4 2 5 1 2 2 avg	2 2 3.7500 2.2500 2.2500 2.2500	

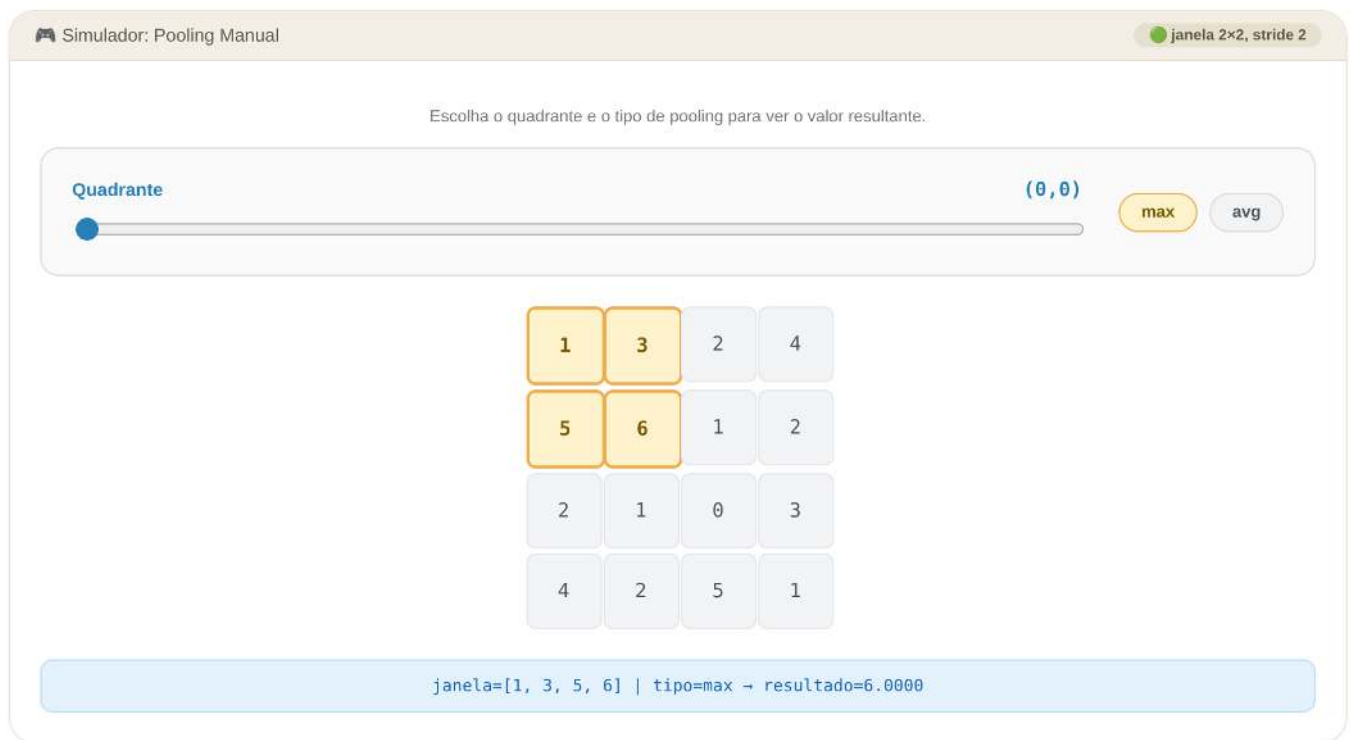


Figura 9.11: Simulador: Pooling Manual (máximo vs. média)

```
%%writefile EP09_02.py
# Código Python
```

Overwriting EP09_02.py

```
TestSuite("EP09_02.py").run()
```

9.13.3 EP09_03 ● Contagem de Parâmetros Treináveis de uma CNN

O Exercício 1 deste capítulo pediu para adicionar uma terceira camada convolucional à CNN do Projeto Prático 1 e comparar o número de parâmetros da rede resultante. Este EP formaliza esse cálculo: dada a descrição textual de uma arquitetura — uma sequência de camadas convolucionais, de *pooling* e totalmente conectadas — determine, para cada camada, o número de parâmetros treináveis e o total da rede.

O ponto central deste exercício é notar que a contagem de parâmetros de uma camada convolucional depende **apenas** do tamanho do *kernel* e do número de canais de entrada/saída — **nunca** das dimensões espaciais ($H \times W$) do mapa de características, graças ao **compartilhamento de pesos**: o mesmo *kernel* desliza por toda a imagem, seja ela 28×28 ou 280×280 . É exatamente esse compartilhamento que torna as CNNs muito mais econômicas em parâmetros do que uma camada totalmente conectada equivalente, na qual cada pixel de entrada tem uma conexão (e um peso) independente para cada neurônio de saída.

9.13.3.1 📋 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro L (número de camadas da arquitetura, na ordem em que são aplicadas).
2. **Camadas:** Ler L linhas, cada uma descrevendo uma camada em um dos três formatos:

- **CONV** *kh kw cin cout bias* — camada convolucional com *kernel* $k_h \times k_w$, *cin* canais de entrada, *cout* canais (filtros) de saída, e *bias* (0 ou 1) indicando se há viés por filtro;
 - **POOL** — camada de *pooling* (máximo ou médio; não possui parâmetros treináveis);
 - **FC** *in out bias* — camada totalmente conectada com *in* entradas, *out* saídas, e *bias* (0 ou 1) indicando se há viés por neurônio.
3. **Parâmetros de uma camada CONV:** $k_h \cdot k_w \cdot c_{in} \cdot c_{out} + c_{out} \cdot \text{bias}$.
 4. **Parâmetros de uma camada FC:** $\text{in} \cdot \text{out} + \text{out} \cdot \text{bias}$.
 5. **Parâmetros de uma camada POOL:** sempre 0.
 6. **Saída:** Para cada camada, na ordem de leitura, imprimir **Camada i: P**, onde *i* começa em 1 e *P* é o número de parâmetros daquela camada. Ao final, imprimir **Total: T**, com *T* igual à soma de parâmetros de todas as camadas.

9.13.3.2 Restrições Computacionais

- **Independência da dimensão espacial:** a entrada **não** informa $H \times W$ — a contagem de uma camada CONV não depende disso, apenas de *kh kw cin cout*.
- **bias sempre 0 ou 1:** multiplique diretamente o termo de viés por esse valor, sem tratamento condicional especial.
- **Camadas POOL sem argumentos adicionais:** a linha contém apenas a palavra POOL.
- Todos os valores de entrada e saída são inteiros não-negativos.

9.13.3.3 Fundamentação Teórica

Elemento	Papel na contagem de parâmetros
Compartilhamento de pesos	O mesmo <i>kernel</i> $k_h \times k_w$ é reutilizado em toda a extensão espacial da entrada — por isso a contagem independe de $H \times W$
Canais de entrada/saída	Cada um dos <i>cout</i> filtros possui um conjunto de pesos por canal de entrada, daí o fator $c_{in} \cdot c_{out}$
Viés	Um único escalar por filtro (CONV) ou por neurônio (FC), independente do tamanho do <i>kernel</i> ou da entrada
Camada FC	Cada uma das <i>in</i> entradas conecta-se a cada uma das <i>out</i> saídas — sem compartilhamento, o que explica seu custo tipicamente muito maior em número de parâmetros

9.13.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro *L*.
- Próximas *L* linhas: descrição de cada camada, no formato CONV *kh kw cin cout bias*, POOL, ou FC *in out bias*.

Saída:

- *L* linhas no formato **Camada i: P**.
- Última linha: **Total: T**.

9.13.3.5 Exemplos

Entrada	Saída	Observação
3	Camada 1: 80	Rede com uma convolução, um pooling e uma camada final.
CONV 3 3 1 8 1	Camada 2: 0	
POOL	Camada 3: 13530	
FC 1352 10 1	Total: 13610	
4	Camada 1: 80	Rede com duas convoluções empilhadas, como no Projeto Prático 1; a última camada não usa viés.
CONV 3 3 1 8 1	Camada 2: 1168	
CONV 3 3 8 16 1	Camada 3: 0	
POOL	Camada 4: 2048	
FC 64 32 0	Total: 3296	

Simulador: Contagem de Parâmetros compartilhamento de pesos

Ajuste o kernel e os canais de uma camada CONV e compare com uma camada FC equivalente, para uma entrada hipotética de 32×32 pixels.

kernel (k×k) 3×3 canais de entrada 1

canais de saída (filtros) 8 ☒ usar viés

CONV: $3 \times 3 \times 1 \times 8 + 8 \times 1 = 80$ parâmetros

FC equivalente (entrada 32×32×1 → 8 saídas): **8.200 parâmetros**

↳ a camada FC teria **102.5×** mais parâmetros que a CONV equivalente, apesar de "ver" a mesma entrada.

Figura 9.12: Simulador: Contagem de Parâmetros — Convolução vs. Camada Totalmente Conectada

```
%%writefile EP09_03.py
# Código Python
```

Overwriting EP09_03.py

```
TestSuite("EP09_03.py").run()
```

9.13.4 EP09_04 ● Interseção sobre União (IoU) e Supressão de Não-Máximos (NMS)

Tanto o Faster R-CNN quanto o YOLO, apresentados na seção “Aplicações em Larga Escala” e no Projeto Prático 2 deste capítulo, produzem internamente **muito mais** caixas delimitadoras candidatas do que objetos realmente existem na imagem — várias delas cobrindo, com pequenas variações, o mesmo objeto. A etapa de

pós-processamento responsável por eliminar essas redundâncias é a **Supressão de Não-Máximos (NMS)**, e sua peça fundamental é a métrica de **Interseção sobre União (IoU)**, que mede o quanto duas caixas se sobrepõem — a mesma métrica usada, por exemplo, para definir o limiar de acerto do **mAP50** calculado no Projeto Prático 2 ($\text{IoU} \geq 0,5$ com a caixa verdadeira).

Você foi encarregado de implementar o algoritmo de NMS do zero: dado um conjunto de caixas candidatas com suas pontuações de confiança, filtrar as redundantes e manter apenas as detecções mais confiáveis e suficientemente distintas entre si.

9.13.4.1 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro N (número de caixas candidatas) e o limiar real τ (limiar de IoU para supressão).
2. **Caixas:** Ler N linhas, cada uma com cinco valores reais: $x_1, y_1, x_2, y_2, \text{score}$ (canto superior-esquerdo, canto inferior-direito, pontuação de confiança).
3. **Interseção sobre União:** Para duas caixas A e B ,

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A) + \text{área}(B) - \text{área}(A \cap B)},$$

onde a área de interseção é calculada pela sobreposição dos intervalos em x e em y (zero se não houver sobreposição).

4. **Algoritmo guloso de NMS:**
 - a. Ordene as caixas por **score** decrescente (em caso de empate, mantenha a ordem de leitura original — ordenação estável).
 - b. Selecione a caixa de maior pontuação entre as restantes; adicione-a ao conjunto de saída e remova-a da lista.
 - c. Calcule o IoU dessa caixa selecionada com **todas** as caixas ainda restantes; remova (suprima) qualquer caixa com $\text{IoU} > \tau$.
 - d. Repita os passos (b)–(c) até que não restem caixas.
5. **Saída:** Para cada caixa mantida, na ordem em que foi selecionada, imprimir seu índice original (posição de leitura, começando em 0) e seu **score**, formatado com 4 casas decimais. Ao final, imprimir **Total mantidas: X**.

9.13.4.2 Restrições Computacionais

- **Supressão estrita:** apenas caixas com $\text{IoU} > \tau$ são suprimidas; caixas com IoU exatamente igual a τ são **mantidas**.
- **Índices originais:** a saída referencia a posição em que cada caixa foi lida na entrada (começando em 0), não sua posição após a ordenação.
- **Retângulos alinhados aos eixos:** todas as caixas são especificadas por dois cantos, com $x_1 < x_2$ e $y_1 < y_2$ garantidos na entrada.

9.13.4.3 Fundamentação Teórica

Elemento	Papel no pós-processamento de detecção
IoU	Quantifica a sobreposição espacial entre duas caixas; $\text{IoU} = 1$ para caixas idênticas, $\text{IoU} = 0$ para caixas disjuntas
Limiar τ	Controla a agressividade da supressão: τ baixo elimina até detecções pouco sobrepostas; τ alto preserva quase todas
Ordenação por confiança	Garante que, entre caixas redundantes, sempre sobrevive a de maior pontuação

Elemento	Papel no pós-processamento de detecção
mAP50 (Projeto Prático 2)	Usa exatamente o mesmo limiar de IoU (0,5) para decidir se uma detecção final “acerta” a caixa verdadeira

9.13.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N e real τ .
- Próximas N linhas: cinco reais x_1 y_1 x_2 y_2 score.

Saída:

- Uma linha por caixa mantida, na ordem de seleção: **índice** **score** (score com 4 casas decimais).
- Última linha: **Total mantidas: X**.

9.13.4.5 Exemplos

Entrada	Saída	Observação
3 0.5	0 0.9000	A caixa 1 é suprimida por sobrepor fortemente a caixa 0 (IoU 0,68 > 0,5).
0 0 10 10 0.9	2 0.8000	
1 1 11 11 0.75	Total mantidas: 2	
50 50 60 60 0.8		
2 0.5	1 0.9500	A caixa 0 é suprimida por sobrepor demais a vencedora (IoU=0,6 > 0,5), mesmo tendo menor score de origem que a caixa 1, aqui a de maior score.
0 0 10 10 0.9	Total mantidas: 1	
0 0 10 6 0.95		

```
%%writefile EP09_04.py
# Código Python
```

```
Overwriting EP09_04.py
```

```
TestSuite("EP09_04.py").run()
```

9.13.5 EP09_05 Pipeline Integrado: Da Detecção à Medição do Mundo Real

Este exercício final integra os dois exercícios anteriores ao princípio de **fotogrametria** apresentado na seção “Integrando Geometria e Aprendizado Profundo” — exatamente o mesmo cálculo implementado na figura de medição por referência de escala deste capítulo. O cenário reproduz uma situação realista: um detector (Faster R-CNN ou YOLO) gera **várias caixas candidatas sobrepostas** para o mesmo objeto de interesse; após filtrá-las por NMS, a caixa sobrevivente de maior confiança é usada, junto de uma caixa de referência de largura real conhecida (como o cartão de 8,56 cm), para estimar as dimensões reais do objeto detectado.

9.13.5.1 Diretrizes de Implementação

1. **Referência conhecida:** Ler o valor real L_{ref} (largura real do objeto de referência, em cm) e, em seguida, os quatro reais x_1 y_1 x_2 y_2 de sua caixa delimitadora em pixels (já conhecida, sem necessidade de detecção).



Figura 9.13: Simulador: IoU e Supressão de Não-Máximos

2. **Candidatas do objeto a medir:** Ler o inteiro N (número de caixas candidatas produzidas pelo detector para o objeto de interesse) e o limiar real τ ; em seguida, ler as N linhas de caixas candidatas, cada uma com $x_1 \ y_1 \ x_2 \ y_2 \ \text{score}$.
3. **Etapa 1 — NMS:** Aplique exatamente o algoritmo de Supressão de Não-Máximos do EP09_04 às N caixas candidatas, usando o limiar τ , para eliminar detecções redundantes do mesmo objeto.
4. **Etapa 2 — Seleção da caixa final:** Após o NMS, a caixa de maior **score** entre as mantidas é a detecção final do objeto (a entrada garante que todas as caixas candidatas correspondem a um único objeto físico, portanto a primeira caixa selecionada pelo NMS já é o resultado final).
5. **Etapa 3 — Medição por referência de escala:** Calcule a razão $\text{cm/pixel} = L_{ref}/\text{largura da referência em pixels}$ e aplique-a tanto à largura quanto à altura (em pixels) da caixa final do objeto, obtendo suas dimensões reais estimadas em centímetros.
6. **Saída:** Primeiro, uma linha por caixa mantida após o NMS (mesmo formato do EP09_04): **índice score**. Em seguida, a linha **Total mantidas: X**. Por fim, a linha **Objeto: L x A cm**, onde L e A são a largura e a altura estimadas do objeto, cada uma com 2 casas decimais.

9.13.5.2 📌 Restrições Computacionais

- **Reaproveite o NMS do EP09_04** integralmente — mesma regra de desempate, mesmo critério de supressão ($\text{IoU} > \tau$).
- **A referência não passa por NMS:** sua caixa é dada diretamente, sem candidatas concorrentes.
- **Razão única para largura e altura:** assim como na figura de fotogrametria do capítulo, a mesma razão cm/pixel (derivada da largura da referência) é aplicada tanto à largura quanto à altura do objeto — não há calibração vertical separada.

9.13.5.3 🧠 Fundamentação Teórica

Etapa	Conceito do capítulo
Múltiplas caixas candidatas	Saída bruta de um detector como o Faster R-CNN ou o YOLO, antes do pós-processamento
NMS (EP09_04)	Filtra a redundância, mantendo apenas a detecção mais confiável do objeto
Referência de escala conhecida	Mesmo princípio da fotogrametria: um objeto de dimensão real conhecida converte pixels em centímetros
Razão cm/pixel	Fator de conversão único, assumindo que a câmera está aproximadamente perpendicular à cena (sem correção de perspectiva)

9.13.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Real L_{ref} .
- Linha 2: Quatro reais x_1 y_1 x_2 y_2 (caixa de referência).
- Linha 3: Inteiro N e real τ .
- Próximas N linhas: cinco reais x_1 y_1 x_2 y_2 score (caixas candidatas do objeto).

Saída:

- Uma linha por caixa mantida após NMS: `índice score` (score com 4 casas decimais).
- Linha `Total mantidas`: `X`.
- Linha final: `Objeto: L x A cm` (2 casas decimais cada).

9.13.5.5 Exemplos

Entrada	Saída	Observação
8.56	0 0.9200	A caixa 1 é suprimida por sobrepor fortemente a caixa 0; a detecção final do objeto é a caixa 0.
30 200 170 288	2 0.4000	
3 0.5	Total mantidas: 2	
250 100 470 250 0.92	Objeto: 13.45 x 9.17 cm	
255 105 468 245 0.88		
600 600 650 650 0.40		

```
%%writefile EP09_05.py
# Código Python
```

```
Overwriting EP09_05.py
```

```
TestSuite("EP09_05.py").run()
```

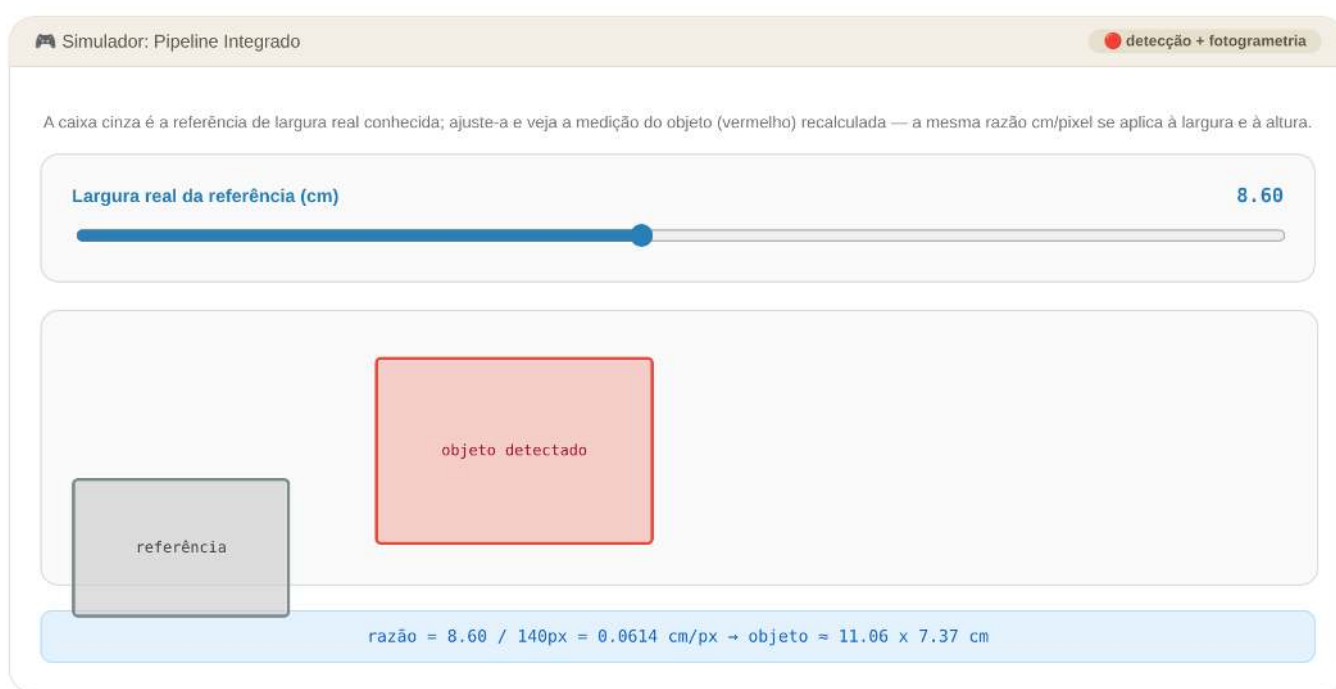


Figura 9.14: Simulador: Pipeline Integrado — NMS + Medição por Referência de Escala

Referências

- BAHDANAU, Dzmitry; CHO, Kyunghyun; BENGIO, Yoshua. Neural Machine Translation by Jointly Learning to Align and Translate. *In*: 2015. Disponível em: <<https://arxiv.org/abs/1409.0473>>
- BARRERA, Junior *et al.* MMach: a mathematical morphology toolbox for the KHOROS system. **Journal of Electronic Imaging**, v. 7, n. 1, p. 174–210, 1998.
- BERGMANN, Paul *et al.* [MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection](#). *In*: 2019.
- BRADSKI, Gary; KAEHLER, Adrian. **Learning OpenCV: Computer vision with the OpenCV library**. [S.l.]: ” O’Reilly Media, Inc.”, 2008.
- COVER, T.; HART, P. [Nearest neighbor pattern classification](#). **IEEE Transactions on Information Theory**, v. 13, n. 1, p. 21–27, 1967.
- DALAL, N.; TRIGGS, B. [Histograms of oriented gradients for human detection](#). *In*: 2005.
- DOUGHERTY, Edward R.; LOTUFO, Roberto A. **Hands-on morphological image processing**. [S.l.]: SPIE press, 2003. v. 59
- DUDA, Richard O.; HART, Peter E.; STORK, David G. **Pattern Classification**. 2nd. ed. New York: Wiley-Interscience, 2001.
- GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4th. ed. New York: Pearson, 2018.
- GOOGLE. **NotebookLM**. <https://notebooklm.google.com>, 2025.
- ITU-R. **Recommendation BT.601: Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios**. <https://www.itu.int/rec/R-REC-BT.601/>, 2011.
- KNUTH, Donald E. [Literate Programming](#). **The Computer Journal**, v. 27, n. 2, p. 97–111, 1984.
- LEWIS, Patrick *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. **Advances in neural information processing systems**, v. 33, p. 9459–9474, 2020.
- LOTUFO, R. A.; ZAMPIROLI, F. A. [Fast multidimensional parallel Euclidean distance transform based on mathematical morphology](#). *In*: 2001.
- LOTUFO, Roberto A. *et al.* MMachLib functions and MMach operators. *In*: 1997.
- MALLAT, Stéphane. **A wavelet tour of signal processing**. [S.l.]: Elsevier, 1999.
- MATHERON, Georges. **Random Sets and Integral Geometry**. New York: John Wiley & Sons, 1975.
- OJALA, Timo; PIETIKÄINEN, Matti; MÄENPÄÄ, Topi. [Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns](#). **IEEE Trans. Pattern Anal. Mach. Intell.**, v. 24, n.

7, p. 971–987, jul. 2002.

OPPENHEIM, Alan V.; SCHAFER, Ronald W. **Discrete-Time Signal Processing**. [S.l.]: Pearson, 2010.

OTSU, Nobuyuki. **A Threshold Selection Method from Gray-Level Histograms**. **IEEE Transactions on Systems, Man, and Cybernetics**, v. 9, n. 1, p. 62–66, 1979.

PEDREGOSA, Fabian *et al.* Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

QUILICI-GONZALEZ, José Artur; ZAMPIROLI, Francisco de Assis. **Sistemas Inteligentes e Mineração de Dados**. [S.l.]: Editora UFABC, 2014.

QUILICI-GONZALEZ, José Artur; ZAMPIROLI, Francisco de Assis; SOUZA, Fábio Rezende de. **Sistemas Inteligentes e Mineração de Dados: Do Weka ao Python**. 2. ed. [S.l.]: <https://fzampirolli.github.io/simd2/>, 2026.

REDMON, Joseph *et al.* **You Only Look Once: Unified, Real-Time Object Detection**. *In*: 2016.

SERRA, Jean. **Image Analysis and Mathematical Morphology**. London: Academic Press, 1982. v. 1

SINGH, Himanshu. **Practical machine learning and image processing**. [S.l.]: Springer, 2019.

SINGH, Shobhna; LLOYD, Jerome; FLICKER, Felix. **Hamiltonian Cycles on Ammann-Beenker Tilings**. **Physical Review X**, v. 14, n. 3, p. 031005, 10 jul. 2024.

SMITH, Ray. **An Overview of the Tesseract OCR Engine**. *In*: IEEE Computer Society, 2007.

SMITH, Ray. **History of the Tesseract OCR Engine: What Worked and What Didn't**. *In*: 2013.

SONG, Kechen; YAN, Yunhui. **A Noise Robust Method Based on Completed Local Binary Patterns for Hot-Rolled Steel Strip Surface Defects**. **Applied Surface Science**, v. 285, p. 858–864, 2013.

SZELISKI, Richard. **Computer Vision: Algorithms and Applications**. 2. ed. [S.l.]: Springer, 2022.

TABERNIK, Domen *et al.* **Segmentation-Based Deep-Learning Approach for Surface-Defect Detection**. **Journal of Intelligent Manufacturing**, v. 31, n. 3, p. 759–776, 2020.

VASWANI, Ashish *et al.* Attention Is All You Need. *In*: 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>

WALLACE, Gregory K. **The JPEG Still Picture Compression Standard**. **Communications of the ACM**, v. 34, n. 4, p. 30–44, 1991.

ZAMPIROLI, Francisco A. **MCTest: Como Criar e Corrigir Exames Parametrizados**. [S.l.]: Independente, 2023.

ZAMPIROLI, Francisco de Assis *et al.* A Practical Digital Image Processing Course with morph.py. *In*: Porto Alegre, RS, Brasil: Sociedade Brasileira de Computação (SBC), 2024. Disponível em: <<https://sol.sbc.org.br/index.php/educomp/article/view/28199>>

ZAMPIROLI, Francisco de Assis *et al.* **Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results**. **Revista Brasileira de Informática na Educação**, v. 33, p.

990–1026, 2025.